

lwIP 开发指南

V1.6

本教程适用于正点原子所有 STM32 开发板

修订历史:

版本	日期	修改内容
V1.0	2022/06/18	第一次发布
V1.1	2022/10/20	添加第 1~8 章的内容
V1.2	2022/10/27	添加阿波罗等开发板移植内容
V1.3	2023/2/1	修改 TCP/IP 协议栈概述
V1.4	2023/12/05	删除 OneNET HTTP 实验，并新版 OneNET MQTT 连接
V1.5	2023/12/25	备注有些开发板没有的实验
V1.6	2024/1/17	删除旧版本原子云连接例程(等待最新原子云上市)
V1.7	2024/6/17	添加 H7R1wIP 移植和删除摄像头章节



正点原子公司名称：广州市星翼电子科技有限公司

原子哥在线教学平台：www.yuanzige.com

开源电子网 / 论坛：www.openedv.com

正点原子官方网站：www.alientek.com

正点原子淘宝店铺：<https://openedv.taobao.com>

正点原子 B 站视频：<https://space.bilibili.com/394620890>

电话：020-38271790 传真：020-36773971

请下载原子哥 APP，数千讲视频免费学习，更快更流畅。

请关注正点原子公众号，资料发布更新我们会通知。



扫码下载“原子哥”APP



扫码关注正点原子公众号

前言	10
第一章 lwIP 初探.....	11
1.1 TCP/IP 协议栈是什么	11
1.1.1 TCP/IP 协议栈架构	11
1.1.2 TCP/IP 协议栈的封包和拆包	12
1.2 lwIP 简介	13
1.2.1 lwIP 源码下载	14
1.2.2 lwIP 文件说明	15
1.3 MAC 内核简介	17
1.4 PHY 芯片介绍	20
1.4.1 YT8512C 简介	20
1.4.2 LAN8720A 简介	27
1.5 以太网接入 MCU 方案	29
第二章 lwIP 无操作系统移植.....	32
2.1 lwIP 前期准备	32
2.2 以太网 DMA 描述符	32
2.2.1 ST 的以太网 DMA 描述符简介	32
2.2.2 DMA 描述符的类型	35
2.2.3 如何追踪 DMA 描述符	37
2.3 lwIP 移植流程(探索者、DMF407).....	38
2.3.1 添加网卡驱动程序	38
2.3.2 添加 lwIP 源文件	47
2.3.3 arch 文件夹下的文件修改	50
2.4 lwIP 移植流程（阿波罗、北极星）	71
2.4.1 添加网卡驱动程序	71
2.4.2 添加 lwIP 源文件	72
2.4.3 arch 文件夹下的文件修改	75
2.4.4 使用 AC6 编译工程（扩展）	85
第三章 lwIP 带操作系统移植.....	88
3.1 lwIP 前期准备	88
3.2 lwIP 移植流程（探索者、DMF407）	88
3.2.1 修改 lwipopts.h	88
3.2.2 添加 sys_arch.c/h 文件	92
3.2.3 修改 lwip_comm.c	93
3.2.4 修改 ethernetif.c/h 文件	95
3.2.5 修改 ethernet.c 文件	97
3.2.6 添加应用程序	98
3.3 lwIP 移植流程（阿波罗、北极星）	109
3.3.1 修改 lwipopts.h	109
3.3.2 添加 sys_arch.c/h 文件	113
3.3.3 修改 lwip_comm.c/h	114

3.3.4 修改 ethernetif.c/h 文件	117
3.3.5 修改 ethernet.c 文件	117
3.3.6 添加应用程序	117
3.4 sys_arch 文件解析	129
第四章 lwIP 内存管理	164
4.1 内存的简介	164
4.2 动态内存堆管理策略	164
4.3 动态内存池管理策略	171
4.4 使用 C 库管理内存策略	179
第五章 lwIP 网络接口管理	181
5.1 网络接口结构 netif	181
第六章 lwIP 网络数据包解析	186
6.1 TCP/IP 协议各层间的操作	186
6.2 lwIP 的线程简介	186
6.3 pbuf 结构	187
第七章 lwIP 简介	196
7.1 网络接口接收数据	196
7.2 lwIP 超时处理	196
7.3 tcpip_thread（协议栈）线程	202
7.4 lwIP 中的消息	204
7.4.1 数据包消息(tcpip_msg)	204
7.4.2 API 消息	208
第八章 ARP 协议	215
8.1 ARP 协议的简介	215
8.1.1 ARP 协议的工作流程	215
8.1.2 ARP 缓存表的超时处理	218
8.2 APR 报文的报文结构	219
8.3 ARP 协议层的接收与发送原理解析	221
8.3.1 发送 ARP 请求数据包	221
8.3.2 接收 ARP 应答数据包	224
第九章 IP 协议	228
9.1 IP 协议的简介	228
9.2 IP 数据报	228
9.2.1 IP 数据报结构	230
9.2.2 IP 数据报的分片解析	230
9.2.3 IP 数据报的分片重装	235
9.3 IP 数据报的输出	236
9.4 IP 数据报的输入	239

第十章 ICMP 协议	247
10.1 ICMP 协议简介	247
10.1.1 ICMP 报文类型	247
10.1.2 ICMP 报文结构	248
10.2 ICMP 的实现	251
10.2.1 ICMP 数据结构体	251
10.2.2 发送 ICMP 差错报文	252
10.2.3 ICMP 报文处理	254
第十一章 RAW 编程接口 UDP 实验	260
11.1 UDP 协议简介	260
11.1.2 UDP 报文的数据结构	261
11.1.3 发送 UDP 报文	262
11.1.4 UDP 报文接收	265
11.2 RAW 的 UDP 接口简介	269
11.3 RAW 的 UDP 实验	271
11.3.1 硬件设计	271
11.3.2 软件设计	271
11.3.3 下载验证	279
第十二章 RAW 编程接口 TCP 客户端实验	281
12.1 TCP 协议	281
12.1.1 TCP 协议简介	281
12.1.2 TCP 的建立连接	281
12.1.3 TCP 终止连接	282
12.1.4 TCP 报文结构	283
12.1.5 lwIP 的 TCP 报文首部数据结构	284
12.1.6 lwIP 的 TCP 连接状态图	285
12.1.7 lwIP 的 TCP 控制块	287
12.1.8 lwIP 的 TCP 编程	293
12.1.9 lwIP 的 TCP 建立与关闭连接原理	301
12.1.10 lwIP 中 RAW API 编程接口中与 TCP 相关的函数	314
12.2 RAW 接口的 TCP 实验	317
12.2.1 硬件设计	317
12.2.2 软件设计	317
12.2.3 下载验证	328
第十三章 RAW 编程接口 TCP 服务器实验	330
13.1 RAW 编程接口 TCP 简介	330
13.2 RAW 接口的 TCP 实验	330
13.2.1 硬件设计	330
13.2.2 软件设计	330
13.2.3 下载验证	337

第十四章 RAW 编程接口 Web Server 实验.....	339
14.1 Web Server 文件以及相关技术简介	339
14.2 Web Server 实验.....	344
14.2.1 硬件设计	344
14.2.2 软件设计	344
14.2.3 下载验证	348
第十五章 NETCONN 编程接口简介.....	350
15.1 netbuf 数据缓冲区	350
15.2 netconn 连接结构	352
15.3 netconn 编程 API 函数.....	353
第十六章 NETCONN 编程接口 UDP 实验	357
16.1 NETCONN 实现 UDP	357
16.2 NETCONN 接口的 UDP 实验	357
16.2.1 硬件设计	357
16.2.2 软件设计	357
16.2.3 下载验证	360
第十七章 NETCONN 编程接口 TCP 客户端实验.....	362
17.1 NETCONN 实现 TCP 客户端连接步骤	362
17.2 NETCONN 接口的 TCPClient 实验	362
17.2.1 硬件设计	362
17.2.2 软件设计	362
17.2.3 下载验证	366
第十八章 NETCONN 编程接口 TCP 服务器实验.....	368
18.1 NETCONN 实现 TCP 服务器步骤	368
18.2 NETCONN 接口的 TCPServer 实验.....	368
18.2.1 硬件设计	368
18.2.2 软件设计	368
18.2.3 下载验证	371
第十九章 Socket 编程接口简介	373
19.1 Socket 编程接口简介	373
19.2 Socket API 函数.....	374
第二十章 Socket 编程接口 UDP 实验	378
20.1 Socket 编程 UDP 连接流程.....	378
20.2 Socket 接口的 UDP 实验.....	378
20.2.1 硬件设计	378
20.2.2 软件设计	378
20.2.3 下载验证	381
第二十一章 Socket 编程接口 TCP 客户端实验	384

21.1 Socket 编程 TCP 客户端流程	384
21.2 Socket 接口的 TCPClient 实验.....	384
21.2.1 硬件设计	384
21.2.2 软件设计	384
21.2.3 下载验证	388
第二十二章 Socket 编程接口 TCP 服务器实验	390
22.1 Socket 编程 TCP 服务器流程	390
22.2 Socket 接口的 TCPServer 实验.....	390
22.2.1 硬件设计	390
22.2.2 软件设计	390
22.2.3 下载验证	394
第二十三章 NTP 协议实验	396
23.1 NTP 简介	396
23.2 NTP 实验	398
23.2.1 硬件设计	398
23.2.2 软件设计	398
23.2.3 下载验证	405
第二十四章 lwIP 测试网速	406
24.1 JPerf 网络测速工具	406
24.2 JPerf 网络实验	408
24.2.1 硬件设计	408
24.2.2 软件设计	409
24.2.3 下载验证	411
第二十五章 基于 MQTT 协议连接阿里云服务器	413
25.1 MQTT 协议简介	413
25.1.1 MQTT 协议实现原理	413
25.1.2 移植 MQTT 协议	416
25.1.3 配置远程服务器	416
25.2 阿里云 MQTT 协议实验.....	419
25.2.1 硬件设计	419
25.2.2 软件设计	419
25.2.3 下载验证	423
第二十六章 基于 MQTT 协议连接 OneNET 服务器	425
26.1 配置 OneNET 平台	425
26.2 工程配置	428
26.3 基于 OneNET 平台 MQTT 实验	429
26.3.1 硬件设计	429
26.3.2 软件设计	429
26.3.3 下载验证	430

第二十七章 网络摄像头（ATK-MC5640）实验	错误!未定义书签。
27.1 ATK-MC5640 简介	错误!未定义书签。
27.1.1 SCCB 简介	错误!未定义书签。
27.1.2 OV5640 DVP 接口说明	错误!未定义书签。
27.1.3 OV5640 窗口设置说明	错误!未定义书签。
27.1.4 OV5640 行像素输出时序介绍	错误!未定义书签。
27.1.5 OV5640 自动对焦介绍	错误!未定义书签。
27.2 网络摄像头实验	错误!未定义书签。
27.2.1 硬件设计	错误!未定义书签。
27.2.2 软件设计	错误!未定义书签。
27.2.3 下载验证	错误!未定义书签。
第二十八章 网络摄像头（ATK-MC2640）实验	错误!未定义书签。
28.1 ATK-MC2640 简介	错误!未定义书签。
28.1.1 SCCB 介绍	错误!未定义书签。
28.1.2 OV2640 行像素输出时序介绍	错误!未定义书签。
28.1.3 OV2640 帧时序介绍	错误!未定义书签。
28.2 网络摄像头实验	错误!未定义书签。
28.2.1 硬件设计	错误!未定义书签。
1. 例程功能	错误!未定义书签。
28.2.2 软件设计	错误!未定义书签。
28.2.2.1 程序流程图	错误!未定义书签。
28.2.2.2 程序解析	错误!未定义书签。
28.2.3 下载验证	错误!未定义书签。
第二十九章 HTTP 服务器实验	431
29.1 HTTP 协议简介	431
29.2 HTTP 服务器实验	432
29.2.1 硬件设计	432
25.2.2 软件设计	432
29.3.3 下载验证	436

前言

随着互联网技术的不断发展和普及，网络通信已经成为现代社会不可或缺的一部分。在各种网络通信协议中，TCP/IP 协议栈作为最基础和核心的协议之一，被广泛应用于各种网络设备和系统中。然而，对于许多嵌入式系统和资源受限的环境来说，标准的 TCP/IP 协议栈过于庞大和复杂，难以实现高效的资源利用和稳定的网络通信。为了解决这一问题，一种轻量级的 TCP/IP 协议栈——lwIP 应运而生。

lwIP (Lightweight IP) 是一个专为嵌入式系统设计的开源 TCP/IP 协议栈。它旨在以较小的资源消耗提供相对完整的 TCP/IP 协议栈功能，成为嵌入式系统网络通信的理想选择。通过优化和裁剪，lwIP 能够在资源受限的环境下实现高效的资源利用，提供稳定可靠的网络连接。

作为一本深入探讨 lwIP 协议栈的书籍，本书将全面介绍 lwIP 的原理、架构、应用和最佳实践。我们将从 lwIP 的起源和发展开始，详细解读其各个组成部分，包括内存管理、底层网络接口管理、ARP 层、IP 层、TCP 层、API 层等。通过深入剖析 lwIP 的工作原理和应用场景，我们将帮助读者全面了解 lwIP 在嵌入式系统中的重要性和应用价值。

除了理论知识的介绍，本书还将分享一些关于 lwIP 的实际应用案例和最佳实践。我们将结合实际项目经验，探讨如何在实际应用中充分利用 lwIP 的功能和优势，解决各种网络通信问题。通过这些案例和实践经验的分享，读者将能够更好地将所学知识应用于实际项目中，提升嵌入式系统网络通信的效率和稳定性。

当然，作为一种不断发展的技术，lwIP 也面临着新的挑战和机遇。在未来的发展中，我们期望看到 lwIP 在以下几个方面取得更大的突破和进展：

1，进一步优化资源占用：随着嵌入式系统硬件资源的不断丰富和优化，我们期望看到 lwIP 在资源占用方面取得更大的突破。通过进一步优化内存管理、代码结构和算法，提高 lwIP 的运行效率，以满足更多应用场景的需求。

2，增强安全性：网络安全已经成为当今互联网发展的重要课题。在未来的发展中，我们期望看到 lwIP 在安全性方面取得更大的进展。通过加强加密算法、防火墙机制和安全协议的支持，提高 lwIP 的安全性，以保障嵌入式系统的网络通信安全。

3，丰富功能和支持更多协议：随着物联网、云计算等新兴技术的发展，嵌入式系统的网络通信需求也在不断变化。我们期望看到 lwIP 不断丰富功能、扩展支持的协议和规范，以满足更多应用场景的需求。例如，支持更多的路由协议、多媒体传输协议等，以适应物联网和云计算的发展趋势。

4，跨平台和可移植性：随着嵌入式系统的多样化和异构化，我们期望看到 lwIP 在跨平台和可移植性方面取得更大的进展。通过改进 lwIP 的跨平台支持和可移植性，使其能够在更多种类的硬件平台上稳定运行，满足不同应用场景的需求。

5，社区发展和生态系统建设：作为一个开源项目，lwIP 的社区发展和生态系统建设对于其可持续发展至关重要。我们期望看到 lwIP 社区不断壮大和完善，吸引更多的开发者、企业和组织参与贡献和支持。同时，通过建设更加完善的生态系统，促进 lwIP 在各个领域的应用和推广。

综上所述，lwIP 作为一种轻量级 TCP/IP 协议栈，在嵌入式系统和资源受限的环境中具有广泛的应用前景。通过深入了解和学习 lwIP 的原理、架构、应用和最佳实践，我们可以更好地应对各种网络通信挑战，提升嵌入式系统的网络通信效率和稳定性。同时，我们也有理由相信，随着 lwIP 的不断发展和完善，它将在未来的网络通信领域发挥更加重要的作用。

第一章 lwIP 初探

计算机网络作为现代社会的关键基础设施，已经深入到各个领域，从日常生活到工业生产，从科学研究到军事应用，几乎无处不在。而在这背后，TCP/IP 协议栈作为网络通信的核心，起到了至关重要的作用。它定义了数据如何在不同的网络设备之间传输，确保了数据的可靠、有序和高效传输。然而，在许多嵌入式系统和资源受限的环境中，标准的 TCP/IP 协议栈由于其庞大的体积和复杂的结构，并不适合直接使用。为了解决这一问题，一种轻量级的 TCP/IP 协议栈——lwIP 应运而生。

本章将分为如下几个部分：

1.1 TCP/IP 协议栈是什么

1.2 lwIP 简介

1.3 MAC 内核简介

1.4 PHY 芯片介绍

1.5 以太网接入单片机的方案

1.1 TCP/IP 协议栈是什么

TCP/IP 协议栈是一系列网络协议的总和，构成网络通信的核心骨架，定义了电子设备如何连入因特网以及数据如何在它们之间进行传输。该协议采用 4 层结构，分别是应用层、传输层、网络层和网络接口层，每一层都使用下一层提供的协议来完成自己的需求。由于大部分时间都在应用层工作，下层的事情不用操心。此外，网络协议体系本身很复杂庞大，入门门槛高，因此很难搞清楚 TCP/IP 的工作原理。如果读者想深入了解 TCP/IP 协议栈的工作原理，建议阅读《计算机网络书籍》。

1.1.1 TCP/IP 协议栈架构

TCP/IP 协议栈是一个分层结构的模型，每一层负责不同的网络功能。整个协议栈可以分为四层，从上到下分别是：应用层、传输层、网络层和网络接口层。

1，应用层：这是最顶层，负责处理特定的应用程序细节。在这一层，用户的数据被处理和解释。一些常见的应用层协议包括 HTTP、FTP、SMTP 和 DNS 等。

2，传输层：这一层负责数据包的分割、打包以及传输控制，确保数据能够可靠、有序地到达目的地。主要的传输层协议有 TCP 和 UDP。

3，网络层：负责确定数据包的路径从源到目的地。这一层的主要协议是 IP（Internet Protocol），它负责在主机之间发送和接收数据包。

4，网络接口层：这是最底层，负责将数据转换为可以在物理媒介上发送的信号。这一层的协议涉及到如何将数据帧封装在数据链路层，以便在网络上进行传输。

每一层都使用下一层提供的服务，同时对上一层提供服务。这种分层结构使得协议栈更加灵活，易于扩展和维护。不同层次上的协议一起工作，协调数据在计算机网络中的传输，使得不同的计算机能够相互通信。

需要注意的是，TCP/IP 协议栈和传统的 OSI 模型并不完全对应。TCP/IP 协议栈是一个简化的模型，强调了实际的协议实现和因特网的实际运作方式。相比之下，OSI 模型更加全面和理想化，它提供了一个框架来描述不同系统之间的交互方式。下图是 IOS 协议栈与 TCP/IP 协议栈分层架构的对比图。

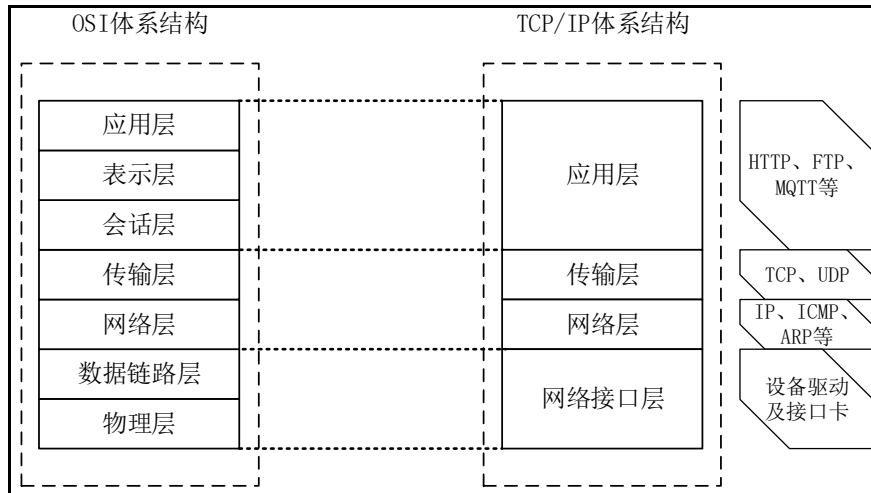


图 1.1.1.1 TCP/IP 协议栈的分层结构

ISO/OSI 分层模型也是一个分层结构，包括七个层次，从上到下分别是：应用层、表示层、会话层、传输层、网络层、数据链路层和物理层。虽然 ISO/OSI 模型为不同的系统之间的通信提供了一个理论框架，但 TCP/IP 协议栈更侧重于实际的协议实现和因特网的实际运作方式。注意：网络技术的发展并不是遵循严格的 ISO/OSI 分层概念。实际上现在的互联网使用的是 TCP/IP 体系结构有时已经演变成为图 1.1.1.2 所示那样，即某些应用程序可以直接使用 IP 层，或甚至直接使用最下面的网络接口层。

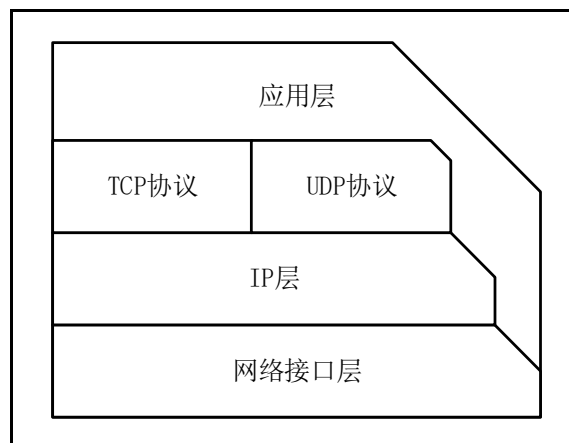


图 1.1.1.2 TCP/IP 体系结构另一种表示方法

无论那种表示方法，TCP/IP 模型各个层次都分别对应于不同的协议。TCP/IP 协议栈负责确保网络设备之间能够通信。它是一组规则，规定了信息如何在网络中传输。其中，这些协议都分布在应用层，传输层和网络层，网络接口层是由硬件来实现。如 Windows 操作系统包含了 CBISC 协议栈，该协议栈就是实现了 TCP/IP 协议栈的应用层，传输层和网络层的功能，网络接口层由网卡实现，所以 CBISC 协议栈和网卡构建了网络通信的核心骨架。因此，无论哪一款以太网产品，都必须符合 TCP/IP 体系结构，才能实现网络通信。注意：路由器和交换机等相关网络设备只实现网络层和网络接口层的功能。

1.1.2 TCP/IP 协议栈的封包和拆包

TCP/IP 协议栈的封包和拆包是指在网络通信中，将数据按照一定的协议和格式进行封装和解析的过程。

在 TCP/IP 协议栈中，数据封装是指在发送端将数据按照协议规定的格式进行打包，以便

项目	说明	
内存占用	lwIP 在内存占用方面进行了优化，使其适用于资源受限的嵌入式系统。具体内存占用取决于配置和版本，但通常来说，它可以仅使用十几 KB 的 RAM 和大约 40K 的 ROM 即可运行	
协议实现	应用层	未实现，需要开发者自行集成或实现特定应用程序
	传输层	实现了 TCP/UDP 协议功能
	网络层	实现了 IP 协议、ARP 协议和 ICMP 协议功能
扩展性	lwIP 支持一些实验性的扩展，例如 IPv6 协议	
硬件接口	WIFI 网络接口、以太网网络接口...	
用户编程接口	RAW、NETCONN、SOCKET 用户编程接口	
硬件支持	网络接口层的实现依赖于底层硬件和驱动程序。开发者需要根据具体的硬件平台和需求进行相应的配置和优化	
其他特性	支持 PPP 协议（点对点通信协议）和 DHCP 协议（动态分配 IP 地址）	

表 1.2.1 lwIP 基本特性

二、lwIP 与 TCP/IP 体系结构的对应关系

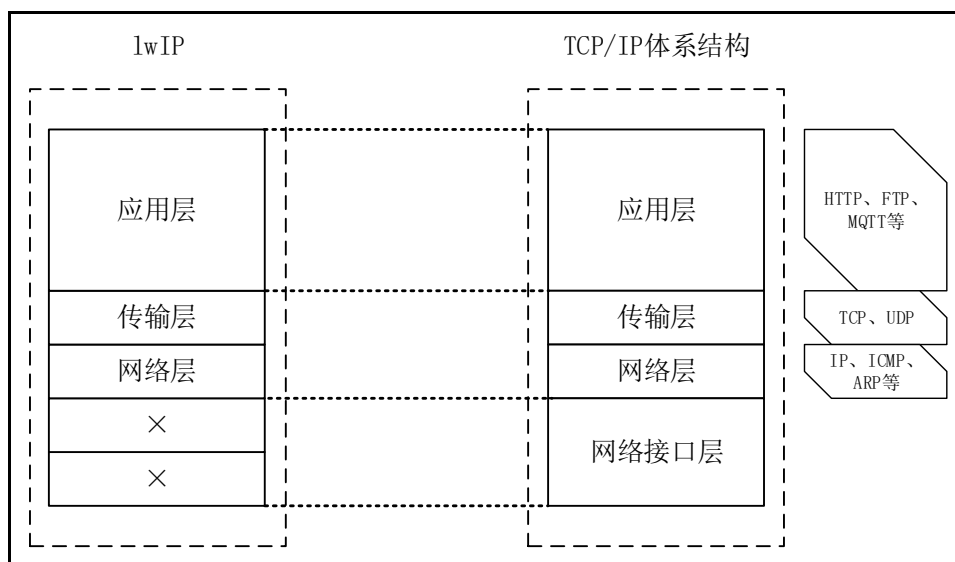


图 1.2.1 lwIP 与 TCP/IP 体系结构的对应图解

从上图可以清晰地看到，lwIP 软件库主要实现了 TCP/IP 体系结构中的三个层次：应用层、传输层和网络层。这些层次共同处理和传输数据包，确保了数据在网络中的可靠传输。然而，网络接口层作为 TCP/IP 协议栈的最底层，其功能并无法通过软件方式完全实现。这一层的主要任务是将数据包转换为光电模拟信号，以便能够在物理媒介上传输。这个过程涉及到与硬件的直接交互，包括数据的调制解调、信号的转换等，这些都是软件难以模拟或实现的。因此，虽然 lwIP 软件库没有实现网络接口层的功能，但通过与底层硬件的紧密配合，它仍然能够提供完整且高效的 TCP/IP 通信功能。这也使得 lwIP 成为一个适用于资源受限的嵌入式系统的理想选择。

在开发过程中，开发者需要根据具体的硬件平台和需求进行相应的配置和优化，以确保 lwIP 能够与硬件完美配合，发挥出最佳的性能。

1.2.1 lwIP 源码下载

lwIP 的源码托管在 Savannah 上，这是一个用于软件开发、维护和分发的平台。开发者可以通过 Savannah 的界面、Git 和邮件列表来下载 lwIP 的源码包。要访问 lwIP 的项目主页，可

以访问 <http://savannah.nongnu.org/projects/lwip/>。在该主页上，应关注“project homepage”和“download area”这两个链接地址。一旦打开 lwIP 项目主页，可以向下滚动并找到“Quick Overview”选项，如下图所示。

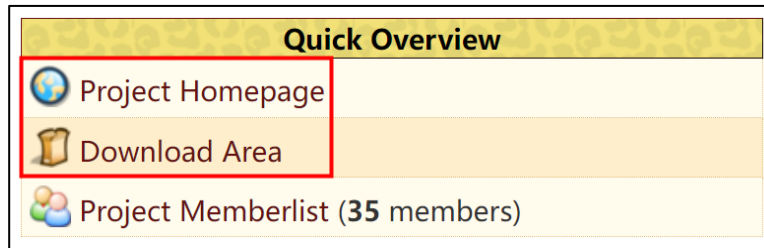


图 1.2.1.1 lwIP 快速预览

点击上图中 Project Homepage 链接地址，读者可以访问到 lwIP 的官方说明文档。该文档详细介绍了 lwIP 的更新日记、常见误解、已发现的 BUG、多线程、优化提示以及相关文件中的函数描述等内容。通过阅读这些信息，读者可以更好地了解 lwIP 的特性和使用方法，为进一步的应用开发提供有力的支持。

点击上图中的 Download Area 链接地址，可以进入 lwIP 源码和 contrib 包的下载页面。由于 lwIP 存在多个版本，本教程选择了最新的版本（2.1.3）作为示例。在下载页面中，除了 lwIP 的源码包，还可以找到一个 contrib 包。这个 contrib 包包含了用户为 lwIP 移植的文件以及与 lwIP 相关的示例程序。需要注意的是，contrib 包并不属于 lwIP 内核的一部分，而是为用户提供移植文件和学习实例的额外资源。下图为 lwIP 源码和 contrib 包的下载页面。

File Name ↓	File Size ↓
Parent directory/	-
drivers/	-
older_versions/	-
contrib-2.1.0.zip	614K
contrib-2.1.0.zip.sig	287
lwip-2.1.3.zip	4M
lwip-2.1.3.zip.sig	310

图 1.2.1.2 lwIP 源码和 contrib 包下载

通过点击上图中的链接，可以下载 lwIP 的 2.1.3 版本源码包（lwip-2.1.3.zip）和 contrib 包（contrib-2.1.0.zip）。下载完成后，可以在本地计算机上找到这两个压缩文件。请确保在下载过程中保持网络连接稳定，以确保完整地下载所需文件。

1.2.2 lwIP 文件说明

根据上一个步骤的步骤，我们已经成功下载了 lwip-2.1.3.zip 和 contrib-2.1.0.zip 这两个压缩包。接下来，作者将为大家介绍这两个压缩包中包含的文件和文件夹。

一、lwIP 源码包文件说明

打开 lwip-2.1.3 文件夹，如下图所示。

doc	文件夹	
src	文件夹	
test	文件夹	
CHANGELOG	文件	186 KB
CMakeLists.txt	文本文档	1 KB
COPYING	文件	2 KB
FEATURES	文件	2 KB
FILES	文件	1 KB
pax_global_header	文件	1 KB
README	文件	5 KB
UPGRADING	文件	13 KB

图 1.2.2.1 lwIP 源码包的文件目录

上图中的文件夹包含了许多文件和子文件夹，这些文件和子文件夹的描述如下。

文件/文件夹	描述
CHANGELOG	记录 lwIP 源码更新
COPYING	记录 lwIP 开源软件 license
FEATURES	描述 lwIP 的特点
FILES	介绍 lwIP 源码包的文件目录信息
README	lwIP 自述文件
UPGRADING	描述 lwIP 升级版本
doc 文件夹	关于 lwIP 的一些文档，这些文档比较零散，不建议参考
src 文件夹	lwIP 内核源码
test 文件夹	官方开发人员测试 lwIP 内部的机制代码

表 1.2.2.1 lwIP 源码包的文件描述

上表中，src 文件夹是 lwIP 源码包中最重要的，它是 lwIP 的内核文件，也是我们移植到工程中的重要文件。接下来，笔者重点讲解 src 文件夹下的文件与文件夹，如下表所示。

文件夹	说明
api 文件夹	实现应用层与传输层递交数据的接口实现
apps 文件夹	实现了多种应用层协议
core 文件夹	包含构建 lwIP 内核的源文件，对应了 TCP/IP 体系架构的传输层、网络层
include 文件夹	包含了 lwIP 软件库的全部头文件
netif 文件夹	实现了网络层与数据链路层交互接口，以及管理不同类型的网卡

表 1.2.2.2 src 文件夹目录介绍

上表详细描述了 lwIP 2.1.3 版本的部分文件和子文件夹的用途和功能。在 api 文件夹中，实现了应用层与传输层之间递交数据的接口，使得应用程序能够与 lwIP 进行通信。apps 文件夹包含了多种应用层协议的实现，如 HTTP、FTP 等，为开发者提供了丰富的应用层支持。core 文件夹则是 lwIP 内核的核心部分，包含了构建 TCP/IP 协议栈的源文件，对应了 TCP/IP 体系架构的传输层、网络层。include 文件夹包含了 lwIP 软件库的全部头文件，为开发者提供了必要的函数声明和数据结构定义。而 netif 文件夹实现了网络层与数据链路层之间的交互接口，并管理不同类型的网卡，确保 lwIP 能够与各种网络硬件设备进行有效的通信。这些文件和子文件夹共同构成了 lwIP 2.1.3 版本，并为开发者提供了全面的 TCP/IP 协议栈实现。

core 文件夹内的子文件主要包括 TCP/IP 协议栈的各种协议、内存管理、数据包管理、网卡管理、网卡接口、基础功能和 API 接口模块等。这些子文件都是 lwIP 协议栈的重要组成部分，负责不同的功能模块，共同实现了 lwIP 的核心功能。这些模块描述如下：

文件	描述
ipv4 文件夹	IPv4 模块相关的源文件（包括 DHCP、ARP、ICMP、IGMP）
ipv6 文件夹	IPv6 模块相关的源文件（包括 DHCP、ARP、ICMP、IGMP）
altcp.c	一种应用程序层（抽象层）用于 TCP 连接的 API 函数
altcp_alloc.c	
altcp_tcp.c.c	
def.c	lwIP 基础类函数定义（主机序和网络序的转换、字符串的查找和比较等）
dns.c	域名解析
inet_chksum.c	计算校验和功能（计算 IP、UDP、TCP 协议的校验和）
init.c	检测 lwIP 宏定义配置错误和提示
ip.c	IP 协议相关的函数（与 ipv4 和 ipv6 文件夹中文件相关）
mem.c	动态内存池管理机制（灵活申请与释放）
memp.c	静态内存堆管理机制（快速申请和释放）
netif.c	lwIP 网卡的操作，例如注册/删除网卡、使能/禁能网卡等功能
pbuf.c	lwIP 对网络数据包的各种操作
raw.c	传输层协议的框架
stats.c	lwIP 内核的统计功能
sys.c	lwIP 的 sys 模块（临界区操作）
tcp.c	TCP 协议相关（TCP 连接的操作、TCP 定时器等）
tcp_in.c	
tcp_out.c	
timeouts.c	lwIP 内核的超时处理机制（实现各个协议的超时处理）
udp.c	UDP 协议相关（对 UDP 连接的操作和 UDP 数据包的操作）

表 1.2.2.3 core 文件描述

二、lwIP 的 contrib 包文件说明

contrib 包提供了 lwIP 移植文件和 lwIP 相关 demo（应用实例），如下图所示。

名称	类型	大小
addons	文件夹	
apps	文件夹	
Coverity	文件夹	
examples	文件夹	
ports	文件夹	
Filelists.cmake	CMAKE 文件	3 KB
Filelists.mk	MK 文件	3 KB

图 1.2.2.2 contrib 包目录

上图中，ports 文件夹提供了基于 FreeRTOS 操作系统的 lwIP 移植文件，以便开发者能够在该操作系统上使用和运行 lwIP。同时，examples 和 apps 文件夹提供了大量的学习实例，让读者更好地了解和学习 lwIP 的应用。至此，我们完成了对 lwIP 源码库和 contrib 包的介绍。

1.3 MAC 内核简介

STM32 的 MAC 内核是一个以太网 MAC 控制器，用于实现以太网通信。它符合 IEEE 802.3 标准，支持 10/100Mbit/s 的数据传输速率，并提供了地址和媒体访问控制方式。

STM32 的 MAC 内核与 TCP/IP 协议栈的层次联系主要体现在网络层和链路层。MAC 内核作为链路层设备，负责与物理层接口（如以太网 PHY）进行通信，并提供了对数据包的封装、

解析和发送接收等操作的支持。同时，MAC 内核还可以通过配置和管理 PHY 设备，实现网络层的地址解析和数据包的路由等功能。因此，STM32 的 MAC 内核是 TCP/IP 协议栈在网络层和链路层的实现基础之一，通过与 TCP/IP 协议栈的配合使用，可以实现以太网通信的各种功能。下图为 STM32 内置以太网架构如下所示：

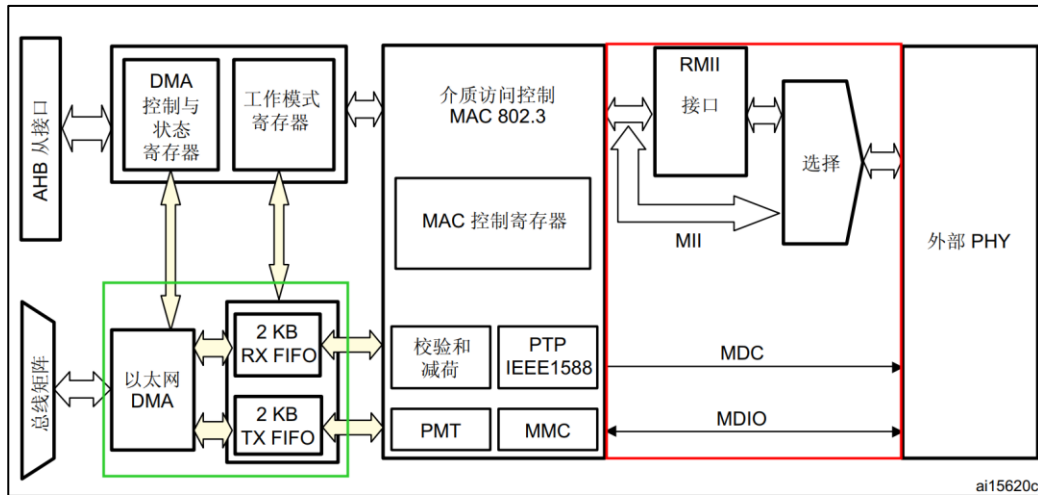


图 1.3.1 ST 以太网框架

从上图描述中可以明确地了解到，绿色框框的 RX FIFO 和 TX FIFO 都是 2KB 的物理存储器，它们分别存储网络层递交的以太网数据和接收的以太网数据。以太网 DMA 是网络层和数据链路层的中间桥梁，是利用存储器到存储器方式传输。红色框框的内容可分为两个部分讲解，RMII 与 MII 是 MAC 内核（数据链路层）与 PHY 芯片（物理层）的数据交互通道，用来传输以太网数据。MDC 和 MDIO 是 MAC 内核对 PHY 芯片的管理和配置，是站管理接口（SMI）所需的通信引脚。站管理接口（SMI）允许应用程序通过 2 条线：时钟（MDC）和数据线（MDIO）访问任意 PHY 寄存器。该接口支持访问多达 32 个 PHY。应用程序可以从 32 个 PHY 中选择一个 PHY，然后从任意 PHY 包含的 32 个寄存器中选择一个寄存器，发送控制数据或接收状态信息。任意给定时间内只能对一个 PHY 中的一个寄存器进行寻址。在 MAC 对 PHY 进行读写操作的时候，应用程序不能修改 MII 的地址寄存器和 MII 的数据寄存器。在此期间对 MII 地址寄存器或 MII 数据寄存器执行的写操作将会被忽略。关于 SMI 接口的详细介绍大家可以参考 STM32F4xx 中文参考手册的 824 页

一、介质独立接口：MII

介质独立接口（MII）是一个媒体独立接口，也称为媒体接口。它是 IEEE-802.3 定义的以太网行业标准。MII 包括一个数据接口，以及一个 MAC 和 PHY 之间的管理接口。数据接口包括分别用于发送器和接收器的两条独立信道，每条信道都有自己的数据、时钟和控制信号。MII 数据接口总共需要 16 个信号。管理接口是个双信号接口：一个是时钟信号，另一个是数据信号。通过管理接口，上层能监视和控制 PHY。下图为 MCU 通过 MII 接口与 PHY 芯片的连接图。

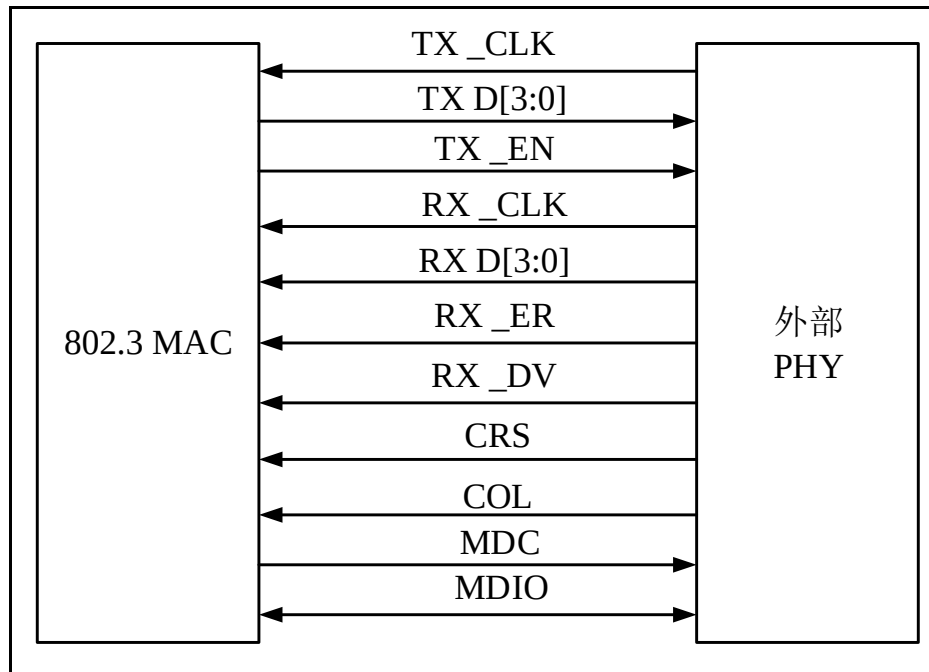


图 1.3.2 MCU 与 PHY 层芯片连接

从上图描述中可以明确地了解到，MII 介质接口使用的引脚数量是非常多的，这也反映出引脚紧缺的 MCU 不适合使用 MII 介质接口来实现以太网数据传输。MII 接口引脚的作用如下。

①：MII_TX_CLK：连续时钟信号。该信号提供进行 TX 数据传输时的参考时序。标称频率为：速率为 10 Mbit/s 时为 2.5 MHz；速率为 100 Mbit/s 时为 25 MHz。

②：MII_RX_CLK：连续时钟信号。该信号提供进行 RX 数据传输时的参考时序。标称频率为：速率为 10 Mbit/s 时为 2.5 MHz；速率为 100 Mbit/s 时为 25 MHz。

③：MII_TX_EN：发送使能信号。

④：MII_TXD[3:0]：数据发送信号。该信号是 4 个一组的数据信号。

⑥：MII_CRS：载波侦听信号。

⑦：MII_COL：冲突检测信号。

⑧：MII_RXD[3:0]：数据接收信号。该信号是 4 个一组的数据信号

⑨：MII_RX_DV：接收数据有效信号。

⑩：MII_RX_ER：接收错误信号。该信号必须保持一个或多个周期（MII_RX_CLK），从而向 MAC 子层指示在帧的某处检测到错误。

二、精简介质独立接口：RMII

简化媒体独立接口（RMII）是一种标准的以太网接口之一，比 MII 有更少的 I/O 传输。RMII 口通常只用两根线来传输数据，而 MII 口需要用四根线。对于 10M 线速，RMII 的速率是 5M，而 MII 的速率是 2.5M；对于 100M 线速，RMII 的速率是 50M，而 MII 的速率是 25M。

RMII 用于传输以太网包，在 MII/RMII 接口是 4/2bit 的，在以太网的 PHY 里需要做串并转换、编解码等才能在双绞线和光纤上进行传输，其帧格式遵循 IEEE 802.3(10M)/IEEE 802.3u(100M)/IEEE 802.1q(VLAN)。下图为 MCU 通过 RMII 接口与 PHY 芯片的连接图。

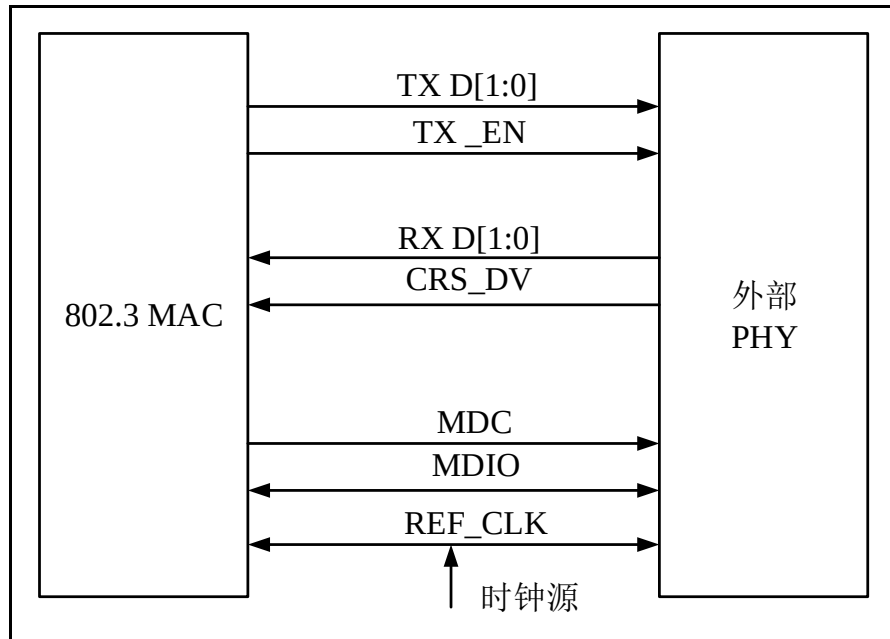


图 1.3.3 MCU 通过 RMII 与 PHY 芯片链接图

从上图描述中可以明确地了解到，REF_CLK 引脚负责提供 50MHz 的时钟频率。这个时钟信号对 MAC 内核和 PHY 芯片都至关重要，确保它们之间的时钟同步，从而保证数据传输的准确性和稳定性。

1.4 PHY 芯片介绍

在 TCP/IP 体系架构中，PHY 芯片是物理层的实现，它负责将数据转换成光电模拟信号以便在网络中进行传输。正点原子常用的 PHY 芯片有两种，分别是 LAN8720A 和 YT8512C。这两款芯片都支持 10/100BASE-T 百兆以太网传输速率。为了更深入地了解这两款以太网芯片，作者将分为两个小节进行讲解。

1.4.1 YT8512C 简介

YT8512C 是一款低功耗的单端口 10/100Mbps 以太网 PHY 芯片，它通过两条标准双绞线电缆收发器实现发送和接收数据所需的所有物理层功能。此外，YT8512C 还通过标准 MII 和 RMII 接口与 MAC 层进行连接。其功能结构图如下所示：

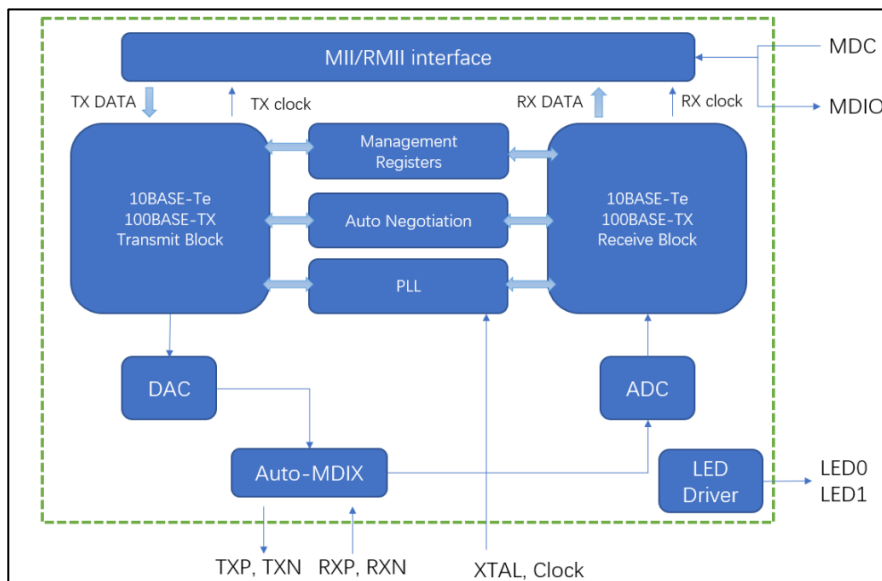


图 1.4.1.1 YT8512C 功能结构图

上图展示了 YT8512C 芯片的内部总架构示意图。从图中我们可以看出，通过 LED0 和 LED1 引脚的电平来设置 PHY 地址，PHY 内部时钟由 XTAL 和 Clock 引脚提供。同时，TXP、TXN、RXP 和 RXN 引脚连接到 RJ45（网口），用于数据的发送和接收。

一、PHY 地址设置

MAC 层通过 SMI 总线对 PHY 芯片进行读写操作。SMI 总线具有强大的控制能力，可以管理多达 32 个 PHY 芯片。通过 PHY 地址的不同，我们可以精确地配置和访问每一个 PHY 芯片。对于 YT8512C 芯片，其 PHY 地址的设置方式如下表所示：

Pin 24 LED0/ PHYAD[0] (PD)	Pin 25 LED1/ PHYAD[1] (PD)	PHY addrss
0	0	00000
0	1	00010
1	0	00001
1	1	00011

表 1.4.1.1 设置 PHYADDR 地址

通过设置 LED0 和 LED1 引脚的电平，我们可以为 YT8512C 芯片分配一个唯一的 PHY 地址。在 MAC 层和 PHY 芯片之间进行通信时，SMI 总线会根据这个地址来识别和操作特定的 PHY 芯片。由于正点原子板载的 PHY 芯片将 LED0 和 LED1 引脚拉低，因此其 PHY 地址为 0x00。在进行 HAL 配置或 PHY 配置时，我们需要在相应的配置文件中指定这个地址。这些配置文件通常包含在开发板的 HAL 库或 PHY 芯片的驱动程序中，如下表所示。

STM32 系列开发板	PHY 配置文件
STM32F4	stm32f4xx_hal_conf.h（探索者、电机）/ethernet_chip.h（阿波罗 429）
STM32F7	ethernet_chip.h
STM32H7	

表 1.4.1.2 不同开发板所对应的文件

在探索者和 DMF407 开发板中，PHY 地址是在 stm32f4xx_hal_conf.h 文件中进行设置的。而阿波罗和北极星开发板的 PHY 地址是在 ethernet_chip.h 文件中进行设置的。由于探索者和 DMF407 开发板所使用的 HAL 库版本较旧，与阿波罗和北极星开发板所使用的 HAL 库版本存在差异，因此它们的移植流程存在较大的不同。对于这部分内容，由于篇幅限制，这里暂时不进行详细讲解。

二、YT8521C 的 RMII 接口模式

YT8521C 的 RMII 接口提供了两种 RMII 模式：

①：**RMII1 模式**：YT8521C 的 TXC 引脚不会输出 50MHz 时钟，这种模式下的连接示意图如下图 1.4.1.2 所示。

②：**RMII2 模式**：YT8521C 的 TXC 引脚会输出 50MHz 时钟，这种模式下的连接示意图如下图 1.4.1.3 所示。

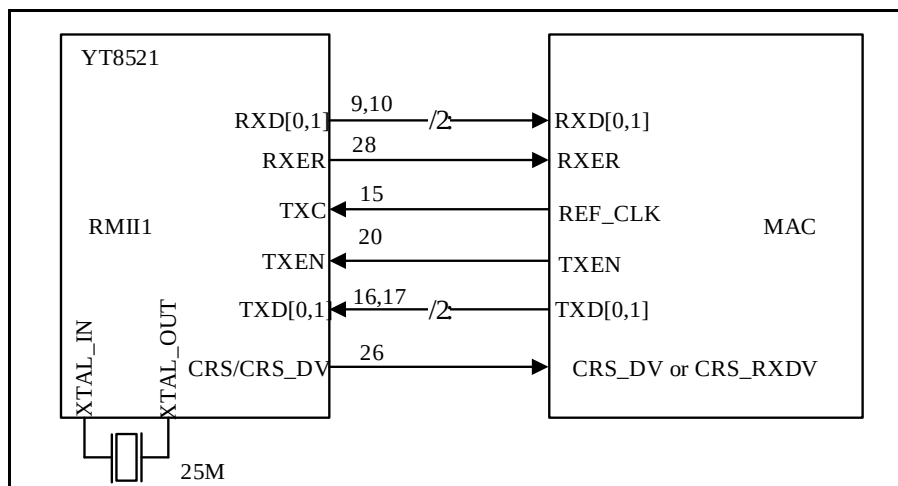


图 1.4.1.2 RMII1 模式连接图

对于 RMII 接口而言，为了使 PHY 芯片与 MAC 内核保持时钟同步操作，需要外部提供 50MHz 的时钟驱动。这个时钟可以来自 PHY 芯片、有源晶振或者 STM32 的 MCO 引脚。如果我们的电路采用 RMII1 模式，PHY 芯片将由 25MHz 晶振经过内部 PLL 倍频达到 50MHz。但是，在这种情况下，MAC 内核并没有被直接提供 50MHz 时钟以与 PHY 芯片保持同步。因此，为了保持时钟同步，我们需要在该基础上使用 MCO 或接入外部 50MHz 晶振来为 MAC 内核提供时钟。简而言之，为了确保 PHY 芯片与 MAC 内核之间的时钟同步，我们需要根据所采用的 RMII 模式，采取相应的措施来提供准确的时钟信号。

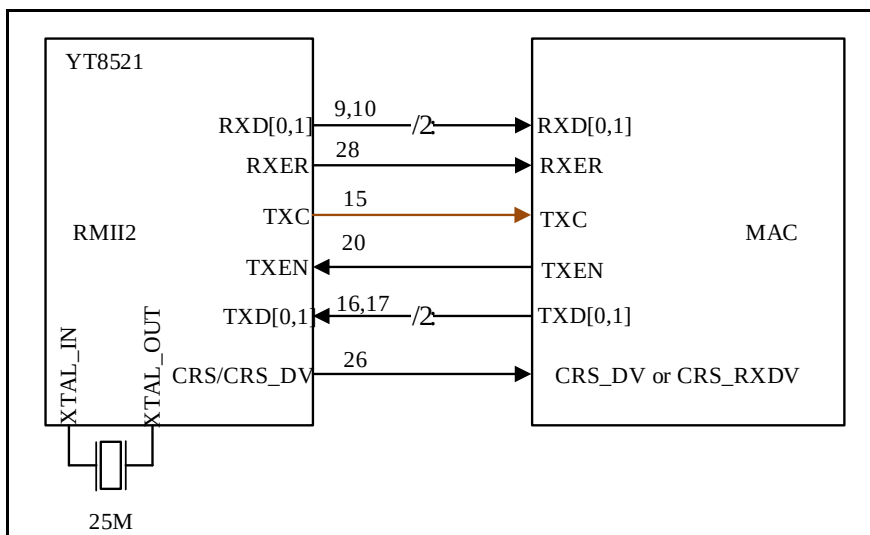


图 1.4.1.3 RMII2 模式

如果电路按照上图模式进行连接，PHY 芯片将通过外接晶振的 25MHz 信号和内部 PLL 倍频操作，生成 50MHz 的内部时钟。接着，PHY 芯片的外围引脚 TXC 将输出 50MHz 时钟频率，这个时钟频率可以直接输入到 MAC 内核，以保持时钟同步。这样，我们就不需要再使用外部晶振或 MCO 引脚来提供 MAC 内核的时钟。

至于 RMII1 模式和 RMII2 模式的选择，这取决于 YT8521C 的 RX_DV (8) 和 RXD3 (12) 引脚的状态。具体如何选择这两种模式，请参考“YT8512C.PDF”手册的 17 到 18 页内容。

三、YT8521C 的寄存器介绍

PHY (物理层芯片) 是由 IEEE 802.3 标准定义的。通常，我们通过 SMI 接口对 PHY 进行管理和控制，也就是读写 PHY 内部寄存器。这些寄存器的地址空间为 5 位，理论上可以定义 0 至 31 共 32 个寄存器。然而，随着 PHY 芯片的功能不断增多，许多 PHY 芯片采用了分页技术来扩展其地址空间，从而定义更多的寄存器。在这里，我们暂时不深入讨论这种情况。IEEE 802.3 标准只定义了 0 至 15 这 16 个寄存器的功能，而 16 至 31 的寄存器则由芯片制造商自由定义。

在 YT8521C 中，存在许多寄存器。在这里，我只会介绍几个我们实际会用到的寄存器 (包括它们的地址，这里使用十进制表示): BCR (0), BSR (1), 以及 PHY 特殊功能寄存器 (17)。首先，让我们看一下 BCR (0) 寄存器。BCR 寄存器的各位功能如下表所示。

位	描述	类型
15	软件复位 1: 软件复位，此位自动清零	R/W
14	回测 0: 正常运行 1: 回测模式	R/W
13	速度选择 0: 10Mbps 1: 100Mbps 注意: 当使用自动协商功能时此位失能	R/W
12	自动协商功能 0: 关闭自动协商功能 1: 打开自动协商功能	R/W
11	掉电 (power down) 0: 正常运行 1: 进入掉电模式 注意: 进入掉电模式前自动协商必须失能	R/W
10	隔离 0: 正常运行 1: PHY 的 RMII 接口电气隔离	R/W
9	重启自动协商功能 0: 正常运行 1: 重启自动协商功能 注意: 此位会被自动清零	R/W SC
8	双工模式 0: 半双工 1: 全双工 注意: 开启自动协商功能后此位失效	R/W
7	碰撞测试 0: 使能 COL 信号测试 1: 关闭 COL 信号测试	RW
6	速度选择	RW
5:0	保留	RO

表 1.4.1.3 BCR 寄存器

从上表描述中可以明确地了解到，以太网速率和双工模式的设置实际上是通过配置 PHY

芯片的 BCR 寄存器来实现的。在 HAL 配置文件或者 ethernet_chip.h 文件中，我们需要定义 BCR 和 BSR 寄存器。具体的代码示例如下：

探索者、DMF407 开发板（HAL 配置文件下）：

```
#define PHY_BCR ((uint16_t)0x0000)
#define PHY_BSR ((uint16_t)0x0001)
```

阿波罗、北极星开发板（PHY 配置文件下）：

```
#define ETH_CHIP_BCR ((uint16_t)0x0000U)
#define ETH_CHIP_BSR ((uint16_t)0x0001U)
```

由于探索者和 DMF407 开发板的例程使用的是 V1.26 版本的 HAL 库，这两个寄存器并不需要读者进行操作。当我们调用 HAL_ETH_Init 函数时，系统会自动根据输入的参数配置 YT8521C 的相应寄存器。然而，阿波罗和北极星开发板的例程采用了当前最新的 HAL 版本。在这个版本中，需要读者手动操作 BCR 寄存器，例如进行自动协商、软复位等操作。

BSR 寄存器各个位介绍如下表所示：

位	描述	类型
15	100BAST-T4 0: 不支持 T4 1: 支持 T4	RO
14	100Base-X_Fd 全双工 0: 不支持 TX 全双工 1: 支持 TX 全双工	RO
13	100Base-X_Fd 半双工 0: 不支持 TX 半双工 1: 支持 TX 半双工	RO
12	10Mbps_Fd 全双工 0: 不支持 10Mbps 全双工 1: 支持 10Mbps 全双工	RO
11	10Mbps_Hd 半双工 0: 不支持 10Mbps 半双工 1: 支持 10Mbps 半双工	RO
10	100Base-T2_Fd 全双工 PHY 不支持 100Base-T2_Fd	RO
9	100Base-T2_Hd PHY 不支持 100Base-T2_Fd	RO
8	扩展状态 0: 不支持 1: 支持	RO
7	链路能力	RO
6	PHY 能够接受管理帧 0: PHY 不接受前导抑制的管理帧 1: PHY 将接受前导抑制的管理帧	RO
5	自协商功能完成 0: 自动协商未完成 1: 自动协商完成	RO
4	远端错误 0: 无远端错误 1: 检测到远端错误	RO/HL
3	自协商功能	RO

	0: 不能执行自协商功能 1: 可以执行自协商功能	
2	连接状态 0: 连接断开 1: 连接建立	RO/LL
1	Jabber 检测 0: 未检测到 jabber 1: 检测到 jabber	RO/LH
0	扩展功能 0: 不支持扩展寄存器 1: 支持扩展寄存器	RO

表 1.4.1.4 BSR 寄存器

SR 寄存器是 YT8521C 的状态寄存器。通过读取该寄存器的值，我们可以获取当前的连接速度、双工状态和连接状态等信息。

接下来，我们将介绍 YT8521C 的特殊功能寄存器。该寄存器的各位功能如下表所示：

位	描述	类型
15:14	速度模式 11: 保留 10: 1000Mbps 01: 100Mbps 00: 10Mbps	RO
13	双工 1: 全双工 0: 半双工	RO
12	页接收实时时间 1: 页支持接收 0: 页不支持接收	RO
11	速度和双工解决 1: 解决 0: 不解决	RO
10	连接状态实时时间 1: 连接 0: 连接失败	RO
9:7	保留	RO
6	MDI 交叉状态 1: MDIX 1: MDIX	RO
5	金属速度下降 1: 下降 0: 不下降	RO
4:2	保留	RO
1	极性实时时间 1: 恢复极性 0: 不恢复极性	RO
0	Jabber 实时时间 1: 设置 Jabber 0: 没有 Jabber	RO

表 1.4.1.5 YT8521C 特殊功能寄存器

在特殊功能寄存器中，我们主要关注 bit13~bit15 这三位。因为系统通过读取这三位值来设置 BCR 寄存器的 bit8 和 bit13。需要注意的是，特殊功能寄存器并不属于 IEEE802.3 规定的前 16 个寄存器，因此每个厂家的实现可能会有所不同。用户需要根据自己实际使用的 PHY 芯片进行相应的修改。

在 ST 提供的以太网驱动文件中，有三个配置项值得读者注意：PHY_SR、PHY_SPEED_STATUS 和 PHY_DUPLEX_STATUS。这些配置项用于描述 PHY 特殊功能寄存器的值，并根据该寄存器的值设置 BCR 寄存器的第 8 位和第 13 位，即双工和网速。在实际应用中，用户需要根据具体的 PHY 芯片和开发环境，参考相应的文档和手册，以确保正确的寄存器配置和操作。

探索者、DMF407 开发板：

```
/* 网卡 PHY 地址设置 */
#define ETHERNET_PHY_ADDRESS    0x00

/* 选择 PHY 芯片 */
#define LAN8720                  0
#define SR8201F                  1
#define YT8512C                  2
#define RTL8201                  3
#define PHY_TYPE                  YT8512C

#if(PHY_TYPE == LAN8720)
#define PHY_SR                    ((uint16_t)0x1F)        /* PHY 状态寄存器地址 */
#define PHY_SPEED_STATUS          ((uint16_t)0x0004)      /* PHY 速度状态 */
#define PHY_DUPLEX_STATUS         ((uint16_t)0x0010)      /* PHY 双工状态 */
#elif(PHY_TYPE == SR8201F)
#define PHY_SR                    ((uint16_t)0x00)        /* PHY 状态寄存器地址 */
#define PHY_SPEED_STATUS          ((uint16_t)0x2020)      /* PHY 速度状态 */
#define PHY_DUPLEX_STATUS         ((uint16_t)0x0100)      /* PHY 双工状态 */
#elif(PHY_TYPE == YT8512C)
#define PHY_SR                    ((uint16_t)0x11)        /* PHY 状态寄存器地址 */
#define PHY_SPEED_STATUS          ((uint16_t)0x4010)      /* PHY 速度状态 */
#define PHY_DUPLEX_STATUS         ((uint16_t)0x2000)      /* PHY 双工状态 */
#elif(PHY_TYPE == RTL8201)
#define PHY_SR                    ((uint16_t)0x10)        /* PHY 状态寄存器地址 */
#define PHY_SPEED_STATUS          ((uint16_t)0x0022)      /* PHY 速度状态 */
#define PHY_DUPLEX_STATUS         ((uint16_t)0x0004)      /* PHY 双工状态 */
```

阿波罗、北极星开发板：

```
/* PHY 地址 */
#define ETH_CHIP_ADDR              ((uint16_t)0x0000U)

/* 选择 PHY 芯片 */
#define LAN8720                    0
#define SR8201F                    1
#define YT8512C                    2
```

```
#define RTL8201                                3
#define PHY_TYPE                               YT8512C

#if(PHY_TYPE == LAN8720)
#define ETH_CHIP_PHYSCSR                      ((uint16_t)0x1F)
#define ETH_CHIP_SPEED_STATUS                 ((uint16_t)0x0004)
#define ETH_CHIP_DUPLEX_STATUS                ((uint16_t)0x0010)
#elif(PHY_TYPE == SR8201F)
#define ETH_CHIP_PHYSCSR                      ((uint16_t)0x00)
#define ETH_CHIP_SPEED_STATUS                 ((uint16_t)0x2020)
#define ETH_CHIP_DUPLEX_STATUS                ((uint16_t)0x0100)
#elif(PHY_TYPE == YT8512C)
#define ETH_CHIP_PHYSCSR                      ((uint16_t)0x11)
#define ETH_CHIP_SPEED_STATUS                 ((uint16_t)0x4010)
#define ETH_CHIP_DUPLEX_STATUS                ((uint16_t)0x2000)
#elif(PHY_TYPE == RTL8201)
#define ETH_CHIP_PHYSCSR                      ((uint16_t)0x10)
#define ETH_CHIP_SPEED_STATUS                 ((uint16_t)0x0022)
#define ETH_CHIP_DUPLEX_STATUS                ((uint16_t)0x0004)
#endif /* PHY_TYPE */
```

作者已经适配了多款 PHY 芯片，并根据 PHY_TYPE 配置项来选择 PHY_SR、PHY_SPEED_STATUS 和 PHY_DUPLEX_STATUS 配置项的数值。这意味着，根据不同的 PHY 芯片类型，需要选择不同的配置项数值来适配不同的 PHY 芯片。

1.4.2 LAN8720A 简介

LAN8720A 是一款低功耗的单端口 10/100Mbps 以太网 PHY 芯片，它通过两条标准双绞线电缆收发器实现发送和接收数据所需的所有物理层功能。此外，LAN8720A 还通过标准 MII 和 RMII 接口与 MAC 层进行连接。其功能结构图如下所示：

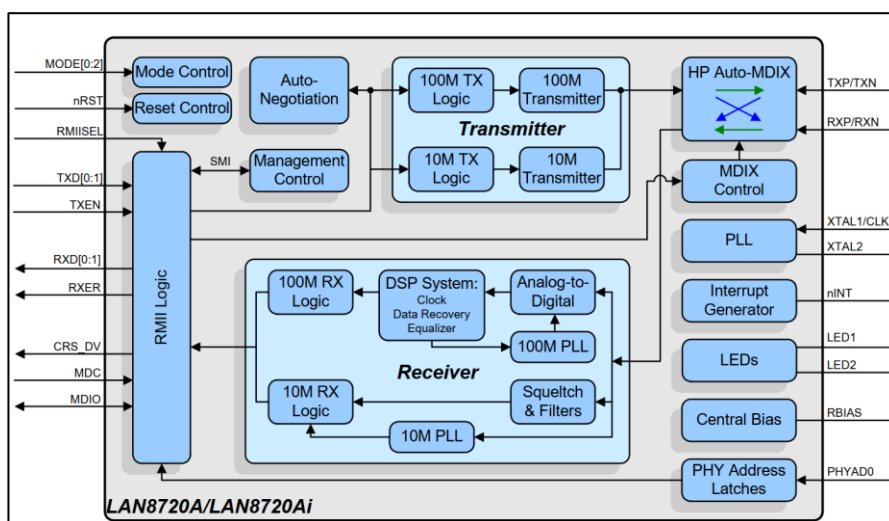


图 1.4.2.1 LAN8720A 能框图

一、LAN8720A 中断管理

LAN8720A 的器件管理接口支持非 IEEE 802.3 规范的中断功能。当中断事件发生且相应中断位被使能时，LAN8720A 会在 nINT（14 脚）产生一个低电平有效的中断信号。LAN8720A 的中断系统提供了两种中断模式：主中断模式和复用中断模式。默认的中断模式为主中断模式，LAN8720A 在上电或复位后即工作在此模式下。模式控制/状态寄存器（十进制地址为 17）的 ALTINT 位决定了工作模式：当 ALTINT 为 0 时，LAN8720A 工作在主模式；当 ALTINT 为 1 时，工作在复用中断模式。正点原子的 STM32 系列开发板并未使用中断功能，如需了解关于中断的具体用法，请参考 LAN8720A 数据手册的 29、30 页。

二、PHY 地址设置

MAC 层通过 SMI 总线对 PHY 芯片进行读写操作。SMI 总线具有强大的控制能力，可以管理多达 32 个 PHY 芯片。通过 PHY 地址的不同，我们可以精确地配置和访问每一个 PHY 芯片。对于 LAN8720A 芯片，其 PHY 地址的设置方式如下表所示。

RXER/PHYAD0 引脚状态	PHY 地址
上拉	0X01
下拉(默认)	0X00

表 1.4.2.1 LAN8720A 地址设置

通过设置 PHYAD0 引脚的电平，我们可以为 LAN8720A 芯片分配一个唯一的 PHY 地址。在 MAC 层和 PHY 芯片之间进行通信时，SMI 总线会根据这个地址来识别和操作特定的 PHY 芯片。由于正点原子板载的 PHY（LAN8720A）芯片将 PHYAD0 引脚拉低，因此其 PHY 地址为 0x00。

三、nINT/REFCLKO 配置

nINTSEL 引脚（2 号引脚）用于设置 nINT/REFCLKO 引脚（14 号引脚）的功能。nINTSEL 配置如下表所示。

nINTSEL 引脚值	模式	nINT/REFCLKO 引脚功能
nINTSEL= 0	REF_CLK Out 模式	nINT/REFCLKO 作为 REF_CLK 时钟源
nINTSEL = 1	REF_CLK In 模式	nINT/REFCLKO 作为中断引脚

表 1.4.2.2 nINTSEL 配置

当工作在 REF_CLK In 模式时，50MHz 的外部时钟信号应接到 LAN8720 的 XTAL1/CLKIN 引脚（5 号引脚）和 STM32 的 RMII_REF_CLK（PA1）引脚上，如下图所示。

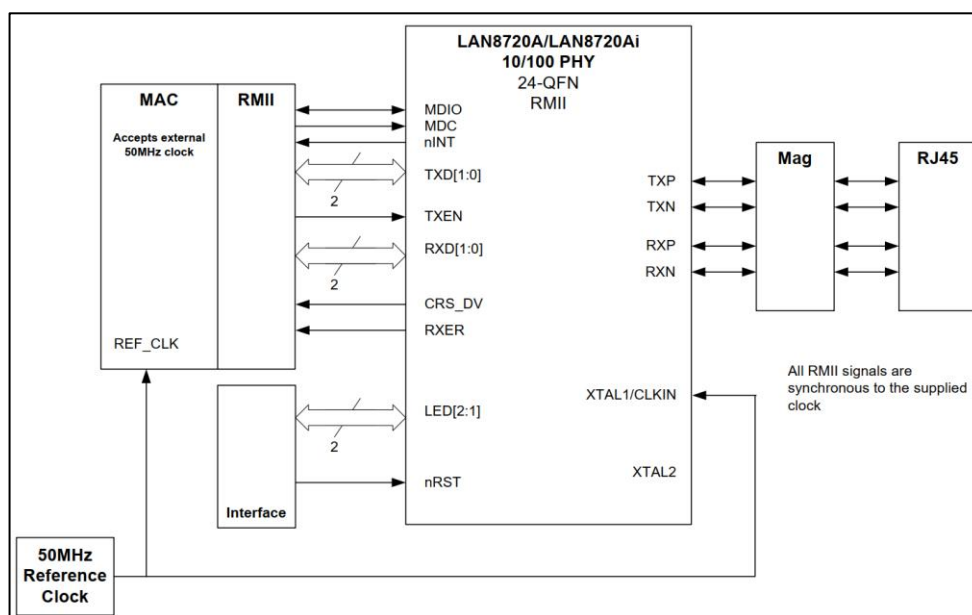


图 1.4.2.2 REF_CLK 连接外部 50MHz 时钟信号

为了降低成本，LAN8720A 可以从外部的 25MHz 的晶振中产生 REF_CLK 时钟。到要使用此功能时应工作在 REF_CLK Out 模式。当工作在 REF_CLO Out 模式时 REF_CLK 的时钟源如下图所示。

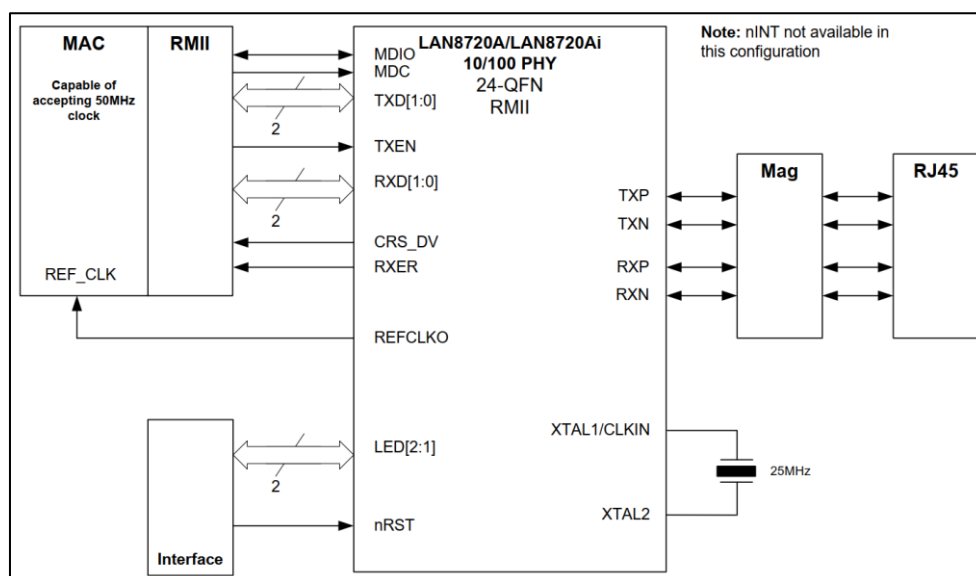


图 1.4.2.3 REF_CLK Out 模式时的 REF_CLK 时钟源

四、LAN8720A 内部寄存器

在 LAN8720A 中，存在许多寄存器。在此，我重点讲解 BCR (0)、BSR (1) 和 PHY 特殊功能寄存器 (31) 这三个寄存器。对于前两个寄存器，我已经在 1.4.1 小节中进行了讲解，这里不再重复。接下来，我将介绍 LAN8720A 的特殊功能寄存器。该寄存器的各位功能如下表所示：

位	描述	类型
15:13	保留	RO
12	自协商完成 0: 自协商未完成或者自协商关闭 1: 自协商完成	RO
11:5	保留	R/W
4:2	速度指示 001: 10BASE-T 半双工 101: 10BAST-T 全双工 010: 100BAST-TX 半双工 110: 100BAST-TX 全双工	RO
1:0	保留	RO

表 1.4.2.5 LAN8720A 特殊功能寄存器

在特殊功能寄存器中我们关心的是 bit2~bit4 这三位，因为系统通过读取这 3 位的值来设置 BCR 寄存器的 bit8 和 bit13。

1.5 以太网接入 MCU 方案

以太网接入方案主要分为两种：全硬件 TCP/IP 协议栈和软件 TCP/IP 协议栈。其中，软件 TCP/IP 协议栈的应用非常广泛，如电脑、交换机等网络设备。而全硬件 TCP/IP 协议栈则是近年来比较新兴的以太网接入方案。下面作者将分别阐述这两种接入方案的差异和优缺点。

一、软件 TCP/IP 协议栈以太网接入方案

这种以太网接入方案是由 lwIP、MAC 内核和 PHY 层芯片共同实现的，用于实现以太网的物理连接。正点原子的探索者、阿波罗、北极星以及电机开发板都采用了这种类型的以太网接入方案。该方案的连接示意图如下：

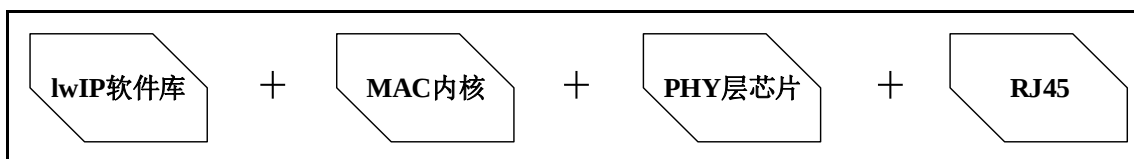


图 1.5.1 MAC 和 PHY 以太网方案示意图

在上述图表中，MCU 需要内置 MAC 内核。该内核对应于 TCP/IP 体系结构中的数据链路层。lwIP 软件库则用于实现 TCP/IP 体系结构中的应用层、传输层和网络层。同时，板载 PHY 层芯片负责实现 TCP/IP 体系结构的物理层功能。通过结合 lwIP、MAC 内核和 PHY 层芯片，我们构建了网络通信的核心骨架。

接下来，我将为大家详细介绍软件 TCP/IP 协议栈方案的优缺点。

优点：

- ①：移植性：可在不同平台、不同编译环境的程序代码经过修改转移到自己的系统中运行。
- ②：可造性：可在 TCP/IP 协议栈的基础上添加和删除相关功能。
- ③：可扩展性：可扩展到其他领域的应用及开发。

缺点：

- ①：内存方面分析：传统的 TCP/IP 方案是移植一个 lwIP 的 TCP/IP 协议（RAM 50K+，ROM 80K+），造成主控可用内存减小。
- ②：从代码量分析：移植 lwIP 可能需要的代码量超过 40KB，对于有些主控芯片内存匮乏来说无疑是一个严重的问题。
- ③：从运行性能方面分析：由于软件 TCP/IP 协议栈方案在通信时候是不断地访问中断机制，造成线程无法运行，如果多线程运行，会使 MCU 的工作效率大大降低。
- ④：从安全性方面分析：软件协议栈会很容易遭受网络攻击，造成单片机瘫痪。

二、硬件 TCP/IP 协议栈以太网接入方案

全硬件 TCP/IP 协议栈是将传统的软件协议 TCP/IP 协议栈用硬件化的逻辑门电路来实现。它具备 TCP、UDP、ICMP 等多种网络协议的内部实现，并且实现了物理层以太网控制（MAC+PHY）以及内存管理等功能。这种方案提供了一套完整的硬件化以太网解决方案。该方案的连接示意图如下：

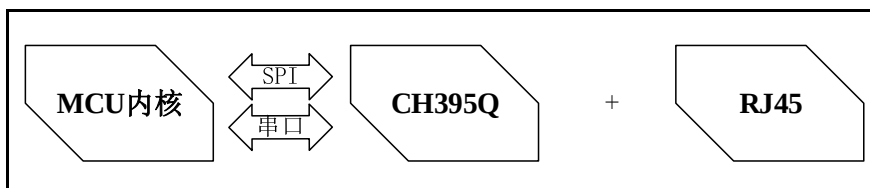


图 1.5.2 硬件协议栈连接示意图

通过这样的方式，全硬件 TCP/IP 协议栈可以大大提高以太网通信的性能和效率，特别是在需要高速、实时处理网络数据的应用场景中。同时，由于将协议栈和网络控制功能集成在芯片内部，它可以降低系统的复杂性和功耗，方便系统集成和部署。然而，这种方案的开发和实现难度较大，需要具备丰富的硬件设计和网络协议经验。

接下来，我将为大家详细介绍硬件 TCP/IP 协议栈的优缺点。

优点：

- ①：从代码量方面来看：相比于传统的接入已经大大减少了代码量。

②：从运行方面来看：极大的减少了中断次数，让单片机更好的完成其他线程的工作。

③：从安全性方面来看：硬件化的逻辑门电路来处理 TCP/IP 协议是不可被攻击的，也就是说网络攻击和病毒对它无效，这也充分弥补了网络协议安全性不足的短板。

缺点：

①：从可扩展性来看：虽然该芯片内部使用逻辑门电路来实现应用层和物理层协议，但是它具有功能局限性，例如给 TCP/IP 协议栈添加一个协议，这样它无法快速添加了。

②：从收发速率来看：全硬件 TCP/IP 协议栈芯片都是采用并口、SPI 以及 IIC 等通讯接口来收发数据，这些数据会受通信接口的速率而影响。

总的来说，全硬件 TCP/IP 协议栈通过简化传统的软件 TCP/IP 协议栈，卸载了 MCU 用于处理 TCP/IP 的线程，从而节约了 MCU 内部的 ROM 等硬件资源。工程师只需进行简单的套接字编程和少量的寄存器操作，即可方便地进行嵌入式以太网上层应用开发。这有助于减少产品开发周期，降低开发成本。通过全硬件 TCP/IP 协议栈的实现，系统可以获得更高的性能和效率，特别是在处理大量网络数据时。此外，由于将协议栈和网络控制功能集成在芯片内部，系统的复杂性和功耗也得到了降低，方便了系统集成和部署。然而，全硬件 TCP/IP 协议栈的开发和实现难度较大，需要具备丰富的硬件设计和网络协议经验。因此，在实际应用中需要根据具体需求和场景进行选择。

第二章 lwIP 无操作系统移植

在本文中，我们将深入探讨如何在无操作系统环境下，将 lwIP（轻量级 IP 协议栈）移植到 STM32 开发板。这个过程通常被称为裸机移植。我们的目标是通过这个教程，帮助读者了解并掌握如何将 lwIP 成功地移植到支持 lwIP 的 STM32 开发板上。请注意，本教程适用于所有正点原子的 STM32 开发板。虽然每个开发板的移植过程可能有所不同，但我们会通过表格和红色字体等形式，明确指出这些差异，以方便读者理解和操作。

本章分为如下几个部分：

- 2.1 lwIP 前期准备
- 2.2 以太网 DMA 描述符
- 2.3 lwIP 移植流程（探索者、DMF407）
- 2.4 lwIP 移植流程（阿波罗、北极星）
- 2.5 lwIP 移植流程（H7R7、H7R3）
- 2.6 使用 AC6 编译工程（扩展）

2.1 lwIP 前期准备

在开始 lwIP 移植之前，我们首先需要有一个基础工程。为了这个目的，作者选择使用了裸机例程中的内存管理实验作为起点，并将其重命名为“lwIP 例程 1 lwIP 裸机移植”工程。

在“lwIP 例程 1 lwIP 裸机移植”工程中，我们在 Middlewares 文件夹下创建了一个名为“lwip”的子文件夹。在“lwip”文件夹下，我们又创建了两个子文件夹：arch 和 lwip_app。arch 文件夹用于存放 lwIP 系统的配置文件；而 lwip_app 文件夹则用于存放用户自定义的文件，例如应用程序的源代码等。通过这样的设置，我们完成了 lwIP 移植的前期准备工作。至于“lwip”文件夹下的具体文件结构，它如下图所示。



图 2.1.1 在 lwip 文件夹下创建 arch、lwip_app 文件夹

2.2 以太网 DMA 描述符

在移植 lwIP 之前，读者需要具备一定的以太网 DMA 描述符相关知识。下面，我将使用 F4_V1.26.0 版本的 HAL 库和 F4_V1.27.0（F7/H7 相同）版本的 HAL 库来详细讲解以太网 DMA 描述符。如果这两个版本之间存在不同的以太网知识，我会进行重点标注，以帮助读者更好地理解和应用。

2.2.1 ST 的以太网 DMA 描述符简介

ST 以太网模块中的以太网数据包传输通过以太网 DMA 描述符在接收/发送 FIFO 和内存之间完成。该模块有两个描述符列表，一个用于接收，另一个用于发送。这两个列表的基地址

被分别写入 DMARDLAR 寄存器和 DMATDLAR 寄存器。此外，这两个描述符列表支持两种链接方式，如下图所示。

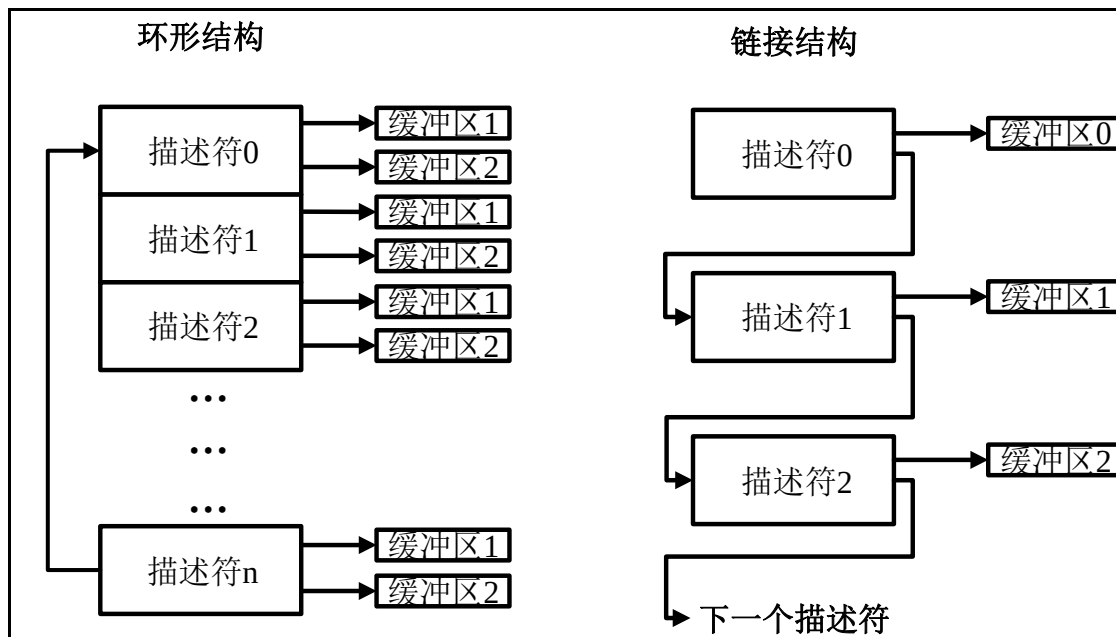


图 2.2.1.1 DMA 描述符列表两种结构

图 2.2.1.1 展示了 DMA 描述符的两种结构：环形结构和链接结构。在 ST 提供的以太网驱动程序 `stm32f4/f7/h7xx_hal_eth.c` 中，初始化发送和接收描述符列表使用的是链接结构。DMA 描述符的链接结构的具体描述如下图 2.2.1.2 和 2.2.1.3 所示。

F4_V1.26.0 版本的 HAL 库（探索者、DMF407）：

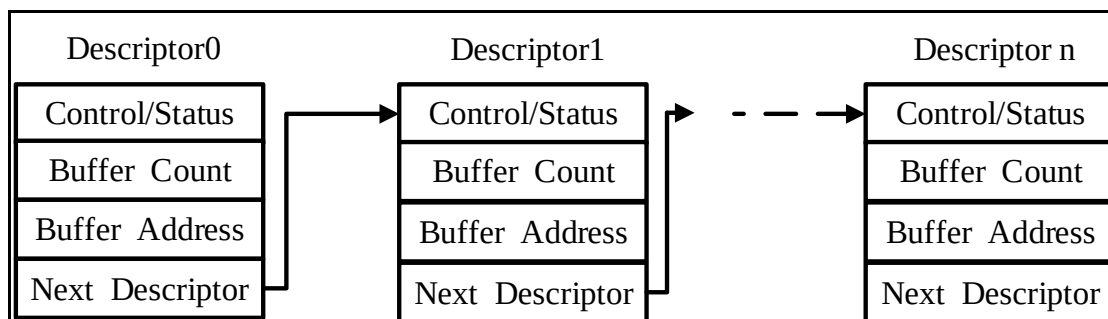


图 2.2.1.2 DMA 描述符链接结构

F4_V1.27.0/F7_V1.17/H7_V1.10 版本的 HAL 库（阿波罗、北极星）：

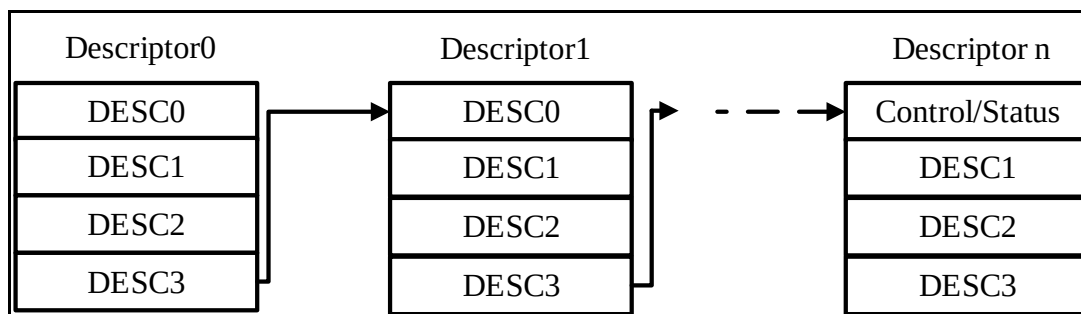


图 2.2.1.3 DMA 描述符链接结构

上图的 DMA 描述符链接结构，由以下函数实现如下表所示：

HAL 版本	函数
F4_V1.26.0	HAL_ETH_DMATxDescListInit()
	HAL_ETH_DMARxDescListInit()
F4_V1.27.0/F7_V1.17/H7_V1.10	ETH_DMATxDescListInit()
	ETH_DMARxDescListInit()

表 2.2.1.1 实现以太网描述符链接结构的函数

这些函数应注意以下几点：

- ✧ 一个以太网数据包可以跨越一个或多个 DMA 描述符。
- ✧ 一个 DMA 描述符只能用于一个以太网数据包。
- ✧ DMA 描述符列表中的最后一个描述符指向第一个，形成循环链式结构。

在 ST 的以太网驱动库 `stm32f4/f7/h7xx_hal_eth.h` 中，有一个结构体 `ETH_DMADescTypeDef`，这个结构体用于定义发送和接收 DMA 描述符。以下是 `ETH_DMADescTypeDef` 结构体的代码示例。

F4_V1.26.0 版本的 HAL 库（探索者、DMF407）：

```
typedef struct {
    __IO uint32_t    Status;           /* 状态 */
    uint32_t    ControlBufferSize;     /* 控制和 buffer1, buffer2 的长度 */
    uint32_t    Buffer1Addr;           /* buffer1 地址 */
    uint32_t    Buffer2NextDescAddr;   /* buffer2 地址或下一个描述符地址 */
    uint32_t    ExtendedStatus;       /* 增强描述符状态 */
    uint32_t    Reserved1;            /* 保留 */
    uint32_t    TimeStampLow;         /* 时间戳低位 */
    uint32_t    TimeStampHigh;        /* 时间戳高位 */
} ETH_DMADescTypeDef;
```

F4_V1.27.0/F7_V1.17/H7_V1.10 版本的 HAL 库（阿波罗、北极星）：

```
typedef struct{
    __IO uint32_t DESC0;
    __IO uint32_t DESC1;
    __IO uint32_t DESC2;
    __IO uint32_t DESC3;
    __IO uint32_t DESC4;
    __IO uint32_t DESC5;
    __IO uint32_t DESC6;
    __IO uint32_t DESC7;
    uint32_t BackupAddr0; /* used to store rx buffer 1 address */
    uint32_t BackupAddr1; /* used to store rx buffer 2 address */
} ETH_DMADescTypeDef;
```

尽管 F4_V1.26.0 版本的 `ETH_DMADescTypeDef` 结构体与 F4_V1.27.0/F7_V1.17/H7_V1.10 版本的 `ETH_DMADescTypeDef` 结构体的成员变量名称存在差异，但它们所实现的功能是一致的。例如，`DESC0` 和 `Status` 都用于描述以太网 DMA 描述符的状态。其他成员变量之间也存在一一对应的关系。这种差异可能是由于版本更新引起的命名或组织更改，但它们的核心功能保持不变。在进行相关编程和配置时，需要根据具体版本的文档和结构体定义进行相应的调整和处理。

2.2.2 DMA 描述符的类型

DMA 描述符分为增强描述符和常规描述符。本教程中使用的是常规描述符，对于常规描述符，我们只需要使用 ETH_DMADescTypeDef 结构体的前四个成员变量。关于常规描述符和增强描述符的区别，建议参考 STM32F4/F7/H7xx 中文参考手册的以太网章节，以获取更详细的信息和指导。

我们知道 STM32F4/F7/H7 的描述符分为发送描述符和接收描述符。发送描述符和接收描述符都使用 ETH_DMADescTypeDef 这个结构体来定义。本教程中，我们将以发送描述符（常规 Tx 的 DMA 描述符）为例进行讲解，而接收描述符（常规 Rx 的 DMA 描述符）与之类似。常规 Tx 的 DMA 描述符可以用下图来表示。

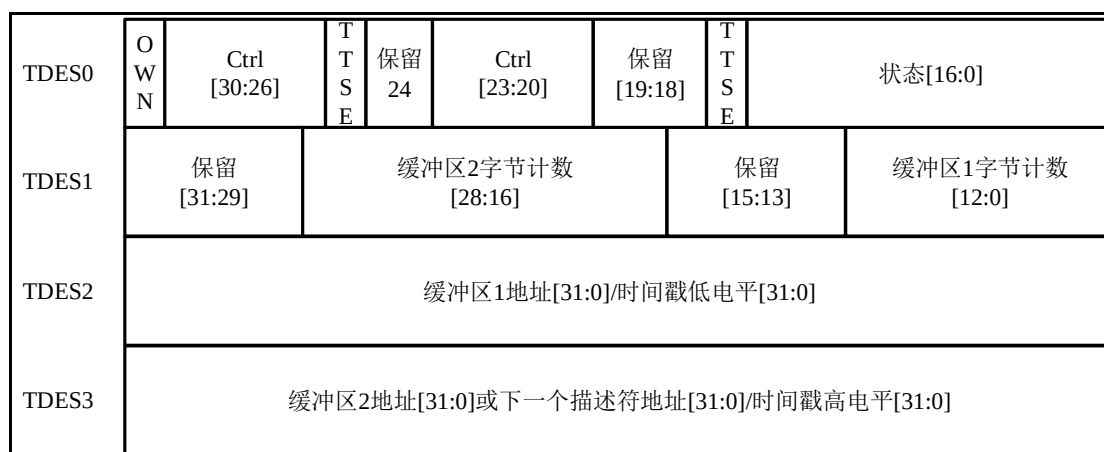


图 2.2.2.1 常规 Tx DMA 描述符图解

从图 2.2.2.1 中可以看到，常规 Tx DMA 描述符包含 4 个“寄存器”：TDES0、TDES1、TDES2 和 TDES3。而如果是增强描述符的话，则会有 8 个“寄存器”。需要注意的是，这 4 个“寄存器”并不是 STM32F4/F7/H7 真实存在的，我们在 STM32F4/F7/H7 的寄存器列表里面是找不到的。它们是由 ETH_DMADescTypeDef 这个结构体描述，被称之为“软件寄存器”。这些“寄存器”也叫做描述符字，它们与 ETH_DMADescTypeDef 结构体的关系如下表所示（此处以 Tx DMA 描述符为例）。

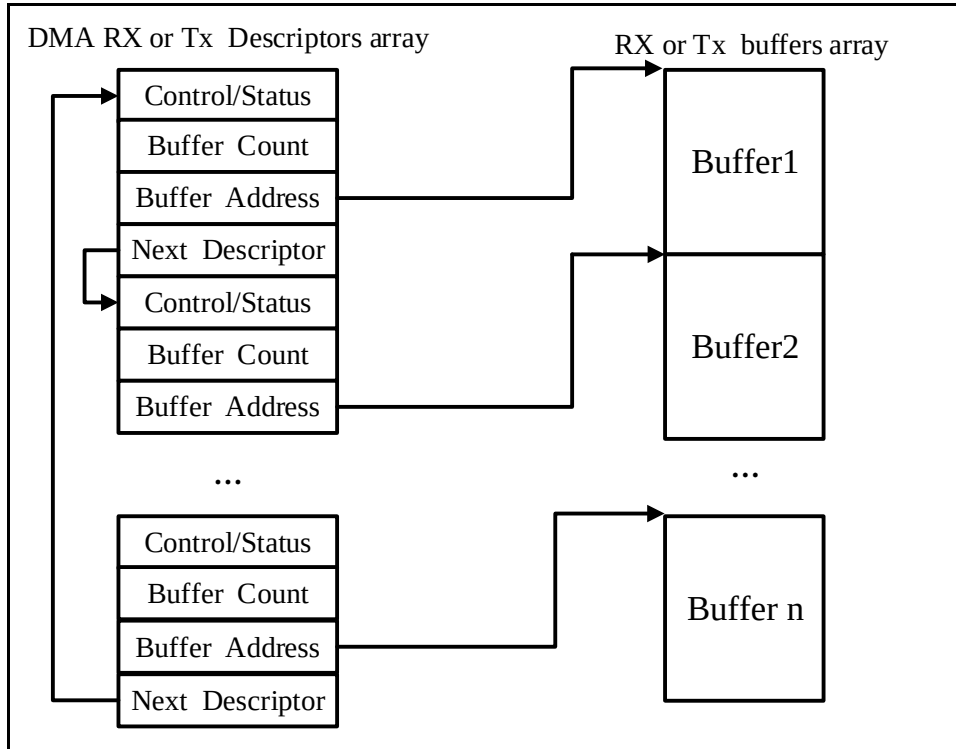
描述符字	结构体成员 (阿波罗、北极星)	结构体成员 (DMF407、探索者)	描述
TDES0	DESC0	Status	常规/增强 Tx DMA 描述符共有
TDES1	DESC1	ControlBufferSize	常规/增强 Tx DMA 描述符共有
TDES2	DESC2	Buffer1Addr	常规/增强 Tx DMA 描述符共有
TDES3	DESC3	Buffer2NextDescAddr	常规/增强 Tx DMA 描述符共有
TDES4	DESC4	ExtendedStatus	增强 Tx DMA 描述符所特有的
TDES5	DESC5	Reserved1	增强 Tx DMA 描述符所特有的
TDES6	DESC6	TimeStampLow	增强 Tx DMA 描述符所特有的
TDES7	DESC7	TimeStampHigh	增强 Tx DMA 描述符所特有的

表 2.2.2.1 描述符字和 ETH_DMADescTypeDef 成员之间的对应关系

以上表格 2.2.2.1 所列的是常规 Tx 的 DMA 描述符，常规 Rx 的 DMA 描述符与之类似。与操作寄存器类似，描述符字的每个位代表不同的状态或控制。关于 Tx 的 DMA 描述符中描述符字各个位的含义，可以参考 STM32F4/F7/H7xx 中文参考手册的以太网章节（Rx DMA 描述符）以获取更详细的信息。通过使用这些描述符字，我们可以配置和操作 DMA 描述符的相关参数和状态，从而实现对以太网数据包传输的全面控制和管理。在理解了这些描述符字和结构体的关系后，我们可以更好地利用 STM32F4/F7/H7 的以太网模块进行数据传输和网络通信。

前面我们了解到，ST 的官方以太网库 `stm32f4/f7/h7xx_hal_eth.c` 中使用了链接结构的 DMA 描述符。在以太网描述符结构体 `ETH_DMADescTypeDef` 中，`BufferAddr`（对应于 `DESC2`）用于保存缓冲区 1 的地址，而 `NextDescriptor`（对应于 `DESC3`）则用于保存下一个描述符的地址。如下图所示。

F4_V1.26.0 版本的 HAL 库（探索者、DMF407）：



F4_V1.27.0/F7_V1.17/H7_V1.10 版本的 HAL 库（阿波罗、北极星）：

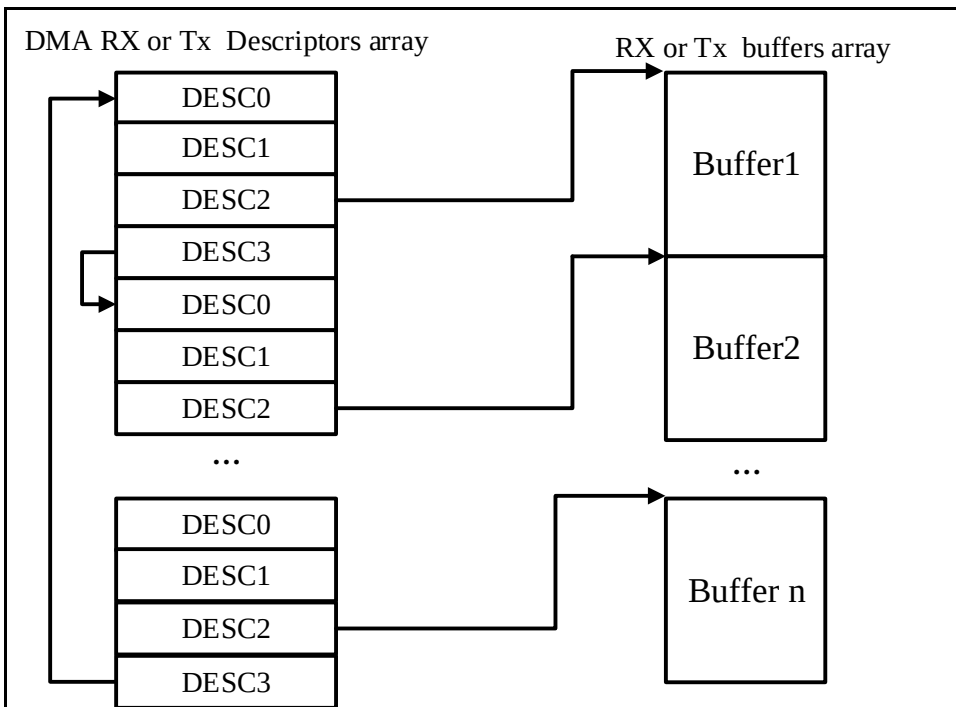


图 2.2.2.2 ST 以太网驱动库中描述符和缓冲区关系

正如图 2.2.2.2 所示，我们需要定义两个 DMA 描述符数组，分别用于发送和接收 DMA。

在例程中，我们将在 ethernet.c 文件中定义两个指向 ETH_DMADescTypeDef 的指针 g_eth_dma_rx_dscr_tab 和 g_eth_dma_tx_dscr_tab。一个用于 DMA 接收，一个用于 DMA 发送。以下是相关的代码示例：

F4_V1.26.0 版本的 HAL 库（探索者、DMF407）：

```
ETH_DMADescTypeDef *g_eth_dma_rx_dscr_tab; /* 以太网 DMA 接收描述符数据结构体指针 */
ETH_DMADescTypeDef *g_eth_dma_tx_dscr_tab; /* 以太网 DMA 发送描述符数据结构体指针 */
```

F4_V1.27.0/F7_V1.17/H7_V1.10 版本的 HAL 库（阿波罗、北极星）：

```
ETH_DMADescTypeDef g_eth_dma_rx_dscr_tab[ETH_RX_DESC_CNT];
ETH_DMADescTypeDef g_eth_dma_tx_dscr_tab[ETH_TX_DESC_CNT];
```

我们可以用内存分配函数给 g_eth_dma_rx_dscr_tab 和 g_eth_dma_tx_dscr_tab 这两个指针分配内存。如果读者不希望使用动态内存，也可以直接定义两个数组。在 ST 以太网驱动库 stm32f4/f7/h7xx_hal_eth.c 中，采用的是 DMA 链接结构。但 g_eth_dma_rx_dscr_tab 和 g_eth_dma_tx_dscr_tab 是两个指针（或数组），因此我们需要将这两个指针（或数组）改为链接结构。在 stm32f4/f7/h7xx_hal_eth.c 文件中，有两个函数可以将两个指针（或数组）变为链接结构，它们分别是：

F4_V1.26.0 版本的 HAL 库（探索者、DMF407）：

HAL_ETH_DMATxDescListInit 和 HAL_ETH_DMARxDescListInit。

F4_V1.27.0/F7_V1.17/H7_V1.10 版本的 HAL 库（阿波罗、北极星）：

ETH_DMATxDescListInit 和 ETH_DMARxDescListInit。

我们通过这两个函数我们就可以将上面两个指针（或数组）变为链接结构了。

2.2.3 如何追踪 DMA 描述符

为了追踪 Rx/Tx 的 DMA 描述符，在下图结构体中定义了两个非常重要的指针变量，如下图所示。

F4_V1.26.0 版本的 HAL 库（探索者、DMF407）：

```
typedef struct
{
    ETH_TypeDef ..... *Instance;
    ETH_InitTypeDef ..... Init;
    uint32_t ..... LinkStatus;
    ETH_DMADescTypeDef ..... *RxDesc;
    ETH_DMADescTypeDef ..... *TxDesc;
    ETH_DMARxFrameInfos ..... RxFrameInfos;
    __IO HAL_ETH_StateTypeDef State;
    HAL_LockTypeDef ..... Lock;
} ETH_HandleTypeDef;
```

F4_V1.27.0/F7_V1.17/H7_V1.10 版本的 HAL 库（阿波罗、北极星）：


```
typedef struct
{
    uint8_t
    *MACAddr;

    ETH_MediaInterfaceTypeDef MediaInterface;

    ETH_DMADescTypeDef
    *TxDesc;

    ETH_DMADescTypeDef
    *RxDesc;

    uint32_t RxBuffLen;
} ETH_InitTypeDef;
```

图 2.2.3.1 结构体 ETH_HandleTypeDef

这两个指针变量指向 ETH_DMADescTypeDef 结构体，在使用过程中它们两个分别指向下一个要发送或者接收的描述符，如下图所示那样：

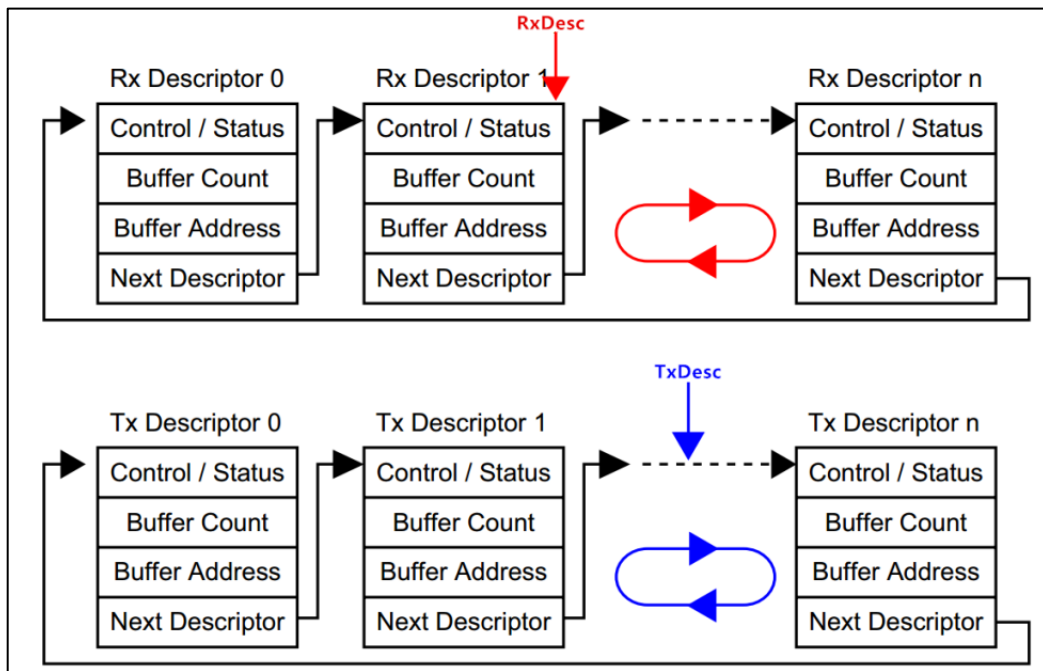


图 2.2.3.2 RxDesc 和 TxDesc 指针的使用

至此 STM32F4/F7/H7 以太网模块中的 DMA 描述符讲解完成了，关于 DMA 描述符的详细内容读者可以参考 STM32 F4/F7/H7xx 中文参考手册。

2.3 lwIP 移植流程(探索者、DMF407)

2.3.1 添加网卡驱动程序

注意：探索者及 DMF407 开发板的移植 lwIP 流程是一致的，它们唯一不同的地方只有三处，稍后笔者会指出区别的部分。

首先，我们需要打开正点原子提供的“lwIP 例程 1 lwIP 裸机移植”实验。然后，在这个实验中，我们需要进入 Drivers\BSP 文件夹。在 Drivers\BSP 文件夹中，可以找到一个名为

ETHERNET 的文件夹。在 ETHERNET 文件夹内，存在两个文件：ethernet.c 和 ethernet.h。这两个文件包含了以太网驱动初始化和 MAC 的驱动程序。在进行移植时，可以将 ethernet.c/h 文件复制到自己工程的 Drivers\BSP\ETHERNET 路径下。此外，还需要将 ethernet.c 文件添加到工程中的 Drivers/BSP 分组。

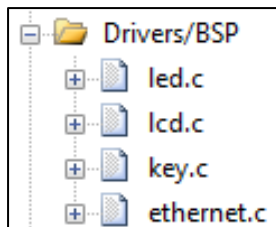


图 2.3.1.1 添加 ethernet.c 后的 Drivers/BSP 分组

下表是 ethernet.c 文件中定义的函数的描述：

函数	描述
ethernet_init()	以太网芯片初始化
HAL_ETH_MspInit()	ETH 底层驱动，时钟使能，引脚配置
ethernet_read_phy()	读取以太网芯片寄存器值
ethernet_write_phy()	向以太网芯片指定地址写入寄存器值
ethernet_chip_get_speed()	获得网络芯片的速度模式
ETH_IRQHandler()	中断服务函数
ethernet_get_eth_rx_size()	获取接收到的帧长度
ethernet_mem_malloc()	为 ETH 底层驱动申请内存
ethernet_mem_free()	释放 ETH 底层驱动申请的内存

表 2.3.1.1 ethernet.c 文件函数

首先我们来看一下 ethernet_init 函数。ethernet_init 函数代码如下所示：

```
/**
 * @brief      以太网芯片初始化
 * @param      无
 * @retval     0, 成功
 *            1, 失败
 */
uint8_t ethernet_init(void)
{
    uint8_t macaddress[6];

    macaddress[0] = lwipdev.mac[0];
    macaddress[1] = lwipdev.mac[1];
    macaddress[2] = lwipdev.mac[2];
    macaddress[3] = lwipdev.mac[3];
    macaddress[4] = lwipdev.mac[4];
    macaddress[5] = lwipdev.mac[5];

    g_eth_handler.Instance = ETH;
    /* 使能自协商模式 */
    g_eth_handler.Init.AutoNegotiation = ETH_AUTONEGOTIATION_ENABLE;
```

```

/* 速度 100M,如果开启了自协商模式,此配置就无效 */
g_eth_handler.Init.Speed = ETH_SPEED_100M;
/* 全双工模式,如果开启了自协商模式,此配置就无效 */
g_eth_handler.Init.DuplexMode = ETH_MODE_FULLDUPLEX;
/* 以太网芯片的地址 */
g_eth_handler.Init.PhyAddress = ETHERNET_PHY_ADDRESS;
/* MAC 地址 */
g_eth_handler.Init.MACAddr = macaddress;
/* 中断接收模式 */
g_eth_handler.Init.RxMode = ETH_RXINTERRUPT_MODE;
/* 硬件帧校验 */
g_eth_handler.Init.ChecksumMode = ETH_CHECKSUM_BY_HARDWARE;
/* RMII 接口 */
g_eth_handler.Init.MediaInterface = ETH_MEDIA_INTERFACE_RMII;

if (HAL_ETH_Init(&g_eth_handler) == HAL_OK)
{
    return 0;    /* 成功 */
}
else
{
    return 1;    /* 失败 */
}
}

```

在这个函数中,我们首先设置了 MAC 地址以及相关的 ETH 配置,然后调用了 HAL_ETH_Init 函数来初始化以太网。这是一个非常简单的函数。

HAL_ETH_MspInit 函数会被 HAL_ETH_Init 函数调用,用于初始化以太网的 IO、使能时钟、配置中断等。以下是该函数的代码示例。

```

/**
 * @brief      ETH 底层驱动,时钟使能,引脚配置
 * @note      此函数会被 HAL_ETH_Init() 调用
 * @param      heth:以太网句柄
 * @retval     无
 */
void HAL_ETH_MspInit(ETH_HandleTypeDef *heth)
{
    GPIO_InitTypeDef gpio_init_struct;

    ETH_CLK_GPIO_CLK_ENABLE();    /* 开启 ETH_CLK 时钟 */
    ETH_MDIO_GPIO_CLK_ENABLE();    /* 开启 ETH_MDIO 时钟 */
    ETH_CRS_GPIO_CLK_ENABLE();    /* 开启 ETH_CRS 时钟 */
    ETH_MDC_GPIO_CLK_ENABLE();    /* 开启 ETH_MDC 时钟 */
    ETH_RXD0_GPIO_CLK_ENABLE();    /* 开启 ETH_RXD0 时钟 */
}

```

```

ETH_RXD1_GPIO_CLK_ENABLE();          /* 开启 ETH_RXD1 时钟 */
ETH_TX_EN_GPIO_CLK_ENABLE();          /* 开启 ETH_TX_EN 时钟 */
ETH_TXD0_GPIO_CLK_ENABLE();          /* 开启 ETH_TXD0 时钟 */
ETH_TXD1_GPIO_CLK_ENABLE();          /* 开启 ETH_TXD1 时钟 */
ETH_RESET_GPIO_CLK_ENABLE();          /* 开启 ETH_RESET 时钟 */
__HAL_RCC_ETH_CLK_ENABLE();           /* 开启 ETH 时钟 */


/* 网络引脚设置 RMII 接口
 * ETH_MDIO -----> PA2
 * ETH_MDC -----> PC1
 * ETH_RMII_REF_CLK-----> PA1
 * ETH_RMII_CRS_DV -----> PA7
 * ETH_RMII_RXD0 -----> PC4
 * ETH_RMII_RXD1 -----> PC5
 * ETH_RMII_TX_EN -----> PG11
 * ETH_RMII_TXD0 -----> PG13
 * ETH_RMII_TXD1 -----> PG14
 * ETH_RESET-----> PD3
 */

/* PA1,2,7 */
gpio_init_struct.Pin = ETH_CLK_GPIO_PIN;
gpio_init_struct.Mode = GPIO_MODE_AF_PP;          /* 推挽复用 */
gpio_init_struct.Pull = GPIO_NOPULL;              /* 不带上下拉 */
gpio_init_struct.Speed = GPIO_SPEED_HIGH;         /* 高速 */
gpio_init_struct.Alternate = GPIO_AF11_ETH;       /* 复用为 ETH 功能 */
HAL_GPIO_Init(ETH_CLK_GPIO_PORT, &gpio_init_struct);

gpio_init_struct.Pin = ETH_MDIO_GPIO_PIN;
HAL_GPIO_Init(ETH_MDIO_GPIO_PORT, &gpio_init_struct);

gpio_init_struct.Pin = ETH_CRS_GPIO_PIN;
HAL_GPIO_Init(ETH_CRS_GPIO_PORT, &gpio_init_struct);

/* PC1 */
gpio_init_struct.Pin = ETH_MDC_GPIO_PIN;
HAL_GPIO_Init(ETH_MDC_GPIO_PORT, &gpio_init_struct); /* ETH_MDC 初始化 */

/* PC4 */
gpio_init_struct.Pin = ETH_RXD0_GPIO_PIN;
HAL_GPIO_Init(ETH_RXD0_GPIO_PORT, &gpio_init_struct); /* ETH_RXD0 初始化 */

```

```

/* PC5 */
gpio_init_struct.Pin = ETH_RXD1_GPIO_PIN;
HAL_GPIO_Init(ETH_RXD1_GPIO_PORT, &gpio_init_struct); /* ETH_RXD1 初始化 */

/* PG11,13,14 */
gpio_init_struct.Pin = ETH_TX_EN_GPIO_PIN;
HAL_GPIO_Init(ETH_TX_EN_GPIO_PORT, &gpio_init_struct); /* ETH_TX_EN 初始化 */

gpio_init_struct.Pin = ETH_TXD0_GPIO_PIN;
HAL_GPIO_Init(ETH_TXD0_GPIO_PORT, &gpio_init_struct); /* ETH_TXD0 初始化 */

gpio_init_struct.Pin = ETH_TXD1_GPIO_PIN;
HAL_GPIO_Init(ETH_TXD1_GPIO_PORT, &gpio_init_struct); /* ETH_TXD1 初始化 */

/* 复位引脚 */
gpio_init_struct.Pin = ETH_RESET_GPIO_PIN; /* ETH_RESET 初始化 */
gpio_init_struct.Mode = GPIO_MODE_OUTPUT_PP; /* 推挽输出 */
gpio_init_struct.Pull = GPIO_NOPULL; /* 无上下拉 */
gpio_init_struct.Speed = GPIO_SPEED_HIGH; /* 高速 */
HAL_GPIO_Init(ETH_RESET_GPIO_PORT, &gpio_init_struct);

ETHERNET_RST(0); /* 硬件复位 */
delay_ms(50);
ETHERNET_RST(1); /* 复位结束 */

HAL_NVIC_SetPriority(ETH_IRQn, 6, 0); /* 网络中断优先级应该高一点 */
HAL_NVIC_EnableIRQ(ETH_IRQn);
}

```

此函数主要用于初始化以太网的引脚，并设置以太网中断的优先级。最后，它会对 PHY 芯片进行硬件复位。值得注意的是，DMF407 电机开发板和探索者在以太网芯片复位引脚的选择上有所不同。DMF407 使用 PI8 作为以太网芯片的复位引脚，而探索者则使用 PD3。如需获取更多关于这两款开发板的信息，建议参考它们各自的用户手册或技术文档。[这里是探索者与 DMF407 开发板第一处区别。](#)

接下来，我们将介绍 ethernet_read_phy 和 ethernet_write_phy 这两个函数。这两个函数用于读取和配置 PHY 芯片的内部寄存器。实际上，它们是对 HAL_ETH_ReadPHYRegister 和 HAL_ETH_WritePHYRegister 函数的简单封装。

ethernet_chip_get_speed 函数可用来获取网络的速度和双工状态，该函数代码如下：

```

/**
 * @brief      获得网络芯片的速度模式
 * @param      无
 * @retval     1:获取 100M 成功

```

```

0:失败

*/
uint8_t ethernet_chip_get_speed(void)
{
    uint8_t speed;
    #if(PHY_TYPE == LAN8720) /* 从 LAN8720 的 31 号寄存器中读取网络速度和双工模式 */
        speed = ((ethernet_read_phy(PHY_SR) & PHY_SPEED_STATUS) >> 4);
    #elif(PHY_TYPE == SR8201F) /* 从 SR8201F 的 0 号寄存器中读取网络速度和双工模式 */
        speed = ((ethernet_read_phy(PHY_SR) & PHY_SPEED_STATUS) >> 13);
    #elif(PHY_TYPE == YT8512C) /* 从 YT8512C 的 17 号寄存器中读取网络速度和双工模式 */
        speed = ((ethernet_read_phy(PHY_SR) & PHY_SPEED_STATUS) >> 14);
    #elif(PHY_TYPE == RTL8201) /* 从 RTL8201 的 16 号寄存器中读取网络速度和双工模式 */
        speed = ((ethernet_read_phy(PHY_SR) & PHY_SPEED_STATUS) >> 1);
    #endif
    return speed;
}

```

由于 YT8512C、SR8201F 和 RTL8201 这三款 PHY 芯片都是兼容一个原理图的，所以我们根据 PHY_TYPE 宏定义来读取 PHY 芯片的状态。

ETH_IRQHandler 函数为以太网 DMA 接收中断服务函数，中断服务函数代码如下：

```

extern void lwip_pkt_handle(void); /* 在 lwip_comm.c 里面定义 */

/**
 * @breif      中断服务函数
 * @param      无
 * @retval     无
 */
void ETH_IRQHandler(void)
{
    if (ethernet_get_eth_rx_size(g_eth_handler.RxDesc))
    {
        lwip_pkt_handle(); /* 处理以太网数据，即将数据提交给 LWIP */
    }
    /* 清除 DMA 中断标志位 */
    __HAL_ETH_DMA_CLEAR_IT(&g_eth_handler, ETH_DMA_IT_NIS);
    /* 清除 DMA 接收中断标志位 */
    __HAL_ETH_DMA_CLEAR_IT(&g_eth_handler, ETH_DMA_IT_R);
}

```

在中断服务函数中，我们首先判断接收到的数据包长度是否为 0。如果数据包长度不为 0，则程序会调用 lwip_pkt_handle 函数来处理接收到的数据包。关于这个函数的具体操作，我们稍后会进行详细讲解。在处理完成后，需要清除 DMA 接收中断标志位和 DMA 总中断标志位，以确保下一次中断能够正确触发。

ethernet_get_eth_rx_size 函数用来获取当前接收到的以太网帧长度，该函数代码如下：

```

/**

```

```

* @brief      获取接收到的帧长度
* @param      dma_rx_desc : 接收 DMA 描述符
* @retval     frameLength : 接收到的帧长度
*/
uint32_t ethernet_get_eth_rx_size(ETH_DMADescTypeDef *dma_rx_desc)
{
    uint32_t frameLength = 0;

    if (((dma_rx_desc->Status & ETH_DMARXDESC_OWN) == (uint32_t)RESET) &&
        ((dma_rx_desc->Status & ETH_DMARXDESC_ES) == (uint32_t)RESET) &&
        ((dma_rx_desc->Status & ETH_DMARXDESC_LS) != (uint32_t)RESET))
    {
        frameLength = ((dma_rx_desc->Status & ETH_DMARXDESC_FL) >>
                       ETH_DMARXDESC_FRAME_LENGTHSHIFT);
    }

    return frameLength;
}

```

ethernet_mem_malloc 函数就是为我们前面提到的那四个指针内存分配，函数代码如下：

```

/**
* @brief      为 ETH 底层驱动申请内存
* @param      无
* @retval     0, 正常
*             1, 失败
*/
uint8_t ethernet_mem_malloc(void)
{
    g_eth_dma_rx_dscr_tab = mymalloc(SRAMIN, ETH_RXBUFNB *
                                      sizeof(ETH_DMADescTypeDef)); /* 申请内存 */
    g_eth_dma_tx_dscr_tab = mymalloc(SRAMIN, ETH_TXBUFNB *
                                      sizeof(ETH_DMADescTypeDef)); /* 申请内存 */
    g_eth_rx_buf = mymalloc(SRAMIN, ETH_RX_BUF_SIZE * ETH_RXBUFNB); /* 申请内存 */
    g_eth_tx_buf = mymalloc(SRAMIN, ETH_TX_BUF_SIZE * ETH_TXBUFNB); /* 申请内存 */

    if (!(uint32_t)&g_eth_dma_rx_dscr_tab || !(uint32_t)&g_eth_dma_tx_dscr_tab
        || !(uint32_t)&g_eth_rx_buf || !(uint32_t)&g_eth_tx_buf)
    {
        ethernet_mem_free();
        return 1; /* 申请失败 */
    }

    return 0; /* 申请成功 */
}

```


ethernet_mem_free 函数为释放内存函数，功能是将 g_eth_rx_buf、g_eth_tx_buf、g_eth_dma_rx_dscr_tab 和 g_eth_dma_tx_dscr_tab 这四个指针的内存释放掉，这里就不贴出具体代码了。

至此，网卡驱动添加完成，我们编译工程会出现很多错误，如下图所示。

```
#20: identifier "ETH_HandleTypeDef" is undefined
/* 以太网句柄 */
#20: identifier "ETH_DMADescTypeDef" is undefined
/* 以太网DMA接收描述符数据结构体指针 */
#20: identifier "ETH_DMADescTypeDef" is undefined
/* 以太网DMA发送描述符数据结构体指针 */
#20: identifier "ETH_DMADescTypeDef" is undefined
```

图 2.3.2.8 缺少以太网 hal 文件

上述的错误就是工程缺少了以太网驱动文件，我们在工程的 Drivers/STM32F4xx_HAL_Driver 分组下添加 stm32f4xx_hal_eth.c 以太网驱动文件。添加驱动文件之后我们打开 stm32f4xx_hal_conf.h 文件，并在该文件下使能 HAL_ETH_MODULE_ENABLED 以太网模块，这个配置项是在该文件的最顶端定义的，最后我们在该文件下添加以下源码：

```
/* MAC ADDRESS: MAC_ADDR0:MAC_ADDR1:MAC_ADDR2:MAC_ADDR3:MAC_ADDR4:MAC_ADDR5 */
#define MAC_ADDR0 2U
#define MAC_ADDR1 0U
#define MAC_ADDR2 0U
#define MAC_ADDR3 0U
#define MAC_ADDR4 0U
#define MAC_ADDR5 0U

/* Definition of the Ethernet driver buffers size and count */
#define ETH_RX_BUF_SIZE ETH_MAX_PACKET_SIZE
#define ETH_TX_BUF_SIZE ETH_MAX_PACKET_SIZE
#define ETH_RXBUFNB ((uint32_t)5)
#define ETH_TXBUFNB ((uint32_t)5)

/* Section 2: PHY configuration section */

/* ETHERNET PHY Address*/
#define ETHERNET_PHY_ADDRESS 0x00

/* PHY Reset delay these values are based on a 1 ms SysTick interrupt*/
#define PHY_RESET_DELAY ((uint32_t)0x00000FFF)
#define PHY_CONFIG_DELAY ((uint32_t)0x00000FFF)
#define PHY_READ_TO ((uint32_t)0x00000FFF)
#define PHY_WRITE_TO ((uint32_t)0x00000FFF)

/* Section 3: Common PHY Registers */
#define PHY_BCR ((uint16_t)0x00)
#define PHY_BSR ((uint16_t)0x01)
#define PHY_RESET ((uint16_t)0x8000)
#define PHY_LOOPBACK ((uint16_t)0x4000)
```

```
#define PHY_FULLDUPLEX_100M          ((uint16_t)0x2100)
#define PHY_HALFDUPLEX_100M          ((uint16_t)0x2000)
#define PHY_FULLDUPLEX_10M           ((uint16_t)0x0100)
#define PHY_HALFDUPLEX_10M           ((uint16_t)0x0000)
#define PHY_AUTONEGOTIATION           ((uint16_t)0x1000)
#define PHY_RESTART_AUTONEGOTIATION  ((uint16_t)0x0200)
#define PHY_POWERDOWN                 ((uint16_t)0x0800)
#define PHY_ISOLATE                   ((uint16_t)0x0400)
#define PHY_AUTONEGO_COMPLETE         ((uint16_t)0x0020)
#define PHY_LINKED_STATUS              ((uint16_t)0x0004)
#define PHY_JABBER_DETECTION          ((uint16_t)0x0002)

/* Section 4: Extended PHY Registers */

#define LAN8720                        0
#define SR8201F                       1
#define YT8512C                       2
#define RTL8201                       3
#define PHY_TYPE                      YT8512C

#if(PHY_TYPE == LAN8720)
#define PHY_SR                        ((uint16_t)0x1F)
#define PHY_SPEED_STATUS              ((uint16_t)0x0004)
#define PHY_DUPLEX_STATUS             ((uint16_t)0x0010)
#elif(PHY_TYPE == SR8201F)
#define PHY_SR                        ((uint16_t)0x00)
#define PHY_SPEED_STATUS              ((uint16_t)0x2020)
#define PHY_DUPLEX_STATUS             ((uint16_t)0x0100)
#elif(PHY_TYPE == YT8512C)
#define PHY_SR                        ((uint16_t)0x11)
#define PHY_SPEED_STATUS              ((uint16_t)0x4010)
#define PHY_DUPLEX_STATUS             ((uint16_t)0x2000)
#elif(PHY_TYPE == RTL8201)
#define PHY_SR                        ((uint16_t)0x10)
#define PHY_SPEED_STATUS              ((uint16_t)0x0022)
#define PHY_DUPLEX_STATUS             ((uint16_t)0x0004)
#endif /* PHY_TYPE */
```

注意：“Section 4”下的代码是作者对多款的 PHY 芯片做了兼容处理，大家请根据 PHY_TYPE 宏定义选择。

至此，我们重新编译了一次，但工程仍然出现了一个错误。这个错误是由于在以太网 DMA 接收中断服务函数 ETH_IRQHandler 中调用了 lwip_pkt_handle 函数，而该函数在 ethernet.c 文件中没有定义。实际上，lwip_pkt_handle 函数是在 lwip_comm.c 文件中定义的。这

个错误提示可以先不用管，关于 `lwip_pkt_handle` 函数的详细讲解会在后续进行。

注意：目前 DMF407 开发板板载的 PHY 芯片为 LAN8720A（可能以后换成 YT8512C），而最新推出的探索者开发板板载的 PHY 芯片为 YT8512C。因此，读者必须在 `PHY_TYPE` 配置项中写入相应的参数。**这是探索者与 DMF407 移植的第二处重要区别。**

2.3.2 添加 lwIP 源文件

将 lwip-2.1.3 文件夹中的 `src` 文件夹复制到“lwIP 例程 1 lwIP 裸机移植”工程的 `Middlewares\lwip` 路径下，如下图所示。



图 2.3.2.1 复制 lwIP 源码到工程中

打开“lwIP 例程 1 lwIP 裸机移植”工程，并添加 `Middlewares\lwip/src/api`、`Middlewares\lwip/src/core`、`Middlewares\lwip/src/netif` 和 `Middlewares\lwip/arch` 这四个分组。下面我们分别说明这些分组需要添加哪些文件：

(1) `Middlewares\lwip/src/api` 分组添加上图中的 `src/api` 路径下的全部.c 文件，如下图所示。

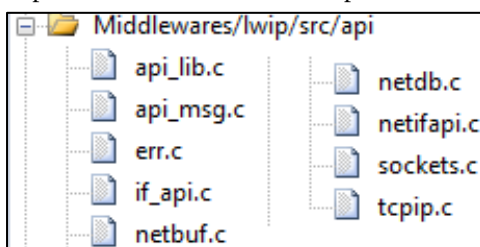


图 2.3.2.2 Middlewares\lwip/src/api 分组添加的文件

(2) `Middlewares\lwip/src/core` 分组添加图 2.3.2.1 中的 `src/core` 路径下除 `ipv6` 文件夹的全部.c 文件，如下图所示。

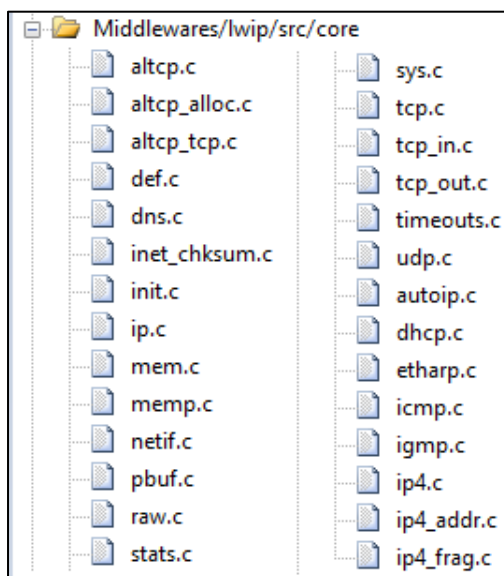


图 2.3.2.3 Middlewares\lwip/src/core 分组添加的文件

(3) `Middlewares\lwip/src/netif` 分组添加图 2.3.2.1 中的 `src/netif` 路径下的 `ethernet.c` 文件，如下图所示。

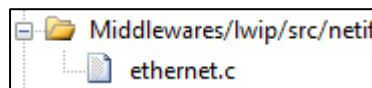


图 2.3.2.4 Middlewares/lwip/src/netif 分组添加的文件

(4) Middlewares/lwip/arch 添加图 2.3.2.1 中的 arch 文件夹下的全部.c 文件，前面笔者没有在该文件夹下放置任何的文件，接下来笔者来讲解这个文件夹到底存放哪些文件。

arch 文件夹主要存放 lwipopts.h、cc.h、ethernetif.c/h 和 lwip_comm.c/h 六个文件，这些文件的作用如下表所示：

文件	描述
lwipopts.h	lwIP 配置文件
cc.h	配置 lwIP 数据类型
ethernetif.c/h	网卡文件，专门处理接收的数据包以及发送的数据包
lwip_comm.c/h	配置 lwIP 相关参数，比如 MAC 地址

表 2.3.2.1 arch 文件夹存放的文件

上表中的文件除了 lwip_comm.c/h 以及 ethernetif.h 之外，都可以在“contrib-2.1.0”包下获取。为了确保正确性，建议从 ST 官方提供的 lwIP 工程中获取 lwipopts.h 和 cc.h。这些文件已经在 ST 芯片上实现了 lwIP 移植，可以直接拷贝到工程中。

要下载 ST 官方 lwIP 对应 MCU 的工程，请访问以下网址：<https://www.st.com/zh/embedded-software/stm32cubef4.html>。在此网址上，下载 HAL 版本的 F4 工程包，即 STM32Cube_FW_F4_V1.26.0 压缩包。这个压缩包包含了所需文件，例如在 STM32Cube_FW_F4_V1.26.0\Middlewares\Third_Party\LwIP\system\arch 路径下获取 cc.h 文件，在 STM32Cube_FW_F4_V1.26.0\Projects\STM32469I_EVAL\Applications\LwIP\LwIP_TCP_Echo_Client\Inc 路径下获取 lwipopts.h 文件。这些文件都是针对 ST 芯片编写的。按照上述地址，将 cc.h 和 lwipopts.h 复制到实验的 arch 文件夹下，如下图所示。

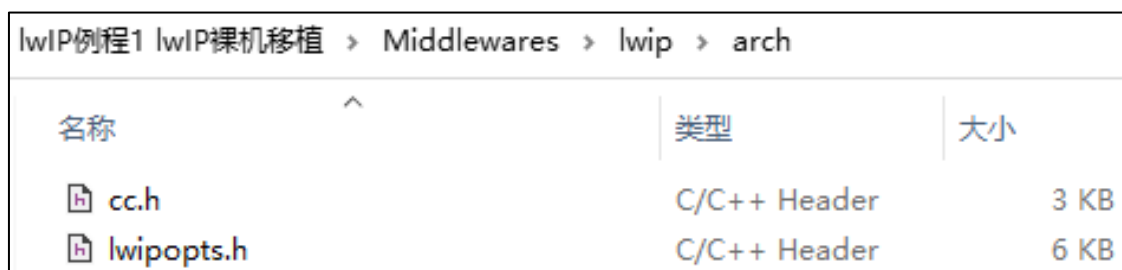


图 2.3.2.5 arch 文件夹保存 cc.h 和 lwipopts.h 文件

ethernetif.c 文件主要定义了底层驱动函数，这些函数在网卡中至关重要。这个文件位于“contrib 包的 contrib-2.1.0\examples\ethernetif”路径下，可以直接拿来使用。需要注意的是，ethernetif.c 是一个通用模板，需要用户根据实际情况进行修改。如果需要，也可以参考 STM32Cube_FW_F4_V1.26.0 压缩包下的 ethernetif.c（裸机工程）文件。在这里，我使用的是正点原子已经编写好的 ethernetif.c 网卡文件。最后，在工程中创建一个 ethernetif.h 文件，用于声明 ethernetif.c 中的函数，并将其保存在 arch 文件夹下，如下图所示。

lwIP例程1 lwIP裸机移植 > Middlewares > lwip > arch	
名称	类型
 cc.h	C++ Header file
 ethernetif.c	C 文件
 ethernetif.h	C++ Header file
 lwipopts.h	C++ Header file

图 2.3.2.6 arch 文件夹添加 ethernetif.c/h 文件

上图中的 ethernetif.h 文件源码如下所示：

```
#ifndef __ETHERNETIF_H__
#define __ETHERNETIF_H__
#include "lwip/err.h"
#include "lwip/netif.h"

err_t ethernetif_init(struct netif *netif); /* 网卡初始化函数 */
void ethernetif_input(struct netif *netif); /* 数据包输入函数 */
u32_t sys_now(void);
#endif
```

根据上述源码，ethernetif.h 文件仅声明了三个函数，这些函数将在其他文件中被引用。

lwip_comm.c 和 lwip_comm.h 文件是连接 lwIP 源码与以太网驱动库的关键桥梁。这两个文件非常重要，由正点原子提供。为了方便起见，可以直接复制正点原子已编写好的 lwip_comm.c/h 文件，并将其放入 arch 文件夹中，如下图所示。

lwIP例程1 lwIP裸机移植 > Middlewares > lwip > arch	
名称	类型
 cc.h	C++ Header file
 ethernetif.c	C 文件
 ethernetif.h	C++ Header file
 lwip_comm.c	C 文件
 lwip_comm.h	C++ Header file
 lwipopts.h	C++ Header file

图 2.3.2.7 arch 文件夹添加 lwip_comm.c/h 文件

至此，我们在工程 Middlewares/lwip/arch 分组添加 arch 文件夹下的全部.c 文件如下图所示。

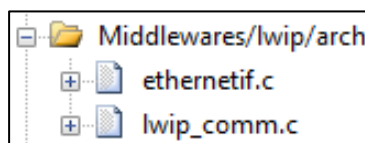


图 2.3.2.8 Middlewares/lwip/arch 分组添加的文件

(5) 添加 lwIP 头文件路径，如下图所示。

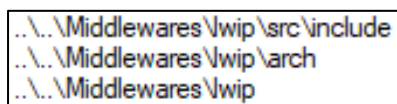


图 2.3.2.9 添加文件路径

注意：如果用户使用 lwip/src/apps 路径下的文件，则必须添加..\..\Middlewares\lwip\src\include\lwip\apps 头文件路径。

(6) 修改工程目标名称

本教程是以标准例程-HAL 库版本的内存管理实验工程为基础工程，内存管理实验工程的工程名为“MALLOC”，为了规范工程，笔者建议将工程的目标名修改为“lwIP”或根据笔者的实际场景进行修改，修改如下图所示。

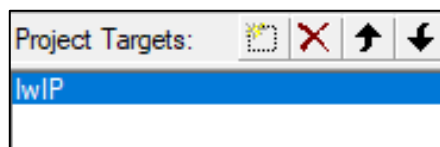


图 2.3.2.10 修改工程目标名称

2.3.3 arch 文件夹下的文件修改

在移植 lwIP 源码后，虽然完成了添加多个分组和设置源码文件路径的工作，但在编译工程时仍会出现许多与 arch 文件夹下的文件相关的错误。为了解决这些问题，我们需要对相关文件进行修改，具体修改方法将在后续部分详细介绍。

(1) 修改 lwipopts.h 文件

lwipopts.h 文件是 lwIP 的配置文件，用户可以通过修改该文件来设置 lwIP 的各种参数和功能。如果用户需要启用 DHCP 功能，可以将 DHCP 宏定义设置为 1。除了 DHCP 外，其他宏定义也可以根据需要进行设置。lwipopts.h 文件的源码如下：

```
#ifndef __LWIPOPTS_H__
#define __LWIPOPTS_H__

/**
SYS_LIGHTWEIGHT_PROT==1:如果您确实需要任务间保护
*/
#define SYS_LIGHTWEIGHT_PROT            0

/* NO_SYS 表示无操作系统模拟层，无操作系统为 1，有操作系统设置为 0
注意这个参数设置会编译不同 */
#define NO_SYS                          1

/**
* NO_SYS_NO_TIMERS==1: Drop support for sys_timeout when NO_SYS==1
* Mainly for compatibility to old versions.
*/
#define NO_SYS_NO_TIMERS                0
```

```

/* ----- 内存选项 ----- */
/* 内存对齐, 按照 4 字节对齐 */
#define MEM_ALIGNMENT                4

/* 堆内存的大小, 如果需要更大的堆内存, 那么设置高一点 */
#define MEM_SIZE                      (30*1024)

/* MEMP_NUM_PBUF: 设置内存池的数量 */
#define MEMP_NUM_PBUF                25
/* MEMP_NUM_UDP_PCB: UDP 协议控制块的数量. */
#define MEMP_NUM_UDP_PCB             4
/* MEMP_NUM_TCP_PCB: TCP 的数量. */
#define MEMP_NUM_TCP_PCB             4
/* MEMP_NUM_TCP_PCB_LISTEN: 监听 TCP 的数量. */
#define MEMP_NUM_TCP_PCB_LISTEN      2
/* MEMP_NUM_TCP_SEG: 同时排队的 TCP 的数量段. */
#define MEMP_NUM_TCP_SEG             150
/* MEMP_NUM_SYS_TIMEOUT: 超时模拟活动的数量. */
#define MEMP_NUM_SYS_TIMEOUT         6

/* ----- Pbuf 选项 ----- */
/* PBUF_POOL 内存池中每个内存块大小 */
#define PBUF_POOL_SIZE               20
/* PBUF_POOL_BUFSIZE: pbuf 池中每个 pbuf 的大小. */
#define PBUF_POOL_BUFSIZE
LWIP_MEM_ALIGN_SIZE(TCP_MSS+40+PBUF_LINK_ENCAPSULATION_HLEN+PBUF_LINK_HLEN)

/* ----- TCP 选项 ----- */
#define LWIP_TCP                      1
#define TCP_TTL                      255

/* 控制 TCP 是否应该对到达的段进行排队
   秩序。如果你的设备内存不足, 定义为 0. */
#define TCP_QUEUE_OOSEQ              0

/* TCP 最大段大小 */
#define TCP_MSS                      (1500 - 40) /* TCP_MSS = (Ethernet MTU -
IP header size - TCP header size) */

/* TCP 发送者缓冲区空间(字节). */
#define TCP_SND_BUF                  (11*TCP_MSS)

```



```

/*  TCP_SND_QUEUELEN: TCP 发送缓冲区空间。这必须是至少
    需要 (2 * TCP_SND_BUF/TCP_MSS) 才能正常工作 */

#define TCP_SND_QUEUELEN                (8* TCP_SND_BUF/TCP_MSS)

/* TCP 接收窗口 */
#define TCP_WND                          (20*TCP_MSS)

/* ----- ICMP 选项 ----- */
#define LWIP_ICMP                        1

/* ----- DHCP 选项 ----- */
/* 如果您希望 DHCP 配置为, 请将 LWIP_DHCP 定义为 1 */
#define LWIP_DHCP                        1

/* ----- UDP 选项 ----- */
#define LWIP_UDP                          1
#define UDP_TTL                          255

/* ----- Statistics 选项 ----- */
#define LWIP_STATS                        0
#define LWIP_PROVIDE_ERRNO               1

/* ----- 链接回调选项 ----- */
/* WIP_NETIF_LINK_CALLBACK==1:支持来自接口的回调函数
    每当链接改变 (例如, 向下链接)
    */
#define LWIP_NETIF_LINK_CALLBACK         1
/*
    -----
    ----- Checksum options -----
    -----
    */
#define CHECKSUM_BY_HARDWARE

#ifndef CHECKSUM_BY_HARDWARE
    #define CHECKSUM_GEN_IP               0

```

```
#define CHECKSUM_GEN_UDP      0
#define CHECKSUM_GEN_TCP      0
#define CHECKSUM_CHECK_IP     0
#define CHECKSUM_CHECK_UDP    0
#define CHECKSUM_CHECK_TCP    0
#define CHECKSUM_GEN_ICMP     0
#else
#define CHECKSUM_GEN_IP       1
#define CHECKSUM_GEN_UDP      1
#define CHECKSUM_GEN_TCP      1
#define CHECKSUM_CHECK_IP     1
#define CHECKSUM_CHECK_UDP    1
#define CHECKSUM_CHECK_TCP    1
#define CHECKSUM_GEN_ICMP     1
#endif

/*
-----
----- Sequential layer options -----
-----
*/
/**
 * LWIP_NETCONN==1: 启用 Netconn API (需要使用 api_lib.c)
 */
#define LWIP_NETCONN          0

/*
-----
----- Socket options -----
-----
*/
/**
 * LWIP_SOCKET==1: 启用 Socket API (要求使用 Socket .c)
 */
#define LWIP_SOCKET           0

#endif /* __LWIPOPTS_H__ */
```

该文件非常重要，它是 lwIP 内核裁剪和配置文件。

(2) 修改 cc.h 文件

该文件主要配置 lwIP 数据类型，cc.h 文件源码如下：

```
#ifndef __CC_H__
```

```
#define __CC_H__

#include <stdlib.h>
#include <stdio.h>

typedef int sys_prot_t;

#if defined (__GNUC__) & !defined (__CC_ARM)

#define LWIP_TIMEVAL_PRIVATE 0
#include <sys/time.h>

#endif

/* define compiler specific symbols */
#if defined (__ICCARM__)

#define PACK_STRUCT_BEGIN
#define PACK_STRUCT_STRUCT
#define PACK_STRUCT_END
#define PACK_STRUCT_FIELD(x) x
#define PACK_STRUCT_USE_INCLUDES

#elif defined (__GNUC__)

#define PACK_STRUCT_BEGIN
#define PACK_STRUCT_STRUCT __attribute__ ((__packed__))
#define PACK_STRUCT_END
#define PACK_STRUCT_FIELD(x) x

#elif defined (__CC_ARM)

#define PACK_STRUCT_BEGIN __packed
#define PACK_STRUCT_STRUCT
#define PACK_STRUCT_END
#define PACK_STRUCT_FIELD(x) x
#elif defined (__TASKING__)

#define PACK_STRUCT_BEGIN
#define PACK_STRUCT_STRUCT
#define PACK_STRUCT_END
#define PACK_STRUCT_FIELD(x) x
#endif
```

```
#define LWIP_PLATFORM_ASSERT(x) do
{printf("Assertion \"%s\" failed at line %d in %s\n", \
x, __LINE__, __FILE__); } while(0)
/* Define random number generator function */
#define LWIP_RAND() ((u32_t)rand())

#endif /* __CC_H__ */
```

该文件没有特别需要讲解的内容。作者在原文件中删除了两处地方：“#include "cpu.h"”和“#define LWIP_PROVIDE_ERRNO”，这是为了避免重复定义和不必要的代码。具体来说，由于 F4 系列芯片没有 cpu.h 文件，所以在该文件中包含这个头文件是不必要的。另外，LWIP_PROVIDE_ERRNO 宏定义已经在 lwipopts.h 文件中定义过了，为了避免重复定义，也不需要再在该文件中再次定义这个宏。因此，这两处被删除的内容是多余的，删除它们可以使代码更加简洁和易于维护。

(3) ethernetif.c 文件

该文件为网卡驱动文件，用来描述硬件网卡的收发函数及相关信息。该文件定义的函数如下表所示：

函数	描述
low_level_init()	底层初始化
low_level_output()	数据包发送函数
low_level_input()	获取数据报函数
ethernetif_input()	处理数据包函数
ethernetif_init()	网卡初始化
sys_now()	以毫秒为单位返回当前时间

表 2.3.3.1 ethernetif.c 文件函数

从上表可知，ethernetif.c 一共定义了六个函数，下面笔者分别地讲解这些函数的作用。

ethernetif_init 函数

该函数主要对网卡描述符初始化，该函数如下所示：

```
err_t
ethernetif_init(struct netif *netif)
{
    struct ethernetif *ethernetif;

    LWIP_ASSERT("netif != NULL", (netif != NULL));

    ethernetif = mem_malloc(sizeof(struct ethernetif));

    if (ethernetif == NULL)
    {
        LWIP_DEBUGF(NETIF_DEBUG, ("ethernetif_init: out of memory\n"));
        return ERR_MEM;
    }

#if LWIP_NETIF_HOSTNAME
    /* Initialize interface hostname */
```

```

netif->hostname = "lwip";
#endif /* LWIP_NETIF_HOSTNAME */

/*
 * Initialize the snmp variables and counters inside the struct netif.
 * The last argument should be replaced with your link speed, in units
 * of bits per second.
 */
MIB2_INIT_NETIF(netif, snmp_ifType_ethernet_csmacd,
                LINK_SPEED_OF_YOUR_NETIF_IN_BPS);

netif->state = ethernetif;
netif->name[0] = IFNAME0;
netif->name[1] = IFNAME1;
/* We directly use etharp_output() here to save a function call.
 * You can instead declare your own function and call etharp_output()
 * from it if you have to do some checks before sending (e.g. if link
 * is available...) */
#if LWIP_IPV4
    netif->output = etharp_output;
#endif /* LWIP_IPV4 */
#if LWIP_IPV6
    netif->output_ip6 = ethip6_output;
#endif /* LWIP_IPV6 */
netif->linkoutput = low_level_output;

ethernetif->ethaddr = (struct eth_addr *) & (netif->hwaddr[0]);

/* initialize the hardware */
low_level_init(netif);

return ERR_OK;
}

```

netif 结构体，它是对网卡进行抽象，例如描述网卡的状态，收发函数和网络参数等。

low_level_init 函数

该函数用来初始化 MAC 地址和初始化 ETH 的 DMA 描述符，该函数如下所示：

```

static void
low_level_init(struct netif *netif)
{
    netif->hwaddr_len = ETHARP_HWADDR_LEN; /*设置 MAC 地址长度, 为 6 个字节*/
    /*初始化 MAC 地址, 设置什么地址由用户自己设置, 但是不能与网络中其他设备 MAC 地址重复*/
    netif->hwaddr[0] = g_lwipdev.mac[0];
    netif->hwaddr[1] = g_lwipdev.mac[1];
}

```

```

netif->hwaddr[2]= g_lwipdev.mac[2];
netif->hwaddr[3]= g_lwipdev.mac[3];
netif->hwaddr[4]= g_lwipdev.mac[4];
netif->hwaddr[5]= g_lwipdev.mac[5];

netif->mtu=1500; /* 最大允许传输单元,允许该网卡广播和 ARP 功能 */

/* 网卡状态信息标志位,是很重要的控制字段,它包括网卡功能使能、广播*/
/* 使能、 ARP 使能等重要控制位*//*广播 ARP 协议 链接检测*/
netif->flags = NETIF_FLAG_BROADCAST|NETIF_FLAG_ETHARP|NETIF_FLAG_LINK_UP;

/*初始化发送描述符*/
HAL_ETH_DMATxDescListInit(&g_eth_handler,g_eth_dma_tx_dscr_tab,
                           g_eth_tx_buf,ETH_TXBUFNB);

/*初始化接收描述符*/
HAL_ETH_DMARxDescListInit(&g_eth_handler,g_eth_dma_rx_dscr_tab,
                           g_eth_rx_buf,ETH_RXBUFNB);

HAL_ETH_Start(&g_eth_handler); /*开启 MAC 和 DMA*/
}

```

此函数负责根据 g_lwipdev.mac 数组来设置 MAC 地址,该数组的具体说明将在 lwip_com m.c 文件中提供。此外,该函数还设置了 lwIP 的最大允许传输单元为 1500 字节。最后,通过调用函数 HAL_ETH_DMATxDescListInit 和 HAL_ETH_DMARxDescListInit,以太网 DMA 描述符被初始化了。

low_level_output 函数

```

static err_t
low_level_output(struct netif *netif, struct pbuf *p)
{
    err_t errval;
    struct pbuf *q;

    uint8_t *buffer = (uint8_t *) (g_eth_handler.TxDesc->Buffer1Addr);
    __IO ETH_DMADescTypeDef *DmaTxDesc;
    uint32_t framelength = 0;
    uint32_t bufferoffset = 0;
    uint32_t byteslefttocopy = 0;
    uint32_t payloadoffset = 0;

    DmaTxDesc = g_eth_handler.TxDesc;
    bufferoffset = 0;

#ifdef ETH_PAD_SIZE
    pbuf_remove_header(p, ETH_PAD_SIZE); /* drop the padding word */
#endif
}

```

```

/* 从 pbuf 中拷贝要发送的数据 */
for(q = p;q != NULL;q = q->next)
{
    /* 判断此发送描述符是否有效，即判断此发送描述符是否归以太网 DMA 所有 */
    if((DmaTxDesc->Status&ETH_DMATXDESC_OWN) != (uint32_t)RESET)
    {
        errval=ERR_USE;
        goto error;          /* 发送描述符无效，不可用 */
    }

    byteslefttocopy = q->len;    /* 要发送的数据长度 */
    payloadoffset = 0;

    /* 将 pbuf 中要发送的数据写入到以太网发送描述符中，
       有时候我们要发送的数据可能大于一个以太网
       描述符的 Tx Buffer，因此我们需要分多次将数据拷贝到多个发送描述符中 */
    while((byteslefttocopy+bufferoffset)>ETH_TX_BUF_SIZE )
    {
        /* 将数据拷贝到以太网发送描述符的 Tx Buffer 中 */
        memcpy((uint8_t*)((uint8_t*)buffer +bufferoffset),
               (uint8_t*)((uint8_t*)q->payload+payloadoffset),
               (ETH_TX_BUF_SIZE-bufferoffset));

        /* DmaTxDesc 指向下一个发送描述符 */
        DmaTxDesc=(ETH_DMADescTypeDef *) (DmaTxDesc->Buffer2NextDescAddr);
        /* 检查新的发送描述符是否有效 */
        if((DmaTxDesc->Status&ETH_DMATXDESC_OWN) != (uint32_t)RESET)
        {
            errval = ERR_USE;
            goto error;      /* 发送描述符无效，不可用 */
        }

        /* 更新 buffer 地址，指向新的发送描述符的 Tx Buffer */
        buffer = (uint8_t *) (DmaTxDesc->Buffer1Addr);
        byteslefttocopy = byteslefttocopy-(ETH_TX_BUF_SIZE-bufferoffset);
        payloadoffset = payloadoffset+(ETH_TX_BUF_SIZE-bufferoffset);
        framelength = framelength+(ETH_TX_BUF_SIZE-bufferoffset);
        bufferoffset = 0;
    }

    /* 拷贝剩余的数据 */
    memcpy((uint8_t*)((uint8_t*)buffer + bufferoffset),
           (uint8_t*)((uint8_t*)q->payload+payloadoffset),
           byteslefttocopy );
    bufferoffset = bufferoffset+byteslefttocopy;
}

```



```

        framelength = framelength+byteslefttocopy;
    }

    /* 当所有要发送的数据都放进发送描述符的 Tx Buffer 以后就可发送此帧了 */
    HAL_ETH_TransmitFrame(&g_eth_handler, framelength);
    errval = ERR_OK;
error:
    /* 发送缓冲区发生下溢，一旦发送缓冲区发生下溢 TxDMA 会进入挂起状态 */
    if((g_eth_handler.Instance->DMASR&ETH_DMASR_TUS) != (uint32_t)RESET)
    {
        /* 清除下溢标志 */
        g_eth_handler.Instance->DMASR = ETH_DMASR_TUS;
        /* 当发送帧中出现下溢错误的时候 TxDMA 会挂起，这时候需要向 DMATPDR 寄存器 */
        /* 随便写入一个值来将其唤醒，此处我们写 0 */
        g_eth_handler.Instance->DMATPDR=0;
    }

#ifdef ETH_PAD_SIZE
    pbuf_add_header(p, ETH_PAD_SIZE); /* reclaim the padding word */
#endif

    return errval;
}

```

此函数的主要功能是将数据包拷贝到以太网发送描述符管理的 Buffer 中。一旦所有要发送的数据都已放入发送描述符管理的 Buffer，就可以调用函数 HAL_ETH_TransmitFrame 来发送该帧。这个过程非常直接和简单，确保数据能够顺利发送出去。

ethernetif_input 函数

该函数用于获取以太网数据包。在示例中，它被放置在 ETH 中断处理程序中。在操作系统移植 lwIP 时，该函数可以被视为一个任务函数来处理。具体实现方式如下所示：

```

void
ethernetif_input(struct netif *netif)
{
    struct pbuf *p;

    /* move received packet into a new pbuf */
    p = low_level_input(netif);
    /* if no packet could be read, silently ignore this */
    if (p != NULL)
    {
        if (netif->input(p, netif) != ERR_OK)
        {
            LWIP_DEBUGF(NETIF_DEBUG, ("ethernetif_input: IP input error\n"));
            pbuf_free(p);
        }
    }
}

```

```

        p = NULL;
    }
}
}

```

此函数非常简单，调用函数 `low_level_input` 获取数据包并调用 `netif->input` 函数往上传输。

low_level_input 函数

该函数用来获取以太网帧的数据并保存在 `pbuf` 当中，`pbuf` 的知识点笔者在第七章中讲解，该函数如下所示：

```

static struct pbuf *
low_level_input(struct netif *netif)
{
    struct pbuf *p, *q;
    u16_t len;
    uint8_t *buffer;
    __IO ETH_DMADescTypeDef *dmarxdesc;
    uint32_t bufferoffset = 0;
    uint32_t payloadoffset = 0;
    uint32_t byteslefttocopy = 0;
    uint32_t i = 0;

    if(HAL_ETH_GetReceivedFrame(&g_eth_handler) != HAL_OK) /*判断是否接收到数据*/
        return NULL;

    len = g_eth_handler.RxFrameInfos.length; /*获取接收到的以太网帧长度*/

#ifdef ETH_PAD_SIZE
    len += ETH_PAD_SIZE; /* allow room for Ethernet padding */
#endif

    /*获取接收到的以太网帧的数据 buffer*/
    buffer = (uint8_t *)g_eth_handler.RxFrameInfos.buffer;

    p = pbuf_alloc(PBUF_RAW, len, PBUF_POOL); /* 申请 pbuf */

    if(p != NULL) /* pbuf 申请成功 */
    {
        /* 获取接收描述符链表中的第一个描述符 */
        dmarxdesc = g_eth_handler.RxFrameInfos.FSRxDesc;
        bufferoffset = 0;

        for(q = p; q != NULL; q = q->next)
        {
            byteslefttocopy = q->len;

```

```

        payloadoffset = 0;

        /* 将接收描述符中 Rx Buffer 的数据拷贝到 pbuf 中 */
        while((byteslefttocopy + bufferoffset) > ETH_RX_BUF_SIZE )
        {
            /* 将数据拷贝到 pbuf 中 */
            memcpy((uint8_t*)((uint8_t*)q->payload+payloadoffset),
                    (uint8_t*)((uint8_t*)buffer + bufferoffset),
                    (ETH_RX_BUF_SIZE - bufferoffset));

            /* dmarxdesc 向下一个接收描述符 */
            dmarxdesc = (ETH_DMADescTypeDef *)
                (dmarxdesc->Buffer2NextDescAddr);

            /* 更新 buffer 地址, 指向新的接收描述符的 Rx Buffer */
            buffer = (uint8_t *) (dmarxdesc->Buffer1Addr);

            byteslefttocopy = byteslefttocopy -
                (ETH_RX_BUF_SIZE - bufferoffset);
            payloadoffset = payloadoffset +
                (ETH_RX_BUF_SIZE - bufferoffset);
            bufferoffset = 0;
        }

        /* 拷贝剩余的数据 */
        memcpy( (uint8_t*)((uint8_t*)q->payload+payloadoffset),
                (uint8_t*)((uint8_t*)buffer + bufferoffset),
                byteslefttocopy);
        bufferoffset = bufferoffset + byteslefttocopy;
    }
}

else
{
    /* drop packet(); 丢包函数自行编写 */
    LINK_STATS_INC(link.memerr);
    LINK_STATS_INC(link.drop);
    MIB2_STATS_NETIF_INC(netif, ifindiscards);
}

/* 释放 DMA 描述符 */
dmarxdesc = g_eth_handler.RxFrameInfos.FSRxDesc;

for(i = 0; i < g_eth_handler.RxFrameInfos.SegCount; i++)
{
    dmarxdesc->Status |= ETH_DMARXDESC_OWN; /* 标记描述符归 DMA 所有 */
    dmarxdesc = (ETH_DMADescTypeDef *) (dmarxdesc->Buffer2NextDescAddr);
}

```

```

}

g_eth_handler.RxFrameInfos.SegCount = 0;          /* 清除段计数器 */
/* 接收缓冲区不可用 */
if((g_eth_handler.Instance->DMASR&ETH_DMASR_RBUS) != (uint32_t)RESET)
{
    /* 清除接收缓冲区不可用标志 */
    g_eth_handler.Instance->DMASR = ETH_DMASR_RBUS;
    /* 当接收缓冲区不可用的时候 RxDMA 会进去挂起状态，
       通过向 DMARPDR 写入任意一个值来唤醒 Rx DMA */
    g_eth_handler.Instance->DMARPDR = 0;
}

return p;
}

```

此函数用来获取以太网发送描述符管理的 Buffer 中的数据，将数据拷贝到 pbuf 当中，最后返回 pbuf 数据包。

sys_now 函数

该函数主要提供 lwIP 时基，该函数如下所示：

```

u32_t sys_now(void)
{
    return HAL_GetTick();
}

```

此函数调用 HAL_GetTick 获取 HAL 的节拍来提供 lwIP 时基。

(4) ethernetif.h 文件

该文件主要声明 ethernetif_init、ethernetif_input 和 sys_now 函数给外部文件使用，该文件源码如下所示：

```

#ifndef __ETHERNETIF_H__
#define __ETHERNETIF_H__
#include "lwip/err.h"
#include "lwip/netif.h"

err_t ethernetif_init(struct netif *netif); /* 网卡初始化函数 */
void ethernetif_input(struct netif *netif); /* 数据包输入函数 */
u32_t sys_now(void);

#endif

```

(5) lwip_comm.h 文件

该文件主要声明了一个名为 __lwip_dev 的结构体，该结构体主要用于保存网络相关的信息，包括 MAC 地址、本地 IP 地址、远程 IP 地址等。以下是这个文件的大致内容：

```

#ifndef __LWIP_COMM_H
#define __LWIP_COMM_H

```

```
#include "../BSP/ETHERNET/ethernet.h"

#define LWIP_MAX_DHCP_TRIES      4          /* DHCP 服务器最大重试次数 */

/*lwip 控制结构体*/
typedef struct
{
    uint8_t mac[6];                /* MAC 地址 */
    uint8_t remoteip[4];           /* 远端主机 IP 地址 */
    uint8_t ip[4];                 /* 本机 IP 地址 */
    uint8_t netmask[4];            /* 子网掩码 */
    uint8_t gateway[4];            /* 默认网关的 IP 地址 */
    uint8_t dhcpstatus;            /* dhcp 状态 */
                                    /* 0, 未获取 DHCP 地址; */
                                    /* 1, 进入 DHCP 获取状态 */
                                    /* 2, 成功获取 DHCP 地址 */
                                    /* 0xFF, 获取失败 */
}__lwip_dev;

extern __lwip_dev lwipdev;        /* lwip 控制结构体 */

void lwip_packet_handler(void);    /* 当接收到数据后调用 */
void lwip_periodic_handle(void);   /* lwip_periodic_handle */
void lwip_comm_default_ip_set(__lwip_dev *lwipx); /* lwip 默认 IP 设置 */
/* 内存堆 ram_heap 和内存池 memp_memory 的内存分配 */
uint8_t lwip_comm_mem_malloc(void);
void lwip_comm_mem_free(void);     /* lwip 中 mem 和 memp 内存释放 */
/* lwIP 初始化(lwIP 启动的时候使用) */
uint8_t lwip_comm_init(void);
void lwip_dhcp_process_handle(void); /* DHCP 处理任务 */

#endif
```

该文件没什么好讲解的，主要声明一些函数。

(6) lwip_comm.c 文件

该文件是将 lwIP 源码与前面的以太网驱动库结合起来的桥梁。它主要包含了五个函数，下面将分别介绍这些函数的作用：

lwip_comm_init 函数

该函数主要对以太网的 IO 初始化以及添加在网卡列表中添加一个网口，该函数如下所示：

```
uint8_t lwip_comm_init(void)
{
    uint8_t retry = 0;
```

```
/* 调用 netif_add() 函数时的返回值,用于判断网络初始化是否成功 */
struct netif *netif_init_flag;

ip_addr_t ipaddr; /* ip 地址 */
ip_addr_t netmask; /* 子网掩码 */
ip_addr_t gw; /* 默认网关 */

if (ethernet_mem_malloc())return 1; /* 内存申请失败*/

lwip_comm_default_ip_set(&lwipdev); /* 设置默认 IP 等信息 */

while (ethernet_init()) /* 初始化以太网芯片,如果失败的话就重试 5 次 */
{
    retry++;

    if (retry > 5)
    {
        retry = 0; /* 以太网芯片初始化失败 */
        return 3;
    }
}

lwip_init(); /* 初始化 LWIP 内核 */

#if LWIP_DHCP /* 使用动态 IP */
    ipaddr.addr = 0;
    netmask.addr = 0;
    gw.addr = 0;
#else /* 使用静态 IP */
    IP4_ADDR(&ipaddr, lwipdev.ip[0], lwipdev.ip[1],
             lwipdev.ip[2], lwipdev.ip[3]);
    IP4_ADDR(&netmask, lwipdev.netmask[0], lwipdev.netmask[1],
             lwipdev.netmask[2], lwipdev.netmask[3]);
    IP4_ADDR(&gw, lwipdev.gateway[0], lwipdev.gateway[1],
             lwipdev.gateway[2], lwipdev.gateway[3]);
    /* 省略代码..... */
#endif /* 向网卡列表中添加一个网口 */

netif_init_flag = netif_add(&lwip_netif, (const ip_addr_t *)&ipaddr,
                           (const ip_addr_t *)&netmask,
                           (const ip_addr_t *)&gw, NULL,
                           &ethernetif_init, &ethernet_input);

if (netif_init_flag == NULL)
```

```

{
    return 4;                                /* 网卡添加失败 */
}
else /* 网口添加成功后,设置 netif 为默认值,并且打开 netif 网口 */
{
    netif_set_default(&lwip_netif);          /* 设置 netif 为默认网口 */

    if (netif_is_link_up(&lwip_netif))
    {
        netif_set_up(&lwip_netif);          /* 打开 netif 网口 */
    }
    else
    {
        netif_set_down(&lwip_netif);
    }
}

#if LWIP_DHCP                                /* 如果使用 DHCP 的话 */
    lwipdev.dhcpstatus = 0;                  /* DHCP 标记为 0 */
    dhcp_start(&lwip_netif);                /* 开启 DHCP 服务 */
#endif
return 0;                                    /* 操作 OK. */
}

```

该函数首先调用 `ethernet_mem_malloc` 函数来为 DMA 描述符分配内存,这个函数之前已经介绍过,这里不再重复。接着,它调用了 `lwip_comm_default_ip_set` 函数来设置网络参数。然后,通过调用 `ethernet_init` 函数来初始化以太网 IO 并初始化 lwIP 内核。最后,使用 `netif_add` 函数向网卡列表添加一个网络接口。

需要注意的是, `netif_add` 函数的参数比较特殊。它的第二到第四个参数是本地 IP 地址、子网掩码和网关。而第五和第六个参数需要用户传入 `ethernetif_init` 和 `ethernet_input` 函数的地址。通过这些参数的设置,可以配置网络接口的各种属性,以便于进行正常的网络通信。

lwip_comm_default_ip_set 函数

此函数非常简单,主要设置网络的信息,该函数如下所示:

```

void lwip_comm_default_ip_set(__lwip_dev *lwipx)
{
    /* 默认远端 IP 为:192.168.1.134 */
    lwipx->remoteip[0] = 192;
    lwipx->remoteip[1] = 168;
    lwipx->remoteip[2] = 1;
    lwipx->remoteip[3] = 134;

    /* MAC 地址设置*/
    lwipx->mac[0] = 0xB8;
    lwipx->mac[1] = 0xAE;
}

```

```

lwipx->mac[2] = 0x1D;
lwipx->mac[3] = 0x00;
lwipx->mac[4] = 0x07;
lwipx->mac[5] = 0x00;

/* 默认本地 IP 为:192.168.1.30 */
lwipx->ip[0] = 192;
lwipx->ip[1] = 168;
lwipx->ip[2] = 1;
lwipx->ip[3] = 30;

/* 默认子网掩码:255.255.255.0 */
lwipx->netmask[0] = 255;
lwipx->netmask[1] = 255;
lwipx->netmask[2] = 255;
lwipx->netmask[3] = 0;

/* 默认网关:192.168.1.1 */
lwipx->gateway[0] = 192;
lwipx->gateway[1] = 168;
lwipx->gateway[2] = 1;
lwipx->gateway[3] = 1;
lwipx->dhcpstatus = 0; /* 没有 DHCP */
}

```

注意：这里是探索者与 DMF407 开发板第三处区别即 MAC 地址的数值。

lwip_pkt_handle 函数

这个函数间接调用 ethernetif_input 函数，该函数在裸机移植时，一般放在 ETH 中断当中。

lwip_periodic_handle 函数

该函数与 DHCP（动态主机配置协议）相关。根据工程是否启用 DHCP，系统的行为会有所不同。如果工程不启用 DHCP，系统只会调用 sys_check_timeouts 来检测超时。这意味着在没有 DHCP 的情况下，系统只会检查是否有任何超时事件发生，并采取相应的处理措施。如果工程启用了 DHCP，系统除了调用 sys_check_timeouts 外，还会根据 DHCP 响应时间来获取动态 IP 地址等网络数据。这意味着在 DHCP 启用的情况下，系统不仅会检查超时，还会利用 DHCP 协议自动获取网络配置信息，如 IP 地址、子网掩码等，以便进行网络通信。

这个函数的具体实现方式如下所示：

```

void lwip_periodic_handle()
{
    sys_check_timeouts();
    #if LWIP_DHCP          /* 如果使用 DHCP */

    /* 每 500ms 调用一次 dhcp_fine_tmr() */
    /* DHCP_FINE_TIMER_MSECS(500) 单位 */
    if (sys_now() - DHCPfineTimer >= DHCP_FINE_TIMER_MSECS)
    {

```



```

    DHCPfineTimer = sys_now();
    dhcp_fine_tmr();

    if ((lwipdev.dhcpstatus != 2) && (lwipdev.dhcpstatus != 0xFF))
    {
        lwip_dhcp_process_handle(); /* DHCP 处理 */
    }
}

/* 每 60s 执行一次 DHCP 粗糙处理 */
if (sys_now() - DHCPcoarseTimer >= DHCP_COARSE_TIMER_MSECS)
{
    DHCPcoarseTimer = sys_now();
    dhcp_coarse_tmr();
}
#endif
}

```

lwip_dhcp_process_handle 函数

该函数也与 DHCP（动态主机配置协议）相关。根据工程是否启用 DHCP，该函数的行为会有所不同。如果工程不启用 DHCP，该函数将默认使用 lwip_comm_default_ip_set 函数中设置的网络信息。这意味着在没有 DHCP 的情况下，系统将使用预先设置的网络参数进行通信。如果工程启用了 DHCP，该函数将通过 DHCP 协议与服务器交互，以获取动态的网络信息。这些信息包括 IP 地址、子网掩码、网关等，以便进行网络通信。

这个函数的具体实现方式如下所示：

```

/* 如果使能 DHCP */
#ifdef LWIP_DHCP

struct dhcp gdhcp;

/**
 * @breif      DHCP 处理任务
 * @param      无
 * @retval     无
 */
void lwip_dhcp_process_handle(void)
{
    uint32_t ip = 0, netmask = 0, gw = 0;

    switch (lwipdev.dhcpstatus)
    {
        case 0: /* 开启 DHCP */
            dhcp_start(&lwip_netif);
            lwipdev.dhcpstatus = 1; /* 等待通过 DHCP 获取到的地址 */

```

```
printf("正在查找 DHCP 服务器,请稍等.....\r\n");
break;

case 1:      /* 等待获取到 IP 地址 */
{
    ip = lwip_netif.ip_addr.addr;      /* 读取新 IP 地址 */
    netmask = lwip_netif.netmask.addr; /* 读取子网掩码 */
    gw = lwip_netif.gw.addr;           /* 读取默认网关 */

    if (ip != 0)      /*正确获取到 IP 地址的时候*/
    {
        lwipdev.dhcpstatus = 2;          /* DHCP 成功 */

        /* 解析出通过 DHCP 获取到的 IP 地址 */
        lwipdev.ip[3] = (uint8_t)(ip >> 24);
        lwipdev.ip[2] = (uint8_t)(ip >> 16);
        lwipdev.ip[1] = (uint8_t)(ip >> 8);
        lwipdev.ip[0] = (uint8_t)(ip);

        /* 解析通过 DHCP 获取到的子网掩码地址 */
        lwipdev.netmask[3] = (uint8_t)(netmask >> 24);
        lwipdev.netmask[2] = (uint8_t)(netmask >> 16);
        lwipdev.netmask[1] = (uint8_t)(netmask >> 8);
        lwipdev.netmask[0] = (uint8_t)(netmask);

    }/* 通过 DHCP 服务获取 IP 地址失败,且超过最大尝试次数 */
    else if (gdhcp.tries > LWIP_MAX_DHCP_TRIES)
    {
        lwipdev.dhcpstatus = 0xFF; /* DHCP 超时失败. */
        /* 使用静态 IP 地址 */
        IP4_ADDR(&(lwip_netif.ip_addr), lwipdev.ip[0],
                                                         lwipdev.ip[1],
                                                         lwipdev.ip[2],
                                                         lwipdev.ip[3]);
        IP4_ADDR(&(lwip_netif.netmask), lwipdev.netmask[0],
                                                         lwipdev.netmask[1],
                                                         lwipdev.netmask[2],
                                                         lwipdev.netmask[3]);
        IP4_ADDR(&(lwip_netif.gw), lwipdev.gateway[0],
                                                         lwipdev.gateway[1],
                                                         lwipdev.gateway[2],
                                                         lwipdev.gateway[3]);
    }
}
```

```

    }

    }

    break;

    default :
        break;

    }

}

#endif

```

至此，我们已经详细介绍了 arch 文件夹下的文件内容。如果仍有疑问，强烈建议观看正点原子 lwIP 视频以获得更深入的理解。在完成这些步骤后，编译工程时应该不会再出现错误（除非程序空间不足，此时可以尝试减小 MEM1_MAX_SIZE 的值，在 malloc.h 文件中进行修改）。

接下来，我们将在 main 函数中编写一些测试代码。完成后，请在 PC 机上打开命令提示符（cmd），并尝试使用 ping 命令来测试开发板的 IP 地址。如果系统能够成功 ping 通，这表示 lwIP 移植已经成功完成。

(7) main 函数

```

int main(void)
{
    uint8_t t = 0;
    /* 一系列初始化 */
    while(lwip_comm_init() != 0)
    {
        lcd_show_string(6, 110, 200, 16, 16, "lwIP Init failed!!", BLUE);
        delay_ms(500);
        lcd_fill(6, 50, 200 + 30, 50 + 16, WHITE);
        lcd_show_string(6, 110, 200, 16, 16, "Retrying...      ", BLUE);
        delay_ms(500);
        LED1_TOGGLE();
    }

    while(!ethernet_read_phy(PHY_SR))          /* 检查 MCU 与 PHY 芯片是否通信成功 */
    {
        printf("MCU 与 PHY 芯片通信失败，请检查电路或者源码！！！！\r\n");
    }

    #if LWIP_DHCP
        /* 等待 DHCP 获取成功/超时溢出 */
        while((lwipdev.dhcpstatus != 2)&&(lwipdev.dhcpstatus != 0xFF))
        {
            lwip_periodic_handle();
        }
    #endif

    while(1)
    {

```

```
lwip_periodic_handle(); /* LWIP 轮询任务 */
delay_ms(2);
t++;
if(t >= 200)
{
    t = 0;
    LED0_TOGGLE();
}
}
```

该函数首先调用 `lwip_comm_init` 函数来初始化以太网 IO 和网卡驱动。这一步确保了以太网接口的正常运行和驱动程序的正确加载。接着，它调用 `lwip_periodic_handle` 函数来获取 DHCP 协议分配的动态 IP 地址。DHCP 是一种自动分配 IP 地址的协议，通过与 DHCP 服务器的交互，该函数能够获取到动态分配的 IP 地址等网络配置信息。最后，在 `while` 循环中不断调用 `lwip_periodic_handle` 函数。这一部分的目的是持续监控和处理网络状态的变化，确保动态 IP 地址的正确性和有效性。

完成上述步骤后，将编译后的工程下载到开发板上。为了验证动态 IP 地址是否正常工作，可以打开串口调试助手来查看动态 IP 地址。具体的显示结果如下图所示。

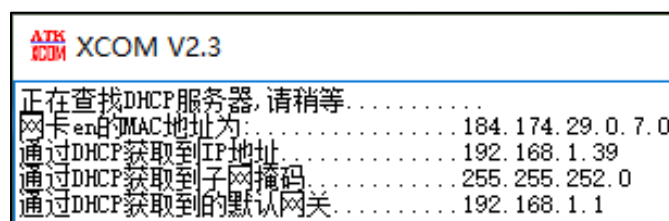
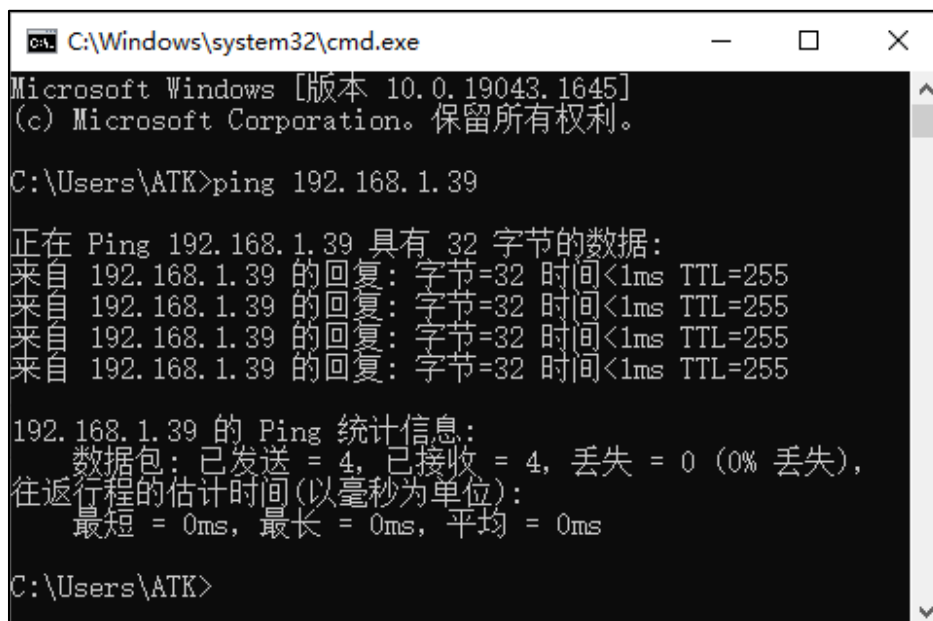


图 2.3.3.1 系统获取动态 IP 地址成功

在 PC 机上，按下“win+r”快捷键打开运行窗口，然后输入“cmd”进入命令行界面。在命令行界面中，输入“ping 192.168.1.39”命令来测试与开发板的通讯。如果命令行显示“无法访问目标主机”的错误信息，这表明系统无法与开发板建立通讯，说明我们移植的工程是失败的。如果命令行显示“字节=32 时间<1ms TTL=255”的结果，这表明系统能够与开发板正常通讯，证明该工程移植成功。

接下来，我们使用命令行工具来 ping 图 2.2.3.1 中显示的 IP 地址。如下图所示。



```

C:\Windows\system32\cmd.exe
Microsoft Windows [版本 10.0.19043.1645]
(c) Microsoft Corporation。保留所有权利。

C:\Users\ATK>ping 192.168.1.39

正在 Ping 192.168.1.39 具有 32 字节的数据:
来自 192.168.1.39 的回复: 字节=32 时间<1ms TTL=255
来自 192.168.1.39 的回复: 字节=32 时间<1ms TTL=255
来自 192.168.1.39 的回复: 字节=32 时间<1ms TTL=255
来自 192.168.1.39 的回复: 字节=32 时间<1ms TTL=255

192.168.1.39 的 Ping 统计信息:
    数据包: 已发送 = 4, 已接收 = 4, 丢失 = 0 (0% 丢失),
    往返行程的估计时间(以毫秒为单位):
        最短 = 0ms, 最长 = 0ms, 平均 = 0ms

C:\Users\ATK>
    
```

图 2.3.3.2 ping IP 地址成功

该实验的实验工程，请参考《lwIP 例程 1 lwIP 裸机移植》。

2.4 lwIP 移植流程（阿波罗、北极星）

2.4.1 添加网卡驱动程序

首先，我要声明的是，阿波罗和北极星开发板的 lwIP 移植流程是相同的。

接下来，打开正点原子提供的“lwIP 例程 1 lwIP 裸机移植”实验，并进入 Drivers\BSP 文件夹。在此文件夹中，你会找到一个 ETHERNET 文件夹。这个文件夹包含了四个文件：ethernet.c、ethernet.h、ethernet_chip.c 和 ethernet_chip.h。其中，ethernet.c 和 ethernet.h 文件主要定义了以太网驱动的初始化和 MAC 驱动程序。而 ethernet_chip.c 和 ethernet_chip.h 文件则主要定义了 PHY 的配置函数。在进行移植时，你需要将这四个文件拷贝到自己的 Drivers\BSP\ETHERNET 文件夹下。同时，你还需要将 ethernet.c 和 ethernet_chip.c 这两个文件添加到工程中的 Drivers/BSP 分组中。

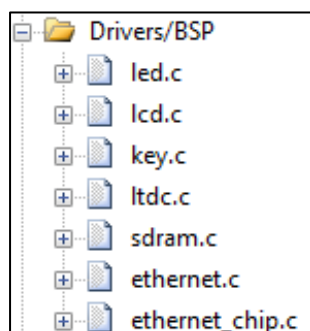


图 2.4.1.1 添加 ethernet.c 后的工程

为了正确管理阿波罗和北极星开发板的 PHY 芯片复位引脚，我们需要添加 PCF8574 和 IIC 驱动，因为这些引脚由 PCF8574 芯片管理，该芯片使用 IIC 协议驱动。这些驱动位于 Drivers/BSP 分组下，以确保开发板的正常工作。

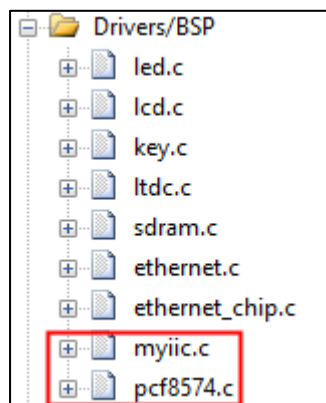


图 2.4.1.2 添加 PCF8574 以及 IIC 驱动

至此，网卡驱动添加完成，我们编译工程会出现很多错误，如下图所示。

```
#20: identifier "ETH_HandleTypeDef" is undefined
/* 以太网句柄 */
#20: identifier "ETH_DMADescTypeDef" is undefined
/* 以太网DMA接收描述符数据结构体指针 */
#20: identifier "ETH_DMADescTypeDef" is undefined
/* 以太网DMA发送描述符数据结构体指针 */
#20: identifier "ETH_DMADescTypeDef" is undefined
```

图 2.4.1.3 缺少以太网 hal 文件

上述的问题导致原因是工程缺少以太网驱动文件。为了解决这个问题，我们需要在工程的 Drivers/STM32F4/F7/H7xx_HAL_Driver 分组下添加一个以太网驱动文件，即 stm32f4/f4/h7xx_hal_eth.c 文件。添加完驱动文件后，我们需要打开 stm32f4/f4/h7xx_hal_conf.h 文件，并在该文件中启用 HAL_ETH_MODULE_ENABLED 以太网模块。这个配置项在该文件的最顶端定义。

2.4.2 添加 lwIP 源文件

把 lwip-2.1.3 文件夹下的 src 文件夹复制到“lwIP 例程 1 lwIP 裸机移植”工程 Middlewares\lwip 路径下，如下图所示。

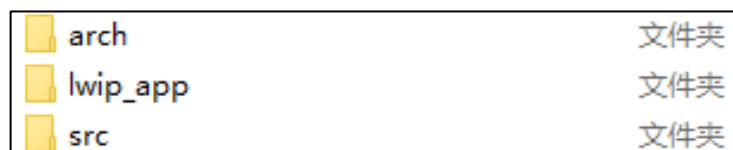


图 2.4.2.1 复制 lwIP 源码到工程中

打开“lwIP 例程 1 lwIP 裸机移植”工程添加 Middlewares/lwip/src/api、Middlewares/lwip/src/core、Middlewares/lwip/src/netif 和 Middlewares/lwip/arch 分组，下面我们分别地讲解这些分组需要添加哪些文件，如下所示：

(1) Middlewares/lwip/src/api 分组添加上图中的 src/api 路径下的全部.c 文件，如下图所示。

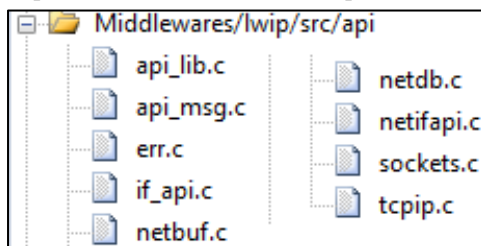


图 2.4.2.2 Middlewares/lwip/src/api 分组添加的文件

(2) Middlewares/lwip/src/core 分组添加图 2.4.2.1 中的 src/core 路径下除 ipv6 文件夹的全部.c 文件，如下图所示。

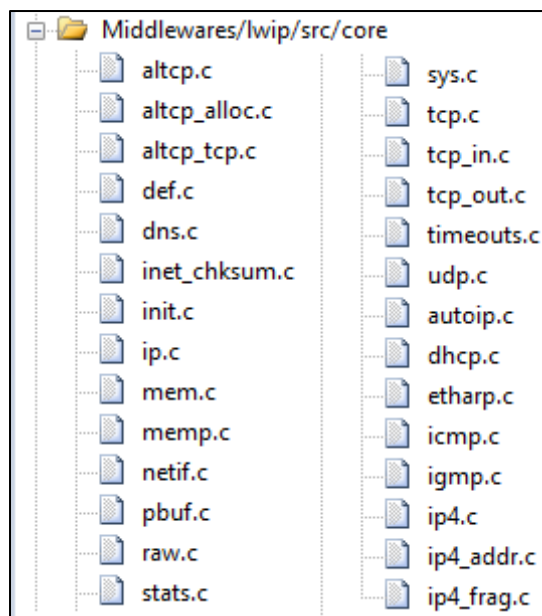


图 2.4.2.3 Middlewares/lwip/src/core 分组添加的文件

(3) Middlewares/lwip/src/netif 分组添加图 2.4.2.1 中的 src/netif 路径下的 ethernet.c 文件，如下图所示。

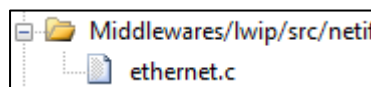


图 2.4.2.4 Middlewares/lwip/src/netif 分组添加的文件

(4) Middlewares/lwip/arch 添加图 2.4.2.1 中的 arch 文件夹下的全部.c 文件，前面我们没有在该文件夹下放置任何的文件，下面我们来讲解这个文件夹到底存放哪些文件。

arch 文件夹主要存放 lwipopts.h、cc.h、ethernetif.c/h 和 lwip_comm.c/h 六个文件，这些文件的作用，如下表所示：

文件	描述
lwipopts.h	lwIP 配置文件
cc.h	配置 lwIP 数据类型
ethernetif.c/h	网卡文件，专门处理接收的数据包以及发送的数据包
lwip_comm.c/h	配置 lwIP 相关参数，比如 MAC 地址

表 2.4.2.1 arch 文件夹存放的文件

除了 lwip_comm.c/h 和 ethernetif.h 文件之外，其他文件可以在“contrib-2.1.0”包下获取。建议使用 ST 官方提供的 lwIP 工程中的 lwipopts.h 和 cc.h 文件，因为它们已经在 ST 芯片上实现了 lwIP 移植。这些文件可以在 ST 官方网站上下载的 HAL 版本的 F4 工程包中找到。将 cc.h 和 lwipopts.h 文件复制到实验的 arch 文件夹下即可使用。

lwIP例程1 lwIP裸机移植 > Middlewares > lwip > arch		
名称	类型	大小
cc.h	C/C++ Header	3 KB
lwipopts.h	C/C++ Header	6 KB

图 2.4.2.5 arch 文件夹保存 cc.h 和 lwipopts.h 文件

ethernetif.c 网卡文件定义了底层驱动函数，这些函数在网卡中至关重要。该文件位于 contrib 包的 contrib-2.1.0/examples/ethernetif 路径下，可以直接使用。但请注意，这个网卡文件是一个通用模板，需要用户自行修改。另外，用户也可以参考 STM32Cube_FW_F4_V1.27.0 压缩包下的 ethernetif.c（裸机工程）文件作为参考。在工程中创建 ethernetif.h 文件来声明 ethernetif.c 中的函数，并将其保存在 arch 文件夹下。



图 2.4.2.6 arch 文件夹添加 ethernetif.c/h 文件

lwip_comm.c 和 lwip_comm.h 是将 lwIP 源码和以太网驱动库结合起来的桥梁，这两个文件非常重要。它们由正点原子提供，用户可以将正点原子已经编写好的 lwip_comm.c/h 文件复制到 arch 文件夹中，以方便使用。文件结构如下图所示。

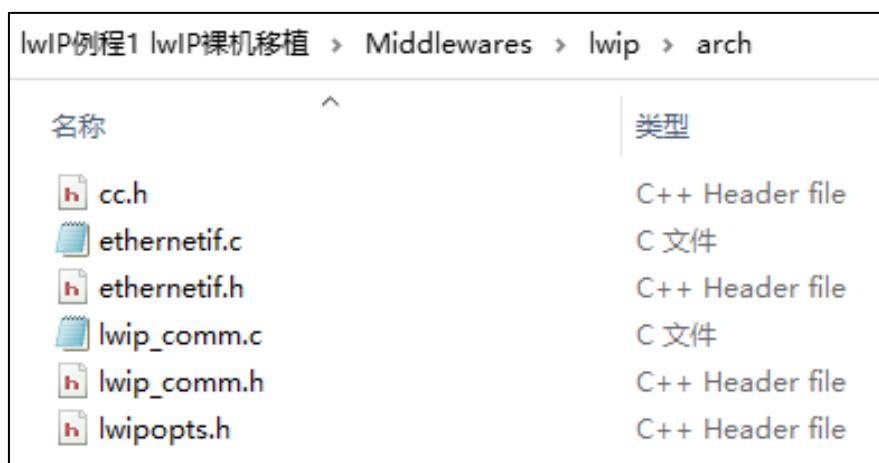


图 2.4.2.7 arch 文件夹添加 lwip_comm.c/h 文件

至此，我们完成了 arch 文件夹添加的.c 文件，然后我们把这些.c 文件添加到 Middlewares/lwip/arch 分组当中，如下图所示。

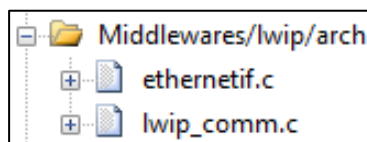


图 2.4.2.8 Middlewares/lwip/arch 分组添加的文件

(5) 添加 lwIP 头文件路径，如下图所示。

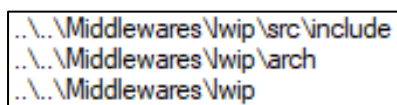


图 2.4.2.9 添加文件路径

注意：用户在使用 lwip/src/apps 路径下的文件时，必须添加....\Middlewares\lwip\src\include

\\lwip\\apps 头文件路径，以确保正确引用所需的头文件。

(6) 修改工程目标名称

本教程以标准例程-HAL 库版本的内存管理实验工程为基础工程，工程名称为“MALLO C”。为了规范工程，建议将工程的目标名称修改为“lwIP”或根据实际场景进行修改，修改过程如下图所示。

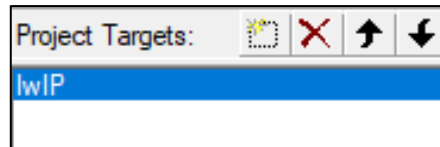


图 2.4.2.10 修改工程目标名称

2.4.3 arch 文件夹下的文件修改

前面我们已经在工程中添加多个分组来描述 lwIP 源码和设置 lwIP 源码文件路径，此时我们已经移植完成了，但是编译工程时会出现很多的错误，这些错误都是指向 arch 文件夹下的文件，下面我们分别地讲解这些文件到底如何修改。

(1) 修改 cc.h 文件

该文件主要配置 lwIP 数据类型，cc.h 文件源码如下：

```
#ifndef __CC_H__
#define __CC_H__

#include <stdlib.h>
#include <stdio.h>

typedef int sys_prot_t;

#if defined (__GNUC__) & !defined (__CC_ARM)

#define LWIP_TIMEVAL_PRIVATE 0
#include <sys/time.h>

#endif

/* define compiler specific symbols */
#if defined (__ICCARM__)

#define PACK_STRUCT_BEGIN
#define PACK_STRUCT_STRUCT
#define PACK_STRUCT_END
#define PACK_STRUCT_FIELD(x) x
#define PACK_STRUCT_USE_INCLUDES

#elif defined (__GNUC__)
```

```
#define PACK_STRUCT_BEGIN
#define PACK_STRUCT_STRUCT __attribute__((packed))
#define PACK_STRUCT_END
#define PACK_STRUCT_FIELD(x) x

#elif defined (__CC_ARM)

#define PACK_STRUCT_BEGIN __packed
#define PACK_STRUCT_STRUCT
#define PACK_STRUCT_END
#define PACK_STRUCT_FIELD(x) x

#elif defined (__TASKING__)

#define PACK_STRUCT_BEGIN
#define PACK_STRUCT_STRUCT
#define PACK_STRUCT_END
#define PACK_STRUCT_FIELD(x) x

#endif

#define LWIP_PLATFORM_ASSERT(x) do
    {printf("Assertion \"%s\" failed at line %d in %s\n", \
        x, __LINE__, __FILE__); } while(0)

/* Define random number generator function */
#define LWIP_RAND() ((u32_t)rand())

#endif /* __CC_H__ */
```

在原文件中删除了两处地方，分别是“#include "cpu.h"”和“#define LWIP_PROVIDE_ERRNO”宏定义，因为 F4 没有 cpu.h 文件，且 LWIP_PROVIDE_ERRNO 宏定义已经在 lwipopts.h 文件中定义了。这些修改是为了避免重复定义。

(2) ethernet_chip.h 文件

ethernet_chip.c 文件主要声明了与开发板以太网芯片相关的函数和配置项。其中，PHY_TYPE 配置项需要根据开发板底板所搭载的以太网芯片类型进行选择。例如，如果开发板的底板搭载了 LAN8720 以太网芯片，则 PHY_TYPE 配置项应填写为 LAN8720；如果搭载的是 YT8512C 以太网芯片，则 PHY_TYPE 配置项应填写为 YT8512C。其他类型的以太网芯片可依此类推进行选择。温馨提示：作者已经编写了自动检测代码，所以用户可不需要设置 PHY_TYPE 配置项来选择 PHY 芯片。

(3) main.c 文件

```
int main(void)
{
    /* 一系列初始化 */
```

```
pcf8574_init();

while(lwip_comm_init() != 0)
{
    lcd_show_string(6, 110, 200, 16, 16, "lwIP Init failed!!", BLUE);
    delay_ms(500);
    lcd_fill(6, 50, 200 + 30, 50 + 16, WHITE);
    lcd_show_string(6, 110, 200, 16, 16, "Retrying... ", BLUE);
    delay_ms(500);
    LED1_TOGGLE();
}

#if LWIP_DHCP
/* 等待 DHCP 获取成功/超时溢出 */
while((lwipdev.dhcpstatus != 2)&&(lwipdev.dhcpstatus != 0xFF))
{
    lwip_pkt_handle();
    lwip_periodic_handle();
}
#endif
while(1)
{
    lwip_periodic_handle(); /* LWIP 轮询任务 */
    lwip_pkt_handle();
    delay_ms(2);
}
}
```

该函数首先调用 `lwip_comm_init` 函数来初始化以太网 IO 和网卡驱动。然后，它调用 `lwip_periodic_handle` 函数来获取 DHCP 分配的动态 IP 地址。最后，在 `while(1)` 循环中不断遍历 `lwip_periodic_handle` 和 `lwip_pkt_handle` 函数。

编译工程并下载到开发板，打开串口调式助手来查看动态 IP 地址，如下图所示。

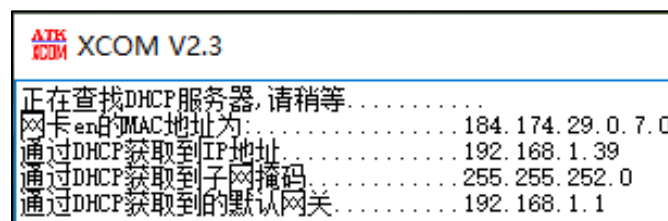
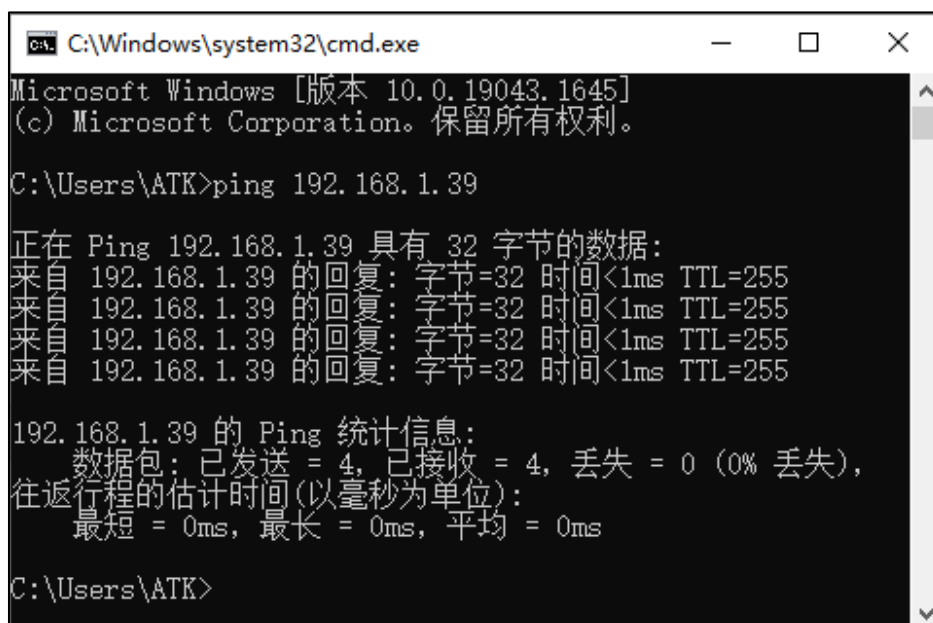


图 2.4.3.1 系统获取动态 IP 地址成功

在 PC 机上按下“win+r”快捷键并输入 `cmd` 进入命令行，然后输入“`ping 192.168.1.39`”命令。如果命令行显示“无法访问目标主机”字符串，则表示系统无法与开发板通讯，表明移植的工程失败。如果命令行显示“字节=32 时间<1ms TTL=255”字符串，则表示系统与开发板通讯正常，证明该工程移植成功。接下来，可以使用类似的方法来 ping 其他 IP 地址，如下

图所示。



```
C:\Windows\system32\cmd.exe
Microsoft Windows [版本 10.0.19043.1645]
(c) Microsoft Corporation。保留所有权利。

C:\Users\ATK>ping 192.168.1.39

正在 Ping 192.168.1.39 具有 32 字节的数据:
来自 192.168.1.39 的回复: 字节=32 时间<1ms TTL=255
来自 192.168.1.39 的回复: 字节=32 时间<1ms TTL=255
来自 192.168.1.39 的回复: 字节=32 时间<1ms TTL=255
来自 192.168.1.39 的回复: 字节=32 时间<1ms TTL=255

192.168.1.39 的 Ping 统计信息:
    数据包: 已发送 = 4, 已接收 = 4, 丢失 = 0 (0% 丢失),
    往返行程的估计时间(以毫秒为单位):
        最短 = 0ms, 最长 = 0ms, 平均 = 0ms

C:\Users\ATK>
```

图 2.4.3.2 ping IP 地址成功

该实验的实验工程，请参考《lwIP 例程 1 lwIP 裸机移植》。

2.5 lwIP 移植流程（H7R7 与 H7R3）

2.5.1 添加网卡驱动程序

首先，我要声明的是，H7R7 和 H7R3 开发板的 lwIP 移植流程是相同的。

接下来，打开正点原子提供的“lwIP 例程 1 lwIP 裸机移植”实验，并进入 Drivers\BSP 文件夹。在此文件夹中，你会找到一个 ETHERNET 文件夹。这个文件夹包含了四个文件：ethernet.c、ethernet.h、ethernet_chip.c 和 ethernet_chip.h。其中，ethernet.c 和 ethernet.h 文件主要定义了以太网驱动的初始化和 MAC 驱动程序。而 ethernet_chip.c 和 ethernet_chip.h 文件则主要定义了 PHY 的配置函数。在进行移植时，你需要将这四个文件拷贝到自己的 Drivers\BSP\ETHERNET 文件夹下。同时，你还需要将 ethernet.c 和 ethernet_chip.c 这两个文件添加到工程中的 Drivers/BSP 分组中。

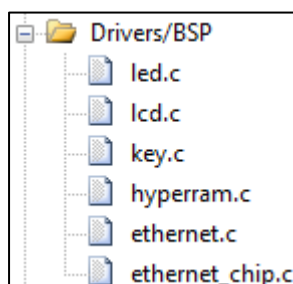


图 2.5.1.1 添加 ethernet.c 后的工程

至此，网卡驱动添加完成，我们编译工程会出现很多错误，如下图所示。

```
#20: identifier "ETH_HandleTypeDef" is undefined
/* 以太网句柄 */
#20: identifier "ETH_DMADescTypeDef" is undefined
/* 以太网DMA接收描述符数据结构体指针 */
#20: identifier "ETH_DMADescTypeDef" is undefined
/* 以太网DMA发送描述符数据结构体指针 */
#20: identifier "ETH_DMADescTypeDef" is undefined
```

图 2.5.1.2 缺少以太网 hal 文件

上述的问题导致原因是工程缺少以太网驱动文件。为了解决这个问题，我们需要在工程的 Drivers/STM32H7RSxx_HAL_Driver 分组下添加一个以太网驱动文件，即 stm32h7rsxx_hal_eth.c 文件。添加完驱动文件后，我们需要打开 stm32h7rsxx_hal_conf.h 文件，并在该文件中启用 HAL_ETH_MODULE_ENABLED 以太网模块。这个配置项在该文件的最顶端定义。

2.5.2 添加 lwIP 源文件

把 lwip-2.1.3 文件夹下的 src 文件夹复制到“lwIP 例程 1 lwIP 裸机移植”工程 Middlewares\lwip 路径下，如下图所示。



图 2.5.2.1 复制 lwIP 源码到工程中

打开“lwIP 例程 1 lwIP 裸机移植”工程添加 Middlewares/lwip/src/api、Middlewares/lwip/src/core、Middlewares/lwip/src/netif 和 Middlewares/lwip/arch 分组，下面我们分别地讲解这些分组需要添加哪些文件，如下所示：

(1) Middlewares/lwip/src/api 分组添加上图中的 src/api 路径下的全部.c 文件，如下图所示。

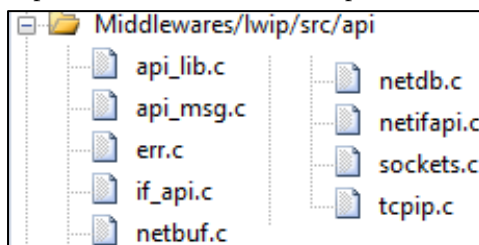


图 2.5.2.2 Middlewares/lwip/src/api 分组添加的文件

(2) Middlewares/lwip/src/core 分组添加图 2.5.2.1 中的 src/core 路径下除 ipv6 文件夹的全部.c 文件，如下图所示。

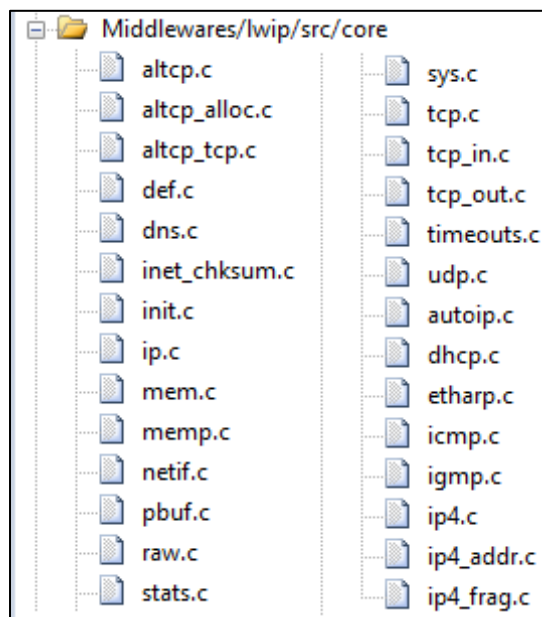


图 2.5.2.3 Middlewares/lwip/src/core 分组添加的文件

(3) Middlewares/lwip/src/netif 分组添加图 2.4.2.1 中的 src/netif 路径下的 ethernet.c 文件，如下图所示。

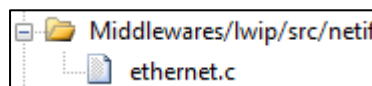


图 2.5.2.4 Middlewares/lwip/src/netif 分组添加的文件

(4) Middlewares/lwip/arch 添加图 2.55.2.1 中的 arch 文件夹下的全部.c 文件，前面我们没有在该文件夹下放置任何的文件，下面我们来讲解这个文件夹到底存放哪些文件。

arch 文件夹主要存放 lwipopts.h、cc.h、ethernetif.c/h 和 lwip_comm.c/h 六个文件，这些文件的作用，如下表所示：

文件	描述
lwipopts.h	lwIP 配置文件
cc.h	配置 lwIP 数据类型
ethernetif.c/h	网卡文件，专门处理接收的数据包以及发送的数据包
lwip_comm.c/h	配置 lwIP 相关参数，比如 MAC 地址

表 2.5.2.1 arch 文件夹存放的文件

除了 lwip_comm.c/h 和 ethernetif.h 文件之外，其他文件可以在“contrib-2.1.0”包下获取。建议使用 ST 官方提供的 lwIP 工程中的 lwipopts.h 和 cc.h 文件，因为它们已经在 ST 芯片上实现了 lwIP 移植。这些文件可以在 ST 官方网站上下载的 HAL 版本的 H7R 工程包中找到。将 cc.h 和 lwipopts.h 文件复制到实验的 arch 文件夹下即可使用。

lwIP例程1 lwIP裸机移植 > Middlewares > lwip > arch		
名称	类型	大小
cc.h	C/C++ Header	3 KB
lwipopts.h	C/C++ Header	6 KB

图 2.4.2.5 arch 文件夹保存 cc.h 和 lwipopts.h 文件

ethernetif.c 网卡文件定义了底层驱动函数，这些函数在网卡中至关重要。该文件位于 contrib 包的 contrib-2.1.0\examples\ethernetif 路径下，可以直接使用。但请注意，这个网卡文件是

一个通用模板，需要用户自行修改。另外，用户也可以参考 en.stm32cubeh7rs-v1-0-0 压缩包下的 ethernetif.c（裸机工程）文件作为参考。在工程中创建 ethernetif.h 文件来声明 ethernetif.c 中的函数，并将其保存在 arch 文件夹下。

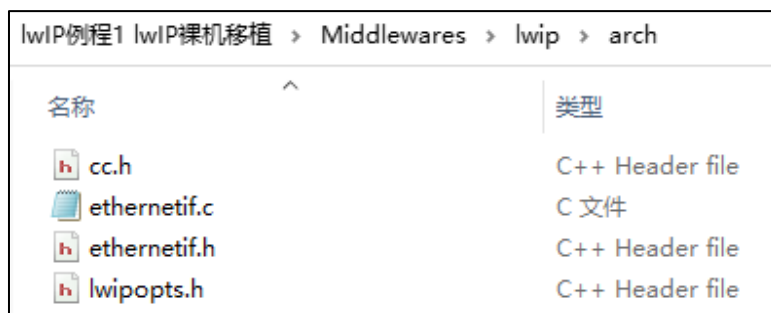


图 2.5.2.6 arch 文件夹添加 ethernetif.c/h 文件

lwip_comm.c 和 lwip_comm.h 是将 lwIP 源码和以太网驱动库结合起来的桥梁，这两个文件非常重要。它们由正点原子提供，用户可以将正点原子已经编写好的 lwip_comm.c/h 文件复制到 arch 文件夹中，以方便使用。文件结构如下图所示。

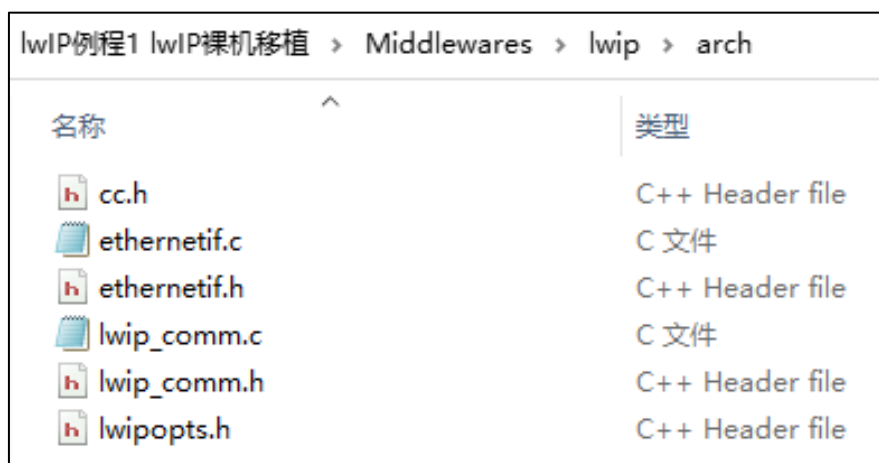


图 2.5.2.7 arch 文件夹添加 lwip_comm.c/h 文件

至此，我们完成了 arch 文件夹添加的.c 文件，然后我们把这些.c 文件添加到 Middlewares/lwip/arch 分组当中，如下图所示。

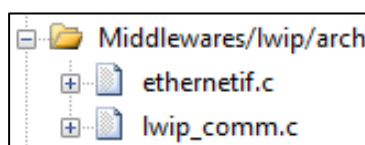


图 2.5.2.8 Middlewares/lwip/arch 分组添加的文件

(5) 添加 lwIP 头文件路径，如下图所示。

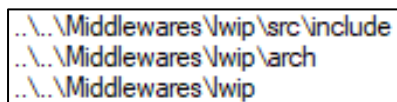


图 2.5.2.9 添加文件路径

注意：用户在使用 lwip/src/apps 路径下的文件时，必须添加....\Middlewares\lwip\src\include \lwip\apps 头文件路径，以确保正确引用所需的头文件。

(6) 修改 sys.c 文件。如下图所示。

```
rcc_osc_init_struct.PLL3.PLLSource = RCC_PLLSOURCE_HSE;
rcc_osc_init_struct.PLL3.PLLM = 25;
rcc_osc_init_struct.PLL3.PLLN = 300;
rcc_osc_init_struct.PLL3.PLLP = 2;
rcc_osc_init_struct.PLL3.PLLQ = 15;
rcc_osc_init_struct.PLL3.PLLR = 2;
rcc_osc_init_struct.PLL3.PLLS = 6; /* 300Mhz/6 = 50Mhz */
rcc_osc_init_struct.PLL3.PLLT = 2;
rcc_osc_init_struct.PLL3.PLLFractional = 0;
```

图 2.5.2.10 设置输出 ETH 时钟

2.4.3 arch 文件夹下的文件修改

前面我们已经在工程中添加多个分组来描述 lwIP 源码和设置 lwIP 源码文件路径，此时我们已经移植完成了，但是编译工程时会出现很多的错误，这些错误都是指向 arch 文件夹下的文件，下面我们分别地讲解这些文件到底如何修改。

(1) 修改 cc.h 文件

该文件主要配置 lwIP 数据类型，cc.h 文件源码如下：

```
#ifndef __CC_H__
#define __CC_H__

#include <stdlib.h>
#include <stdio.h>

typedef int sys_prot_t;

#if defined (__GNUC__) & !defined (__CC_ARM)

#define LWIP_TIMEVAL_PRIVATE 0
#include <sys/time.h>

#endif

/* define compiler specific symbols */
#if defined (__ICCARM__)

#define PACK_STRUCT_BEGIN
#define PACK_STRUCT_STRUCT
#define PACK_STRUCT_END
#define PACK_STRUCT_FIELD(x) x
#define PACK_STRUCT_USE_INCLUDES

#elif defined (__GNUC__)

#define PACK_STRUCT_BEGIN
#define PACK_STRUCT_STRUCT __attribute__((packed))
#define PACK_STRUCT_END
```



```
#define PACK_STRUCT_FIELD(x) x

#elif defined (__CC_ARM)

#define PACK_STRUCT_BEGIN __packed
#define PACK_STRUCT_STRUCT
#define PACK_STRUCT_END
#define PACK_STRUCT_FIELD(x) x

#elif defined (__TASKING__)

#define PACK_STRUCT_BEGIN
#define PACK_STRUCT_STRUCT
#define PACK_STRUCT_END
#define PACK_STRUCT_FIELD(x) x

#endif

#define LWIP_PLATFORM_ASSERT(x) do
    {printf("Assertion \"%s\" failed at line %d in %s\n", \
        x, __LINE__, __FILE__); } while(0)

/* Define random number generator function */
#define LWIP_RAND() ((u32_t)rand())

#endif /* __CC_H__ */
```

在原文件中删除了两处地方，分别是“#include "cpu.h"”和“#define LWIP_PROVIDE_ERRNO”宏定义，因为 H7R 没有 cpu.h 文件，且 LWIP_PROVIDE_ERRNO 宏定义已经在 lwipopts.h 文件中定义了。这些修改是为了避免重复定义。

(2) ethernet_chip.h 文件

ethernet_chip.c 文件主要声明了与开发板以太网芯片相关的函数和配置项。其中，PHY_TYPE 配置项需要根据开发板底板所搭载的以太网芯片类型进行选择。例如，如果开发板的底板搭载了 LAN8720 以太网芯片，则 PHY_TYPE 配置项应填写为 LAN8720；如果搭载的是 YT8512C 以太网芯片，则 PHY_TYPE 配置项应填写为 YT8512C。其他类型的以太网芯片可依此类推进行选择。温馨提示：作者已经编写了自动检测代码，所以用户可不需要设置 PHY_TYPE 配置项来选择 PHY 芯片。

(3) main.c 文件

```
int main(void)
{
    /* 一系列初始化 */

    while(lwip_comm_init() != 0)
    {
```

```

    lcd_show_string(6, 110, 200, 16, 16, "lwIP Init failed!!", BLUE);
    delay_ms(500);
    lcd_fill(6, 50, 200 + 30, 50 + 16, WHITE);
    lcd_show_string(6, 110, 200, 16, 16, "Retrying... ", BLUE);
    delay_ms(500);
    LED1_TOGGLE();
}

#if LWIP_DHCP
/* 等待 DHCP 获取成功/超时溢出 */
while((lwipdev.dhcpstatus != 2)&&(lwipdev.dhcpstatus != 0XFF))
{
    lwip_pkt_handle();
    lwip_periodic_handle();
}
#endif
while(1)
{
    lwip_periodic_handle(); /* LWIP 轮询任务 */
    lwip_pkt_handle();
    delay_ms(2);
}
}

```

该函数首先调用 `lwip_comm_init` 函数来初始化以太网 IO 和网卡驱动。然后，它调用 `lwip_periodic_handle` 函数来获取 DHCP 分配的动态 IP 地址。最后，在 `while(1)` 循环中不断遍历 `lwip_periodic_handle` 和 `lwip_pkt_handle` 函数。

编译工程并下载到开发板，打开串口调式助手来查看动态 IP 地址，如下图所示。

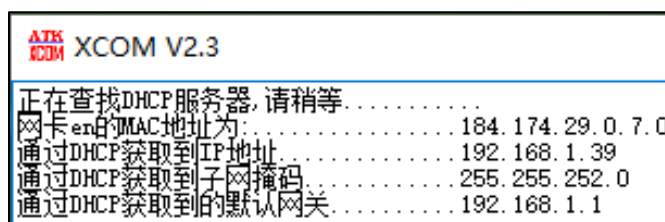


图 2.5.3.1 系统获取动态 IP 地址成功

在 PC 机上按下“win+r”快捷键并输入 `cmd` 进入命令行，然后输入“`ping 192.168.1.39`”命令。如果命令行显示“无法访问目标主机”字符串，则表示系统无法与开发板通讯，表明移植的工程失败。如果命令行显示“字节=32 时间<1ms TTL=255”字符串，则表示系统与开发板通讯正常，证明该工程移植成功。接下来，可以使用类似的方法来 ping 其他 IP 地址，如下图所示。

```
C:\Windows\system32\cmd.exe
Microsoft Windows [版本 10.0.19043.1645]
(c) Microsoft Corporation。保留所有权利。

C:\Users\ATK>ping 192.168.1.39

正在 Ping 192.168.1.39 具有 32 字节的数据:
来自 192.168.1.39 的回复: 字节=32 时间<1ms TTL=255
来自 192.168.1.39 的回复: 字节=32 时间<1ms TTL=255
来自 192.168.1.39 的回复: 字节=32 时间<1ms TTL=255
来自 192.168.1.39 的回复: 字节=32 时间<1ms TTL=255

192.168.1.39 的 Ping 统计信息:
    数据包: 已发送 = 4, 已接收 = 4, 丢失 = 0 (0% 丢失),
    往返行程的估计时间(以毫秒为单位):
        最短 = 0ms, 最长 = 0ms, 平均 = 0ms

C:\Users\ATK>
```

图 2.5.3.2 ping IP 地址成功

该实验的实验工程，请参考《lwIP 例程 1 lwIP 裸机移植》。

注意：使用 H7R7 和 H7R3 开发板的用户，可以不看 2.6 小节的内容。

2.6 使用 AC6 编译工程（扩展）

首先说明，本教程配套的实验工程除了 H7R7 和 H7R3 开发板使用 AD6 开发之外，其他都采用 AC5 进行开发。

AC 是“ARM Compiler”的简称，是用于编译 ARM 处理器代码的编译工具链，AC 后面的 5 和 6 表示的是 AC 的版本号，在 Keil MDK 中集成了 AC5 和 AC6 编译工具链，这里简单地对比一下 AC5 和 AC6 的差异，如下表所示：

对比项	AC5	AC6	说明
中文支持	较好	较差	AC6 对中文支持极差，goto definition 无法使用，报错等
代码兼容性	较好	较差	AC6 对某些代码优化可能导致运行异常，需慢慢调试
编译速度	较慢	较快	AC6 编译速度比 AC5 快
语法检查	一般	严格	AC6 语法检查非常严格，代码严谨性较好

表 2.6.4.1 AC5&AC6 简单对比

虽然 AC6 的编译速度比较快，但是 AC5 在各方面的兼容性都比 AC6 要好，因此笔者推荐新手读者使用 AC5。当然读者可以自由地选择使用 AC5 或 AC6，正点原子的源码都是支持 AC6 的，只不过在 Keil MDK 的选项配置方面有些差异，具体差异如下表所示：

选项	AC5	AC6	说明
Target -> ARM Compiler	V5.xx	V6.xx	选择对应的编译工具

表 2.6.4.2 AC5&AC6 配置差异

通过以上配置正点原子的裸机程序已经能够正常使用 AC6 编译工具进行编译了，但是对于 FreeRTOS 和 lwIP 还需要稍作修改，需要修改四个地方，如下所示：

1. FreeRTOS 的 port.c 文件

在前面小节中，往工程添加 port.c 文件的路径为 FreeRTOS/portable/RVDS，这是在使用 AC5 编译工具的前提下，如果使用 AC6 编译工具，那么 port.c 文件的路径应该改为 FreeRTOS/portable/GCC，对于正点原子不同的 STM32 开发板，只要根据表 2.1.2.4 将对应的 port.c 文件添加到工程中即可。

2. 头文件路径

在前面小节中，添加了一个以 port.c 文件路径为路径的头文件路径，由于使用 AC6 编译工具时，使用了别的 port.c 文件，因此需要将原来以 port.c 文件路径为路径的头文件路径修改为新的 port.c 文件的路径为路径的头文件路径，修改后如下图所示（这里以正点原子的 STM32F4 系列开发板为例，其他类型的开发板类似）：

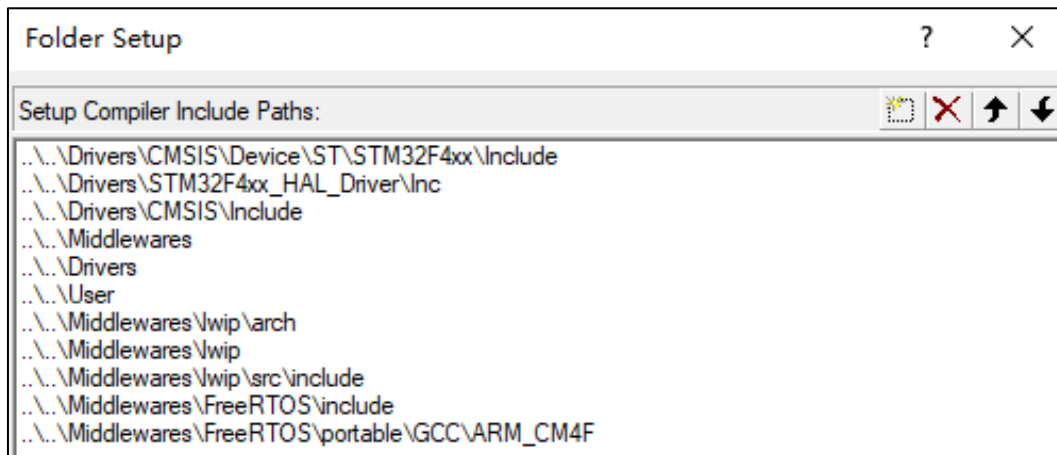


图 2.6.4.1 修改头文件路径

这里要注意的是，在切换了 Keil MDK 编译工具之后，最好完整地重新编译一遍工程，即 Rebuild all target files 而不是 Build Target，以免出现一些莫名的问题。

3. 设置 MiscControls

编译工程时，出现“.warning: illegal character encoding in string literal [-Winvalid-source-encoding]”警告，我们可在 MiscControls 选项中添加“-Wno-invalid-source-encoding”字段来屏蔽这个警告，如下图所示。

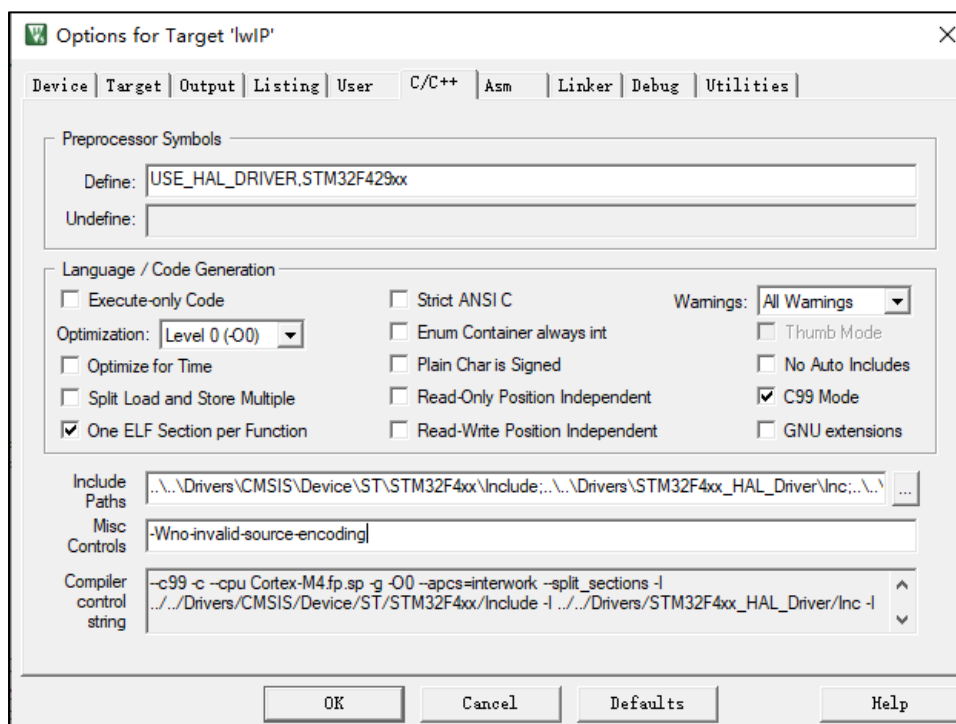


图 2.6.4.2 屏蔽警告

4. 配置 LWIP_TIMEVAL_PRIVATE

在 cc.h 文件中，注销#include <sys/time.h>头文件路径，并设置 LWIP_TIMEVAL_PRIVATE 配置项为 1。

第三章 lwIP 带操作系统移植

在本章中，我们将深入探讨带操作系统的 lwIP 移植技术，并选用 FreeRTOS 作为操作系统。建议参考正点原子的《FreeRTOS 开发指南》教程，该教程提供了详细的移植流程。此外，本章的移植例程将基于上一章的内容进行展开。

本章分为如下几个部分：

3.1 lwIP 前期准备

3.2 lwIP 移植流程（探索者、DMF407）

3.3 lwIP 移植流程（阿波罗、北极星）

3.4 sys_arch 文件解析

3.1 lwIP 前期准备

本章将使用第二章的例程作为移植工程的基础，并将其命名为《lwIP 例程 6 lwIP_FreeRTOS 移植》。关于 FreeRTOS 操作系统的移植，建议参考正点原子的《FreeRTOS 开发指南》，但此处不会详细讲解 FreeRTOS 的移植过程。

3.2 lwIP 移植流程（探索者、DMF407）

3.2.1 修改 lwipopts.h

这个文件是 lwIP 的配置文件，用于设置 lwIP 的相关参数。在第二章例程中，使用了针对 lwIP 无操作系统下的配置文件，即 lwipopts.h。在本实验中，我们将使用 STM32Cube_FW_F4_V1.26.0\Projects\STM32469I_EVAL\Applications\LwIP\LwIP_HTTP_Server_Netconn_RTOS\Inc\路径下的 lwipopts.h 文件来替换原有的 lwipopts.h 文件。新的 lwipopts.h 文件源码如下所示：

```
#ifndef __LWIPOPTS_H__
#define __LWIPOPTS_H__

/* NO_SYS 表示无操作系统模拟层，无操作系统为 1，有操作系统设置为 0
   注意这个参数设置会编译不同 */
#define NO_SYS 0

/* ----- 内存选项 ----- */
/* 内存对齐，按照 4 字节对齐 */
#define MEM_ALIGNMENT 4

/* 堆内存的大小，如果需要更大的堆内存，那么设置高一点 */
#define MEM_SIZE (10*1024)

/* MEMP_NUM_PBUF：设置内存池的数量 */
#define MEMP_NUM_PBUF 10

/* MEMP_NUM_UDP_PCB：UDP 协议控制块的数量。*/
```

```

#define MEMP_NUM_UDP_PCB                6

/* MEMP_NUM_TCP_PCB: TCP 的数量. */
#define MEMP_NUM_TCP_PCB                10

/* MEMP_NUM_TCP_PCB_LISTEN: 监听 TCP 的数量. */
#define MEMP_NUM_TCP_PCB_LISTEN        5

/* MEMP_NUM_TCP_SEG: 同时排队的 TCP 的数量段. */
#define MEMP_NUM_TCP_SEG                8

/* MEMP_NUM_SYS_TIMEOUT: 超时模拟活动的数量. */
#define MEMP_NUM_SYS_TIMEOUT            10


/* ----- Pbuf 选项 ----- */
/* PBUF_POOL 内存池中每个内存块大小 */
#define PBUF_POOL_SIZE                  8

/* PBUF_POOL_BUFSIZE: pbuf 池中每个 pbuf 的大小. */
#define PBUF_POOL_BUFSIZE               1524


/* ----- TCP 选项 ----- */
#define LWIP_TCP                        1
#define TCP_TTL                        255

/* 控制 TCP 是否应该对到达的段进行排队
   秩序。如果你的设备内存不足，定义为 0. */
#define TCP_QUEUE_OOSEQ                0

/* TCP 最大段大小 */ /* TCP_MSS = (Ethernet MTU - IP header size - TCP header size)
*/
#define TCP_MSS                        (1500 - 40)

/* TCP 发送者缓冲区空间 (字节). */
#define TCP_SND_BUF                    (4*TCP_MSS)

/* TCP_SND_QUEUELEN: TCP 发送缓冲区空间。这必须是至少
   需要 (2 * TCP_SND_BUF/TCP_MSS) 才能正常工作 */
#define TCP_SND_QUEUELEN                (2* TCP_SND_BUF/TCP_MSS)

/* TCP 接收窗口 */
#define TCP_WND                        (2*TCP_MSS)


/* ----- ICMP 选项 ----- */

```

```

#define LWIP_ICMP                                1

/* ----- DHCP 选项 ----- */
/* 如果您希望 DHCP 配置为, 请将 LWIP_DHCP 定义为 1 */
#define LWIP_DHCP                                1

/* ----- UDP 选项 ----- */
#define LWIP_UDP                                1
#define UDP_TTL                                  255

/* ----- Statistics 选项 ----- */
#define LWIP_STATS                              0

/* ----- 链接回调选项 ----- */
/* WIP_NETIF_LINK_CALLBACK==1:支持来自接口的回调函数
   每当链接改变(例如, 向下链接)
   */
#define LWIP_NETIF_LINK_CALLBACK                1
/*
   ----- 帧校验和选项 -----
   */

/*
The STM32F4x7 allows computing and verifying the IP, UDP, TCP and ICMP
checksums by hardware:
- To use this feature let the following define uncommented.
- To disable it and process by CPU comment the the checksum.
*/
#define CHECKSUM_BY_HARDWARE

#ifdef CHECKSUM_BY_HARDWARE
#define CHECKSUM_GEN_IP                          0
#define CHECKSUM_GEN_UDP                        0
#define CHECKSUM_GEN_TCP                       0
#define CHECKSUM_CHECK_IP                      0
#define CHECKSUM_CHECK_UDP                     0
#define CHECKSUM_CHECK_TCP                     0
#define CHECKSUM_GEN_ICMP                      0

```



```
#else

#define CHECKSUM_GEN_IP                1
#define CHECKSUM_GEN_UDP                1
#define CHECKSUM_GEN_TCP                1
#define CHECKSUM_CHECK_IP               1
#define CHECKSUM_CHECK_UDP              1
#define CHECKSUM_CHECK_TCP              1
#define CHECKSUM_GEN_ICMP               1
#endif

/*
-----
----- 连续层的选择 -----
-----

*/
/**
 * LWIP_NETCONN==1: 启用 Netconn API (需要使用 api_lib.c)
 */
#define LWIP_NETCONN                    1

/*
-----
----- Socket 选项 -----
-----

*/
/**
 * LWIP_SOCKET==1: 启用 Socket API (要求使用 Socket .c)
 */
#define LWIP_SOCKET                     0

/*
-----
----- httpd options -----
-----

*/
/** Set this to 1 to include "fsdata_custom.c" instead of "fsdata.c" for the
 * file system (to prevent changing the file included in CVS) */
#define HTTPD_USE_CUSTOM_FSDATA        1

/*
-----
----- 操作系统选项 -----
-----

```

```

*/

#define TCPIP_THREAD_NAME           " TCP/IP"
#define TCPIP_THREAD_STACKSIZE      1000
#define TCPIP_MBOX_SIZE             6
#define DEFAULT_UDP_RECVMBOX_SIZE  6
#define DEFAULT_TCP_RECVMBOX_SIZE  6
#define DEFAULT_ACCEPTMBOX_SIZE     6
#define DEFAULT_THREAD_STACKSIZE    500
#define TCPIP_THREAD_PRIO           5

#endif /* __LWIPOPTS_H__ */

```

作者对新的 lwipopts.h 文件进行了中文注释，并修改了某些配置项的数值。例如，将 TCPIP_THREAD_PRIO 配置项修改为 5，该配置项用于设置协议栈任务的优先级。此外，读者也可以使用正点原子《lwIP 例程 5 lwIP_FreeRTOS 移植》例程中的 lwipopts.h 文件作为参考，该文件的内容是基于上述文件内容改编而来的。

3.2.2 添加 sys_arch.c/h 文件

如果 lwIP 使用操作系统，那么 sys_arch.c/h 这两个文件是必要的，因为它们是 lwIP 内核与操作系统交互的接口文件。在实验中使用 FreeRTOS 操作系统时，sys_arch.c 文件中包含的邮箱、信号量以及互斥锁等 IPC 策略都是由 FreeRTOS 操作系统提供的。需要注意的是，FreeRTOS 操作系统没有邮箱的概念，可以使用消息队列来替代邮箱机制。打开 contrib-2.1.0\ports\freertos 路径下的文件夹，可以找到与 FreeRTOS 相关的 sys_arch.c/h 文件，将这些文件复制到 Middlewares\lwip\arch 路径下，以完成文件结构的配置。

lwIP例程5 lwIP_FreeRTOS移植 > Middlewares > lwip > arch		
名称	类型	大小
cc.h	C++ Header file	3 KB
ethernetif.c	C 文件	15 KB
ethernetif.h	C++ Header file	2 KB
lwip_comm.c	C 文件	11 KB
lwip_comm.h	C++ Header file	3 KB
lwipopts.h	C++ Header file	8 KB
sys_arch.c	C 文件	18 KB
sys_arch.h	C++ Header file	4 KB

图 3.2.2.1 arch 文件夹的文件结构

打开工程并在 Middlewares/lwip/arch 分组下添加 sys_arch.c 文件，如下图所示。

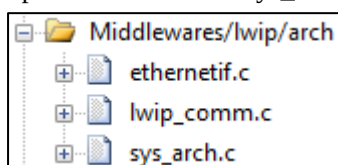


图 3.2.2.2 添加 sys_arch.c 文件

在编译工程时，我们遇到了很多错误，这些错误都指向 `err.c` 文件中的 `err_to_errno_table` 数组。MDK 提示这些错误信息如下图所示。

```
error: #20: identifier "EINPROGRESS" is undefined
    Operation in progress    */
error: #20: identifier "EINVAL" is undefined
    Illegal value.          */
error: #20: identifier "EADDRINUSE" is undefined
    Address in use.         */
error: #20: identifier "EALREADY" is undefined
```

图 3.2.2.3 `err.c` 文件错误

为了解决上述的错误，我们需要在 `lwipopts.h` 文件中添加 `LWIP_PROVIDE_ERRNO` 配置项，并将其设置为 1。然后，我们再次编译工程，这次出现了一个指向 `sys_now` 函数的错误。MDK 提示错误信息如下图所示。

```
Error: L6200E: Symbol sys_now multiply defined (by sys_arch.o and ethernetif.o).
```

图 3.2.2.4 重复定义 `sys_now` 函数

由于 `sys_now` 函数原本定义在 `ethernetif.c` 文件中，而添加的 `sys_arch.c` 文件已经为用户定义了 `sys_now` 函数，因此我们需要将 `ethernetif.c` 文件中的 `sys_now` 函数去除。同时，注意将 `cc.h` 文件中的 “`typedef int sys_prot_t;`” 代码注释掉，因为 `sys_prot_t` 变量已经在 `sys_arch.c` 文件中定义了，无需重复定义。完成这些修改后，再次编译工程就不会出现错误了。

3.1.3 修改 `lwip_comm.c`

该文件主要用于修改网络配置信息，包括默认 IP 的配置、lwIP 的初始化和 DHCP 处理等。在 `lwip_comm.c` 文件中进行简单修改，修改流程如下。

- (1) 删除 lwIP 轮询函数 `lwip_periodic_handle`。
- (2) 修改 `lwip_comm_init` 函数，修改内容如下：

```
/**
 * @breif      LWIP 初始化 (LWIP 启动的时候使用)
 * @param      无
 * @retval     0, 成功
 *             1, 内存错误
 *             2, 以太网芯片初始化失败
 *             3, 网卡添加失败.
 */
uint8_t lwip_comm_init(void)
{
    /* 此处省略代码..... */

    tcpip_init(NULL, NULL);

    /* 此处省略代码..... */

    //lwip_init();
#if LWIP_DHCP          /* 使用动态 IP */
    /* 此处省略代码..... */
```

```
#else    /* 使用静态 IP */
    /* 此处省略代码..... */
#endif  /* 向网卡列表中添加一个网口 */

    //netif_init_flag = netif_add(&lwip_netif, (const ip_addr_t *)&ipaddr,
                                (const ip_addr_t *)&netmask,
                                (const ip_addr_t *)&gw, NULL,
                                &ethernetif_init, &ethernet_input);

    netif_init_flag = netif_add(&lwip_netif, (const ip_addr_t *)&ipaddr,
                                (const ip_addr_t *)&netmask,
                                (const ip_addr_t *)&gw, NULL,
                                &ethernetif_init, &tcpip_input);

    /* 此处省略代码..... */

#if LWIP_DHCP                                /* 如果使用 DHCP 的话 */
    /* 此处省略代码..... */
#endif

    return 0;    /* 操作 OK. */
}
```

首先，我们将 `lwip_init` 函数注释掉，并添加 `tcpip_init` 函数。该函数主要用于创建邮箱和 `tcpip` 线程，该线程专门处理数据消息并将其传递给上层。其次，我们修改了 `netif_add` 函数的第七个形参。原本该形参指向的是 `ethernet_input` 函数，现在我们将其修改为指向 `tcpip_input` 函数。最后，我们对 `lwip_pkt_handle` 函数进行了修改。修改内容如下。

```
#include "FreeRTOS.h"
#include "semphr.h"
#include "task.h"

extern xSemaphoreHandle s_xSemaphore; /* 定义一个信号量 */
/**
 * @brief      当接收到数据后调用
 * @param      无
 * @retval     无
 */
void lwip_pkt_handle(void)
{
    BaseType_t xHigherPriorityTaskWoken;
    /* 释放二值信号量 */
    xSemaphoreGiveFromISR(s_xSemaphore, &xHigherPriorityTaskWoken);
    portYIELD_FROM_ISR(xHigherPriorityTaskWoken); /* 进行一次任务切换 */
}
```

当接收到数据时，该函数会释放一个信号量。这个信号量在 `ethernetif.c` 文件中创建，用于实现同步操作。当有数据发送过来时，ETH 中断会释放这个信号量。接收线程会获取这个信号量，并将 RX 缓冲区的数据复制到一个 `pbuf` 中。然后，程序将 `pbuf` 中的数据以邮箱的形式递交给 `tcpip` 线程进行处理。通过这种方式，数据接收和处理的流程得以顺利进行。

3.2.4 修改 `ethernetif.c/h` 文件

`ethernetif.c` 文件是网卡底层驱动文件，负责接收数据包和处理数据包。在 `ethernetif.c` 文件中，我们需要修改两个函数，如下所示。

(1) 修改 `low_level_init` 函数

`ethernetif_init` 函数主要用于初始化底层相关的信息，包括设置 MAC 地址、初始化 ETH 的 DMA 描述符等操作。在 lwIP 操作系统中，接收和协议处理被分为两个线程或任务，接收线程可以在该函数中创建。具体实现细节可能涉及底层硬件操作和网络协议处理，以确保网络通信的正常运行，该函数的源码如下。

```
static void
low_level_init(struct netif *netif)
{
    netif->hwaddr_len = ETHARP_HWADDR_LEN; /*设置 MAC 地址长度,为 6 个字节*/
    /*初始化 MAC 地址,设置什么地址由用户自己设置,但是不能与网络中其他设备 MAC 地址重复*/
    netif->hwaddr[0]=lwipdev.mac[0];
    netif->hwaddr[1]=lwipdev.mac[1];
    netif->hwaddr[2]=lwipdev.mac[2];
    netif->hwaddr[3]=lwipdev.mac[3];
    netif->hwaddr[4]=lwipdev.mac[4];
    netif->hwaddr[5]=lwipdev.mac[5];

    netif->mtu=1500; /*最大允许传输单元,允许该网卡广播和 ARP 功能*/

    /* 网卡状态信息标志位,是很重要的控制字段,它包括网卡功能使能、广播*/
    /* 使能、ARP 使能等等重要控制位*/ /*广播 ARP 协议 链接检测*/
    netif->flags = NETIF_FLAG_BROADCAST|NETIF_FLAG_ETHARP|NETIF_FLAG_LINK_UP;

    /* 创建一个信号量 */
    s_xSemaphore = xSemaphoreCreateBinary();

    /* 创建处理 ETH_MAC 的任务 */
    sys_thread_new("eth_thread",
                   ethernetif_input,          /* 任务入口函数 */
                   netif,                     /* 任务入口函数参数 */
                   NETIF_IN_TASK_STACK_SIZE, /* 任务栈大小 */
                   NETIF_IN_TASK_PRIORITY);   /* 任务的优先级 */

#endif /* LWIP_ARP || LWIP_ETHERNET */
```

```

HAL_ETH_DMATxDscrListInit(&g_eth_handler,g_eth_dma_tx_dscr_tab,
                           g_eth_tx_buf,ETH_TXBUFNB); /* 初始化发送描述符 */
HAL_ETH_DMARxDscrListInit(&g_eth_handler,g_eth_dma_rx_dscr_tab,
                           g_eth_rx_buf,ETH_RXBUFNB); /* 初始化接收描述符 */
HAL_ETH_Start(&g_eth_handler); /* 开启 ETH */

}

```

在 `ethernetif_init` 函数中，与第二章实验中的 `low_level_init` 函数相比，新增了两处内容。第一处是调用 `xSemaphoreCreateBinary` 函数创建信号量，该信号量用于实现同步操作，确保数据接收和处理的正确性。第二处是调用 `sys_thread_new` 函数创建接收任务，该任务函数为 `ethernetif_input` 函数，用于处理接收到的数据包。同时，将任务函数的形参指向 `netif`，以便正确处理网络接口的相关操作。这两处新增的内容使得 `ethernetif_init` 函数能够更好地适应网络通信的需求，提高数据接收和处理的效率和准确性。

(2) 修改 `ethernetif_input` 函数

该函数主要获取底层的数据包并递交给 `tcpip` 线程处理，该函数如下所示：

```

void
ethernetif_input(void *pParams)
{
    struct netif *netif;
    struct pbuf *p = NULL;
    netif = (struct netif *) pParams;
    LWIP_DEBUGF(NETIF_DEBUG, ("ethernetif_input: IP input error\n"));

    while (1)
    {
        if (xSemaphoreTake( s_xSemaphore, portMAX_DELAY ) == pdTRUE)
        {
            /* 将接收到的包移动到新的 pbuf 中 */
            taskENTER_CRITICAL();
REGAIN_PBUF:
            /* 调用 low_level_input 函数接收数据 */
            p = low_level_input(netif);
            taskEXIT_CRITICAL();

            /* 指向包有效负载，它从一个以太网报头开始 */
            if (p != NULL)
            {
                taskENTER_CRITICAL();

                /* 调用 netif 结构体中的 input 字段 (一个函数) 来处理数据包 */
                if (netif->input(p, netif) != ERR_OK)
                {
                    pbuf_free(p);

```

```

        p = NULL;
    }
    else
    {
        xSemaphoreTake( s_xSemaphore, 0);
        goto REGAIN_PBUF;
    }

    taskEXIT_CRITICAL();
}
}
}
}
}

```

ethernetif_input 函数的结构已经发生了变化，不再是传统的函数形式。现在，该函数以任务的形式呈现出来，作为接收线程的一部分。该任务函数首先获取信号量，该信号量是由 ETH 中断释放的。当有数据包进入时，ETH 中断会释放信号量，此时任务函数成功获取信号量。然后，在 low_level_input 函数中获取数据包。最后，数据包通过邮箱的形式，由 netif->input 指针指向的函数递交给 tcpip 线程处理。通过这样的设计，数据包的接收、处理和传递流程更加清晰和高效。信号量的使用确保了数据处理的同步性，避免了可能的竞态条件。这种设计方式使得网络通信更加可靠和高效。

最后打开 ethernetif.h 文件，该文件修改如下：

```

#ifndef __ETHERNETIF_H__
#define __ETHERNETIF_H__
#include "lwip/err.h"
#include "lwip/netif.h"

err_t ethernetif_init(struct netif *netif); /* 网卡初始化函数 */
#endif

```

这里没什么好讲解的，作者是把 void ethernetif_input(struct netif *netif)和 sys_now 函数声明去除了而已。

3.2.5 修改 ethernet.c 文件

在 ETH_IRQHandler 中断服务函数中，我们将 while 语句修改为 if 语句判断。具体来说，我们将原有的 while 循环条件判断改为 if 条件判断，以确保在满足特定条件时执行相应的操作。该函数源码如下所示：

```

void ETH_IRQHandler(void)
{
    if (ethernet_get_eth_rx_size(g_eth_handler.RxDesc))
    {
        lwip_pkt_handle(); /* 处理以太网数据，即将数据提交给 LWIP */
    }
}

```

```

/* 清除 DMA 中断标志位 */
__HAL_ETH_DMA_CLEAR_IT(&g_eth_handler, ETH_DMA_IT_NIS);

/* 清除 DMA 接收中断标志位 */
__HAL_ETH_DMA_CLEAR_IT(&g_eth_handler, ETH_DMA_IT_R);
}

```

在 ETH_IRQHandler 中断服务函数中，对 while 语句进行了修改，改为 if 语句判断。这个函数是在 ethernet.c 文件下的 HAL_ETH_MspInit 函数内被调用的。通过这样的修改，我们可以确保在数据接收时，操作系统能够正确处理中断。至此，修改工作已经完成，编译工程应该不会出现错误。

由于 ETH 中断服务函数会涉及到 FreeRTOS 操作系统信号量相关的函数调用，因此 ETH 中断必须由 FreeRTOS 操作系统管理。为了确保正确的中断处理，我们需要设置 ETH 中断的优先级。在给定的源码中，ETH 中断的优先级被设置为 6。如下源码所示：

```
HAL_NVIC_SetPriority(ETH_IRQn, 6, 0); /* 网络中断优先级应该高一点 */
```

3.2.6 添加应用程序

移植好 lwIP 之后，当然要测试一下移植是否成功。在本步骤中，一共需要修改 1 个文件并添加 4 个文件，修改的 1 个文件为 main.c，添加的 4 个文件为 freertos_demo.c、freertos_demo.h、lwip_demo.c 和 lwip_demo.h。对于 main.c 主要是在 main 函数中完成一些硬件的初始化，最后调用 freertos_demo.c 文件中的 freertos_demo 函数。freertos_demo.c 则是用于编写 FreeRTOS 的相关应用程序代码。而 lwip_demo.c 则是用于编写 lwIP 的相关应用程序代码。

注意：freertos_demo.c、freertos_demo.h 文件保存在 User 文件夹下，而 lwip_demo.c、lwip_demo.h 文件保存在 Middlewares\lwip\lwip_app 文件夹下。

1. main.c 文件

由于正点原子的 STM32 系列开发板众多，这里以正点原子探索者开发板为例为读者进行演示，读者可以根据自己所移植的目标开发板在本教程的配套例程源码的《lwIP 例程 6 lwIP_FreeRTOS 移植》中查看对应的 main.c 文件。正点原子探索者开发板《lwIP 例程 6 lwIP_FreeRTOS 移植》中的 main.c 文件如下所示：

```

/**
 * *****
 * @file      main.c
 * @author    正点原子团队 (ALIENTEK)
 * @version   V1.0
 * @date      2022-6-18
 * @brief     lwIP+FreeRTOS 操作系统移植实验
 * @license   Copyright (c) 2020-2032, 广州市星翼电子科技有限公司
 * *****
 *
 * @attention
 *
 * 实验平台：正点原子 探索者 开发板
 * 在线视频：www.yuanzige.com
 * 技术论坛：http://www.openedv.com/forum.php
 * 公司网址：www.alientek.com

```



```

* 购买地址: zhengdianyuanzi.tmall.com
*
*****
*/

#include "./SYSTEM/sys/sys.h"
#include "./SYSTEM/usart/usart.h"
#include "./SYSTEM/delay/delay.h"
#include "./BSP/LED/led.h"
#include "./BSP/LCD/lcd.h"
#include "./USMART/usmart.h"
#include "./BSP/KEY/key.h"
#include "./MALLOC/malloc.h"
#include "freertos_demo.h"

int main(void)
{
    HAL_Init(); /* 初始化 HAL 库 */
    sys_stm32_clock_init(336, 8, 2, 7); /* 设置时钟, 168Mhz */
    delay_init(168); /* 延时初始化 */
    usart_init(115200); /* 串口初始化为 115200 */
    usmart_dev.init(84); /* 初始化 USMART */
    led_init(); /* 初始化 LED */
    lcd_init(); /* 初始化 LCD */
    key_init(); /* 初始化按键 */

    my_mem_init(SRAMIN); /* 初始化内部 SRAM 内存池 */
    my_mem_init(SRAMEX); /* 初始化外部 SRAM 内存池 */
    my_mem_init(SRAMCCM); /* 初始化内部 SRAMCCM 内存池 */

    freertos_demo(); /* 创建 lwIP 的任务函数 */
}

```

可以看到，在 main.c 文件中只包含了一个 main 函数，main 函数主要就是完成了一些外设的初始化，如串口、LED、LCD、按键等，并在最后调用了函数 freertos_demo。

2. freertos_demo.c 文件

```

/**
*****
* @file      freertos_demo.c
* @author    正点原子团队 (ALIENTEK)
* @version   V1.0
* @date      2022-01-11
* @brief     lwIP+FreeRTOS 操作系统移植实验

```

```

* @license      Copyright (c) 2020-2032, 广州市星翼电子科技有限公司

*****

* @attention
*
* 实验平台:正点原子 探索者 开发板
* 在线视频:www.yuanzige.com
* 技术论坛:www.openedv.com
* 公司网址:www.alientek.com
* 购买地址:openedv.taobao.com
*

*****

*/

#include "freertos_demo.h"
#include "../BSP/LED/led.h"
#include "../BSP/LCD/lcd.h"
#include "../SYSTEM/usart/usart.h"
#include "../SYSTEM/delay/delay.h"
#include "lwip_comm.h"
#include "lwipopts.h"
#include "FreeRTOS.h"
#include "task.h"

/*****
/*FreeRTOS 配置*/

/* START_TASK 任务 配置
* 包括: 任务句柄 任务优先级 堆栈大小 创建任务
*/

#define START_TASK_PRIO          5                /* 任务优先级 */
#define START_STK_SIZE           128              /* 任务堆栈大小 */
TaskHandle_t StartTask_Handler;                  /* 任务句柄 */
void start_task(void *pvParameters);              /* 任务函数 */

/* LWIP_DEMO 任务 配置
* 包括: 任务句柄 任务优先级 堆栈大小 创建任务
*/

#define LWIP_DME0_TASK_PRIO      11                /* 任务优先级 */
#define LWIP_DME0_STK_SIZE       1024             /* 任务堆栈大小 */
TaskHandle_t LWIP_Task_Handler;                  /* 任务句柄 */

```

```

void lwip_demo_task(void *pvParameters);          /* 任务函数 */

/* LED_TASK 任务 配置
 * 包括：任务句柄 任务优先级 堆栈大小 创建任务
 */

#define LED_TASK_PRIO          10                  /* 任务优先级 */
#define LED_STK_SIZE           128                 /* 任务堆栈大小 */
TaskHandle_t LEDTask_Handler;                     /* 任务句柄 */
void led_task(void *pvParameters);                 /* 任务函数 */
/*****

/**
 * @brief      freertos_demo
 * @param      无
 * @retval     无
 */
void freertos_demo(void)
{
    /* lwip_task 任务 */
    xTaskCreate((TaskFunction_t    )start_task,
                (const char *      )"start_task",
                (uint16_t           )START_STK_SIZE,
                (void *             )NULL,
                (UBaseType_t        )START_TASK_PRIO,
                (TaskHandle_t *     )&StartTask_Handler);

    vTaskStartScheduler(); /* 开启任务调度 */
}

/**
 * @brief      start_task
 * @param      pvParameters : 传入参数(未用到)
 * @retval     无
 */
void start_task(void *pvParameters)
{
    pvParameters = pvParameters;

    while(lwip_comm_init() != 0)
    {
        lcd_show_string(30, 110, 200, 16, 16, "lwIP Init failed!!", RED);
    }
}

```

```

        delay_ms(500);
        lcd_fill(30, 50, 200 + 30, 50 + 16, WHITE);
        lcd_show_string(30, 110, 200, 16, 16, "Retrying... ", RED);
        delay_ms(500);
        LED1_TOGGLE();
    }

    while(!ethernet_read_phy(PHY_SR)) /* 检查 MCU 与 PHY 芯片是否通信成功 */
    {
        printf("MCU 与 PHY 芯片通信失败, 请检查电路或者源码!!! \r\n");
    }
    /* 等待 DHCP 获取成功/超时溢出 */
    while((lwipdev.dhcpstatus != 2)&&(lwipdev.dhcpstatus != 0xFF))
    {
        lwip_dhcp_process_handle();
        vTaskDelay(5);
    }

    taskENTER_CRITICAL(); /* 进入临界区 */

    /* 创建 lwIP 任务 */
    xTaskCreate((TaskFunction_t) lwip_demo_task,
                (const char*) "lwip_demo_task",
                (uint16_t) LWIP_DMEM_STK_SIZE,
                (void*) NULL,
                (UBaseType_t) LWIP_DMEM_TASK_PRIO,
                (TaskHandle_t*) &LWIP_Task_Handler);

    /* LED 测试任务 */
    xTaskCreate((TaskFunction_t) led_task,
                (const char*) "led_task",
                (uint16_t) LED_STK_SIZE,
                (void*) NULL,
                (UBaseType_t) LED_TASK_PRIO,
                (TaskHandle_t*) &LEDTask_Handler);

    vTaskDelete(StartTask_Handler); /* 删除开始任务 */
    taskEXIT_CRITICAL(); /* 退出临界区 */
}

/**
 * @brief      lwIP 运行例程

```

```

* @param      pvParameters : 传入参数 (未用到)
* @retval      无
*/
void lwip_demo_task(void *pvParameters)
{
    pvParameters = pvParameters;

    uint8_t t = 0;

    while(1)
    {
        t++;

        if((t%40) == 0)
        {
            LED0_TOGGLE();    /* 翻转一次 LED0 */
        }

        vTaskDelay(5);
    }
}

/**
* @brief      系统再运行
* @param      pvParameters : 传入参数 (未用到)
* @retval      无
*/
void led_task(void *pvParameters)
{
    pvParameters = pvParameters;

    while(1)
    {
        LED1_TOGGLE();
        vTaskDelay(1000);
    }
}

```

对于 freertos_demo.c 文件，这里先简单的介绍一下这个文件的代码结构，这个文件的代码结构可分为 6 个部分，分别是包含头文件、FreeRTOS 相关配置、全局变量及自定义函数、应用程序入口函数、开始任务入口函数、其他任务入口函数，接下来分别地介绍以上这几个部分的代码。

(1) 包含头文件

包含的头文件分成两个部分，分别为 FreeRTOS 头文件、lwIP 头文件和其他头文件，如下

所示：

```
#include "freertos_demo.h"
#include "../BSP/LED/led.h"
#include "../BSP/LCD/lcd.h"
#include "../SYSTEM/usart/usart.h"
#include "../SYSTEM/delay/delay.h"
#include "lwip_comm.h"
#include "lwipopts.h"
#include "FreeRTOS.h"
#include "task.h"
```

(2) FreeRTOS 相关配置

FreeRTOS 的配置主要包括所创建 FreeRTOS 任务的相关定义（任务优先级、任务堆栈大小、任务句柄、任务函数）以及 FreeRTOS 相关变量（信号量、事件、列表、软件定时器等）的定义，如下所示：

```
/* ***** */
/* FreeRTOS 配置 */

/* START_TASK 任务 配置
 * 包括：任务句柄 任务优先级 堆栈大小 创建任务
 */
#define START_TASK_PRIO          5                /* 任务优先级 */
#define START_STK_SIZE           128              /* 任务堆栈大小 */
TaskHandle_t StartTask_Handler;                  /* 任务句柄 */
void start_task(void *pvParameters);              /* 任务函数 */

/* LWIP_DEMO 任务 配置
 * 包括：任务句柄 任务优先级 堆栈大小 创建任务
 */
#define LWIP_DME0_TASK_PRIO      11               /* 任务优先级 */
#define LWIP_DME0_STK_SIZE       1024             /* 任务堆栈大小 */
TaskHandle_t LWIP_Task_Handler;                  /* 任务句柄 */
void lwip_demo_task(void *pvParameters);          /* 任务函数 */

/* LED_TASK 任务 配置
 * 包括：任务句柄 任务优先级 堆栈大小 创建任务
 */
#define LED_TASK_PRIO            10               /* 任务优先级 */
#define LED_STK_SIZE             128              /* 任务堆栈大小 */
TaskHandle_t LEDTask_Handler;                    /* 任务句柄 */
void led_task(void *pvParameters);               /* 任务函数 */
/* ***** */
```

(3) 应用程序入口函数

这部分就是函数 freertos_demo，函数 freertos_demo 一开始就是创建开始任务，最后开启

FreeRTOS 系统任务调度，如下所示：

```
/**
 * @brief      freertos_demo
 * @param      无
 * @retval     无
 */
void freertos_demo(void)
{
    /* lwip_task 任务 */
    xTaskCreate((TaskFunction_t) start_task,
                (const char *) "start_task",
                (uint16_t) START_STK_SIZE,
                (void *) NULL,
                (UBaseType_t) START_TASK_PRIO,
                (TaskHandle_t *) &StartTask_Handler);

    vTaskStartScheduler(); /* 开启任务调度 */
}
```

(4) 开始任务入口函数

这部分就是开始任务的入口函数，开始任务主要用于创建或初始化特定实验中使用到的一些硬件外设、lwIP 初始化和 FreeRTOS 相关的软件（信号量、事件、列表、软件定时器等）以及创建其他用于实验演示的任务，如下所示：

```
/**
 * @brief      start_task
 * @param      pvParameters : 传入参数(未用到)
 * @retval     无
 */
void start_task(void *pvParameters)
{
    pvParameters = pvParameters;

    while(lwip_comm_init() != 0)
    {
        lcd_show_string(30, 110, 200, 16, 16, "lwIP Init failed!!", RED);
        delay_ms(500);
        lcd_fill(30, 50, 200 + 30, 50 + 16, WHITE);
        lcd_show_string(30, 110, 200, 16, 16, "Retrying... ", RED);
        delay_ms(500);
        LED1_TOGGLE();
    }

    while(!ethernet_read_phy(PHY_SR)) /* 检查 MCU 与 PHY 芯片是否通信成功 */
    {
        lcd_show_string(30, 110, 200, 16, 16, "PHY init failed!!", RED);
        delay_ms(500);
        lcd_fill(30, 50, 200 + 30, 50 + 16, WHITE);
        lcd_show_string(30, 110, 200, 16, 16, "Retrying... ", RED);
        delay_ms(500);
        LED1_TOGGLE();
    }
}
```

```

{
    printf("MCU 与 PHY 芯片通信失败, 请检查电路或者源码!!! \r\n");
}

/* 等待 DHCP 获取成功/超时溢出 */
while((lwipdev.dhcpstatus != 2)&&(lwipdev.dhcpstatus != 0xFF))
{
    lwip_dhcp_process_handle();
    vTaskDelay(5);
}

taskENTER_CRITICAL();          /* 进入临界区 */

/* 创建 lwIP 任务 */
xTaskCreate((TaskFunction_t) lwip_demo_task,
            (const char*) "lwip_demo_task",
            (uint16_t) LWIP_DMEO_STK_SIZE,
            (void*) NULL,
            (UBaseType_t) LWIP_DMEO_TASK_PRIO,
            (TaskHandle_t*) &LWIP_Task_Handler);

/* LED 测试任务 */
xTaskCreate((TaskFunction_t) led_task,
            (const char*) "led_task",
            (uint16_t) LED_STK_SIZE,
            (void*) NULL,
            (UBaseType_t) LED_TASK_PRIO,
            (TaskHandle_t*) &LEDTask_Handler);

vTaskDelete(StartTask_Handler); /* 删除开始任务 */
taskEXIT_CRITICAL();           /* 退出临界区 */
}

```

(6) 其他任务入口函数

```

/**
 * @brief      lwIP 运行例程
 * @param      pvParameters : 传入参数(未用到)
 * @retval     无
 */
void lwip_demo_task(void *pvParameters)
{
    pvParameters = pvParameters;

    uint8_t t = 0;

```



```

while(1)
{
    t++;

    if((t%40) == 0)
    {
        LED0_TOGGLE();    /* 翻转一次 LED0 */
    }

    vTaskDelay(5);
}

/**
 * @brief      系统再运行
 * @param      pvParameters : 传入参数(未用到)
 * @retval     无
 */
void led_task(void *pvParameters)
{
    pvParameters = pvParameters;

    while(1)
    {
        LED1_TOGGLE();
        vTaskDelay(1000);
    }
}

```

我们一般在 `lwip_demo_task` 任务函数调用用户编写的函数执行 lwIP 例程，该函数是在 `lwip_demo.c` 文件定义。而 `led_task` 任务函数主要表示系统是否再运行。本教程配套的实验例程 6 以下的例程都会按着这个代码结构来进行实验代码的编写，建议读者先熟悉这个代码结构。

3. freertos_demo.h

```

#ifndef __FREERTOS_DEMO_H
#define __FREERTOS_DEMO_H

void freertos_demo(void);

#endif

```

`freertos_demo.h` 这个文件很简单，就是将函数 `freertos_demo` 导出给其他 C 源文件调用。

4. lwip_demo.c

该文件主要由用户编写的 lwIP 程序，如下所示：

```

/**

```

```
* @brief      lwIP(lwIP 程序入口)
* @param      无
* @retval     无
*/
void lwip_demo(void)
{
    /* 编写 lwIP 代码 */
}
```

5. lwip_demo.h

```
#ifndef _LWIP_DEMO_H
#define _LWIP_DEMO_H
#include "../SYSTEM/sys/sys.h"

void lwip_demo(void); /* lwIP 例程入口 */
#endif
```

lwip_demo.h 这个文件很简单，就是将函数 lwip_demo 导出给其他 C 源文件调用。

至此，我们已经移植完成了，把代码下载到开发板中来验证移植释放成功，打开串口调试助手获取网络信息如下图所示。



图 3.2.6.1 串口调试助手接收到 IP 信息

在 PC 机上按下 win+r 快捷键打开命令行，并在命令行输入“ping192.168.1.37”命令，如下图所示。

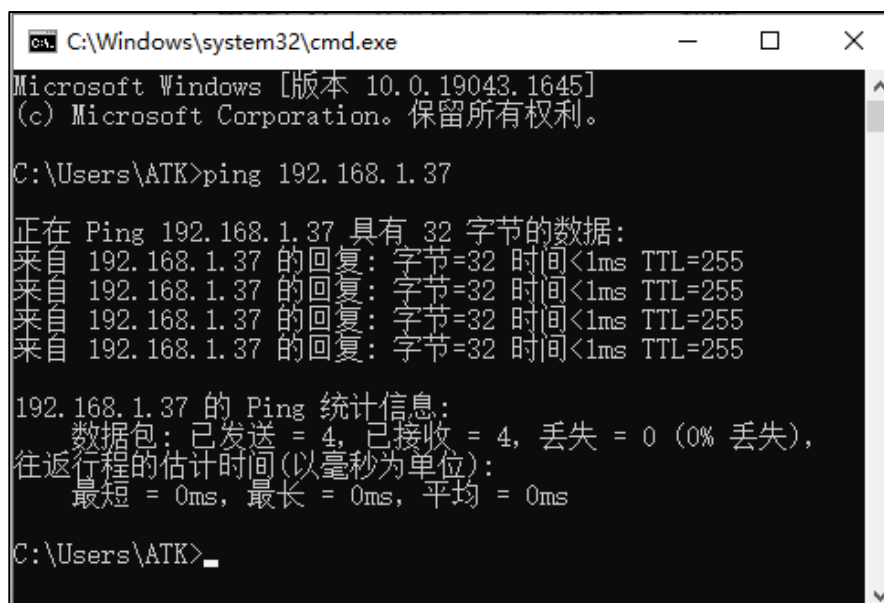


图 3.2.6.2 ping IP 地址成功

该实验的实验工程，请参考《lwIP 例程 6 lwIP_FreeRTOS 移植》。

3.3 lwIP 移植流程（阿波罗、北极星）

3.3.1 修改 lwipopts.h

这个文件是 lwIP 的配置文件，在第二章例程的 lwipopts.h 文件是针对 lwIP 无操作系统下的配置文件，这里我们在 STM32Cube_FW_F4_V1.27.0\Projects\STM32469I_EVAL\Applications\LwIP\LwIP_HTTP_Server_Netconn_RTOS\Inc\路径下的 lwipopts.h 替换本实验的 lwipopts.h 文件，新的 lwipopts.h 文件源码如下所示：

```
#ifndef __LWIPOPTS_H__
#define __LWIPOPTS_H__

/* NO_SYS 表示无操作系统模拟层，无操作系统为 1，有操作系统设置为 0
   注意这个参数设置会编译不同 */
#define NO_SYS 0

/* ----- 内存选项 ----- */
/* 内存对齐，按照 4 字节对齐 */
#define MEM_ALIGNMENT 4

/* 堆内存的大小，如果需要更大的堆内存，那么设置高一点 */
#define MEM_SIZE (10*1024)

/* MEMP_NUM_PBUF: 设置内存池的数量 */
#define MEMP_NUM_PBUF 10
/* MEMP_NUM_UDP_PCB: UDP 协议控制块的数量. */
#define MEMP_NUM_UDP_PCB 6
/* MEMP_NUM_TCP_PCB: TCP 的数量. */
#define MEMP_NUM_TCP_PCB 10
/* MEMP_NUM_TCP_PCB_LISTEN: 监听 TCP 的数量. */
#define MEMP_NUM_TCP_PCB_LISTEN 5
/* MEMP_NUM_TCP_SEG: 同时排队的 TCP 的数量段. */
#define MEMP_NUM_TCP_SEG 8
/* MEMP_NUM_SYS_TIMEOUT: 超时模拟活动的数量. */
#define MEMP_NUM_SYS_TIMEOUT 10

/* ----- Pbuf 选项 ----- */
/* PBUF_POOL 内存池中每个内存块大小 */
#define PBUF_POOL_SIZE 8
/* PBUF_POOL_BUFSIZE: pbuf 池中每个 pbuf 的大小. */
#define PBUF_POOL_BUFSIZE 1524
```

```

/* ----- TCP 选项 ----- */
#define LWIP_TCP 1
#define TCP_TTL 255

/* 控制 TCP 是否应该对到达的段进行排队
   秩序。如果你的设备内存不足，定义为 0. */
#define TCP_QUEUE_OOSEQ 0

/* TCP 最大段大小 */ /* TCP_MSS = (Ethernet MTU - IP header size - TCP header size)
*/
#define TCP_MSS (1500 - 40)

/* TCP 发送者缓冲区空间(字节). */
#define TCP_SND_BUF (4*TCP_MSS)

/* TCP_SND_QUEUELEN: TCP 发送缓冲区空间。这必须是至少
   需要 (2 * TCP_SND_BUF/TCP_MSS) 才能正常工作 */
#define TCP_SND_QUEUELEN (2* TCP_SND_BUF/TCP_MSS)

/* TCP 接收窗口 */
#define TCP_WND (2*TCP_MSS)

/* ----- ICMP 选项 ----- */
#define LWIP_ICMP 1

/* ----- DHCP 选项 ----- */
/* 如果您希望 DHCP 配置为，请将 LWIP_DHCP 定义为 1 */
#define LWIP_DHCP 1

/* ----- UDP 选项 ----- */
#define LWIP_UDP 1
#define UDP_TTL 255

/* ----- Statistics 选项 ----- */
#define LWIP_STATS 0

/* ----- 链接回调选项 ----- */

```

```

/* WIP_NETIF_LINK_CALLBACK==1:支持来自接口的回调函数
   每当链接改变(例如, 向下链接)
   */
#define LWIP_NETIF_LINK_CALLBACK 1
/*
   -----
   ----- 帧校验和选项 -----
   -----
   */

/*
The STM32F4x7 allows computing and verifying the IP, UDP, TCP and ICMP
checksums by hardware:
- To use this feature let the following define uncommented.
- To disable it and process by CPU comment the the checksum.
*/
#define CHECKSUM_BY_HARDWARE

#ifdef CHECKSUM_BY_HARDWARE
#define CHECKSUM_GEN_IP 0
#define CHECKSUM_GEN_UDP 0
#define CHECKSUM_GEN_TCP 0
#define CHECKSUM_CHECK_IP 0
#define CHECKSUM_CHECK_UDP 0
#define CHECKSUM_CHECK_TCP 0
#define CHECKSUM_GEN_ICMP 0
#else
#define CHECKSUM_GEN_IP 1
#define CHECKSUM_GEN_UDP 1
#define CHECKSUM_GEN_TCP 1
#define CHECKSUM_CHECK_IP 1
#define CHECKSUM_CHECK_UDP 1
#define CHECKSUM_CHECK_TCP 1
#define CHECKSUM_GEN_ICMP 1
#endif

/*
   -----
   ----- 连续层的选择 -----
   -----
   */

```

```

/**
 * LWIP_NETCONN==1: 启用 Netconn API (需要使用 api_lib.c)
 */
#define LWIP_NETCONN                1

/**
 * ----- Socket 选项 -----
 */
/**
 * LWIP_SOCKET==1: 启用 Socket API (要求使用 Socket .c)
 */
#define LWIP_SOCKET                  0

/**
 * ----- httpd options -----
 */
/** Set this to 1 to include "fsdata_custom.c" instead of "fsdata.c" for the
 * file system (to prevent changing the file included in CVS) */
#define HTTPD_USE_CUSTOM_FSDATA      1
/**
 * ----- 操作系统选项 -----
 */

#define TCPIP_THREAD_NAME             "TCP/IP"
#define TCPIP_THREAD_STACKSIZE        1000
#define TCPIP_MBOX_SIZE               6
#define DEFAULT_UDP_RECVMBOX_SIZE     6
#define DEFAULT_TCP_RECVMBOX_SIZE     6
#define DEFAULT_ACCEPTMBOX_SIZE       6
#define DEFAULT_THREAD_STACKSIZE      500
#define TCPIP_THREAD_PRIO              5

#endif /* __LWIPOPTS_H__ */

```

笔者对新的 lwipopts.h 文件做了中文注释以及配置项的数值修改，例如修改了 TCPIP_THREAD_PRIO 配置项为 5，这个配置项是设置协议栈任务的优先级，当然大家也可以使用正点原子《lwIP 例程 5 lwIP-FreeRTOS 移植》例程中的 lwipopts.h 文件，该文件的内容

是参考上述的源码改编得来的。

3.3.2 添加 sys_arch.c/h 文件

如果 lwIP 使用操作系统的话，sys_arch.c/h 这两个文件是有必要了解的，这些文件是 lwIP 内核与操作系统交互的接口文件，也可以说，它们是 lwIP 与操作系统的交接文件，本章的实验是使用 FreeRTOS 操作系统，显然 sys_arch.c 文件包含的邮箱、信号量以及互斥锁等 IPC 策略都是由 FreeRTOS 操作系统提供的，注意：FreeRTOS 操作系统没有邮箱的概念，可使用消息队列替代邮箱机制。

打开“contrib-2.1.0\ports\freertos\路径下的文件夹，该文件夹包含了与 FreeRTOS 相关的 sys_arch.c/h 文件，笔者把这两个文件复制到本章实验的 Middlewares\lwip\arch 路径下，该文件夹结构如下图所示。

lwIP例程5 lwIP_FreeRTOS移植 > Middlewares > lwip > arch		
名称	类型	大小
cc.h	C++ Header file	3 KB
ethernetif.c	C 文件	15 KB
ethernetif.h	C++ Header file	2 KB
lwip_comm.c	C 文件	11 KB
lwip_comm.h	C++ Header file	3 KB
lwipopts.h	C++ Header file	8 KB
sys_arch.c	C 文件	18 KB
sys_arch.h	C++ Header file	4 KB

图 3.3.2.1 arch 文件夹的文件结构

打开工程并在 Middlewares/lwip/arch 分组下添加 sys_arch.c 文件，如下图所示。

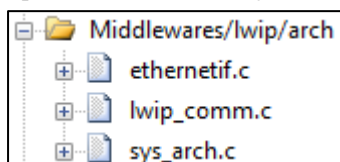


图 3.3.2.2 添加 sys_arch.c 文件

至此，我们编译工程会发现有很多的错误，这些错误都是指向 err.c 文件中的 err_to_errno_table 数组如下图所示。

```
error: #20: identifier "EINPROGRESS" is undefined
    Operation in progress    */
error: #20: identifier "EINVAL" is undefined
    Illegal value.          */
error: #20: identifier "EADDRINUSE" is undefined
    Address in use.         */
error: #20: identifier "EALREADY" is undefined
```

图 3.3.2.3 err.c 文件错误

这些错误的解决方法是在 lwipopts.h 文件添加 LWIP_PROVIDE_ERRNO 配置项并且把它设置为 1。打开 cc.h 文件，在此文件把“typedef int sys_prot_t;”代码注释掉，因为这个 sys_prot_t 变量已经在 sys_arch.c 文件中定义了，这里无需重复定义。

3.3.3 修改 lwip_comm.c/h

该文件主要修改网络配置的信息，比如默认 IP 的配置、lwIP 的初始化以及 DHCP 等处理，下面我们对 lwip_comm.c 文件简单修改，如下所示：

(1) FreeRTOS 相关配置

```
/* LINK 线程配置 */
#define LWIP_LINK_TASK_PRIO          1                /* 任务优先级 */
#define LWIP_LINK_STK_SIZE           128 * 2          /* 任务堆栈大小 */
void lwip_link_thread( void * argument );             /* 链路线程 */

/* DHCP 线程配置 */
#define LWIP_DHCP_TASK_PRIO          0                /* 任务优先级 */
#define LWIP_DHCP_STK_SIZE           128 * 2          /* 任务堆栈大小 */
void lwip_periodic_handle(void *argument);            /* DHCP 线程 */
```

声明 LINK 任务和 DHCP 任务配置，它们分别处理网口状态和 DHCP 分配。

(2) 删除 lwip_periodic_handle、lwip_dhcp_process 和 lwip_pkt_handle 函数。

(3) 修改 lwip_comm_init 函数，该函数如下所示：

```
/**
 * @breif      LWIP 初始化(LWIP 启动的时候使用)
 * @param      无
 * @retval     0,成功
 *             1,内存错误
 *             2,以太网芯片初始化失败
 *             3,网卡添加失败.
 */
uint8_t lwip_comm_init(void)
{
    /* 此处省略代码..... */

    tcpip_init(NULL, NULL);

    /* 此处省略代码..... */

    //lwip_init();
#if LWIP_DHCP                /* 使用动态 IP */
    /* 此处省略代码..... */
#else    /* 使用静态 IP */
    /* 此处省略代码..... */
#endif    /* 向网卡列表中添加一个网口 */

    //netif_init_flag = netif_add(&lwip_netif, (const ip_addr_t *)&ipaddr,
                                (const ip_addr_t *)&netmask,
                                (const ip_addr_t *)&gw, NULL,
                                &ethernetif_init, &ethernet_input);
```



```

netif_init_flag = netif_add(&lwip_netif, (const ip_addr_t *)&ipaddr,
                            (const ip_addr_t *)&netmask,
                            (const ip_addr_t *)&gw, NULL,
                            &ethernetif_init, &tcpip_input);

if (netif_init_flag == NULL)
{
    return 1;          /* 网卡添加失败 */
}
else                  /* 网口添加成功后,设置 netif 为默认值,并且打开 netif 网口 */
{
    netif_set_default(&g_lwip_netif);          /* 设置 netif 为默认网口 */

#ifdef LWIP_NETIF_LINK_CALLBACK
    lwip_link_status_updated(&g_lwip_netif);    /* DHCP 链接状态更新函数 */
    netif_set_link_callback(&g_lwip_netif, lwip_link_status_updated);
    /* 查询 PHY 连接状态任务 */
    sys_thread_new("eth_link",
                   lwip_link_thread,          /* 任务入口函数 */
                   &g_lwip_netif,          /* 任务入口函数参数 */
                   LWIP_LINK_STK_SIZE,      /* 任务栈大小 */
                   LWIP_LINK_TASK_PRIO);    /* 任务的优先级 */
#endif

    }

    g_lwipdev.link_status = LWIP_LINK_OFF;          /* 链接标记为 0 */
#ifdef LWIP_DHCP
    /* 如果使用 DHCP 的话 */
    g_lwipdev.dhcpstatus = 0;          /* DHCP 标记为 0 */
    /* DHCP 轮询任务 */
    sys_thread_new("eth_dhcp",
                   lwip_periodic_handle,      /* 任务入口函数 */
                   &g_lwip_netif,          /* 任务入口函数参数 */
                   LWIP_DHCP_STK_SIZE,      /* 任务栈大小 */
                   LWIP_DHCP_TASK_PRIO);    /* 任务的优先级 */
#endif

    return 0;          /* 操作 OK. */
}

```

此函数主要修改三处地方，首先我们把 `lwip_init` 函数注释掉并添加 `tcpip_init` 函数，该函数主要创建邮箱以及 `tcpip` 线程，它专门处理数据消息并把它往上层递交，接着修改 `netif_add` 函数的第七个形参，该形参不再指向 `ethernet_input` 函数而是指向 `tcpip_input` 函数，最后创建 `eth_link` 和 `eth_dhcp` 任务，它们分别处理网口状态和 DHCP 分配。

打开 lwip_comm.h 文件，把它的内容修改为以下源码：

```
#ifndef _LWIP_COMM_H
#define _LWIP_COMM_H

#include "../BSP/ETHERNET/ethernet.h"
#include "netif/etharp.h"
#include "lwip/timeouts.h"
#include "lwip/snmp.h"

/* DHCP 进程状态 */
#define LWIP_DHCP_OFF (uint8_t) 0 /* DHCP 服务器关闭状态 */
#define LWIP_DHCP_START (uint8_t) 1 /* DHCP 服务器启动状态 */
#define LWIP_DHCP_WAIT_ADDRESS (uint8_t) 2 /* DHCP 服务器等待分配 IP 状态 */
#define LWIP_DHCP_ADDRESS_ASSIGNED (uint8_t) 3 /* DHCP 服务器地址已分配状态 */
#define LWIP_DHCP_TIMEOUT (uint8_t) 4 /* DHCP 服务器超时状态 */
#define LWIP_DHCP_LINK_DOWN (uint8_t) 5 /* DHCP 服务器链接失败状态 */

/* 链接状态 */
#define LWIP_LINK_OFF (uint8_t) 0 /* 链接关闭状态 */
#define LWIP_LINK_ON (uint8_t) 1 /* 链接开启状态 */
#define LWIP_LINK_AGAIN (uint8_t) 2 /* 重复开启 */

/* DHCP 服务器最大重试次数 */
#define LWIP_MAX_DHCP_TRIES (uint8_t) 4

typedef void (*display_fn)(uint8_t index);

/*lwip 控制结构体*/
typedef struct
{
    uint8_t mac[6]; /* MAC 地址 */
    uint8_t remoteip[4]; /* 远端主机 IP 地址 */
    uint8_t ip[4]; /* 本机 IP 地址 */
    uint8_t netmask[4]; /* 子网掩码 */
    uint8_t gateway[4]; /* 默认网关的 IP 地址 */
    uint8_t dhcpstatus; /* dhcp 状态
        0, 未获取 DHCP 地址
        1, 进入 DHCP 获取状态
        2, 成功获取 DHCP 地址
        0xFF, 获取失败 */
    uint8_t link_status; /* 连接状态 */
    display_fn lwip_display_fn; /* 显示函数指针 */
}__lwip_dev;
```

```
extern __lwip_dev g_lwipdev; /* lwip 控制结构体 */

void    lwip_comm_default_ip_set(__lwip_dev *lwipx); /* lwip 默认 IP 设置 */
uint8_t lwip_comm_init(void); /* LWIP 初始化 (LWIP 启动的时候使用) */
#endif
```

3.3.4 修改 ethernetif.c/h 文件

这些文件是网卡底层驱动文件，它要负责接收数据包和处理数据包，该文件修改的地方非常多，这里笔者不做讲解，读者可参考正点原子编写好的 ethernetif.c/h 或者 STM32Cube_FW_F4_V1.27.0\Projects\STM32469I_EVAL\Applications\LwIP\LwIP_HTTP_Server_Netconn_RTOS 工程下的 ethernetif.c/h 文件。

3.3.5 修改 ethernet.c 文件

这个文件我们只修改 HAL_ETH_MspInit 函数，在此函数下添加两段代码，如下所示：

```
/* Enable the Ethernet global Interrupt */
HAL_NVIC_SetPriority(ETH_IRQn, 0x07, 0);
HAL_NVIC_EnableIRQ(ETH_IRQn);
```

注意：以太网中断优先级必须归 FreeRTOS 内核管理，具体原因请看正点原子《FreeRTOS 开发指南》教程。

3.3.6 添加应用程序

移植好 lwIP 之后，当然要测试一下移植是否成功。在本步骤中，一共需要修改 1 个文件并添加 4 个文件，修改的 1 个文件为 main.c，添加的 4 个文件为 freertos_demo.c、freertos_demo.h、lwip_demo.c 和 lwip_demo.h。对于 main.c 主要是在 main 函数中完成一些硬件的初始化，最后调用 freertos_demo.c 文件中的 freertos_demo 函数。freertos_demo.c 则是用于编写 FreeRTOS 的相关应用程序代码。而 lwip_demo.c 则是用于编写 lwIP 的相关应用程序代码。

注意：freertos_demo.c、freertos_demo.h 文件保存在 User 文件夹下，而 lwip_demo.c、lwip_demo.h 文件保存在 Middlewares\lwip\lwip_app 文件夹下。

1. main.c 文件

由于正点原子的 STM32 系列开发板众多，这里以正点原子阿波罗 429 开发板为例为读者进行演示，读者可以根据自己所移植的目标开发板在本教程的配套例程源码的《lwIP 例程 6 lwIP_FreeRTOS 移植》中查看对应的 main.c 文件。正点原子阿波罗 429 开发板《lwIP 例程 6 lwIP_FreeRTOS 移植》中的 main.c 文件如下所示：

```
/**
*****
* @file      main.c
* @author    正点原子团队 (ALIENTEK)
* @version   V1.0
* @date      2022-8-01
* @brief     lwIP+FreeRTOS 操作系统移植实验
* @license   Copyright (c) 2020-2032, 广州市星翼电子科技有限公司
*****
**/
```

```

* @attention
*
* 实验平台:正点原子 阿波罗 F429 开发板
* 在线视频:www.yuanzige.com
* 技术论坛:www.openedv.com
* 公司网址:www.alientek.com
* 购买地址:openedv.taobao.com
*
*****
*/

#include "./SYSTEM/sys/sys.h"
#include "./SYSTEM/usart/usart.h"
#include "./SYSTEM/delay/delay.h"
#include "./BSP/LED/led.h"
#include "./BSP/LCD/lcd.h"
#include "./BSP/KEY/key.h"
#include "./USMART/usmart.h"
#include "./BSP/SDRAM/sdram.h"
#include "./MALLOC/malloc.h"
#include "./BSP/PCF8574/pcf8574.h"
#include "lwip_comm.h"
#include "lwipopts.h"
#include "freertos_demo.h"

int main(void)
{
    HAL_Init(); /* 初始化 HAL 库 */
    sys_stm32_clock_init(360, 25, 2, 8); /* 设置时钟,180Mhz */
    delay_init(180); /* 延时初始化 */
    usart_init(115200); /* 串口初始化为 115200 */
    usmart_dev.init(90); /* 初始化 USMART */
    led_init(); /* 初始化 LED */
    key_init(); /* 初始化按键 */
    sdram_init(); /* SRAM 初始化 */
    lcd_init(); /* 初始化 LCD */

    while (pcf8574_init()) /* 检测不到 PCF8574 */
    {
        lcd_show_string(30, 170, 200, 16, 16, "PCF8574 Check Failed!", RED);
        delay_ms(500);
        lcd_show_string(30, 170, 200, 16, 16, "Please Check! ", RED);
    }
}

```

```

        delay_ms(500);
        LED0_TOGGLE();      /* 红灯闪烁 */
    }

    my_mem_init(SRAMIN);      /* 初始化内部内存池 */
    my_mem_init(SRAMEX);      /* 初始化外部内存池 */
    my_mem_init(SRAMCCM);     /* 初始化 CCM 内存池 */

    freertos_demo();
}

```

可以看到，在 main.c 文件中只包含了一个 main 函数，main 函数主要就是完成了一些外设的初始化，如串口、LED、LCD、按键等，并在最后调用了函数 freertos_demo。

2. freertos_demo.c 文件

```

/**
*****
* @file      freertos_demo.c
* @author    正点原子团队 (ALIENTEK)
* @version   V1.0
* @date      2022-8-01
* @brief     lwIP+FreeRTOS 操作系统移植实验
* @license   Copyright (c) 2020-2032, 广州市星翼电子科技有限公司
*****
* @attention
*
* 实验平台:正点原子 阿波罗 F429 开发板
* 在线视频:www.yuanzige.com
* 技术论坛:www.openedv.com
* 公司网址:www.alientek.com
* 购买地址:openedv.taobao.com
*
*****
*/

#include "freertos_demo.h"
#include "../BSP/LED/led.h"
#include "../BSP/LCD/lcd.h"
#include "../SYSTEM/usart/usart.h"
#include "../SYSTEM/delay/delay.h"
#include "../BSP/PCF8574/pcf8574.h"
#include "lwip_comm.h"
#include "lwipopts.h"
#include "FreeRTOS.h"
#include "task.h"

```

```

/*****
/*FreeRTOS 配置*/

/* START_TASK 任务 配置
 * 包括：任务句柄 任务优先级 堆栈大小 创建任务
 */

#define START_TASK_PRIO          1                /* 任务优先级 */
#define START_STK_SIZE           256              /* 任务堆栈大小 */
TaskHandle_t StartTask_Handler;                  /* 任务句柄 */
void start_task(void *pvParameters);              /* 任务函数 */

/* LWIP_DEMO 任务 配置
 * 包括：任务句柄 任务优先级 堆栈大小 创建任务
 */

#define LWIP_DME0_TASK_PRIO      11               /* 任务优先级 */
#define LWIP_DME0_STK_SIZE       1024             /* 任务堆栈大小 */
TaskHandle_t LWIP_Task_Handler;                  /* 任务句柄 */
void lwip_demo_task(void *pvParameters);          /* 任务函数 */

/* LED_TASK 任务 配置
 * 包括：任务句柄 任务优先级 堆栈大小 创建任务
 */

#define LED_TASK_PRIO            10               /* 任务优先级 */
#define LED_STK_SIZE             128              /* 任务堆栈大小 */
TaskHandle_t LEDTask_Handler;                    /* 任务句柄 */
void led_task(void *pvParameters);               /* 任务函数 */
*****/

/**
 * @brief      加载 UI
 * @param      mode :   bit0:0,不加载;1,加载前半部分 UI
 *                bit1:0,不加载;1,加载后半部分 UI
 * @retval     无
 */
void lwip_test_ui(uint8_t mode)
{
    uint8_t speed;
    uint8_t buf[30];

    if (mode & 1 << 0)

```

```
{  
    lcd_fill(5, 30, lcddev.width, 110, WHITE);  
    lcd_show_string(5, 30, 200, 16, 16, "STM32", RED);  
    lcd_show_string(5, 50, 200, 16, 16, "lwIP ping Test", RED);  
    lcd_show_string(5, 70, 200, 16, 16, "ATOM@ALIENTEK", RED);  
}  
  
if (mode & 1 << 1)  
{  
    lcd_fill(5, 110, lcddev.width, lcddev.height, WHITE);  
    lcd_show_string(5, 110, 200, 16, 16, "lwIP Init Succeeded", BLUE);  
  
    if (g_lwipdev.dhcpstatus == 2)  
    {  
        sprintf((char*)buf, "DHCP IP:%d.%d.%d.%d",  
                g_lwipdev.ip[0],  
                g_lwipdev.ip[1],  
                g_lwipdev.ip[2],  
                g_lwipdev.ip[3]);    /* 显示动态 IP 地址 */  
    }  
    else  
    {  
        sprintf((char*)buf, "Static IP:%d.%d.%d.%d",  
                g_lwipdev.ip[0],  
                g_lwipdev.ip[1],  
                g_lwipdev.ip[2],  
                g_lwipdev.ip[3]);    /* 打印静态 IP 地址 */  
    }  
  
    lcd_show_string(5, 130, 200, 16, 16, (char*)buf, BLUE);  
  
    speed = ethernet_chip_get_speed(); /* 得到网速 */  
  
    if (speed)  
    {  
        lcd_show_string(5, 150, 200, 16, 16, "Ethernet Speed:100M", BLUE);  
    }  
    else  
    {  
        lcd_show_string(5, 150, 200, 16, 16, "Ethernet Speed:10M", BLUE);  
    }  
}  
}
```

```

/**
 * @brief      freertos_demo
 * @param      无
 * @retval     无
 */
void freertos_demo(void)
{
    /* start_task 任务 */
    xTaskCreate((TaskFunction_t      )start_task,
                (const char *        )"start_task",
                (uint16_t             )START_STK_SIZE,
                (void *               )NULL,
                (UBaseType_t          )START_TASK_PRIO,
                (TaskHandle_t *       )&StartTask_Handler);

    vTaskStartScheduler(); /* 开启任务调度 */
}

/**
 * @brief      start_task
 * @param      pvParameters : 传入参数(未用到)
 * @retval     无
 */
void start_task(void *pvParameters)
{
    pvParameters = pvParameters;

    g_lwipdev.lwip_display_fn = lwip_test_ui;

    lwip_test_ui(1); /* 加载后前部分 UI */

    while (lwip_comm_init() != 0)
    {
        lcd_show_string(30, 110, 200, 16, 16, "lwIP Init failed!!", RED);
        delay_ms(500);
        lcd_fill(30, 50, 200 + 30, 50 + 16, WHITE);
        lcd_show_string(30, 110, 200, 16, 16, "Retrying...      ", RED);
        delay_ms(500);
        LED1_TOGGLE();
    }

    /* 等待静态和动态分配完成 */
    while (g_lwipdev.dhcpstatus != 2 && g_lwipdev.dhcpstatus != 0xff)

```



```

{
    vTaskDelay(500);
}

taskENTER_CRITICAL();          /* 进入临界区 */

/* 创建 lwIP 任务 */
xTaskCreate((TaskFunction_t    )lwip_demo_task,
            (const char*       )"lwip_demo_task",
            (uint16_t          )LWIP_DMEM_STK_SIZE,
            (void*              )NULL,
            (UBaseType_t       )LWIP_DMEM_TASK_PRIO,
            (TaskHandle_t*)    )&lwip_demo_handler);

/* LED 测试任务 */
xTaskCreate((TaskFunction_t    )led_task,
            (const char*       )"led_task",
            (uint16_t          )LED_STK_SIZE,
            (void*              )NULL,
            (UBaseType_t       )LED_TASK_PRIO,
            (TaskHandle_t*)    )&led_task_handler);

vTaskDelete(StartTask_Handler); /* 删除开始任务 */
taskEXIT_CRITICAL();          /* 退出临界区 */
}

/**
 * @brief      lwIP 运行例程
 * @param      pvParameters : 传入参数 (未用到)
 * @retval     无
 */
void lwip_demo_task(void *pvParameters)
{
    pvParameters = pvParameters;

    uint8_t t = 0;

    while (1)
    {
        t++;

        if ((t % 40) == 0)
        {

```

```

        LED0_TOGGLE();    /* 翻转一次 LED0 */
    }

    vTaskDelay(5);
}
}

/**
 * @brief      系统再运行
 * @param      pvParameters : 传入参数(未用到)
 * @retval     无
 */
void led_task(void *pvParameters)
{
    pvParameters = pvParameters;

    while (1)
    {
        LED1_TOGGLE();
        vTaskDelay(1000);
    }
}

```

对于 freertos_demo.c 文件，这里先简单的介绍一下这个文件的代码结构，这个文件的代码结构可分为 6 个部分，分别是包含头文件、FreeRTOS 相关配置、全局变量及自定义函数、应用程序入口函数、开始任务入口函数、其他任务入口函数，接下来分别地介绍以上这几个部分的代码。

(1) 包含头文件

包含的头文件分成两个部分，分别为 FreeRTOS 头文件、lwIP 头文件和其他头文件，如下所示：

```

#include "freertos_demo.h"
#include "../BSP/LED/led.h"
#include "../BSP/LCD/lcd.h"
#include "../SYSTEM/usart/usart.h"
#include "../SYSTEM/delay/delay.h"
#include "../BSP/PCF8574/pcf8574.h"
#include "lwip_comm.h"
#include "lwipopts.h"
#include "FreeRTOS.h"
#include "task.h"

```

(2) FreeRTOS 相关配置

FreeRTOS 的配置主要包括所创建 FreeRTOS 任务的相关定义（任务优先级、任务堆栈大小、任务句柄、任务函数）以及 FreeRTOS 相关变量（信号量、事件、列表、软件定时器等）的定义，如下所示：

```

/*****
/*FreeRTOS 配置*/

/* START_TASK 任务 配置
 * 包括：任务句柄 任务优先级 堆栈大小 创建任务
 */
#define START_TASK_PRIO          1                /* 任务优先级 */
#define START_STK_SIZE           256              /* 任务堆栈大小 */
TaskHandle_t StartTask_Handler;                  /* 任务句柄 */
void start_task(void *pvParameters);              /* 任务函数 */

/* LWIP_DEMO 任务 配置
 * 包括：任务句柄 任务优先级 堆栈大小 创建任务
 */
#define LWIP_DME0_TASK_PRIO      11               /* 任务优先级 */
#define LWIP_DME0_STK_SIZE       1024             /* 任务堆栈大小 */
TaskHandle_t LWIP_Task_Handler;                  /* 任务句柄 */
void lwip_demo_task(void *pvParameters);          /* 任务函数 */

/* LED_TASK 任务 配置
 * 包括：任务句柄 任务优先级 堆栈大小 创建任务
 */
#define LED_TASK_PRIO            10               /* 任务优先级 */
#define LED_STK_SIZE             128              /* 任务堆栈大小 */
TaskHandle_t LEDTask_Handler;                    /* 任务句柄 */
void led_task(void *pvParameters);               /* 任务函数 */
*****/
    
```

(3) 应用程序入口函数

这部分就是函数 `freertos_demo`，函数 `freertos_demo` 一开始就是创建开始任务，最后开启 FreeRTOS 系统任务调度，如下所示：

```

/**
 * @brief      freertos_demo
 * @param      无
 * @retval     无
 */
void freertos_demo(void)
{
    /* lwip_task 任务 */
    xTaskCreate((TaskFunction_t) start_task,
                (const char *) "start_task",
                (uint16_t) START_STK_SIZE,
                (void *) NULL,
                (UBaseType_t) START_TASK_PRIO,
    
```

```

        (TaskHandle_t *      ) &StartTask_Handler);

vTaskStartScheduler(); /* 开启任务调度 */
}

```

(4) 开始任务入口函数

这部分就是开始任务的入口函数，开始任务主要用于创建或初始化特定实验中使用到的一些硬件外设、lwIP 初始化和 FreeRTOS 相关的软件（信号量、事件、列表、软件定时器等）以及创建其他用于实验演示的任务，如下所示：

```

/**
 * @brief      start_task
 * @param      pvParameters : 传入参数(未用到)
 * @retval     无
 */
void start_task(void *pvParameters)
{
    pvParameters = pvParameters;

    g_lwipdev.lwip_display_fn = lwip_test_ui;

    lwip_test_ui(1); /* 加载后前部分 UI */

    while (lwip_comm_init() != 0)
    {
        lcd_show_string(30, 110, 200, 16, 16, "lwIP Init failed!!", RED);
        delay_ms(500);
        lcd_fill(30, 50, 200 + 30, 50 + 16, WHITE);
        lcd_show_string(30, 110, 200, 16, 16, "Retrying...      ", RED);
        delay_ms(500);
        LED1_TOGGLE();
    }
    /* 等待静态和动态分配完成 */
    while (g_lwipdev.dhcpstatus != 2 && g_lwipdev.dhcpstatus != 0xff)
    {
        vTaskDelay(500);
    }

    taskENTER_CRITICAL(); /* 进入临界区 */

    /* 创建 lwIP 任务 */
    xTaskCreate((TaskFunction_t      ) lwip_demo_task,
                (const char*         ) "lwip_demo_task",
                (uint16_t             ) LWIP_DMEM_STK_SIZE,
                (void*                ) NULL,
                (UBaseType_t          ) LWIP_DMEM_TASK_PRIO,

```

```

        (TaskHandle_t*          ) &LWIP_Task_Handler);

/* LED 测试任务 */
xTaskCreate((TaskFunction_t      ) led_task,
            (const char*         ) "led_task",
            (uint16_t             ) LED_STK_SIZE,
            (void*                ) NULL,
            (UBaseType_t          ) LED_TASK_PRIO,
            (TaskHandle_t*        ) &LEDTask_Handler);

vTaskDelete(StartTask_Handler); /* 删除开始任务 */
taskEXIT_CRITICAL();             /* 退出临界区 */
}

```

(6) 其他任务入口函数

```

/**
 * @brief      lwIP 运行例程
 * @param      pvParameters : 传入参数(未用到)
 * @retval     无
 */
void lwip_demo_task(void *pvParameters)
{
    pvParameters = pvParameters;
    uint8_t t = 0;

    while (1)
    {
        t++;

        if ((t % 40) == 0)
        {
            LED0_TOGGLE(); /* 翻转一次 LED0 */
        }

        vTaskDelay(5);
    }
}

/**
 * @brief      系统再运行
 * @param      pvParameters : 传入参数(未用到)
 * @retval     无
 */
void led_task(void *pvParameters)

```

```
{
    pvParameters = pvParameters;
    while (1)
    {
        LED1_TOGGLE();
        vTaskDelay(1000);
    }
}
```

我们一般在 `lwip_demo_task` 任务函数调用用户编写的函数执行 lwIP 例程，该函数是在 `lwip_demo.c` 文件定义。而 `led_task` 任务函数主要表示系统是否再运行。本教程配套的实验例程 6 以下的例程都会按着这个代码结构来进行实验代码的编写，建议读者先熟悉这个代码结构。

3. freertos_demo.h

```
#ifndef __FREERTOS_DEMO_H
#define __FREERTOS_DEMO_H

void freertos_demo(void);

#endif
```

`freertos_demo.h` 这个文件很简单，就是将函数 `freertos_demo` 导出给其他 C 源文件调用。

4. lwip_demo.c

该文件主要由用户编写的 lwIP 程序，如下所示：

```
/**
 * @brief      lwIP(lwIP 程序入口)
 * @param      无
 * @retval     无
 */
void lwip_demo(void)
{
    /* 编写 lwIP 代码 */
}
```

5. lwip_demo.h

```
#ifndef _LWIP_DEMO_H
#define _LWIP_DEMO_H
#include "../SYSTEM/sys/sys.h"

void lwip_demo(void); /* lwIP 例程入口 */
#endif
```

`lwip_demo.h` 这个文件很简单，就是将函数 `lwip_demo` 导出给其他 C 源文件调用。

至此，我们已经移植完成了，把代码下载到开发板中来验证移植释放成功，打开串口调试助手获取网络信息如下图所示。

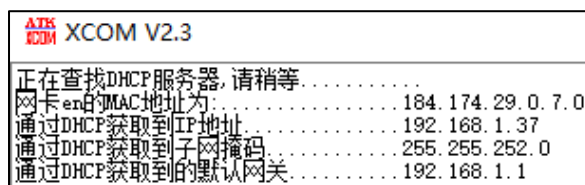


图 3.3.6.1 串口调试助手接收到 IP 信息

在 PC 机上按下 win+r 快捷键打开命令行，并在命令行输入“ping192.168.1.37”命令，如下图所示。

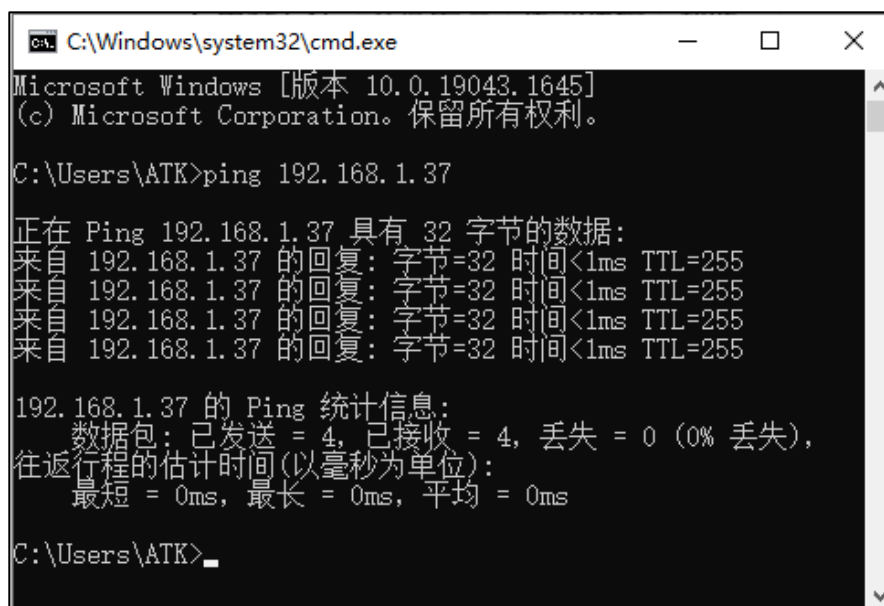


图 3.3.6.2 ping IP 地址成功

该实验的实验工程，请参考《lwIP 例程 6 lwIP_FreeRTOS 移植》。

3.4 lwIP 移植流程（H7R7、H7R3）

3.4.1 修改 lwipopts.h

这个文件是 lwIP 的配置文件，在第二章例程的 lwipopts.h 文件是针对 lwIP 无操作系统下的配置文件，这里我们在 STM32Cube_FW_H7RS_V1.0.0\Projects\NUCLEO-H7S3L8\Applications\LwIP\LwIP_HTTP_Server_Netconn_RTOS\Inc\路径下的 lwipopts.h 替换本实验的 lwipopts.h 文件，新的 lwipopts.h 文件源码如下所示：

```
#ifndef __LWIPOPTS_H__
#define __LWIPOPTS_H__

/* NO_SYS 表示无操作系统模拟层，无操作系统为 1，有操作系统设置为 0
   注意这个参数设置会编译不同 */
#define NO_SYS 0

/* ----- 内存选项 ----- */
/* 内存对齐，按照 4 字节对齐 */
```

```

#define MEM_ALIGNMENT                4

/* 堆内存的大小，如果需要更大的堆内存，那么设置高一点 */
#define MEM_SIZE                      (10*1024)

/* MEMP_NUM_PBUF: 设置内存池的数量 */
#define MEMP_NUM_PBUF                10
/* MEMP_NUM_UDP_PCB: UDP 协议控制块的数量. */
#define MEMP_NUM_UDP_PCB              6
/* MEMP_NUM_TCP_PCB: TCP 的数量. */
#define MEMP_NUM_TCP_PCB              10
/* MEMP_NUM_TCP_PCB_LISTEN: 监听 TCP 的数量. */
#define MEMP_NUM_TCP_PCB_LISTEN       5
/* MEMP_NUM_TCP_SEG: 同时排队的 TCP 的数量段. */
#define MEMP_NUM_TCP_SEG               8
/* MEMP_NUM_SYS_TIMEOUT: 超时模拟活动的数量. */
#define MEMP_NUM_SYS_TIMEOUT           10

/* ----- Pbuf 选项 ----- */
/* PBUF_POOL 内存池中每个内存块大小 */
#define PBUF_POOL_SIZE                8
/* PBUF_POOL_BUFSIZE: pbuf 池中每个 pbuf 的大小. */
#define PBUF_POOL_BUFSIZE              1524

/* ----- TCP 选项 ----- */
#define LWIP_TCP                        1
#define TCP_TTL                        255

/* 控制 TCP 是否应该对到达的段进行排队
   秩序。如果你的设备内存不足，定义为 0. */
#define TCP_QUEUE_OOSEQ                0

/* TCP 最大段大小 */ /* TCP_MSS = (Ethernet MTU - IP header size - TCP header size) */
#define TCP_MSS                        (1500 - 40)

/* TCP 发送者缓冲区空间 (字节). */
#define TCP_SND_BUF                     (4*TCP_MSS)

/* TCP_SND_QUEUELEN: TCP 发送缓冲区空间。这必须是至少
   需要 (2 * TCP_SND_BUF/TCP_MSS) 才能正常工作 */

```



```

#define TCP_SND_QUEUELEN                (2* TCP_SND_BUF/TCP_MSS)

/* TCP 接收窗口 */
#define TCP_WND                          (2*TCP_MSS)

/* ----- ICMP 选项 ----- */
#define LWIP_ICMP                        1

/* ----- DHCP 选项 ----- */
/* 如果您希望 DHCP 配置为, 请将 LWIP_DHCP 定义为 1 */
#define LWIP_DHCP                        1

/* ----- UDP 选项 ----- */
#define LWIP_UDP                          1
#define UDP_TTL                          255

/* ----- Statistics 选项 ----- */
#define LWIP_STATS                        0

/* ----- 链接回调选项 ----- */
/* WIP_NETIF_LINK_CALLBACK==1:支持来自接口的回调函数
   每当链接改变(例如, 向下链接)
   */
#define LWIP_NETIF_LINK_CALLBACK        1
/*
   ----- 帧校验和选项 -----
   */

/*
The STM32F4x7 allows computing and verifying the IP, UDP, TCP and ICMP
checksums by hardware:
- To use this feature let the following define uncommented.
- To disable it and process by CPU comment the the checksum.
*/
#define CHECKSUM_BY_HARDWARE

```

```

#ifdef CHECKSUM_BY_HARDWARE
    #define CHECKSUM_GEN_IP            0
    #define CHECKSUM_GEN_UDP          0
    #define CHECKSUM_GEN_TCP          0
    #define CHECKSUM_CHECK_IP         0
    #define CHECKSUM_CHECK_UDP        0
    #define CHECKSUM_CHECK_TCP        0
    #define CHECKSUM_GEN_ICMP         0
#else
    #define CHECKSUM_GEN_IP            1
    #define CHECKSUM_GEN_UDP          1
    #define CHECKSUM_GEN_TCP          1
    #define CHECKSUM_CHECK_IP         1
    #define CHECKSUM_CHECK_UDP        1
    #define CHECKSUM_CHECK_TCP        1
    #define CHECKSUM_GEN_ICMP         1
#endif

/*
-----
----- 连续层的选择 -----
-----
*/
/**
 * LWIP_NETCONN==1: 启用 Netconn API (需要使用 api_lib.c)
 */
#define LWIP_NETCONN                1

/*
-----
----- Socket 选项 -----
-----
*/
/**
 * LWIP_SOCKET==1: 启用 Socket API (要求使用 Socket .c)
 */
#define LWIP_SOCKET                  0

/*
-----
----- httpd options -----
-----

```

```

*/
/** Set this to 1 to include "fsdata_custom.c" instead of "fsdata.c" for the
 * file system (to prevent changing the file included in CVS) */
#define HTTPD_USE_CUSTOM_FSDATA      1
*/

-----
----- 操作系统选项 -----
-----

*/

#define TCPIP_THREAD_NAME              " TCP/IP"
#define TCPIP_THREAD_STACKSIZE        1000
#define TCPIP_MBOX_SIZE                6
#define DEFAULT_UDP_RECVMBOX_SIZE     6
#define DEFAULT_TCP_RECVMBOX_SIZE     6
#define DEFAULT_ACCEPTMBOX_SIZE       6
#define DEFAULT_THREAD_STACKSIZE      500
#define TCPIP_THREAD_PRIO              5

#endif /* __LWIPOPTS_H__ */

```

笔者对新的 lwipopts.h 文件做了中文注释以及配置项的数值修改，例如修改了 TCPIP_THREAD_PRIO 配置项为 5，这个配置项是设置协议栈任务的优先级，当然大家也可以使用正点原子《lwIP 例程 5 lwIP_FreeRTOS 移植》例程中的 lwipopts.h 文件，该文件的内容是参考上述的源码改编得来的。

3.4.2 添加 sys_arch.c/h 文件

如果 lwIP 使用操作系统的话，sys_arch.c/h 这两个文件是有必要了解的，这些文件是 lwIP 内核与操作系统交互的接口文件，也可以说，它们是 lwIP 与操作系统的交接文件，本章的实验是使用 FreeRTOS 操作系统，显然 sys_arch.c 文件包含的邮箱、信号量以及互斥锁等 IPC 策略都是由 FreeRTOS 操作系统提供的，注意：FreeRTOS 操作系统没有邮箱的概念，可使用消息队列替代邮箱机制。

打开“contrib-2.1.0\ports\freertos\路径下的文件夹，该文件夹包含了与 FreeRTOS 相关的 sys_arch.c/h 文件，笔者把这两个文件复制到本章实验的 Middlewares\lwip\arch 路径下，该文件夹结构如下图所示。

lwIP例程5 lwIP_FreeRTOS移植 > Middlewares > lwip > arch		
名称	类型	大小
cc.h	C++ Header file	3 KB
ethernetif.c	C 文件	15 KB
ethernetif.h	C++ Header file	2 KB
lwip_comm.c	C 文件	11 KB
lwip_comm.h	C++ Header file	3 KB
lwipopts.h	C++ Header file	8 KB
sys_arch.c	C 文件	18 KB
sys_arch.h	C++ Header file	4 KB

图 3.4.2.1 arch 文件夹的文件结构

打开工程并在 Middlewares/lwip/arch 分组下添加 sys_arch.c 文件，如下图所示。

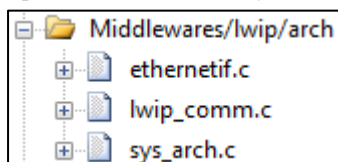


图 3.4.2.2 添加 sys_arch.c 文件

至此，我们编译工程会发现有很多的错误，这些错误都是指向 err.c 文件中的 err_to_errno_table 数组如下图所示。

```
error: #20: identifier "EINPROGRESS" is undefined
    Operation in progress    */
error: #20: identifier "EINVAL" is undefined
    Illegal value.         */
error: #20: identifier "EADDRINUSE" is undefined
    Address in use.        */
error: #20: identifier "EALREADY" is undefined
```

图 3.4.2.3 err.c 文件错误

这些错误的解决方法是在 lwipopts.h 文件添加 LWIP_PROVIDE_ERRNO 配置项并且把它设置为 1。打开 cc.h 文件，在此文件把“typedef int sys_prot_t;”代码注释掉，因为这个 sys_prot_t 变量已经在 sys_arch.c 文件中定义了，这里无需重复定义。

3.4.3 修改 lwip_comm.c/h

该文件主要修改网络配置的信息，比如默认 IP 的配置、lwIP 的初始化以及 DHCP 等处理，下面我们对 lwip_comm.c 文件简单修改，如下所示：

(1) FreeRTOS 相关配置

```
/* LINK 线程配置 */
#define LWIP_LINK_TASK_PRIO          1                /* 任务优先级 */
#define LWIP_LINK_STK_SIZE          128 * 2          /* 任务堆栈大小 */
void lwip_link_thread( void * argument );            /* 链路线程 */

/* DHCP 线程配置 */
#define LWIP_DHCP_TASK_PRIO          0                /* 任务优先级 */
```

```
#define LWIP_DHCP_STK_SIZE          128 * 2          /* 任务堆栈大小 */
void lwip_periodic_handle(void *argument);           /* DHCP 线程 */
```

声明 LINK 任务和 DHCP 任务配置，它们分别处理网口状态和 DHCP 分配。

(2) 删除 `lwip_periodic_handle`、`lwip_dhcp_process` 和 `lwip_pkt_handle` 函数。

(3) 修改 `lwip_comm_init` 函数，该函数如下所示：

```
/**
 * @breif      LWIP 初始化(LWIP 启动的时候使用)
 * @param      无
 * @retval     0,成功
 *             1,内存错误
 *             2,以太网芯片初始化失败
 *             3,网卡添加失败.
 */
uint8_t lwip_comm_init(void)
{
    /* 此处省略代码..... */

    tcpip_init(NULL, NULL);

    /* 此处省略代码..... */

    //lwip_init();
#if LWIP_DHCP          /* 使用动态 IP */
    /* 此处省略代码..... */
#else    /* 使用静态 IP */
    /* 此处省略代码..... */
#endif    /* 向网卡列表中添加一个网口 */

    //netif_init_flag = netif_add(&lwip_netif, (const ip_addr_t *)&ipaddr,
                                (const ip_addr_t *)&netmask,
                                (const ip_addr_t *)&gw, NULL,
                                &ethernetif_init, &ethernet_input);

    netif_init_flag = netif_add(&lwip_netif, (const ip_addr_t *)&ipaddr,
                                (const ip_addr_t *)&netmask,
                                (const ip_addr_t *)&gw, NULL,
                                &ethernetif_init, &tcpip_input);

    if (netif_init_flag == NULL)
    {
        return 1;          /* 网卡添加失败 */
    }
}
```

```

else /* 网口添加成功后,设置 netif 为默认值,并且打开 netif 网口 */
{
    netif_set_default(&g_lwip_netif); /* 设置 netif 为默认网口 */

#ifdef LWIP_NETIF_LINK_CALLBACK
    lwip_link_status_updated(&g_lwip_netif); /* DHCP 链接状态更新函数 */
    netif_set_link_callback(&g_lwip_netif, lwip_link_status_updated);
    /* 查询 PHY 连接状态任务 */
    sys_thread_new("eth_link",
                    lwip_link_thread, /* 任务入口函数 */
                    &g_lwip_netif, /* 任务入口函数参数 */
                    LWIP_LINK_STK_SIZE, /* 任务栈大小 */
                    LWIP_LINK_TASK_PRIO); /* 任务的优先级 */
#endif

    }

    g_lwipdev.link_status = LWIP_LINK_OFF; /* 链接标记为 0 */
#ifdef LWIP_DHCP /* 如果使用 DHCP 的话 */
    g_lwipdev.dhcpstatus = 0; /* DHCP 标记为 0 */
    /* DHCP 轮询任务 */
    sys_thread_new("eth_dhcp",
                    lwip_periodic_handle, /* 任务入口函数 */
                    &g_lwip_netif, /* 任务入口函数参数 */
                    LWIP_DHCP_STK_SIZE, /* 任务栈大小 */
                    LWIP_DHCP_TASK_PRIO); /* 任务的优先级 */
#endif

    return 0; /* 操作 OK. */
}

```

此函数主要修改三处地方, 首先我们把 `lwip_init` 函数注释掉并添加 `tcpip_init` 函数, 该函数主要创建邮箱以及 `tcpip` 线程, 它专门处理数据消息并把它往上层递交, 接着修改 `netif_add` 函数的第七个形参, 该形参不再指向 `ethernet_input` 函数而是指向 `tcpip_input` 函数, 最后创建 `eth_link` 和 `eth_dhcp` 任务, 它们分别处理网口状态和 DHCP 分配。

打开 `lwip_comm.h` 文件, 把它的内容修改为以下源码:

```

#ifndef _LWIP_COMM_H
#define _LWIP_COMM_H

#include "../BSP/ETHERNET/ethernet.h"
#include "netif/etharp.h"
#include "lwip/timeouts.h"
#include "lwip/snmp.h"

/* DHCP 进程状态 */
#define LWIP_DHCP_OFF (uint8_t) 0 /* DHCP 服务器关闭状态 */

```

```
#define LWIP_DHCP_START (uint8_t) 1 /* DHCP 服务器启动状态 */
#define LWIP_DHCP_WAIT_ADDRESS (uint8_t) 2 /* DHCP 服务器等待分配 IP 状态 */
#define LWIP_DHCP_ADDRESS_ASSIGNED (uint8_t) 3 /* DHCP 服务器地址已分配状态 */
#define LWIP_DHCP_TIMEOUT (uint8_t) 4 /* DHCP 服务器超时状态 */
#define LWIP_DHCP_LINK_DOWN (uint8_t) 5 /* DHCP 服务器链接失败状态 */

/* 链接状态 */
#define LWIP_LINK_OFF (uint8_t) 0 /* 链接关闭状态 */
#define LWIP_LINK_ON (uint8_t) 1 /* 链接开启状态 */
#define LWIP_LINK_AGAIN (uint8_t) 2 /* 重复开启 */

/* DHCP 服务器最大重试次数 */
#define LWIP_MAX_DHCP_TRIES (uint8_t) 4

typedef void (*display_fn)(uint8_t index);

/*lwip 控制结构体*/
typedef struct
{
    uint8_t mac[6]; /* MAC 地址 */
    uint8_t remoteip[4]; /* 远端主机 IP 地址 */
    uint8_t ip[4]; /* 本机 IP 地址 */
    uint8_t netmask[4]; /* 子网掩码 */
    uint8_t gateway[4]; /* 默认网关的 IP 地址 */
    uint8_t dhcpstatus; /* dhcp 状态
        0, 未获取 DHCP 地址
        1, 进入 DHCP 获取状态
        2, 成功获取 DHCP 地址
        0xFF, 获取失败 */
    uint8_t link_status; /* 连接状态 */
    display_fn lwip_display_fn; /* 显示函数指针 */
}__lwip_dev;

extern __lwip_dev g_lwipdev; /* lwip 控制结构体 */

void lwip_comm_default_ip_set(__lwip_dev *lwipx); /* lwip 默认 IP 设置 */
uint8_t lwip_comm_init(void); /* LWIP 初始化(LWIP 启动的时候使用) */
#endif
```

3.4.4 修改 ethernetif.c/h 文件

这些文件是网卡底层驱动文件，它要负责接收数据包和处理数据包，该文件修改的地方非常多，这里笔者不做讲解，读者可参考正点原子编写好的 ethernetif.c/h 或者 STM32Cube_FW_H7RS_V1.0.0\Projects\NUCLEO-H7S3L8\Applications\LwIP\LwIP_HTTP_Server_Netconn_RT

OS 工程下的 ethernetif.c/h 文件。

3.4.5 修改 ethernet.c 文件

这个文件我们只修改 HAL_ETH_MspInit 函数，在此函数下添加两段代码，如下所示：

```
/* Enable the Ethernet global Interrupt */
HAL_NVIC_SetPriority(ETH_IRQn, 0x07, 0);
HAL_NVIC_EnableIRQ(ETH_IRQn);
```

注意：以太网中断优先级必须归 FreeRTOS 内核管理，具体原因请看正点原子《FreeRTOS 开发指南》教程。

3.4.6 添加应用程序

移植好 lwIP 之后，当然要测试一下移植是否成功。在本步骤中，一共需要修改 1 个文件并添加 4 个文件，修改的 1 个文件为 main.c，添加的 4 个文件为 freertos_demo.c、freertos_demo.h、lwip_demo.c 和 lwip_demo.h。对于 main.c 主要是在 main 函数中完成一些硬件的初始化，最后调用 freertos_demo.c 文件中的 freertos_demo 函数。freertos_demo.c 则是用于编写 FreeRTOS 的相关应用程序代码。而 lwip_demo.c 则是用于编写 lwIP 的相关应用程序代码。

注意：freertos_demo.c、freertos_demo.h 文件保存在 User 文件夹下，而 lwip_demo.c、lwip_demo.h 文件保存在 Middlewares\lwip\lwip_app 文件夹下。

1. main.c 文件

由于正点原子的 STM32 系列开发板众多，这里以 H7R7 为例为读者进行演示，读者可以根据自己所移植的目标开发板在本教程的配套例程源码的《lwIP 例程 6 lwIP_FreeRTOS 移植》中查看对应的 main.c 文件。H7R7 开发板《lwIP 例程 6 lwIP_FreeRTOS 移植》中的 main.c 文件如下所示：

```
/**
 * *****
 * @file      main.c
 * @author    正点原子团队 (ALIENTEK)
 * @version   V1.0
 * @date      2024-8-01
 * @brief     lwIP+FreeRTOS 操作系统移植实验
 * @license   Copyright (c) 2020-2032, 广州市星翼电子科技有限公司
 * *****
 * @attention
 *
 * 实验平台:正点原子 STM32H7R7 开发板
 * 在线视频:www.yuanzige.com
 * 技术论坛:www.openedv.com
 * 公司网址:www.alientek.com
 * 购买地址:openedv.taobao.com
 *
 * *****
 */
```



```
#include "../SYSTEM/sys/sys.h"
#include "../SYSTEM/usart/usart.h"
#include "../SYSTEM/delay/delay.h"
#include "../BSP/LED/led.h"
#include "../BSP/LCD/lcd.h"
#include "../BSP/KEY/key.h"
#include "../USMART/usmart.h"
#include "../MALLOC/malloc.h"
#include "lwip_comm.h"
#include "lwipopts.h"
#include "freertos_demo.h"
```

```
int main(void)
{
    /* 初始化.....*/

    freertos_demo();
}
```

可以看到，在 main.c 文件中只包含了一个 main 函数，main 函数主要就是完成了一些外设的初始化，如串口、LED、LCD、按键等，并在最后调用了函数 freertos_demo。

2. freertos_demo.c 文件

```
/**
*****
* @file      freertos_demo.c
* @author    正点原子团队 (ALIENTEK)
* @version   V1.0
* @date      2022-8-01
* @brief     lwIP+FreeRTOS 操作系统移植实验
* @license   Copyright (c) 2020-2032, 广州市星翼电子科技有限公司
*****
* @attention
*
* 实验平台:正点原子 STM32H7R7 开发板
* 在线视频:www.yuanzige.com
* 技术论坛:www.openedv.com
* 公司网址:www.alientek.com
* 购买地址:openedv.taobao.com
*
*****
*/

#include "freertos_demo.h"
```

```
#include "../BSP/LED/led.h"
#include "../BSP/LCD/lcd.h"
#include "../SYSTEM/usart/usart.h"
#include "../SYSTEM/delay/delay.h"
#include "lwip_comm.h"
#include "lwipopts.h"
#include "FreeRTOS.h"
#include "task.h"

/*****
/*FreeRTOS 配置*/

/* START_TASK 任务 配置
 * 包括：任务句柄 任务优先级 堆栈大小 创建任务
 */
#define START_TASK_PRIO          1                /* 任务优先级 */
#define START_STK_SIZE           256              /* 任务堆栈大小 */
TaskHandle_t StartTask_Handler;                  /* 任务句柄 */
void start_task(void *pvParameters);              /* 任务函数 */

/* LWIP_DEMO 任务 配置
 * 包括：任务句柄 任务优先级 堆栈大小 创建任务
 */
#define LWIP_DME0_TASK_PRIO      11               /* 任务优先级 */
#define LWIP_DME0_STK_SIZE       1024             /* 任务堆栈大小 */
TaskHandle_t LWIP_Task_Handler;                  /* 任务句柄 */
void lwip_demo_task(void *pvParameters);          /* 任务函数 */

/* LED_TASK 任务 配置
 * 包括：任务句柄 任务优先级 堆栈大小 创建任务
 */
#define LED_TASK_PRIO            10               /* 任务优先级 */
#define LED_STK_SIZE             128              /* 任务堆栈大小 */
TaskHandle_t LEDTask_Handler;                    /* 任务句柄 */
void led_task(void *pvParameters);                /* 任务函数 */
*****/

/**
 * @brief      加载 UI
 * @param      mode :   bit0:0,不加载;1,加载前半部分 UI
 *              bit1:0,不加载;1,加载后半部分 UI
 */
```

```

* @retval      无
*/
void lwip_test_ui(uint8_t mode)
{
    uint8_t speed;
    uint8_t buf[30];

    if (mode & 1 << 0)
    {
        lcd_fill(5, 30, lcddev.width, 110, WHITE);
        lcd_show_string(5, 30, 200, 16, 16, "STM32", RED);
        lcd_show_string(5, 50, 200, 16, 16, "lwIP ping Test", RED);
        lcd_show_string(5, 70, 200, 16, 16, "ATOM@ALIENTEK", RED);
    }

    if (mode & 1 << 1)
    {
        lcd_fill(5, 110, lcddev.width, lcddev.height, WHITE);
        lcd_show_string(5, 110, 200, 16, 16, "lwIP Init Succeeded", BLUE);

        if (g_lwipdev.dhcpstatus == 2)
        {
            sprintf((char*)buf, "DHCP IP:%d.%d.%d.%d",
                g_lwipdev.ip[0],
                g_lwipdev.ip[1],
                g_lwipdev.ip[2],
                g_lwipdev.ip[3]); /* 显示动态 IP 地址 */
        }
        else
        {
            sprintf((char*)buf, "Static IP:%d.%d.%d.%d",
                g_lwipdev.ip[0],
                g_lwipdev.ip[1],
                g_lwipdev.ip[2],
                g_lwipdev.ip[3]); /* 打印静态 IP 地址 */
        }

        lcd_show_string(5, 130, 200, 16, 16, (char*)buf, BLUE);

        speed = ethernet_chip_get_speed(); /* 得到网速 */

        if (speed)
        {

```

```

        lcd_show_string(5, 150, 200, 16, 16, "Ethernet Speed:100M", BLUE);
    }
    else
    {
        lcd_show_string(5, 150, 200, 16, 16, "Ethernet Speed:10M", BLUE);
    }
}

/**
 * @breif      freertos_demo
 * @param      无
 * @retval      无
 */
void freertos_demo(void)
{
    /* start_task 任务 */
    xTaskCreate((TaskFunction_t      )start_task,
                (const char *        )"start_task",
                (uint16_t             )START_STK_SIZE,
                (void *               )NULL,
                (UBaseType_t          )START_TASK_PRIO,
                (TaskHandle_t *       )&StartTask_Handler);

    vTaskStartScheduler(); /* 开启任务调度 */
}

/**
 * @brief      start_task
 * @param      pvParameters : 传入参数(未用到)
 * @retval      无
 */
void start_task(void *pvParameters)
{
    pvParameters = pvParameters;

    g_lwipdev.lwip_display_fn = lwip_test_ui;

    lwip_test_ui(1); /* 加载后前部分 UI */

    while (lwip_comm_init() != 0)
    {
        lcd_show_string(30, 110, 200, 16, 16, "lwIP Init failed!!", RED);
    }
}

```

```

        delay_ms(500);
        lcd_fill(30, 50, 200 + 30, 50 + 16, WHITE);
        lcd_show_string(30, 110, 200, 16, 16, "Retrying... ", RED);
        delay_ms(500);
        LED1_TOGGLE();
    }
    /* 等待静态和动态分配完成 */
    while (g_lwipdev.dhcpstatus != 2 && g_lwipdev.dhcpstatus != 0xff)
    {
        vTaskDelay(500);
    }

    taskENTER_CRITICAL(); /* 进入临界区 */

    /* 创建 lwIP 任务 */
    xTaskCreate((TaskFunction_t) lwip_demo_task,
                (const char*) "lwip_demo_task",
                (uint16_t) LWIP_DMEM_STK_SIZE,
                (void*) NULL,
                (UBaseType_t) LWIP_DMEM_TASK_PRIO,
                (TaskHandle_t*) &LWIP_Task_Handler);

    /* LED 测试任务 */
    xTaskCreate((TaskFunction_t) led_task,
                (const char*) "led_task",
                (uint16_t) LED_STK_SIZE,
                (void*) NULL,
                (UBaseType_t) LED_TASK_PRIO,
                (TaskHandle_t*) &LEDTask_Handler);

    vTaskDelete(StartTask_Handler); /* 删除开始任务 */
    taskEXIT_CRITICAL(); /* 退出临界区 */
}

/**
 * @brief      lwIP 运行例程
 * @param      pvParameters : 传入参数(未用到)
 * @retval     无
 */
void lwip_demo_task(void *pvParameters)
{
    pvParameters = pvParameters;

```

```
uint8_t t = 0;

while (1)
{
    t++;

    if ((t % 40) == 0)
    {
        LED0_TOGGLE();    /* 翻转一次 LED0 */
    }

    vTaskDelay(5);
}

/**
 * @brief      系统再运行
 * @param      pvParameters : 传入参数(未用到)
 * @retval     无
 */
void led_task(void *pvParameters)
{
    pvParameters = pvParameters;

    while (1)
    {
        LED1_TOGGLE();
        vTaskDelay(1000);
    }
}
```

对于 freertos_demo.c 文件，这里先简单的介绍一下这个文件的代码结构，这个文件的代码结构可分为 6 个部分，分别是包含头文件、FreeRTOS 相关配置、全局变量及自定义函数、应用程序入口函数、开始任务入口函数、其他任务入口函数，接下来分别地介绍以上这几个部分的代码。

(1) 包含头文件

包含的头文件分成两个部分，分别为 FreeRTOS 头文件、lwIP 头文件和其他头文件，如下所示：

```
#include "freertos_demo.h"
#include "../BSP/LED/led.h"
#include "../BSP/LCD/lcd.h"
#include "../SYSTEM/usart/usart.h"
#include "../SYSTEM/delay/delay.h"
#include "lwip_comm.h"
```

```
#include "lwipopts.h"
#include "FreeRTOS.h"
#include "task.h"
```

(2) FreeRTOS 相关配置

FreeRTOS 的配置主要包括所创建 FreeRTOS 任务的相关定义（任务优先级、任务堆栈大小、任务句柄、任务函数）以及 FreeRTOS 相关变量（信号量、事件、列表、软件定时器等）的定义，如下所示：

```
/* ***** */
/* FreeRTOS 配置 */

/* START_TASK 任务 配置
 * 包括：任务句柄 任务优先级 堆栈大小 创建任务
 */
#define START_TASK_PRIO          1                /* 任务优先级 */
#define START_STK_SIZE           256              /* 任务堆栈大小 */
TaskHandle_t StartTask_Handler;                  /* 任务句柄 */
void start_task(void *pvParameters);              /* 任务函数 */

/* LWIP_DEMO 任务 配置
 * 包括：任务句柄 任务优先级 堆栈大小 创建任务
 */
#define LWIP_DMEMO_TASK_PRIO     11               /* 任务优先级 */
#define LWIP_DMEMO_STK_SIZE      1024             /* 任务堆栈大小 */
TaskHandle_t LWIP_Task_Handler;                   /* 任务句柄 */
void lwip_demo_task(void *pvParameters);          /* 任务函数 */

/* LED_TASK 任务 配置
 * 包括：任务句柄 任务优先级 堆栈大小 创建任务
 */
#define LED_TASK_PRIO            10               /* 任务优先级 */
#define LED_STK_SIZE             128              /* 任务堆栈大小 */
TaskHandle_t LEDTask_Handler;                     /* 任务句柄 */
void led_task(void *pvParameters);                /* 任务函数 */
/* ***** */
```

(3) 应用程序入口函数

这部分就是函数 freertos_demo，函数 freertos_demo 一开始就是创建开始任务，最后开启 FreeRTOS 系统任务调度，如下所示：

```
/**
 * @brief      freertos_demo
 * @param      无
 * @retval     无
 */
void freertos_demo(void)
```

```
{
    /* lwip_task 任务 */
    xTaskCreate((TaskFunction_t      )start_task,
                (const char *        )"start_task",
                (uint16_t             )START_STK_SIZE,
                (void *               )NULL,
                (UBaseType_t          )START_TASK_PRIO,
                (TaskHandle_t *       )&StartTask_Handler);

    vTaskStartScheduler(); /* 开启任务调度 */
}
```

(4) 开始任务入口函数

这部分就是开始任务的入口函数，开始任务主要用于创建或初始化特定实验中使用到的一些硬件外设、lwIP 初始化和 FreeRTOS 相关的软件（信号量、事件、列表、软件定时器等）以及创建其他用于实验演示的任务，如下所示：

```
/**
 * @brief      start_task
 * @param      pvParameters : 传入参数(未用到)
 * @retval     无
 */
void start_task(void *pvParameters)
{
    pvParameters = pvParameters;

    g_lwipdev.lwip_display_fn = lwip_test_ui;

    lwip_test_ui(1); /* 加载后前部分 UI */

    while (lwip_comm_init() != 0)
    {
        lcd_show_string(30, 110, 200, 16, 16, "lwIP Init failed!!", RED);
        delay_ms(500);
        lcd_fill(30, 50, 200 + 30, 50 + 16, WHITE);
        lcd_show_string(30, 110, 200, 16, 16, "Retrying...      ", RED);
        delay_ms(500);
        LED1_TOGGLE();
    }

    /* 等待静态和动态分配完成 */
    while (g_lwipdev.dhcpstatus != 2 && g_lwipdev.dhcpstatus != 0xff)
    {
        vTaskDelay(500);
    }

    taskENTER_CRITICAL(); /* 进入临界区 */
```



```

/* 创建 lwIP 任务 */
xTaskCreate((TaskFunction_t      )lwip_demo_task,
            (const char*         )"lwip_demo_task",
            (uint16_t             )LWIP_DMEM_STK_SIZE,
            (void*                )NULL,
            (UBaseType_t          )LWIP_DMEM_TASK_PRIO,
            (TaskHandle_t*)       )&LWIP_Task_Handler);

/* LED 测试任务 */
xTaskCreate((TaskFunction_t      )led_task,
            (const char*         )"led_task",
            (uint16_t             )LED_STK_SIZE,
            (void*                )NULL,
            (UBaseType_t          )LED_TASK_PRIO,
            (TaskHandle_t*)       )&LEDTask_Handler);

vTaskDelete(StartTask_Handler); /* 删除开始任务 */
taskEXIT_CRITICAL();             /* 退出临界区 */
}

```

(6) 其他任务入口函数

```

/**
 * @brief      lwIP 运行例程
 * @param      pvParameters : 传入参数 (未用到)
 * @retval     无
 */
void lwip_demo_task(void *pvParameters)
{
    pvParameters = pvParameters;
    uint8_t t = 0;

    while (1)
    {
        t++;

        if ((t % 40) == 0)
        {
            LED0_TOGGLE(); /* 翻转一次 LED0 */
        }

        vTaskDelay(5);
    }
}

```

```
/**
 * @brief      系统再运行
 * @param      pvParameters : 传入参数(未用到)
 * @retval     无
 */
void led_task(void *pvParameters)
{
    pvParameters = pvParameters;
    while (1)
    {
        LED1_TOGGLE();
        vTaskDelay(1000);
    }
}
```

我们一般在 `lwip_demo_task` 任务函数调用用户编写的函数执行 lwIP 例程，该函数是在 `lwip_demo.c` 文件定义。而 `led_task` 任务函数主要表示系统是否再运行。本教程配套的实验例程 6 以下的例程都会按着这个代码结构来进行实验代码的编写，建议读者先熟悉这个代码结构。

3. freertos_demo.h

```
#ifndef __FREERTOS_DEMO_H
#define __FREERTOS_DEMO_H

void freertos_demo(void);

#endif
```

`freertos_demo.h` 这个文件很简单，就是将函数 `freertos_demo` 导出给其他 C 源文件调用。

4. lwip_demo.c

该文件主要由用户编写的 lwIP 程序，如下所示：

```
/**
 * @brief      lwIP(lwIP 程序入口)
 * @param      无
 * @retval     无
 */
void lwip_demo(void)
{
    /* 编写 lwIP 代码 */
}
```

5. lwip_demo.h

```
#ifndef _LWIP_DEMO_H
#define _LWIP_DEMO_H
#include "../SYSTEM/sys/sys.h"
```

```
void lwip_demo(void); /* lwIP 例程入口 */
#endif
```

lwip_demo.h 这个文件很简单，就是将函数 lwip_demo 导出给其他 C 源文件调用。

至此，我们已经移植完成了，把代码下载到开发板中来验证移植释放成功，打开串口调试助手获取网络信息如下图所示。



图 3.4.6.1 串口调试助手接收到 IP 信息

在 PC 机上按下 win+r 快捷键打开命令行，并在命令行输入 “ping192.168.1.37” 命令，如下图所示。

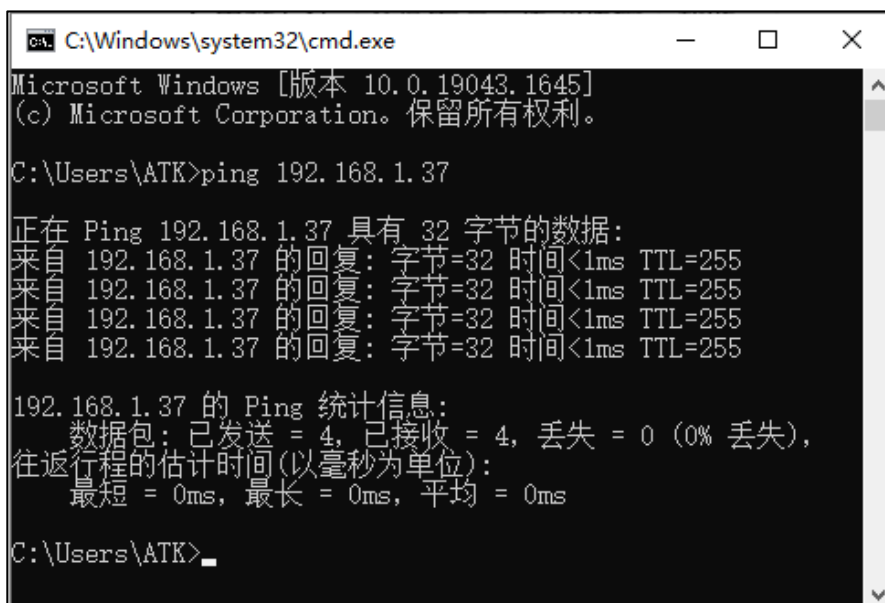


图 3.4.6.2 ping IP 地址成功

该实验的实验工程，请参考《lwIP 例程 6 lwIP_FreeRTOS 移植》。

3.5 sys_arch 文件解析

下面我们分别地讲解 sys_arch.c 和 sys_arch.h 实现源码，如下所示：

1. sys_arch.c 文件

该文件是 lwIP 内核与操作系统交互的接口文件，该文件定义了非常多的函数，这些函数如下表所示：

函数	类型	描述
sys_mbox_new	函数	创建一个邮箱
sys_mbox_free	函数	删除一个邮箱
sys_mbox_post	函数	向邮箱发送消息，一直阻塞
sys_mbox_trypost	函数	向邮箱发送消息，非阻塞
sys_mbox_trypost_fromisr	函数(中断)	向邮箱发送消息，非阻塞
sys_arch_mbox_fetch	函数	从邮箱中获取消息，阻塞
sys_arch_mbox_tryfetch	函数	从邮箱中获取消息，非阻塞

sys_mbox_valid	函数	判断邮箱是否有效
sys_mbox_set_invalid	函数	设置邮箱为无效状态
sys_sem_new	函数	创建一个信号量
sys_arch_sem_wait	函数	等待一个信号量
sys_sem_signal	函数	释放一个信号量
sys_sem_free	函数	删除一个信号量
sys_sem_valid	函数	判断信号量是否有效
sys_sem_set_invalid	函数	将信号量设置无效状态
sys_init	函数	初始化系统
sys_now	函数	内核时钟
sys_jiffies	函数	获取节拍
sys_mutex_new	函数	创建一个互斥量
sys_mutex_free	函数	删除一个互斥量
sys_mutex_lock	函数	获取一个互斥量（阻塞）
sys_mutex_unlock	函数	释放一个互斥量
sys_thread_new	函数	创建一个线程
sys_arch_protect	函数	进入临界段
sys_arch_unprotect	函数	退出临界段
sys_arch_msleep	函数	lwip 延时函数

表 3.5.1 sys_arch.c 的相关函数

上述的函数很多，但是我们仔细发现，无非就是三个 IPC 策略和几个系统配置函数，下面笔者一一讲解它们的作用。

(1) 邮箱机制

邮箱本质上就是一个指向数据的指针，邮箱在操作系统里是作为数据交互的 IPC，这样我们已经明了了，lwIP 与操作系统的数据交互和 API 消息是使用邮箱来完成的，例如用户将数据传递给内核也是通过邮箱将一个指针进行传递，如下源码所示：

```
/**
 * @brief      创建一个邮箱
 * @param      mbox : 邮箱的句柄
 * @param      size : 邮箱的大小
 * @retval     ERR_OK 或者 ERR_MEM
 */
err_t
sys_mbox_new(sys_mbox_t *mbox, int size)
{
    LWIP_ASSERT("mbox != NULL", mbox != NULL);
    LWIP_ASSERT("size > 0", size > 0);

    mbox->mbx = xQueueCreate((UBaseType_t)size, sizeof(void *));

    if (mbox->mbx == NULL)
    {
        SYS_STATS_INC(mbox.err);
        return ERR_MEM;
    }
}
```

```

        SYS_STATS_INC_USED(mbox);
        return ERR_OK;
    }

/**
 * @brief      向邮箱发送消息，一直阻塞
 * @param      mbox : 邮箱的句柄
 * @param      msg  : 邮箱的参数
 * @retval     无
 */
void
sys_mbox_post(sys_mbox_t *mbox, void *msg)
{
    BaseType_t ret;
    LWIP_ASSERT("mbox != NULL", mbox != NULL);
    LWIP_ASSERT("mbox->mbx != NULL", mbox->mbx != NULL);

    ret = xQueueSendToBack(mbox->mbx, &msg, portMAX_DELAY);
    LWIP_ASSERT("mbox post failed", ret == pdTRUE);
}

/**
 * @brief      向邮箱发送消息，非阻塞
 * @param      mbox : 邮箱的句柄
 * @param      msg  : 邮箱的参数
 * @retval     ERR_OK 或者 ERR_MEM
 */
err_t
sys_mbox_trypost(sys_mbox_t *mbox, void *msg)
{
    BaseType_t ret;
    LWIP_ASSERT("mbox != NULL", mbox != NULL);
    LWIP_ASSERT("mbox->mbx != NULL", mbox->mbx != NULL);

    ret = xQueueSendToBack(mbox->mbx, &msg, 0);

    if (ret == pdTRUE)
    {
        return ERR_OK;
    }
    else
    {

```

```

        LWIP_ASSERT("mbox trypost failed", ret == errQUEUE_FULL);
        SYS_STATS_INC(mbox.err);
        return ERR_MEM;
    }
}

/**
 * @brief      在中断中向邮箱发送消息
 * @param      mbox : 邮箱的句柄
 * @param      msg  : 邮箱的参数
 * @retval     ERR_OK 或者 ERR_MEM
 */
err_t
sys_mbox_trypost_fromisr(sys_mbox_t *mbox, void *msg)
{
    BaseType_t ret;
    BaseType_t xHigherPriorityTaskWoken = pdFALSE;
    LWIP_ASSERT("mbox != NULL", mbox != NULL);
    LWIP_ASSERT("mbox->mbx != NULL", mbox->mbx != NULL);

    ret = xQueueSendToBackFromISR(mbox->mbx, &msg, &xHigherPriorityTaskWoken);

    if (ret == pdTRUE)
    {
        if (xHigherPriorityTaskWoken == pdTRUE)
        {
            return ERR_NEED_SCHED;
        }

        return ERR_OK;
    }
    else
    {
        LWIP_ASSERT("mbox trypost failed", ret == errQUEUE_FULL);
        SYS_STATS_INC(mbox.err);
        return ERR_MEM;
    }
}

/**
 * @brief      从邮箱中获取消息，阻塞
 * @param      mbox : 邮箱的句柄
 * @param      msg  : 邮箱的参数

```

```

* @param      timeout_ms  超时时间
* @retval      SYS_ARCH_TIMEOUT
*/
u32_t
sys_arch_mbox_fetch(sys_mbox_t *mbox, void **msg, u32_t timeout_ms)
{
    BaseType_t ret;
    void *msg_dummy;
    LWIP_ASSERT("mbox != NULL", mbox != NULL);
    LWIP_ASSERT("mbox->mbx != NULL", mbox->mbx != NULL);

    if (!msg)
    {
        msg = &msg_dummy;
    }

    if (!timeout_ms)
    {
        /* wait infinite */
        ret = xQueueReceive(mbox->mbx, &(*msg), portMAX_DELAY);
        LWIP_ASSERT("mbox fetch failed", ret == pdTRUE);
    }
    else
    {
        TickType_t timeout_ticks = timeout_ms / portTICK_RATE_MS;
        ret = xQueueReceive(mbox->mbx, &(*msg), timeout_ticks);

        if (ret == errQUEUE_EMPTY)
        {
            /* timed out */
            *msg = NULL;
            return SYS_ARCH_TIMEOUT;
        }

        LWIP_ASSERT("mbox fetch failed", ret == pdTRUE);
    }

    return 1;
}

/**
* @brief      从邮箱中获取消息，非阻塞
* @param      mbox : 邮箱的句柄

```

```

* @param      msg    : 邮箱的参数
* @retval     SYS_MBOX_EMPTY
*/
u32_t
sys_arch_mbox_tryfetch(sys_mbox_t *mbox, void **msg)
{
    BaseType_t ret;
    void *msg_dummy;
    LWIP_ASSERT("mbox != NULL", mbox != NULL);
    LWIP_ASSERT("mbox->mbx != NULL", mbox->mbx != NULL);

    if (!msg)
    {
        msg = &msg_dummy;
    }

    ret = xQueueReceive(mbox->mbx, &(*msg), 0);

    if (ret == errQUEUE_EMPTY)
    {
        *msg = NULL;
        return SYS_MBOX_EMPTY;
    }

    LWIP_ASSERT("mbox fetch failed", ret == pdTRUE);

    return 1;
}

/**
* @brief      删除一个邮箱
* @param      mbox : 邮箱的句柄
* @retval     无
*/
void
sys_mbox_free(sys_mbox_t *mbox)
{
    LWIP_ASSERT("mbox != NULL", mbox != NULL);
    LWIP_ASSERT("mbox->mbx != NULL", mbox->mbx != NULL);

#ifdef LWIP_FREERTOS_CHECK_QUEUE_EMPTY_ON_FREE
    {
        UBaseType_t msgs_waiting = uxQueueMessagesWaiting(mbox->mbx);

```



```

    LWIP_ASSERT("mbbox quence not empty", msgs_waiting == 0);

    if (msgs_waiting != 0)
    {
        SYS_STATS_INC(mbox.err);
    }
}
#endif

vQueueDelete(mbox->mbx);

SYS_STATS_DEC(mbox.used);
}

```

上述源码很简单，由于 FreeRTOS 操作系统没有邮箱的 IPC，所以我们只能使用 FreeRTOS 的消息队列模拟邮箱，关于 FreeRTOS 操作系统的消息队列知识，请大家参考正点原子《FreeRTOS 开发指南》第十三章的内容。

(2) 信号量机制

信号量在操作系统中起到同步的作用，所以 lwIP 与操作系统的信号量是为内核提供同步机制。例如用户发送数据到内核时需要调用上层 API 接口，在这个过程中用户需要去获取一个信号量，此时系统没有信号量释放，所以用户线程只能在堵塞状态，当 lwIP 内核获取到用户的数据并调用网卡去发送数据，直到数据发送完毕之后就会释放一个信号量来告知用户线程发送数据完成，最后用户线程从堵塞状态转成运行态。如下源码所示：

```

/**
 * @brief      创建一个信号量
 * @param      sem : 信号量的句柄
 * @param      initial_count 初始值
 * @retval     ERR_OK 或者 ERR_MEM
 */
err_t
sys_sem_new(sys_sem_t *sem, u8_t initial_count)
{
    LWIP_ASSERT("sem != NULL", sem != NULL);
    LWIP_ASSERT("initial_count invalid (not 0 or 1)",
        (initial_count == 0) || (initial_count == 1));

    sem->sem = xSemaphoreCreateBinary();

    if (sem->sem == NULL)
    {
        SYS_STATS_INC(sem.err);
        return ERR_MEM;
    }
}

```

```

SYS_STATS_INC_USED(sem);

if (initial_count == 1)
{
    BaseType_t ret = xSemaphoreGive(sem->sem);
    LWIP_ASSERT("sys_sem_new: initial give failed", ret == pdTRUE);
}

return ERR_OK;
}

/**
 * @brief      释放一个信号量
 * @param      sem : 信号量的句柄
 * @retval     无
 */
void
sys_sem_signal(sys_sem_t *sem)
{
    BaseType_t ret;
    LWIP_ASSERT("sem != NULL", sem != NULL);
    LWIP_ASSERT("sem->sem != NULL", sem->sem != NULL);

    ret = xSemaphoreGive(sem->sem);
    /* queue full is OK, this is a signal only... */
    LWIP_ASSERT("sys_sem_signal: sane return value",
        (ret == pdTRUE) || (ret == errQUEUE_FULL));
}

/**
 * @brief      等待一个信号量
 * @param      sem : 信号量的句柄
 * @param      timeout_ms : 无限的等待
 * @retval     SYS_ARCH_TIMEOUT 或者 1
 */
u32_t
sys_arch_sem_wait(sys_sem_t *sem, u32_t timeout_ms)
{
    BaseType_t ret;
    LWIP_ASSERT("sem != NULL", sem != NULL);
    LWIP_ASSERT("sem->sem != NULL", sem->sem != NULL);

    if (!timeout_ms)

```

```

{
    /* wait infinite */
    ret = xSemaphoreTake(sem->sem, portMAX_DELAY);
    LWIP_ASSERT("taking semaphore failed", ret == pdTRUE);
}
else
{
    TickType_t timeout_ticks = timeout_ms / portTICK_RATE_MS;
    ret = xSemaphoreTake(sem->sem, timeout_ticks);

    if (ret == errQUEUE_EMPTY)
    {
        /* 请求超时 */
        return SYS_ARCH_TIMEOUT;
    }

    LWIP_ASSERT("taking semaphore failed", ret == pdTRUE);
}

/* 旧版本的 lwIP 要求我们返回等待的时间。
   现在情况已经不一样了。刚刚返回 != SYS_ARCH_TIMEOUT
   这是足够的 */
return 1;
}

/**
 * @brief      删除一个信号量
 * @param      sem : 信号量的句柄
 * @retval     无
 */
void
sys_sem_free(sys_sem_t *sem)
{
    LWIP_ASSERT("sem != NULL", sem != NULL);
    LWIP_ASSERT("sem->sem != NULL", sem->sem != NULL);

    SYS_STATS_DEC(sem.used);
    vSemaphoreDelete(sem->sem);
    sem->sem = NULL;
}

```

上述源码是关于 FreeRTOS 操作系统的信号量知识，请大家参考正点原子《FreeRTOS 开发指南》第十四章的内容。

(3) 互斥信号量机制

互斥信号量主要作用是防止优先级翻转，当然互斥信号量是在信号量的基础上修改得来的，所以也有同步功能的作用，操作系统的互斥信号量为 lwIP 内核提供互斥机制，如下源码所示：

```
/**
 * @brief      创建一个互斥信号量
 * @param      mutex : 互斥信号量的句柄
 * @retval     ERR_OK 或者 ERR_MEM
 */
err_t
sys_mutex_new(sys_mutex_t *mutex)
{
    LWIP_ASSERT("mutex != NULL", mutex != NULL);

    mutex->mut = xSemaphoreCreateRecursiveMutex();

    if (mutex->mut == NULL)
    {
        SYS_STATS_INC(mutex.err);
        return ERR_MEM;
    }

    SYS_STATS_INC_USED(mutex);
    return ERR_OK;
}

/**
 * @brief      等待一个互斥信号量（阻塞）
 * @param      mutex : 互斥信号量的句柄
 * @retval     无
 */
void
sys_mutex_lock(sys_mutex_t *mutex)
{
    BaseType_t ret;
    LWIP_ASSERT("mutex != NULL", mutex != NULL);
    LWIP_ASSERT("mutex->mut != NULL", mutex->mut != NULL);

    ret = xSemaphoreTakeRecursive(mutex->mut, portMAX_DELAY);
    LWIP_ASSERT("failed to take the mutex", ret == pdTRUE);
}

/**
 * @brief      释放一个互斥信号量

```

```

* @param      mutex : 互斥信号量的句柄
* @retval     无
*/
void
sys_mutex_unlock(sys_mutex_t *mutex)
{
    BaseType_t ret;
    LWIP_ASSERT("mutex != NULL", mutex != NULL);
    LWIP_ASSERT("mutex->mut != NULL", mutex->mut != NULL);

    ret = xSemaphoreGiveRecursive(mutex->mut);
    LWIP_ASSERT("failed to give the mutex", ret == pdTRUE);
}

/**
* @brief      删除一个互斥信号量
* @param      mutex : 互斥信号量的句柄
* @retval     无
*/
void
sys_mutex_free(sys_mutex_t *mutex)
{
    LWIP_ASSERT("mutex != NULL", mutex != NULL);
    LWIP_ASSERT("mutex->mut != NULL", mutex->mut != NULL);

    SYS_STATS_DEC(mutex.used);
    vSemaphoreDelete(mutex->mut);
    mutex->mut = NULL;
}

```

关于互斥信号量的相关知识请大家参考正点原子《FreeRTOS 开发指南》第十四章的 14.8 小节内容。

(4) 其他系统配置

sys_arch.c 文件里有几个配置信息函数，如下源码所示：

```

/**
* @brief      系统初始化
* @param      无
* @retval     无
*/
void
sys_init(void)
{
}

```

```
#if LWIP_FREERTOS_SYS_NOW_FROM_FREERTOS
/**
 * @brief      提供 lwIP 内核时钟
 * @param      无
 * @retval     无
 */
u32_t
sys_now(void)
{
    return xTaskGetTickCount() * portTICK_PERIOD_MS;
}
#endif

/**
 * @brief      获取时钟节拍
 * @param      无
 * @retval     无
 */
u32_t
sys_jiffies(void)
{
    return xTaskGetTickCount();
}

#if SYS_LIGHTWEIGHT_PROT
/**
 * @brief      进入临界区
 * @param      无
 * @retval     无
 */
sys_prot_t
sys_arch_protect(void)
{
    taskENTER_CRITICAL();
}

/**
 * @brief      退出临界区
 * @param      无
 * @retval     无
 */
void
```

```
sys_arch_unprotect(sys_prot_t pval)
{
    taskEXIT_CRITICAL();
}

#endif /* SYS_LIGHTWEIGHT_PROT */

/**
 * @brief      lwip 延时函数
 * @param      delay_ms : 延时时间
 * @retval     无
 */
void
sys_arch_msleep(u32_t delay_ms)
{
    TickType_t delay_ticks = delay_ms / portTICK_RATE_MS;
    vTaskDelay(delay_ticks);
}

/**
 * @brief      创建线程
 * @param      name : 线程名称
 * @param      thread : 线程函数
 * @param      arg : 传入的参数
 * @param      stacksize : 堆栈的大小
 * @param      prio : 线程优先级
 * @retval     任务控制块
 */
sys_thread_t
sys_thread_new(const char *name, lwip_thread_fn thread, void *arg,
               int stacksize, int prio)
{
    TaskHandle_t rtos_task;
    BaseType_t ret;
    sys_thread_t lwip_thread;
    size_t rtos_stacksize;

    LWIP_ASSERT("invalid stacksize", stacksize > 0);
#if LWIP_FREERTOS_THREAD_STACKSIZE_IS_STACKWORDS
    rtos_stacksize = (size_t)stacksize;
#else
    rtos_stacksize = (size_t)stacksize / sizeof(StackType_t);
#endif
}
```

```
#endif

/* lwIP's lwip_thread_fn matches FreeRTOS' TaskFunction_t, so we can pass the
   thread function without adaption here. */
ret = xTaskCreate(thread, name, (configSTACK_DEPTH_TYPE)rtos_stacksize,
                  arg, prio, &rtos_task);
LWIP_ASSERT("task creation failed", ret == pdTRUE);

lwip_thread.thread_handle = rtos_task;
return lwip_thread;
}
```

上述函数非常简单，这里笔者不做讲解。

2. sys_arch.h 文件

该文件主要声明 sys_arch.c 文件的信息的，如下源码所示：

```
#ifndef LWIP_ARCH_SYS_ARCH_H
#define LWIP_ARCH_SYS_ARCH_H

#include "lwip/opt.h"
#include "lwip/arch.h"

/** This is returned by _fromisr() sys functions to tell the outermost function
 * that a higher priority task was woken and the scheduler needs to be invoked.
 */
#define ERR_NEED_SCHED 123

/* This port includes FreeRTOS headers in sys_arch.c only.
 * FreeRTOS uses pointers as object types. We use wrapper structs instead of
 * void pointers directly to get a tiny bit of type safety.
 */

void sys_arch_msleep(u32_t delay_ms);
#define sys_msleep(ms) sys_arch_msleep(ms)

#if SYS_LIGHTWEIGHT_PROT
typedef u32_t sys_prot_t;
#endif /* SYS_LIGHTWEIGHT_PROT */

#if !LWIP_COMPAT_MUTEX
struct _sys_mut {
    void *mut;
};
typedef struct _sys_mut sys_mutex_t;
#define sys_mutex_valid_val(mutex) ((mutex).mut != NULL)
```



```
#define sys_mutex_valid(mutex)      (((mutex) != NULL) &&  
sys_mutex_valid_val(*(mutex)))  
#define sys_mutex_set_invalid(mutex) ((mutex)->mut = NULL)  
#endif /* !LWIP_COMPAT_MUTEX */  
  
struct _sys_sem {  
    void *sem;  
};  
typedef struct _sys_sem sys_sem_t;  
#define sys_sem_valid_val(sema)     ((sema).sem != NULL)  
#define sys_sem_valid(sema)         (((sema) != NULL) &&  
sys_sem_valid_val(*(sema)))  
#define sys_sem_set_invalid(sema) ((sema)->sem = NULL)  
  
struct _sys_mbox {  
    void *mbx;  
};  
typedef struct _sys_mbox sys_mbox_t;  
#define sys_mbox_valid_val(mbox)     ((mbox).mbx != NULL)  
#define sys_mbox_valid(mbox)         (((mbox) != NULL) &&  
sys_mbox_valid_val(*(mbox)))  
#define sys_mbox_set_invalid(mbox) ((mbox)->mbx = NULL)  
  
struct _sys_thread {  
    void *thread_handle;  
};  
typedef struct _sys_thread sys_thread_t;  
  
#if LWIP_NETCONN_SEM_PER_THREAD  
sys_sem_t* sys_arch_netconn_sem_get(void);  
void sys_arch_netconn_sem_alloc(void);  
void sys_arch_netconn_sem_free(void);  
#define LWIP_NETCONN_THREAD_SEM_GET() sys_arch_netconn_sem_get()  
#define LWIP_NETCONN_THREAD_SEM_ALLOC() sys_arch_netconn_sem_alloc()  
#define LWIP_NETCONN_THREAD_SEM_FREE() sys_arch_netconn_sem_free()  
#endif /* LWIP_NETCONN_SEM_PER_THREAD */  
  
#endif /* LWIP_ARCH_SYS_ARCH_H */
```

上述源码没什么好讲解的，主要封装了几个宏定义来判断邮箱，信号量以及互斥信号量是否有效。至此，sys_arch.c/h 文件已经讲解完了。

第四章 lwIP 内存管理

对于嵌入式系统而言，内存管理是至关重要的环节。内存管理的选择将从根本上影响内存分配和回收的效率，进而决定整个系统的性能。为了满足这一需求，lwIP 提供了两种简单而高效的内存管理机制：动态内存池管理和动态内存堆管理。这两种机制都能够有效地管理内存资源，确保系统的高效运行。通过合理地选择和使用这两种机制，开发人员可以更好地应对嵌入式系统中的内存管理挑战，提高系统的性能和稳定性。

本章分为如下几个部分：

- 4.1 内存的类型
- 4.2 内存对齐
- 4.3 动态内存池管理策略
- 4.4 动态内存堆管理策略
- 4.5 使用 C 库管理内存策略

4.1 内存的简介

在 lwIP 中，内存分配策略主要有两种：动态内存池管理和动态内存堆管理。这两种策略在 lwIP 中起到了互补的作用，使内核可以根据不同场景选择合适的分配方式，从而大大提高系统的内存开销和分配效率。与 C 语言提供的内存分配方式不同，lwIP 也支持使用 malloc 和 free 进行内存分配。然而，lwIP 并不建议使用 C 标准库的内存分配策略。关于这一点的主要原因，将在本章的 4.5 小节中详细讲解。

1, lwIP 的宏配置及内存管理

在 lwIP 中，内存管理策略的选择是由几个宏定义的值来决定的。这些宏定义的值决定了内存管理的方式和行为。用户可以根据这些宏值来判断 lwIP 将使用哪种内存管理策略。通过合理地设置这些宏值，用户可以灵活地调整内存管理策略，以满足具体的应用需求。这些宏配置的作用如下表所示：

宏定义	描述
MEM_LIBC_MALLOC	选择 C 标准库分配策略（默认为 0）
MEMP_MEM_MALLOC	是否使用 lwIP 内存堆分配策略实现内存池分配（默认为 0）
MEM_USE_POOLS	是否使用 lwIP 内存池分配策略实现内存堆的分配（默认为 0）

表 4.1.1 内存管理配置描述

2, lwIP 的宏配置及内存管理的注意事项

在 lwIP 中，内存堆管理策略和 C 标准库管理策略只能选择其中一种。如果 MEM_LIBC_MALLOC 被设置为 0，那么 lwIP 内核将选择内存堆管理策略。这意味着用户可以根据自己的需求选择是否使用 C 标准库的内存分配策略，并通过设置相应的宏值来决定使用哪种内存管理策略。

4.2 动态内存堆管理策略

动态内存堆是一种可变长分配方式，常用于需要动态申请和释放内存的系统中。随着系统的运行，内存块的大小和数量会不断变化，严重时可能导致内存碎片。为了解决这个问题，lwIP 采用了 First Fit（首次拟合）内存管理算法。该算法优先利用内存中低地址部分的空闲分区，保留高地址部分的大空闲区，有利于为将来到达的大作业分配大的内存空间。然而，这种

算法也存在明显的缺点。由于是从低地址不断划分，系统会留下许多难以利用的小型空闲分区，即内存碎片。每次申请内存时，系统需要从低地址部分开始查找可用空闲分区，增加了查找时间。因此，使用这种算法时需要注意权衡利弊，根据实际需求进行选择。

下面，作者将深入解析 lwIP 内存堆算法的实现代码。这个算法由两个关键文件组成：mem.c 和 mem.h。其中，mem.c 文件尤为关键，因为它包含了实现 lwIP 内存堆分配和释放原理的代码。通过分析这些文件，作者将详细解析该算法的工作原理和实现细节。这将有助于读者更好地理解 lwIP 内存管理的核心机制，并能够根据实际需求进行优化和调整。通过深入了解和掌握 lwIP 内存堆算法的实现，开发人员可以更好地应对嵌入式系统中的内存管理挑战，提高系统的性能和稳定性。

(1) 内存堆的结构体

管理内存块的结构体，如下源码所示：

```
struct mem {
    mem_size_t next;      /* 保存下一个内存块的索引 */
    mem_size_t prev;      /* 保存前一个内存块的索引 */
    u8_t used;            /* 此内存块是否被用。1 使用、0 未使用 */
};
```

这个结构体 mem 定义了三个成员变量，其中 next 和 prev 变量用于保存下一个和前一个内存块的索引，而 used 变量用于标记被管理的内存块是否可用。这个结构体用于描述内存块的状态和相互之间的关系，为内存管理提供基础数据结构。

(2) 内存堆的对齐及最小配置值

```
#ifndef MIN_SIZE
#define MIN_SIZE          12
#endif /* MIN_SIZE */

/* 最小大小做对齐处理，后面均用对齐后的该宏值 */
#define MIN_SIZE_ALIGNED  LWIP_MEM_ALIGN_SIZE(MIN_SIZE)

/* 内存块头大小做对齐处理，后面均用对齐后的该宏值 */
#define SIZEOF_STRUCT_MEM  LWIP_MEM_ALIGN_SIZE(sizeof(struct mem))

/* 用户定义的堆大小做对齐处理，后面均用对齐后的该宏值 */
#define MEM_SIZE_ALIGNED  LWIP_MEM_ALIGN_SIZE(MEM_SIZE)
```

为了有效防止内存碎片，lwIP 内核定义了最小分配大小 MIN_SIZE。当用户申请的内存小于最小分配内存时，系统将分配 MIN_SIZE 大小的内存资源。此外，SIZEOF_STRUCT_MEM 宏定义用于确保内存大小进行 4 字节对齐。内存对齐的作用主要有两个方面：平台原因和性能原因。在平台原因方面，某些硬件平台只能从特定地址处获取特定类型的数据，否则会抛出硬件异常。在性能原因方面，经过内存对齐后，CPU 的内存访问速度大大提升。因此，通过定义最小分配大小和进行内存对齐，lwIP 内核能够有效地管理内存并提高系统的性能和稳定性。

(3) 定义内存堆的空间

```
#ifndef LWIP_RAM_HEAP_POINTER

/* 定义堆内存空间 */
LWIP_DECLARE_MEMORY_ALIGNED(ram_heap, MEM_SIZE_ALIGNED +
                             (2U*SIZEOF_STRUCT_MEM));

#define LWIP_RAM_HEAP_POINTER ram_heap
#endif
```

无论是内存堆还是内存池，它们都是对一个大数组进行操作。这个大数组通常被称为 `ram_heap` 数组。在 lwIP 中，这个数组的大小被定义为 `MEM_SIZE_ALIGNED + (2U*SIZEOF_STRUCT_MEM)`。通过这个数组，lwIP 内存堆可以分配所需的内存。这意味着，当用户请求内存时，lwIP 会从这个大数组中分配相应大小的内存块。通过合理地管理这个大数组，lwIP 能够提供高效的内存分配机制，满足不同场景的需求。这种基于大数组的内存管理方式有助于减少内存碎片，提高内存利用率。

(4) 操作内存堆的变量

```
/* 指向对齐后的内存堆的地址*/
static u8_t *ram;
/* 指向对齐后的内存堆的最后一个内存块 */
static struct mem *ram_end;
/* 指向已被释放的索引号最小的内存块（内存堆最前面的已被释放的）*/
static struct mem * LWIP_MEM_LFREE_VOLATILE lfree;
```

`ram_heap` 数组是 lwIP 定义的内存堆总空间，用于存储和管理内存。为了从这个总空间中申请合适大小的内存，lwIP 内核使用三个指针：`ram` 指针、`ram_end` 指针和 `lfree` 指针。`ram` 指针指向对齐后的内存堆总空间首地址，`ram_end` 指针指向内存堆总空间尾地址（接近总空间的尾地址），而 `lfree` 指针指向最低内存地址的空闲内存块。

在分配内存时，lwIP 内核根据 `lfree` 指针指向的空闲内存块进行分配。通过这种方式，内核能够快速找到可用的内存块，并有效地分配内存给需要的任务。同时，`ram_end` 指针用于检测总内存堆空间中是否存在空闲的内存，以便进行进一步的内存分配或释放操作。

(5) 内存堆的初始化

结合(1)~(4)的分析，我们来看一下 lwIP 动态内存堆是如何实现的。

1. mem_init 函数

```
void mem_init(void)
{
    struct mem *mem;

    /* 对内存堆的地址（全局变量的名）进行对齐指向 ram_heap。*/
    ram = (u8_t *)LWIP_MEM_ALIGN(LWIP_RAM_HEAP_POINTER);
    /* 建立第一个内存块，内存块由内存块头+空间组成。 */
    mem = (struct mem *) (void *) ram;
    /* 下一个内存块不存在，因此指向内存堆的结束 */
    mem->next = MEM_SIZE_ALIGNED;
    /* 前一个内存块就是它自己，因为这是第一个内存块 */
    mem->prev = 0;
    /* 第一个内存块没有被使用 */
    mem->used = 0;
    /* 初始化堆的末端（指向 MEM_SIZE_ALIGNED 底部位置） */
    ram_end = ptr_to_mem(MEM_SIZE_ALIGNED);

    /* 最后一个内存块被使用。因为其后面没有可用空间，必须标记为已被使用 */
    ram_end->used = 1;
```

```
/* 下一个不存在，因此指向内存堆的结束 */
ram_end->next = MEM_SIZE_ALIGNED;
/* 前一个不存在，因此指向内存堆的结束 */
ram_end->prev = MEM_SIZE_ALIGNED;
/* 已释放的索引最小的内存块就是上面建立的第一个内存块。 */
lfree = (struct mem *) (void *) ram;
/* 这里建立一个互斥信号量，主要是用来进行内存的申请、释放的保护 */
if (sys_mutex_new(&mem_mutex) != ERR_OK) {
}
}
```

上述源码是对堆空间进行初始化。初始化后，lfree 指针指向第一个内存块，该内存块由两部分组成：控制块和可用内存。控制块用于标记内存是否可用，而可用内存则是实际可分配的内存区域。ram_end 指针指向堆空间的尾部，用于判断堆空间中是否存在可用内存。如果 lfree 指针指向 ram_end 指针，则表示该堆空间没有可用内存可供分配。

在内存分配过程中，lfree 指针从低地址开始不断查找和划分内存，直到达到 ram_end 指针所指向的地址。这意味着 lfree 指针从堆空间的起始位置开始，逐个遍历内存块，直到找到可用的内存或到达堆空间的末尾才能结束分配。内存堆初始化示意图如下所示：

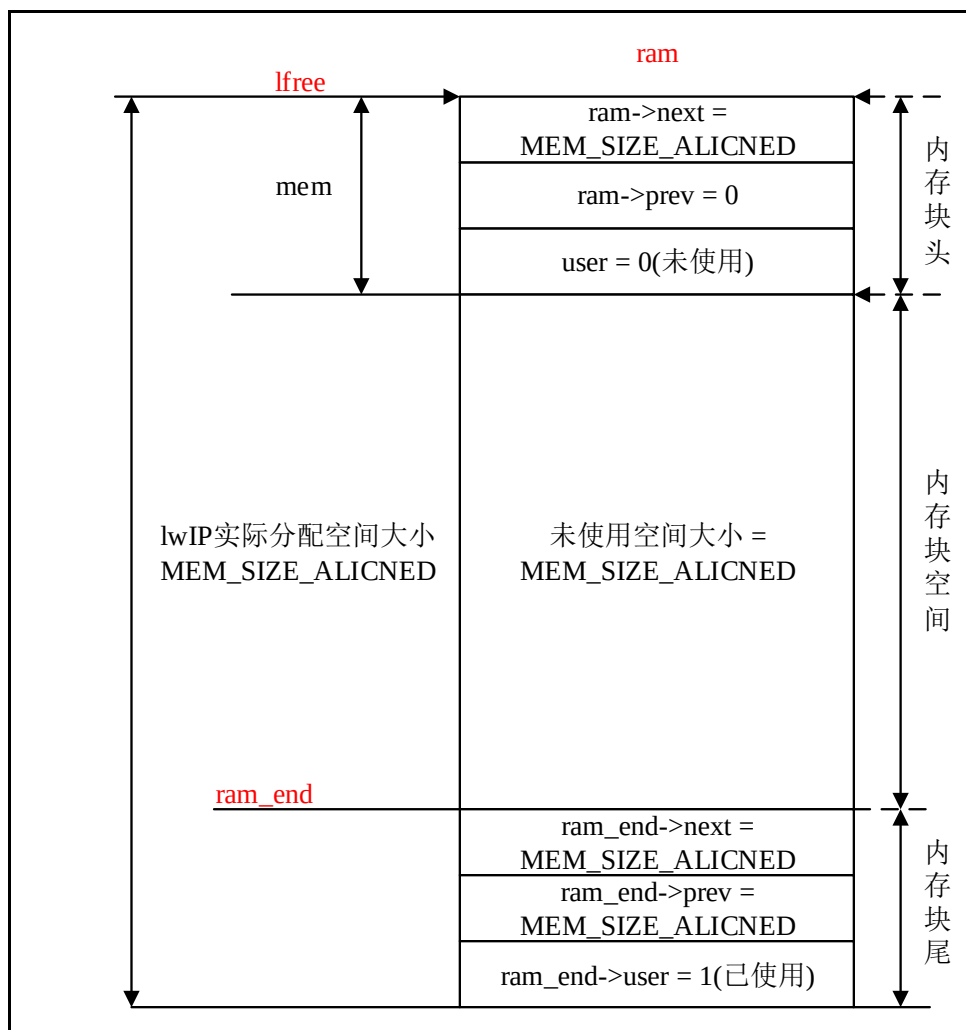


图 4.2.1 内存堆初始化示意图

struct mem 结构体的 next 和 prev 变量不是指针类型，而是用于保存内存块的索引号。这意味着它们保存的是数字索引，而不是实际的内存地址。例如，如果有一个长度为 10 的数组 a，next 和 prev 变量将保存 0 到 9 的索引号。通过这些索引号，lwIP 内核可以快速地定位到数组 a 中的特定元素的内存地址。

2. mem_malloc 函数

```
void *
mem_malloc(mem_size_t size_in){
    mem_size_t ptr, ptr2, size;
    struct mem *mem, *mem2;

    /*****第一：检测用户申请的内存块释放满足 LWIP 的规则*****/
    /*****第二：从内存堆中划分用户的内存块*****/
    /* 寻找足够大的空闲块，从最低的空闲块开始.*/
    for (ptr = mem_to_ptr(lfree); ptr < MEM_SIZE_ALIGNED - size;
        ptr = ((struct mem *) (void *)&ram[ptr])->next)
    {
        mem = ptr_to_mem(ptr); /* 取它的地址 */
        /* 空间大小必须排除内存块头大小 */
        if ((!mem->used) &&
            (mem->next - (ptr + SIZEOF_STRUCT_MEM)) >= size) {
            /* 这个地方需要判断 剩余的内存块是否可以申请 size 内存块 */
            if (mem->next - (ptr + SIZEOF_STRUCT_MEM) >= (size +
                SIZEOF_STRUCT_MEM + MIN_SIZE_ALIGNED)) {
                /* 上面注释一大堆，主要就是说，
                   剩余内存可能连一个内存块的头都放不下了，
                   这个时候就没法新建空内存块。其索引也就不能移动 */
                /* 指向申请后的位置，即：
                   建立下一个未使用的内存块的头部。
                   即：插入一个新空内存块 */
                ptr2 = (mem_size_t)(ptr + SIZEOF_STRUCT_MEM + size);
                /*从 Ptr2 地址开始创建 mem2 的结构体 */
                mem2 = ptr_to_mem(ptr2); /* 调用(struct mem *) (void *)&ram[ptr]; */
                mem2->used = 0;
                /* 这个根据下面的 if(mem2->next != MEM_SIZE_ALIGNED)判定 */
                mem2->next = mem->next;
                mem2->prev = ptr; /* 空闲内存块的前一个指向上面分配的内存块 */
                /* 前一个内存块指向上面建立的空闲内存块 */
                mem->next = ptr2;
                mem->used = 1; /* 将当前分配的内存块标记为 已使用 */
                /* 如果 mem2 内存块的下一个内存块不是链表中最后一个内存块（结束地址），
                   那就将它下一个的内存块的 prve 指向 mem2 */
                if (mem2->next != MEM_SIZE_ALIGNED) {
                    ((struct mem *) (void *)&ram[mem2->next])->prev = ptr2;
                }
            }
        }
    }
}
```

```

    } else { /* 内存块太小了会产生的碎片 */
        mem->used = 1;
    }

    /* 这里处理：当分配出去的内存正好是 lfree 时，
       因为该内存块已经被分配出去了，
       必须修改 lfree 的指向下一个最其前面的已释放的内存块*/
    if (mem == lfree) {
        struct mem *cur = lfree;
        /* 只要内存块已使用且没到结尾，则继续往后找 */
        while (cur->used && cur != ram_end) {

            cur = ptr_to_mem(cur->next); /* 下一个内存块 */
        }
        /* 指向找到的 第一个已释放的内存块。如果上面没有找到，则 lfree = lfree 不变 */
        lfree = cur;
    }
    /* 这里返回 内存块的空间的地址，排除内存块的头 */
    return (u8_t *)mem + SIZEOF_STRUCT_MEM + MEM_SANITY_OFFSET;
}

}

return NULL;
}
}

```

从上述源码可以看出，lwIP 内存堆申请内存时，是从低地址往高地址方向查找合适的内存块。每个内存块由两部分组成：一个与 struct mem 结构体大小相当的内存块，用于描述和管理可用内存块；另一个是可用内存块，用户可以直接对其进行操作。

根据图解 4.2.1，lfree 指针指向的是未被使用的控制块。当用户申请一定大小的内存时，lwIP 内核会将 lfree 指针指向的控制块标记为已用内存。随后，它会从当前控制块的地址开始，根据 size 大小和对齐需求进行偏移。偏移完成后，lfree 指针指向的地址处会附加一个新的 struct mem 结构体，作为下一个控制块。值得注意的是，这个新的控制块被初始化为未使用状态（used=0），下图是 lwIP 申请一块内存的示意图。

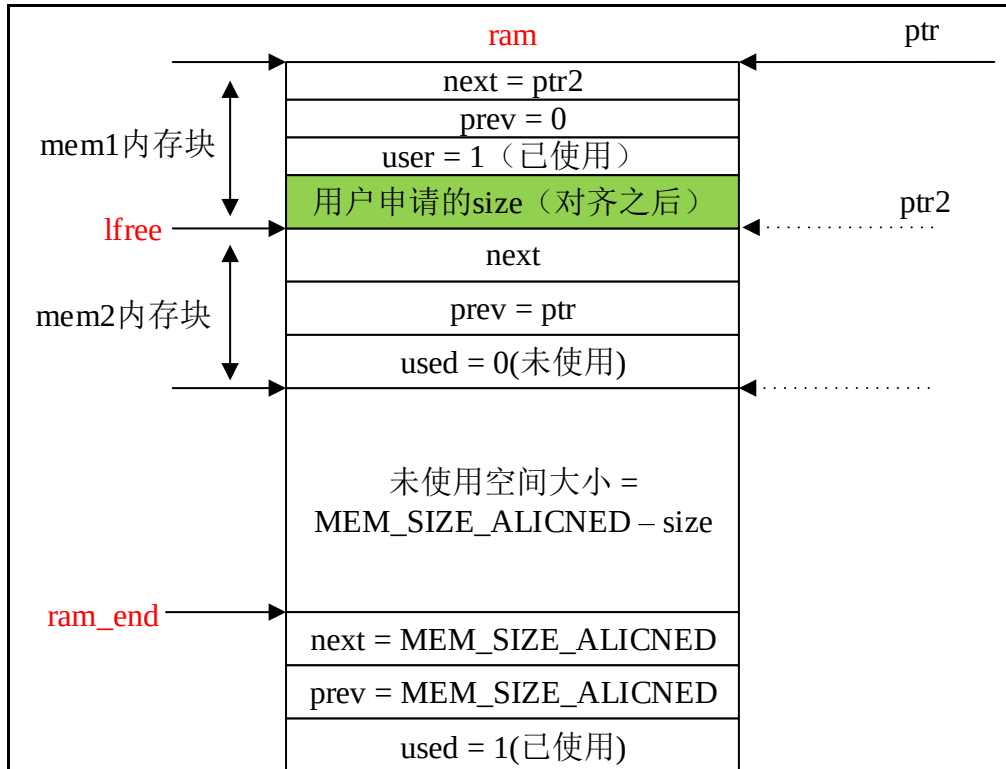


图 4.2.2 内存堆初始化第一次分配内存示意图

3. mem_free 函数

```
void
mem_free(void *rmem) {
    struct mem *mem;

    /* 第一步：检查内存块的参数 */
    /* 判断释放的内存块释放为空 */
    if (rmem == NULL) {
        return; /* 为空则返回 */
    }

    /* 除去指针就剩下内存块了，通过 mem_malloc 的到的地址是不含 struct mem 的 */
    mem = (struct mem *) (void *) ((u8_t *) rmem - (sizeof_STRUCT_MEM +
                                                    MEM_SANITY_OFFSET));

    /* 第二步：查找指定的内存块，标记为未使用 */
    mem->used = 0;

    /* 第三步：需要移动全局的释放指针，因为 lfree 始终指向内存堆中最小索引的
       那个已经释放的内存块 */
    if (mem < lfree) {
        /* 新释放的结构现在是最低的 */
        lfree = mem;
    }
}
```

lwIP 内存堆释放内存的过程相对简单，主要分为三个步骤。首先，系统会检测传入的地址是否正确。这一步是为了确保释放的内存块是有效且未被重复释放的。接下来，系统会对该

地址进行偏移操作，偏移大小为一个 struct mem 结构体的大小。通过偏移，可以获得释放内存的控制块首地址，并将该控制块设置为未使用状态。这一步的目的是将已使用的内存块标记为未使用，以便后续重新分配。最后，系统会判断该控制块的地址是否小于 lfree 指针指向的地址。如果是，这意味着释放的内存块位于 lfree 指针指向的内存块之前，更接近堆空间的起始地址。在这种情况下，系统会将 lfree 指针指向这个释放的内存块（控制块 + 可用内存）。这样，在后续申请内存时，系统会从 lfree 指针指向的内存块开始查找合适的内存。

值得注意的是，申请内存时，lwIP 内核会从 lfree 指针指向的内存块开始查找。如果该内存块不满足申请要求，系统会根据该内存块的 next 变量保存的数值作为下一跳查询的地址。这种机制允许系统按顺序遍历内存块，找到合适的内存进行分配。下图是内存堆释放内存示意图。

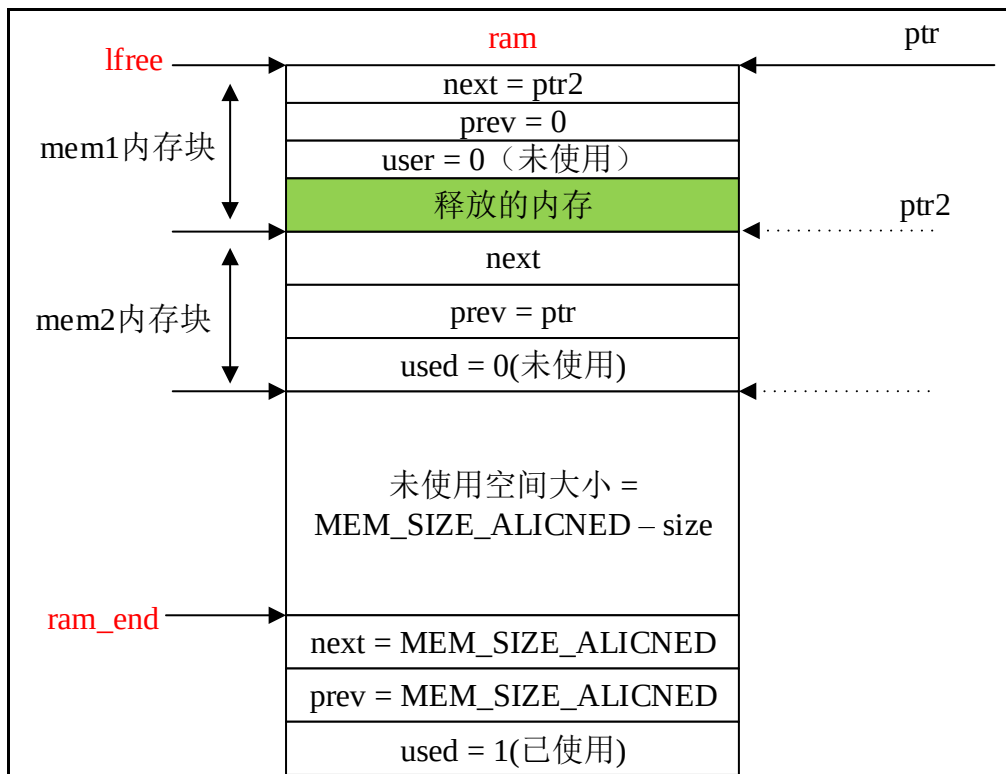


图 4.2.3 内存堆使用函数示意图

当系统需要申请内存，而当前 lfree 指针指向的内存块无法满足需求时，系统会根据该内存块的 next 变量所保存的数值作为下一跳查询的索引。这意味着系统会根据 next 变量所指示的内存块进行下一步的查找，以找到合适的内存进行分配。同时，需要注意的是，lfree 指针始终指向最低地址的内存空间。这意味着每次释放内存后，lfree 指针会更新为指向释放后的内存块，以便于下一次申请内存时从低地址开始查找。

4.3 动态内存池管理策略

内存池是一种有效的内存管理技术，它通过预先分配一定数量的内存块来满足程序中频繁的内存分配请求，提高了内存分配的速度。在内存池初始化时，系统将可用的内存划分为多个固定大小的内存块，并通过单链表的方式连接起来。当用户申请内存时，系统会从单链表的头部取出一个内存块进行分配，释放内存时只需将内存块放回链表的头部。这种分配方式具有高效的特点，但也可能导致资源的浪费。

在 lwIP 内存池的实现中，受到 MEMP_MEM_MALLOC 和 MEM_USE_POOLS 两个宏值

的限制。虽然这种技术有其优点和缺点，但它仍然是内存管理中的重要工具之一

1, IP 内存池的应用场景

在 lwIP 中，存在多种固定大小的数据结构，这些数据结构的特点是预先知道其大小并且在整个生命周期中保持不变。例如，在建立 TCP 连接时，需要使用 TCP 控制块这种数据结构，其大小是固定的。为了满足这些数据结构的内存分配需求，lwIP 在内存初始化时创建了动态内存池 POOL，预先分配一定数量的内存块。这种内存管理方式有助于提高内存分配的效率和性能。

2, IP 内存池实现的文件

对于内存堆的动态内存池分配，其实现涉及复杂的宏运用。为了更好地理解其原理，我们需要深入探究这四个关键文件：memp.c、memp.h、memp_std.h 和 memp_priv.h。接下来，我将详细解析每个文件的作用。

(1) memp_std.h 文件

该文件是 lwIP 内存池的核心定义，它针对 lwIP 内核所需的内存池进行定义。由于 lwIP 内核涉及多种固定数据结构，使用宏定义来确定是否启用特定类型的内存池，例如 TCP、UDP、DHCP、ICMP 等协议。这些宏定义通常在 lwippools.h 文件中进行声明启用。以下是该文件的源码示例：

```
#if LWIP_RAW
LWIP_MEMPOOL(RAW_PCB, MEMP_NUM_RAW_PCB, sizeof(struct raw_pcb), "RAW_PCB")
#endif /* LWIP_RAW */

#if LWIP_UDP
LWIP_MEMPOOL(UDP_PCB, MEMP_NUM_UDP_PCB, sizeof(struct udp_pcb), "UDP_PCB")
#endif /* LWIP_UDP */

#if LWIP_TCP
LWIP_MEMPOOL(TCP_PCB, MEMP_NUM_TCP_PCB, sizeof(struct tcp_pcb), "TCP_PCB")
LWIP_MEMPOOL(TCP_PCB_LISTEN, MEMP_NUM_TCP_PCB_LISTEN,
              sizeof(struct tcp_pcb_listen), "TCP_PCB_LISTEN")
LWIP_MEMPOOL(TCP_SEG, MEMP_NUM_TCP_SEG, sizeof(struct tcp_seg), "TCP_SEG")
#endif /* LWIP_TCP */

/* .....忽略以下源码.....
*/
```

从源码中可以发现两个关键点：首先，不同类型的内存池是通过相应的宏定义来声明启用的；其次，LWIP_MEMPOOL 宏定义用于初始化各种类型的内存池。这些宏定义和初始化过程对于实现 lwIP 内存池的内存管理至关重要。

(2) memp_priv.h 文件

```
/* 管理内存块 */
struct memp {
    struct memp *next;
};

/* 管理和描述各类型的内存池 */
```

```
struct memp_desc {
    /** 每个内存块的大小 */
    u16_t size;
    /** 内存块的数量 */
    u16_t num;
    /** 指向内存的基地址 */
    u8_t *base;
    /** 每个池的第一个空闲元素。元素形成一个链表 */
    struct memp **tab;
};
```

这个文件主要定义了两个核心结构体：`memp` 和 `memp_desc`。`memp` 结构体将同一类型的内存池以链表的形式连接起来，而 `memp_desc` 结构体则用于管理和描述各种类型的内存池，包括数量、大小、内存池的起始地址以及指向空闲内存池的指针。它们之间的关系可以通过下图进行可视化展示。

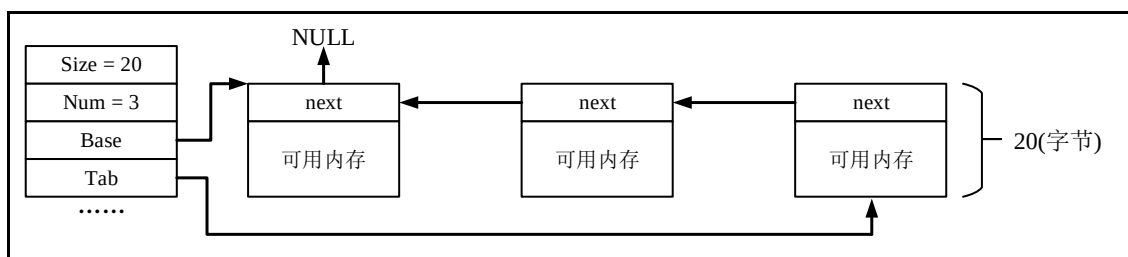


图 4.3.1 memp 和 memp_desc 结构体的关系

通过上图可以清晰地看到，每一个 `memp_desc` 结构体都是用于管理同一类型的内存池。这些内存池，也就是内存块，是通过链表的形式相互连接起来的。这样的设计使得内存管理更加高效和灵活。

(3) memp.h 文件

在 `memp.h` 文件中，我们重点关注了 `memp_t` 枚举类型以及 `LWIP_MEMPOOL_DECLARE` 宏定义。这些元素在内存池的声明和定义中起到了关键作用。`memp_t` 枚举类型用于描述内存池的类型，而 `LWIP_MEMPOOL_DECLARE` 宏则用于声明并初始化特定类型的内存池。它们共同协作，使 lwIP 能够有效地管理内存资源。

```
typedef enum {
    /** ##为C语言的连接符，例如 MEMP_##A, A = NAME, 所以等于 MEMP_NAME */
#define LWIP_MEMPOOL(name,num,size,desc) MEMP_##name,
#include "lwip/priv/memp_std.h"
    MEMP_MAX
} memp_t;

#include "lwip/priv/memp_priv.h" /* 该文件需要使用上面的枚举 */
#include "lwip/stats.h"
```

该文件主要定义了 `memp_t` 枚举类型，该类型用于获取各类内存池的数量。通过巧妙的宏运用，根据 `memp_std.h` 文件中启用的内存池类型来计算 `MEMP_MAX`，即各类内存池的最大数量。计算方法如下：

- 1, `LWIP_MEMPOOL` 宏定义指向 `MEMP_##name`（##为 C 语言中的连接符）。

2, 通过#include "lwip/priv/memp_std.h"文件来启用所需的内存池类型。

通过这些步骤, 可以准确地确定各类内存池的数量, 为内存管理提供必要的依据。

如果 memp_std.h 文件中仅启用了 LWIP_RAW 和 LWIP_UDP 类型的内存池, 则 MEMP_MAX 变量的值将为 2。以下是枚举类型的展开源码示例:

```
typedef enum {
    MEMP_RAW_PCB,
    MEMP_UDP_PCB,
    MEMP_MAX
} memp_t;
```

在枚举类型中, MEMP_RAW_PCB 对应的值为 0, MEMP_UDP_PCB 对应的值为 1, 以此类推。memp.h 文件的主要任务是计算各类内存池的数量, 并得出 MEMP_MAX 的数值。通过枚举类型的定义, 我们可以方便地管理和访问不同类型的内存池。

LWIP_MEMPOOL_DECLARE 宏是 lwIP 用于声明和初始化 lwIP 内存池的宏, 它通常在 memp.h 文件中使用, 用于定义不同类型的内存池。通过使用 LWIP_MEMPOOL_DECLARE 宏, 您可以声明一个内存池, 指定内存池中每个内存块的大小, 并定义内存池中的内存块数量。这个宏在编译时会自动生成相应的内存池结构和函数, 以便在运行时进行内存分配和释放。

LWIP_MEMPOOL_DECLARE 宏的内容如下所示:

```
#define LWIP_MEMPOOL_DECLARE(name,num,size,desc) \
    LWIP_DECLARE_MEMORY_ALIGNED(memp_memory_ ## name ## _base, \
                                ((num) * (MEMP_SIZE + MEMP_ALIGN_SIZE(size)))); \
    \
    LWIP_MEMPOOL_DECLARE_STATS_INSTANCE(memp_stats_ ## name) \
    \
    static struct memp *memp_tab_ ## name; \
    \
    const struct memp_desc memp_ ## name = { \
        DECLARE_LWIP_MEMPOOL_DESC(desc) \
        LWIP_MEMPOOL_DECLARE_STATS_REFERENCE(memp_stats_ ## name) \
        LWIP_MEM_ALIGN_SIZE(size), \
        (num), \
        memp_memory_ ## name ## _base, \
        &memp_tab_ ## name \
    };
```

LWIP_MEMPOOL_DECLARE 是一个关键的宏定义, 用于声明不同类型的内存池。它涉及到内存池的多个方面, 包括内存的来源、各类型内存池的数量、大小、内存来源的地址以及指向空闲内存的指针。通过使用这个宏定义, 您可以有效地管理内存池, 确保内存的合理分配和释放。以下是一个展开后的源码示例, 展示了如何使用 LWIP_MEMPOOL_DECLARE 宏定义:

```
#define LWIP_MEMPOOL_DECLARE(name,num,size,desc) \
u8_t memp_memory_ ## name ## _base[((((((num) * (MEMP_SIZE + (((size) + \
    MEM_ALIGNMENT - 1U) & ~(MEM_ALIGNMENT-1U)))) + MEM_ALIGNMENT - \
1U)))];\
static struct memp *memp_tab_ ## name;
```

```
const struct memp_desc memp_ ## name = { \
    LWIP_MEM_ALIGN_SIZE(size), \
    (num), \
    memp_memory_ ## name ## _base, \
    &memp_tab_ ## name \
};
```

通过展开 `LWIP_MEMPOOL_DECLARE` 宏定义，我们可以发现各类型内存池的内存来源与 lwIP 内存堆类似，都是由数组进行分配的。这个宏定义的具体使用将在 `memp.c` 文件中进行详细讲解。

(4) memp.c 文件

在深入探讨这个文件之前，我们需要了解 `LWIP_MEMPOOL` 和 `const memp_pools[MEMP_MAX]` 这两个核心概念。其中，`LWIP_MEMPOOL` 指向 `LWIP_MEMPOOL_DECLARE` 宏定义，该宏定义已在 `memp.h` 文件中展开，并将作为重点内容进行讲解。而 `const memp_pools[MEMP_MAX]` 数组则用于管理各类内存池的描述符。接下来，我们将分别深入探讨这两个部分的内容：

```
#define LWIP_MEMPOOL(name,num,size,desc) \

LWIP_MEMPOOL_DECLARE(name,num,size,desc)
#include "lwip/priv/memp_std.h"
```

同样，这里也展现了宏的巧妙运用。例如，当 `memp_std.h` 文件只启用 `LWIP_RAW` 和 `LWIP_UDP` 类型的内存池时，展开后的枚举类型如下：

```
u8_t memp_memory_RAW_PCB_base[((((num) * (MEMP_SIZE +
    (((size) + MEM_ALIGNMENT - 1U) & ~(MEM_ALIGNMENT-1U)))) +
    MEM_ALIGNMENT - 1U))];
static struct memp *memp_tab_RAW_PCB;
const struct memp_desc memp_RAW_PCB= { \
    LWIP_MEM_ALIGN_SIZE(size), \
    (num), \
    memp_memory_TCPIP_MSG_API_base, \
    &memp_tab_TCPIP_MSG_API \
};

u8_t memp_memory_UDP_PCB_base[((((num) * (MEMP_SIZE +
    (((size) + MEM_ALIGNMENT - 1U) & ~(MEM_ALIGNMENT-1U)))) +
    MEM_ALIGNMENT - 1U))];
static struct memp *memp_tab_UDP_PCB;

const struct memp_desc memp_UDP_PCB= { \
    LWIP_MEM_ALIGN_SIZE(size), \
    (num), \
    memp_memory_UDP_PCB_base, \
    &memp_tab_UDP_PCB \
};
```

这段代码通过使用 LWIP_MEMPOOL_DECLARE 宏定义，声明了各类内存池的描述和管理信息。具体来说，它定义了 memp_desc 结构体，例如 memp_RAW_PCB，用于描述该类型的内存池的数量、大小、分配内存地址以及指向空闲内存池的指针。

```
const struct memp_desc* const memp_pools[MEMP_MAX] = {
#define LWIP_MEMPOOL(name,num,size,desc) &memp_ ## name,
#include "lwip/priv/memp_std.h"
};
```

这个数组的大小由 MEMP_MAX 变量声明，该变量已经在前面的内容中进行了讲解。如果 memp_std.h 文件只启用了 LWIP_RAW 和 LWIP_UDP 类型的内存池，那么这个数组展开后如下所示：

```
const struct memp_desc* const memp_pools[MEMP_MAX] = {
    &memp_memp_RAW_PCB,
    &memp_memp_UDP_PCB,
};
```

这个数组的第一个元素是 memp_pools[MEMP_RAW_PCB]，它取自 memp_RAW_PCB 变量的地址。这是我们在前面展开 LWIP_MEMPOOL_DECLARE 宏定义时定义的变量。通过这种方式，我们可以方便地管理和访问不同类型的内存池，确保程序的稳定性和性能。

memp_init 函数和 memp_init_pool 函数

该函数是内存池的初始化，该函数如下所示：

```
void memp_init(void)
{
    ul6_t i;
    /* 遍历，需要多少个内存池 */
    for (i = 0; i < LWIP_ARRAYSIZE(memp_pools); i++) {
        memp_init_pool(memp_pools[i]);
    }
}

void memp_init_pool(const struct memp_desc *desc)
{
    int i;
    struct memp *memp;
    *desc->tab = NULL;
    /* 内存对齐 */
    memp = (struct memp*)LWIP_MEM_ALIGN(desc->base);
    /* 将内存块链接成链表形式 */
    for (i = 0; i < desc->num; ++i) {
        memp->next = *desc->tab;
        *desc->tab = memp;
        /* 地址偏移 */
        memp = (struct memp *) (void *) ((u8_t *)memp +
            MEMP_SIZE + desc->size);
    }
}
```

从上述源码中可以看出，每个类型的描述符都是用于管理和描述该类型的内存池。这些同一类型的内存池内部通过指向下一个节点的指针链接起来，形成一个链表。通过第二个 for 循环语句，我们可以遍历这些同一类型的内存池，并将其以链表的形式进行链接。经过不断循环，我们可以得到以下示意图：

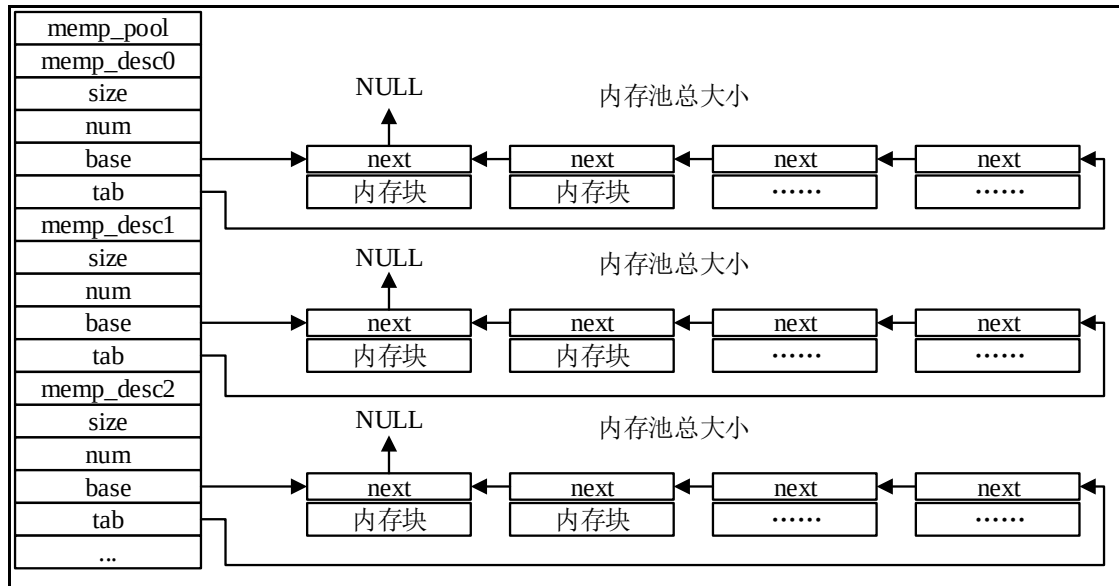


图 4.3.2 内存池初始化

根据上图，mempool 数组包含了不同类型的内存池描述符，每个描述符负责管理同类型的内存池。这些内存池通过指针链接成一个单向链表，方便管理和访问。同一类型的内存池都在同一个数组中分配，通过 base 指针可以找到该数组的首地址。tab 指针指向第一个空闲的内存池，当用户申请内存池时，将从 tab 指针指向的内存池进行分配。分配完成后，tab 指针将偏移到下一个空闲内存池的地址，以便下次分配。这种链表结构使得内存分配和释放更加高效，提高了程序的稳定性和性能。

memp_malloc 函数和 mempool_malloc_pool 函数

在前面讲解中提到，内存池有多种类型，因此用户在申请内存池时需要明确申请的类型。lwIP 内存池的申请函数是 mempool_malloc，其函数定义如下：

```
void *
memp_malloc(memp_t type)
{
    void *memp;
    mempool_malloc_pool(memp_pools[type]);
    return mempool;
}

static void*
memp_malloc_pool(const struct mempool_desc *desc)
{
    struct mempool *memp;
    mempool = *desc->memp;
}
```

```
if (memp != NULL)
{
    *desc->tab = memp->next;
    return ((u8_t*)memp + MEMP_SIZE);
}
else
{
    return NULL;
}
```

memp_malloc 函数根据用户传入的内存池类型，如 UDP_PCB 等，在 memp_pool 数组中查找对应的内存池描述符。一旦找到对应的描述符，该函数会根据描述符中的 tab 指针来分配内存给用户，并将 tab 指针偏移至下一个空闲内存池。这样，每次内存分配都会从当前空闲内存池开始，并更新 tab 指针以指向下一个空闲内存池，实现了高效的内存分配和管理。下图是 lwIP 内存池申请示意图。

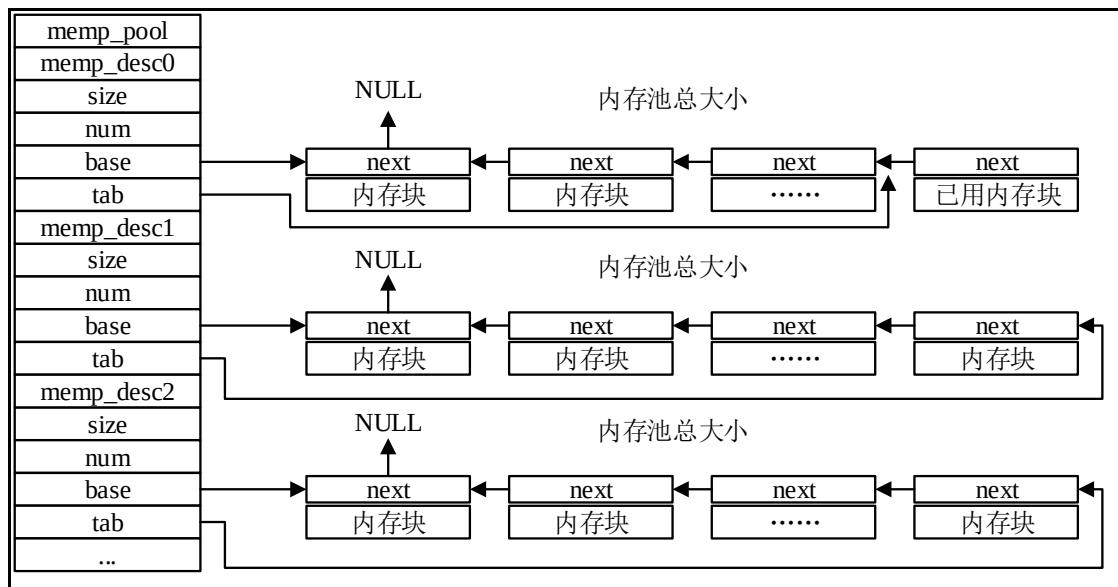


图 4.4.2 内存池中申请内存块

memp_free 函数与 mem_free_pool 函数

内存池的释放函数相对简单，它需要传入两个参数：内存池的类型和要释放的内存池的地址。通过这两个参数，lwIP 内核可以确定该类型内存池描述符的位置，以及需要释放的内存池的具体位置。内存池释放函数的定义如下：

```
void memp_free(memp_t type, void *mem)
{
    if (mem == NULL) /* 判断内存块的起始地址释放为空 */
    {
        return;
    }

    do_memp_free_pool(memp_pools[type], mem);
}
```



```

}

static void do_memp_free_pool(const struct memp_desc* desc, void *mem)
{
    struct memp *memp;

    /* 据内存块的地址偏移得到内存块的起始地址 */
    memp = (struct memp *) (void *) ((u8_t*)mem - MEMP_SIZE);

    /* 内存块的下一个就是链表中的第一个空闲内存块 */
    memp->next = *desc->tab;

    /* *desc->tab 指向 memp 内存块中 */
    *desc->tab = memp;
}

```

释放函数很简单，只需要将内存池描述符的 tab 指针偏移至要释放的内存池。这样，释放的内存块就会被返回到相应的内存池中，以供后续使用。释放流程如下图所示。

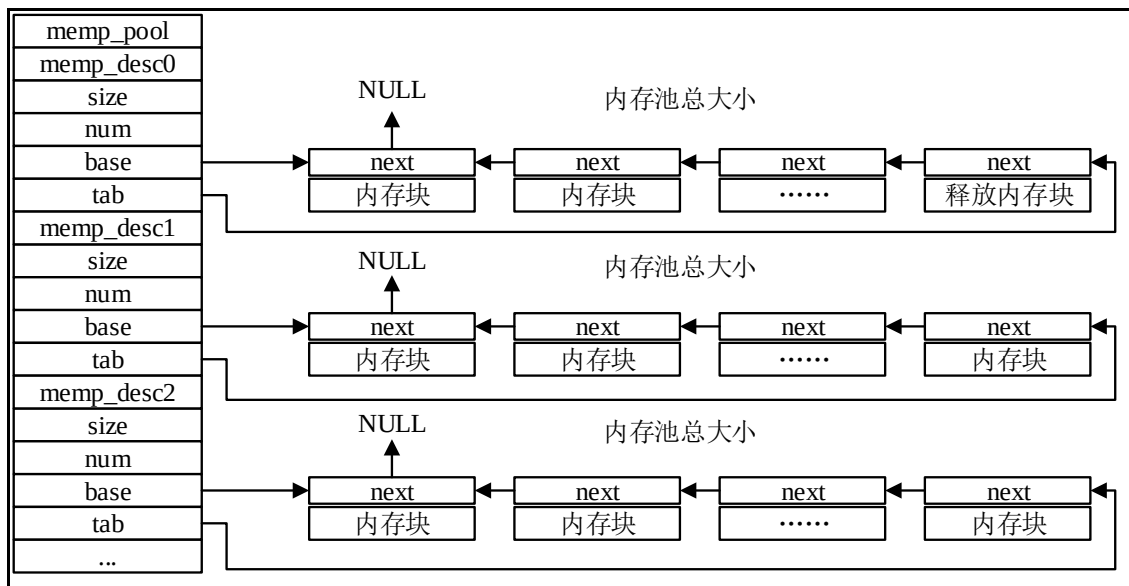


图 4.2.4 内存池释放内存

4.4 使用 C 库管理内存策略

lwIP 内核支持 C 标准库管理策略，但与 lwIP 内存堆管理策略只能选其一。在 mem.c 文件中，我们可以找到 MEM_LIBC_MALLOC 配置项的源码示例如下：

```

/* in case C library malloc() needs extra protection,
 * allow these defines to be overridden.
 */

#ifndef mem_clib_free
#define mem_clib_free    free
#endif

#ifndef mem_clib_malloc
#define mem_clib_malloc  malloc
#endif

```

```
#ifndef mem_clib_malloc
#define mem_clib_malloc      malloc
#endif

#if LWIP_STATS && MEM_STATS
#define MEM_LIBC_STATSHELPER_SIZE LWIP_MEM_ALIGN_SIZE(sizeof(mem_size_t))
#else
#define MEM_LIBC_STATSHELPER_SIZE 0
#endif
```

上述的 `free`、`malloc` 和 `calloc` 函数是 C 标准库中用于内存管理的函数。然而，需要注意的是，C 标准库的内存管理策略不能合并相邻的空闲内存块，容易导致内存碎片化。因此，在使用 C 标准库进行内存管理时，需要注意内存碎片化问题，并采取相应的措施进行优化。相比之下，lwIP 内存堆管理策略可以合并相邻的空闲内存块，减少内存碎片化，提高内存利用率。

第五章 lwIP 网络接口管理

lwIP 支持多网口设计，通过使用 `netif` 结构来描述每种网络接口的特性，如 IP 地址和接口状态等。为了有效管理所有网络接口，协议栈内部采用了一个名为 `netif` 的网络接口结构来描述各种网络设备。当项目中使用多个网卡时，lwIP 通过 `netif_list` 链表来管理这些网卡。该链表负责管理多个 `netif` 网络接口，以便在发送数据时从链表中选出一个合适的网卡进行数据传输。简而言之，lwIP 通过 `netif` 结构体和 `netif_list` 链表实现了对多网口的有效管理，确保了数据传输的准确性和高效性。

5.1 网络接口结构 `netif`

5.1 网络接口结构 `netif`

众所周知，网卡的种类繁多，而 lwIP 是如何兼容众多网卡的呢？lwIP 通过使用一个数据结构——`netif` 来描述一个网卡，并提供了统一接口函数来管理这些网卡。由于网卡与硬件紧密相关，不同硬件的处理方式各异，因此 lwIP 的底层函数需要用户自行完成，例如网卡的初始化、接收和发送数据等。当 lwIP 底层接收到网络数据时，需要经过层层递交才会传入内核处理；相反地，lwIP 发送数据时也会调用网卡的发送函数。对于没有接触过 lwIP 的学员来说，如何编写底层驱动呢？lwIP 已经做得很好，它提供了一个 `ethernetif.c` 文件作为底层接口的驱动模板。用户只需根据自己的网络设备进行参照修改即可。

在 lwIP 中，`netif` 数据结构用于描述一个网络接口的特性。这个结构体是在 `netif.h` 文件中定义的，如下所示：

```
struct netif {
    /* 指向下一个 netif 结构的指针 */
    struct netif *next;

    /* IP 地址相关配置 */
    ip_addr_t ip_addr;      /* 网络接口的 IP 地址 */
    ip_addr_t netmask;      /* 子网掩码 */
    ip_addr_t gw;          /* 网关地址 */

    /* 该函数向 IP 层输入数据包 */
    netif_input_fn input;

    /* 该函数发送 IP 包 */
    netif_output_fn output;

    /* 该函数实现底层数据包发送 */
    netif_linkoutput_fn linkoutput;

    /* 该字段用户可以自由设置，例如用于指向一些底层设备相关的信息 */
    void *state;
}
```

```
void* client_data[LWIP_NETIF_CLIENT_DATA_INDEX_MAX +
                  LWIP_NUM_NETIF_CLIENT_DATA];

/* 该接口允许的最大数据包长度 */
u16_t mtu;

/* 该接口物理地址长度 */
u8_t hwaddr_len;

/* 该接口的物理地址 */
u8_t hwaddr[NETIF_MAX_HWADDR_LEN];

/* 该接口的状态、属性字段 */
u8_t flags;

/* 该接口的名字 */
char name[2];

/* 接口的编号 */
u8_t num;

/* 需要发送的路由器请求消息的数量 */
u8_t rs_count;
};
```

该结构体包含了多个字段，这些字段的作用如下：

- (1) next: 该字段指向下一个 netif 类型的结构体，因为 lwIP 可以支持多个网络接口，当设备有多个网络接口的话 lwIP 就会把所有的 netif 结构体组成链表来管理这些网络接口。
- (2) ipaddr, netmask 和 gw: 分别为网络接口的 IP 地址、子网掩码和默认网关。
- (3) input: 此字段为一个函数，这个函数将网卡接收到的数据交给 IP 层。
- (4) output: 此字段为一个函数，当 IP 层向接口发送一个数据包时调用此函数。这个函数通常首先解析硬件地址，然后发送数据包。此字段我们一般使用 etharp.c 中的 etharp_output() 函数。
- (5) linkoutput: 此字段为一个函数，该函数被 ARP 模块调用，完成网络数据的发送。上面说的 etharp_output 函数将 IP 数据包封装成以太网数据帧以后就会调用 linkoutput 函数将数据发送出去。
- (6) state: 用来定义一些关于接口的信息，用户可以自行设置。
- (7) mtu: 网络接口所能传输的最大数据长度，一般设置为 1500。
- (8) hwaddr_len: 网卡 MAC 地址长度，6 个字节。
- (9) hwaddr: MAC 地址。
- (10) flags: 网络的接口状态，属性信息字段。
- (11) name: 网卡的名字。
- (12) num: 编号从 0 开始，此字段为协议栈为每个网络接口设置的一个编号。
- (13) rs_count: 发送的路由器请求消息的数量。

这些字段用于描述各个网卡的差异，每个网卡都通过一个 netif 结构体进行抽象表示。多个网卡对应多个 netif，这些 netif 以链表的形式链接起来，形成一个单向链表。

这个链表的首个节点由 netif_list 指针指向，lwIP 内核使用该指针遍历和查询 netif 链表。管理和描述该链表涉及三个全局变量，如下所示：

```
struct netif *netif_list;      /* 网络接口链表指针 */
struct netif *netif_default;  /* 哪个网络接口 (多网口时候) */
static u8_t netif_num;        /* 为网口分配唯一标识 */
```

在 lwIP 中，netif.c 文件包含了管理网络接口的核心函数。其中，netif_default 指针指向默认的网络接口，当网络层需要发送数据包时，系统优先选择该指针所指向的网卡进行数据发送。如果该网卡没有响应，则会选择其他网卡进行发送。接下来，我将重点介绍几个重要的函数：

(1) netif_add 函数

该函数用于将新创建的 netif 插入到 netiflist 队列中，以表示添加了一个网络接口。以下是函数的示例代码：

```
struct netif *
netif_add( struct netif *netif,
           const ip4_addr_t *ipaddr, const ip4_addr_t *netmask,
           const ip4_addr_t *gw,
           void *state, netif_init_fn init, netif_input_fn input)
{

    /* 清空主机 IP 地址、子网掩码、网关等信息。 */
    ip_addr_set_zero_ip4(&netif->ip_addr);
    ip_addr_set_zero_ip4(&netif->netmask);
    ip_addr_set_zero_ip4(&netif->gw);
    netif->output = netif_null_output_ip4;
    /* 传输的最大数据长度 */
    netif->mtu = 0;
    /* 网络的接口状态 */
    netif->flags = 0;

    memset(netif->client_data, 0, sizeof(netif->client_data));

    /* 传递进来的参数填写网卡 state、input 等字段的相关信息 */
    netif->state = state;
    /* 并为当前网卡 分配唯一标识 num */
    netif->num = netif_num;
    /* 网卡输入 */
    netif->input = input;

    /* 调用网卡设置函数 netif_set_addr() 设置网卡 IP 地址、子网掩码、网关 */
    netif_set_addr(netif, ipaddr, netmask, gw);

    /* 为 netif 调用用户指定的初始化函数 */
    if (init(netif) != ERR_OK) {
        return NULL;
    }

    /* 将这个 netif 添加到列表中 */
    netif->next = netif_list;
    netif_list = netif;
}
```

```
mib2_netif_added(netif);
netif_invoke_ext_callback(netif, LWIP_NSC_NETIF_ADDED, NULL);
return netif;
}
```

根据上述源码，可以得出结论：每一个 netif 结构体是对一个网卡进行抽象，包含了该网卡的收发函数、状态等信息。根据上述函数的运行流程，可以得到以下示意图：

① 只有一个网络接口

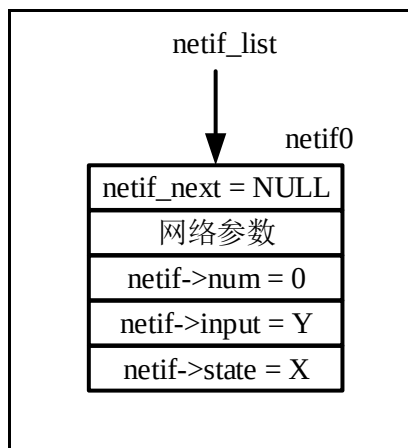


图 5.1.1 只有一个网卡的链表示意图

② 两个网络接口

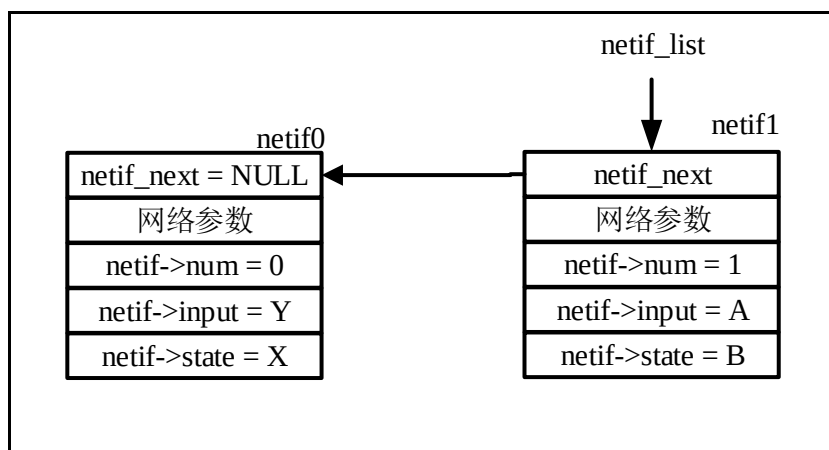


图 5.1.2 多个网卡的链表结构

注：新插入的 netif 结构体是在 netiflist 队列的首部插入。

(2) netif_set_default 函数

该函数用于将指定的 netif 结构体设置为默认网卡，使 lwIP 内核优先对该网卡进行操作。

以下是函数的示例代码：

```
void
netif_set_default(struct netif *netif)
{
    if (netif == NULL) {
        /* 删除默认路由 */
        mib2_remove_route_ip4(1, netif);
    } else {
```

```
/* 添加默认路由 */
mib2_add_route_ip4(1, netif);
}

netif_default = netif; /* 选择那个网络接口 */
}

/*****怎么使用函数 netif_set_default()*****/
/* 通过该函数，将网络接口添加到链表中 */
netif_add(&xnetif, &ipaddr, &netmask, &gw, NULL, &ethernetif_init,
&tcpip_input);
/* 注册默认的网络接口 */
netif_set_default(&xnetif);
```

通过调用该函数，可以设置指定的网络接口为默认网卡。在函数内部，会更新相应的默认网卡指针，并确保后续的网络操作优先使用该网卡进行数据传输。

第六章 lwIP 网络数据包解析

TCP/IP 协议的核心是对数据包的处理。为了提高数据包处理效率，lwIP 提供了一种高效的数据包管理机制。这种机制使得各层之间能够灵活操作数据包，同时避免了在各层之间复制数据的开销，减少了传递时间。类似于 Linux 的 BSD 协议中的 mbuf 结构体，lwIP 采用了类似的结构名为 pbuf。pbuf 的数据包种类和大小多样，从网卡读取的数据包可以是几个字节的 IP 数据报，也可以是一千个字节。这些数据包可能存在于 RAM 和 ROM 中，具体取决于用户配置。为了高效处理数据，lwIP 需要对这些数据进行统一管理。

本章分为如下几个部分：

6.1 TCP/IP 协议各层间的操作

6.2 lwIP 的线程简介

6.3 pbuf 结构

6.1 TCP/IP 协议各层间的操作

传统的 TCP/IP 协议各层之间是独立存在的，每一层只处理该层的数据，不能越界读写数据。如果按照这种严格的分层模式来实现 TCP/IP 协议，会导致数据包在各层间的递交变得非常慢，涉及一系列的内存拷贝问题，进而影响系统总体性能。因此，lwIP 内部并没有采用完整的分层结构，而是假设各层间的部分数据结构和实现原理在其他层是可见的。这样在数据包递交过程中，各层协议可以直接对数据包中属于其他层次协议的字段进行操作，从而提高了数据包处理的效率。

从上述可以看出，lwIP 的优点有以下几个：

- ① 不需要数据层层拷贝。
- ② 用户程序可以直接访问内部各层数据包。
- ③ 各个层次之间存在交叉存取数据的现象，既节省系统的空间也节省处理的时间，而且更加灵活。
- ④ lwIP 的内存共享机制，使得应用程序能直接对协议栈内核的内存区域直接操作，减少时间和空间的损耗。

6.2 lwIP 的线程简介

操作系统中的任务创建与管理是常见的技术，如果将协议栈各层作为独立的任务或线程，虽然编程简便、代码组织灵活，但会导致各层之间严格分层。这种模式下，数据递交需要进行拷贝和频繁的任务切换，可能导致用户程序无法准时处理。一个数据包在各层次间的递交至少需要进行 3 次任务切换，导致协议栈效率低下。

另一种方法是将协议栈与操作系统结合，使协议栈成为操作系统的一部分。这样用户任务与协议栈通过操作系统的 API 函数实现，提高了效率，各层也可以交叉存取。但协议栈与操作系统的融合会导致严重的后果，降低操作系统的实时性。

lwIP 采用了协议栈与操作系统相互隔离的方法，不影响操作系统的实时性。协议栈作为操作系统的一个独立任务，有两种方法实现用户与协议栈之间的数据交互：一是让用户程序驻留在协议栈任务中，通过回调函数实现数据交互；二是用户程序作为操作系统的独立任务，与协议栈任务通过 IPC 通信机制交互。这种在 lwIP 中称为 RAW API 和 NETCONN API 以及 Socket API 编程。

6.3 pbuf 结构

lwIP 使用 pbuf 对数据进行发送与接收，其灵活的 pbuf 结构体可以减少数据在不同层之间传输时的内存开销和内存复制所占用的时间，从而提高数据在不同层之间传递的速度并节约内存。在 lwIP 源码中，关于 pbuf 的文件是 pbuf.c 和 pbuf.h。pbuf 结构体的源码定义如下：

```
struct pbuf {  
    /* pbuf 链表中指向下一个 pbuf 结构 */  
    struct pbuf *next;  
    /* 数据指针，指向该 pbuf 所记录的数据区域 */  
    void *payload;  
    /* 当前 pbuf 及后续所有 pbuf 中所包含的数据总长度 */  
    u16_t tot_len;  
    /* 当前 pbuf 中数据的长度 */  
    u16_t len;  
    /* 当前 pbuf 的类型 */  
    u8_t type;  
    /* 状态位未用到 */  
    u8_t flags;  
    /* 指向该 pbuf 的指针数，即该 pbuf 被引用的次数 */  
    LWIP_PBUF_REF_T ref;  
    /* 对于传入的数据包，它包含输入 netif 的索引 */  
    u8_t if_idx;  
};
```

pbuf 结构体具有多个字段，这些字段的作用如下所示：

参数	描述
next	这个指向下一个 pbuf 结构
payload	数据指针，指向该 pbuf 管理的数据起始地址
totlen	当前 pbuf 有效数据的总长度
type	字段表示 pbuf 的类型，具体来说 pbuf 有四种类型
	PBUF_RAM pbuf 数据存储在内存堆
	PBUF_ROM pbuf 数据存储在 ROM 中
	PBUF_REF pbuf 数据存储在 RAM 中
flags	PBUF_POOL pbuf 有数据存储在内存池
	这里 lwIP 暂时没有用到。
ref	pbuf 被引用的次数。

表 6.3.1 结构体 pbuf 的参数描述

从上文可以看出，pbuf 具有四种类型，它们的数据存储在不同的区域。以下是这四种类型 pbuf 的详细描述。

(1) PBUF_RAM 类型

PBUF_RAM 是 lwIP 中最常用的 pbuf 类型。当协议栈需要发送数据时，通常会采用这种形式。协议栈会在内存堆中为 PBUF_RAM 类型的 pbuf 分配相应空间，包括 pbuf 结构本身和相应的数据缓冲区。这些空间在物理存储上是连续的，以确保高效的内存访问。分配完成后，pbuf 的结构如下图所示。

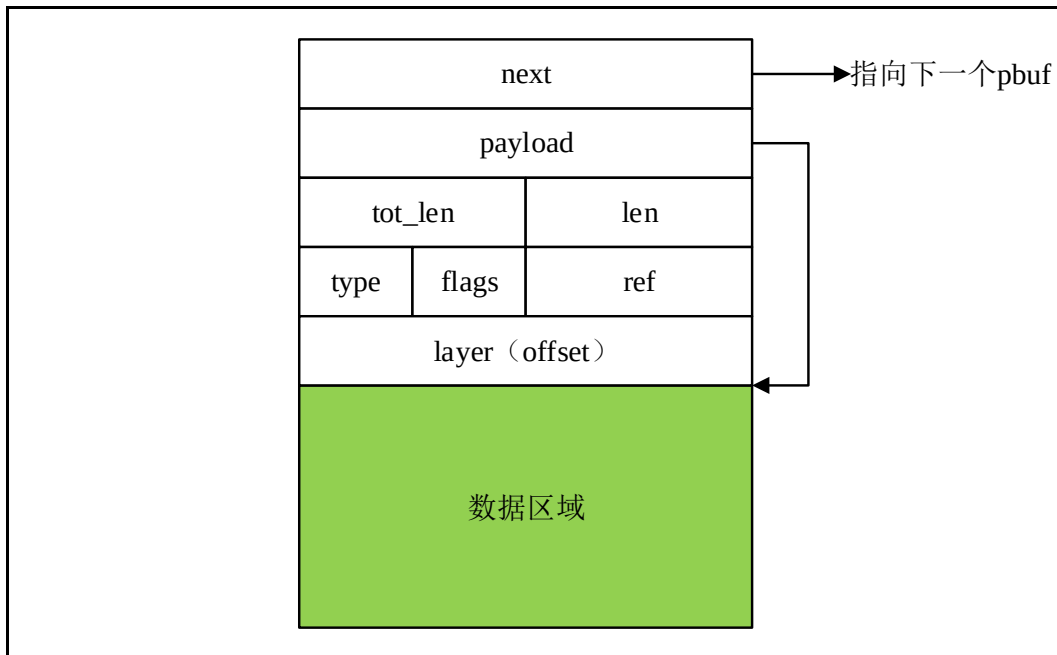


图 6.3.1 PBUF_RAM 类型的 pbuf

注：payload 指针并不一定指向数据区域的首地址，而是可以设定一定的 offset 偏移量。这种偏移量常用于存储 TCP 报文首部、IP 首部等。此外，layer 的大小可以为 0，具体取决于数据包的申请方式。

(2) PBUF_POOL 类型

PBUF_POOL 类型的 pbuf 与 PBUF_RAM 类型的 pbuf 在许多方面相似，但它们在空间分配方式上存在显著差异。PBUF_POOL 类型的 pbuf 通过内存池分配空间，这意味着它们可以在极短的时间内快速分配。在网卡接收数据包时，通常使用这种类型的 pbuf 来包装或存储接收到的数据。此外，在系统初始化阶段，还会初始化两个与数据报 pbuf 密切相关的内存池，如下源码所示：

```
LWIP_PBUF_MEMPOOL(PBUF, MEMP_NUM_PBUF, 0, "PBUF_REF/ROM")
LWIP_PBUF_MEMPOOL(PBUF_POOL, PBUF_POOL_SIZE, PBUF_POOL_BUFSIZE, "PBUF_POOL")
```

内存池是一个固定大小的内存块，用于存储用户数据。当用户数据超过内存池的大小时，lwIP 内核会使用多个内存池来存储这些数据。存储完成后，系统将多个 pbuf 以链表的形式链接起来，形成一个单向链表，如下图所示。

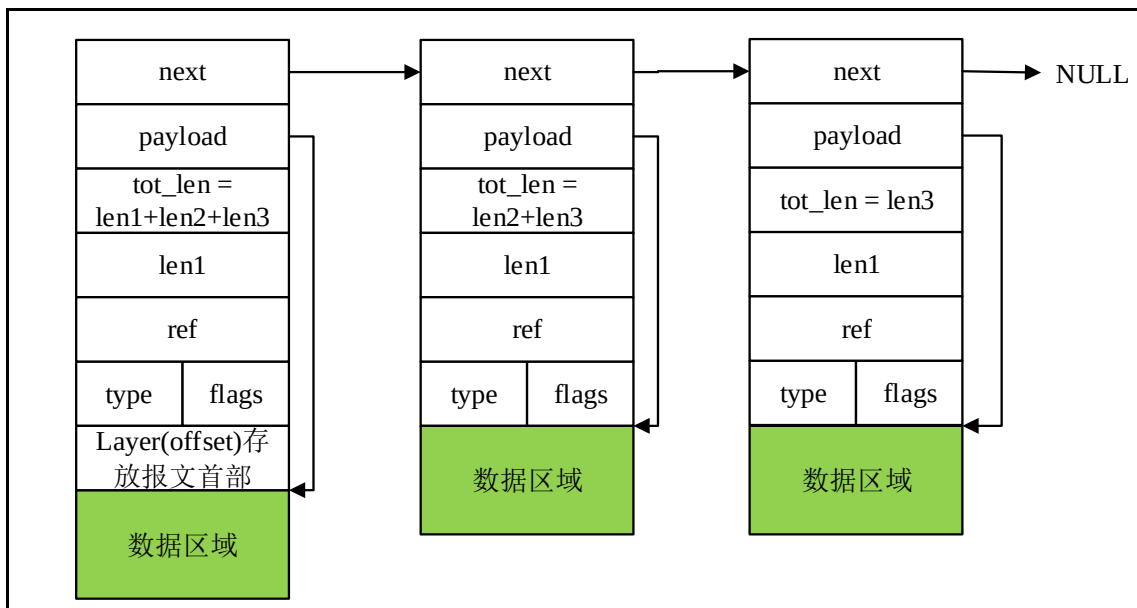


图 6.3.2 PBUF_POOL 类型 pbuf（组成 pbuf 链表）

(3) PBUF_ROM和和 PBUF_REF 类型

PBUF_ROM 和 PBUF_REF 是另外两种类型的 pbuf，它们在内存池中分配相应的 pbuf 结构，但并不申请数据区的空间。两者之间的区别在于 PBUF_ROM 指向 ROM 空间内的数据，而 PBUF_REF 则指向 RAM 空间内的某段数据。在发送某些静态数据时，可以采用这两种类型的 pbuf，这有助于大大节省协议栈的内存空间。结构如下图所示。

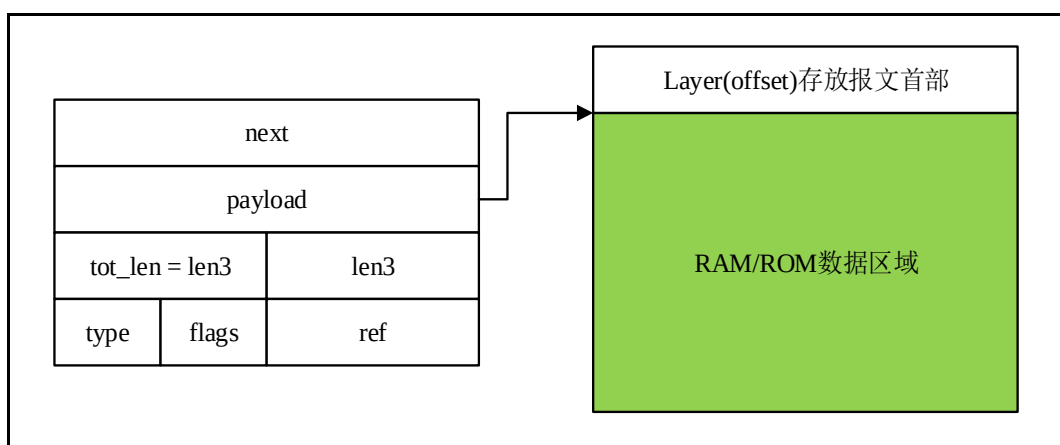


图 6.3.3 PBUF_ROM 和 PBUF_REF 类型 pbuf

对于一个数据包，可以使用上述任意的 pbuf 类型来描述。一个数据包的数据可以由一大串不同类型的 pbuf 组成，这些 pbuf 可以连续存储，形成一个链表。这样的设计使得 lwIP 能够灵活地处理不同类型的数据包，并优化内存的使用。结构如下图所示。

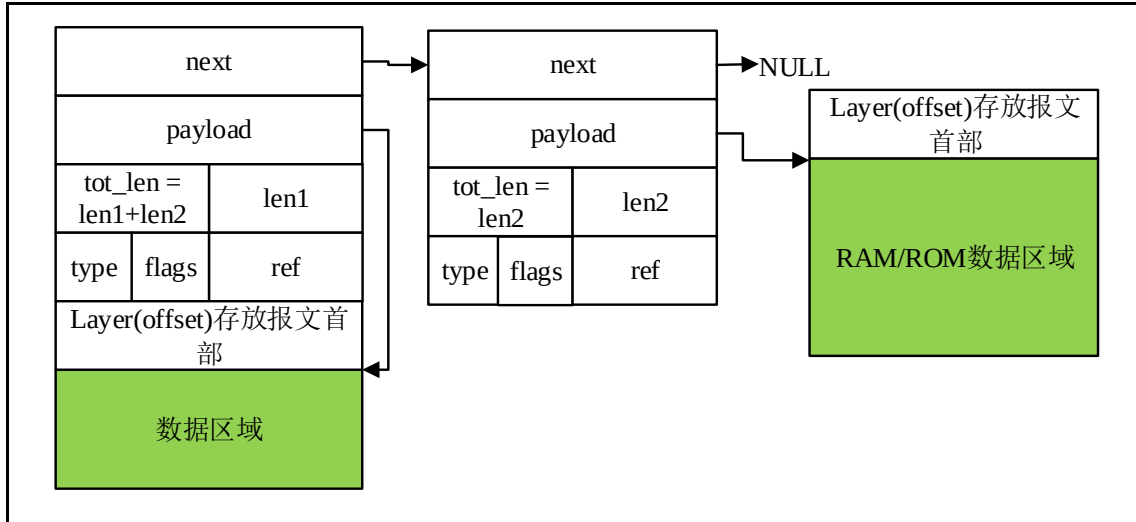


图 6.3.4 多种类型的组合分配 pbuf

lwIP 网络数据包 pbuf 提供了 5 个函数，这些函数如下所示：

函数	描述
pbuf_alloc()	pbuf 的申请
pbuf_free()	pbuf 的释放
pbuf_realloc()	相应 pbuf（链表）尾部释放一定的空间
pbuf_header()	调整 pbuf 的 payload 指针（向前或向后移动一定字节数）
pbuf_take()	于向 pbuf 的数据区域拷贝数据

表 6.3.2 pbuf 相关的函数描述

1. pbuf_alloc 函数

该函数根据类型、大小和偏移来申请 pbuf 空间，若该函数返回 NULL，则申请失败。

```
static void
pbuf_init_allocated_pbuf(struct pbuf *p, void *payload, u16_t tot_len, u16_t len,
                        pbuf_type type, u8_t flags)
{
    p->next = NULL; /* 指向 NULL */
    p->payload = payload; /* 指向数据区域 */
    p->tot_len = tot_len; /* 总长度 */
    p->len = len; /* 该 pbuf 长度 */
    p->type_internal = (u8_t)type; /* 申请的 pbuf 类型 */
    p->flags = flags; /* 状态位 */
    p->ref = 1; /* 指向该 pbuf 的指针数，即该 pbuf 被引用的次数 */
    p->if_idx = NETIF_NO_INDEX; /* 对于传入的数据包，它包含输入 netif 的索引 */
}

struct pbuf *
pbuf_alloc(pbuf_layer layer, u16_t length, pbuf_type type)
{
    struct pbuf *p;
    u16_t offset = (u16_t)layer; /* 申请那个层的首部 */
    /* 判断以太网首部 */
}
```

```

switch (type)
{
    case PBUF_REF: /* 失败 */
    case PBUF_ROM:
        p = pbuf_alloc_reference(NULL, length, type);
        break;

    case PBUF_POOL:
    {
        struct pbuf *q, *last;
        u16_t rem_len; /* 总大小 */
        p = NULL;
        last = NULL;
        rem_len = length; /* rem_len 赋值为总长度 */

        do
        {
            u16_t qlen; /* 减去首部的长度 */
            /* 申请内存池 */
            q = (struct pbuf *)memp_malloc(MEMP_PBUF_POOL);

            if (q == NULL) /* 申请内存池失败 */
            {
                PBUF_POOL_IS_EMPTY();

                if (p)
                {
                    pbuf_free(p);
                }

                return NULL;
            }

            /* 总长度减去 offset (首部大小) 并赋值给 qlen (去除首部的长度)
               LWIP_MIN(x, y) ((x) < (y)) ? (x) : (y) */
            qlen = LWIP_MIN(rem_len, (u16_t)(PBUF_POOL_BUFSIZE_ALIGNED -
                                             LWIP_MEM_ALIGN_SIZE(offset)));
            /* 分配后初始化 struct pbuf 成员 */
            pbuf_init_allocated_pbuf(q, LWIP_MEM_ALIGN((void *)((u8_t *)q +
                                                                    sizeof_struct_pbuf + offset)),
                                     rem_len, qlen, type, 0);

            if (p == NULL) /* 第一次分配 p 必定指向 NULL */

```

```

        {
            /* pbuf 链分配头 */
            p = q;
        }
        else

            /* 让前面的 pbuf 指向这个 pbuf */
            last->next = q;
        }

        last = q;
        /* 判断是否还有剩余长度 */
        rem_len = (u16_t)(rem_len - qlen);
        offset = 0;

    } while (rem_len > 0); /* 如果有剩余，还需要执行一次 do 语句 */

    break;
}

case PBUF_RAM:
{
    u16_t payload_len = (u16_t)(LWIP_MEM_ALIGN_SIZE(offset) +
                                LWIP_MEM_ALIGN_SIZE(length));

    mem_size_t alloc_len =
        (mem_size_t)(LWIP_MEM_ALIGN_SIZE(sizeof_struct_pbuf) + payload_len);

    if ((payload_len < LWIP_MEM_ALIGN_SIZE(length)) ||
        (alloc_len < LWIP_MEM_ALIGN_SIZE(length)))
    {
        return NULL;
    }

    /* 如果要在 RAM 中分配 pbuf，请为它分配内存。 */
    p = (struct pbuf *)mem_malloc(alloc_len);

    if (p == NULL)
    {
        return NULL;
    }

    pbuf_init_allocated_pbuf(p, LWIP_MEM_ALIGN((void *)((u8_t *)p +
                                                sizeof_struct_pbuf + offset)),

```

```

                                length, length, type, 0);

        break;
    }

    default:
        return NULL;
}

return p;
}

```

此函数首先根据 `type` 的值判断申请 `pbuf` 的类型，并执行相应的代码段。`layer` 变量用于让 `pbuf` 中的 `payload` 指针偏移，以便在 `lwIP` 网络数据包中添加各层首部。通过这种方式，`lwIP` 能够灵活地处理不同类型的数据包，并优化内存的使用。

2. `pbuf_free` 函数

此函数是对各类型的数据包 `pbuf` 进行释放，该函数实现原理如下所示：

```

u8_t
pbuf_free(struct pbuf *p)
{
    u8_t alloc_src;
    struct pbuf *q;
    u8_t count;

    /* 如果数据包为空则返回 0 */
    if (p == NULL)
    {
        return 0;
    }

    PERF_START;

    count = 0;

    /* 判断数据包不为空 */
    while (p != NULL)
    {
        LWIP_PBUF_REF_T ref;
        SYS_ARCH_DECL_PROTECT(old_level);
        SYS_ARCH_PROTECT(old_level);
        /* 减少引用计数 (指向 pbuf 的指针数) */
        ref = --(p->ref);
        SYS_ARCH_UNPROTECT(old_level);

        if (ref == 0)

```

```
{
    /* 为了下一次迭代，请记住链中的下一个 pbuf */
    q = p->next;
    alloc_src = pbuf_get_allocsrc(p);
#ifdef LWIP_SUPPORT_CUSTOM_PBUF

    /* is this a custom pbuf? */
    if ((p->flags & PBUF_FLAG_IS_CUSTOM) != 0)
    {
        struct pbuf_custom *pc = (struct pbuf_custom *)p;
        pc->custom_free_function(p);
    }
    else
#endif /* LWIP_SUPPORT_CUSTOM_PBUF */
    {
        /* 判断释放的内存池的类型 */
        if (alloc_src == PBUF_TYPE_ALLOC_SRC_MASK_STD_MEMP_PBUF_POOL)
        {
            memp_free(MEMP_PBUF_POOL, p);
            /* is this a ROM or RAM referencing pbuf? */
        }
        else if (alloc_src == PBUF_TYPE_ALLOC_SRC_MASK_STD_MEMP_PBUF)
        {
            memp_free(MEMP_PBUF, p);
            /* type == PBUF_RAM */
        }
        else if (alloc_src == PBUF_TYPE_ALLOC_SRC_MASK_STD_HEAP)
        {
            mem_free(p);
        }
        else
        {
        }
    }

    count++;
    /* 继续到下一个 pbuf */
    p = q;
}
else
{
    p = NULL;
}
```



```
}  
    return count;  
}
```

此函数主要涉及两个部分：一是减 1 操作 `ref` 参数并释放内存，二是处理多 `pbuf` 链表的数据包。

3. `pbuf_realloc` 函数

该函数通过释放 `pbuf` 链表的尾部空间，将数据包 `pbuf` 的数据长度减少到特定长度值。需要注意的是，该函数仅修改 `pbuf` 中的长度字段值，并不实际释放对应的内存池空间。

4. `pbuf_header` 函数

该函数用于调整 `pbuf` 的 `payload` 指针，使其向前或向后移动一定字节数。通过调用 `pbuf_header` 函数，`payload` 指针可以指向数据区前的首部字段，为各层对数据包首部的操作提供便利。在进行指针调整时，`len` 和 `tot_len` 字段值也会相应更改。

5. `pbuf_take` 函数

`pbuf_copy` 函数用于将任意类型的 `pbuf` 中的数据拷贝到 `PBUF_RAM` 类型的 `pbuf` 中。
`pbuf_chain` 函数用于连接两个 `pbuf`（链表）为一个 `pbuf` 链表。`pbuf_ref` 函数用于将 `pbuf` 中的引用计数加 1。

第七章 lwIP 简介

在前面的章节中，我们简要探讨了 lwIP 的内存管理、网络接口和网络数据包等核心概念。这些内容为我们揭示了 lwIP 内部结构的基础知识。然而，这只是 lwIP 这座知识冰山的一角。为了真正掌握 lwIP，我们不仅要学习如何使用它，更要深入理解其实现原理。接下来，我们将一起揭开 lwIP 的神秘面纱，深入探索其核心框架、超时处理和数据收发机制。通过这些内容，我们将建立起对 lwIP 牢固的知识基础，为进一步理解和应用打下坚实基础。

本章分为如下几个部分：

7.1 网络接口接收数据

7.2 lwIP 超时处理

7.3 tcpip_thread（协议栈）线程

7.4 lwIP 中的消息

7.1 网络接口接收数据

STM32 通过 ETH 接口接收数据，并在产生 ETH 中断时释放信号量（s_xSemaphore），以通知网络接口任务（ethernetif_input）处理接收的数据。该任务将数据封装成消息并传递给 tcpip_mbox 邮箱，再通过邮箱发送消息。lwIP 内核中的协议栈线程负责接收 tcpip_mbox 邮箱的消息，并根据消息类型解析处理。处理前，它会判断消息类型，并根据类型执行相应的代码段。下图是网络接口接收数据示意图。

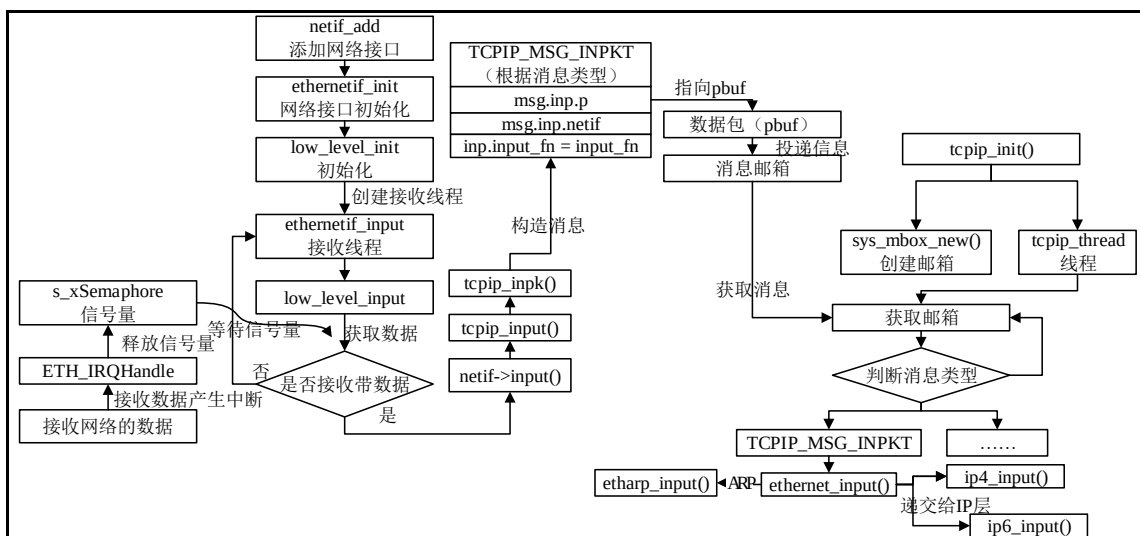


图 7.1.1 带操作系统的 lwIP 收发示意图

从上图可以看出，ethernetif_input 是一个任务函数，用于接收 ETH 中断释放的信号量。当接收到信号量时，它调用 low_level_input 函数获取描述符管理缓冲区的数据，并使用 tcpip_input 函数构造消息，再通过 tcpip_mbox 邮箱发送消息。在初始化时，lwIP 内核创建了 TCP/IP 线程，该线程负责接收 tcpip_mbox 邮箱的消息，并根据消息类型进行解析处理。在处理之前，它会判断消息类型，并根据类型执行相应的代码段。

7.2 lwIP 超时处理

在 lwIP 中很多时候会使用到超时处理，超时处理的实现是 TCP/IP 协议栈中一个重要部分。

它为每个与外界网络连接的任务都设定了 `timeout` 属性，即等待超时时间。

lwIP 为什么需要超时定时器

lwIP 中为什么需要做超时处理呢？这可从其实现的 TCP/IP 协议栈功能可以知道，TCP 的建立连接超时、重传超时机制，IP 分片数据报的重装等待超时，ARP 缓存表项的时间管理、ping 接收数据包超时处理等等，都需要使用超时操作来处理。超时处理的相关代码在 `timeouts.c/h` 中实现，下面笔者分别地讲解这两个文件的内容。

(1) `timeouts.h` 文件

该文件主要定义了两个结构体，它们分别为 `lwip_cyclic_timer` 和 `sys_timeo`，第一个结构体定义了超时等待时间和超时处理函数，另外一个管理这些超时的定时器，着两个结构体的原型如下所示：

1. `lwip_cyclic_timer` 结构体：

```
struct lwip_cyclic_timer
{
    u32_t interval_ms;                /* 超时间隔 */
    lwip_cyclic_timer_handler handler; /* 超时处理 */
};

const struct lwip_cyclic_timer lwip_cyclic_timers[] = {
    {TCP_TMR_INTERVAL, HANDLER(tcp_tmr)},
    {IP_TMR_INTERVAL, HANDLER(ip_reass_tmr)},
    {ARP_TMR_INTERVAL, HANDLER(etharp_tmr)},
    {DHCP_COARSE_TIMER_MSECS, HANDLER(dhcp_coarse_tmr)},
    {DHCP_FINE_TIMER_MSECS, HANDLER(dhcp_fine_tmr)},
    {AUTOIP_TMR_INTERVAL, HANDLER(autoip_tmr)},
    {IGMP_TMR_INTERVAL, HANDLER(igmp_tmr)},
    {DNS_TMR_INTERVAL, HANDLER(dns_tmr)},
};
```

可以看到，`interval_ms` 变量就是超时等待时间，而 `handler` 就是超时处理函数，即超时事件，若超时了，则触发一个超时事件。`lwip_cyclic_timers` 数组就是定义了 lwIP 内核所需的超时定时器，即超时事件。

2. `sys_timeo` 结构体：

```
typedef void (* sys_timeout_handler)(void *arg);

struct sys_timeo
{
    struct sys_timeo *next;          /* 下一个超时事件的指针 */
    u32_t time;                      /* 当前超时事件的等待时间 */
    sys_timeout_handler h;           /* 指向超时的回调函数 */
    void *arg;                      /* 超时的回调函数参数 */
};
```

这个结构体是用来管理这些超时事件，它的 `next` 指针指向下一个超时事件，最后这些超时事件形成了单向链表。这些超时事件都调用同一的超时回调函数，这个函数由 `h` 函数指针指向，最后根据 `arg` 回调函数形参来调用哪个超时事件处理。注：`time` 变量等于系统节拍加上超

时等待时间，例如系统当前节拍是 1s，超时定时器的等待时间为 5s，所以系统在节拍等于 6s 时才执行超时事件。

(2) timeouts.c 文件

1. 注册超时事件：

lwip_cyclic_timers 保存了 lwIP 所需的超时事件，这些超时事件由 sys_timeouts_init 函数插入到超时链表当中，该函数如下所示：

```
void sys_timeouts_init(void)
{
    size_t i;

    for (i = (LWIP_TCP ? 1 : 0); i < LWIP_ARRAYSIZE(lwip_cyclic_timers); i++)
    {
        sys_timeout(lwip_cyclic_timers[i].interval_ms, cyclic_timer,
                    LWIP_CONST_CAST(void*, &lwip_cyclic_timers[i]));
    }
}
```

此函数很简单，获取 lwip_cyclic_timers 元素地址和等待超时时间之后调用 sys_timeout 函数把超时事件插入到超时链表当中。sys_timeout 函数如下所示：

```
#define LWIP_MAX_TIMEOUT 0x7fffffff
/* 当前插入超时事件时间与 next_timeout 指向超时事件时间对比是否大于 0x7fffffff
   如果 t - compare_to 为负值的话，由于类型为 u32_t 所以导致该值比 0x7fffffff 大，
   如果比 LWIP_MAX_TIMEOUT 大则为 1，否则为 0*/
#define TIME_LESS_THAN(t, compare_to) ( (((u32_t)((t)-(compare_to))) > \
                                         LWIP_MAX_TIMEOUT) ? 1 : 0 )

void
sys_timeout(u32_t msecs, sys_timeout_handler handler, void *arg)
{
    u32_t next_timeout_time;
    /* 由 TIME_LESS_THAN 宏处理的溢出 */
    next_timeout_time = (u32_t)(sys_now() + msecs);
    sys_timeout_abs(next_timeout_time, handler, arg);
}
```

从这里可以看出，next_timeout_time 变量等于系统当前节拍加上某个超时事件的等待时间，其实 next_timeout_time 变量最终赋给 sys_timeo 结构体下的 time 成员变量。这个超时事件由这个 sys_timeout_abs 函数插入到超时链表当中，该函数如下所示：

```
static void
sys_timeout_abs(u32_t abs_time, sys_timeout_handler handler, void *arg)
{
    struct sys_timeo *timeout, *t;
    /* 申请节点内存 */
    timeout = (struct sys_timeo *)memp_malloc(MEMP_SYS_TIMEOUT);
    if (timeout == NULL) { /* 申请内存失败直接返回 */
        return;
    }
```

```

/* 节点各变量赋值 */
timeout->next = NULL;
timeout->h = handler;
timeout->arg = arg;
/* abs_time = (u32_t)(sys_now() + msecs) */
timeout->time = abs_time;
/* 如果创建的是第一个定时器，则不用特殊处理，
   next_timeout 是一个全局指针，指向定时器链表中第一个定时器 */
if (next_timeout == NULL) {
    next_timeout = timeout;
    return;
}
/* 如果新添加的定时器小于当前链首定时器的时长， 则进入该代码段*/
if (TIME_LESS_THAN(timeout->time, next_timeout->time)) {
    timeout->next = next_timeout;
    next_timeout = timeout;
}
else
{
    for (t = next_timeout; t != NULL; t = t->next) {
        if ((t->next == NULL) || TIME_LESS_THAN(timeout->time, t->next->time)) {
            timeout->next = t->next;
            t->next = timeout;
            break;}}}
}

```

首先此函数为超时事件申请内存，以内存池的方式申请，接着对超时事件各个成员变量赋值，可以看到 h 函数指针指向超时回调函数，arg 指针指向 lwip_cyclic_timers 数组的某个元素地址，超时回调函数就是根据 arg 形参来运行某个超时事件，time 变量等于了 next_timeout_time 变量即当前系统节拍加上超时等待时间，最后插入到超时链表当中。下面笔者使用几个示意图来讲解这个函数，如下所示：

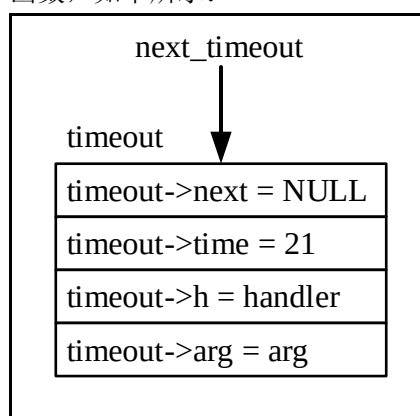


图 7.2.1 插入一个节点的示意图

从上图可以知道，该超时事件的 time 等于 21 即当前系统节拍加上超时事件等待函数，它的 next 指针指向为 NULL，因为一开始这个超时链表没有超时事件，所以 next_timeout 指向新插入的超时事件。

当我们插入第二个超时事件时，系统需要逐一判断这个超时事件的 `time` 是否大于超时链表挂载的超时事件 `time`，逐一对比之后发送插入的超时事件 `time` 比超时链表挂载的超时事件 `time` 要大，则系统把这个超时事件插入这张链表的尾部。如下图所示。

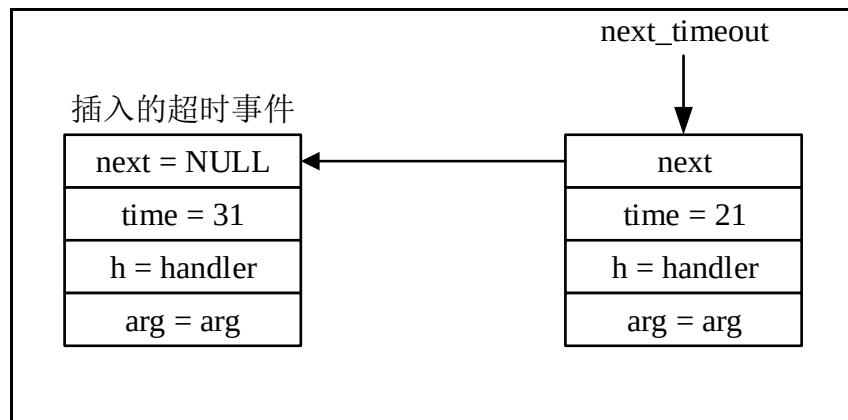


图 7.2.2 插入第二个节点的示意图

如果插入的超时事件 `time` 与挂载超时链表的超时事件 `time` 对比之后，发现插入的超时事件 `time` 在两个挂载的超时事件 `time` 之间即 $a < \text{time} < b$ ，那么要插入的超时事件挂载至这两个挂载的超时事件中间，如下图所示。

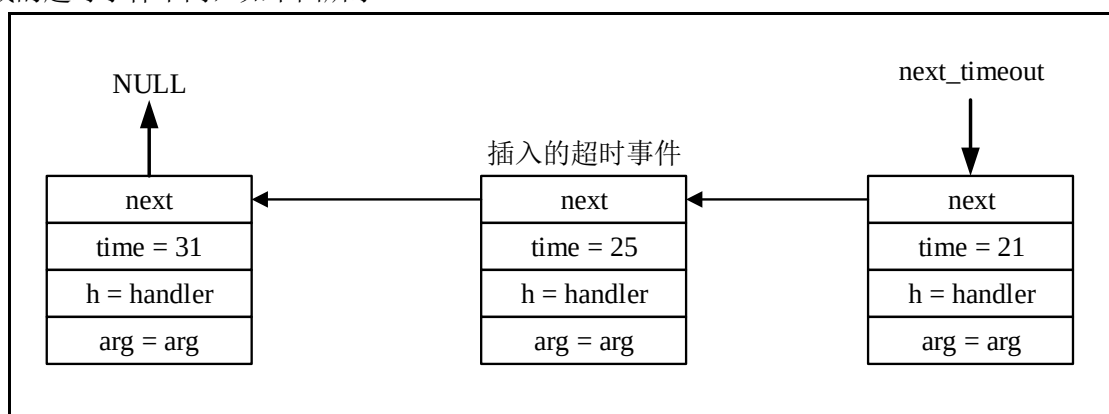


图 7.2.3 插入第三个节点示意图

2. 删除超时事件:

从超时事件链表中删除一个超时事件可调用 `sys_untimeout` 函数删除，如下源码所示：

```

void
sys_untimeout(sys_timeout_handler handler, void *arg)
{
    struct sys_timeo *prev_t, *t;
    /* 从链表头开始遍历这个链表 */
    for (t = next_timeout, prev_t = NULL; t != NULL; prev_t = t, t = t->next)
    {
        /* 查找删除的超时事件，判断超时事件的回调函数与函数参数是否一致 */
        if ((t->h == handler) && (t->arg == arg))
        {
            if (prev_t == NULL)
            {

```

```

        next_timeout = t->next;
    }
    else
    {
        prev_t->next = t->next;

        memp_free(MEMP_SYS_TIMEOUT, t);
        return;
    }
}
return;
}

```

此函数非常简单，只需遍历这个超时链表，在遍历过程中判断超时事件的回调函数与函数参数是否一致，若一致，则对超时链表的超时事件排序，排序完成之后调用 `memp_free` 删除这个超时事件。

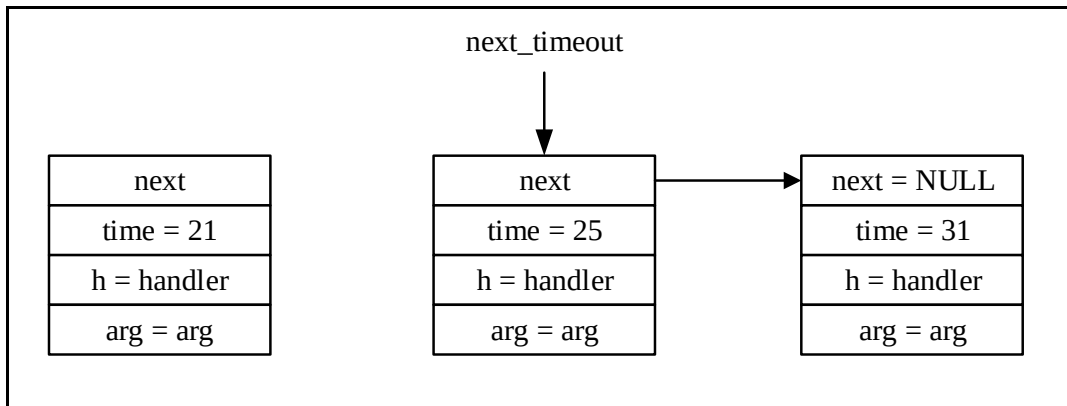


图 7.2.4 从超时链表删除超时事件

超时定时器检查：

不管是 OS 的还是裸机的都可以对其进行超时检查和处理，lwIP 使用两个函数来实现超时检查处理。

1. void sys_check_timeouts(void)函数

这个函数是用于裸机部分的，用户可以在裸机的应用中周期性调用该函数，每次进来检查定时器链表上定时最短的定时器是否到期，如果没有到期，直接退出该函数，否则，执行该定时器回调函数，并从链表上删除该定时器，然后继续检查下一个定时器，直到没有一个定时器到期退出。

2. tcpip_timeouts_mbox_fetch(sys_mbox_t *mbox, void **msg)函数

这个函数在 OS 线程中循环执行的，主要等待 mbox 消息并可阻塞，如果等待 mbox 时超时，则会同时执行超时事件处理，即调用超时回调函数，否则一直没有收到 mbox 消息就会一直等待直到下一个超时时间并循环将所有超时定时器检查一遍(内部调用了 `void sys_check_timeouts(void)`)，lwIP 中 tcpip 线程就是靠这种方法，即处理了上层及底层的 mbox 消息，同时处理了所有需要定时处理的事件。

7.3 tcpip_thread（协议栈）线程

这个线程由 tcpip_init 函数创建，该函数如下所示：

```
void
tcpip_init(tcpip_init_done_fn initfunc, void *arg)
{
    lwip_init();

    tcpip_init_done = initfunc;
    tcpip_init_done_arg = arg;
    if (sys_mbox_new(&tcpip_mbox, TCPIP_MBOX_SIZE) != ERR_OK) {
        LWIP_ASSERT("failed to create tcpip_thread mbox", 0);
    }
#ifdef LWIP_TCPIP_CORE_LOCKING
    if (sys_mutex_new(&lock_tcpip_core) != ERR_OK) {
        LWIP_ASSERT("failed to create lock_tcpip_core", 0);
    }
#endif /* LWIP_TCPIP_CORE_LOCKING */

    sys_thread_new(TCPIP_THREAD_NAME, tcpip_thread, NULL,
                  TCPIP_THREAD_STACKSIZE, TCPIP_THREAD_PRIO);
}
```

这个函数在 lwip_init 函数调用，它负责几个任务，第一、创建邮箱为数据传输准备，第二、创建互斥锁为防止优先级翻转问题，第三、创建 TCP/IP 线程。下面笔者重点讲解 tcpip_thread 任务函数的实现源码，如下所示：

```
static void
tcpip_thread(void *arg)
{
    struct tcpip_msg *msg;
    LWIP_UNUSED_ARG(arg);

    LWIP_MARK_TCPIP_THREAD();

    LOCK_TCPIP_CORE();

    if (tcpip_init_done != NULL)
    {
        tcpip_init_done(tcpip_init_done_arg);
    }

    while (1)
    {
        LWIP_TCPIP_THREAD_ALIVE();
        /* 第一步：等待消息时，将在等待时处理超时 */
    }
}
```



```

    /* TCPIP_MBOX_FETCH 的宏定义为 sys_timeouts_mbox_fetch
       等待消息并且处理超时事件 */
    TCPIP_MBOX_FETCH(&tcPIP_mbox, (void **) &msg);

    if (msg == NULL) /* 如果没有等到消息就继续等待 */
    {
        continue;
    }

    tcPIP_thread_handle_msg(msg);
}

static void
tcPIP_thread_handle_msg(struct tcPIP_msg *msg)
{
    /* 第二步：等待到消息就对消息进行处理 */
    /* 不同类型进行不同的处理 */
    switch (msg->type)
    {
#ifdef LWIP_TCPIP_CORE_LOCKING
        /* 执行对应的 API 函数 */
        case TCPIP_MSG_API:
            msg->msg.api_msg.function(msg->msg.api_msg.msg);
            break;

        case TCPIP_MSG_API_CALL:
            msg->msg.api_call.arg->err =
            msg->msg.api_call.function(msg->msg.api_call.arg);
            sys_sem_signal(msg->msg.api_call.sem);
            break;
#endif /* !LWIP_TCPIP_CORE_LOCKING */

#ifdef LWIP_TCPIP_CORE_LOCKING_INPUT
        /* 直接交给 ARP 层处理 */
        case TCPIP_MSG_INPKT:

            if (msg->msg.inp.input_fn(msg->msg.inp.p,
                msg->msg.inp.netif) != ERR_OK)
            {
                pbuf_free(msg->msg.inp.p);
            }

```

```

        memp_free(MEMP_TCPIP_MSG_INPKT, msg);
        break;
#endif /* !LWIP_TCPIP_CORE_LOCKING_INPUT */

#if LWIP_TCPIP_TIMEOUT && LWIP_TIMERS
    /* 注册一个超时事件 */
    case TCPIP_MSG_TIMEOUT:
        sys_timeout(msg->msg.tmo.msecs, msg->msg.tmo.h, msg->msg.tmo.arg);
        memp_free(MEMP_TCPIP_MSG_API, msg);
        break;

    /* 删除一个超时事件 */
    case TCPIP_MSG_UNTIMEOUT:
        sys_untimeout(msg->msg.tmo.h, msg->msg.tmo.arg);
        memp_free(MEMP_TCPIP_MSG_API, msg);
        break;
#endif /* LWIP_TCPIP_TIMEOUT && LWIP_TIMERS */

    /* 通过回调方式执行一个回调函数
    他们的回调函数相同 */
    case TCPIP_MSG_CALLBACK:
        msg->msg.cb.function(msg->msg.cb.ctx);
        memp_free(MEMP_TCPIP_MSG_API, msg);
        break;

    case TCPIP_MSG_CALLBACK_STATIC:
        msg->msg.cb.function(msg->msg.cb.ctx);
        break;

    default:
        break;
}
}

```

协议栈线程主要负责接收邮箱的消息、递交数据至网络层、遍历超时链表等任务。

7.4 lwIP 中的消息

在上一个小节笔者讲解了 `tcpip_thread` 线程的作用，其中接收邮箱的消息到底如何构建，这里涉及到 lwIP 数据包消息机制，它专门把 ETH 中断接收的数据封装成消息，以邮箱的方式发送至 `tcpip_thread` 线程处理，注：这里以带操作系统为例。

7.4.1 数据包消息(tcpip_msg)

```

/* 7 种 tcpip_msg 消息类型 */
enum tcpip_msg_type {
    TCPIP_MSG_API,          /* 用户调用应用层的接口时，就属于 API 消息类型 */
    TCPIP_MSG_API_CALL,    /* API 函数调用 */

```

```

TCPIP_MSG_INPKT,                /* 底层数据包输入 */
TCPIP_MSG_TIMEOUT,              /* 注册超时事件 */
TCPIP_MSG_UNTIMEOUT,           /* 删除超时事件 */
TCPIP_MSG_CALLBACK,            /* 执行回调函数 */
TCPIP_MSG_CALLBACK_STATIC      /* 执行静态回调函数 */
};

/* tcpip_msg 结构体 */
struct tcpip_msg {
    /* tcpip_msg 消息的类型 */
    enum tcpip_msg_type type;
    /* 消息内容,共用体,不同消息类型使用不同的结构 */
    union {

        struct {
            /* 内核执行函数 */
            tcpip_callback_fn function;
            /* 执行函数的参数 */
            void* msg;
        } api_msg;

        struct {
            /* 回调函数 */
            tcpip_api_call_fn function;
            /* 回调函数的参数 */
            struct tcpip_api_call_data *arg;
            /* 用户同步的信号量 */
            sys_sem_t *sem;
        } api_call;

        struct {
            /* 接收的数据包 */
            struct pbuf *p;
            /* 接收的数据包的接口 */
            struct netif *netif;
            /* 输入的函数接口 */
            netif_input_fn input_fn;
        } inp;

        struct {
            /* tcpip 回调函数 */
            tcpip_callback_fn function;
            /* 回调函数参数 */

```

```

    void *ctx;
} cb;

struct {
    /* 超时时间 */
    u32_t msecs;
    /* 超时执行的回调函数 */
    sys_timeout_handler h;
    /* 传入超时回调函数的形参 */
    void *arg;
} tmo;
#endif /* LWIP_TCPIP_TIMEOUT && LWIP_TIMERS */
} msg;
};

```

上述的源码中，我们可以看到消息结构的 `msg` 字段是一个共用体 `union`，共用体中定义了各类型消息的具体内容，每种类型的消息对应了共用体中的一个字段，其中注册超时事件和删除超时事件消息共用一个 `tmo` 结构体；回调事件与静态回调事件消息也共用一个 `cb` 结构体；API 调用与 NETIF 的 API 调用相关的消息具体内容比较多，不宜直接放在 `tcpip_msg` 中，系统用了专门的结构体 `api_msg` 来描述对应消息的具体内容。注：`tcpip_msg` 中只保存了一个指向 `api_msg` 指针。

`tcpip_thread` 线程处理每种类型的消息时，lwIP 内核就会产生与之对应的消息函数，首先产生的消息传递到系统邮箱（`tcpip_mbox`），`tcpip_thread` 线程需要判断该消息的类型，从而做出相应的处理，在图 7.1.1 中，笔者大概描述了 lwIP 接收数据的流程图，直观上它是通过函数 `tcpip_input` 对消息进行构造和投递，当然该函数真正执行的是函数 `tcpip_inpkt`，如下源码所示：

```

err_t
tcpip_input(struct pbuf *p, struct netif *inp)
{
    #if LWIP_ETHERNET

        if (inp->flags & (NETIF_FLAG_ETHARP | NETIF_FLAG_ETHERNET))
        {
            /* 把 ethernet_input() 作用该函数的一部分
               内核接收到这个数据包就调用该函数 */
            return tcpip_inpkt(p, inp, ethernet_input);
        }
        else
    #endif /* LWIP_ETHERNET */
        return tcpip_inpkt(p, inp, ip_input);
}

err_t
tcpip_inpkt(struct pbuf *p, struct netif *inp, netif_input_fn input_fn)
{

```

```
#if LWIP_TCPIP_CORE_LOCKING_INPUT
    err_t ret;
    ret = input_fn(p, inp);
    return ret;
#else /* LWIP_TCPIP_CORE_LOCKING_INPUT */
    struct tcpip_msg *msg;
    msg = (struct tcpip_msg *)memp_malloc(MEMP_TCPIP_MSG_INPKT);

    if (msg == NULL)
    {
        return ERR_MEM;
    }

    msg->type = TCPIP_MSG_INPKT;
    msg->msg.inp.p = p;                /* 指向 pbuf 数据包 */
    msg->msg.inp.netif = inp;          /* 网络接口 */
    /* 构造消息，消息的类型是数据包消息，处理函数是 ethernet_input() */
    msg->msg.inp.input_fn = input_fn;

    if (sys_mbox_trypost(&mbox, msg) != ERR_OK) /* 构造消息完成，发送邮箱 */
    {
        memp_free(MEMP_TCPIP_MSG_INPKT, msg); /* 释放内存池 */
        return ERR_MEM;
    }

    return ERR_OK;
#endif /* LWIP_TCPIP_CORE_LOCKING_INPUT */
}
```

总的来说，接收数据都是通过函数 `ethernet_input`，当然无操作系统也是如此，只不过就是传递消息的方式不同，无操作系统一般使用回调函数传递消息，而操作系统一般使用 IPC 通信机制，例如邮箱，信号量等通信机制，lwIP 的 IPC 通讯示意图如下图所示。

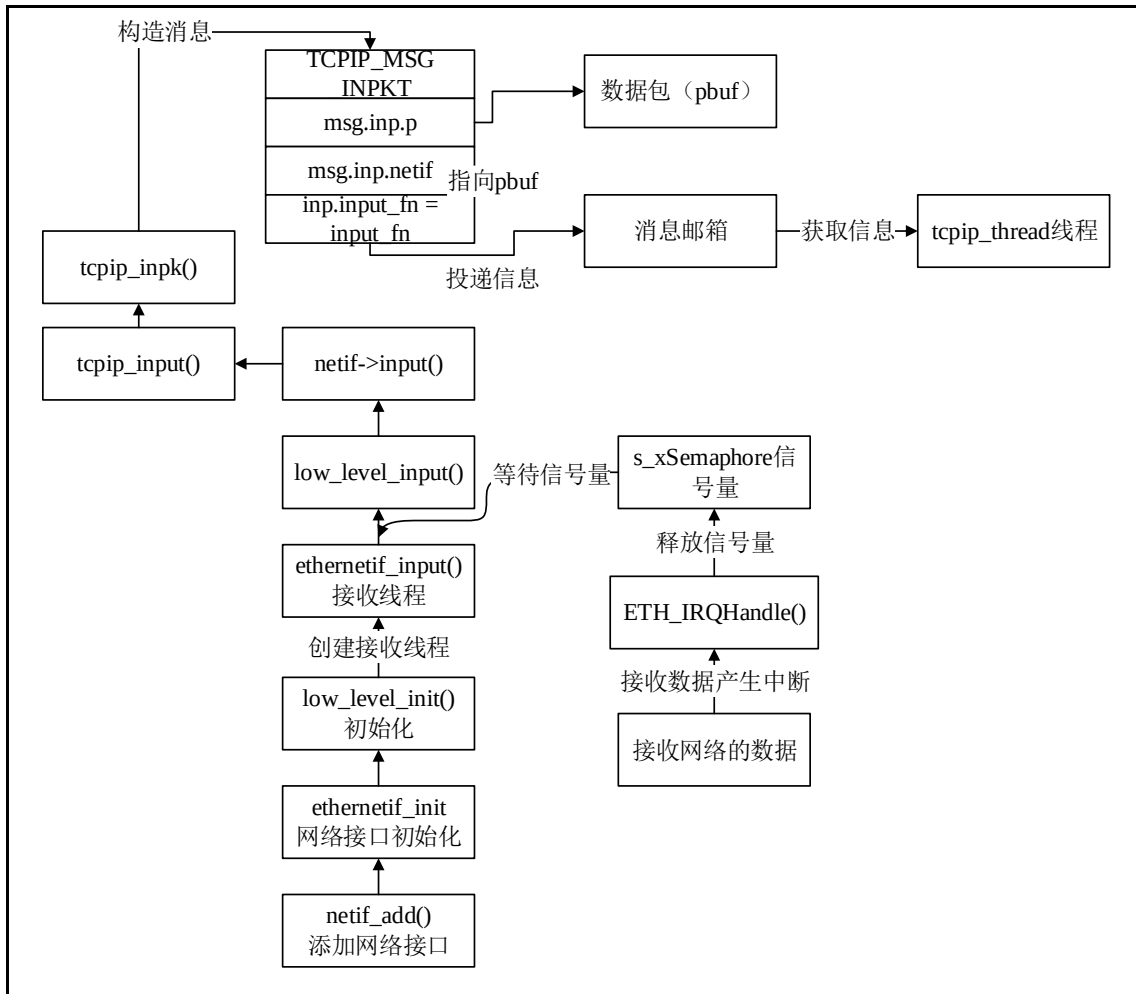


图 7.4.1.1 接收数据包的消息运作示意图

7.4.2 API 消息

所谓 API 消息，其实就是两个 API 部分的交互的消息，它是由用户调用 API 函数为起点，使用 IPC 通信机制告诉内核需要执行那个部分的 API 函数，内核的具体消息内容都可以直接包含内核消息结构 tcpip_msg，但是 API 消息除外，由于它的消息内容实在庞大，所以协议栈专门用结构体 api_msg 来描述 API 消息内容，而在 tcpip_msg 结构体中只维护该类型的指针，前面笔者也讲解到 tcpip_msg 时候，它里面包含了一个 api_msg 指针，这个指针只是指向 api_msg 结构体，现在的 api_msg 结构体在 api_msg.h 文件定义的，该结构体如下所示：

```

struct api_msg
{
    struct netconn *conn; /* 当前连接 */

    err_t err; /* 返回结果 */

    union
    {
        /* 用于函数 lwip_netconn_do_send() 参数 */
        struct netbuf *b;
    };
}
    
```

```
/* 用于函数 lwip_netconn_do_newconn() 参数 */
struct
{
    u8_t proto;
} n;

/* 用于函数 lwip_netconn_do_bind() 和函数 lwip_netconn_do_connect() 参数 */
struct
{
    API_MSG_M_DEF_C(ip_addr_t, ipaddr);          /* ip 地址 */
    u16_t port;                                   /* 端口号 */
    u8_t if_idx;
} bc;

/* 用于函数 lwip_netconn_do_getaddr() 参数 */
struct
{
    ip_addr_t API_MSG_M_DEF(ipaddr);             /* ip 地址 */
    u16_t API_MSG_M_DEF(port);                   /* 端口号 */
    u8_t local;
} ad;

/* 用于函数 lwip_netconn_do_write() 参数 */
struct {
    /** 当前要写的向量 e */
    const struct netvector *vector;
    /** 未写向量的个数 */
    u16_t vector_cnt;
    /** 偏移成矢量 */
    size_t vector_off;
    /** 向量的总长度 */
    size_t len;
    /** 当 err == ERR_OK 时写入的字节的总长度/输出的偏移量 */
    size_t offset;
    u8_t apiflags;
#ifdef LWIP_SO_SNDTIMEO
    u32_t time_started;
#endif /* LWIP_SO_SNDTIMEO */
} w;

/** 用于函数 lwip_netconn_do_recv() 参数 */
struct
```

```

    {
        u32_t len;
    } r;

#if LWIP_TCP
    /* 用于函数 wip_netconn_do_close (/shutdown) 参数 */
    struct
    {
        u8_t shut;
#if LWIP_SO_SNDTIMEO || LWIP_SO_LINGER
        u32_t time_started;
#else /* LWIP_SO_SNDTIMEO || LWIP_SO_LINGER */
        u8_t polls_left;
#endif /* LWIP_SO_SNDTIMEO || LWIP_SO_LINGER */
    } sd;
#endif /* LWIP_TCP */
#if LWIP_IGMP || (LWIP_IPV6 && LWIP_IPV6_MLD)

    /* 用于函数 lwip_netconn_do_join_leave_group() 参数 */
    struct
    {
        API_MSG_M_DEF_C(ip_addr_t, netif_addr);
        u8_t if_idx;
        enum netconn_igmp join_or_leave;
    } jl;
#endif /* LWIP_IGMP || (LWIP_IPV6 && LWIP_IPV6_MLD) */
#if TCP_LISTEN_BACKLOG
    struct
    {
        u8_t backlog;
    } lb;
#endif /* TCP_LISTEN_BACKLOG */
    } msg;
#if LWIP_NETCONN_SEM_PER_THREAD
    sys_sem_t *op_completed_sem;
#endif /* LWIP_NETCONN_SEM_PER_THREAD */
};

```

这个结构体只包含了三个字段，分别为描述连接信息的 `conn`、内核返回的执行结果 `err`、以及 `msg`。在 `api_msg` 结构体中保存 `conn` 字段是必须的，因为 `conn` 结构中包含了与该连接相关的邮箱和信号量等信息，协议栈进程要用这些信息来完成与应用进程间的同步与通信；共用体类型 `msg` 的各个成员与调用它的函数密切相关，如 `lwip_netconn_do_xxx`（`xxx` 表示不一样的 `NETCONN` 的 API 接口）类型的函数执行需要用这些信息来完成与应用线程的通信与同步；

内核执行 `lwip_netconn_do_xxx` 类型的函数返回结果会被记录在 `err` 中；`msg` 的各个参数记录各个函数执行时需要的详细参数。

到了这里，我们已经理解了底层数据包消息，同理 API 函数的调用也是如此，如果用户要与内核进行数据传递，也是需要 lwIP 的消息机制，毕竟用户和内核都是独立的线程或者任务，例如我们使用 SOCKET 的 API 接口和 NETCONN 的 API 接口时候，lwIP 会把用户调用的函数与参数做成消息传递给 `tcpip_thraed` 线程，这个消息就是 lwIP 中的 API 消息，lwIP 为什么会使用这些方式呢？首先对于用户来说，不需要很深入的理解 lwIP 内核，只需要调用 API 函数接口就可以完成实验，例如在 NETCONN 的 API 中构造数据包时，就会调用 `netconn_apimsg` 函数进行投递消息。

其实 lwIP 的协议栈 API 实现有两个部分组成，一部分为用户编程接口函数提供给用户，这些函数在用户进程执行，另一部分驻留在内核进程，这两个部分的通信方法是使用 IPC 通信机制，被用到的进程通信机制有以下四种：

1. 邮箱：用于数据交互。
2. 信号量：用于用户和系统同步。
3. 互斥信号量：用于优先级翻转的问题。
4. 共享内存：内核消息结构 `tcpip_msg` 和 API 消息内容 `api_msg`。

根据上述的通信机制，我们可以得到用户调用函数与内核进程的关系图，如下图所示。

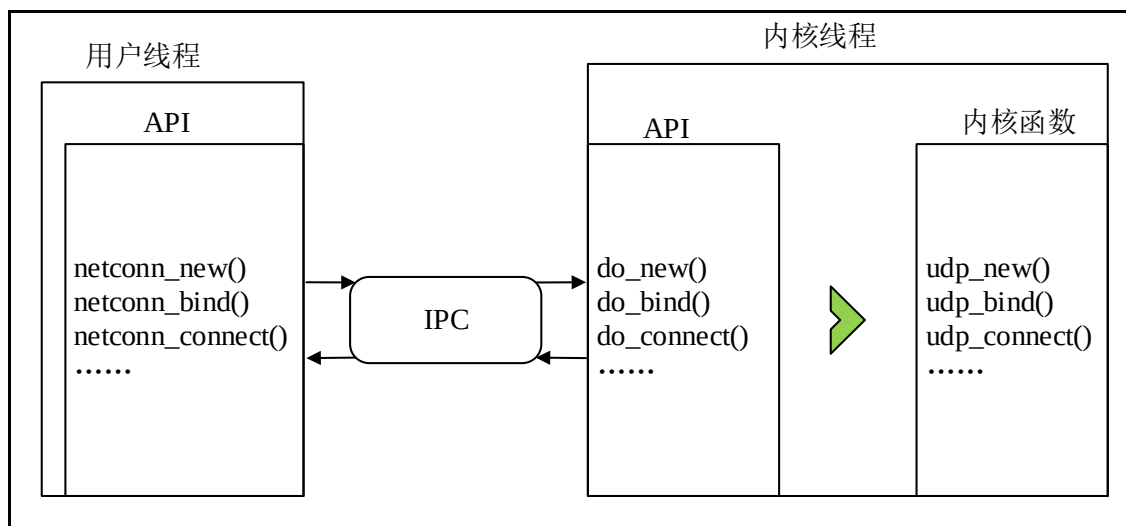


图 7.4.2.1 用户 API 与内核进程的关系图

接下来笔者就以 NETCONN 的 API 为例，来讲解 lwIP 的 API 消息使用，如下源码所示：

```
err_t
netconn_bind(struct netconn *conn, const ip_addr_t *addr, u16_t port)
{
    /*声明 api_msg 消息结构体*/
    API_MSG_VAR_DECLARE(msg);
    err_t err;

#ifdef LWIP_IPV4
    if (addr == NULL) {
        addr = IP4_ADDR_ANY;
    }
}
```

```
#endif /* LWIP_IPV4 */

/* 第一步：构建 api_msg 结构体 */
API_MSG_VAR_ALLOC(msg); /* 申请内存 */
/* 连接的信息 */
API_MSG_VAR_REF(msg).conn = conn;
/* IP 地址 */
API_MSG_VAR_REF(msg).msg.bc.ipaddr = API_MSG_VAR_REF(addr);
/* 端口号 */
API_MSG_VAR_REF(msg).msg.bc.port = port;
/* 发送 API 消息并等待信号量 */
err = netconn_apimsg(lwip_netconn_do_bind, &API_MSG_VAR_REF(msg));
API_MSG_VAR_FREE(msg);

return err;
}

static err_t
netconn_apimsg(tcpip_callback_fn fn, struct api_msg *apimsg)
{
    err_t err;
    /* 发送 API 消息并等待信号量 */
    err = tcpip_send_msg_wait_sem(fn, apimsg, LWIP_API_MSG_SEM(apimsg));
    if (err == ERR_OK) {
        return apimsg->err; /* 返回 API 的错误码 */
    }
    return err;
}

err_t
tcpip_send_msg_wait_sem(tcpip_callback_fn fn, void *apimsg, sys_sem_t *sem)
{
    #if LWIP_TCPIP_CORE_LOCKING
        LWIP_UNUSED_ARG(sem);
        LOCK_TCPIP_CORE();
        fn(apimsg);
        UNLOCK_TCPIP_CORE();
        return ERR_OK;
    #else /* LWIP_TCPIP_CORE_LOCKING */

        /* 声明 tcpip_msg 消息结构体 */
        TCPIP_MSG_VAR_DECLARE(msg);
```

```

/* 第二步：构造 tcpip_msg 消息 */
TCPIP_MSG_VAR_ALLOC(msg); /* 申请内存 */
TCPIP_MSG_VAR_REF(msg).type = TCPIP_MSG_API; /* 消息类型 */
TCPIP_MSG_VAR_REF(msg).msg.api_msg.function = fn; /* 设置回调函数 */
TCPIP_MSG_VAR_REF(msg).msg.api_msg.msg = apimsg; /* 指向 api_msg 消息 */
/* 第三步：释放邮箱 */
sys_mbox_post(&mbox, &TCPIP_MSG_VAR_REF(msg));
/* 第四步：等待信号量 */
sys_arch_sem_wait(sem, 0);
TCPIP_MSG_VAR_FREE(msg);
return ERR_OK;
#endif /* LWIP_TCPIP_CORE_LOCKING */
}

```

上述的源码，我们可分为四步讲解，它们分别构造 tcpip_msg、api_msg、发送邮箱消息和等待信号量，如下图所示。

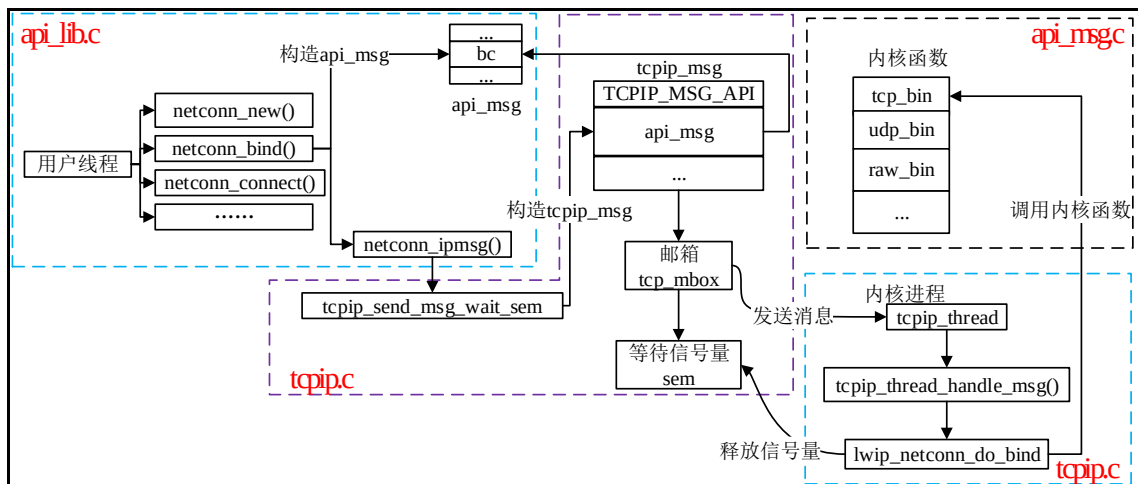


图 7.4.2.2 用户的应用线程与内核交互示意图

其实上图并不是 lwIP 的最优先的流程图，因为在函数 tcpip_send_msg_wait_sem 中宏定义 LWIP_TCPIP_CORE_LOCKING 是为 1 的，表示无需操作系统的邮箱与信号量参与，在该函数只执行以下源码：

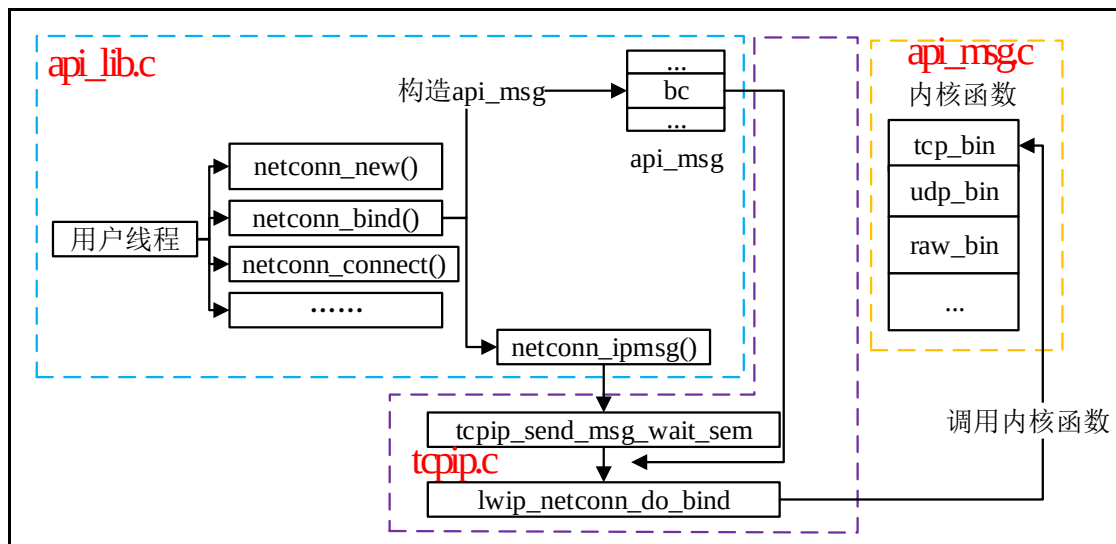
```

err_t
tcpip_send_msg_wait_sem(tcpip_callback_fn fn, void *apimsg, sys_sem_t *sem)
{
#if LWIP_TCPIP_CORE_LOCKING
    LWIP_UNUSED_ARG(sem);
    LOCK_TCPIP_CORE();
    fn(apimsg);
    UNLOCK_TCPIP_CORE();
    return ERR_OK;
#else /* LWIP_TCPIP_CORE_LOCKING */
    /* 代码省略 */
#endif /* LWIP_TCPIP_CORE_LOCKING */
}

```

}

上述函数也非常简单理解，首先此函数调用 LOCK_TCPIP_CORE 函数上锁，记者系统直接调用函数 lwip_netconn_do_bind 对 api_msg 消息做处理，这样的方法省去了 tcpip_mag 消息构建、邮箱以及信号量等操作。如下图所示。



7.4.2.3 最优的应用线程与内核交互示意图

第八章 ARP 协议

本章，我们了解 ARP 协议，ARP 全称为 Address Resolution Protocol（地址解析协议），是根据 IP 地址获取物理地址的一个 TCP/IP 协议。主机发送信息时将包含目标 IP 地址的 ARP 请求广播到局域网络上的所有主机，并接收返回消息，以此确定目标的物理地址；收到返回消息后将该 IP 地址和物理地址存入本机 ARP 缓存中并保留一定时间，下次请求时直接查询 ARP 缓存以节约资源。本章分为如下几个部分：

8.1 ARP 协议的简介

8.2 ARP 协议的报文结构

8.3 ARP 协议层的接收与发送原理解析

8.1 ARP 协议的简介

ARP 协议是根据 IP 地址获取物理地址的一个 TCP/IP 协议。主机发送信息时将包含目标 IP 地址的 ARP 请求广播到局域网络上的所有主机，并接收返回消息，以此确定目标的物理地址；收到返回消息后将该 IP 地址和物理地址存入本机 ARP 缓存中并保留一定时间，下次请求时直接查询 ARP 缓存以节约资源。地址解析协议是建立在网络中各个主机互相信任的基础上的，局域网络上的主机可以自主发送 ARP 应答消息，其他主机收到应答报文时不会检测该报文的真实性就会将其记入本机 ARP 缓存；总的来说，ARP 协议是透过目标设备的 IP 地址，查询目标设备的 MAC 地址，以保证通信的顺利进行。

8.1.1 ARP 协议的工作流程

假设由两台主机，分别为主机 A（192.168.0.10）与主机 B（192.168.0.11），它们两个都是同一网段的，如果主机 A 向主机 B 发送信息或者数据，ARP 的地址解析过程有以下几个步骤：

① 主机 A 首先查自己的 ARP 表是否有包含主机 B 的信息，例如主机 B 的 MAC 地址，如果主机 A 的 ARP 表包含主机 B 的 MAC 地址，则主机 A 直接利用 ARP 表的主机 B 的 MAC 地址对 IP 数据包进行封装并把数据包发给主机 B。

② 如果主机 A 的 ARP 表没有包含主机 B 的 MAC 地址或者没有找到主机 B 的 MAC 地址，则主机 A 就把数据包缓存起来，然后以广播的方式发送一个 ARP 包的请求报文，该 ARP 包的内容包含主机 A 的 IP 地址、MAC 地址、主机 B 的 IP 地址和主机 B 的全 0 的 MAC 地址，由于主机 A 发送 ARP 包是使用广播形式，那么同一网段的主机都可以收到该 ARP 包，主机 B 接收到这个 ARP 包会进行处理。

③ 主机 B 接收到主机 A 的 ARP 包之后，主机 B 会对这个 ARP 解析并比较自己的 IP 地址和 ARP 包的目的 IP 地址是否相同，如果相同，则主机 B 将 ARP 请求报文中的发送端（即主机 A）的 IP 地址和 MAC 地址存入自己的 ARP 表中。之后以单播方式发送 ARP 响应报文给主机 A，其中包含了自己的 MAC 地址。

④ 当主机 A 收到了主机 B 的 ARP 包也是同样的处理，首先比较 ARP 包的 IP 地址是否和自己的 IP 地址相同，如果 IP 地址相同，则把 ARP 包的信息存入自己的 ARP 表中，最后对 IP 数据包进行封装并把数据包发给主机 B。

从上述步骤的内容，可得到 ARP 包的流程图，如下图所示。

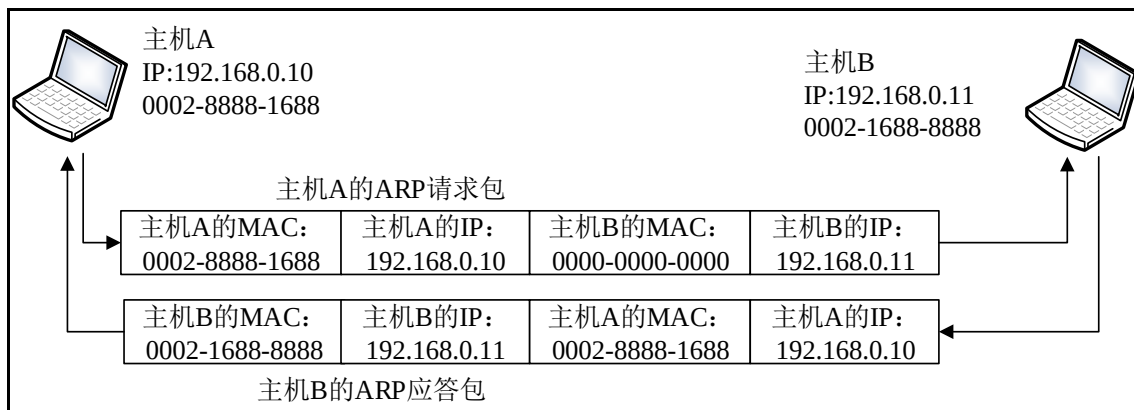


图 8.1.1.1 ARP 协议示意图

可以看到，主机 A 发送数据之前先判断主机 A 的 ARP 缓存表是否包含主机 B 的 MAC 地址，若主机 A 的 ARP 缓存表没有主机 B 的 MAC 地址，则主机 A 把要发送的数据挂起并发送一个 ARP 请求包，发送完成之后等待主机 B 应答，直到收到主机 B 的应答包之后才把挂起的数据包添加以太网首部发送至主机 B 当中。

lwIP 描述 ARP 缓存表和 ARP 相关处理函数由 `etharp.c/h` 文件定义，下面笔者重点讲解 ARP 缓存表的表项信息和挂起流程。ARP 缓存表结构如下所示：

```
struct etharp_entry {
#ifdef ARP_QUEUEING
    /* 数据包缓存队列指针 */
    struct etharp_q_entry *q;
#else /* ARP_QUEUEING */
    /* 指向此 ARP 表项上的单个挂起数据包的指针 */
    struct pbuf *q;
#endif
    ip4_addr_t ipaddr; /* 目标 IP 地址 */
    struct netif *netif; /* 对应网卡信息 */
    struct eth_addr ethaddr; /* 对应的 MAC 地址 */
    u16_t ctime; /* 生存时间信息 */
    u8_t state; /* 表项的状态 */
};

static struct etharp_entry arp_table[ARP_TABLE_SIZE];
```

可以看出，ARP 缓存表（`arp_table`）最大存放 10 个表项，每一个表项描述符了 IP 地址映射 MAC 地址的信息和表项生存时间与状态。这个 ARP 缓存表很小，lwIP 根据传入的目标 IP 地址对 ARP 缓存表直接采用遍历方式查找对应的 MAC 地址。

注：每一个表项都有一个生存时间，若超出了自身生存时间，则 lwIP 内核会把这个表项删除，这里用到了超时处理机制。

每一个表项从创建、请求等都设置了一个状态，不同状态的表项都需要特殊的处理，这些状态如下所示：

```
enum etharp_state {
    ETHARP_STATE_EMPTY = 0,
    ETHARP_STATE_PENDING,
    ETHARP_STATE_STABLE,
};
```

```
ETHARP_STATE_STABLE_REREQUESTING_1,
ETHARP_STATE_STABLE_REREQUESTING_2
};
```

下面笔者讲解一下每一个表项的作用及任务。

(1) ETHARP_STATE_EMPTY 状态

这个状态表示 ARP 缓存表处于初始化的状态，所有表项初始化之后才可以被使用，如果需要添加表项，lwIP 内核就会遍历 ARP 缓存表并找到合适的表项进行添加。

(2) ETHARP_STATE_PENDING 状态

该状态表示该表项处于不稳定状态，此时该表项只记录到了 IP 地址，但是还未记录到对应的 MAC 地址。很可能的情况是：lwIP 内核已经发出一个关于该 IP 地址的 ARP 请求到数据链路上且 lwIP 内核还未收到 ARP 应答，此时 ETHARP_STATE_PENDING 状态下会设定超时时间（5 秒），当计数超时后，对应的表项将被删除，超时时间需要宏定义 ARP_MAXPENDING 来指定，默认为 5 秒，如果在 5 秒之前收到应答数据包，那么系统会更新缓存表的信息，记录目标 IP 地址与目标 MAC 地址的映射关系并且开始记录表项的生存时间，同时该表项的状态会变成 ETHARP_STATE_STABLE 状态。

(3) ETHARP_STATE_STABLE 状态

当收到应答之前，这些数据包会暂时挂载到表项的数据包缓冲队列上，收到应答之后，系统已经更新 ARP 缓存表，那么系统发送数据就会进入该状态

(4) ETHARP_STATE_STABLE_REREQUESTING_1 & ETHARP_STATE_STABLE_REREQUESTING_2 状态

如果系统再一次发送 ARP 请求数据包，则表项状态会暂时被设置为 ETHARP_STATE_STABLE_REREQUESTING_1，之后设置为 ETHARP_STATE_STABLE_REREQUESTING_2 状态，其实这两个状态为过渡状态，如果 5 秒之前收到 ARP 应答后，表项又会被设置为 ETHARP_STATE_STABLE 状态，这样子能保持表项的有效性。

这些状态也是和超时处理相关，在 ARP 超时事件中，需要定时遍历 ARP 缓存表各个表项的状态和检测各个表项的生存时间。稍后笔者也会讲解 ARP 超时事件的作用。

表项挂起数据包

之前讲解过，lwIP 发送数据包时需要检测 ARP 缓存表是否包含对方主机的 MAC 地址，若 ARP 缓存表没有包含对方主机的 MAC 地址，则 lwIP 内核在 ARP 缓存表上创建一个表项并且构建一个 ARP 请求包，发送完成之后 lwIP 内核把要发送的数据包挂载到新创建的表项当中。在表项中包含了 etharp_q_entry 结构体和 pbuf 结构体指针，这两个都是用来挂载数据包的，一般来说，lwIP 内核不使用 etharp_q_entry 结构体挂载数据包，而是直接使用指针指向 pbuf 数据包，下面笔者使用一张图来描述上面的内容。

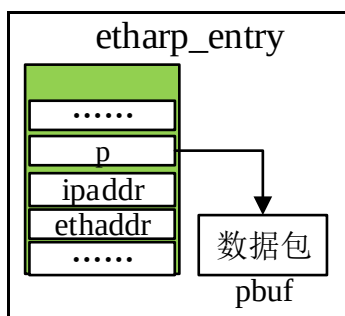


图 8.1.1.2 ARP 表项单个挂起的数据包

8.1.2 ARP 缓存表的超时处理

上一个小节写了这么多，无非就是为了 ARP 表项的 `ctime`（生存时间）这个参数准备的，其实这个参数笔者在上面也有所涉及，因为系统以周期的形式调用函数 `etharp_trm`。例如，5 秒之前收到 ARP 的应答包就会更新 ARP 缓存表，这个函数的作用就是使每个 ARP 缓存表项 `ctime` 字段加 1 处理，如果某个表项的生存时间计数值大于系统规定的某个值，系统就会删除该表项。`etharp_trm` 函数如下所示：

```
void
etharp_trm(void)
{
    u8_t i;

    /* 第一步：ARP 缓存表遍历，ARP_TABLE_SIZE = 10 */
    for (i = 0; i < ARP_TABLE_SIZE; ++i)
    {
        /* 获取表项的状态 */
        u8_t state = arp_table[i].state;

        /* 第二步：判断该状态不等于空（初始化的状态） */
        if (state != ETHARP_STATE_EMPTY)
        {
            /* ARP 缓存表项的生存时间+1 */
            arp_table[i].ctime++;

            /* 第三步：发送 ARP 请求数据包并判断 ctime 是否大于 5 秒 */
            if ((arp_table[i].ctime >= ARP_MAXAGE) ||
                ((arp_table[i].state == ETHARP_STATE_PENDING) &&
                 (arp_table[i].ctime >= ARP_MAXPENDING)))
            {
                /* 从 ARP 缓存表中删除该表项 */
                etharp_free_entry(i);
            }
            else if (arp_table[i].state == ETHARP_STATE_STABLE_REREQUESTING_1)
            {
                /* 这是一个过度形式 */
                arp_table[i].state = ETHARP_STATE_STABLE_REREQUESTING_2;
            }
            else if (arp_table[i].state == ETHARP_STATE_STABLE_REREQUESTING_2)
            {
                /* 将状态重置为稳定状态，使下一个传输的数据包将
                 重新发送一个 ARP 请求 */
                arp_table[i].state = ETHARP_STATE_STABLE;
            }
            else if (arp_table[i].state == ETHARP_STATE_PENDING)
```



```

{
    /* 仍然挂起，重新发送一个 ARP 查询 */
    etharp_request(arp_table[i].netif, &arp_table[i].ipaddr);
}
}
}

```

此函数非常简单，这里笔者使用一个流程图来讲解这个函数的实现流程，如下图所示。

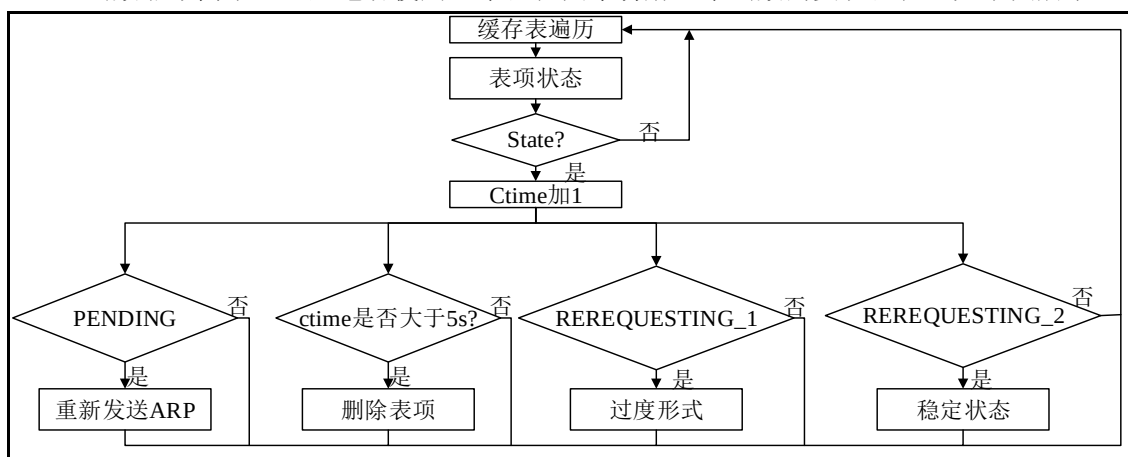


图 8.1.2.1 函数 etharp_tmr()流程图

从上图可以看出，这些 ARP 缓存表的表项都会定期检测，如果这些表项超时最大生存时间，那么 lwIP 内核会把这些表项统一删除。

8.2 APR 报文的报文结构

典型的 ARP 报文结构，该结构如下图所示。

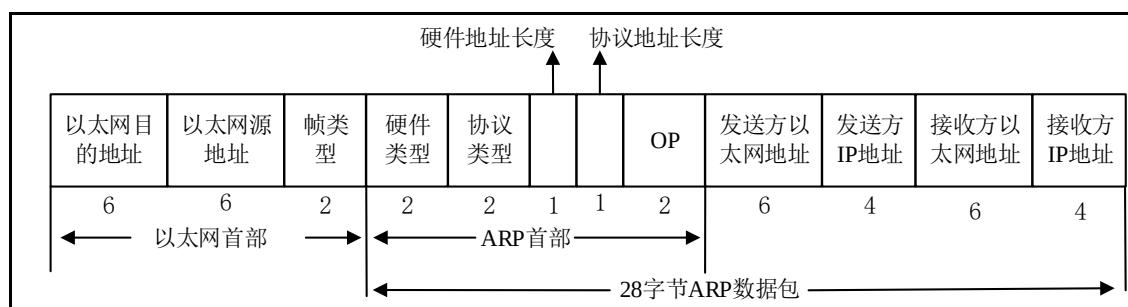


图 8.2.1 ARP 报文结构

左边的是以太网首部，数据发送时必须添加以太网首部，添加完成之后才能把数据发往到网络当中（这里解答了为什么需要对方主机的 MAC 地址），而右边是 ARP 报文结构，它一共定义了 5 个字段，它们分别为：

硬件类型：如果这个类型设置为 1 表示以太网 MAC 地址。

协议类型：表示要映射的协议地址类型，0x0800--映射为 IP 地址。

硬件地址长度和协议地址长度：以太网 ARP 请求和应答分别设置为 6 和 4，它们代表 MAC 地址长度和 IP 地址长度。在 ARP 协议包中留出硬件地址长度字段和协议地址长度字段可以使得 ARP 协议在任何网络中被使用，而不仅仅只在以太网中。

op：ARP 数据包的类型，ARP 请求设置为 1，ARP 应答设置为 2。

剩下的字段就是填入本地 IP 地址与本地 MAC 地址和目标 IP 地址与目标 MAC 地址。

关于 ARP 报文结构可在 ethernet.h 找到一些数据结构和宏描述，如下所示：

```

/*****ethernet.h*****/
#define ETH_HWADDR_LEN    6  /* 以太网地址长度 */

struct eth_addr { /* 一个以太网 MAC 地址 */
    PACK_STRUCT_FLD_8(u8_t addr[ETH_HWADDR_LEN]);
} PACK_STRUCT_STRUCT;

struct eth_hdr { /* 以太网首部 */
#ifdef ETH_PAD_SIZE
    PACK_STRUCT_FLD_8(u8_t padding[ETH_PAD_SIZE]);
#endif

    PACK_STRUCT_FLD_S(struct eth_addr dest);          /* 以太网目标地址(6 字节) */
    PACK_STRUCT_FLD_S(struct eth_addr src);           /* 以太网源 MAC 地址(6 字节) */
    PACK_STRUCT_FIELD(u16_t type);                    /* 帧类型(2 字节) */
} PACK_STRUCT_STRUCT;

/*****etharp.h*****/
struct etharp_hdr { /* ARP 报文 */
    /* ARP 报文首部 */
    PACK_STRUCT_FIELD(u16_t hwtype);                  /* 硬件类型(2 字节) */
    PACK_STRUCT_FIELD(u16_t proto);                    /* 协议类型(2 字节) */
    PACK_STRUCT_FLD_8(u8_t hwlen);                    /* 硬件地址长度(1 字节) */
    PACK_STRUCT_FLD_8(u8_t protolen);                 /* 协议地址长度(2 字节) */
    PACK_STRUCT_FIELD(u16_t opcode);                  /* op 字段(2 字节) */
    PACK_STRUCT_FLD_S(struct eth_addr shwaddr);       /* 源 MAC 地址(6 字节) */
    PACK_STRUCT_FLD_S(struct ip4_addr2 sipaddr);      /* 源 ip 地址(4 字节) */
    PACK_STRUCT_FLD_S(struct eth_addr dhwaddr);       /* 目标 MAC 地址(6 字节) */
    PACK_STRUCT_FLD_S(struct ip4_addr2 dipaddr);      /* 目标 ip 地址(4 字节) */
} PACK_STRUCT_STRUCT;

/* op 字段操作 */
enum etharp_opcode {
    ARP_REQUEST = 1, /* 请求包 */
    ARP_REPLY   = 2  /* 应答包 */
};

```

前面的 eth_hdr 结构体就是定义了以太网首部字段，而 etharp_hdr 结构体定义了 ARP 首部的字段信息。下面笔者使用 Wireshark 网络抓包工具形象地讲解报文格式和内容，如下图所示。

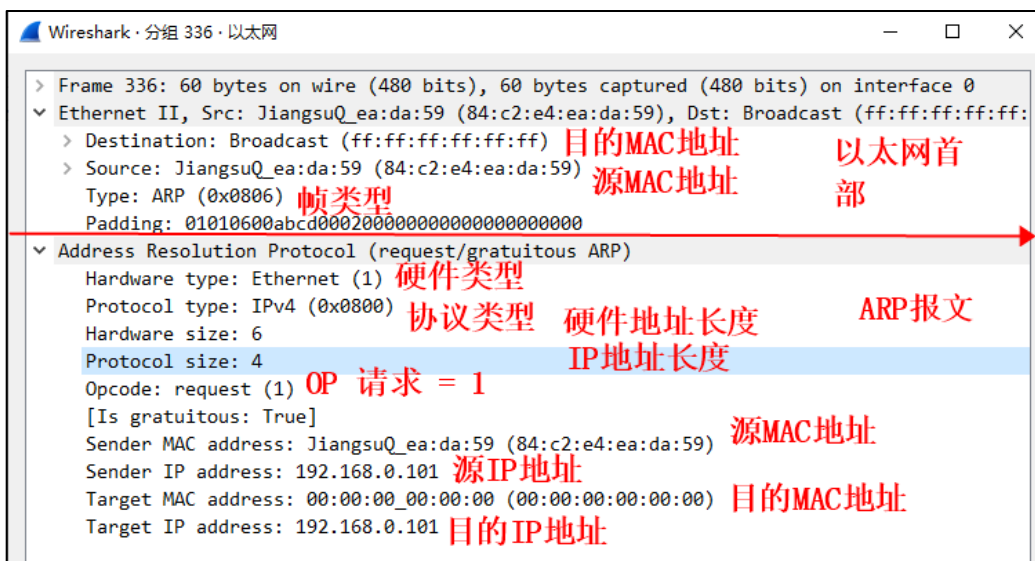


图 8.2.2 ARP 请求数据包

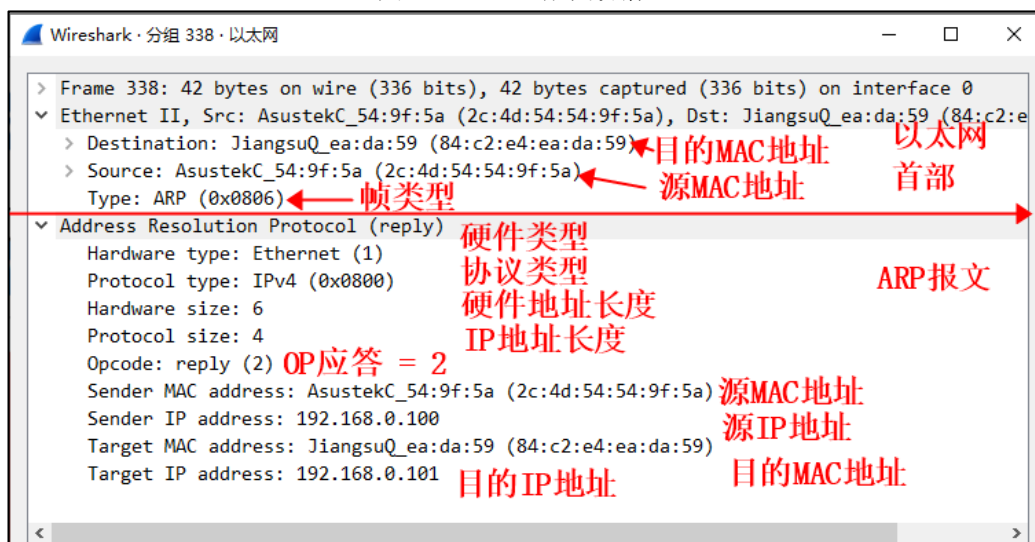


图 8.2.3 ARP 应答数据包

从这两张图可以看出，图一的 ARP 数据包是以广播的方式发送，它的 OP 字段类型为 1 表示 ARP 数据包为 ARP 请求包。图二的 ARP 数据包为 ARP 应答包，因为它的 OP 字段为 2，所以该包不是以广播的方式发送。

8.3 ARP 协议层的接收与发送原理解析

8.3.1 发送 ARP 请求数据包

构建 ARP 请求包函数是在 etharp_raw 函数下实现，该函数如下所示：

```
static err_t
etharp_raw(struct netif *netif, /* 发送 ARP 数据包的 lwip 网络接口 */
            const struct eth_addr *ethsrc_addr, /* 以太网源 MAC 地址 */
            const struct eth_addr *ethdst_addr, /* 以太网目标 MAC 地址 */
            const struct eth_addr *hwsrc_addr, /* ARP 协议源 MAC 地址 */
            const ip4_addr_t *ipsrc_addr, /* ARP 协议源 IP 地址 */
```

```

        const struct eth_addr *hwdst_addr,          /* ARP 协议目标 MAC 地址 */
        const ip4_addr_t *ipdst_addr,              /* ARP 协议目标 IP 地址 */
        const u16_t opcode) /* ARP 数据包的类型：1 为请求包类型、2 为应答包类型 */
{
    struct pbuf *p;
    err_t result = ERR_OK;
    struct etharp_hdr *hdr;

    /* 申请 ARP 报文的内存池空间 */
    p = pbuf_alloc(PBUF_LINK, SIZEOF_ETHARP_HDR, PBUF_RAM);

    /* 申请内存池是否成功 */
    if (p == NULL)
    {
        ETHARP_STATS_INC(etharp.memerr);
        return ERR_MEM;
    }

    /* ARP 报文的数据区域，并且强制将起始地址转化成 ARP 报文首部 */
    hdr = (struct etharp_hdr *)p->payload;
    /* ARP 数据包的 op 字段 */
    hdr->opcode = lwip_htons(opcode);

    /* 源 MAC 地址 */
    SMEMCPY(&hdr->shwaddr, hwsrc_addr, ETH_HWADDR_LEN);
    /* 目的 MAC 地址 */
    SMEMCPY(&hdr->dhwaddr, hwdst_addr, ETH_HWADDR_LEN);
    /* 源 IP 地址 */
    IPADDR_WORDALIGNED_COPY_FROM_IP4_ADDR_T(&hdr->sipaddr, ipsrc_addr);
    /* 目的 IP 地址 */
    IPADDR_WORDALIGNED_COPY_FROM_IP4_ADDR_T(&hdr->dipaddr, ipdst_addr);

    /* 硬件类型 */
    hdr->hwtype = PP_HTONS(HWTYPE_ETHERNET);
    /* 协议类型 */
    hdr->proto = PP_HTONS(ETHTYPE_IP);

    /* 硬件地址长度 */
    hdr->hwlen = ETH_HWADDR_LEN;
    /* 协议地址长度 */
    hdr->protolen = sizeof(ip4_addr_t);

```

```
#if LWIP_AUTOIP
    if (ip4_addr_islinklocal(ipsrc_addr))
    {
        ethernet_output(netif, p, ethsrc_addr, &ethbroadcast, ETHTYPE_ARP);
    }
    else
#endif /* LWIP_AUTOIP */
    {
        /* 调用底层发送函数将以太网数据帧发送出去 */
        ethernet_output(netif, p, ethsrc_addr, ethdst_addr, ETHTYPE_ARP);
    }

    ETHARP_STATS_INC(etharp.xmit);

    /* 发送完成释放内存 */
    pbuf_free(p);
    p = NULL;

    /* 发送完成返回结果 */
    return result;
}

/* 定义以太网广播地址 */
const struct eth_addr ethbroadcast = {{0xff,0xff,0xff,0xff,0xff,0xff}};
/* 填写 ARP 请求包的接收方 MAC 字段 */
const struct eth_addr ethzero = {{0,0,0,0,0,0}};

static err_t
etharp_request_dst(struct netif *netif, const ip4_addr_t *ipaddr,
                  const struct eth_addr *hw_dst_addr)
{
    return etharp_raw(netif, (struct eth_addr *)netif->hwaddr, hw_dst_addr,
                      (struct eth_addr *)netif->hwaddr,
                      netif_ip4_addr(netif),
                      &ethzero,
                      ipaddr, ARP_REQUEST);
}

/* 发送一个要求 ipaddr 的 ARP 请求包 */
err_t
etharp_request(struct netif *netif, const ip4_addr_t *ipaddr)
{
    return etharp_request_dst(netif, ipaddr, &ethbroadcast);
}
```

```
}

```

发送 ARP 请求报文之前先申请 pbuf 内存，接着由 pbuf 的 payload 指针指向的地址添加 ARP 首部，添加完成之后设置 ARP 首部字段的信息，最后由 ethernet_output 函数为 pbuf 添加以太网首部和发送，发送完成之后把要发送的数据挂载到 ARP 缓存表项当中。。

8.3.2 接收 ARP 应答数据包

虽然 ARP 和 IP 协议同属于网络层的协议，但是从分层的结构来看，ARP 处于网络层的最底层，而 IP 处于网络层的顶层。总的来说，ARP 最接近网卡驱动文件，发送的数据经过 ARP 检测和操作发送至网卡驱动文件处理，由网卡驱动文件调用 ETH 外设把数据发送至 PHY 设备当中。下面笔者来讲解网卡驱动文件的函数如何把接收的数据发送至 ARP 或者 IP 处理，这个函数为 ethernet_input，如下所示：

```
err_t
ethernet_input(struct pbuf *p, struct netif *netif)
{
    struct eth_hdr *ethhdr;
    u16_t type;
#ifdef LWIP_ARP || ETHARP_SUPPORT_VLAN || LWIP_IPV6
    s16_t ip_hdr_offset = SIZEOF_ETH_HDR;
#endif /* LWIP_ARP || ETHARP_SUPPORT_VLAN */
    /* 第一步：判断数据包是否小于等于以太网头部的大小
       如果是，则释放内存，直接返回 */
    if (p->len <= SIZEOF_ETH_HDR)
    {
        ETHARP_STATS_INC(etharp.proterr);
        ETHARP_STATS_INC(etharp.drop);
        MIB2_STATS_NETIF_INC(netif, ifinerrors);
        goto free_and_return;
    }
    if (p->if_idx == NETIF_NO_INDEX)
    {
        p->if_idx = netif_get_index(netif);
    }
    /* 第二步：p->payload 表示指向缓冲区中实际数据的指针
       相当于指向以太网的头部 */
    ethhdr = (struct eth_hdr *)p->payload;

    /* 第三步：获取数据包的类型 */
    type = ethhdr->type;

#ifdef LWIP_ARP_FILTER_NETIF
    netif = LWIP_ARP_FILTER_NETIF_FN(p, netif, lwip_htons(type));
#endif /* LWIP_ARP_FILTER_NETIF*/
}
```

```

/* 第四步：判断数据包是以怎么样的类型发来的 */
if (ethhdr->dest.addr[0] & 1)
{
    /* 这可能是一个多播或广播包 */
    if (ethhdr->dest.addr[0] == LL_IP4_MULTICAST_ADDR_0)
    {
#ifdef LWIP_IPV4

        if ((ethhdr->dest.addr[1] == LL_IP4_MULTICAST_ADDR_1) &&
            (ethhdr->dest.addr[2] == LL_IP4_MULTICAST_ADDR_2))
        {
            /* 将 pbuf 标记为链路层多播 */
            p->flags |= PBUF_FLAG_LLMCAST;
        }

#endif /* LWIP_IPV4 */

    }
    else if (eth_addr_cmp(&ethhdr->dest, &ethbroadcast))
    {
        /* 将 pbuf 标记为链路层广播 */
        p->flags |= PBUF_FLAG_LLBCAST;
    }
}

/* 第五步：判断数据包的类型 */
switch (type)
{
#ifdef LWIP_IPV4 && LWIP_ARP

    /* IP 数据包 */
    case PP_HTONS(ETHTYPE_IP):
        if (!(netif->flags & NETIF_FLAG_ETHARP))
        {
            goto free_and_return;
        }

        /* 去除以太网报头*/
        if ((p->len < ip_hdr_offset) ||
            pbuf_header(p, (s16_t) - ip_hdr_offset))
        {
            /* 去除以太网首部失败，则直接返回 */
            goto free_and_return;
        }
    else

```

```

        {
            /* 传递到 IP 协议去处理 */
            ip4_input(p, netif);
        }

        break;

/* 对于是 ARP 包 */
case PP_HTONS(ETHTYPE_ARP):
    if (!(netif->flags & NETIF_FLAG_ETHARP))
    {
        goto free_and_return;
    }

    /* 去除以太网首部 */
    if ((p->len < ip_hdr_offset) ||
        pbuf_header(p, (s16_t) - ip_hdr_offset))
    { /* 去除以太网首部失败, 则直接返回 */
        ETHARP_STATS_INC(etharp.lenerr);
        ETHARP_STATS_INC(etharp.drop);
        goto free_and_return;
    }
    else
    {
        /* 传递到 ARP 协议处理 */
        etharp_input(p, netif);
    }

    break;
#endif /* LWIP_IPV4 && LWIP_ARP */

    default:
#ifdef LWIP_HOOK_UNKNOWN_ETH_PROTOCOL
        if (LWIP_HOOK_UNKNOWN_ETH_PROTOCOL(p, netif) == ERR_OK)
        {
            break;
        }
#endif

#ifdef LWIP_HOOK_UNKNOWN_ETH_PROTOCOL
    ETHARP_STATS_INC(etharp.proterr);
    ETHARP_STATS_INC(etharp.drop);
    MIB2_STATS_NETIF_INC(netif, ifinunknownprotos);
    goto free_and_return;
}

```



```
return ERR_OK;

free_and_return:
    pbuf_free(p);
    return ERR_OK;
}
```

为了理解整个以太网的数据帧在 ARP 层处理，笔者就以图形展示整个数据包递交流程，如下图所示。

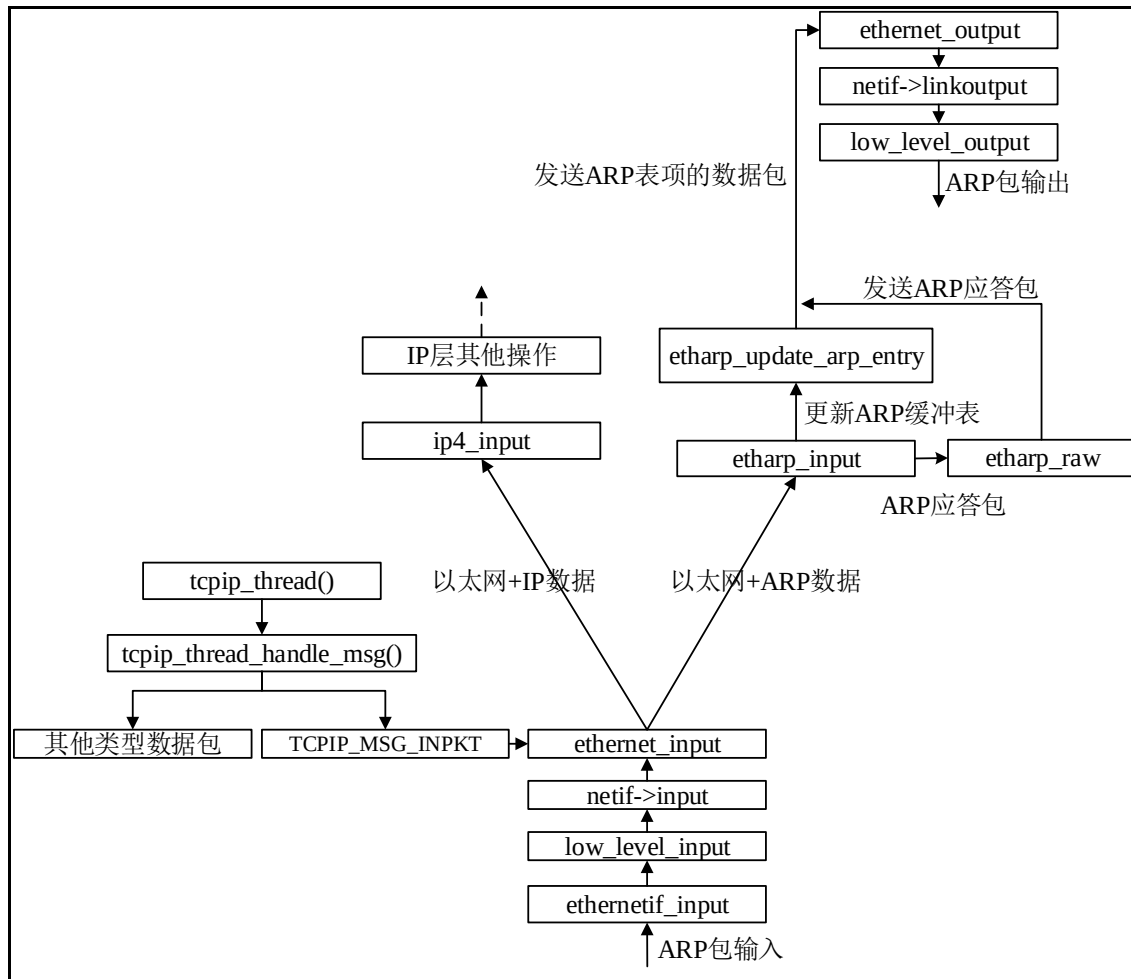


图 8.3.1 ARP 数据包输入

可以看出，数据包在 `ethernet_input` 中需要判断该数据包的类型，若该数据包的类型为 IP 数据包，则 lwIP 内核把该数据包递交给 `ip4_input` 函数处理。若该数据包的类型为 ARP 数据包，则 lwIP 内核把该数据包递交给 `etharp_input` 函数处理，递交完成之后该函数需要判断 ARP 数据包的类型，如果它是 ARP 请求包，则 lwIP 内核调用 `etharp_raw` 函数构建 ARP 应答包并且更新 ARP 缓存表；如果它是 ARP 应答包，则 lwip 内核更新 ARP 缓存表并且把表项挂载的数据包以 `ethernet_output` 函数发送。

第九章 IP 协议

IP 指网际互连协议，Internet Protocol 的缩写，是 TCP/IP 体系中的网络层协议。设计 IP 的目的是提高网络的可扩展性：一是解决互联网问题，实现大规模、异构网络的互联互通；二是分割顶层网络应用和底层网络技术之间的耦合关系，以利于两者的独立发展。根据端到端的设计原则，IP 只为主机提供一种无连接、不可靠的、尽力而为的数据包传输服务。本章分为如下几个部分：

- 9.1 IP 协议的简介
- 9.2 IP 数据报
- 9.3 IP 数据报的输出
- 9.4 IP 数据报的输入

9.1 IP 协议的简介

IP 协议是整个 TCP/IP 协议族的核心，也是构成互联网的基础。IP 位于 TCP/IP 模型的网络层（相当于 OSI 模型的网络层），它可以向传输层提供各种协议的信息，例如 TCP、UDP 等；对下可将 IP 信息包放到链路层，通过以太网、令牌环网络等各种技术来传送。为了能适应异构网络，IP 强调适应性、简洁性和可操作性，并在可靠性做了一定的牺牲。这里我们不过多深入了解 IP 协议了，本章笔者重点讲解 IP 数据报的分片与重组原理。

9.2 IP 数据报

IP 层数据报也叫做 IP 数据报或者 IP 分组，IP 数据报组装在以太网帧中发送的，它通常由两个部分组成，即 IP 首部与数据区域，其中 IP 的首部是 20 字节大小，数据区域理论上可以多达 65535 个字节，由于以太网网络接口的最大传输单元为 1500，所以一个完整的数据包不能超出 1500 字节大小。IP 数据报结构如以下图所示：

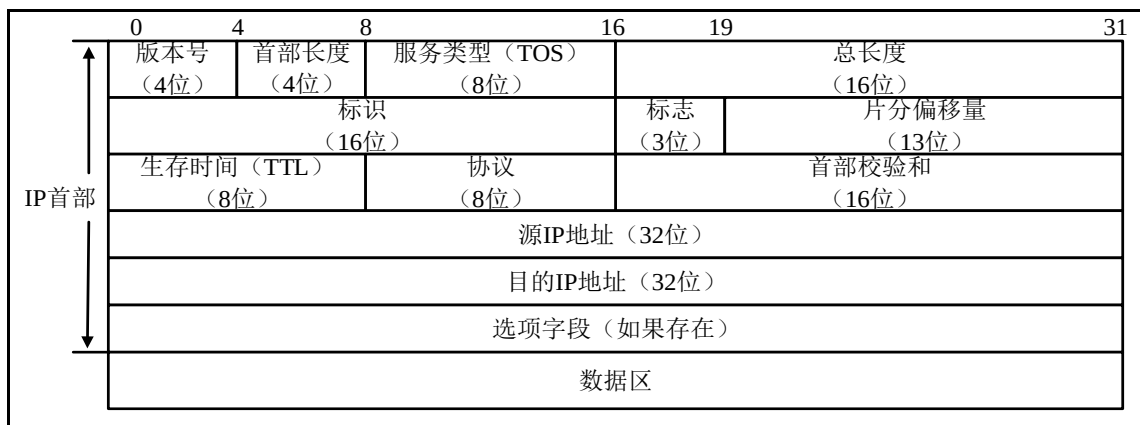


图 9.2.1 IP 数据报结构

(1) 版本：占 4 位指 IP 协议的版本。通信双方使用的 IP 协议版本必须一致。广泛使用的 IP 协议版本号为 4（即 IPv4）。

(2) 首部长度：占 4 位可表示的最大十进制数值是 15。请注意，这个字段所表示数的单位是 32 位字长（1 个 32 位字长是 4 字节），因此，当 IP 的首部长度为 1111 时（即十进制的 15），首部长度就达到 60 字节。当 IP 分组的首部长度不是 4 字节的整数倍时，必须利用最后的填充

字段加以填充。因此数据部分永远在 4 字节的整数倍开始，这样在实现 IP 协议时较为方便。首部长度限制为 60 字节的缺点是有时可能不够用。但这样做是希望用户尽量减少开销。最常用的首部长度就是 20 字节（即首部长度为 0101），这时不使用任何选项。

(3) 区分服务：占 8 位，用来获得更好的服务。这个字段在旧标准中叫做服务类型，但实际上一直没有被使用过。

(4) 总长度：总长度指首部和数据之和的长度，单位为字节。总长度字段为 16 位，因此数据报的最大长度为 $2^{16}-1=65534$ 字节。

在 IP 层下面的每一种数据链路层都有自己的帧格式，其中包括帧格式中的数据字段的最大长度，这称为最大传送单元 MTU。当一个数据报封装成链路层的帧时，此数据报的总长度（即首部加上数据部分）一定不能超过下面的数据链路层的 MTU 值。

(5) 标识 (identification)：占 16 位 IP 软件在存储器中维持一个计数器，每产生一个数据报，计数器就加 1，并将此值赋给标识字段。但这个“标识”并不是序号，因为 IP 是无连接服务，数据报不存在按序接收的问题。当数据报由于长度超过网络的 MTU 而必须分片时，这个标识字段的值就被复制到所有的数据报的标识字段中。相同的标识字段的值使分片后的各数据报片最后能正确地重装成为原来的数据报。

(6) 标志 (flag)：占 3 位但只有 2 位有意义的。

1. 标志字段中的最低位记为 MF (More Fragment)。MF=1 即表示后面“还有分片”的数据报。MF=0 表示这已是若干数据报片中的最后一个。

2. 标志字段中间的一位记为 DF (Don't Fragment)，意思是“不能分片”。只有当 DF=0 时才允许分片。

(7) 片偏移：占 13 位片偏移指出：较长的分组在分片后，某片在原分组中的相对位置。也就是说，相对用户数据字段的起点，该片从何处开始。片偏移以 8 个字节为偏移单位。这就是说，除了最后一个分片，每个分片的长度一定是 8 字节（64 位）的整数倍。

(8) 生存时间：占 8 位生存时间字段常用的英文缩写是 TTL (Time To Live)，表明是数据报在网络中的寿命。由发出数据报的源点设置这个字段。其目的是防止无法交付的数据报无限地在因特网中兜圈子，因而白白消耗网络资源。最初的设计是以秒作为 TTL 的单位。每经过一个路由器时，就把 TTL 减去数据报在路由器消耗掉的一段时间。若数据报在路由器消耗的时间小于 1 秒，就把 TTL 值减 1。当 TTL 值为 0 时，就丢弃这个数据报。后来把 TTL 字段的功能改为“跳数限制”（但名称不变）。路由器在转发数据报之前就把 TTL 值减 1。若 TTL 值减少到零，就丢弃这个数据报，不再转发。因此，TTL 的单位不再是秒，而是跳数。TTL 的意义是指明数据报在网络中至多可经过多少个路由器。显然，数据报在网上经过的路由器的最大数值是 255。若把 TTL 的初始值设为 1，就表示这个数据报只能在本局域网中传送。

(9) 协议：占 8 位协议字段指出此数据报携带的数据是使用何种协议，以便使目的主机的 IP 层知道应将数据部分上交给哪个处理过程。

(10) 首部检验和：占 16 位这个字段只检验数据报的首部，但不包括数据部分。这是因为数据报每经过一个路由器，路由器都要重新计算一下首部检验和（一些字段，如生存时间、标志、片偏移等都可能发生变化）。不检验数据部分可减少计算的工作量。

(11) 源地址：占 32 位。

(12) 目的地址：占 32 位。

(13) 数据区域：这是 IP 数据报的最后一个字段，也是最重要的内容，lwIP 发送数据报是把该层的首部封装到数据包里面，在 IP 层也是把 IP 首部封装在其中，因为有数据区域才会有数据报首部的存在，在大多数情况下，IP 数据报中的数据字段包含要交付给目标 IP 地址的运输层 (TCP 协议或 UDP 协议)，当然数据区域也可承载其他类型的报文，如 ICMP 报文等。

9.2.1 IP 数据报结构

在 lwIP 中，为了描述 IP 报文结构，它在 ip4.h 文件中定义了一个 ip_hdr 结构体来描述 IP 数据报的内容，该结构体如下所示：

```
struct ip_hdr {
    /* 版本号+首部长度+服务类型 */
    PACK_STRUCT_FLD_8(u8_t _v_hl);
    /* 服务类型 */
    PACK_STRUCT_FLD_8(u8_t _tos);
    /* 总长度(IP 首部+数据区) */
    PACK_STRUCT_FIELD(u16_t _len);
    /* 数据包标识(编号) */
    PACK_STRUCT_FIELD(u16_t _id);
    /* 标志+片偏移 */
    PACK_STRUCT_FIELD(u16_t _offset);
    /* IP 首部标志定义 */
#define IP_RF 0x8000U          /* 保留 */
#define IP_DF 0x4000U          /* 是否允许分片 */
#define IP_MF 0x2000U          /* 后续是否还有更多分片 */
#define IP_OFFMASK 0x1fffU     /* 片偏移域掩码 */
    /* 生存时间(最大转发次数)+协议类型(IGMP:1、UDP:17、TCP:6) */
    PACK_STRUCT_FLD_8(u8_t _ttl);
    /* 协议 */
    PACK_STRUCT_FLD_8(u8_t _proto);
    /* 校验和(IP 首部) */
    PACK_STRUCT_FIELD(u16_t _chksum);
    /* 源 IP 地址/目的 IP 地址 */
    PACK_STRUCT_FLD_S(ip4_addr_p_t src);
    PACK_STRUCT_FLD_S(ip4_addr_p_t dest);
} PACK_STRUCT_STRUCT;
PACK_STRUCT_END
```

可以看出，此结构体的成员变量和上图 9.2.1 的字段一一对应。

9.2.2 IP 数据报的分片解析

TCP/IP 协议栈为什么具备分片的概念，因为应用程序处理的数据是不确定的，可能超出网络接口最大传输单元，为此 TCP/IP 协议栈引入了分片概念，它是以 MTU 为界限对这个大型的数据切割成多个小型的数据包。这些小型的数据叫做 IP 的分组和分片，它们在接收方进行重组处理，这样，接收方的应用程序接收到这个大型的数据了。总的来讲，IP 数据报的分片概念是为了解决 IP 数据报数据过大的问题而诞生。注：以太网最大传输单元 MTU 为 1500。

现在笔者举个示例，让大家更好的理解 IP 分片的原理：

假设 IP 数据报整体的大小为 4000 字节，IP 首部默认为 20 字节，而数据区域为 3980。由于以太网最大传输单元为 1500，所以 lwIP 内核会把这个数据报进行分片处理。

1. 第一个 IP 分片：

分片数据大小：20（IP 首部）+ 1480（数据区域）。

标识：888。

标志：IP_MF = 1 后续还有分片。

片偏移量：片偏移量是 0，单位是 8 字节，本片偏移量相当于 0 字节。

2. 第二片 IP 数据报：

分片数据大小：20（IP 首部）+ 1480（数据区域）。

标识：888。

标志：IP_MF = 1 后续还有分片。

片偏移量：片偏移量是 185（1480/8），单位是 8 字节，本片偏移量相当于 1480 字节。

3. 第三片 IP 数据报：

分片数据大小：20（IP 首部）+ 1020（数据区域）。

标识：888。

标志：IP_MF = 0，后续没有分片。

片偏移量：片偏移量是 370（185+185），单位是 8 字节，本片偏移量相当于 2960 字节。

注：这些分片的标识都是一致的，而 IP_MF 表示后续有没有分片，若 IP_MF 为 0，则这个分片为最后一个分片。

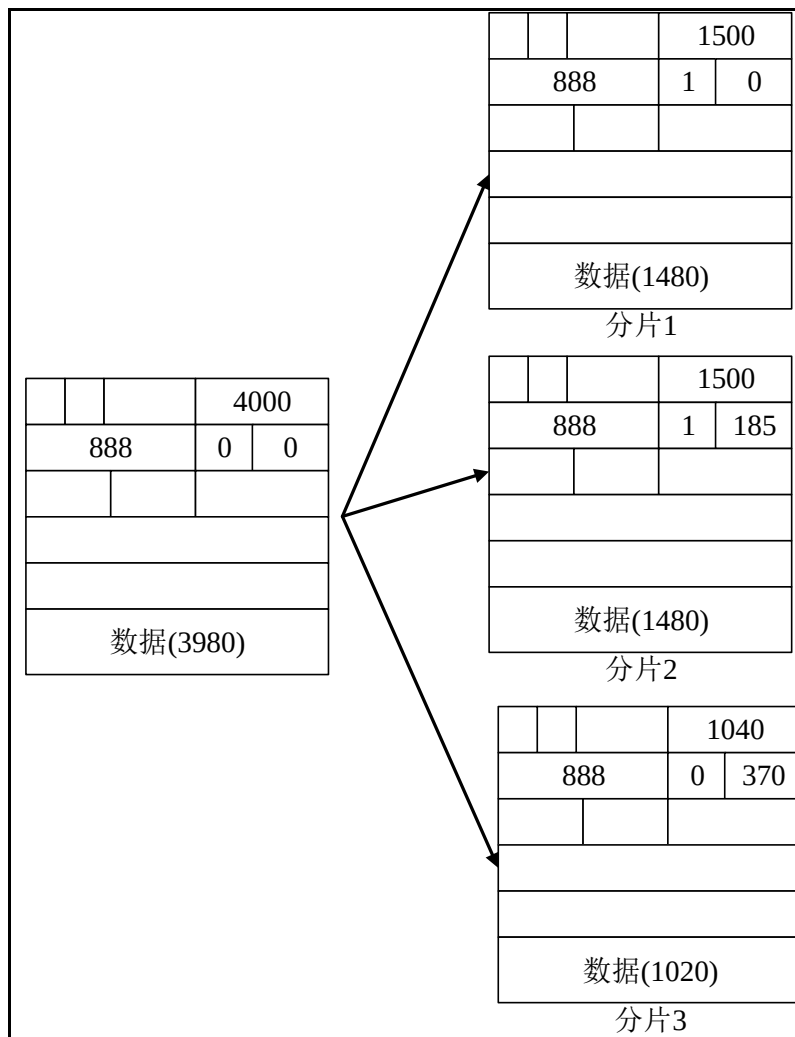


图 10.2.2.1 IP 数据报分片示意图

从上图可以看出，一个大型的 IP 数据包经过网络层处理，它会被分成两个或者两个以上

的 IP 分片，这些分片的数据组合起来就是应用程序发送的数据与传输层的首部。

至此，我们已经明白了 IP 分片的原理，下面笔者讲解 lwIP 内核如何实现这个原理，它的实现函数为 ip4_frag，该函数如下所示：

```
/**
 * 如果 IP 数据报对 netif 来说太大，则将其分片，
 * 将数据报切成 MTU 大小的块，然后按顺序发送通过将 pbuf_ref 指向 p
 * @param p:要发送的 IP 数据包
 * @param netif:发送的 netif
 * @param dest:目的 IP 地址
 * @return ERR_OK:发送成功, err_t:其他
 */
err_t
ip4_frag(struct pbuf *p, struct netif *netif, const ip4_addr_t *dest)
{
    struct pbuf *rambuf;
#ifdef LWIP_NETIF_TX_SINGLE_PBUF
    struct pbuf *newpbuf;
    u16_t newpbuflen = 0;
    u16_t left_to_copy;
#endif
    struct ip_hdr *original_iphdr;
    struct ip_hdr *iphdr;
    /* (1500 - 20)/8 = 偏移 185 */
    const u16_t nfb = (u16_t)((netif->mtu - IP_HLEN) / 8);
    u16_t left, fragsize;
    u16_t ofo;
    int last;
    u16_t poff = IP_HLEN; /* IP 头部长度 */
    u16_t tmp;
    int mf_set;
    original_iphdr = (struct ip_hdr *)p->payload; /* 指向数据报 */
    iphdr = original_iphdr;
    /* 判断 IP 头部是否为 20 */
    if (IPH_HL_BYTES(iphdr) != IP_HLEN) {
        return ERR_VAL;
    }
    /* tmp 变量获取标志和片偏移数值 */
    tmp = lwip_ntohs(IPH_OFFSET(iphdr));
    /* ofo = 片偏移 */
    ofo = tmp & IP_OFFMASK;
    /* mf_set = 分片标志 */
    mf_set = tmp & IP_MF;
    /* left = 总长度减去 IP 头部等于有效数据长度, 4000 - 20 = 3980 */

```

```

left = (u16_t)(p->tot_len - IP_HLEN);
/* 判断 left 是否为有效数据 */
while (left) {
    /* 判断有效数据和偏移数据大小, fragsize = 1480 (3980 < 1480 ? 3980 : 1480) */
    fragsize = LWIP_MIN(left, (u16_t)(nfb * 8));
    /* rambuf 申请 20 字节大小的内存块 */
    rambuf = pbuf_alloc(PBUF_LINK, IP_HLEN, PBUF_RAM);
    if (rambuf == NULL) {
        goto memerr;
    }
    /* 这个 rambuf 有效数据指针指向 original_iphdr 数据报 */
    SMEMCPY(rambuf->payload, original_iphdr, IP_HLEN);

    /* iphdr 指向有效区域地址 rambuf->payload */
    iphdr = (struct ip_hdr *)rambuf->payload;
    /* left_to_copy = 偏移数据大小(1480) */
    left_to_copy = fragsize;
    while (left_to_copy) {
        struct pbuf_custom_ref *pcr;
        /* 当前 pbuf 中数据的长度, plen = 3980 - 20 = 3960 */
        u16_t plen = (u16_t)(p->len - poff);
        /* newpbuflen = 1480 (1480 < 3960 ? 1480 : 3960) */
        newpbuflen = LWIP_MIN(left_to_copy, plen);

        if (!newpbuflen) {
            poff = 0;
            p = p->next;
            continue;
        }
        /* pcr 申请内存 */
        pcr = ip_frag_alloc_pbuf_custom_ref();
        if (pcr == NULL) {
            pbuf_free(rambuf);
            goto memerr;
        }
        /* newpbuf 申请内存 1480 字节,
        保存了这个数据区域偏移 poff 字节的数据 (p->payload + poff) */
        newpbuf = pbuf_alloc_custom(PBUF_RAW, newpbuflen, PBUF_REF, &pcr->pc,
                                     (u8_t *)p->payload + poff, newpbuflen);
        if (newpbuf == NULL) {
            /* 释放内存 */
            ip_frag_free_pbuf_custom_ref(pcr);
            pbuf_free(rambuf);
        }
    }
}

```

```

        goto memerr;
    }

    /* 增加 pbuf 的引用计数 */
    pbuf_ref(p);
    pcr->original = p;
    pcr->pc.custom_free_function = ipfrag_free_pbuf_custom;
    /* 将它添加到 rambuf 的链的末尾 */
    pbuf_cat(rambuf, newpbuf);
    /* left_to_copy = 0 (1480 - 1480) */
    left_to_copy = (u16_t)(left_to_copy - newpbuf->len);

    if (left_to_copy) {
        poff = 0;
        p = p->next;
    }
}

/* poff = 1500 (20 + 1480) */
poff = (u16_t)(poff + newpbuf->len);
/* last = 0 (3980 <= (1500 - 20)) */
last = (left <= netif->mtu - IP_HLEN);
/* 设置新的偏移量和 MF 标志 */
tmp = (IP_OFFMASK & (ofo));
/* 判断是否是最后一个分片 */
if (!last || mf_set) {
    /* 最后一个片段设置了 MF 为 0 */
    tmp = tmp | IP_MF;
}

/* 分段偏移与标志字段 */
IPH_OFFSET_SET(iphdr, lwip_htons(tmp));
/* 设置数据报总长度 = 1500 (1480 + 20) */
IPH_LEN_SET(iphdr, lwip_htons((u16_t)(fragsize + IP_HLEN)));
/* 校验为 0 */
IPH_CHKSUM_SET(iphdr, 0);
/* 发送 IP 数据报 */
netif->output(netif, rambuf, dest);
IPFRAG_STATS_INC(ip_frag.xmit);
/* rambuf 释放内存 */
pbuf_free(rambuf);
/* left = 2500 (3980 - 1480) */
left = (u16_t)(left - fragsize);
/* 片偏移 ofo = 185(0 + 185) */
ofo = (u16_t)(ofo + nfb);
}

```



```

MIB2_STATS_INC(mib2.ipfragoks);
return ERR_OK;
memerr:
MIB2_STATS_INC(mib2.ipfragfails);
return ERR_MEM;
}

MIB2_STATS_INC(mib2.ipfragoks);
return ERR_OK;
memerr:
MIB2_STATS_INC(mib2.ipfragfails);
return ERR_MEM;
}

```

此函数非常简单，首先判断这个大型数据包的有效区域总长度，系统根据这个总长度划分数据区域，接着申请 $20 + \text{sizeof}(\text{struct pbuf})$ 字节的 `rambuf` 来存储 IP 首部，然后根据 `poff` 数值让被分片数据包的 `payload` 指针偏移 `poff` 大小，它所指向的地址由 `newpbuf` 数据包的 `payload` 指针指向，最后调用 `netif->output` 函数发送该分片，其他分片一样操作。

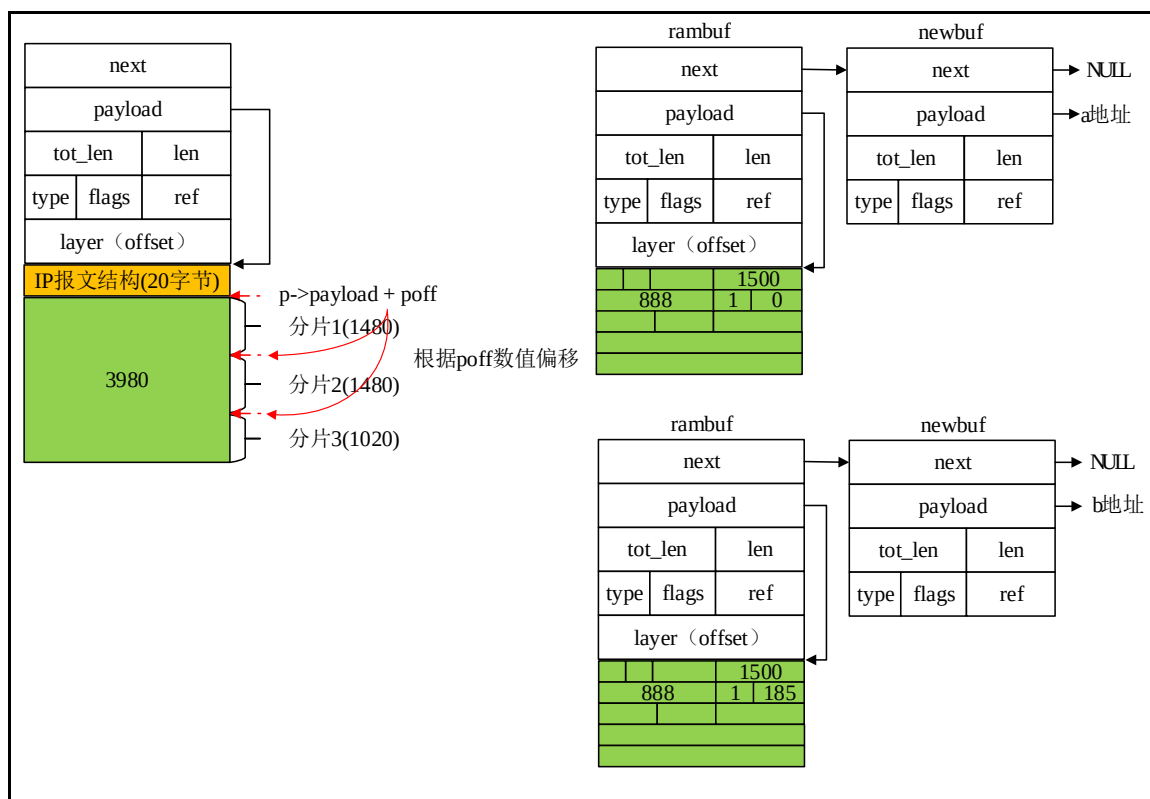


图 9.2.2.2 IP 数据报分片原理示意图

上图中，`newpbuf` 的 `payload` 指针指向的地址由左边的 `payload` 指针经过偏移得来的。

9.2.3 IP 数据报的分片重装

由于 IP 分组在网络传输过程中到达目的地点的时间是不确定的，所以后面的分组可能比前面的分组先达到目的地点。为此，lwIP 内核需要将接收到的分组暂存起来，等所有的分组都接收完成之后，再将数据传递给上层。

在 lwIP 中，有专门的结构体负责缓存这些分组，这个结构体为 ip_reassdata 重装数据链表，该结构体在 ip4_frag.h 文件中定义，如下所示：

```
/* 重装数据结构体 */
struct ip_reassdata {
    struct ip_reassdata *next; /* 指向下一个重装节点 */
    struct pbuf *p; /* 指向分组的 pbuf */
    struct ip_hdr iphdr; /* IP 数据报的首部 */
    u16_t datagram_len; /* 已收到数据的长度 */
    u8_t flags; /* 标志是否最后一个分组 */
    u8_t timer; /* 超时间隔 */
};
```

这个结构体描述了同类型的 IP 分组信息，同类型的 IP 分组会挂载到该重装节点上，如下图所示。

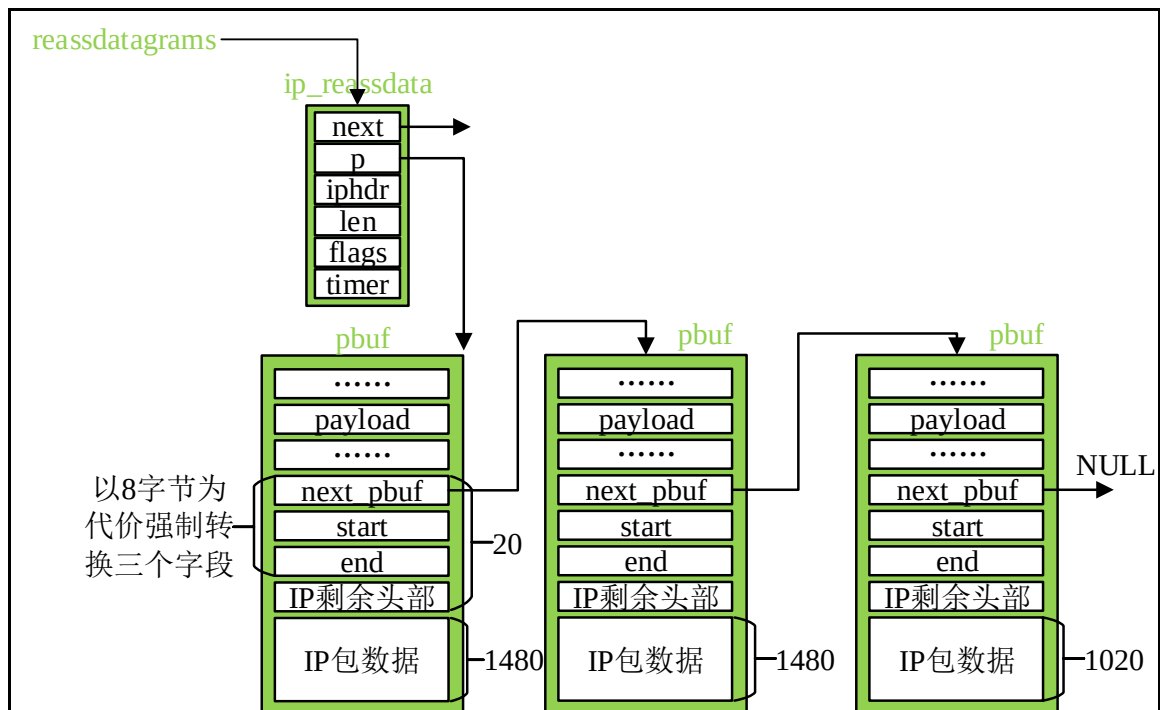


图 9.2.3.3 IP 分组重装示意图

可以看到，这些分片挂载到同一个重装节点上，它们挂载之前，是把 IP 首部的前 8 字节强制转换成三个字段，其中 next_pbuf 指针用来链接这些 IP 分组，形成了单向链表，而 start 和 end 字段用来描述分组的顺序，lwIP 系统根据这些数值对分组进行排序。

lwIP 内核的 IP 重组功能由 ip4_reass 函数实现，该函数的代码量比较长，这里笔者不深入讲解了，我们会在视频当中讲解 IP 重装流程。

9.3 IP 数据报的输出

无论是 UDP 还是 TCP，它们的数据段递交至网络层的接口是一致的，这个接口函数如下所示：

```
err_t
ip4_output_if_src(struct pbuf *p, const ip4_addr_t *src, const ip4_addr_t *dest,
```

```

        u8_t ttl, u8_t tos,
        u8_t proto, struct netif *netif)
{
    struct ip_hdr *iphdr;
    ip4_addr_t dest_addr;
    if (dest != LWIP_IP_HDRINCL){
        u16_t ip_hlen = IP_HLEN;
        /* 第一步: 生成 IP 报头 */
        if (pbuf_header(p, IP_HLEN)){
            return ERR_BUF;
        }
        /* 第二步: iphdr 指向 IP 头部指针 */
        iphdr = (struct ip_hdr *)p->payload;
        /* 设置生存时间(最大转发次数) */
        IPH_TTL_SET(iphdr, ttl);
        /* 设置协议类型(IGMP:1、UDP:17、TCP:6) */
        IPH_PROTO_SET(iphdr, proto);
        /* 设置目的 IP 地址 */
        ip4_addr_copy(iphdr->dest, *dest);
        /* 设置版本号+设置首部长度 */
        IPH_VHL_SET(iphdr, 4, ip_hlen / 4);
        /* 服务类型 */
        IPH_TOS_SET(iphdr, tos);
        /* 设置总长度(IP 首部+数据区) */
        IPH_LEN_SET(iphdr, lwip_htons(p->tot_len));
        /* 设置标志+片偏移 */
        IPH_OFFSET_SET(iphdr, 0);
        /* 设置数据包标识(编号) */
        IPH_ID_SET(iphdr, lwip_htons(ip_id));
        /* 每发送一个数据包, 编号加一 */
        ++ip_id;
        /* 没有指定源 IP 地址 */
        if (src == NULL){
            /* 将当前网络接口 IP 地址设置为源 IP 地址 */
            ip4_addr_copy(iphdr->src, *IP4_ADDR_ANY4);
        }
        else{
            /* 复制源 IP 地址 */
            ip4_addr_copy(iphdr->src, *src);
        }
    }
    else{
        /* IP 头部已经包含在 pbuf 中 */

```

```

    iphdr = (struct ip_hdr *)p->payload;
    ip4_addr_copy(dest_addr, iphdr->dest);
    dest = &dest_addr;
}
IP_STATS_INC(ip.xmit);
ip4_debug_print(p);
/* 如果数据包总长度大于 MTU，则分片发送 */
if (netif->mtu && (p->tot_len > netif->mtu)){
    return ip4_frag(p, netif, dest);
}
/* 如果数据包总长度不大于 MTU，则直接发送 */
return netif->output(netif, p, dest);
}

```

此函数非常简单，这里笔者使用一个流程图来描述该函数的实现原理，如下图所示。

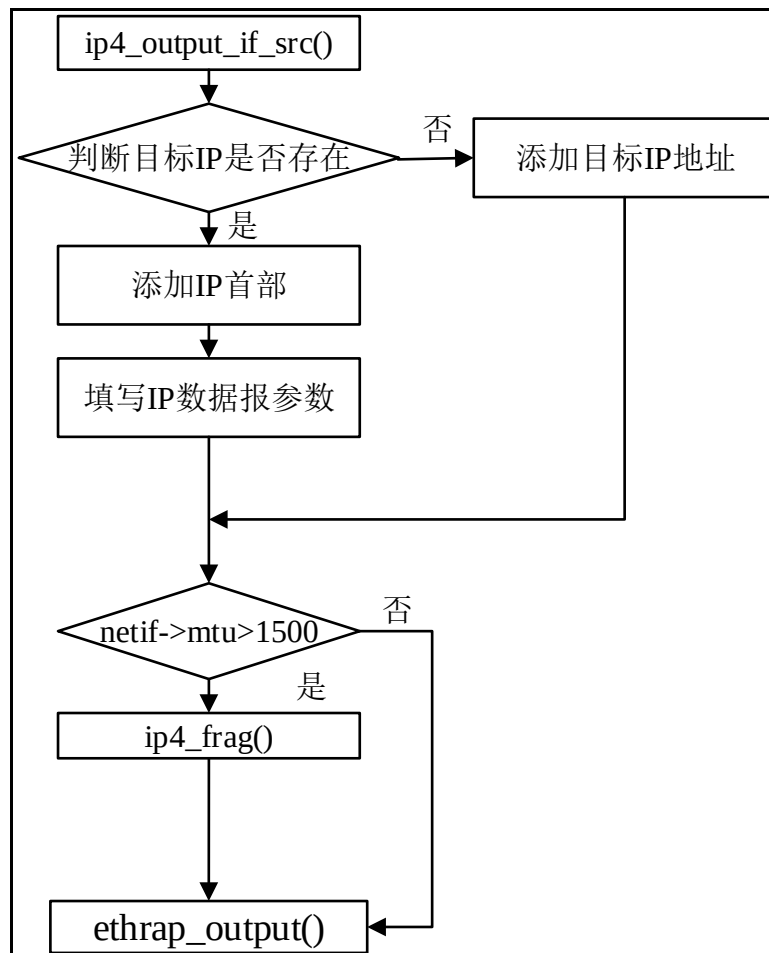


图 9.3.1 函数 ip4_output_if_src()流程图

此函数首先判断目标 IP 地址是否为 NULL，若目标 IP 地址不为空，则偏移 payload 指针添加 IP 首部，偏移完成之后设置 IP 首部字段信息，接着判断该数据包的总长度是否大于以太网传输单元，若大于，则调用 ip4_frag 函数对这个数据包分组并且逐一发送，否则直接调用 ethrap_output 函数把数据包递交给 ARP 层处理。

9.4 IP 数据报的输入

数据包提交给网络层之前，系统需要判断接收到的数据包是 IP 数据包还是 ARP 数据包，若接收到的是 IP 数据包，则 lwIP 内核调用 ip4_input 函数处理这个数据包，该函数如下所示：

```
err_t
ip4_input(struct pbuf *p, struct netif *inp)
{
    struct ip_hdr *iphdr;
    struct netif *netif;
    u16_t iphdr_hlen;
    u16_t iphdr_len;
#ifdef IP_ACCEPT_LINK_LAYER_ADDRESSING || LWIP_IGMP
    int check_ip_src = 1;
#endif /* IP_ACCEPT_LINK_LAYER_ADDRESSING || LWIP_IGMP */

    IP_STATS_INC(ip.recv);
    MIB2_STATS_INC(mib2.ipinreceives);

    /* 识别 IP 报头 */
    iphdr = (struct ip_hdr *)p->payload;
    /* 第一步：判断版本是否为 IPv4 */
    if (IPH_V(iphdr) != 4)
    {
        ip4_debug_print(p);
        pbuf_free(p);          /* 释放空间 */
        IP_STATS_INC(ip.err);
        IP_STATS_INC(ip.drop);
        MIB2_STATS_INC(mib2.ipinhdrrerrors);
        return ERR_OK;
    }

    /* 以 4 字节（32 位）字段获得 IP 头的长度 */
    iphdr_hlen = IPH_HL(iphdr);
    /* 以字节计算 IP 报头长度 */
    iphdr_hlen *= 4;
    /* 以字节为单位获取 ip 长度 */
    iphdr_len = lwip_ntohs(IPH_LEN(iphdr));

    /* 修剪 pbuf。这对于 < 60 字节的数据包尤其需要。 */
    if (iphdr_len < p->tot_len)
    {
        pbuf_realloc(p, iphdr_len);
    }
}
```

```

/* 第二步：标头长度超过第一个 pbuf 长度，或者 ip 长度超过总 pbuf 长度 */
if ((iphdr_hlen > p->len) || (iphdr_len > p->tot_len)
    || (iphdr_hlen < IP_HLEN))
{
    if (iphdr_hlen < IP_HLEN)
    {
    }
    if (iphdr_hlen > p->len)
    {
    }
    if (iphdr_len > p->tot_len)
    {
    }
    /* 释放空间 */
    pbuf_free(p);
    IP_STATS_INC(ip.lenerr);
    IP_STATS_INC(ip.drop);
    MIB2_STATS_INC(mib2.ipindiscards);
    return ERR_OK;
}

/* 第三步：验证校验和 */
#ifdef CHECKSUM_CHECK_IP
    /* 省略代码 */
#endif

/* 将源 IP 地址与目标 IP 地址复制到对齐的 ip_data.current_iphdr_src 和
   ip_data.current_iphdr_dest */
ip_addr_copy_from_ip4(ip_data.current_iphdr_dest, iphdr->dest);
ip_addr_copy_from_ip4(ip_data.current_iphdr_src, iphdr->src);

/* 第四步：匹配数据包和接口，即这个数据包是否发给本地 */
if (ip4_addr_ismulticast(ip4_current_dest_addr()))
{
#ifdef LWIP_IGMP
    /* 省略代码 */
#else /* LWIP_IGMP */
    /* 如果网卡已经挂载了和 IP 地址有效 */
    if ((netif_is_up(inp)) && (!ip4_addr_isany_val(*netif_ip4_addr(inp))))
    {
        netif = inp;
    }
}

```

```
        else
        {
            netif = NULL;
        }

#endif /* LWIP_IGMP */
    }

    /* 如果数据报不是发给本地 */
    else
    {
        int first = 1;
        netif = inp;

        do
        {
            /* 接口已启动并配置? */
            if ((netif_is_up(netif)) &&
                (!ip4_addr_isany_val(*netif_ip4_addr(netif))))
            {
                /* 单播到此接口地址? */
                if (ip4_addr_cmp(ip4_current_dest_addr(),
                                netif_ip4_addr(netif)) ||
                    /* 或广播在此接口网络地址? */
                    ip4_addr_isbroadcast(ip4_current_dest_addr(), netif)

#if LWIP_NETIF_LOOPBACK && !LWIP_HAVE_LOOPIF
                    || (ip4_addr_get_u32(ip4_current_dest_addr()) ==
                        PP_HTONL(IPADDR_LOOPBACK))
#endif
                )
                {
                    break;
                }
            }

#if LWIP_AUTOIP
            if (autoip_accept_packet(netif, ip4_current_dest_addr()))
            {
                /* 跳出 if 循环 */
                break;
            }
#endif

        } while (1);

#endif /* LWIP_AUTOIP */
    }
}
```

```
        if (first)
        {
#if !LWIP_NETIF_LOOPBACK || LWIP_HAVE_LOOPIF

            /* 检查一下目标 IP 地址是否是环回地址 */
            if (ip4_addr_isloopback(ip4_current_dest_addr()))
            {
                netif = NULL;
                break;
            }

#endif /* !LWIP_NETIF_LOOPBACK || LWIP_HAVE_LOOPIF */
            first = 0;
            netif = netif_list;
        }
        else
        {
            netif = netif->next;
        }

        if (netif == inp)
        {
            netif = netif->next;
        }
    } while (netif != NULL);
}

#if IP_ACCEPT_LINK_LAYER_ADDRESSING
if (netif == NULL)
{
    /* 远程端口是 DHCP 服务器? */
    if (IPH_PROTO(iphdr) == IP_PROTO_UDP)
    {
        struct udp_hdr *udphdr = (struct udp_hdr *)
            ((u8_t *)iphdr + iphdr_hlen);

        if (IP_ACCEPT_LINK_LAYER_ADDRESSED_PORT(udphdr->dest))
        {
            netif = inp;
            check_ip_src = 0;
        }
    }
}
```



```

    }

#endif /* IP_ACCEPT_LINK_LAYER_ADDRESSING */

#if LWIP_IGMP || IP_ACCEPT_LINK_LAYER_ADDRESSING

    if (check_ip_src
#ifdef IP_ACCEPT_LINK_LAYER_ADDRESSING
        && !ip4_addr_isany_val(*ip4_current_src_addr())
#endif /* IP_ACCEPT_LINK_LAYER_ADDRESSING */
    )
#endif /* LWIP_IGMP || IP_ACCEPT_LINK_LAYER_ADDRESSING */
    {
        /* 第五步: IP 地址, 源 IP 地址不能是多播或者广播地址 */
        if ((ip4_addr_isbroadcast(ip4_current_src_addr(), inp)) ||
            (ip4_addr_ismulticast(ip4_current_src_addr())))
        {
            /* 释放空间 */
            pbuf_free(p);
            IP_STATS_INC(ip.drop);
            MIB2_STATS_INC(mib2.ipinaddrerrors);
            MIB2_STATS_INC(mib2.ipindiscards);
            return ERR_OK;
        }
    }

    /* 第六步: 如果还没找到对应的网卡, 数据包不是给我们的 */
    if (netif == NULL)
    {
        /* 路由转发或者丢弃。如果 IP_FORWARD 宏定义被使能, 则进行转发 */
#ifdef IP_FORWARD
        /* 非广播包? */
        if (!ip4_addr_isbroadcast(ip4_current_dest_addr(), inp))
        {
            /* 尝试在 (其他) 网卡上转发 IP 数据包 */
            ip4_forward(p, iphdr, inp);
        }
        else
#endif /* IP_FORWARD */
        {
            IP_STATS_INC(ip.drop);
            MIB2_STATS_INC(mib2.ipinaddrerrors);
            MIB2_STATS_INC(mib2.ipindiscards);
        }
    }
}

```

```

    }

    /* 释放空间 */
    pbuf_free(p);
    return ERR_OK;
}

/* 第七步：如果数据报由多个片段组成（分片处理）？ */
if ((IPH_OFFSET(iphdr) & PP_HTONS(IP_OFFMASK | IP_MF)) != 0)
{
    /* 重装数据报 */
    p = ip4_reass(p);

    /* 如果重装没有完成 */
    if (p == NULL)
    {
        return ERR_OK;
    }

    /* 分片重装完成，将数据报首部强制转换为 ip_hdr 类型 */
    iphdr = (struct ip_hdr *)p->payload;
}

#if IP_OPTIONS_ALLOWED == 0

#if LWIP_IGMP

    if ((iphdr_hlen > IP_HLEN) && (IPH_PROTO(iphdr) != IP_PROTO_IGMP))
    {
#else
    /* 第八步：如果 IP 数据报首部长度大于 20 字节，就表示错误 */
    if (iphdr_hlen > IP_HLEN)
    {
#endif /* LWIP_IGMP */
        /* 释放空间 */
        pbuf_free(p);
        IP_STATS_INC(ip.opterr);
        IP_STATS_INC(ip.drop);
        /* u 不受支持的协议特性 */
        MIB2_STATS_INC(mib2.ipinunknownprotos);
        return ERR_OK;
    }
#endif /* IP_OPTIONS_ALLOWED == 0 */

    /* 第九步：发送到上层协议 */
    ip4_debug_print(p);

```

```

ip_data.current_netif = netif;
ip_data.current_input_netif = inp;
ip_data.current_ip4_header = iphdr;
ip_data.current_ip_header_tot_len = IPH_HL(iphdr) * 4;
#if LWIP_RAW

/* RAW API 输入 */
if (raw_input(p, inp) == 0)
#endif /* LWIP_RAW */
{
    /* 转移到有效载荷（数据区域），不需要检查 */
    pbuf_header(p, -(s16_t)iphdr_hlen);
    /* 根据 IP 数据报首部的协议的类型处理 */
    switch (IPH_PROTO(iphdr))
    {
#if LWIP_UDP
        /* UDP 协议 */
        case IP_PROTO_UDP:
#if LWIP_UDPLITE
            case IP_PROTO_UDPLITE:
#endif /* LWIP_UDPLITE */
                MIB2_STATS_INC(mib2.ipindeliivers);
                /* IP 层递交给网络层的函数 */
                udp_input(p, inp);
                break;
#endif /* LWIP_UDP */
#if LWIP_TCP
        /* TCP 协议 */
        case IP_PROTO_TCP:
            MIB2_STATS_INC(mib2.ipindeliivers);
            /* IP 层递交给网络层的函数 */
            tcp_input(p, inp);
            break;
#endif /* LWIP_TCP */

        pbuf_free(p); /* 释放空间 */

        IP_STATS_INC(ip.proterr);
        IP_STATS_INC(ip.drop);
        MIB2_STATS_INC(mib2.ipinunknownprotos);
    }
}

/* 全局变量清零 */
ip_data.current_netif = NULL;

```

```
ip_data.current_input_netif = NULL;
ip_data.current_ip4_header = NULL;
ip_data.current_ip_header_tot_len = 0;
ip4_addr_set_any(ip4_current_src_addr());
ip4_addr_set_any(ip4_current_dest_addr());

return ERR_OK;
}
```

上述的源码篇幅很长，也不容易理解，下面笔者把上述的源码分成十步来讲解：

第一步：判断 IP 数据报的版本是否是 IPv4，如果不是，那么 lwIP 会掉弃该数据报。

第二步：判断标头长度超过第一个 pbuf 长度，或者 ip 长度超过总 pbuf 长度，如果是，那么 lwIP 会丢弃该数据报。

第三步：验证校验和，如果不正确，那么 lwIP 会掉弃该数据报。

第四步：匹配数据包和接口，这个数据包是否发给本地。

第五步：判断 IP 数据报是否是广播或者多播，如果是，那么 lwIP 会丢弃该数据报。

第六步：如果到了这一步，没有发现网络接口，那么 lwIP 会丢弃该数据报。

第七步：如果如 IP 数据报不能分片处理，那么 lwIP 会丢弃该数据报。

第八步：如果 IP 数据报的 IP 首部大于 20 字节，那么 lwIP 会丢弃该数据报。

第九步：把数据包递交给上层。

第十步：判断该数据报的协议为 TCP/UDP/ICMP/IGMP，如果不是这四个协议，则丢弃该数据报。

第十章 ICMP 协议

ICMP (Internet Control Message Protocol) Internet 控制报文协议。它是 TCP/IP 协议簇的一个子协议，用于在 IP 主机、路由器之间传递控制消息。控制消息是指网络通不通、主机是否可达、路由是否可用等网络本身的消息，这些控制消息虽然并不传输到用户数据，但是对于用户数据的传递起着重要的作用。本章分为如下几个部分：

10.1 ICMP 协议简介

10.2 ICMP 的实现

10.1 ICMP 协议简介

IP 协议虽然是 TCP/IP 协议中的核心部分，但是它是一种无连接的不可靠数据报交付，这个协议本身没有任何错误检验和恢复机制，为了弥补 IP 协议中的缺陷，ICMP 协议登场了，ICMP 协议是一种面向无连接的协议，用于传输出错报告控制信息。它是一个非常重要的协议，它对于网络安全具有极其重要的意义。它属于网络层协议，主要用于在主机与路由器之间传递控制信息，包括报告错误、交换受限控制和状态信息等。当遇到 IP 数据无法访问目标、IP 路由器无法按当前的传输速率转发数据包等情况时，会自动发送 ICMP 消息。

ICMP 协议用于 IP 主机、路由器之间递交控制消息，在网络中，控制消息分为很多种，例如数据报错信息、网络状况信息和主句状况信息等，虽然这些信息不会递交给用户数据，但对于用户来说数据报有效性得到提高。

ICMP 应用场景

IP 协议本身不提供差错报告和差错控制机制来保证数据报递交的有效性，如果在路由器无法递交一个数据报或者数据报生存时间为 0 时，那么路由器会直接掉弃这个数据报，虽然 IP 层这样处理是合理的，但是对于源主机来说，比较希望得到数据报递交过程中出现异常相关信息，以便重新递交数据报或者其他处理。

IP 协议不能进行主机管理与查询机制，简单来说：不知道对方主机或者路由器的活跃，对于不活跃的主机和路由器就没有必要发送数据报，所以对于主机管理员来说：更希望得到对方主机和路由器的信息，这样可以根据相关的信息对自身配置、数据报发送控制。

为了解决上述的两个问题，TCP/IP 设计人员在协议上引入了特殊用途报文，这个报文为网际报文控制协议简称 ICMP，从 TCP/IP 的协议结构来看，它是和 IP 协议一样，都是处于网络层，但是 ICMP 协议有自己一套报文结构，这样数据报就变成了 IP 首部+ICMP 首部+数据区域，ICMP 协议不为任何的应用程序服务，它的目的是目的主机的网络层处理软件。

10.1.1 ICMP 报文类型

在没有引入 ICMP 报文之前，IP 数据报一般分为 IP 首部+IP 数据区域，现在添加了 ICMP 协议，则 IP 数据报分为 IP 首部+ICMP 首部+数据区域。ICMP 报文分为两类：一类是 ICMP 差错报告报文，另一类是 ICMP 查询报文，这两类报文分别解决上小节的问题。

① ICMP 差错报告报文主要用来向 IP 数据报源主机返回一个差错报告信息，这个信息就是判断路由器和主机对当前的数据报进行正常处理，例如无法将数据报递交给上层处理，或者数据报因为生存时间而被删除。

② ICMP 查询报文用于一台主机向另一台主机查询特定的信息，这个类型的报文是成对出现的，例如源主机发送查询报文，当目标主机收到该报文之后，它会根据查询报文的约定的格

式为源主机放回应答报文。

ICMP 差错报告报文和 ICMP 查询报文常见类型如下表所示：

ICMP 报文类型	具体类型	描述
ICMP 差错报文	3	目的不可达
	4	源站抑制
	5	重定义
	11	超时
	12	参数错误报文
ICMP 查询报文	8/0	回显请求或回显应答
	9/10	路由器询问或通告
	13/14	时间戳请求或应答
	15/16	信息请求或信息应答
	17/18	掩码请求或应答

表 10.1.1 差错报文与查询报文常见类型

注：lwIP 只实现差错报文的类型 3 和 11，而查询报文只处理回显请求。

10.1.2 ICMP 报文结构

ICMP 报文有 8 字节首部和可变长度的数据部分组成，因为 ICMP 有两种类型的报文，其中不同的报文其首部的格式也会有点差异，当然也有通用的地方，例如首部的前 4 个字节是通用的，ICMP 报文结构如下图所示。



图 10.1.2.1 ICMP 报文格式

类型字段：表示使用 ICMP 的两类类型中的哪一个。

代码字段：产生 ICMP 报文的具体原因。

校验和字段：用于记录包括 ICMP 报文数据部分在内的整个 ICMP 数据报的校验和。

首部剩余的 4 字节在每种类型的报文有特殊的定义，总的来说：不同类型的报文，数据部分长度和含义存在差异，例如差错报文会引起差错的据报的信息，而查询报文携带查询请求和查询结果数据。

1. ICMP 差错报文

(1) 目的站不可到达

当路由器发送的数据报不能发送到指定目的地时，或者说当路由器不能够给数据报找到路由或主机不能够交付数据报时，就丢弃这个数据报，然后向发送数据报的源主机设备发回一个终点不可达数据报文。如下图所示。

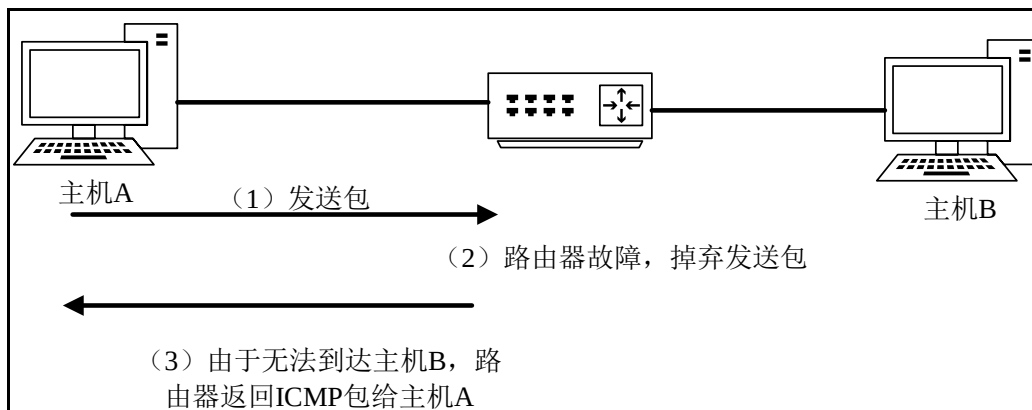


图 10.1.2.2 数据报不可到达

举个例子：主机 A 给主机 B 发送一个数据报，在网络中传输时中间可能要经过很多台路由器，主机 A 先把这个数据报发送给路由器，路由器收到这个数据报后，此时路由 R1 发生了故障，它不知道这个数据报下一步该发给哪个路由设备或者那台主机设备，也就是说这个数据报不能发送到目的地主机 B，这时路由器会把这个数据报丢弃并向主机 A 发回一个终点不可达的数据报文。

ICMP 目的不可达差错报告报文产生差错的原因有很多，如网络不可达、主机不可达、协议不可达、端口不可达等，引起差错的原因会在 ICMP 报文中的代码字段（Code）记录。对于不同的差错代码字段的值是不一样的，但是 lwIP 实现的只有前 6 种，如下图所示。

代码值	描述
0	网络不可达
1	主机不可达
2	协议不可达
3	端口不可达
4	需要分片但设置了不分片
5	源站路由失败

表 10.1.2.3 目的不可达的代码字段取值描述

当然 ICMP 目的不可达报文首部剩下的 4 字节是未使用，而 ICMP 报文数据区装载了 IP 数据报首部及 IP 数据报的数据区域前 8 字节，为什么需要装载 IP 数据报的数据区域中前 8 字节的数据呢？因为 IP 数据报的数据区域前 8 个字节刚好覆盖了传输层协议中的端口号字段，而 IP 数据报首部就拥有目标 IP 地址与源 IP 地址，当源主机收到这样子的 ICMP 报文后，它能根据 ICMP 报文的数据区域判断出是哪个数据包出现问题，并且 IP 层能够根据端口号将报文传递给对应的上层协议处理，差错报文结构如下图所示。

类型	代码:(0-15)	校验和
未使用 (0)		
引起差错的IP首部 + 数据区域（前8字节）		

图 10.1.2.4 ICMP 目的不可达报文

可以看出：首部剩下的 4 个字节是未使用的，而数据区域保存的是引起差错 IP 首部和引起差错数据包的数据区域前 8 字节数据。准确来说，就是把引起差错 IP 数据包的 IP 部和数据区域的前 8 字节数据拷贝到差错报文的数据区域。

(2) 源站抑制

由于 IP 协议是面向无连接的，没有流量控制机制，数据在传输过程中是很容易造成拥塞的现象。而 ICMP 源点抑制报文就是给 IP 协议提供一种流量监控的机制，因为 ICMP 源点抑制机制并不能控制流量的大小，但是能根据流量的使用情况，给源主机提供一些建议。这个报文的作用就是通知数据报在拥塞时被丢弃了，另外还会警告源主机流量出现了拥塞的情况，然后源主机根据反馈的 ICMP 源点抑制报文信息作出处理，至于源主机怎么就不关它的事了。如下图所示。

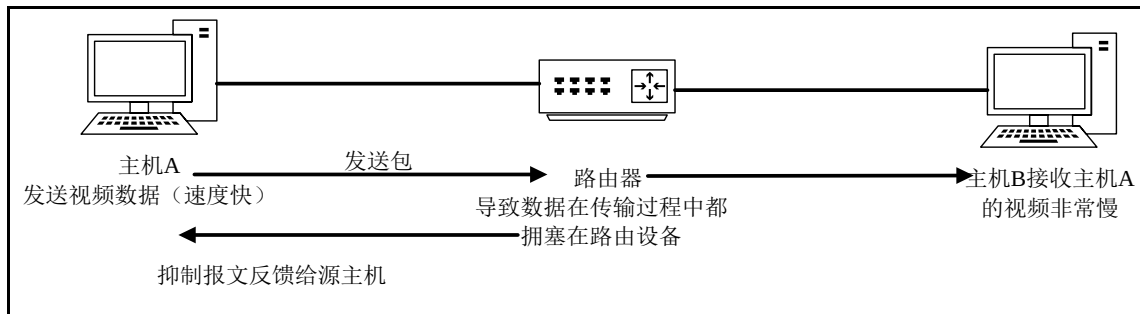


图 10.1.2.5 源站抑制示意图

(3) 端口不可达

当目标系统收到一个 IP 数据报的某个服务请求时，如果本地没有此服务，则本地会向源头返回 ICMP 端口不可达信息。常见的端口不可达有：主机 A 向主机 B 发起一个 ftp 的传输请求，从主机 B 传输一个文件到主机 A，由于主机 B 设备没有开启 ftp 服务的 69 端口，因此主机 A 在请求主机 B 时，会收到主机 B 回复的一个 ICMP 端口不可达的差错报文。

(4) 超时

ICMP 差错报告报文主要在以下几种情况中，会发送 ICMP 超时报文：

1. 当路由器接收到的数据报的 TTL 生命周期字段值为 0 时，路由器会把该数据报丢弃掉，并向源主机发回一个 ICMP 超时报文。
2. 另外，当目标主机在规定时间内没有收到所有的数据分片时，会把已经收到的所有数据分片丢弃，并向源主机发回一个 ICMP 超时报文。在超时报文中，代码 0 只能给路由器使用，表示生存周期字段值为 0，代码 1 只能给目的主机使用，它表示在规定的时间内，目的主机没有收到所有的数据分片。

(5) 参数错误

当数据报在因特网上传送时，在其首部中出现的任何二义性或者首部字段值被修改都可能会产生非常严重的问题。如果路由器或目的主机发现了这种二义性，或在数据报的某个字段中缺少某个值，就丢弃这个数据报，并回送参数问题报文。

2. ICMP 查询报文

ping 程序利用 ICMP 回显请求报文和回显应答报文（而不用经过传输层）来测试目标主机是否可达。它是一个检查系统连接性的基本诊断工具。

ICMP 回显请求和 ICMP 回显应答报文是配合工作的。当源主机向目标主机发送了 ICMP 回显请求数据包后，它期待着目标主机的回答。目标主机在收到一个 ICMP 回显请求数据包后，它会交换源、目的主机的地址，然后将收到的 ICMP 回显请求数据包中的数据部分原封不动地封装在自己的 ICMP 回显应答数据包中，然后发回给发送 ICMP 回显请求的一方。如果校验正确，发送者便认为目标主机的回显服务正常，也即物理连接畅通。查询报文结构如下图所示。

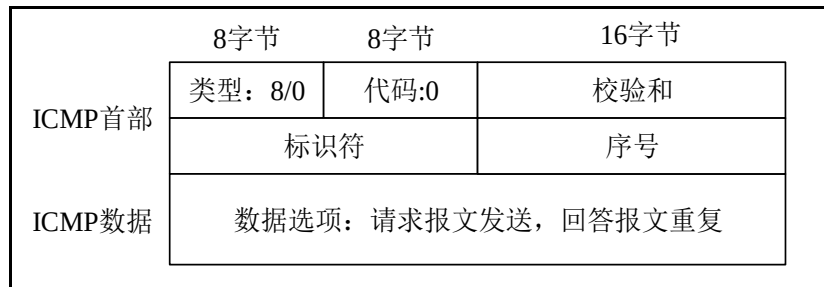


图 10.1.2.7 ICMP 回显请求和回显应答报文格式

类型字段是指请求报文（8）和回答报文（0），代码段在 ICMP 查询报文没有特殊取值，其值为 0，首部中的标识符和序号在 ICMP 中没有正式定义该值的范围，所以发送方可以自由定义这两个字段，可以用来记录源主机发送出去的请求报文编号。数据可选区域标识回送请求报文包含数据和长度是可选的，发送方应该选择适合的长度和填充数据。在接收方它可以根据这个回送请求产生一个回送回答报文，回送报文的数据与回送请求报文的数据是相同的。

10.2 ICMP 的实现

我们可以总结一下 ICMP 协议的作用，ICMP 协议是 IP 协议的辅助协议，为什么 ICMP 协议是 IP 协议的辅助协议呢？由于 IP 协议本身不提供差错报告和差错控制机制来保证数据报递交的有效性和进行主机管理与查询机制，简单来说：ICMP 协议为了解决 IP 协议的缺陷而诞生的，ICMP 报文分为差错报文和查询报文，这两个报文分别解决 IP 协议的两大缺陷，本小节主要讲解 lwIP 是怎么样实现 ICMP 协议发送及处理的。

10.2.1 ICMP 数据结构体

在讲述 IP 协议时，它是有自己的数据结构，同样 ICMP 也有它自己的数据结构 icmp_echo_hdr，该数据结构在 lwIP 的 icmp.h 文件中定义，该结构体如下源码所示：

```
PACK_STRUCT_BEGIN
struct icmp_echo_hdr {
    PACK_STRUCT_FLD_8(u8_t type);           /* ICMP 类型 */
    PACK_STRUCT_FLD_8(u8_t code);           /* ICMP 代码号 */
    PACK_STRUCT_FIELD(u16_t chksum);        /* ICMP 校验和 */
    PACK_STRUCT_FIELD(u16_t id);            /* ICMP 的标识符 */
    PACK_STRUCT_FIELD(u16_t seqno);        /* 序号 */
} PACK_STRUCT_STRUCT;
PACK_STRUCT_END
```

此外 lwIP 还定义了很多宏与枚举类型的变量对 ICMP 的类型及代码字段进行描述，如下源码所示：

```
#define ICMP_ER    0    /* 回送应答 */
#define ICMP_DUR   3    /* 目标不可达 */
#define ICMP_SQ    4    /* 源站抑制 */
#define ICMP_RD    5    /* 重定向 */
#define ICMP_ECHO  8    /* 回送 */
#define ICMP_TE    11   /* 超时 */
#define ICMP_PP    12   /* 参数问题 */
#define ICMP_TS    13   /* 时间戳 */
```

```
#define ICMP_TSR 14      /* 时间戳应答 */
#define ICMP_IRQ 15      /* 信息请求 */
#define ICMP_IR 16       /* 信息应答 */
#define ICMP_AM 17       /* 地址掩码请求 */
#define ICMP_AMR 18      /* 地址掩码应答 */

/* ICMP 目标不可到达的代码 */
enum icmp_dur_type {
    /* 网络不可到达 */
    ICMP_DUR_NET = 0,
    /* 主机不可达 */
    ICMP_DUR_HOST = 1,
    /* 协议不可到达 */
    ICMP_DUR_PROTO = 2,
    /* 端口不可达 */
    ICMP_DUR_PORT = 3,
    /* 需要进行分片但设置不分片比特 */
    ICMP_DUR_FRAG = 4,
    /* 源路由失败 */
    ICMP_DUR_SR = 5
};

/* ICMP 时间超时代码 */
enum icmp_te_type {
    /* 在运输过程中超出了生存时间 */
    ICMP_TE_TTL = 0,
    /* 分片重组时间超时 */
    ICMP_TE_FRAG = 1
};
```

可以看出，这些宏定义描述了 ICMP 数据报文的类型字段，下面的 `icmp_dur_type` 和 `icmp_te_type` 枚举用来描述 ICMP 数据报文的代码字段，它们分别为目的不可到达和超时差错报文。

lwIP 的作者为了快速读取和填写 ICMP 报文首部，在 `icmp.h` 文件还定义了 ICMP 报文首部的宏定义，如下源码所示：

```
#define ICMPH_TYPE(hdr) ((hdr)->type)      /* 读取类型字段 */
#define ICMPH_CODE(hdr) ((hdr)->code)      /* 读取代码字段 */
#define ICMPH_TYPE_SET(hdr, t) ((hdr)->type = (t)) /* 填写类型字段 */
#define ICMPH_CODE_SET(hdr, c) ((hdr)->code = (c)) /* 填写代码字段*/
```

使用这些宏定义能快速设置 ICMP 各个字段的数值。

10.2.2 发送 ICMP 差错报文

lwIP 只实现目的不可到达和超时差错报文，它们的实现函数分别为 `icmp_dest_unreach` 和 `icmp_time_exceeded`，这两个函数转入的参数与 `icmp_dur_type` 和 `icmp_te_type` 枚举相关。如目

的不可到达报文的代码字段由 `icmp_dur_type` 枚举描述，而超时报文的代码字段由 `icmp_te_type` 枚举描述。

好了，废话不多说，打开 `icmp.c` 文件查看 `icmp_dest_unreach` 和 `icmp_time_exceeded` 这两个函数，如下所示：

```
/* 发送目标不可达报文，该函数实际调用函数
   icmp_send_response 来发送 ICMP 差错报文
   ICMP_DUR 为目的不可到达 */
void
icmp_dest_unreach(struct pbuf *p, enum icmp_dur_type t)
{
    MIB2_STATS_INC(mib2.icmpoutdestunreachs);
    icmp_send_response(p, ICMP_DUR, t);
}

/* 发送超时报文，该函数实际调用函数
   icmp_send_response 来发送 ICMP 差错报文
   ICMP_TE 为超时 */
void
icmp_time_exceeded(struct pbuf *p, enum icmp_te_type t)
{
    MIB2_STATS_INC(mib2.icmpouttimeexcds);
    icmp_send_response(p, ICMP_TE, t);
}
```

从上述源码可以看出，差错报文的类型已经固定为目的不可到达或者超时，它们唯一不同的是差错报文的代码值，这个代码值就是由 `icmp_dur_type` 和 `icmp_te_type` 枚举定义的，最后调用相同的 `icmp_send_response` 函数发送差错报文，这个发送函数如下所示：

```
static void
icmp_send_response(struct pbuf *p, u8_t type, u8_t code)
{
    struct pbuf *q;
    struct ip_hdr *iphdr;
    struct icmp_echo_hdr *icmphdr;
    ip4_addr_t iphdr_src;
    struct netif *netif;

    MIB2_STATS_INC(mib2.icmpoutmsgs);

    /* 为差错报文申请 pbuf，pbuf 预留以太网首部和 ip 首部，
       申请数据长度为 icmp 首部长度+icmp 数据长度(ip 首部长度+8) */
    q = pbuf_alloc(PBUF_IP, sizeof(struct icmp_echo_hdr) + IP_HLEN +
                  ICMP_DEST_UNREACH_DATASIZE, PBUF_RAM);

    if (q == NULL) {
        MIB2_STATS_INC(mib2.icmpouterrors);
    }
```

```

    return;
}

/* 指向 IP 数据报首部 */
iphdr = (struct ip_hdr *)p->payload;
/* 指向带填写的 icmp 首部 */
icmphdr = (struct icmp_echo_hdr *)q->payload;
/* 填写类型字段 */
icmphdr->type = type;
/* 填写代码字段 */
icmphdr->code = code;
icmphdr->id = 0;
icmphdr->seqno = 0;

/* 从原始数据包中复制字段, IP 数据报首部+8 字节的数据区域 */
SMEMCPY((u8_t *)q->payload + sizeof(struct icmp_echo_hdr), (u8_t *)p->payload,
        IP_HLEN + ICMP_DEST_UNREACH_DATASIZE);
/* 得到源 IP 地址 */
ip4_addr_copy(iphdr_src, iphdr->src);
/* 判断是否同一网段 */
netif = ip4_route(&iphdr_src);
if (netif != NULL) {
    /* 计算校验和 */
    icmphdr->chksum = 0;
    ICMP_STATS_INC(icmp.xmit);
    /* 发送 ICMP 差错报文 */
    ip4_output_if(q, NULL, &iphdr_src, ICMP_TTL, 0, IP_PROTO_ICMP, netif);
}
/* 释放 icmp pbuf */
pbuf_free(q);
}

```

可以看到，此函数申请了一个 pbuf 内存，它的数据区域存储了 ICMP 首部，接着对这个首部各个字段设置数值，然后在 ICMP 首部后面添加引起差错数据包的 IP 首部和引起差错的前 8 字节数据区域，这样 lwIP 内核构建差错报文完成，最后调用 ip4_output_if 函数发送该差错报文。

10.2.3 ICMP 报文处理

IP 层把数据报递交至传输层之前，lwIP 内核会判断 IP 首部的上层协议字段，若这个上层协议字段不为 TCP 和 UDP，则该数据报不会递交给传输层处理；若上层协议字段为 ICMP，则该数据报递交给 icmp_input 函数处理，该函数如下所示：

```

void
icmp_input(struct pbuf *p, struct netif *inp)
{
    u8_t type;

```

```
struct icmp_echo_hdr *iecho;
const struct ip_hdr *iphdr_in;
u16_t hlen;
const ip4_addr_t *src;

ICMP_STATS_INC(icmp.recv);
MIB2_STATS_INC(mib2.icmpinmsgs);

iphdr_in = ip4_current_header();
hlen = IPH_HL_BYTES (iphdr_in);
/* 判断 IP 首部的大小 */
if (hlen < IP_HLEN)
{
    goto lenerr;
}
/* 判断 pbud 的大小 */
if (p->len < sizeof(u16_t) * 2)
{
    goto lenerr;
}
/* 获取 ICMP 的类型字段 */
type = *((u8_t *)p->payload);

switch (type)
{
    case ICMP_ER: /* 回送应答 */
        MIB2_STATS_INC(mib2.icmpinechoreps);
        break;

    case ICMP_ECHO: /* 回送 */
        MIB2_STATS_INC(mib2.icmpinechos);
        src = ip4_current_dest_addr();
        /* 判断是否为多播 */
        if (ip4_addr_ismulticast(ip4_current_dest_addr()))
        {
            goto icmperr;
        }
        /* 判断是否为广播 */
        if (ip4_addr_isbroadcast(ip4_current_dest_addr(),
                                ip_current_netif()))
        {
            goto icmperr;
        }
}
```

```
if (p->tot_len < sizeof(struct icmp_echo_hdr))
{
    goto lenerr;
}

if (pbuf_header(p, (s16_t)(hlen + PBUF_LINK_HLEN +
                          PBUF_LINK_ENCAPSULATION_HLEN)))
{
    struct pbuf *r;
    r = pbuf_alloc(PBUF_LINK, p->tot_len + hlen, PBUF_RAM);

    if (r == NULL)
    {
        goto icmperr;
    }

    if (r->len < hlen + sizeof(struct icmp_echo_hdr))
    {
        pbuf_free(r);
        goto icmperr;
    }

    MEMCPY(r->payload, iphdr_in, hlen);

    if (pbuf_header(r, (s16_t) - hlen))
    {
        pbuf_free(r);
        goto icmperr;
    }

    if (pbuf_copy(r, p) != ERR_OK)
    {
        pbuf_free(r);
        goto icmperr;
    }

    pbuf_free(p);
    p = r;
}
else
{
    if (pbuf_header(p, -(s16_t)(hlen + PBUF_LINK_HLEN +
                                PBUF_LINK_ENCAPSULATION_HLEN)))
```

```

        {
            goto icmperr;
        }
    }

    /* 强制将数据区域转换为 ICMP 报文首部 */
    iecho = (struct icmp_echo_hdr *)p->payload;

    if (pbuf_header(p, (s16_t)hlen))
    {
    }
    else
    {
        err_t ret;
        struct ip_hdr *iphdr = (struct ip_hdr *)p->payload;
        /* 拷贝源 IP 地址 */
        ip4_addr_copy(iphdr->src, *src);
        /* 拷贝目标 IP 地址 */
        ip4_addr_copy(iphdr->dest, *ip4_current_src_addr());
        /* 填写报文类型 */
        ICMPH_TYPE_SET(iecho, ICMP_ER);
        iecho->chksum = 0;

        /* 设置正确的 TTL 并重新计算头校验和。 */
        IPH_TTL_SET(iphdr, ICMP_TTL);
        IPH_CHKSUM_SET(iphdr, 0);
        ICMP_STATS_INC(icmp.xmit);
        MIB2_STATS_INC(mib2.icmpoutmsgs);
        MIB2_STATS_INC(mib2.icmpoutechoreps);

        /* 发送一个应答 ICMP 数据包 */
        ret = ip4_output_if(p, src, LWIP_IP_HDRINCL,
                           ICMP_TTL, 0, IP_PROTO_ICMP, inp);

        if (ret != ERR_OK)
        {
        }
    }

    break;

default: /* 对于其他类型的报文，直接丢掉 */
    if (type == ICMP_DUR)

```

```
{
    MIB2_STATS_INC(mib2.icmpindestunreachs);
}
else if (type == ICMP_TE)
{
    MIB2_STATS_INC(mib2.icmpintimeexcds);
}
else if (type == ICMP_PP)
{
    MIB2_STATS_INC(mib2.icmpinparmprobs);
}
else if (type == ICMP_SQ)
{
    MIB2_STATS_INC(mib2.icmpinsrcquenchs);
}
else if (type == ICMP_RD)
{
    MIB2_STATS_INC(mib2.icmpinredirects);
}
else if (type == ICMP_TS)
{
    MIB2_STATS_INC(mib2.icmpintimestamps);
}
else if (type == ICMP_TSR)
{
    MIB2_STATS_INC(mib2.icmpintimestamppreps);
}
else if (type == ICMP_AM)
{
    MIB2_STATS_INC(mib2.icmpinaddrmask);
}
else if (type == ICMP_AMR)
{
    MIB2_STATS_INC(mib2.icmpinaddrmaskreps);
}

ICMP_STATS_INC(icmp.proterr);
ICMP_STATS_INC(icmp.drop);
}
```

```
pbuf_free(p);
```

```
return;
```

```
lenerr:
```



```
pbuf_free(p);  
ICMP_STATS_INC(icmp.lenerr);  
MIB2_STATS_INC(mib2.icmpinerrors);  
return;  
icmperr:  
pbuf_free(p);  
ICMP_STATS_INC(icmp.err);  
MIB2_STATS_INC(mib2.icmpinerrors);  
return;  
}
```

可以看出，lwIP 接收到回显请求报文时，系统会把这个回显请求报文的 ICMP 类型字段修改为 0（回显应答类型），接着偏移 payload 指针添加 IP 首部并设置 IP 首部的各个字段，最后调用 ip4_output_if 函数发送这个回显应答报文。注：lwIP 只处理回显请求报文，而其他类型的请求报文一律不处理。

第十一章 RAW 编程接口 UDP 实验

本章，我们一起学习传输层 UDP 协议与 RAW API 编程接口，总的来说，本章节的内容会涉及到传输层和应用层的知识。本章分为如下几个部分：

11.1 UDP 协议简介

11.2 RAW 的 UDP 接口简介

11.3 RAW 接口的 UDP 实验

11.1 UDP 协议简介

UDP 协议是 TCP/IP 协议栈的传输层协议，是一个简单的面向数据报的协议，在传输层中还有另一个重要的协议，那就是 TCP 协议，TCP 协议的知识笔者会在下一章节中讲解。UDP 不提供数据包分组、组装，不能对数据包进行排序，当报文发送出去后无法知道是否安全、完整的到达。UDP 除了这些缺点外肯定有它自身的优势，由于 UDP 不属于连接型协议，因而消耗资源小，处理速度快，所以通常在音频、视频和普通数据传输时使用 UDP 较多。UDP 数据报结构如下图所示。



图 11.1.1 UDP 数据报结构

UDP 首部有 8 个字节，由 4 个字段构成，每个字段都是两个字节，这些字段的作用如下：

- ①源端口：源端口号，需要对方回信时选用，不需要时全部置 0。
- ②目的端口：目的端口号，在终点交付报文的时候需要用到。
- ③长度：UDP 的数据报的长度（包括首部和数据）其最小值为 8（只有首部）。
- ④校验和：检测 UDP 数据报在传输中是否有错，有错则丢弃。

UDP 协议使用端口号为不同的应用保留各自的数据传输通道，UDP 和 TCP 协议都是采用端口号对同一时刻内多项应用同时发送和接收数据，而数据接收方则通过目标端口接收数据。有的网络应用只能使用预先为其预留或注册的静态端口；而另外一些网络应用则可以使用未被注册的动态端口。因为 UDP 报头使用两个字节存放端口号，所以端口号的有效范围是从 0 到 65535。一般来说，大于 49151 的端口号都代表动态端口。

数据报的长度是指包括报头和数据部分在内的总字节数。因为报头的长度是固定的，所以该数据区域主要被用来计算可变长度的数据部分（又称为数据负载）。数据报的最大长度根据操作环境的不同而各异。从理论上说，包含报头在内的数据报的最大长度为 65535 字节。

UDP 协议使用报头中的校验和来保证数据的安全。校验和首先在数据发送方通过特殊的算法计算得出，在传递到接收方之后，还需要再重新计算。如果某个数据报在传输过程中被第三方篡改或者由于线路噪音等原因受到损坏，发送和接收方的校验计算和将不会相符，由此 UDP 协议可以检测是否出错。

UDP 报文封装流程

UDP 报文与 TCP 报文一样也是由 UDP/TCP 首部+数据区域组成，UDP 协议是位于传输层，该层是应用层的下一层，当用户发送数据时候，需要选择使用那种协议发送出去，如果使用 UDP 协议，则 UDP 协议就会简单的把数据封装起来，UDP 报文结构如下图所示。

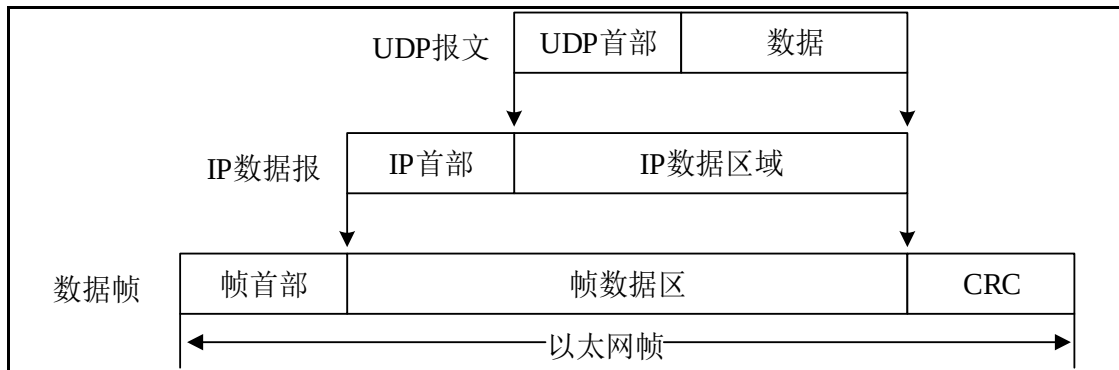


图 11.1.2 UDP 报文封装

11.1.2 UDP 报文的数据结构

(1) UDP 首部结构

从上面可知，UDP 首部包含了四个字段，这些字段在 lwIP 内核中由结构体 `udp_hdr` 描述，该结构体如下所示：

```
struct udp_hdr {
    PACK_STRUCT_FIELD(u16_t src);    /* 源端口 */
    PACK_STRUCT_FIELD(u16_t dest);   /* 目的端口 */
    PACK_STRUCT_FIELD(u16_t len);    /* 长度 */
    PACK_STRUCT_FIELD(u16_t chksum); /* 校验和 */
} PACK_STRUCT_STRUCT;
```

可见，这个结构体的成员变量与图 11.1.1 的 UDP 首部字段一一对应。

(2) UDP 控制块

lwIP 为了更好的管理 UDP 报文，它定义了一个 UDP 控制块，使用该控制块来记录 UDP 的通讯信息，例如源端口、目的端口，源 IP 地址和目的 IP 地址以及收到的数据回调函数等信息，lwIP 把多个 UDP 控制块使用链表形式连接起来，在处理时候遍历列表即可，该 UDP 控制块结构如下所示：

```
#define IP_PCB \
    ip_addr_t local_ip;          /* 本地 ip 地址与远端 IP 地址 */ \
    ip_addr_t remote_ip;        \
    u8_t netif_idx;             /* 绑定 netif 索引 */ \
    u8_t so_options;            /* Socket 选项 */ \
    u8_t tos;                   /* 服务类型 */ \
    u8_t ttl                    /* 生存时间 */ \
    IP_PCB_NETIFHINT /* 链路层地址解析提示 */ \
struct ip_pcb { \
    IP_PCB; \
}; \
struct udp_pcb { \
    IP_PCB;
```

```
struct udp_pcb *next;           /* 指向下一个控制块 */
u8_t flags;                   /* 控制块状态 */
u16_t local_port, remote_port; /* 本地端口和目标端口 */
udp_recv_fn recv;             /* 接收回调函数 */
void *recv_arg;               /* 用户为 recv 回调提供的参数 */
};
```

可以看到，结构体 `udp_pcb` 包含了指向下一个节点的指针 `next`，多个 UDP 控制块构建了一个单向链表且各个控制块指向独立的接收回调函数，如下图所示。

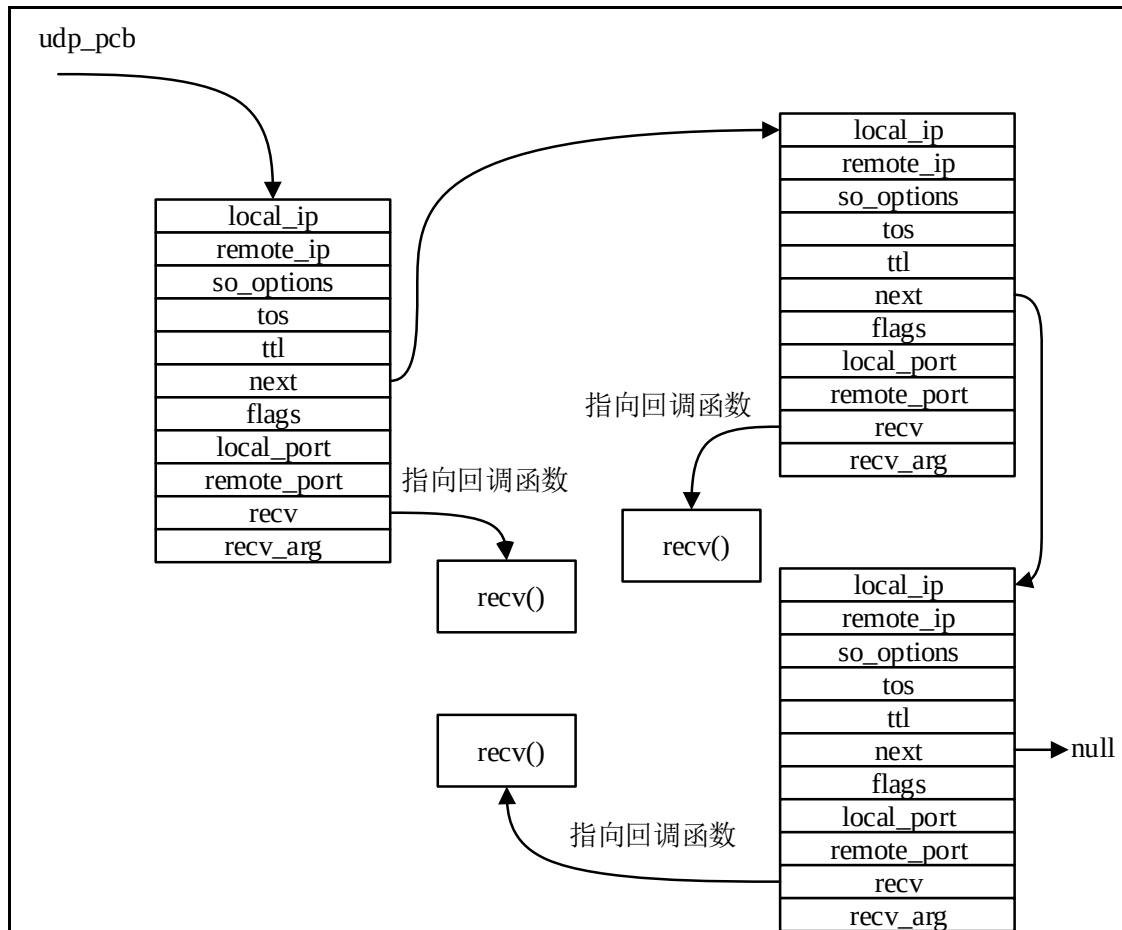


图 11.1.2.1 多个 UDP 控制块的 UDP 控制块的链表

对于 RAW 的 API 接口来讲，上图中的 `recv` 由用户提供这个函数，而 NETCONN 和 SOCKET 接口无需用户提供回调函数，因为 lwIP 内核已经注册了该回调函数，所以数据到来时，该函数把数据以邮箱的方式发送至 NETCONN 和 SOCKET 对应的接口。

11.1.3 发送 UDP 报文

UDP 报文发送函数是由 `udp_sendto_if_src` 实现，其实它最终调用 `ip_output_if_src` 函数把数据报递交给网络层处理，`udp_sendto_if_src` 函数如下所示：

```
err_t
udp_sendto_if_src(struct udp_pcb *pcb,           /* udp 控制块 */
                  struct pbuf *p,                /* pbuf 网络数据包 */
                  const ip_addr_t *dst_ip,       /* 目的 IP 地址 */
```

```

        u16_t dst_port,           /* 目的端口 */
        struct netif *netif,      /* 网卡信息 */
        const ip_addr_t *src_ip)  /* 源 IP 地址 */
{
    struct udp_hdr *udphdr;
    err_t err;
    struct pbuf *q;
    u8_t ip_proto;
    u8_t ttl;

    /* 第一步：判断控制块是否为空和远程 IP 地址是否为空 */
    if (!IP_ADDR_PCB_VERSION_MATCH(pcb, src_ip) ||
        !IP_ADDR_PCB_VERSION_MATCH(pcb, dst_ip))
    {
        return ERR_VAL; /* 放回错误 */
    }

    /* 如果 PCB 还没有绑定到一个端口，那么在这里绑定它 */
    if (pcb->local_port == 0)
    {
        err = udp_bind(pcb, &pcb->local_ip, pcb->local_port);

        if (err != ERR_OK)
        {
            return err;
        }
    }

    /* 判断添加 UDP 首部会不会溢出 */
    if ((u16_t)(p->tot_len + UDP_HLEN) < p->tot_len)
    {
        return ERR_MEM;
    }

    /* 第二步：没有足够的空间将 UDP 首部添加到给定的 pbuf 中 */
    if (pbuf_add_header(p, UDP_HLEN))
    {
        /* 在单独的新 pbuf 中分配标头 */
        q = pbuf_alloc(PBUF_IP, UDP_HLEN, PBUF_RAM);

        /* 在单独的新 pbuf 中分配标头 */
        if (q == NULL)
        {
            return ERR_MEM; /* 返回错误 */
        }
    }
}

```

```

    if (p->tot_len != 0)
    {
        /* 把首部 pbuf 和数据 pbuf 连接到一个 pbuf 链表上 */
        pbuf_chain(q, p);
    }
}

else /* 如果有足够的空间 */
{
    /* 在数据 pbuf 中已经预留 UDP 首部空间 */
    /* q 指向 pbuf */
    q = p;

    /* 第三步：设置 UDP 首部信息 */
    /* 指向它的 UDP 首部 */
    udphdr = (struct udp_hdr *)q->payload;
    /* 填写本地 IP 端口 */
    udphdr->src = lwip_htons(pcb->local_port);
    /* 填写目的端口 */
    udphdr->dest = lwip_htons(dst_port);
    /* 填写校验和 */
    udphdr->chksum = 0x0000;
    /* 设置长度 */
    udphdr->len = lwip_htons(q->tot_len);

    /* 设置协议类型 */
    ip_proto = IP_PROTO_UDP;
    /* 设置生存时间 */
    ttl = pcb->ttl;

    /* 第四步：发送到 IP 层 */
    NETIF_SET_HWADDRHINT(netif, &(pcb->addr_hint));
    err = ip_output_if_src(q, src_ip, dst_ip, ttl, pcb->tos, ip_proto, netif);
    NETIF_SET_HWADDRHINT(netif, NULL);

    MIB2_STATS_INC(mib2.udpoutdatagrams);
    if (q != p)
    {
        /* 释放内存 */
        pbuf_free(q);
        q = NULL;
    }
}

```

```

    UDP_STATS_INC(udp.xmit);
    return err;
}

```

此函数非常简单，首先判断源 IP 地址和目标 IP 地址是否为空，接着判断本地端口是否为空，判断完成之后添加 UDP 首部，最后调用 ip_output_if_src 函数把数据报递交给网络层处理。

11.1.4 UDP 报文接收

网络层处理数据报完成之后，由 udp_input 函数把数据报递交给传输层，该函数源码所示：

```

void
udp_input(struct pbuf *p, struct netif *inp)
{
    struct udp_hdr *udphdr;
    struct udp_pcb *pcb, *prev;
    struct udp_pcb *uncon_pcb;
    u16_t src, dest;
    u8_t broadcast;
    u8_t for_us = 0;

    LWIP_UNUSED_ARG(inp);

    PERF_START;

    UDP_STATS_INC(udp.recv);

    /* 第一步：判断数据报长度少于 UDP 首部 */
    if (p->len < UDP_HLEN)
    {
        UDP_STATS_INC(udp.lenerr);
        UDP_STATS_INC(udp.drop);
        MIB2_STATS_INC(mib2.udpinerrors);
        pbuf_free(p); /* 释放内存，掉弃该数据报 */
        goto end;
    }

    /* 指向 UDP 首部 */
    udphdr = (struct udp_hdr *)p->payload;

    /* 判断是否是广播包 */
    broadcast = ip_addr_isbroadcast(ip_current_dest_addr(), ip_current_netif());

    /* 得到源端口号 */
    src = lwip_ntohs(udphdr->src);

    /* 得到目的端口号 */

```

```

dest = lwip_ntohs(udphdr->dest);

udp_debug_print(udphdr);

pcb = NULL;
prev = NULL;
uncon_pcb = NULL;

/* 第二步：遍历 UDP pcb 列表以找到匹配的 pcb */
for (pcb = udp_pcb; pcb != NULL; pcb = pcb->next)
{

    /* 第三步：比较 PCB 本地 IP 地址与端口 */
    if ((pcb->local_port == dest) &&
        (udp_input_local_match(pcb, inp, broadcast) != 0))
    {
        /* 判断 UDP 控制块的状态 */
        if (((pcb->flags & UDP_FLAGS_CONNECTED) == 0) &&
            ((uncon_pcb == NULL)))
        {
            /* 如果未找到使用第一个 UDP 控制块 */
            uncon_pcb = pcb;
        }
        /* 判断目的 IP 是否为广播地址 */
        else if (broadcast &&
            ip4_current_dest_addr()->addr == IPADDR_BROADCAST)
        {
            /* 全局广播地址 (仅对 IPv4 有效;之前检查过匹配) */
            if (!IP_IS_V4_VAL(uncon_pcb->local_ip)
                || !ip4_addr_cmp(ip_2_ip4(&uncon_pcb->local_ip),
                    netif_ip4_addr(inp)))
            {
                /* 检查此 pcb，uncon_pcb 与输入 netif 不匹配 */
                if (IP_IS_V4_VAL(pcb->local_ip) &&
                    ip4_addr_cmp(ip_2_ip4(&pcb->local_ip),
                        netif_ip4_addr(inp)))
                {
                    /* 更好的匹配 */
                    uncon_pcb = pcb;
                }
            }
        }
    }
}

```



```

/* 比较 PCB 远程地址+端口和 UDP 源地址+端口 */
if ((pcb->remote_port == src) &&
    (ip_addr_isany_val(pcb->remote_ip) ||
     ip_addr_cmp(&pcb->remote_ip, ip_current_src_addr())))
{
    /* 第一个完全匹配的 PCB */
    if (prev != NULL)
    {
        /* 将 pcb 移到 udp_pcbs 前面 */
        prev->next = pcb->next;
        pcb->next = udp_pcbs;
        udp_pcbs = pcb;
    }
    else
    {
        UDP_STATS_INC(udp.cachehit);
    }

    break;
}

prev = pcb;
}

/* 第五步：找不到完全匹配的 UDP 控制块
   将第一个未使用的 UDP 控制块作为匹配结果 */
if (pcb == NULL)
{
    pcb = uncon_pcb;
}

/* 检查校验和是否匹配或是否匹配 */
if (pcb != NULL)
{
    for_us = 1;
}
else
{
    #if LWIP_IPV4

    if (!ip_current_is_v6())
    {

```

```

        for_us = ip4_addr_cmp(netif_ip4_addr(inp), ip4_current_dest_addr());
    }

#endif /* LWIP_IPV4 */
}

/* 第六步：如果匹配 */
if (for_us)
{
    /* 调整报文的数据区域指针 */
    if (pbuf_header(p, -UDP_HLEN) )
    {
        UDP_STATS_INC(udp.drop);
        MIB2_STATS_INC(mib2.udpinerrors);
        pbuf_free(p);
        goto end;
    }

    /* 如果找到对应的控制块 */
    if (pcb != NULL)
    {
        MIB2_STATS_INC(mib2.udpinetagrams);
        /* 回调函数，将数据递交给上层应用 */
        if (pcb->recv != NULL)
        {
            /* 回调函数 recv 需要负责释放 p */
            pcb->recv(pcb->recv_arg, pcb, p, ip_current_src_addr(), src);
        }
        else
        {
            /* 如果 recv 函数没有注册，直接释放 p */
            pbuf_free(p);
            goto end;
        }
    }

    else /* 第七步：没有找到匹配的控制块，返回端口不可达 ICMP 报文 */
    {
        if (!broadcast && !ip_addr_ismulticast(ip_current_dest_addr()))
        {
            /* 将数据区域指针移回 IP 数据报首部 */
            pbuf_header_force(p, (s16_t)(ip_current_header_tot_len() +
                                         UDP_HLEN));

            /* 返回一个端口不可达 ICMP 差错控制报文到源主机中 */
            icmp_port_unreach(ip_current_is_v6(), p);
        }
    }
}

```

```

        UDP_STATS_INC(udp.proterr);
        UDP_STATS_INC(udp.drop);
        MIB2_STATS_INC(mib2.udpnoports);
        pbuf_free(p); /* 掉弃该数据包 */
    }
}

/* 如果不匹配，则掉弃该数据包 */
else
{
    pbuf_free(p);
}
end:
    PERF_STOP("udp_input");
    return;
}

```

可以看出，此函数根据接收数据包的 UDP 首部信息遍历 UDP 控制块链表，找到对应的控制块之后 lwIP 内核把接收到的数据包递交给 pcb->recv 回调函数处理。

11.2 RAW 的 UDP 接口简介

下表给出了 UDP 协议的 RAW 的 API 功能函数，我们使用这些函数来完成 UDP 的数据发送和接收功能。

RAW 的 API 函数	描述
udp_new	新建一个 UDP 的 PCB 块
udp_remove	将一个 PCB 控制块从链表中删除，并且释放这个控制块的内存
udp_bind	为 UDP 的 PCB 控制块绑定一个本地 IP 地址和端口号
udp_connect	连接到指定 IP 地址主机的指定端口上
udp_disconnect	断开连接，将控制块设置为非连接状态
udp_send	通过一个 PCB 控制块发送数据
udp_recv	创建的一个回调函数

表 11.2.1 UDP 的 RAW API 功能函数

表 11.2.1 只是列出了我们在编程时需要使用到的函数，接下来我们来看一下上述表中重要的函数：

(1) udp_new 函数

此函数用来创建一个 UDP 控制块，这个控制块用来描述 IP 地址、端口号和状态等信息，该函数实现源码如下所示：

```

struct udp_pcb *
udp_new(void)
{
    struct udp_pcb *pcb;
    /* 申请一个 UDP 内存池 */
    pcb = (struct udp_pcb *)memp_malloc(MEMP_UDP_PCB);

    /* 申请成功*/
}

```

```

if (pcb != NULL)
{
    /* 初始化 PCB 控制块所有零 */
    memset(pcb, 0, sizeof(struct udp_pcb));
    /* pcb->ttd = 255 */
    pcb->ttd = UDP_TTL;
}

return pcb;
}

```

可以看到，该控制块的内存由内存池申请，申请成功之后设置该控制块的生存时间。

(2) udp_remove 函数

从 PCB 控制块链表中移除一个控制块，并且把移除的控制块释放内存，该函数实现源码如下所示：

```

void
udp_remove(struct udp_pcb *pcb)
{
    struct udp_pcb *pcb2;
    mib2_udp_unbind(pcb);

    /* 判断 pcb 被删除在列表的第一个 */
    if (udp_pcbs == pcb)
    {
        /* 从第二 pcb 开始制作列表 */
        udp_pcbs = udp_pcbs->next;
    }
    else /* pcb 不在列表的第一个 */
    {
        /* 遍历 pcb 列表 */
        for (pcb2 = udp_pcbs; pcb2 != NULL; pcb2 = pcb2->next)
        {
            /* 在 udp_pcbs 列表中查找 pcb */
            if (pcb2->next != NULL && pcb2->next == pcb)
            {
                /* 从列表中删除 pcb */
                pcb2->next = pcb->next;
                break;
            }
        }
    }
    memp_free(MEMP_UDP_PCB, pcb);
}

```

以传入的控制块为条件，遍历 PCB 控制块链表，若链表中的控制块等于要移除的控制块，

则该控制块移除 PCB 控制块链表，移除完成之后释放该控制块的内存。

(3) udp_recv 函数

此函数用来设置接收回调函数及函数参数，若用户使用 RAW 接口实现 UDP，则用户必须调用此函数设置接收回调函数，该函数的源码如下所示：

```
void
udp_recv(struct udp_pcb *pcb, udp_recv_fn recv, void *recv_arg)
{
    /* 调用 recv() 回调和用户数据 */
    pcb->recv = recv;
    pcb->recv_arg = recv_arg;
}
```

可以看出，设置的函数和形参都是由 UDP 控制块的字段指向。

11.3 RAW 的 UDP 实验

11.3.1 硬件设计

1. 例程功能

本章实验的目标是 PC 端和开发板通过 UDP 协议连接起来，PC 端使用网络调试助手向开发板发送数据，开发板接收到以后在 LCD 上显示接收到的数据，我们也可以通过开发板上的按键发送数据给 PC。

该实验的实验工程，请参考《lwIP 例程 2 lwIP_RAW_UDP 实验》。

11.3.2 软件设计

11.3.2.1 UDP 配置步骤

1) 创建 UDP 控制块

调用函数 `udp_new` 创建 UDP 控制块。

2) 连接指定的 IP 地址和端口号

调用函数 `udp_connect` 绑定远程 IP 地址和远程端口号。

3) 绑定本地 IP 地址与端口号

调用函数 `udp_bind` 绑定本地 IP 地址和本地端口号。

4) 注册接收回调函数

`udp_recv` 是注册接收回调函数，该函数需要自己编写。

5) 发送数据

调用函数 `udp_send` 发送数据。

11.3.2.2 程序流程图

本实验的程序流程图，如下图所示。

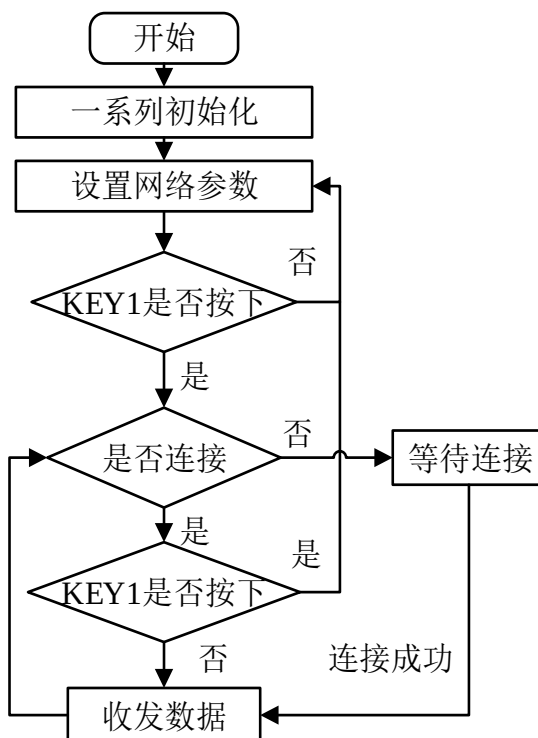


图 11.4.2.1 RAW 编程 UDP 实验流程图

11.3.2.3 程序解析

本实验重点讲解 lwip_demo.c 这个文件，该文件定义了五个函数，它们的作用如下表所示：

函数	描述
lwip_demo_set_remoteip()	设置远端 IP 地址，UDP 连接的时候要要用到这个地址
lwip_demo()	UDP 实验测试函数，本实验的主要内容就在这个函数里面
lwip_demo_recv()	UDP 接收数据的回调函数，回调函数是要我们自己编写的
lwip_demo_senddata()	发送数据，将我们要发送的数据发送出去
lwip_demo_connection_close()	关闭 UDP 连接

表 11.4.3.1 UDP 实验函数

我们首先看一下 lwip_demo 函数，该函数的代码如下。

```

/**
 * @brief      lwip_demo 实验入口
 * @param      无
 * @retval     无
 */
void lwip_demo(void)
{
    err_t err;
    struct udp_pcb *udppcb; /* 定义一个 TCP 服务器控制块 */
    ip_addr_t rmtipaddr;    /* 远端 ip 地址 */

    char *tbuf;
    uint8_t key;
    uint8_t res = 0;

```

```
uint8_t t = 0;

lwip_demo_set_remoteip(); /* 先选择 IP */
lcd_clear(BLACK);        /* 清屏 */
g_point_color = WHITE;
lcd_show_string(30, 30, 200, 16, 16, "STM32", g_point_color);
lcd_show_string(30, 50, 200, 16, 16, "UDP Test", g_point_color);
lcd_show_string(30, 70, 200, 16, 16, "ATOM@正点原子", g_point_color);
lcd_show_string(30, 90, 200, 16, 16, "KEY0:Send data", g_point_color);
tbuf = mymalloc(SRAMIN, 200); /* 申请内存 */

if (tbuf == NULL) return ;    /* 内存申请失败了,直接退出 */

sprintf((char *)tbuf, "Local IP:%d.%d.%d.%d", lwipdev.ip[0],
                                              lwipdev.ip[1],
                                              lwipdev.ip[2],
                                              lwipdev.ip[3]); /* 服务器 IP */
lcd_show_string(30, 130, 210, 16, 16, tbuf, g_point_color);
/* 远端 IP */
sprintf((char *)tbuf, "Remote IP:%d.%d.%d.%d", lwipdev.remoteip[0],
                                              lwipdev.remoteip[1],
                                              lwipdev.remoteip[2],
                                              lwipdev.remoteip[3]);
lcd_show_string(30, 150, 210, 16, 16, tbuf, g_point_color);
sprintf((char *)tbuf, "Remote Port:%d", UDP_DEMO_PORT); /* 客户端端口号 */
lcd_show_string(30, 170, 210, 16, 16, tbuf, g_point_color);
g_point_color = BLUE;
lcd_show_string(30, 190, 210, 16, 16, "STATUS:Disconnected", g_point_color);
udppcb = udp_new();

if (udppcb) /* 创建成功 */
{
    IP4_ADDR(&rmtipaddr, lwipdev.remoteip[0], lwipdev.remoteip[1],
            lwipdev.remoteip[2], lwipdev.remoteip[3]);
    /* UDP 客户端连接到指定 IP 地址和端口号的服务器 */
    err = udp_connect(udppcb, &rmtipaddr, UDP_DEMO_PORT);

    if (err == ERR_OK)
    {
        /* 绑定本地 IP 地址与端口号 */
        err = udp_bind(udppcb, IP_ADDR_ANY, UDP_DEMO_PORT);

        if (err == ERR_OK) /* 绑定完成 */

```

```

        {
            udp_recv(udppcb, lwip_demo_callback, NULL); /* 注册接收回调函数 */
            /* 标记连接上了 (UDP 是非可靠连接, 这里仅仅表示本地 UDP 已经准备好) */
            lcd_show_string(30, 190, 210, 16, 16, "STATUS:Connected  ",
                           g_point_color);
            g_point_color = WHITE;
            lcd_show_string(30, 210, lcddev.width - 30,
                           lcddev.height - 190, 16,
                           "Receive Data:", g_point_color); /* 提示消息 */
            g_point_color = BLUE;
        }
        else res = 1;
    }
    else res = 1;
}
else res = 1;

while (res == 0)
{
    key = key_scan(0);

    if (key == KEY1_PRES) break;

    if (key == KEY0_PRES) /* KEY0 按下了, 发送数据 */
    {
        lwip_demo_senddata(udppcb);
    }

    if (lwip_send_flag & 1 << 6) /* 是否收到数据 */
    {
        /* 清上一次数据 */
        lcd_fill(30, 230, lcddev.width - 1, lcddev.height - 1, BLACK);
        /* 显示接收到的数据 */
        lcd_show_string(30, 230, lcddev.width - 30, lcddev.height - 230, 16,
                        (char *)udp_demo_recvbuf, g_point_color);
        lwip_demo_flag &= ~(1 << 6); /* 标记数据已经被处理了 */
    }

    lwip_periodic_handle();
    delay_ms(2);
    t++;

    if (t == 200)

```



```

    {
        t = 0;
        LED0_TOGGLE();
    }
}

lwip_demo_connection_close(udppcb);
myfree(SRAMIN, tbuf);
}

```

此函数非常简单，它首先设置 IP 地址等信息，接着调用 RAW 相关 API 函数配置 UDP 连接，值得注意的是，UDP 的接收函数由用户编写，并且调用 `udp_recv` 函数注册到 UDP 控制块当中。

设置远程 IP 地址的函数为 `lwip_udp_set_remoteip`，如下源码所示：

```

/**
 * @brief      设置远端 IP 地址
 * @param      无
 * @retval     无
 */
void lwip_udp_set_remoteip(void)
{
    char *tbuf;
    uint16_t xoff;
    uint8_t key;
    lcd_clear(BLACK);
    g_point_color = WHITE;
    lcd_show_string(30, 30, 200, 16, 16, "STM32", g_point_color);
    lcd_show_string(30, 50, 200, 16, 16, "UDP Test", g_point_color);
    lcd_show_string(30, 70, 200, 16, 16, "Remote IP Set", g_point_color);
    lcd_show_string(30, 90, 200, 16, 16, "KEY0:+ KEY2:-", g_point_color);
    lcd_show_string(30, 110, 200, 16, 16, "KEY1:OK", g_point_color);
    tbuf = mymalloc(SRAMIN, 100); /* 申请内存 */

    if (tbuf == NULL) return;

    /* 前三个 IP 保持和 DHCP 得到的 IP 一致 */
    lwipdev.remoteip[0] = lwipdev.ip[0];
    lwipdev.remoteip[1] = lwipdev.ip[1];
    lwipdev.remoteip[2] = lwipdev.ip[2];

    /* 远端 IP */
    sprintf((char *)tbuf, "Remote IP:%d.%d.%d.", lwipdev.remoteip[0],
                                   lwipdev.remoteip[1], lwipdev.remoteip[2]);
    lcd_show_string(30, 150, 210, 16, 16, tbuf, g_point_color);
    g_point_color = BLUE;
}

```

```

xoff = strlen((char *)tbuf) * 8 + 30;
lcd_show_xnum(xoff, 150, lwipdev.remoteip[3], 3, 16, 0, g_point_color);

while (1)
{
    key = key_scan(0);

    if (key == KEY1_PRES)break;
    else if (key)
    {
        if (key == KEY0_PRES)lwipdev.remoteip[3]++; /* IP 增加 */

        if (key == KEY2_PRES)lwipdev.remoteip[3]--; /* IP 减少 */
        /* 显示新 IP */
        lcd_show_xnum(xoff, 150, lwipdev.remoteip[3], 3, 16, 0x80,
                      g_point_color);
    }
}

myfree(SRAMIN, tbuf);
}

```

可见，此函数根据开发板上的 KEY0 和 KEY2 按键来设置远程 IP 地址，设置好 IP 地址之后按下 KEY1 退出设置界面。

函数 `udp_recv` 为注册接收回调函数 `lwip_udp_callback`，如下源码所示：

```

/**
 * @brief      UDP 服务器回调函数
 * @param      arg : 传入参数
 * @param      upcb: UDP 控制块
 * @param      p   : 网络数据包
 * @param      addr: IP 地址
 * @param      port: 端口号
 * @retval     无
 */
static void lwip_udp_callback(void *arg, struct udp_pcb *upcb, struct pbuf *p,
                              const ip_addr_t *addr, u16_t port)
{
    uint32_t data_len = 0;
    struct pbuf *q;

    if (p != NULL) /* 接收到不为空的数据时 */
    {
        memset(udp_demo_recvbuf, 0, UDP_DEMO_RX_BUFSIZE); /* 数据接收缓冲区清零 */
    }
}

```

```

for (q = p; q != NULL; q = q->next) /* 遍历完整个 pbuf 链表 */
{
    /* 判断要拷贝到 UDP_DEMO_RX_BUFSIZE 中的数据是否大于
       UDP_DEMO_RX_BUFSIZE 的剩余空间, 如果大于 */
    /* 的话就只拷贝 UDP_DEMO_RX_BUFSIZE 中剩余长度的数据,
       否则的话就拷贝所有的数据 */
    /* 拷贝数据 */
    if (q->len > (UDP_DEMO_RX_BUFSIZE - data_len))
        memcpy(udp_demo_recvbuf + data_len, q->payload,
                (UDP_DEMO_RX_BUFSIZE - data_len));
    else memcpy(udp_demo_recvbuf + data_len, q->payload, q->len);

    data_len += q->len;
    /* 超出 TCP 客户端接收数组, 跳出 */
    if (data_len > UDP_DEMO_RX_BUFSIZE) break;
}

upcb->remote_ip = *addr; /* 记录远程主机的 IP 地址 */
upcb->remote_port = port; /* 记录远程主机的端口号 */
lwipdev.remoteip[0] = upcb->remote_ip.addr & 0xff; /* IADDR4 */
lwipdev.remoteip[1] = (upcb->remote_ip.addr >> 8) & 0xff; /* IADDR3 */
lwipdev.remoteip[2] = (upcb->remote_ip.addr >> 16) & 0xff; /* IADDR2 */
lwipdev.remoteip[3] = (upcb->remote_ip.addr >> 24) & 0xff; /* IADDR1 */
udp_demo_flag |= 1 << 6; /* 标记接收到数据了 */
pbuf_free(p); /* 释放内存 */
}
else
{
    udp_disconnect(upcb);
    lcd_clear(BLACK); /* 清屏 */
    lcd_show_string(30, 30, 200, 16, 16, "STM32", g_point_color);
    lcd_show_string(30, 50, 200, 16, 16, "UDP Test", g_point_color);
    lcd_show_string(30, 70, 200, 16, 16, "ATOM@正点原子", g_point_color);
    lcd_show_string(30, 90, 200, 16, 16, "KEY1:Connect", g_point_color);
    lcd_show_string(30, 190, 210, 16, 16, "Connect break!", g_point_color);
}
}

```

之前笔者讲解过, 使用 RAW 接口实现远程通讯的话, 它的接收函数由用户编写且调用 `udp_recv` 函数让控制块的 `recv` 函数指针指向, 因为 lwIP 内核接收到的数据会递交给这个接收函数, 所以不注册该函数就无法接收到数据。

`lwip_udp_senddata` 函数是用来发送数据的, 在发送数据前我们先通过 `pbuf_alloc` 函数申请内存, 当内存申请成功以后我们将发送缓冲区 `lwip_demo_sendbuf` 的首地址填入到 `ptr` 的 `payload` 字段, 然后调用 `udp_send` 函数将数据发送出去, 最后释放申请到的内存, 代码如下。

```
/**
 * @brief      UDP 服务器发送数据
 * @param      upcb: UDP 控制块
 * @retval     无
 */
void lwip_udp_senddata(struct udp_pcb *upcb)
{
    struct pbuf *ptr;
    /* 申请内存 */
    ptr = pbuf_alloc(PBUF_TRANSPORT, strlen((char *)udp_demo_sendbuf),
                     PBUF_POOL);

    if (ptr)
    {
        pbuf_take(ptr, (char *)udp_demo_sendbuf, strlen((char *)udp_demo_sendbuf));
        /* 将 tcp_demo_sendbuf 中的数据打包进 pbuf 结构中 */
        udp_send(upcb, ptr); /* udp 发送数据 */
        pbuf_free(ptr); /* 释放内存 */
    }
}
```

lwip_demo_connection_close 函数是用来关闭 UDP 连接的，这个函数很简单，通过调用函数 udp_disconnect 来关闭连接，然后调用 udp_remove 函数将当前被关闭的连接控制块从当前连接控制块链表中删除，代码如下。

```
/**
 * @brief      关闭 tcp 连接
 * @param      upcb: UDP 控制块
 * @retval     无
 */
void lwip_demo_connection_close(struct udp_pcb *upcb)
{
    udp_disconnect(upcb);
    udp_remove(upcb); /* 断开 UDP 连接 */
    udp_demo_flag &= ~(1 << 5); /* 标记连接断开 */
    lcd_clear(BLACK); /* 清屏 */
    lcd_show_string(30, 30, 200, 16, 16, "STM32", g_point_color);
    lcd_show_string(30, 50, 200, 16, 16, "UDP Test", g_point_color);
    lcd_show_string(30, 70, 200, 16, 16, "ATOM@正点原子", g_point_color);
    lcd_show_string(30, 90, 200, 16, 16, "KEY1:Connect", g_point_color);
    lcd_show_string(30, 190, 210, 16, 16, "STATUS:Disconnected", g_point_color);
}
```

main 函数一开始完成外设的初始化，如果开启 DHCP 的话通过 DHCP 获取 IP 地址，IP 地址获取成功以后就调用 udp_demo_test 函数进入 UDP 实验。我们知道在 lwip_demo 函数中有一

个 while() 循环，当从这个循环退出来以后就会进入 main 函数的 while() 循环中，在 main 函数的 while() 循环中当 KEY1 按下并且 UDP 连接已经断开就调用 lwip_demo 函数重新开始 UDP 实验，如下源码所示：

```
key = key_scan(0);
if (key == KEY1_PRES)
{
    if((lwip_send_flag & 1<<5))
    {
        printf("UDP 连接已经建立,不能重复连接\r\n"); /* 如果连接成功,不做任何处理 */
    }
    else
    {
        lwip_demo_test(); /* 当断开连接后,调用 udp_demo_test() 函数 */
    }
}
```

11.3.3 下载验证

下载完代码后，打开网络调试助手，等待开发板的 LCD 出现如图 11.4.1 所示界面，在这个界面上我们通过按键 KEY2 和 KEY0 设置远端 IP 地址，也就是电脑的 IP 地址，设置好以后按 KEY_UP 确认，确认完了以后 LCD 就如图 11.4.2 所示的数据接收界面。

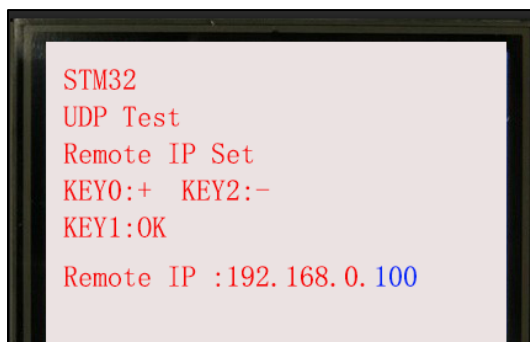


图 11.3.3.1 远端 IP 地址设置

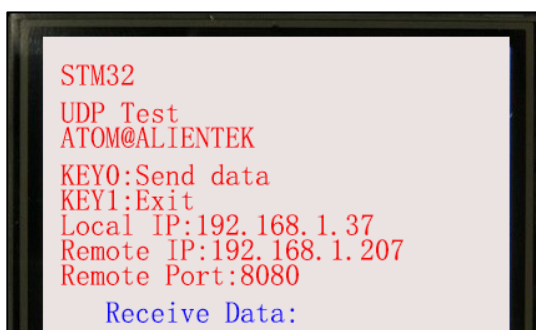


图 11.3.3.2 数据接收界面

接下来设置电脑端的网络调试助手，设置完成后点击网络调试助手的“连接”，操作完后的网络调试助手如下图所示，



图 11.3.3.3 网络调试助手设置

设置完网络调试助手后在发送填入要发送的数据，这里输入要发送的数据：ALIENTEK DATA，然后点击发送，这时我们开发板的 LCD 上显示，我们可以看到在 LCD 上显示出了电脑端发送过来的数据，我们通过按下 KEY0，向电脑端发送数据“ALIENTEK DATA”图 11.4.4 所示，表明网络调试助手接收到开发板发送的数据，这里我们按了 12 次 KEY0，因此在网络调试助手上有 12 行数据。

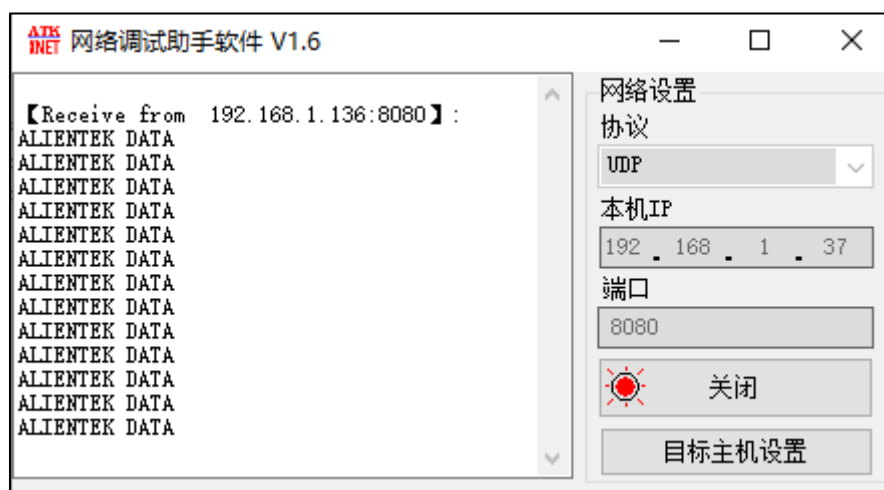


图 11.3.3.4 UDP 测试

第十二章 RAW 编程接口 TCP 客户端实验

本章，我们学习传输层的另一个协议，它是 TCP 协议，TCP 协议对于 UDP 协议来说，可能有点晦涩难懂，读者可以参考相关网络书籍，来学习 TCP 协议。本章分为如下几个部分：

12.1 TCP 协议

12.2 RAW 接口的 TCP 实验

12.1 TCP 协议

12.1.1 TCP 协议简介

TCP (Transmission Control Protocol 传输控制协议) 是一种面向连接的、可靠的、基于字节的传输层通信协议。

TCP 为了保证数据包传输的可靠性，会给每个包一个序号，同时此序号也保证了发送到接收端主机能够按序接收。然后接收端主机对成功接收到的数据包发回一个相应的确认字符 (ACK, Acknowledgement)，如果发送端主机在合理的往返时延 (RTT) 内未收到确认字符 ACK，那么对应的数据包就被认为丢失并将被重传。TCP 协议，它是基于连接的一种传输层协议，在发送数据之前要求系统需要在不可靠的信道上建立可靠连接，我们称之为“三次握手”。建立连接完成之后客户端与服务器才能互发数据，不需要发送数据时，可以断开连接，这里我们称之为“四次挥手”。下面笔者带大家了解一下 TCP 协议建立连接的过程和断开连接的过程，即三次握手和四次挥手的过程。

12.1.2 TCP 的建立连接

握手之前主动打开连接的客户端结束 CLOSED 阶段，被动打开的服务器端也结束 CLOSED 阶段，并进入 LISTEN 阶段。随后开始“三次握手”：

① TCP 服务器进程先创建传输控制块 TCB，时刻准备接受客户进程的连接请求，此时服务器就进入了 LISTEN (监听) 状态。

② TCP 客户进程也是先创建传输控制块 TCB，然后向服务器发出连接请求报文，这是报文首部中的同部位 SYN=1，同时选择一个初始序列号 seq=x，此时，TCP 客户端进程进入了 SYN-SENT (同步已发送状态) 状态。TCP 规定，SYN 报文段 (SYN=1 的报文段) 不能携带数据，但需要消耗掉一个序号。

③ TCP 服务器收到请求报文后，如果同意连接，则发出确认报文。确认报文中应该 ACK=1，SYN=1，确认号是 ack=x+1，同时也要为自己初始化一个序列号 seq=y，此时，TCP 服务器进程进入了 SYN-RCVD (同步收到) 状态。这个报文也不能携带数据，但是同样要消耗一个序号。

④ TCP 客户进程收到确认后，还要向服务器给出确认。确认报文的 ACK=1，ack=y+1，自己的序列号 seq=x+1，此时，TCP 连接建立，客户端进入 ESTABLISHED (已建立连接) 状态。TCP 规定，ACK 报文段可以携带数据，但是如果不携带数据则不消耗序号。

当服务器收到客户端的确认后也进入 ESTABLISHED 状态，此后双方就可以开始通信了。这就是“三次握手”的过程，如下图所示。

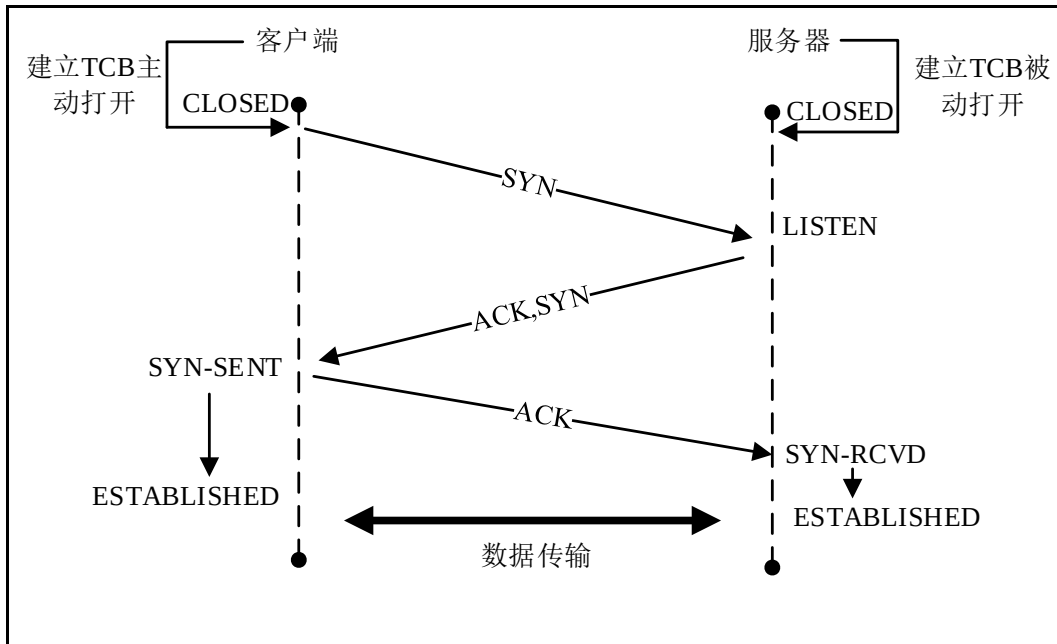


图 12.1.2.1 TCP 连接（三次握手）

12.1.3 TCP 终止连接

建立一个连接需要三次握手而终止一个连接需要四次挥手，终止连接有以下过程。

(1) 第一次挥手：客户端发送释放报文，并停止发送数据。释放数据报文首部，FIN=1,其序列号为 seq=u,此时，客户端进入 FIN-WAIT1（等待服务器应答 FIN 报文）。

(2) 第二次挥手：服务器收到客户端的 FIN 报文后，发出确认报文 ACK=1、ack=u+1，并携带自己的序列号 seq=v。此时，服务器进入 CLOSE-WAIT（关闭等待）状态。客户端收到服务端确认请求，此时，客户端进入 FIN-WAIT2（终止等待 2）状态，等待服务器发送连接释放报文。

(3) 第三次挥手：服务器向客户端发送连接释放报文 FIN=1、ack=u+1,此时，服务器进入了 LAST-ACK（最后确认）等待客户端的确认。客户端接收到服务器的连接释放报文后，必须发送确认 ack=1、ack=w+1,客户端的序列号为 seq=u+1，此时，客户端进入 TIME-WAIT（时间等待）。

(4) 第四次挥手：服务器接收到客户端的确认报文，立刻进入 CLOSED 状态。

这四次挥手就是终止 TCP 协议连接，如下图所示。

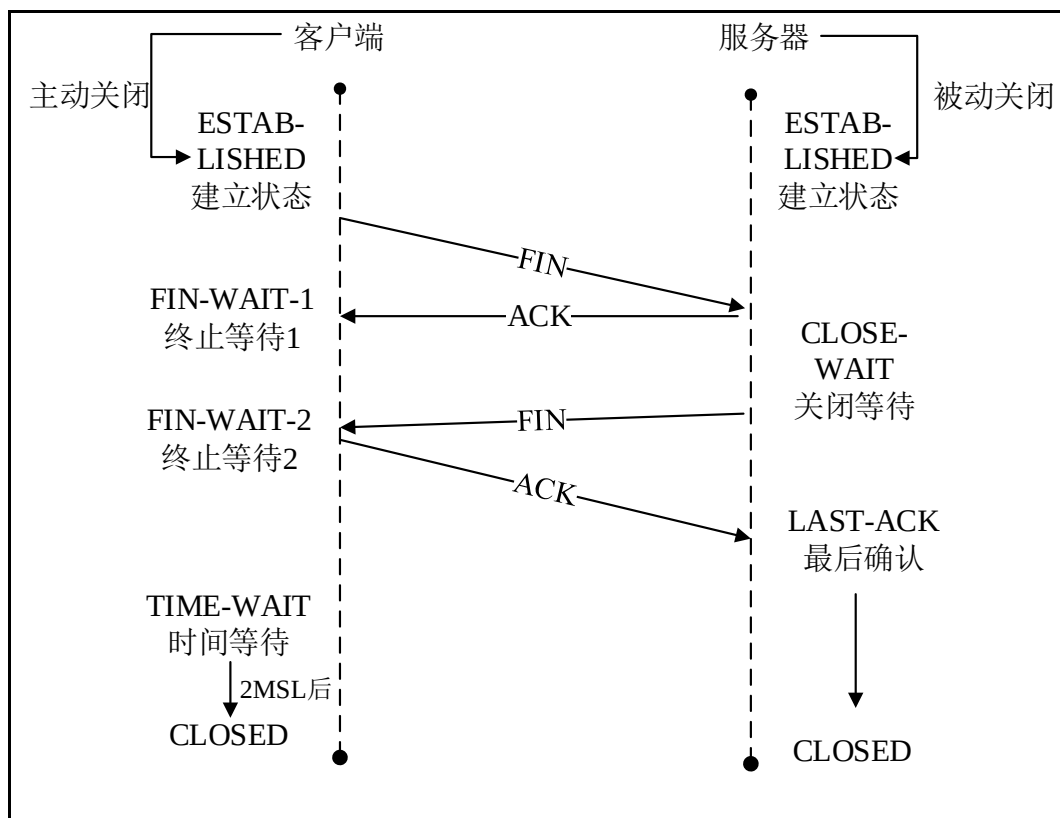


图 12.1.2.2 TCP 协议四次挥手终止连接

上图的终止连接由客户端发起，当然服务器也可以发起终止连接。

12.1.4 TCP 报文结构

在传输层中，TCP 的数据包称为数据段，TCP 报文段与 UDP 报文段一样都是封装在 IP 数据报中发送。TCP 首部包含建立与断开、数据确认、窗口大小通告、数据发送相关的所有标志和控制信息，TCP 报文结构如下图所示。

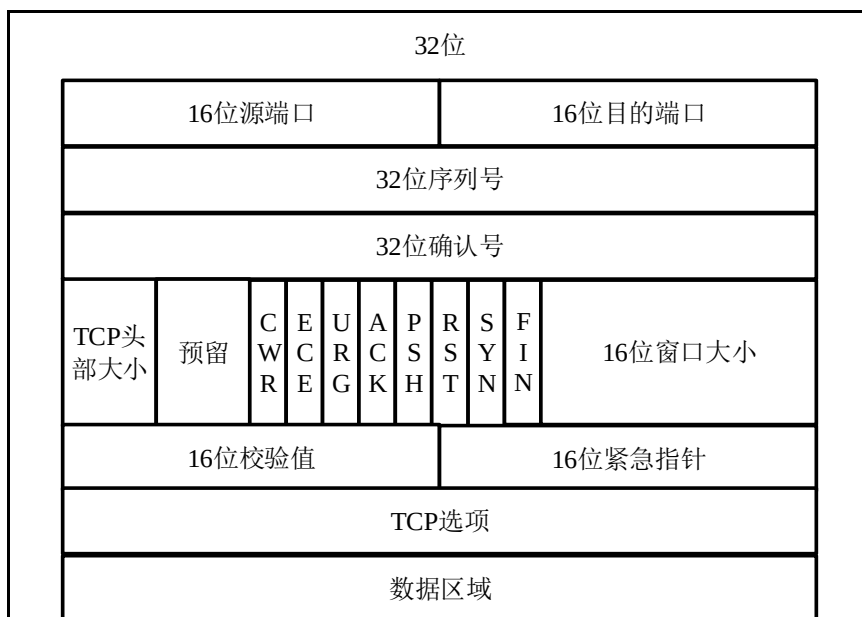


图 12.1.4.1 TCP 报文结构

(1) 源、目标端口号字段：占 16 比特。TCP 协议通过使用”端口”来标识源端和目标端的应用进程。端口号可以使用 0 到 65535 之间的任何数字。在收到服务请求时，操作系统动态地为客户端的应用程序分配端口号。在服务器端，每种服务在”众所周知的端口”(Well-Know Port) 为用户提供服务。

(2) 序列号字段：占 32 比特。用来标识从 TCP 源端向 TCP 目标端发送的数据字节流，它表示在这个报文段中的第一个数据字节。

(3) 确认号字段：占 32 比特。只有 ACK 标志为 1 时，确认号字段才有效。它包含目标端所期望收到源端的下一个数据字节。

(4) 头部长度的字段：占 4 比特。给出头部占 32 比特的数目。没有任何选项字段的 TCP 头长度为 20 字节；最多可以有 60 字节的 TCP 头部。

(5) 标志位字段 (U、A、P、R、S、F)：占 6 比特。各比特的含义如下：

① URG：紧急指针有效。

② ACK：为 1 时，确认序号有效。

③ PSH：为 1 时，接收方应该尽快将这个报文段交给应用层。

④ RST：为 1 时，重建连接。

⑤ SYN：为 1 时，同步程序，发起一个连接。

⑥ FIN：为 1 时，发送端完成任务，释放一个连接。

(6) 窗口大小字段：占 16 比特。此字段用来进行流量控制。单位为字节数，这个值是本机期望一次接收的字节数。

(7) TCP 校验和字段：占 16 比特。对整个 TCP 报文段，即 TCP 头部和 TCP 数据进行校验和计算，并由目标端进行验证。

(8) 紧急指针字段：占 16 比特。它是一个偏移量，和序号字段中的值相加表示紧急数据最后一个字节的序号。

(9) 选项字段：占 32 比特。可能包括”窗口扩大因子”、”时间戳”等选项。

上述的内容讲解的是 TCP 首部信息，这些信息被封装在一个 IP 数据报中，该数据报结构如下图所示。

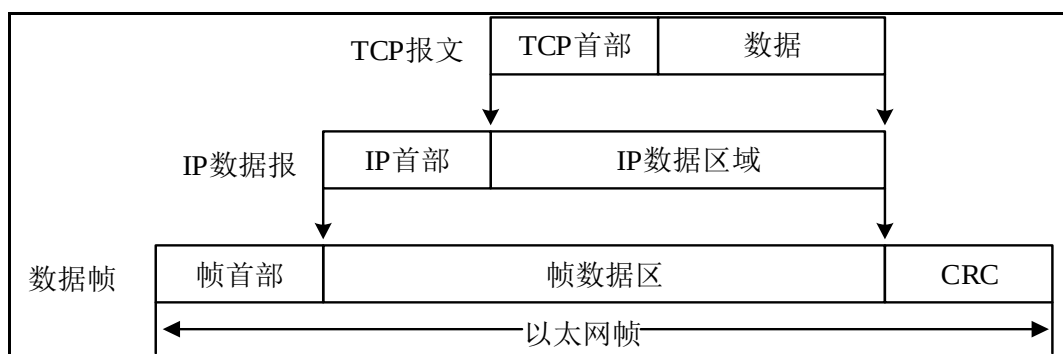


图 12.1.4.2 TCP 数据在 IP 数据报中的封装

12.1.5 lwIP 的 TCP 报文首部数据结构

实现 TCP 协议的文件有 tcp.h、tcp.c、tcp_in.c 和 tcp_out.c，这四个文件实现了 TCP 协议全部数据结构和函数，其中 tcp.c 文件包含了与 TCP 编程、TCP 定时器相关的函数，而 tcp_in.c 文件包含了 TCP 报文段输入处理函数，而 tcp_out.c 文件包含了 TCP 报文输出处理函数，当然 tcp.h 定义了宏和结构体。首先我们看一下 TCP 首部结构，这个结构为 tcp_hdr，如下源码所示：

```
struct tcp_hdr {
    PACK_STRUCT_FIELD(u16_t src);           /* 源端口 */
    PACK_STRUCT_FIELD(u16_t dest);          /* 目的端口 */
    PACK_STRUCT_FIELD(u32_t seqno);          /* 序号 */
    PACK_STRUCT_FIELD(u32_t ackno);          /* 确认序号 */
    PACK_STRUCT_FIELD(u16_t _hdrlen_rsvd_flags); /* 首部长度+保留位+标志位 */
    PACK_STRUCT_FIELD(u16_t wnd);            /* 窗口大小 */
    PACK_STRUCT_FIELD(u16_t chksum);          /* 校验位 */
    PACK_STRUCT_FIELD(u16_t urgptr);          /* 紧急指针 */
} PACK_STRUCT_STRUCT;
```

可见，lwIP 使用 tcp_hdr 结构体描述 TCP 首部各个字段，值得注意的是，该结构体的 _hdrlen_rsvd_flags 变量用来描述下图黄色部分的内容。

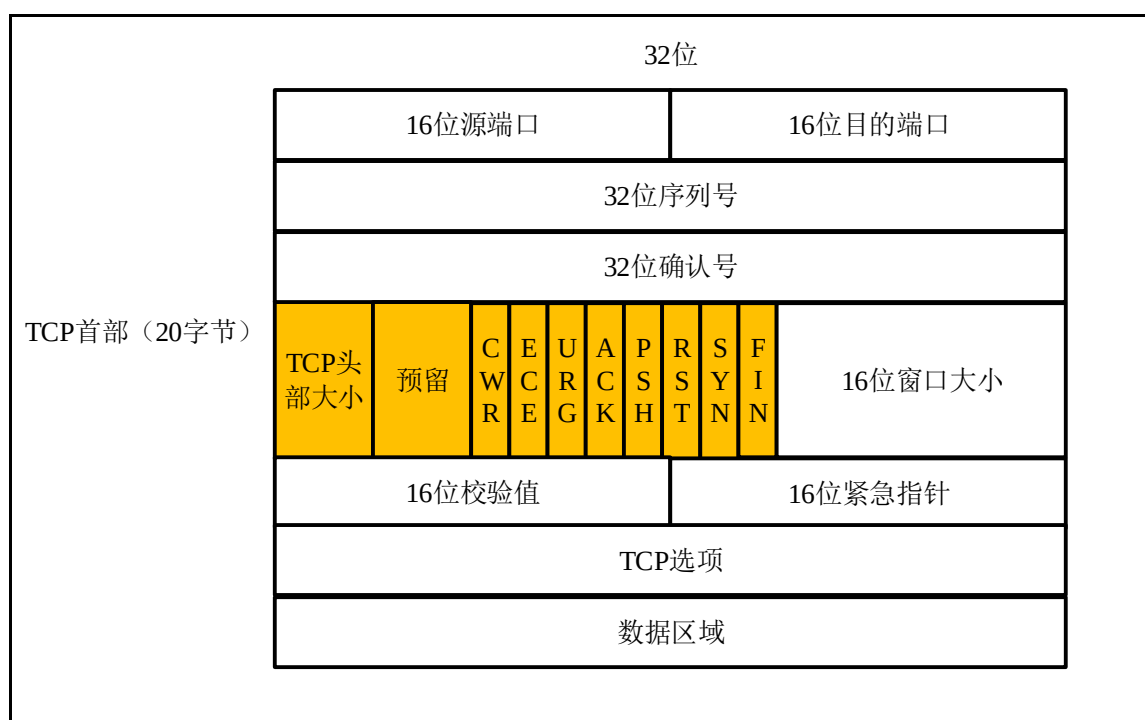


图 12.1.5.1 TCP 首部信息

12.1.6 lwIP 的 TCP 连接状态图

根据图 12.1.2.1 和 12.1.2.2 所示，发送端与接收端发送的指令会进入不同的状态，因此，lwIP 在 tcpbase.h 文件中定义了枚举类型 tcp_state，它是用来描述 TCP 的状态，该枚举 tcp_state 如下源码所示：

```
enum tcp_state {
    CLOSED          = 0,          /* 关闭状态 */
    LISTEN          = 1,          /* 监听状态 */
    SYN_SENT        = 2,          /* 发送请求连接 */
    SYN_RCVD        = 3,          /* 接收请求连接 */
    ESTABLISHED     = 4,          /* 连接状态已建立 */
    FIN_WAIT_1      = 5,          /* 程序已关闭该连接 */
}
```

```

FIN_WAIT_2    = 6,      /* 另一端已关闭连接 */
CLOSE_WAIT    = 7,      /* 等待程序关闭连接 */
CLOSING       = 8,      /* 两端同时收到对方的关闭请求 */
LAST_ACK      = 9,      /* 服务器等待对方接收关闭操作 */
TIME_WAIT     = 10     /* 关闭成功 */
};

```

下面笔者使用 TCP 状态转换图来描述连接可能在各个状态之间的转换关系，如下图所示。

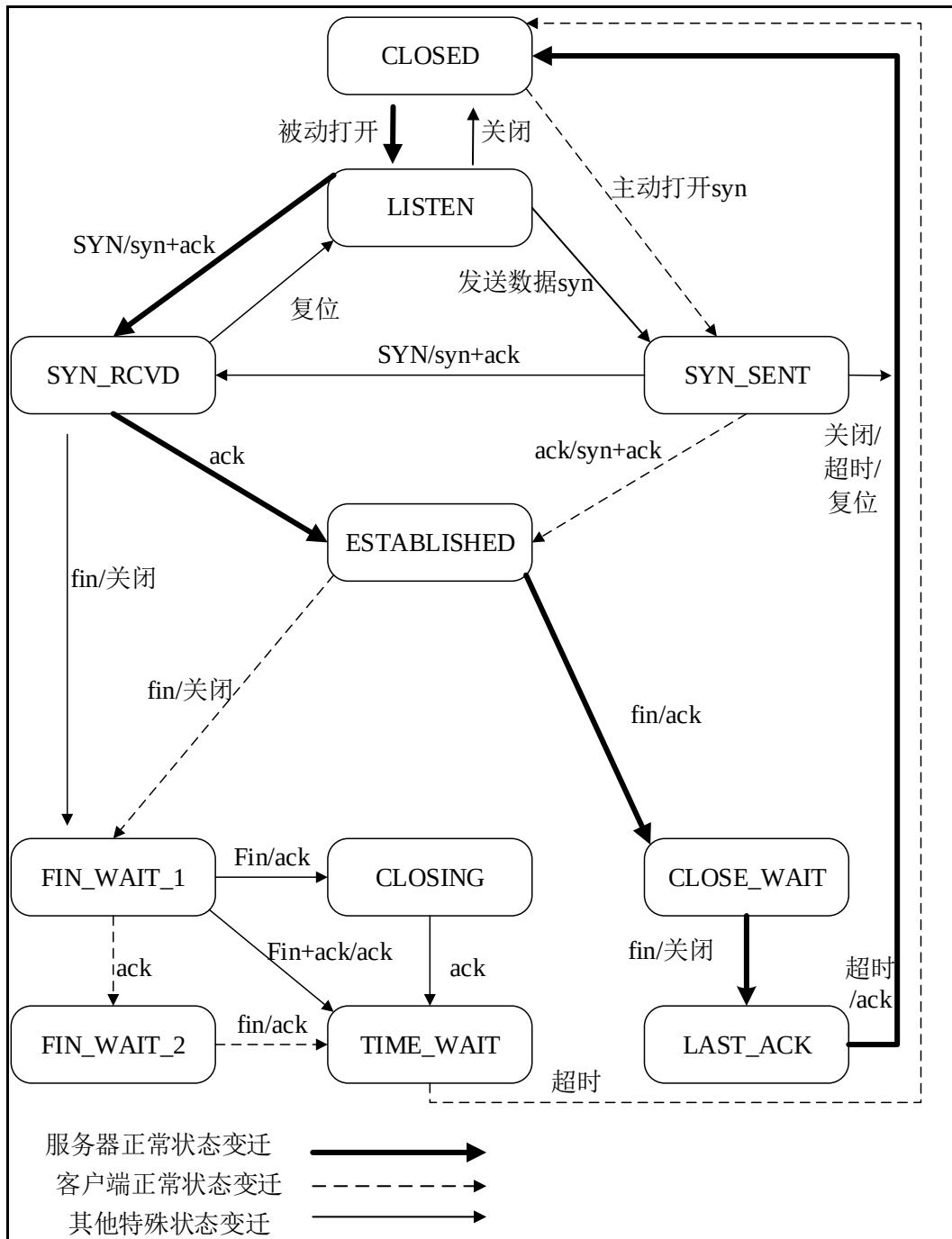


图 12.1.4.2 TCP 状态变迁图

如果 TCP 需要建立连接，则系统需要三次握手；如果 TCP 中断连接，则系统需要四次挥

手，现在笔者以上图 12.1.4.2 的 TCP 状态变迁图来绘制三次握手与四次挥手的状态图，不得不图片让我们更直观了解 TCP 连接和关闭，如下图所示。

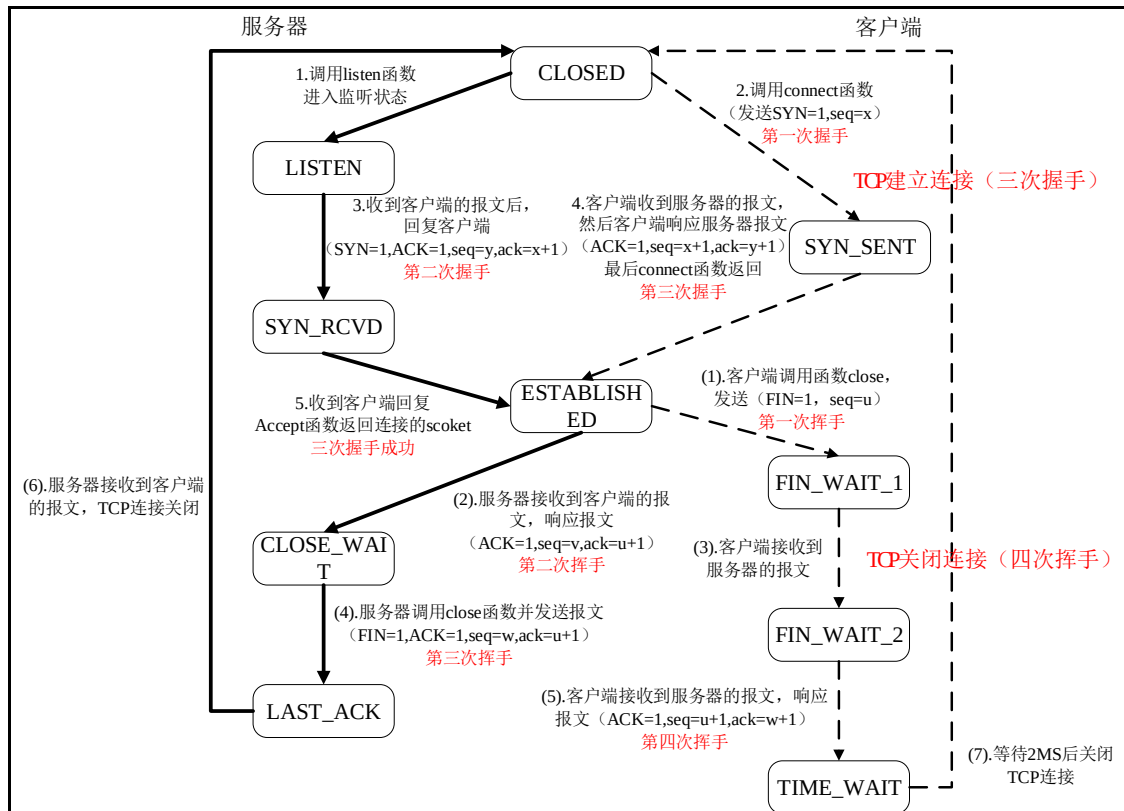


图 12.1.4.3 TCP 建立与关闭连接流程图

12.1.7 lwIP 的 TCP 控制块

到目前为止，笔者已经讲解了太多 TCP 协议理论的知识，这一小节我们正式踏入 lwIP 的 TCP 协议大门。在此之前我们先了解一下 TCP 控制块，这个控制块定义了 TCP 协议运作过程中的参数，例如发送窗口、数据缓冲区等，如下源码所示：

```

/** TCP 协议控制块 */
struct tcp_pcb {
    /** common PCB members */
    IP_PCB;
    TCP_PCB_COMMON(struct tcp_pcb);

    /** 远端端口号 */
    u16_t remote_port;

    /** 附加状态信息，如连接是快速恢复、一个被延迟的 ACK 是否被发送等 */
    tcpflags_t flags;

#define TF_ACK_DELAY 0x01U  /* 延迟发送 ACK. */
#define TF_ACK_NOW 0x02U  /* 延迟发送 ACK. */
#define TF_INFR 0x04U  /* 在快速恢复. */
#define TF_CLOSEPEND 0x08U  /* 关闭挂起 */
#define TF_RXCLOSED 0x10U  /* rx 由 tcp_shutdown 关闭 */

```

```
#define TF_FIN          0x20U    /* 连接在本地关闭 (FIN 段入队) */
#define TF_NODELAY      0x40U    /* 纳格尔禁用算法 */
#define TF_NAGLEMEMERR  0x80U    /* nagle 启用, 本地缓冲区溢出 */

/* Timers */
u8_t polltmr, pollinterval;
/* 控制块被最后一次处理的时间 */
u8_t last_timer;
/* 该字段记录该 PCB 被创建的时刻 */
u32_t tmr;

/* 接收变量 */
u32_t rcv_nxt;          /* 下一个期望收到的序号 */
tcpwnd_size_t rcv_wnd;  /* 当前接收窗口的大小, 会随着数据的接收与递交动态变化 */
tcpwnd_size_t rcv_ann_wnd; /* 将向对方通告的窗口大小, 随着数据的接收与递交动态变化 */
u32_t rcv_ann_right_edge; /* 上一次窗口通告时窗口的右边界值 */

/* 重传定时器, 该值随时间递增, 当大于 rto 的值时重传报文 */
s16_t rtime;

u16_t mss;    /* 对方可接收的最大报文段大小 */

/* RTT (往返时间) 估计变量 */
u32_t rttest;    /* RTT 估计每秒 500 毫秒 */
u32_t rtseq;     /* 序列号定时 */
s16_t sa, sv;    /* RTT 估计得到的平均值与时间差 */

s16_t rto;       /* 重新传输超时 (以 TCP_SLOW_INTERVAL 为单位) */
u8_t nrtx;       /* 重新发送的 */

/* 快速重新传输/恢复 */
u8_t dupacks;    /* 上述最大确认号被重复收到的次数 */
u32_t lastack;   /* 接收到的最大确认序号 */

/* 拥塞避免/控制变量 */
tcpwnd_size_t cwnd;    /* 连接当前的窗口大小 */
tcpwnd_size_t ssthresh; /* 拥塞避免算法启动的阈值 */

/* 第一个字节后面最后一个 rto 字节 */
u32_t rto_end;

/* 发送变量 */
u32_t snd_nxt;        /* 下一个要发送的序号 */
```

```

u32_t snd_wl1, snd_wl2;      /* 上一次收到的序号和确认号 */
u32_t snd_lbb;               /* 要缓冲的下一个字节的序列号 */
tcpwnd_size_t snd_wnd;       /* 发送窗口 */
tcpwnd_size_t snd_wnd_max;    /* 对方的最大发送窗口 */
/* 可用的缓冲区空间 */
tcpwnd_size_t snd_buf;

#define TCP_SNDQUEUELEN_OVERFLOW (0xffffU-3)
u16_t snd_queuelen; /* 可用的发送包数 */

#if TCP_OVERSIZE
/* Extra bytes available at the end of the last pbuf in unsent. */
u16_t unsent_oversize;
#endif /* TCP_OVERSIZE */

tcpwnd_size_t bytes_acked;

/* These are ordered by sequence number: */
struct tcp_seg *unsent;      /* 未发送的报文段 */
struct tcp_seg *unacked;     /* 已发送但未收到确认的报文段. */

struct pbuf *refused_data;    /* 以前收到但上层尚未取得的数据 */

#if LWIP_CALLBACK_API || TCP_LISTEN_BACKLOG
struct tcp_pcb_listen* listener;
#endif /* LWIP_CALLBACK_API || TCP_LISTEN_BACKLOG */

/* TCP 协议相关的回调函数 */
#if LWIP_CALLBACK_API
/* 当数据发送成功后被调用. */
tcp_sent_fn sent;
/* 接收数据完成后被调用 */
tcp_recv_fn recv;
/* 建立连接后被调用 . */
tcp_connected_fn connected;
/* 该函数被内核周期调用. */
tcp_poll_fn poll;
/* 发送错误时候被调用. */
tcp_err_fn errf;
#endif /* LWIP_CALLBACK_API */

/* 保持活性 */
u32_t keep_idle;

```

```
/* 坚持计时器计数器值 */
u8_t persist_cnt;

/* 坚持计时器关闭 */
u8_t persist_backoff;

/* 持续探测数 */
u8_t persist_probe;

/* 保持活性报文发送次数 */
u8_t keep_cnt_sent;

};
```

TCP 协议控制块的成员变量有点多，由于 TCP 协议在 lwIP 源码中占了 50% 之多，所以深入的去了解 TCP 协议可能会花很多精力和时间，这里笔者讲解重要的知识即可。首先我们先讲解一下接收数据相关的字段 `rcv_nxt`，`rcv_wnd`，`rcv_ann_wnd` 和数据发送的相关字段 `snd_nxt`，`snd_max`，`snd_wnd`，`acked`，这些字段和 TCP 中滑动窗口协议有密切关系的。

声明：下面的内容参考自《嵌入式网络那些事 LWIP 协议深度剖析与实战演练》，作者朱升林！。

1. TCP 控制块接收窗口

在 TCP 控制块中，关于接收窗口有四个变量来描述，如下图所示。

① `rcv_nxt`：是自己期望收到的下一个数据字节编号。

② `rcv_wnd`：表示接收窗口的大小。

③ `rcv_ann_wnd`：表示将向对方通告的窗口大小值，这个值在报文发送时会被填在首部中的窗口大小字段。

④ `rcv_ann_right_edge`：记录了上一次窗口通告时窗口右边界取值，该字段在窗口滑动过程中经常被用到。

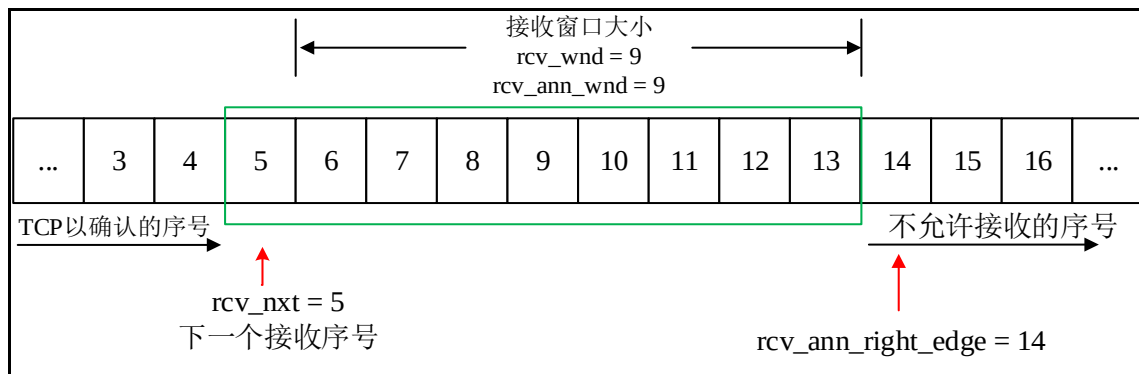


图 12.1.7.1 接收窗口

在上图中绿色框是窗口大小（`rcv_wnd = 9`），也就是说可以发送 9 个数据，而 `rcv_ann_wnd = 9` 就是通知对方窗口大小的值，而 `rcv_ann_right_edge` 记录了上一次窗口通告时窗口右边界取值（14），当然下一次发送时，这四个变量就不一定是上述图中的值了，它们会随着数据的发送与接收动态改变。当接收到数据后，数据会被放在接收窗口中等待上层调用，`rcv_nxt` 字段会指向下一个期望接收的编号，同时窗口值 `rcv_wnd` 值会减少，当上层取走相关的数据后，窗口的值会增加；`rcv_ann_wnd` 在整个过程中都是动态计算的，当 `rcv_wnd` 值改变时，内核会计算一个合理的窗口值 `rcv_ann_wnd`（并不一定与 `rcv_wnd` 相等），在下一次报文发送时，通告窗口的值（`rcv_ann_wnd`）会被填入报文的首部，同时右边界值

rcv_ann_right_edge 也在报文发送后更新数值。

2. TCP 控制块发送窗口

在 lwIP 源码描述 TCP 的发送窗口涉及 4 个变量，它们之间的关系如下图所示。

- ① lastack: 字段记录了被接收方确认的最高序列号。
- ② snd_nxt: 表示自己将要发送的下一个数据的起始编号。
- ③ snd_wnd: 记录了当前的发送窗口大小，它常被设置为接收方通告的接收窗口值。
- ④ snd_lbb: 记录了下一个将被应用程序缓存的数据的起始编号。

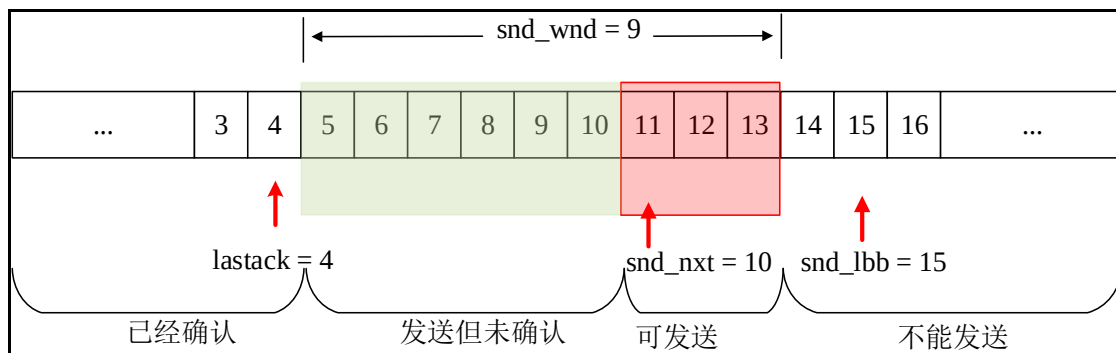


图 12.1.7.2 发送窗口

可以看出，左边部分是已经发送并确认的数据，绿色框是已经发送但未确认的数据（需要等待对方确认），红色框可以发送的数据，最右边的是不能发送的。上面这四个字段的值也是动态变化的，每当收到接收方的一个有效 ACK 后，lastack 的值就做相应的增加，指向下一个待确认数据的编号，当发送一个报文后，snd_nxt 的值就做相应的增加，指向下一个待发送数据。snd_nxt 和 lastack 之间的差值不能超过 sndwnd 的大小。由于实际数据发送时是按照报文段的形式组织的，因此可能存在这样的情况：即使发送窗口允许，但并不是窗口内的所有数据都能被发送以填满窗口，如上图中编号为 11~13 的数据，可能因为它们太小不能组织成一个有效的报文段，因此不会被发送。发送方会等到新的确认到来，从而使发送窗口向右滑动，使得更多的数据被包含在窗口中，这样再启动下一个报文段的发送。

3. 监听控制块

lwIP 除了定义结构体 tcp_pcb，它还定义了结构体 tcp_pcb_listen，前者我们知道有这个就行，后者结构体 tcp_pcb_listen 主要描述 LISTEN 状态的连接，一般用于描述处于监听状态的连接，在处于 LISTEN 状态的连接只记录本地端口的信息，不记录任何远程端口的信息，当然处于该状态不会进行数据发送、连接握手之类的服务，主要是分配完整的 TCP 控制块是比较消耗内存资源的，在 TCP 协议在连接之初，是无法进行数据交互，那么在监听的时候只需要把对方主机的相关信息得到，然后无缝切换到完整的 TCP 控制块中，这样子就能节省不少资源，tcp_pcb_listen 的庐山真面目，如下源码所示：

```
/** 用于监听 pcb 的 TCP 协议控制块 */
struct tcp_pcb_listen {
    /** 该宏包含源 IP 地址、目的 IP 地址两个重要字段 */
    IP_PCB;

    /** 两种控制块都具有的字段 */
    TCP_PCB_COMMON(struct tcp_pcb_listen);
#ifdef LWIP_CALLBACK_API
    /** 函数在连接侦听器时调用 */
    tcp_accept_fn accept;
#endif
};
```

```
#endif /* LWIP_CALLBACK_API */
};
```

4. 控制块链表:

为了描述 TCP 控制块，lwIP 内核定义了四条链表来链接处于不同状态下的控制块，TCP 操作一般对于链表上的控制块进行查找，这四个控制块链表在 tcp.c 文件中，如下源码所示：

```
/*连接所有进行了端口号绑定，但是还没有发起连接（主动连接）或进入侦听状态（被动连接）的控制块*/
struct tcp_pcb *tcp_bound_pcbs;
/* 连接所有进入侦听状态（被动连接）的控制块 */
union tcp_listen_pcbs_t tcp_listen_pcbs;
/* 连接所有处于其他状态的控制块. */
struct tcp_pcb *tcp_active_pcbs;
/* 连接所有处于 TIME-WAIT 状态的控制块 */
struct tcp_pcb *tcp_tw_pcbs;
```

5. TCP 报文段缓冲

在内核中，所有待发送的数据或者已经接收的数据都会以报文的形式保存，一般都是保存在 pbuf 中，为了更好的管理报文段的 pbuf，内核引用了一个 tcp_seg 的结构体，该结构体的作用就是把所有报文段连接起来，当然这些报文段可以是无发送、已发送并未确认的或者是以收到的报文，它们都保存在 TCP 控制块缓冲区中，该结构体如下源码所示：

```
/* 定义组织 TCP 报文段的结构 */
struct tcp_seg {
    struct tcp_seg *next;          /* 该指针用于将报文段组织为队列的形式 */
    struct pbuf *p;                /* 指向装载报文段的 pbuf */
    u16_t len;                     /* 报文段中的数据长度 */
    u8_t flags;

#define TF_SEG_OPTS_MSS            (u8_t)0x01U /* 包含了最大报文段大小选项 */
#define TF_SEG_OPTS_TS            (u8_t)0x02U /* 包含了时间戳选项 */
#define TF_SEG_DATA_CHECKSUMMED (u8_t)0x04U /* 所有数据(不是 header)都是校验和为 */
#define TF_SEG_OPTS_WND_SCALE    (u8_t)0x08U /* 包括 WND 规模选项(仅用于 SYN 段) */
#define TF_SEG_OPTS_SACK_PERM    (u8_t)0x10U /* 包括 SACK 允许选项(仅在 SYN 段中使用) */
    /* 指向报文段中的 TCP 首部 */
    struct tcp_hdr *tcphdr; /* TCP 报头 */
};
```

每个控制块中都维护了三个缓冲队列，unsent、unacked、ooseq 三个字段（这三个字段已经在 TCP 控制块时候讲解了）。unsent 用于连接还未被发送出去的报文段、unacked 用于连接已经发送出去但是还未被确认的报文段、ooseq 用于连接接收到的无序报文段，如下图所示。

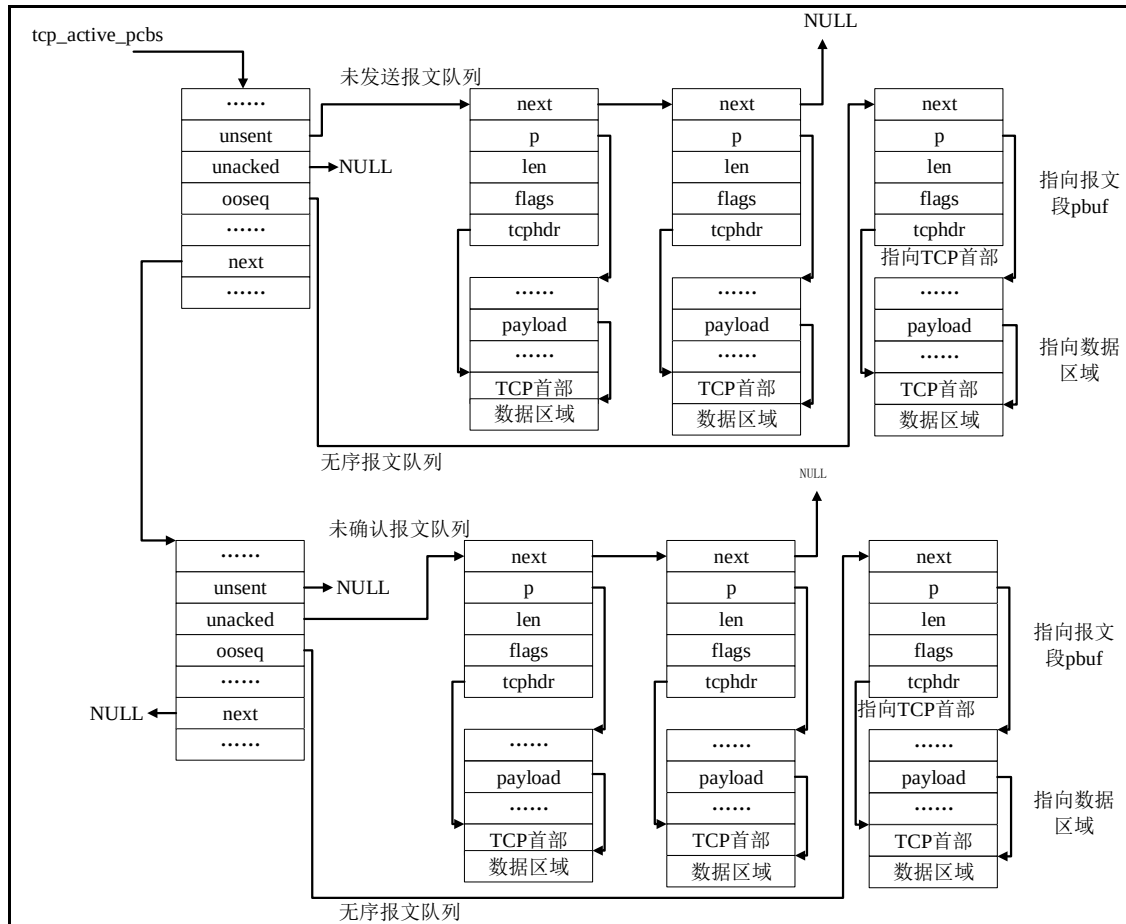


图 12.1.7.3 报文段缓冲队列

12.1.8 lwIP 的 TCP 编程

(2) TCP 报文段的接收

报文段的接收函数是 `tcp_input`，该函数位于 `tcp_inc.c` 文件中，如下源码所示：

```
void
tcp_input(struct pbuf *p, struct netif *inp)
{
    struct tcp_pcb *pcb, *prev;
    struct tcp_pcb_listen *lpcb;

    u8_t hdrlen_bytes;
    err_t err;

    /* 指向 TCP 首部 */
    tcphdr = (struct tcp_hdr *)p->payload;

    /* 第一步：检查 TCP 报头是否少于 20 */
    if (p->len < TCP_HLEN)
    {

```

```

        /* 释放空间掉弃报文段 */
        goto dropped;
    }

    /* 第二步：判断是否是广播与多播类型 */
    if (ip_addr_isbroadcast(ip_current_dest_addr(), ip_current_netif()) ||
        ip_addr_ismulticast(ip_current_dest_addr()))
    {
        /* 释放空间掉弃报文段 */
        goto dropped;
    }

    /* 获取 tcphdr 首部字节 */
    hdrlen_bytes = TCPH_HDRLEN_BYTES(tcphdr);
    /* 第三步：检测 TCP 报文长度 */
    if ((hdrlen_bytes < TCP_HLEN) || (hdrlen_bytes > p->tot_len))
    {
        /* 释放空间掉弃报文段 */
        goto dropped;
    }

    /* 移动 pbuf 中的有效负载指针，使其指向 TCP 数据 */
    /* tcphdr_optlen = TCP 报头选项长度 (TCP 报头总长度 - TCP 标准报头 20 字节) */
    tcphdr_optlen = (u16_t)(hdrlen_bytes - TCP_HLEN);
    tcphdr_opt2 = NULL; /* tcphdr_opt2 指向 NULL */
    /* 判断 TCP 报头是否在一个 pbuf 中 */
    if (p->len >= hdrlen_bytes)
    {
        /* 若 TCP 报头在第一个 pbuf 中 */
        tcphdr_opt1len = tcphdr_optlen; /* tcphdr_opt1len = TCP 报头选项长度 */
        pbuf_remove_header(p, hdrlen_bytes); /* 将指针移动到 pbuf 数据中 */
    }
    else
    {
        u16_t opt2len;

        /* 删除 TCP 首部 */
        pbuf_remove_header(p, TCP_HLEN);

        /* 确定选项的第一部分和第二部分长度 */
        tcphdr_opt1len = p->len;
        opt2len = (u16_t)(tcphdr_optlen - tcphdr_opt1len);
    }
}

```

```

/* 移除 tcphdr_optlen 选项 */
pbuf_remove_header(p, tcphdr_optlen);

/* 检查 TCP 报头选项部分是否在第二个 pbuf 中 */
if (opt2len > p->next->len)
{
    /* 丢弃过短的报文 */
    goto dropped;
}

/* 记住指向 TCP 报头选项的第二部分的指针
   (有部分选项在第二个 pbuf 中, 记录 TCP 报头选项的开始部分) */
tcphdr_opt2 = (u8_t *)p->next->payload;

/* 将第二个 pbuf 的指针指向 pbuf 的数据部分 */
pbuf_remove_header(p->next, opt2len);
p->tot_len = (u16_t) (p->tot_len - opt2len);
}

/* 提取源端口 */
tcphdr->src = lwip_ntohs(tcphdr->src);
/* 提取目标端口 */
tcphdr->dest = lwip_ntohs(tcphdr->dest);
/* 提取序号 */
seqno = tcphdr->seqno = lwip_ntohl(tcphdr->seqno);
/* 提取确认号 */
ackno = tcphdr->ackno = lwip_ntohl(tcphdr->ackno);
/* 提取窗口 */
tcphdr->wnd = lwip_ntohs(tcphdr->wnd);
/* 6 位标志位 */
flags = TCPH_FLAGS(tcphdr);
/* TCP 数据包中数据的总长度, 对于有 FIN 或 SYN 标志的数据包, 该长度要加 1 */
tcplen = p->tot_len;
if (flags & (TCP_FIN | TCP_SYN))
{
    tcplen++;

    if (tcplen < p->tot_len)
    {
        /* 释放空间掉弃报文段 */
        goto dropped;
    }
}
}

```

```

/* *****省略代码***** */

/* 如果 pcb 在回调中被中止(通过调用 tcp_abort()), 则跳转目标。 */
aborted:
    tcp_input_pcb = NULL;
    recv_data = NULL;

    if (inseg.p != NULL)
    {
        pbuf_free(inseg.p);
        inseg.p = NULL;
    }
}
else
{
    /*如果在 3 张链表里都未找到匹配的 pcb, 则调用 tcp_rst 向源主机发送一个 TCP 复位数据包*/
    if (!(TCPH_FLAGS(tcphdr) & TCP_RST))
    {
        TCP_STATS_INC(tcp.proterr);
        TCP_STATS_INC(tcp.drop);
        tcp_rst(NULL, ackno, seqno + tcplen, ip_current_dest_addr(),
                ip_current_src_addr(), tcphdr->dest, tcphdr->src);
    }

    pbuf_free(p);
}

return;
dropped:
    pbuf_free(p);
}

```

上述的源码大概 400 多行, 该函数可以分为上部分与下部分, 上部分主要讲述了对 IP 层递交传输层的数据报检验, 例如检验数据报是否正常操作、是否包含数据、该数据报是否为广播或者多播, 如果以上检验成立, 则系统把该数据报掉弃处理, 并释放 pbuf。下部分主要对 tcp_active_pcb 链表寻找对应的 TCP 控制块, 如果找到了 TCP 控制块, 则调用 tcp_process 函数处理; 如果找不到 TCP 控制块, 则内核转换到 tcp_tw_pcb 链表中查找; 如果在 tcp_tw_pcb 链表中找到 TCP 控制块, 则内核调用 tcp_timewait_input 函数处理它; 如果这两个链表没有找到 TCP 控制块, 则系统会进入 tcp_listen_pcb 链表中查找; 如果找到了就调用 tcp_listen_input 函数处理; 如果三个链表都找不到的话, 则系统就释放 pbuf 内存。

(3) TCP 报文段的发送

传输层与网络层的交互函数为 tcp_output, 它在 tcp_output.c 文件中定义, 如下源码所示:

```

/* 发送控制块缓冲队列中的报文段 */
err_t
tcp_output(struct tcp_pcb *pcb)

```

```

{
    struct tcp_seg *seg, *useg;
    u32_t wnd, snd_nxt;
    err_t err;
    struct netif *netif;

    /* 如果控制块当前正有数据被处理，这里不做任何输出，直接返回*/
    if (tcp_input_pcb == pcb) /* 在控制块的数据处理完成后，内核会再次调用 */
    {
        return ERR_OK; /* 调用 tcp_output 发送数据，见函数 tcp_input */
    }

    /* 从发送窗口和阻塞窗口取小者得到有效发送窗口，拥塞避免会讲解到这个原理 */
    wnd = LWIP_MIN(pcb->snd_wnd, pcb->cwnd);
    /* 未发送队列 */
    seg = pcb->unsent;

    if (seg == NULL)
    {
        /* 若要求立即确认，但该 ACK 又不能带数据出去，则只发送一个纯 ACK 的报文段 */
        if (pcb->flags & TF_ACK_NOW)
        {
            return tcp_send_empty_ack(pcb); /* 发送只带 ACK 的报文段 */
        }

        /* 没什么可送的 */
        goto output_done;
    }
    else
    {
        /* 判断本地 IP 地址与远程 IP 地址是否同一网段 */
        netif = tcp_route(pcb, &pcb->local_ip, &pcb->remote_ip);

        if (netif == NULL)
        {
            return ERR_RTE;
        }

        /* 如果没有本地 IP 地址，从 netif 获得一个 */
        if (ip_addr_isany(&pcb->local_ip))
        {

```

```

    const ip_addr_t *local_ip = ip_netif_get_local_ip(netif,
                                                         &pcb->remote_ip);

    if (local_ip == NULL)
    {
        return ERR_RTE;
    }

    ip_addr_copy(pcb->local_ip, *local_ip);
}

/* 处理窗口中不匹配的当前段 */
if (lwip_ntohl(seg->tcphdr->seqno) - pcb->lastack + seg->len > wnd)
{
    /* 开始持续定时器 */
    if (wnd == pcb->snd_wnd && pcb->unacked == NULL && \
        pcb->persist_backoff == 0)
    {
        pcb->persist_cnt = 0;
        pcb->persist_backoff = 1;
        pcb->persist_probe = 0;
    }

    /* 我们需要一个 ACK，但是现在不能发送数据，所以发送一个空 ACK */
    if (pcb->flags & TF_ACK_NOW)
    {
        return tcp_send_empty_ack(pcb);
    }

    goto output_done;
}

/* 停止持续计时器，如果以上条件不满足 */
pcb->persist_backoff = 0;

/* useg 应该指向未处理队列的最后一个 tcp_seg 结构 */
useg = pcb->unacked;

if (useg != NULL)
{
    for (; useg->next != NULL; useg = useg->next);
}

```



```

/* 可用数据和窗口允许它发送报文段，直到把数据全部发送出去或者填满发送窗口 */
while (seg != NULL &&
        lwip_ntohl(seg->tcphdr->seqno) - pcb->lastack + seg->len <= wnd)
{

    /* 如果 nagle 算法可以阻止发送，就停止发送 */
    if ((tcp_do_output_nagle(pcb) == 0) &&
        ((pcb->flags & (TF_NAGLEMEMERR | TF_FIN)) == 0))
    {
        break;
    }

    if (pcb->state != SYN_SENT)    /* 当前不为 SYN_SENT 状态 */
    {
        TCPH_SET_FLAG(seg->tcphdr, TCP_ACK); /* 填写首部中的 ACK 标志 */
    }

    /* 调用函数发送报文段 */
    err = tcp_output_segment(seg, pcb, netif);

    if (err != ERR_OK)
    {
        /* segment could not be sent, for whatever reason */
        tcp_set_flags(pcb, TF_NAGLEMEMERR);
        return err;
    }

    /* 得到下一个未发送的 tcp_seg */
    pcb->unsent = seg->next;

    if (pcb->state != SYN_SENT)
    {
        tcp_clear_flags(pcb, TF_ACK_DELAY | TF_ACK_NOW);
    }

    /* 计算 snd_nxt 的值 */
    snd_nxt = lwip_ntohl(seg->tcphdr->seqno) + TCP_TCPLen(seg);
    /* 更新下一个要发送的数据编号 */
    if (TCP_SEQ_LT(pcb->snd_nxt, snd_nxt))
    {
        pcb->snd_nxt = snd_nxt;
    }

    /* 如果发送出去的报文段数据长度不为 0，或者带有 SYN、FIN 标志，则将该报
       文段加入到未确认队列中以便超时后重传 */

```

```
if (TCP_TCPLLEN(seg) > 0)
{
    seg->next = NULL; /* 空报文段 next 字段 */

    /* 若未确认队列为空，则直接挂接 */
    if (pcb->unacked == NULL)
    {
        pcb->unacked = seg;
        useg = seg; /* 变量 useg 指向未确认队列尾部 */
    }
    else
    {
        /* 如果未确认队列不为空，则需要把当前报文按照顺序组织在队列中 */
        if (TCP_SEQ_LT(lwip_ntohl(seg->tcphdr->seqno),
                       lwip_ntohl(useg->tcphdr->seqno)))
        {
            /* 如果当前报文的序列号比队列尾部报文的序列号低，则从队列首部开始
              查找合适的位置，插入报文段*/
            struct tcp_seg **cur_seg = &(pcb->unacked);

            while (*cur_seg &&
                  TCP_SEQ_LT(lwip_ntohl((*cur_seg)->tcphdr->seqno),
                              lwip_ntohl(seg->tcphdr->seqno)))
            {
                cur_seg = &((*cur_seg)->next );
            } /* 找到插入位置，将报文段插入到队列中 */

            seg->next = (*cur_seg);
            (*cur_seg) = seg;
        }
        else
        {
            /* 报文段序号最高，则放在未确认队列尾部 */
            useg->next = seg;
            useg = useg->next;
        }
    }
}

else /* 报文段长度为 0，不需要重传，直接删除 */
{
    tcp_seg_free(seg);
}
```

```

        seg = pcb->unsent; /* 发送下一个报文段 */
    }

#ifdef TCP_OVERSIZE

    if (pcb->unsent == NULL)
    {
        /* 清 0 已发送的窗口探测包数目 */
        pcb->unsent_oversize = 0;
    }

#endif /* TCP_OVERSIZE */

output_done:
    tcp_clear_flags(pcb, TF_NAGLEMEMERR);
    return ERR_OK;
}

```

从整体来看，此函数首先检测报文是否满足发送要求，接着判断控制块的 flags 字段是否被设置为 TF_ACK_NOW 状态，如果是，则发送一个纯粹 ACK 报文段，因此，此时 unsent 队列中无数据发送或者发送窗口不允许发送数据。如果内核能发送数据，则就将 ACK 应答捎带发送出去，同时在发送的时候先找到未发送链表，然后调用 tcp_output_segment()->ip_output_if() 函数进行发送，直到把未发送链表的数据完全发送出去或者直到填满发送窗口，并且更新发送窗口相关字段，当然也要将这些已发送但是未确认的数据存储在未确认链表中，以防丢失数据进行重发操作，放入未确认链表的时候是按序号升序进行排序的。

12.1.9 lwIP 的 TCP 建立与关闭连接原理

下面笔者来讲解一下 lwIP 如何实现 TCP 客户端以及服务器连接，这里我们可以根据 TCP 连接示意图来讲解 lwIP 源码是如何实现 TCP 连接的。在讲解之前，我们先了解 TCP 客户端的配置流程，如下所示：

1. TCP 客户端建立连接原理：

① 创建 TCP 控制块

调用函数 tcp_new 创建 TCP 控制块。

② 连接指定的 IP 地址和端口号

调用函数 tcp_connect 连接到目的地址的指定端口上，注意：当连接成功后进入回调 tcp_client_connected 函数。

③ 接收数据

调用函数 tcp_recved 接收数据。

④ 发送数据

调用函数 tcp_write 发送数据。

从上述步骤可知，我们主要调用函数 tcp_connect 连接远程服务器，这个函数和 TCP 连接图存在某种联系，下面笔者简单的讲解这个函数到底如何连接服务器，该函数如下所示：

```
err_t
```

```
tcp_connect(struct tcp_pcb *pcb, const ip_addr_t *ipaddr, u16_t port,
            tcp_connected_fn connected)
{
    /*.....前面省略大部分代码.....*/
    /* 发送 SYN 与 MSS 选项一起发送 */
    ret = tcp_enqueue_flags(pcb, TCP_SYN);
    if (ret == ERR_OK) {
        /* 设置当前 TCP 控制块为 SYN_SENT 状态 */
        pcb->state = SYN_SENT;
        if (old_local_port != 0) {
            TCP_RMV(&tcp_bound_pcb, pcb);
        }
        TCP_REG_ACTIVE(pcb);
        MIB2_STATS_INC(mib2.tcpactiveopens);
        tcp_output(pcb);
    }
    return ret;
}
```

可见，上述的(1)表示程序调用函数 `tcp_enqueue_flags` 构建连接请求报文（TCP_SYN）；上述的(2)表示当前 TCP 控制块设置为 `SYN_SENT` 状态；上述的(3)表示程序调用函数 `tcp_output` 向服务器发送连接请求报文。下面笔者使用一个示意图来描述上述的内容，如下图所示。

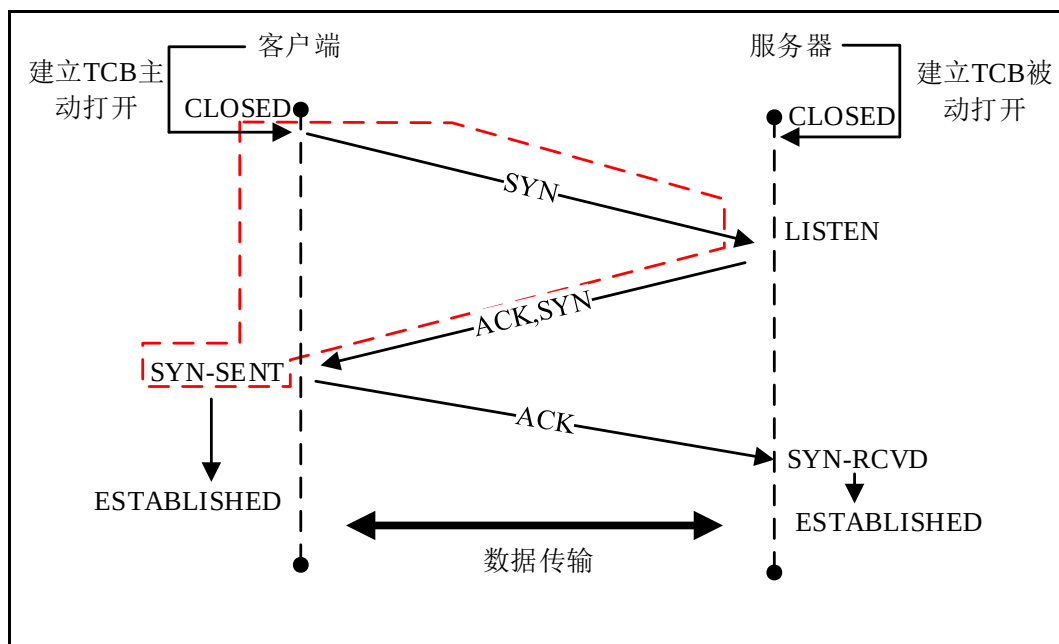


图 12.1.9.1 客户端从 CLOSED 状态转换成 SYN-SENT 状态

上图中红色框框的是 `tcp_connect` 函数实现流程，这里可以称之为 TCP 第一次握手，此时客户端等待服务器的连接应答报文（TCP_ACK）。当客户端接收服务器应答报文（TCP_ACK）时，系统会在 `tcp_input` 这个函数处理该应答报文。这个函数在上小节也讲解过，这里我们无需重复讲解了，该连接应答报文会在 `tcp_input-->tcp_process` 函数下处理，注意：`tcp_input` 函数中 `flags` 的全局变量是获取接收数据报的首部标志位（TCP_ACK+ TCP_SYN），这个过程请

看 tcp_in.c 文件 234 行的代码，如下源码所示：

```
static err_t
tcp_process(struct tcp_pcb *pcb)
{
    /*.....此处省略了很多代码..... */
    switch (pcb->state) {
        case SYN_SENT:
            /* 收到 SYN ACK 与预期的序列号? */
            if ((flags & TCP_ACK) && (flags & TCP_SYN)      (1)
                && (ackno == pcb->lastack + 1)) {
                pcb->rcv_nxt = seqno + 1;
                pcb->rcv_ann_right_edge = pcb->rcv_nxt;
                pcb->lastack = ackno;
                pcb->snd_wnd = tcphdr->wnd;
                pcb->snd_wnd_max = pcb->snd_wnd;
                pcb->snd_wll = seqno - 1;
                pcb->state = ESTABLISHED;      (2)
            }

            /*.....此处省略了很多代码..... */
    }
    /*.....此处省略了很多代码..... */
}
```

上述的(1)就是为了判断服务器应答报文的标志位是否包含 TCP_ACK 和 TCP_SYN，如果该应答报文包含这些标志位，则系统执行上述(2)的代码设置 TCP 控制块为 ESTABLISHED 状态。这里笔者也是使用一个示意图来描述上述的内容，如下图所示。

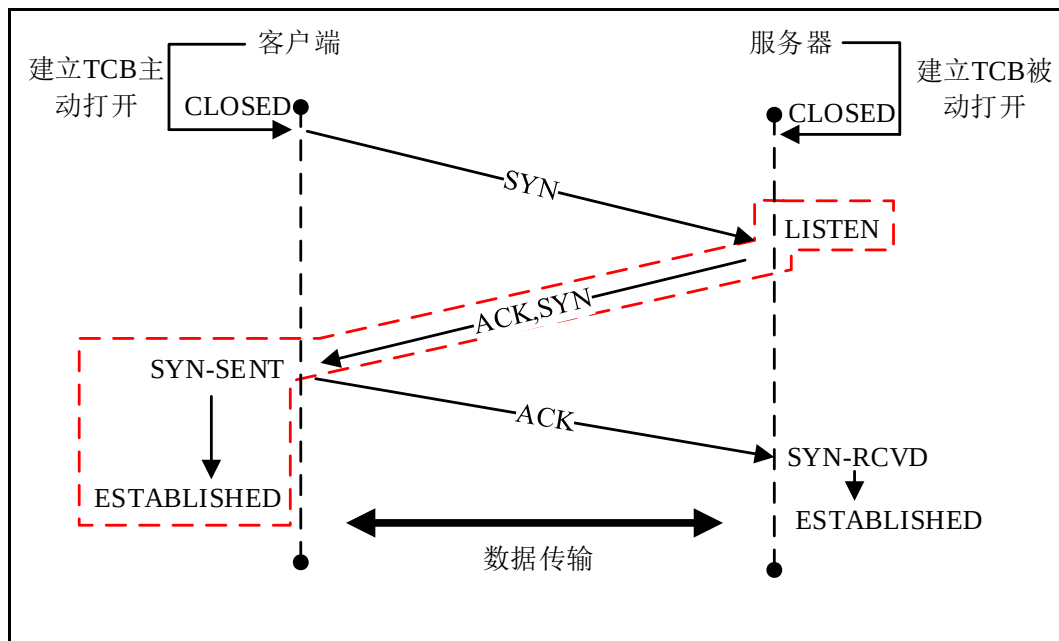


图 12.1.9.2 客户端从 SYN-SENT 状态转换成 ESTABLISHED 状态

上图的红色框框就是上述内容实现的过程，这里可以称之为 TCP 第二次握手，此时客户

端必须发送 TCP_ACK 应答报文给服务器才能实现第三次握手。上面的函数 tcp_process 执行完成之后返回到 tcp_input 函数，该函数的 553 行代码调用了 tcp_output 函数发送应答报文，该函数如下所示：

```
err_t
tcp_output(struct tcp_pcb *pcb)
{
    /*.....此处省略了很多代码..... */
    if (pcb->state != SYN_SENT) {
        TCPH_SET_FLAG(seg->tcphdr, TCP_ACK);
    }
    /* 发送应答包 */
    err = tcp_output_segment(seg, pcb, netif);
    /*.....此处省略了很多代码..... */
}
```

因为 TCP 控制块已经是 ESTABLISHED 状态了，所以这个 if 语句判断为真且执行 if 语句内的代码，这个代码主要添加该数据报的首部标志位 TCP_ACK，接着系统调用 tcp_output_segmen 发送该应答包，这里就完成了三次握手的动作。下面笔者使用一个示意图来讲解这个过程，如下图所示。

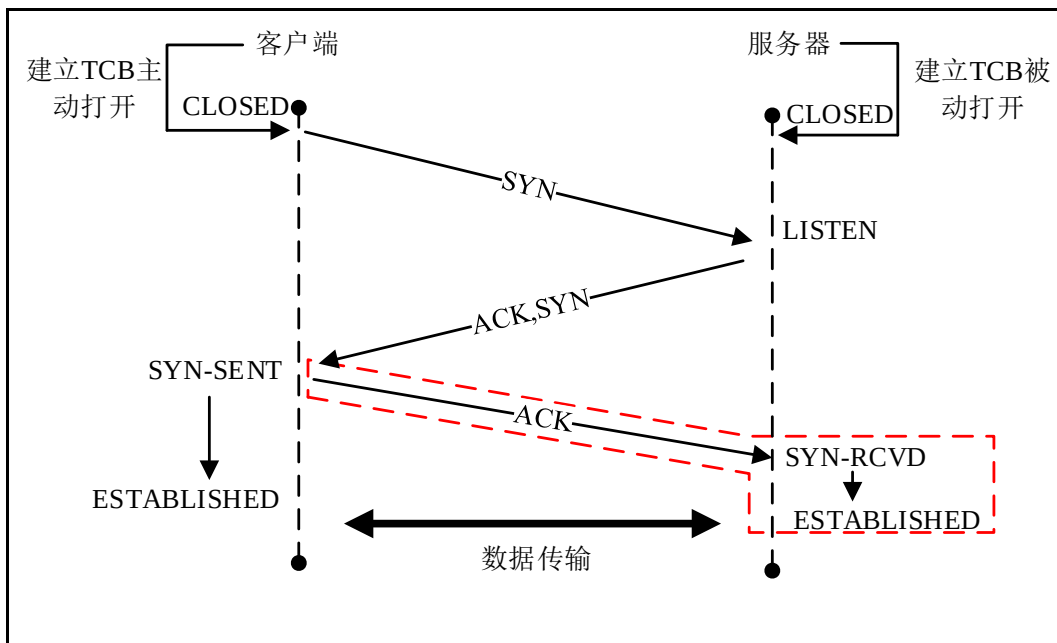


图 12.1.9.3 客户端发送应答包给服务器

2. TCP 服务器建立连接原理

TCP 服务器的配置流程，如下步骤所示：

① 创建 TCP 控制块

调用函数 tcp_new 创建 TCP 控制块。

② 绑定本地 IP 地址和端口号

调用函数 tcp_bind 绑定本地 IP 地址和端口号。

③ 连接请求

调用函数 tcp_accept 等待连接。注意：有连接时，会调用函数 lwip_tcp_server_accept 处理

④ 接收数据

调用函数 `tcp_recved` 接收数据。

⑤ 发送数据

调用函数 `tcp_write` 发送数据。

首先我们调用 `tcp_listen` 函数让服务器进去监听状态，简单来说，TCP 服务器控制块从 CLOSER 转换成 LISTEN 状态，如下源码所示：

```
#define tcp_listen(pcb) tcp_listen_with_backlog(pcb, TCP_DEFAULT_LISTEN_BACKLOG)
struct tcp_pcb *
tcp_listen_with_backlog(struct tcp_pcb *pcb, u8_t backlog){
    LWIP_ASSERT_CORE_LOCKED();
    return tcp_listen_with_backlog_and_err(pcb, backlog, NULL);
}
struct tcp_pcb *
tcp_listen_with_backlog_and_err(struct tcp_pcb *pcb, u8_t backlog, err_t *err){
    /* .....省略代码..... */
    lpcb->callback_arg = pcb->callback_arg;
    lpcb->local_port = pcb->local_port;
    lpcb->state = LISTEN;                                (1)
    lpcb->prio = pcb->prio;
    lpcb->so_options = pcb->so_options;
    lpcb->netif_idx = pcb->netif_idx;
    lpcb->ttl = pcb->ttl;
    lpcb->tos = pcb->tos;
    /* .....省略代码..... */
}
```

上述的(1)就是让 TCP 服务器控制块从 CLOSER 状态转换成 LISTEN 状态，下面笔者使用一个图来描述上述的内容，如下图所示。

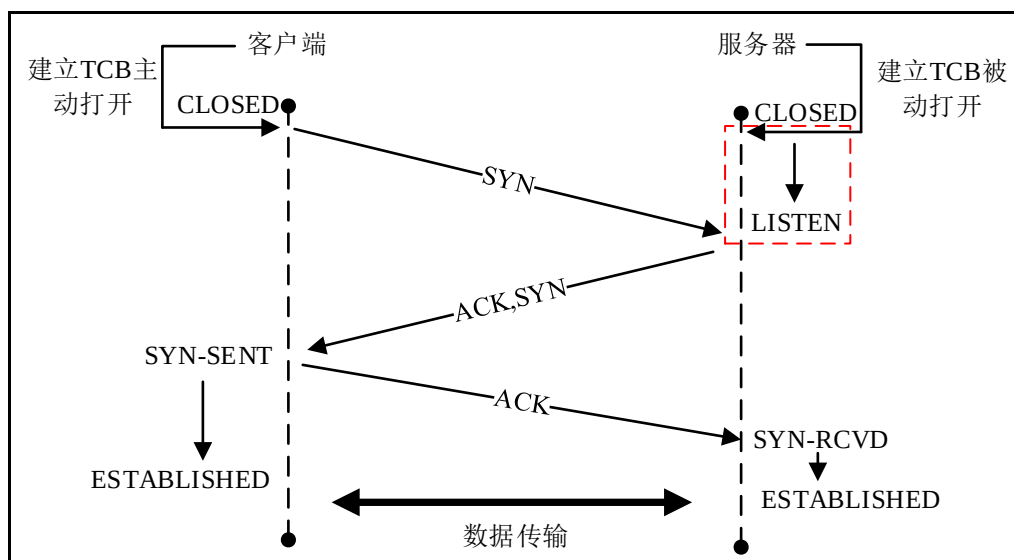


图 12.1.9.4 `tcp_listen` 函数实现流程

上图的红色框框就是由 `tcp_listen` 函数实现的，下面开始讲解 TCP 第一次握手流程，对于

服务器而言，它是先接收客户端发来的连接请求包并判断该请求报文的首部标志位是否包含 TCP_SYN，这个请求报文的处理是由 tcp_input→tcp_listen_input 函数处理的，该函数与客户端请求包相关的源码如下所示：

```
static void
tcp_listen_input(struct tcp_pcb_listen *pcb)
{
    /* .....此处省略多行代码..... */

    if (flags & TCP_RST) {
        return;
    }

    if (flags & TCP_ACK) {
        /* .....此处省略多行代码..... */
    } else if (flags & TCP_SYN) {
        /* .....此处省略多行代码..... */
        ip_addr_copy(npcb->local_ip, *ip_current_dest_addr());
        ip_addr_copy(npcb->remote_ip, *ip_current_src_addr());
        npcb->local_port = pcb->local_port;
        npcb->remote_port = tcphdr->src;
        npcb->state = SYN_RCVD; (1)
        npcb->rcv_nxt = seqno + 1;
        npcb->rcv_ann_right_edge = npcb->rcv_nxt;
        iss = tcp_next_iss(npcb);
        npcb->snd_wl2 = iss;
        npcb->snd_nxt = iss;
        npcb->lastack = iss;
        npcb->snd_lbb = iss;
        npcb->snd_wl1 = seqno - 1;
        npcb->callback_arg = pcb->callback_arg;
        /* .....此处省略多行代码..... */

        /* 发送 SYN|ACK 和 MSS 选项 */
        rc = tcp_enqueue_flags(npcb, TCP_SYN | TCP_ACK); (2)
        if (rc != ERR_OK) {
            tcp_abandon(npcb, 0);
            return;
        }
        /* 发送连接应答包 */
        tcp_output(npcb); (3)
    }
    return;
}
```


可见，lwIP 内核首先判断连接请求报文的首部标志位是否包含 TCP_SYN，显然这个符合第一次 TCP 握手，然后系统把服务器控制块的状态从 LISTEN 转换成 SYN-RCVD，这个过程请看上述的(1)，其次系统构建连接应答 TCP_ACK| TCP_SYN 报文（上述源码中的(2)），最后系统调用函数 tcp_output 发送该连接应答 TCP_ACK| TCP_SYN 报文到客户端当中（上述源码中的(3)）。至此我们已经实现了 TCP 第二次握手了，下面笔者使用一个示意图来讲解上述的内容，如下图所示。

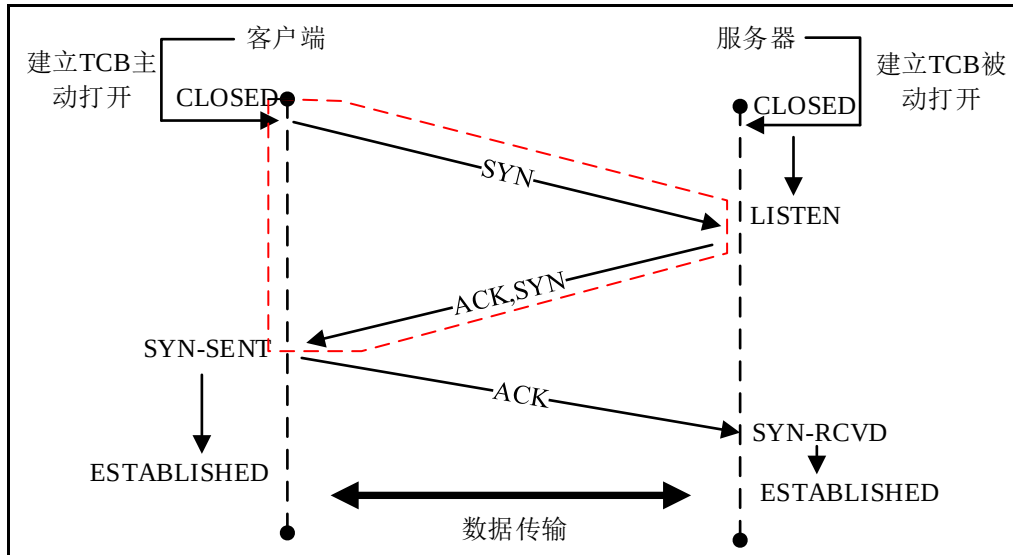


图 12.1.9.5 TCP 第一次和第二次握手

上图的红色框框就是服务器接收客户端的连接请求报文之后发送连接应答报文，到了这里服务器必须接收客户端的确认连接应答 TCP_ACK 报文才能实现 TCP 第三次握手，下面笔者带大家讲解一下最后一次握手，它是在 tcp_input → tcp_process 函数下处理的，该函数如下所示：

```
static err_t
tcp_process(struct tcp_pcb *pcb)
{
    /* .....此处省略多行代码..... */
    case SYN_RCVD:
        if (flags & TCP_ACK) {
            if (TCP_SEQ_BETWEEN(ackno, pcb->lastack + 1, pcb->snd_nxt)) {
                pcb->state = ESTABLISHED;
                /* .....此处省略多行代码..... */
            } else {
                /* .....此处省略多行代码..... */
            }
        } else if ((flags & TCP_SYN) && (seqno == pcb->rcv_nxt - 1)) {
            /* .....此处省略多行代码..... */
        }
        break;
    /* .....此处省略多行代码..... */
}
```

服务器接收到客户端的应答 ACK 报文之后会把自身的状态 SYN-RCVD 转换成 ESTABLISHED，至此客户端和服务端可以相互发送数据了。TCP 客户端和服务端握手流程已经很详细讲解了，如有疑问，请大家联系笔者，我们可以一起讨论研究。

3. TCP 关闭连接原理：

(1) 客户端发送 FIN 报文

程序关闭 TCP 连接是调用 tcp_close 函数实现的，在调用这个函数之前，我们必须把 tcp_pcb 的 recv 回调函数指针设置为 NULL（应用层不再接收数据，所有数据直接被丢弃，协议层的处理仍按正常流程走，认为应用层已经接收到数据），tcp_close 函数主要作用是发送 FIN 报文，进入 FIN_WAIT_1 状态（第一次挥手），下面我们来看一下第一次挥手的源码，注意：以下源码的路径：tcp_close→tcp_close_shutdown→tcp_close_shutdown_fin 函数下，该函数如下所示：

```
static err_t
tcp_close_shutdown_fin(struct tcp_pcb *pcb)
{
    err_t err;

    /* 省略多余的代码行 */

    switch (pcb->state) {
        case SYN_RCVD:
            err = tcp_send_fin(pcb);
            if (err == ERR_OK) {
                tcp_backlog_accepted(pcb);
                MIB2_STATS_INC(mib2.tcpattemptfails);
                pcb->state = FIN_WAIT_1;
            }
            break;
        case ESTABLISHED:
            err = tcp_send_fin(pcb);
            if (err == ERR_OK) {
                MIB2_STATS_INC(mib2.tcpestabresets);
                /* 设置 TCP 控制块的状态为 FIN_WAIT_1 */
                pcb->state = FIN_WAIT_1;
            }
            break;
        case CLOSE_WAIT:
            err = tcp_send_fin(pcb);
            if (err == ERR_OK) {
                MIB2_STATS_INC(mib2.tcpestabresets);
                pcb->state = LAST_ACK;
            }
            break;
        default:
            return ERR_OK;
    }
}
```

```

}

/* 发送关闭连接请求包 */
if (err == ERR_OK) {
    tcp_output(pcb);
} else if (err == ERR_MEM) {
    tcp_set_flags(pcb, TF_CLOSEPEND);
    return ERR_OK;
}

return err;
}

```

大家请看上述有注释的代码，这些代码是客户端发送关闭连接请求报文过程，该包的首部包含 FIN 标志位并调用函数 tcp_output 发送到服务器当中。由此可见，客户端从 ESTABLISHED 状态转换成 FIN-WAIT-1 状态，下面笔者使用一个示意图来描述上述的内容，如下图所示。

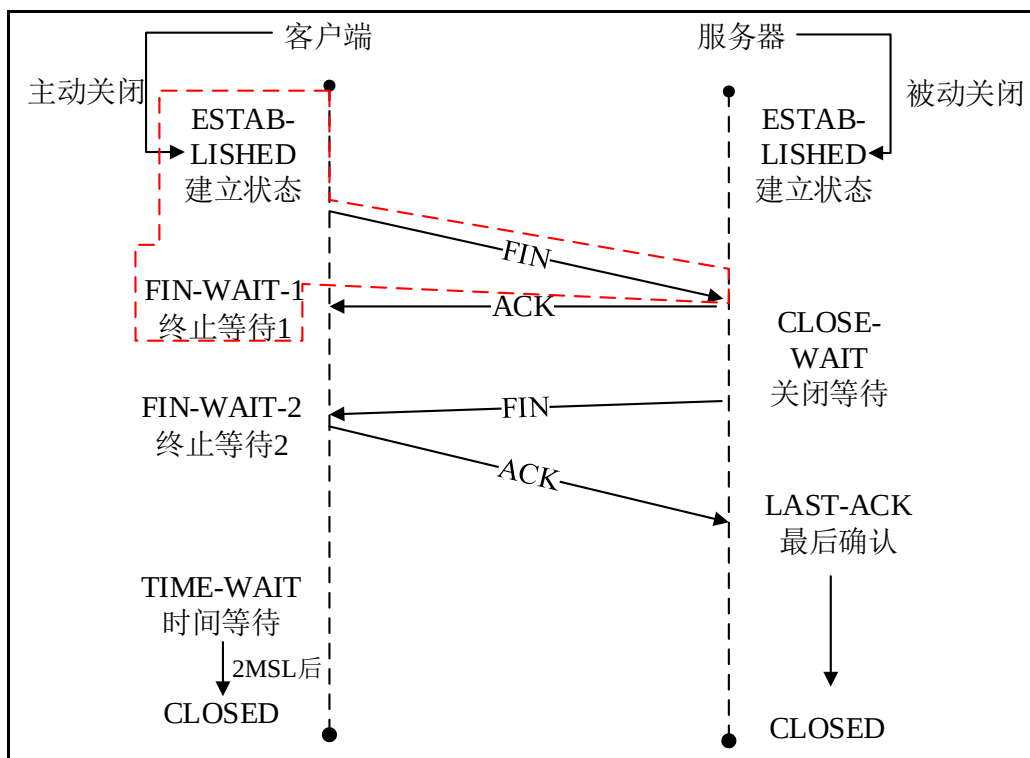


图 12.1.9.6 客户端发送 FIN 报文示意图

上图红色框框表示 tcp_close 函数处理过程，这里也可以称之为 TCP 第一次挥手的动作。

(2) 服务器接收到 FIN 报文并发送 ACK 报文

当服务器接收到客户端的 FIN 报文时，它会进入到 CLOSE_WAIT 状态，这个 FIN 报文交由 tcp_process 函数处理，当然它接收到的数据可以发送给应用层，但是它递交一个空的 EOF 数据给应用层（应用层知道接收数据已经完成，不需要再从协议栈读数据），最后系统发送客户端 ACK 报文给客户端（第二次挥手），进入 CLOSE_WAIT 状态，如下源码所示：

```

static err_t
tcp_process(struct tcp_pcb *pcb)
{

```

```

/* .....省略多行代码..... */
switch (pcb->state) {
    /* .....省略多行代码..... */
    case ESTABLISHED:
        tcp_receive(pcb);
        if (recv_flags & TF_GOT_FIN) { /* 收到 FIN 被动关闭 */
            tcp_ack_now(pcb);          /* 构建 ACK 报文 */
            pcb->state = CLOSE_WAIT;    /* 进入 CLOSE_WAIT 状态 */
        }
        break;
}
/* .....省略多行代码..... */
}

```

上述源码是服务器接收到客户端的 FIN 报文时，它构建了一个 ACK 报文发送到客户端当中，然后它的状态从 ESTABLISHED 转换成 CLOSE-WAIT，下面笔者也是使用一个示意图来描述上述的内容，如下图所示。

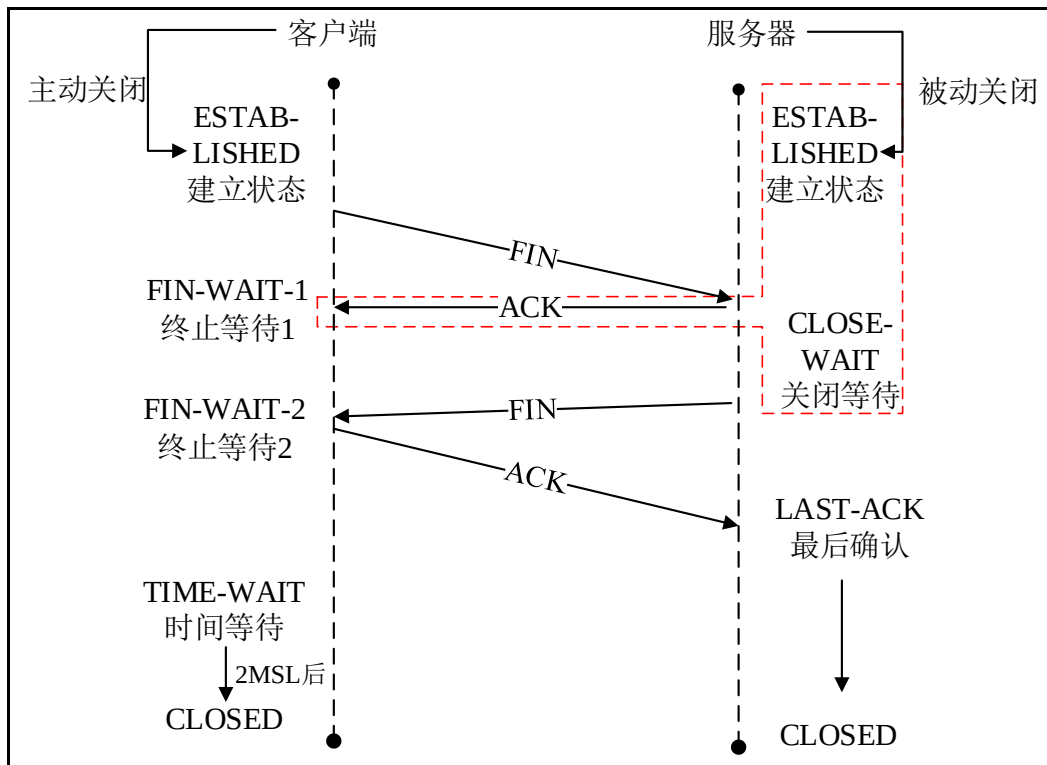


图 12.1.9.7 服务器发送 ACK 报文示意图

上图红色框框就是上述源码运行的流程，为了理解，笔者没有把全部的代码列举出来。

(3) 客户端接收到 ACK 报文并转换成 FIN-WAIT-2 状态

当 FIN_WAIT_1 状态的客户端收到服务器的 ACK 报文时，它的状态从 FIN-WAIT-1 转换成 FIN-WAIT-2 状态，这个过程的源码如下所示：

```

static err_t
tcp_process(struct tcp_pcb *pcb)
{

```

```

/* .....省略多行代码..... */
switch (pcb->state) {
/* .....省略多行代码..... */

case FIN_WAIT_1:
/* 接收数据 */
tcp_receive(pcb);
/* 服务器还没有确认 FIN 报文 */
if (recv_flags & TF_GOT_FIN) {
/* 非同时关闭 */
if ((flags & TCP_ACK) && (ackno == pcb->snd_nxt)) {
/* .....省略多行代码..... */
/* 发送 ACK 应答对端的 FIN 报文 */
tcp_ack_now(pcb);
TCP_RMV(&tcp_active_pcbs, pcb); /* 从 tcp_active_pcbs 删除 tcp_pcb */
/* tcp_timewait_input 处理，所有数据都丢弃，不发送给应用层，
直接确认当前收到的报文，rcv_nxt 设置为当前报文的下一个字节 */
pcb->state = TIME_WAIT;
TCP_REG(&tcp_tw_pcbs, pcb); /* 添加 tcp_pcb 到 tcp_tw_pcbs */
}
else
{ /* (客户端、服务器同时调用 tcp_close，都在 FIN_WAIT_1 状态收到对方的 FIN 报文) */
tcp_ack_now(pcb); /* 发送 FIN 报文的 ACK */
pcb->state = CLOSING; /* 进入 CLOSING 状态 */
}
} /* 服务器确认了 FIN 报文 */
else if ((flags & TCP_ACK) && (ackno == pcb->snd_nxt)) {
pcb->state = FIN_WAIT_2; /* 进入 FIN_WAIT_2 状态 */
}
}
/* .....省略多行代码..... */
}

```

上述源码可分为两个部分讲解，第一部分：处于 FIN_WAIT_1 客户端会判断服务器有没有确认 FIN 报文，如果它没有发送 ACK 报文，则系统进入 if 语句执行，该 if 语句的代码主要为了判断服务器和客户端是否同时调用 tcp_close 函数关闭连接，如果不同时，则将 TCP 控制块从 tcp_active_pcbs 队列移除并设置该控制块的状态为 TIME_WAIT。最后把该控制块挂在 tcp_tw_pcbs 队列当中；如果客户端和服务器同时关闭连接，则系统发送一个 ACK 报文到服务器当中并设置 TCP 控制块的状态为 CLOSING；第二部分：服务器发送 ACK 报文给客户端了，显然它直接设置 TCP 控制块的状态为 FIN_WAIT_2，下面我们使用一个示意图来描述上述的内容，如下图所示。

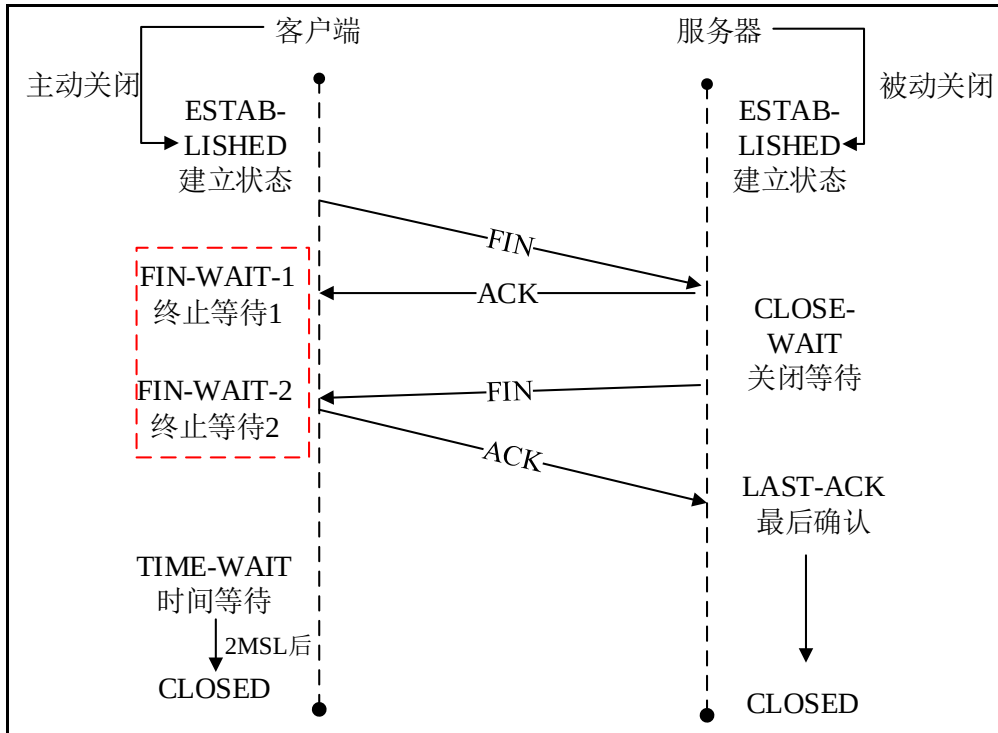


图 12.1.9.8 TCP 客户端接收服务器的 ACK 报文示意图

从上图可知：服务器的状态从 FIN-WAIT-1 转换成 FIN-WAIT-2 状态，FIN-WAIT-2 状态的客户端需要等待服务器发送 FIN 报文。

(4) CLOSE-WAIT 状态的服务器发送 FIN 报文

CLOSE-WAIT 状态的服务器发送 FIN 报文如下源码所示：

```

static err_t
tcp_close_shutdown_fin(struct tcp_pcb *pcb){
    err_t err;
    /* 省略多余的代码行 */
    switch (pcb->state) {
        case SYN_RCVD:
            err = tcp_send_fin(pcb);
            if (err == ERR_OK) {
                tcp_backlog_accepted(pcb);
                MIB2_STATS_INC(mib2.tcpattemptfails);
                pcb->state = FIN_WAIT_1;
            }
            break;
        case ESTABLISHED:
            err = tcp_send_fin(pcb);
            if (err == ERR_OK) {
                MIB2_STATS_INC(mib2.tcpestabresets);
                pcb->state = FIN_WAIT_1;
            }
            break;
        case CLOSE_WAIT:
            /* 发送 FIN 报文 */
    
```

```
err = tcp_send_fin(pcb);
if (err == ERR_OK) {
    MIB2_STATS_INC(mib2.tcpestabresets);
    /* 设置状态为 LAST_ACK */
    pcb->state = LAST_ACK;
}
break;
default:
    return ERR_OK;
}
/* 发送关闭连接请求包 */
if (err == ERR_OK) {
    tcp_output(pcb);
} else if (err == ERR_MEM) {
    tcp_set_flags(pcb, TF_CLOSEPEND);
    return ERR_OK;
}
return err;
}
```

此函数很简单，主要发送 FIN 报文以及设置 CLOSE_WAIT 状态的服务器为 LAST_ACK 状态。下面笔者也是使用一个示意图来描述上述的内容，如下图所示。

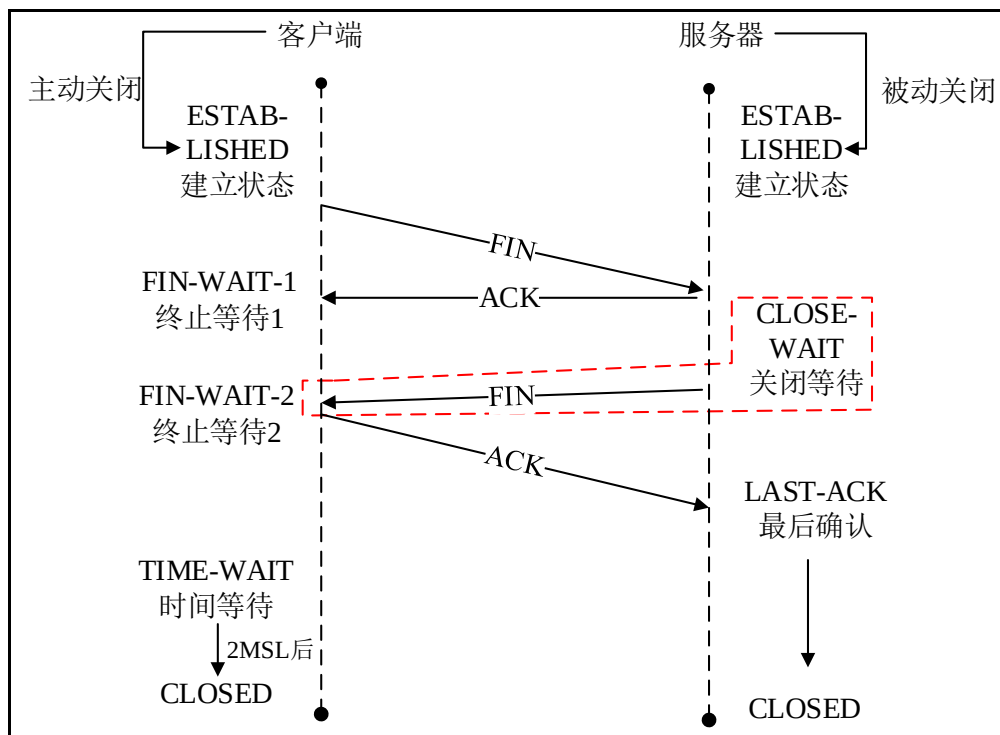


图 12.1.9.10 服务器发送 FIN 报文

这里称之为 TCP 第三次挥手，最后就是 FIN-WAIT-2 状态的客户端接收服务器的 FIN 报文并发送 ACK 报文确认关闭。

(5) FIN-WAIT-2 状态的客户端接收 FIN 报文并发送 ACK 报文确认

这个过程是在 tcp_input→tcp_process 函数下处理，该函数如下所示：

```
static err_t
```

```
tcp_process(struct tcp_pcb *pcb){
    /* .....省略多行代码..... */
    switch (pcb->state) {
        /* .....省略多行代码..... */
        case FIN_WAIT_2:
            /* 接收报文 */
            tcp_receive(pcb);

            if (recv_flags & TF_GOT_FIN) {
                /* 构建ACK 报文 */
                tcp_ack_now(pcb);

                tcp_pcb_purge(pcb);

                TCP_RMV_ACTIVE(pcb);

                /* 设置状态为 TIME_WAIT */
                pcb->state = TIME_WAIT;
                TCP_REG(&tcp_tw_pcb, pcb);
            }
            break;
        /* .....省略多行代码..... */
    }
}
```


lwIP 提供了很多关于 TCP 协议的 RAW 编程 API 函数，我们可以使用这些函数来完成有关 TCP 的实验，我们在下表列出了一部分函数。

函数分组	API 函数	函数功能描述
TCP 连接建立	tcp_new()	创建一个 TCP 的 PCB 控制块
	tcp_bind()	为 TCP 的 PCB 控制块绑定一个本地 IP 地址和端口号
	tcp_listen()	开始 TCP 的 PCB 监听
	tcp_accept()	控制块 accept 字段注册的回调函数，侦听到连接时被调用
	tcp_accepted()	通知 LWIP 协议栈一个 TCP 连接被接受了
	tcp_connect()	连接远端主机
发送 TCP 数据	tcp_write()	构造一个报文并放到控制块的发送缓冲队列中
	tcp_sent()	控制块 sent 字段注册的回调函数，数据发送成功后被回调
	tcp_output()	将发送缓冲队列中的数据发送出去
接收 TCP 数据	tcp_recv()	控制块 recv 字段注册的回调函数，当接收到新数据时被调用
	tcp_recved()	当程序处理完数据后一定要调用这个函数，通知内核更新接收窗口
轮询函数	tcp_poll()	控制块 poll 字段注册的回调函数，该函数周期性调用
关闭和中止连接	tcp_close()	关闭一个 TCP 连接
	tcp_err()	控制块 err 字段注册的回调函数，遇到错误时被调用
	tcp_abort()	中断 TCP 连接

表 12.1.10.2 TCP RAW API 函数

(1) tcp_new 函数

该函数的为创建一个 TCP 控制块，如下表所示：

功能	建立一个新的连接标志控制块
原型	struct tcp_pcb * tcp_new(void)
参数	无
返回值	pcb: 创建成功; NULL:创建失败

表 12.1.10.3 函数 tcp_new()

(2) tcp_bind 函数

该函数为 TCP 的 PCB 控制块绑定一个本地 IP 地址和端口号，如下表所示：

功能	绑定本地 IP 地址和端口号
原型	err_t tcp_bind(struct tcp_pcb *pcb, const ip_addr_t *ipaddr, u16_t port)
参数	pcb:控制块 ipaddr:本地 IP 地址 port:端口号
返回值	ERR_OK: 绑定成功; ERR_USE:绑定失败

表 12.1.10.4 函数 tcp_bind()

(3) tcp_listen 函数

开启 TCP 的 PCB 监听，该函数实际调用函数 tcp_listen_with_backlo 来实现功能，如下表所示：

功能	指定连接进入监听状态
原型	struct tcp_pcb*tcp_listen_with_backlog(struct tcp_pcb *pcb, u8_t backlog)
参数	pcb:控制块 backlog: 用户参数，这里未使用到
返回值	指向处于侦听状态的控制块

表 12.1.10.5 函数 tcp_listen()

(4) tcp_accept 函数

当处于监听连接与一个新来的连接并连接上时，该函数指定的回调函数被调用，该函数通常在 tcp_listen 后面调用，如下表所示：

功能	指定处于监听状态的连接并接通后将要调用的回调函数
原型	void tcp_accept(struct tcp_pcb *pcb, tcp_accept_fn accept)
参数	pcb:处于监听状态的控制块 accept: 连接后回调函数
返回值	无

表 12.1.10.6 函数 tcp_accept()

上述的源码很简单，就是判断是否满足条件，然后使用 tcp_pcb_listen 链表指向该控制块，并设置该链表的回调函数。

(5) tcp_connect 函数

该函数为请求参数 pcb 指定的连接连接到远程主机，并发送打开连接的最初 SYN 段。该函数调用后不会立刻返回，它并不会等待连接，但是一定需要正确连接，如果正确连接，那么它会直接调用第四个参数指定函数（connected 参数），相反，如果连接不正确，可能就是远程主机拒绝连接，或者远程主机不应答，如下表所示：

功能	指定处于监听状态的连接并接通后将要调用的回调函数
原型	err_t tcp_connect(struct tcp_pcb *pcb, const ip_addr_t *ipaddr, u16_t port, tcp_connected_fn connected)
参数	pcb:控制块 ipaddr: 指定连接的远程 IP 地址 port:指定连接远程的端口号 connected:连接成功调用的函数
返回值	ERR_VAL: 无效参数 ERR_OK: 连接成功

表 12.1.10.7 函数 tcp_connect()

(6) tcp_write 函数

该函数的功能是发送 TCP 数据，并不是一经使用就会立刻发送数据，而是指定的数据放到发送队列，由协议内核来决定发送，发送队列中可用字节大小并通过函数 tcp_sendbuf()来获取，使用该函数就是返回字节大小的数据，如果该函数返回值为 ERR_MEM，那么应用程序需要等待一段时间，直到发送队列中的数据被远程主机成功接收，然后才可以发送出去。

该函数可以用来构建 TCP 报文段，在前面我们已经讲解过，如果使用 RAW 编程 API，那么回调函数需要我们自己来注册，如果使用 netconn 和 Socket 编程，那么不需要我们来处理，lwIP 内核已经帮我们注册该函数来构建 TCP 报文段，如下表所示：

功能	发送 TCP 数据
原型	err_t tcp_write(struct tcp_pcb *pcb, const void *dataptr, u16_t len, u8_t apiflags);
参数	pcb:控制块 dataptr: 准备发送的数据

	len:指定数据长度 apiflags: 数据是否进行拷贝, 以及报文段首部是否设置 PSH 标志
返回值	ERR_OK: 发送成功

表 12.1.10.8 函数 tcp_write()

(7) tcp_sent 函数

该函数用于设定远程主机成功接收到数据后才调用的回调函数, 如下表所示:

功能	指定远程主机成功接收数据后才调用的回调函数
原型	void tcp_sent(struct tcp_pcb *pcb, tcp_sent_fn sent)
参数	pcb:控制块 sent: 回调函数
返回值	ERR_OK: 发送成功

表 12.1.10.9 函数 tcp_send()

(8) tcp_recv 函数

该函数当有新的数据接收到时, 调用的回调函数, 如下表所示:

功能	新的数据接收到时, 调用的回调函数
原型	void tcp_recv(struct tcp_pcb *pcb, tcp_recv_fn recv)
参数	pcb:控制块 recv: 回调函数
返回值	无

表 12.1.10.10 函数 tcp_recv()

(9) tcp_close 函数

该函数为关闭指定连接, 调用该函数删除 TCP 控制块, 如下表所示:

功能	关闭一个 TCP 连接
原型	err_t tcp_close(struct tcp_pcb *pcb)
参数	pcb:控制块
返回值	ERR_OK: 正常关闭, ERR_MEM: 没有关闭可用的内存

表 12.1.10.11 函数 tcp_recv()

12.2 RAW 接口的 TCP 实验

12.2.1 硬件设计

1. 例程功能

本章实验的目标是 PC 端和开发板通过 TCP 协议连接起来, 开发板做 TCP 客户端, PC 端的网络调试助手配置成服务器。开发板接收服务器发送的数据在 LCD 上显示, 我们也可以通过开发板上的按键发送数据给 PC。

该实验的实验工程, 请参考《lwIP 例程 3 lwIP_RAW_TCP 客户端实验》。

12.2.2 软件设计

12.2.2.1 TCP 客户端配置步骤

1) 创建 TCP 控制块

调用函数 tcp_new 创建 TCP 控制块。

2) 连接指定的 IP 地址和端口号

调用函数 `tcp_connect` 连接到目的地址的指定端口上。

4) 接收数据

调用函数 `tcp_recved` 接收数据。

5) 发送数据

调用函数 `tcp_write` 发送数据。

12.2.2.2 程序流程图

本实验的程序流程图，如下图所示。

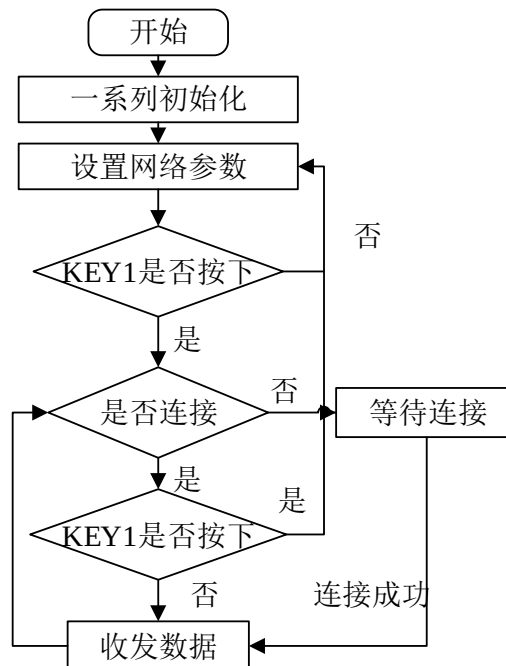


图 12.2.2.1 RAW 编程接口 TCP 客户端流程图

12.2.2.3 程序解析

本章实验只讲解 `lwip_demo.c` 文件，该文件定义了 9 个函数，这些函数的作用如下所示：

TCP 客户端函数	函数说明
<code>lwip_tcp_client_set_remoteip()</code>	设置远端 IP 地址，也就是要连接的服务器 IP 地址
<code>lwip_demo()</code>	TCP 客户端测试程序
<code>lwip_tcp_client_connected()</code>	TCP 连接建立后调用的回调函数
<code>lwip_tcp_client_recv()</code>	接收到数据时的回调函数
<code>lwip_tcp_client_error()</code>	接收错误时的回调函数
<code>lwip_tcp_client_poll()</code>	轮询函数
<code>lwip_tcp_client_sent()</code>	发送回调函数
<code>lwip_tcp_client_senddata()</code>	发送数据
<code>lwip_tcp_client_connection_close()</code>	关闭 TCP 连接

表 12.2.2.3.1 TCP 客户端函数

程序首先执行 `lwip_demo` 函数，此函数为 `lwip_demo.c` 文件的入口处，如下源码所示：

```

/**
 * @brief      lwip_demo 程序入口
 * @param      无

```

```

* @retval      无
*/
void lwip_demo(void)
{
    struct tcp_pcb *tcppcb;      /* 定义一个 TCP 服务器控制块 */
    ip_addr_t rmtipaddr;         /* 远端 ip 地址 */
    char *tbuf;
    uint8_t key;
    uint8_t res = 0;
    uint8_t t = 0;
    uint8_t connflag = 0;        /* 连接标记 */

    lwip_tcp_client_set_remoteip(); /* 先选择 IP */
    lcd_clear(BLACK);             /* 清屏 */
    g_point_color = WHITE;
    lcd_show_string(30, 30, 200, 16, 16, "STM32", g_point_color);
    lcd_show_string(30, 50, 200, 16, 16, "TCP Client Test", g_point_color);
    lcd_show_string(30, 70, 200, 16, 16, "ATOM@正点原子", g_point_color);
    lcd_show_string(30, 90, 200, 16, 16, "KEY0:Send data", g_point_color);
    lcd_show_string(30, 110, 200, 16, 16, "KEY1:Quit", g_point_color);
    tbuf = mymalloc(SRAMIN, 200); /* 申请内存 */

    if (tbuf == NULL) return ;     /* 内存申请失败了,直接退出 */

    sprintf((char *)tbuf, "Local IP:%d.%d.%d.%d", lwipdev.ip[0],
                                                    lwipdev.ip[1],
                                                    lwipdev.ip[2],
                                                    lwipdev.ip[3]); /* 服务器 IP */
    lcd_show_string(30, 130, 210, 16, 16, tbuf, g_point_color);
    /* 远端 IP */
    sprintf((char *)tbuf, "Remote IP:%d.%d.%d.%d", lwipdev.remoteip[0],
                                                    lwipdev.remoteip[1],
                                                    lwipdev.remoteip[2],
                                                    lwipdev.remoteip[3]);

    lcd_show_string(30, 150, 210, 16, 16, tbuf, g_point_color);
    sprintf((char *)tbuf, "Remote Port:%d", TCP_CLIENT_PORT); /* 客户端端口号 */
    lcd_show_string(30, 170, 210, 16, 16, tbuf, g_point_color);
    g_point_color = BLUE;
    lcd_show_string(30, 190, 210, 16, 16, "STATUS:Disconnected", g_point_color);
    tcppcb = tcp_new(); /* 创建一个新的 pcb */

    if (tcppcb)           /* 创建成功 */

```

```

{
    IP4_ADDR(&rmtipaddr, lwipdev.remoteip[0], lwipdev.remoteip[1],
            lwipdev.remoteip[2], lwipdev.remoteip[3]);
    /* 连接到目的地址的指定端口上,当连接成功后回调 tcp_client_connected() 函数 */
    tcp_connect(tcpcb, &rmtipaddr, TCP_CLIENT_PORT,
            lwip_tcp_client_connected);
}

else res = 1;
while (res == 0)
{
    key = key_scan(0);
    if (key == KEY1_PRES)break;
    if (key == KEY0_PRES)                                /* KEY0 按下了,发送数据 */
    {
        lwip_tcp_client_usersent(tcpcb);                /* 发送数据 */
    }
    if (lwip_client_flag & 1 << 6)                      /* 是否收到数据 */
    {
        /* 清上一次数据 */
        lcd_fill(30, 230, lcddev.width - 1, lcddev.height - 1, BLACK);
        /* 显示接收到的数据 */
        lcd_show_string(30, 230, lcddev.width - 30, lcddev.height - 230, 16,
            lwip_client_recvbuf, g_point_color);
        lwip_client_flag &= ~(1 << 6);                  /* 标记数据已经被处理了 */
    }
    if (lwip_client_flag & 1 << 5)                      /* 是否连接上 */
    {
        if (connflag == 0)
        {
            lcd_show_string(30, 190, lcddev.width - 30,
                lcddev.height - 190, 16,
                "STATUS:Connected  ",
                g_point_color);                          /* 提示消息 */
            g_point_color = WHITE;
            lcd_show_string(30, 210, lcddev.width - 30,
                lcddev.height - 190, 16,
                "Receive Data:", g_point_color); /* 提示消息 */
            g_point_color = BLUE;
            connflag = 1; /* 标记连接了 */
        }
    }
    else if (connflag)
    {

```

```

        lcd_show_string(30, 190, 190, 16, 16, "STATUS:Disconnected",
                        g_point_color);
        lcd_fill(30, 210, lcddev.width - 1,
                lcddev.height - 1, BLACK); /* 清屏 */
        connflag = 0; /* 标记连接断开了 */
    }
    lwip_periodic_handle ();
    delay_ms(2);
    t++;
    if (t == 200)
    {
        /* 未连接上,则尝试重连 */
        if (connflag == 0 && (tcp_client_flag & 1 << 5) == 0)
        {
            lwip_tcp_client_connection_close(tcpcb, 0); /* 关闭连接 */
            tcpcb = tcp_new(); /* 创建一个新的 pcb */
            if (tcpcb) /* 创建成功 */
            {
                /* 连接到目的地址的指定端口上,
                当连接成功后回调 tcp_client_connected()函数 */
                tcp_connect(tcpcb, &rmtipaddr, TCP_CLIENT_PORT,
                            tcp_client_connected);
            }
        }
        t = 0;
        LED0_TOGGLE();
    }
}

lwip_tcp_client_connection_close(tcpcb, 0); /* 关闭 TCP Client 连接 */
myfree(SRAMIN, tbuf);
}

```

可见，此函数和 UDP 实验一样，根据开发板上的 KEY0 和 KEY1 设置远程 IP 地址，接着调用 RAW 接口函数配置 TCP 客户端，配置完成之后连接服务器。

设置远程 IP 地址的函数 `lwip_tcp_client_set_remoteip`，如下源码所示：

```

/**
 * @brief      设置远端 IP 地址
 * @param      无
 * @retval     无
 */
void lwip_tcp_client_set_remoteip(void)
{
    char *tbuf;
    uint16_t xoff;

```

```
uint8_t key;
lcd_clear(BLACK);
g_point_color = WHITE;
lcd_show_string(30, 30, 200, 16, 16, "STM32", g_point_color);
lcd_show_string(30, 50, 200, 16, 16, "TCP Client Test", g_point_color);
lcd_show_string(30, 70, 200, 16, 16, "Remote IP Set", g_point_color);
lcd_show_string(30, 90, 200, 16, 16, "KEY0:+ KEY2:-", g_point_color);
lcd_show_string(30, 110, 200, 16, 16, "KEY_UP:OK", g_point_color);
tbuf = mymalloc(SRAMIN, 100); /* 申请内存 */

if (tbuf == NULL) return;

/* 前三个 IP 保持和 DHCP 得到的 IP 一致 */
lwipdev.remoteip[0] = lwipdev.ip[0];
lwipdev.remoteip[1] = lwipdev.ip[1];
lwipdev.remoteip[2] = lwipdev.ip[2];
/* 远端 IP */
sprintf((char *)tbuf, "Remote IP:%d.%d.%d.", lwipdev.remoteip[0],
                                              lwipdev.remoteip[1],
                                              lwipdev.remoteip[2]);

lcd_show_string(30, 150, 210, 16, 16, tbuf, g_point_color);
g_point_color = BLUE;
xoff = strlen((char *)tbuf) * 8 + 30;
lcd_show_xnum(xoff, 150, lwipdev.remoteip[3], 3, 16, 0, g_point_color);

while (1)
{
    key = key_scan(0);

    if (key == KEY1_PRES) break;
    else if (key)
    {
        if (key == KEY0_PRES) lwipdev.remoteip[3]++; /* IP 增加 */

        if (key == KEY2_PRES) lwipdev.remoteip[3]--; /* IP 减少 */
        /* 显示新 IP */
        lcd_show_xnum(xoff, 150, lwipdev.remoteip[3], 3, 16, 0x80,
                      g_point_color);
    }
}

myfree(SRAMIN, tbuf);
}
```

此函数根据开发板上的按键设置远程 IP 地址，设置完成之后按下 KEY1 退出设置。

TCP 连接建立后的回调函数 `lwip_tcp_client_connected`，如下源码所示：

```
/**
 * @brief      lwIP TCP 连接建立后回调函数
 * @param      arg    : 回调函数传入的参数
 * @param      tpcb   : TCP 控制块
 * @param      err    : 错误码
 * @retval     返回错误码
 */
err_t lwip_tcp_client_connected(void *arg, struct tcp_pcb *tpcb, err_t err)
{
    struct tcp_client_struct *es = NULL;
    if (err == ERR_OK)
    {
        es = (struct tcp_client_struct *)mem_malloc(sizeof(struct
                                                    tcp_client_struct)); /* 申请内存 */

        if (es) /* 内存申请成功 */
        {
            es->state = ES_TCPCLIENT_CONNECTED; /* 状态为连接成功 */
            es->pcb = tpcb;
            es->p = NULL;
            tcp_arg(tpcb, es); /* 使用 es 更新 tpcb 的 callback_arg */
            /* 初始化 LwIP 的 tcp_recv 回调功能 */
            tcp_recv(tpcb, lwip_tcp_client_recv);
            tcp_err(tpcb, lwip_tcp_client_error); /* 初始化 tcp_err() 回调函数 */
            /* 初始化 LwIP 的 tcp_sent 回调功能 */
            tcp_sent(tpcb, lwip_tcp_client_sent);
            /* 初始化 LwIP 的 tcp_poll 回调功能 */
            tcp_poll(tpcb, lwip_tcp_client_poll, 1);
            tcp_client_flag |= 1 << 5; /* 标记连接到服务器了 */
            err = ERR_OK;
        }
        else
        {
            lwip_tcp_client_connection_close(tpcb, es); /* 关闭连接 */
            err = ERR_MEM; /* 返回内存分配错误 */
        }
    }
    else{
        lwip_tcp_client_connection_close(tpcb, 0); /* 关闭连接 */
    }
    return err;
}
```

这个回调函数由用户编写，由 `tcp_connect` 函数注册此函数。简单来讲，就是让 TCP 控制

块内的函数指针指向该函数。

lwip_tcp_client_recv 函数是当接收到数据时的回调函数，在这个函数中我们根据不同的状态有不同的处理，这里最重要的就是当处于连接状态并且接收到数据时的处理，这个时候我们将遍历完接收数据的 pbuf 链表，将链表中的所有数据拷贝到 lwip_tcp_client_recvbuf 中，这个过程和 UDP 的接收处理过程相似。数据接收成功以后我们将 lwip_client_flag 的 bit5 置 1，表示接收到数据，lwip_tcp_client_recv 函数代码如下。

```
/**
 * @brief      lwIP tcp_recv() 函数的回调函数
 * @param      arg    : 回调函数传入的参数
 * @param      tpcb   : TCP 控制块
 * @param      p      : 网络数据包
 * @param      err    : 错误码
 * @retval     返回错误码
 */
err_t lwip_tcp_client_recv(void *arg, struct tcp_pcb *tpcb,
                           struct pbuf *p, err_t err)
{
    uint32_t data_len = 0;
    struct pbuf *q;
    struct tcp_client_struct *es;
    err_t ret_err;
    LWIP_ASSERT("arg != NULL", arg != NULL);
    es = (struct tcp_client_struct *)arg;

    if (p == NULL) /* 如果从服务器接收到空的数据帧就关闭连接 */
    {
        es->state = ES_TCPCLIENT_CLOSING; /* 需要关闭 TCP 连接了 */
        es->p = p;
        ret_err = ERR_OK;
    }
    else if (err != ERR_OK) /* 当接收到一个非空的数据帧,但是 err!=ERR_OK */
    {
        if (p) pbuf_free(p); /* 释放接收 pbuf */

        ret_err = err;
    }
    else if (es->state == ES_TCPCLIENT_CONNECTED) /* 当处于连接状态时 */
    {
        if (p != NULL) /* 当处于连接状态并且接收到的数据不为空时 */
        {
            /* 数据接收缓冲区清零 */
            memset(lwip_client_recvbuf, 0, TCP_CLIENT_RX_BUFSIZE);
        }
    }
}
```

```

for (q = p; q != NULL; q = q->next) /* 遍历完整个 pbuf 链表 */
{
    /* 判断要拷贝到 TCP_CLIENT_RX_BUFSIZE 中的数据是否大于
    TCP_CLIENT_RX_BUFSIZE 的剩余空间, 如果大于 */
    /* 的话就只拷贝 TCP_CLIENT_RX_BUFSIZE 中剩余长度的数据,
    否则的话就拷贝所有的数据 */
    if (q->len > (LWIP_DEMO_RX_BUFSIZE - data_len))
        memcpy(lwip_client_recvbuf + data_len, q->payload,
                (LWIP_DEMO_RX_BUFSIZE - data_len)); /* 拷贝数据 */
    else memcpy(lwip_client_recvbuf + data_len, q->payload, q->len);

    data_len += q->len;
    /* 超出 TCP 客户端接收数组, 跳出 */
    if (data_len > LWIP_DEMO_RX_BUFSIZE) break;
}

tcp_client_flag |= 1 << 6; /* 标记接收到数据了 */
/* 用于获取接收数据, 通知 LWIP 可以获取更多数据 */
tcp_recved(tpcb, p->tot_len);
pbuf_free(p); /* 释放内存 */
ret_err = ERR_OK;
}
}

else /* 接收到数据但是连接已经关闭 */
{
    /* 用于获取接收数据, 通知 LWIP 可以获取更多数据 */
    tcp_recved(tpcb, p->tot_len);
    es->p = NULL;
    pbuf_free(p); /* 释放内存 */
    ret_err = ERR_OK;
}

return ret_err;
}

```

lwip_tcp_client_error 函数是控制块中 `errf` 字段的回调函数, 当出现知名错误的时候就会被调用, 这里我们没有实现这个函数, 用户可以根据自己的实际情况来实现这个函数。

lwip_tcp_client_poll 函数为控制块中 `poll` 字段的回调函数, 这个函数会被周期调用, 因此在这个函数中我们可以将要发送的数据发送出去。通过 `lwip_client_flag` 的 `bit7` 来判断是否有数据要发送, 因为 lwIP 中处理数据用的是 `pbuf` 结构体组成的链表, 因此如果有数据要发送的话就将发送缓冲区 `lwip_tcp_client_sendbuf` 中的待发送数据放进 `pbuf` 链表中, 这个我们使用 `pbuf_take` 来实现这个过程, 然后我们调用 `lwip_tcp_client_senddata` 函数将数据发送出去, 发送完成以后记得将 `lwip_client_flag` 的 `bit7` 清零, 如下源码所示:

```

/**
 * @brief      lwIP tcp_poll 的回调函数
 * @param      arg : 回调函数传入的参数

```

```

* @param      tcpb: TCP 控制块
* @retval      ERR_OK
*/
err_t lwip_tcp_client_poll(void *arg, struct tcp_pcb *tcpb)
{
    err_t ret_err;
    struct tcp_client_struct *es;
    es = (struct tcp_client_struct *)arg;

    if (es->state == ES_TCPCLIENT_CLOSING)          /* 连接断开 */
    {
        lwip_tcp_client_connection_close(tcpb, es); /* 关闭 TCP 连接 */
    }

    ret_err = ERR_OK;
    return ret_err;
}

```

lwip_tcp_client_sent 函数为控制块中的 sent 字段的回调函数，这个函数中主要调用了我们下面要讲的 lwip_tcp_client_senddata 这个函数，lwip_tcp_client_sent 函数源码如下。

```

/**
* @brief      lwIP tcp_sent 的回调函数(当从远端主机接收到 ACK 信号后发送数据)
* @param      arg : 回调函数传入的参数
* @param      tcpb: TCP 控制块
* @param      len : 长度
* @retval      ERR_OK
*/
err_t lwip_tcp_client_sent(void *arg, struct tcp_pcb *tcpb, u16_t len)
{
    struct tcp_client_struct *es;
    es = (struct tcp_client_struct *)arg;

    if (es->p)lwip_tcp_client_senddata(tcpb, es); /* 发送数据 */

    return ERR_OK;
}

```

lwip_tcp_client_senddata 函数用来发送数据，在这个函数中我们使用 tcp_write 函数将要发送的数据加入到发送缓冲队列中，最后调用 tcp_output 函数将发送缓冲队列中的数据发送出去，这个函数的代码如下。

```

/**
* @brief      用来发送数据
* @param      tcpb: TCP 控制块
* @param      es  : LWIP 回调函数使用的结构体
* @retval      无

```

```

*/
void lwip_tcp_client_senddata(struct tcp_pcb *tpcb,
                             struct tcp_client_struct *es)
{
    struct pbuf *ptr;
    err_t wr_err = ERR_OK;
    /* 将要发送的数据加入到发送缓冲队列中 */
    while ((wr_err == ERR_OK) && es->p && (es->p->len <= tcp_sndbuf(tpcb)))
    {
        ptr = es->p;
        wr_err = tcp_write(tpcb, ptr->payload, ptr->len, 1);
        if (wr_err == ERR_OK)
        {
            es->p = ptr->next;          /* 指向下一个 pbuf */
            if (es->p) pbuf_ref(es->p); /* pbuf 的 ref 加一 */
            pbuf_free(ptr);            /* 释放 ptr */
        }
        else if (wr_err == ERR_MEM) es->p = ptr;
        tcp_output(tpcb);             /* 将发送缓冲队列中的数据立即发送出去 */
    }
}

```

lwip_tcp_client_connection_close 函数的功能是关闭与服务器的连接，通过调用 tcp_abort 函数来关闭与服务器的连接，然后注销掉控制块中的回调函数，将 lwip_client_flag 的 bit5 置 1，标记连接断开，lwip_tcp_client_connection_close 函数源码如下。

```

/**
 * @brief      关闭与服务器的连接
 * @param      tpcb: TCP 控制块
 * @param      es : LWIP 回调函数使用的结构体
 * @retval     无
 */
void lwip_tcp_client_connection_close(struct tcp_pcb *tpcb,
                                      struct tcp_client_struct *es)
{
    /* 移除回调 */
    tcp_abort(tpcb); /* 终止连接, 删除 pcb 控制块 */
    tcp_arg(tpcb, NULL);
    tcp_recv(tpcb, NULL);
    tcp_sent(tpcb, NULL);
    tcp_err(tpcb, NULL);
    tcp_poll(tpcb, NULL, 0);
    if (es) mem_free(es);
    tcp_client_flag &= ~(1 << 5); /* 标记连接断开了 */
    lcd_show_string(30, 30, 200, 16, 16, "STM32", g_point_color);
}

```

```

lcd_show_string(30, 50, 200, 16, 16, "TCPclient Test", g_point_color);
lcd_show_string(30, 70, 200, 16, 16, "ATOM@正点原子", g_point_color);
lcd_show_string(30, 90, 200, 16, 16, "KEY1:Connect", g_point_color);
lcd_show_string(30, 190, 210, 16, 16, "STATUS:Disconnected", g_point_color);
}

```

至此，lwip_demo.c 文件就讲完了，接下来就是编写 main 函数，main 函数基本和 UDP 实验的相同。

12.2.3 下载验证

代码编译成功之后下载代码到开发板中。打开网络调试助手软件设置为如下图的信息。



图 12.3.3.1 网络调试助手设置

开发板上电，等待出现 12.3.3.2 所示画面，我们设置远端 IP 地址为电脑的 IP 地址，也就是图 12.3.1 中的本地 IP 地址，设置好以后按 KEY_UP 键确认，确认后进入图 12.2.3.3 所示界面，当 STATUS 为 Connected 的时候就可以和网络调试助手互相发送数据了。

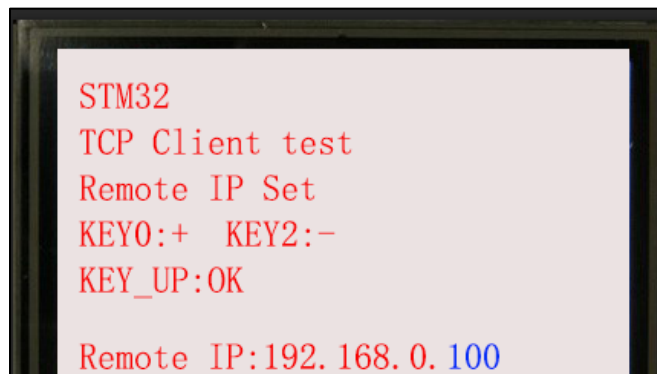


图 12.2.3.2 设置服务器 IP 地址

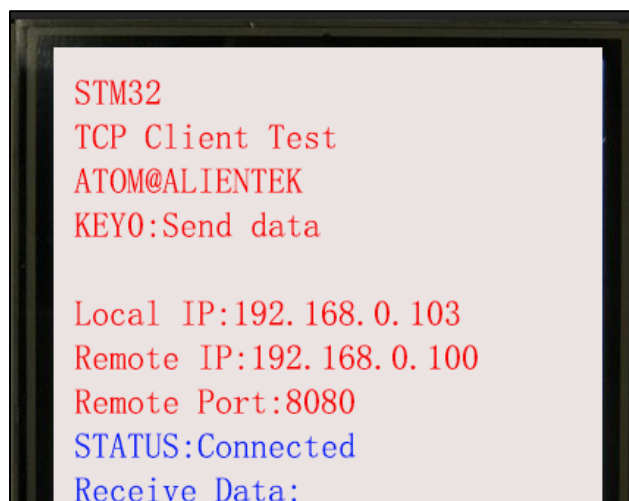


图 12.2.3.3 连接到服务器

我们通过网络调试助手向开发板发送：<http://www.openedv.com>，此时开发板 LCD 上显示接收到的数据如图 12.2.3.4 所示，按下 KEY0 键向网络调试助手发送数据。

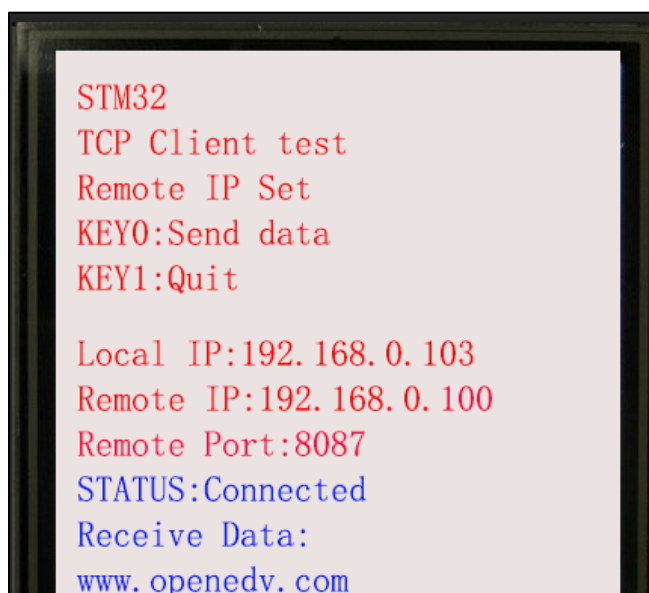


图 12.2.3.4 LCD 显示接收到的数据

第十三章 RAW 编程接口 TCP 服务器实验

在本章中开发板做 TCP 服务器，网络调试助手做 TCP 客户端，实验中我们通过电脑端的网络调试助手给开发板发送数据，开发板接收并在 LCD 上显示接收到的数据，同时也可以通过按键从开发板向网络调试助手发送数据。本章分为如下几个部分：

13.1 RAW 编程接口 TCP 简介

13.2 RAW 接口的 TCP 实验

13.1 RAW 编程接口 TCP 简介

在上一章 RAW 编程接口的 TCP 客户端实验中我们已经讲解过了 TCP 的基础知识，这里就不做讲解。

13.2 RAW 接口的 TCP 实验

13.2.1 硬件设计

1. 例程功能

本章实验的目标是 PC 端和开发板通过 TCP 协议连接起来，开发板做 TCP 服务器，PC 端的网络调试助手配置成客户端。网络调试助手连接到开发板服务器时，网络调试助手可向开发板发送数据并且在 LCD 上显示，我们也可以通过开发板上的按键发送数据给 PC。

该实验的实验工程，请参考《lwIP 例程 4 lwIP_RAW_TCP 服务器实验》。

13.2.2 软件设计

13.2.2.1 TCP 服务器配置步骤

1) 创建 TCP 控制块

调用函数 `tcp_new` 创建 TCP 控制块。

2) 绑定本地 IP 地址和端口号

调用函数 `tcp_bind` 绑定本地 IP 地址和端口号。

3) 连接请求

调用函数 `tcp_accept` 等待连接。

4) 接收数据

调用函数 `tcp_recved` 接收数据。

5) 发送数据

调用函数 `tcp_write` 发送数据。

13.2.2.2 程序流程图

本实验的程序流程图，如下图所示。

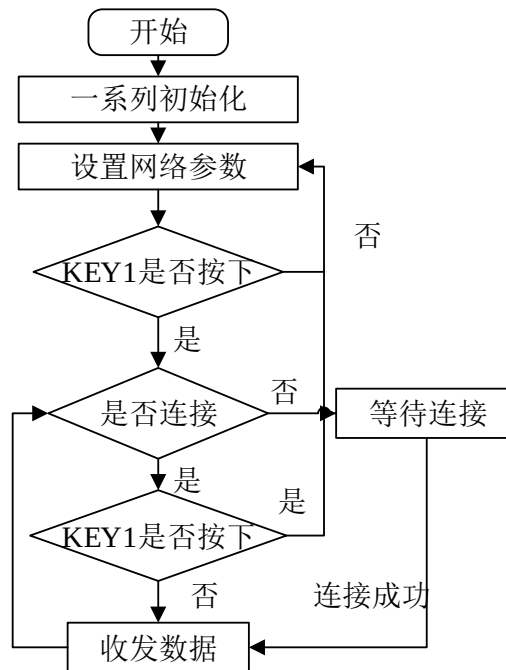


图 14.3.2.1 RAW 编程 TCP 服务器实验流程图

13.2.2.4 程序解析

上一章中我们简单的介绍了几个 lwIP 中关于 TCP 的函数，本节中我们就用这几个函数编写我们本章的例程，本章实验的目标是 PC 端和开发板通过 TCP 协议连接起来，开发板做 TCP 服务器，PC 端的网络调试助手配置成客户端。网络调试助手连接到开发板服务器，网络调试助手向开发板发送数据并在 LCD 上显示接收到的数据，我们也可以通过开发板上的按键发送数据给 PC。本章实验中我们主要有两个文件 lwip_demo.c 和 lwip_demo.h。lwip_demo.h 文件很简单，这里就不讲解，我们重点讲解一下 lwip_demo.c 这个文件，在 tcp_server_demo.c 文件中我们一共定义了 9 个函数，如下表所示。

TCP 客户端函数	函数说明
lwip_demo()	TCP 服务器测试程序
lwip_tcp_server_accept()	控制块 accept 字段的回调函数
lwip_tcp_server_recv()	控制块 recv 字段的回调函数
lwip_tcp_server_error()	控制块 errf 字段的回调函数
lwip_tcp_server_poll()	控制块 poll 字段的回调函数
lwip_tcp_server_sent()	控制块 sent 字段的回调函数
lwip_tcp_server_senddata()	发送数据
lwip_tcp_server_connection_close()	关闭 TCP 连接
lwip_tcp_server_remove_timewait()	删除处于等待状态的 pcb 控制块

表 13.2.2.4.1 TCP 服务器函数

lwip_tcp_server_accept 函数为控制块 accpet 字段的回调函数，当一个侦听和其他主机连接上以后调用，在这个函数中我们主要是为控制块的相应字段注册回调函数，函数的代码如下。

```

/**
 * @brief      lwIP tcp_accept() 的回调函数
 * @param      arg : 传入的参数
 * @param      newpcb: TCP 控制块

```

```

* @param      err : 错误码
* @retval      返回 ret_err
*/
err_t lwip_tcp_server_accept(void *arg, struct tcp_pcb *newpcb, err_t err)
{
    err_t ret_err;
    struct tcp_server_struct *es;
    LWIP_UNUSED_ARG(arg);
    LWIP_UNUSED_ARG(err);
    tcp_setprio(newpcb, TCP_PRIO_MIN); /* 设置新创建的 pcb 优先级 */
    /* 分配内存 */
    es = (struct tcp_server_struct *)mem_malloc(
                                                sizeof(struct tcp_server_struct));

    if (es != NULL) /* 内存分配成功 */
    {
        es->state = ES_TCPSEVER_ACCEPTED; /* 接收连接 */
        es->pcb = newpcb;
        es->p = NULL;

        tcp_arg(newpcb, es);
        tcp_recv(newpcb, lwip_tcp_server_recv); /* 初始化 tcp_recv() 的回调函数 */
        tcp_err(newpcb, lwip_tcp_server_error); /* 初始化 tcp_err() 回调函数 */
        tcp_poll(newpcb, lwip_tcp_server_poll, 1); /* 初始化 tcp_poll 回调函数 */
        tcp_sent(newpcb, lwip_tcp_server_sent); /* 初始化发送回调函数 */

        lwip_send_flag |= 1 << 5; /* 标记有客户端连上了 */
        lwipdev.remoteip[0] = newpcb->remote_ip.addr & 0xff;
        lwipdev.remoteip[1] = (newpcb->remote_ip.addr >> 8) & 0xff;
        lwipdev.remoteip[2] = (newpcb->remote_ip.addr >> 16) & 0xff;
        lwipdev.remoteip[3] = (newpcb->remote_ip.addr >> 24) & 0xff;
        ret_err = ERR_OK;
    }
    else ret_err = ERR_MEM;

    return ret_err;
}

```

lwip_tcp_server_recv 函数和上一章 TCP 客户端实验的 lwip_tcp_client_recv 函数功能基本差不多，大家可以对照这两个函数的源码看一下，这里就不做讲解了。

lwip_tcp_server_error 为当出现重大错误的时候的回调函数，在 TCP 客户端实验中我们没有实现这个函数，在本章实验中我们将这个函数的参数通过串口打印出来，当然也可以根据自己的实际情况来实现这个函数。

lwip_tcp_server_poll, lwip_tcp_server_sent 和 lwip_tcp_server_senddata 这三个函数分别和

上一章 TCP 客户端实验中的 `lwip_tcp_client_poll`, `lwip_tcp_client_sent` 和 `lwip_tcp_client_senddata` 函数功能类似, 大家可以参考 TCP 客户端实验中关于这三个函数的讲解。

`lwip_tcp_server_connection_close` 函数用来关闭 TCP 连接, 我们通过调用 `tcp_close` 函数来关闭连接, 注意这里和 TCP 客户端实验的不同, 然后就是注销掉控制块中的回调函数, 最后释放内存, 清零 `lwip_send_flag` 的 bit5, 标记连接断开, 函数代码如下。

```
/**
 * @brief      关闭 tcp 连接
 * @param      tpcb : TCP 控制块
 * @param      es   : LWIP 回调函数使用的结构体
 * @retval     无
 */
void
tcp_server_connection_close(struct tcp_pcb *tpcb, struct tcp_server_struct *es)
{
    tcp_close(tpcb);
    tcp_arg(tpcb, NULL);
    tcp_sent(tpcb, NULL);
    tcp_recv(tpcb, NULL);
    tcp_err(tpcb, NULL);
    tcp_poll(tpcb, NULL, 0);

    if (es) mem_free(es);

    lwip_send_flag &= ~(1 << 5); /* 标记连接断开了 */
    lcd_show_string(30, 30, 200, 16, 16, "STM32", g_point_color);
    lcd_show_string(30, 50, 200, 16, 16, "TCP Server Test", g_point_color);
    lcd_show_string(30, 70, 200, 16, 16, "ATOM@正点原子", g_point_color);
    lcd_show_string(30, 90, 200, 16, 16, "KEY1:Connect", g_point_color);
    lcd_show_string(30, 190, 210, 16, 16, "STATUS:Disconnected", g_point_color);
}
```

最后一个函数是 `lwip_tcp_server_remove_timewait`, 这个函数用来强制删除处于 TIME-WAIT 状态的控制块, 函数代码如下。

```
/**
 * @brief      强制删除 TCP Server 主动断开时的 time wait
 * @param      无
 * @retval     无
 */
void lwip_tcp_server_remove_timewait(void)
{
    struct tcp_pcb *pcb, *pcb2;
    uint8_t t = 0;

    while (tcp_active_pcbs != NULL && t < 200)
```

```

{
    lwip_periodic_handle();    /* 继续轮询 */
    t++;
    delay_ms(10);             /* 等待 tcp_active_pcbs 为空 */
}

pcb = tcp_tw_pcbs;

while (pcb != NULL)          /* 如果有等待状态的 pcbs */
{
    tcp_pcb_purge(pcb);
    tcp_tw_pcbs = pcb->next;
    pcb2 = pcb;
    pcb = pcb->next;
    memp_free(MEMP_TCP_PCB, pcb2);
}
}

```

接下来是本实验最重要的函数 `lwip_demo`，同 UDP 实验和 TCP 客户端实验一样，这个函数一开始也是显示一些提示信息，不过不同的是和本实验不用设置需要连接的远端主机的 IP 地址，因为服务器是等待其他主机来连接的。显示完提示信息以后就是本函数的重点了，代码如下。

```

/**
 * @brief      lwip_demo 测试
 * @param      无
 * @retval     无
 */
void lwip_demo(void)
{
    err_t err;
    struct tcp_pcb *tcppcbnew;    /* 定义一个 TCP 服务器控制块 */
    struct tcp_pcb *tcppcbconn;  /* 定义一个 TCP 服务器控制块 */

    char *tbuf;
    uint8_t key;
    uint8_t res = 0;
    uint8_t t = 0;
    uint8_t connflag = 0;        /* 连接标记 */

    lcd_clear(BLACK);           /* 清屏 */
    g_point_color = WHITE;
    lcd_show_string(30, 30, 200, 16, 16, "STM32", g_point_color);
    lcd_show_string(30, 50, 200, 16, 16, "TCP Server Test", g_point_color);
    lcd_show_string(30, 70, 200, 16, 16, "ATOM@正点原子", g_point_color);

```

```

lcd_show_string(30, 90, 200, 16, 16, "KEY0:Send data", g_point_color);
lcd_show_string(30, 110, 200, 16, 16, "KEY_UP:Quit", g_point_color);
tbuf = mymalloc(SRAMIN, 200); /* 申请内存 */

if (tbuf == NULL) return ;      /* 内存申请失败了,直接退出 */

sprintf((char *)tbuf, "Server IP:%d.%d.%d.%d",      lwipdev.ip[0],
                                                    lwipdev.ip[1],
                                                    lwipdev.ip[2],
                                                    lwipdev.ip[3]);

lcd_show_string(30, 130, 210, 16, 16, tbuf, g_point_color);
sprintf((char *)tbuf, "Server Port:%d", LWIP_DEMO_PORT); /* 服务器端口号 */
lcd_show_string(30, 150, 210, 16, 16, tbuf, g_point_color);
tcppcbnew = tcp_new();          /* 创建一个新的 pcb */

if (tcppcbnew)                  /* 创建成功 */
{
    /* 将本地 IP 与指定的端口号绑定在一起,IP_ADDR_ANY 为绑定本地所有的 IP 地址 */
    err = tcp_bind(tcppcbnew, IP_ADDR_ANY, LWIP_DEMO_PORT);

    if (err == ERR_OK) /* 绑定完成 */
    {
        tcppcbconn = tcp_listen(tcppcbnew); /* 设置 tcppcb 进入监听状态 */
        /* 初始化 LWIP 的 tcp_accept 的回调函数 */
        tcp_accept(tcppcbconn, tcp_server_accept);
    }
    else res = 1;
}
else res = 1;

g_point_color = BLUE;

while (res == 0)
{
    key = key_scan(0);

    if (key == KEY1_PRES) break;

    if (key == KEY0_PRES) /* KEY0 按下了,发送数据 */
    {
        lwip_tcp_server_usersent(tcppcbnew); /* 发送数据 */
    }

    if (lwip_send_flag & 1 << 6) /* 是否收到数据 */

```

```

{
    /* 清上一次数据 */
    lcd_fill(30, 210, lcddev.width - 1, lcddev.height - 1, BLACK);
    /* 显示接收到的数据 */
    lcd_show_string(30, 210, lcddev.width - 30, lcddev.height - 210, 16,
                    (char *)lwip_demo_recvbuf, g_point_color);
    lwip_send_flag &= ~(1 << 6); /* 标记数据已经被处理了 */
}
if (lwip_send_flag & 1 << 5) /* 是否连接上 */
{
    if (connflag == 0)
    {
        /* 客户端 IP */
        sprintf((char *)tbuf,
                "ClientIP:%d.%d.%d.%d", lwipdev.remoteip[0],
                                     lwipdev.remoteip[1],
                                     lwipdev.remoteip[2],
                                     lwipdev.remoteip[3]);
        lcd_show_string(30, 170, 230, 16, 16, tbuf, g_point_color);
        g_point_color = WHITE;
        connflag = 1; /* 标记连接了 */
    }
}
else if (connflag)
{
    lcd_fill(30, 170, lcddev.width - 1, lcddev.height - 1, BLACK);
    connflag = 0; /* 标记连接断开了 */
}
lwip_periodic_handle();
delay_ms(2);
t++;
if (t == 200)
{
    t = 0;
    LED0_TOGGLE();
}
}

lwip_tcp_server_connection_close(tcpcbnew, 0); /* 关闭 TCP Server 连接 */
lwip_tcp_server_connection_close(tcpcbconn, 0); /* 关闭 TCP Server 连接 */
lwip_tcp_server_remove_timewait();
memset(tcpcbnew, 0, sizeof(struct tcp_pcb));
memset(tcpcbconn, 0, sizeof(struct tcp_pcb));

```

```
myfree(SRAMIN, tbuf);
}
```

在上面代码中主要完成一下几个功能。

(1) 通过调用 `tcp_new` 函数创建一个 `tcp` 控制块 `tcppcbnew`，这个控制块用来进行监听，如果未创建成功的话就令 `res` 等于 1。

(2) 当控制块 `tcppcbnew` 创建成功以后就将其绑定到指定的 IP 地址和端口好上，绑定成功以后将控制块设置为监听状态，并且注册控制块 `accept` 字段的回调函数，如果绑定未成功的话就让 `res` 等于 1。

(3) 当 `res` 等于 0 的话就进入 `while` 循环，在 `while` 循环的处理过程基本和 TCP 客户端的一样。

(4) 当从 `while` 循环退出来后，我们就关闭 TCP 连接，这里我们要关闭两个：`tcppcbnew` 和 `tcppcbconn`，最后还要调用 `lwip_tcp_server_remove_timewait` 函数将处于 TIME—WAIT 状态的 `pcb` 控制块删除。

到这里 `lwip_demo.c` 文件中的函数已经讲完。`mian` 函数就不必讲解了，代码很简单。

13.2.3 下载验证

在代码编译成功以后，我们下载代码到开发板中，通过网线连接开发板到路由器上，如果没有路由器的话就连接到电脑端的 RJ45 上，电脑端还要进行设置，设置过程很简单。开发板上电，等待出现 13.2.3.1 所示画面，打开网络调试助手按图 13.2.3.2 所示设置好以后点击“连接”按钮。

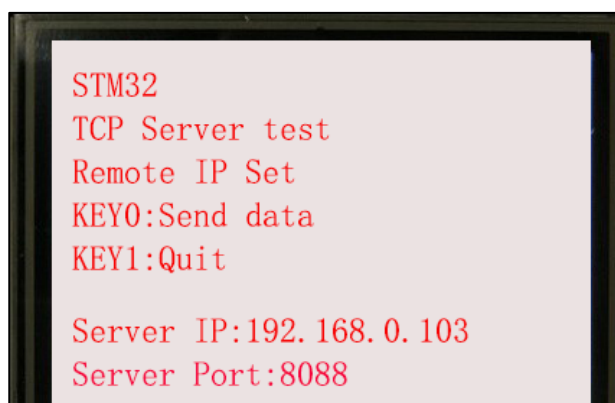


图 13.2.3.1 开机 LCD 显示画面



图 13.2.3.2 网络调试助手设置

当网络调试助手连接上开发板以后，开发板 LCD 上显示如图 13.2.3.3 所示，表明网络调试助手已经连接上服务器。

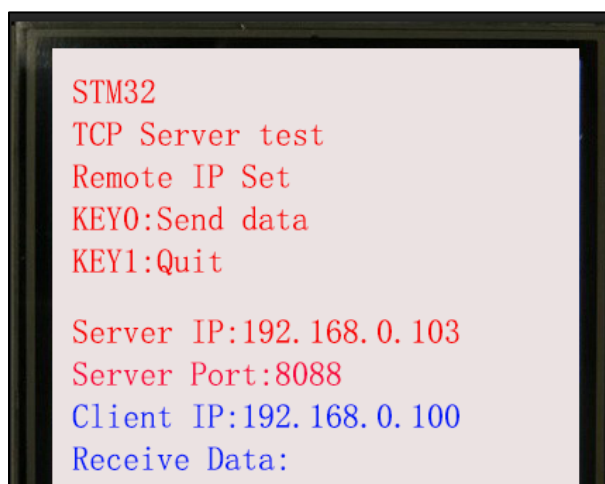


图 13.2.3.3 连接上以后的 LCD 显示

我们通过网络调试助手向开发板发送：www.openedv.com，此时开发板 LCD 上显示接收到的数据如图 13.2.3.4 所示，按下开发板上的 KEY0 键向网络调试助手发送数据。

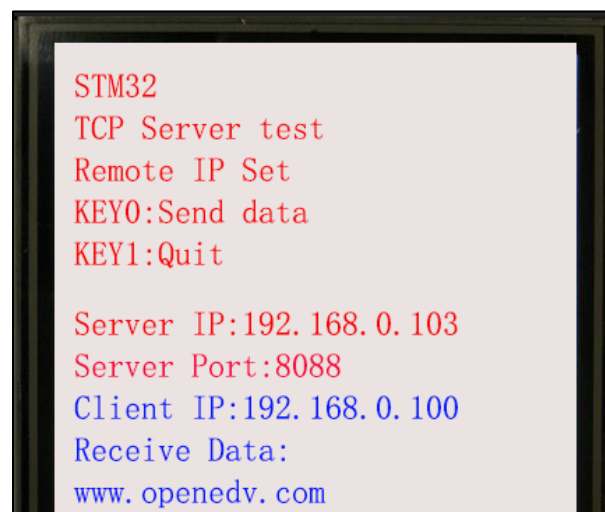


图 13.2.3.4 LCD 显示接收到的数据

第十四章 RAW 编程接口 Web Server 实验

本章采用 RAW 编程 API 接口在开发板上实现一个 WebServer。在本章中我们通过移植并修改 ST 官方的一个 WebServer 的例程来展示。在浏览器中输入开发板的 IP 地址来访问开发板，这时开发板会返回一个网页，在这个网页上我们可以控制开发板的 LED 灯和蜂鸣器并查看 ADC 和内部温度等外设的数据。本章分为如下几个部分：

14.1 Web Server 文件以及相关技术简介

14.2 Web Server 实验

14.1 Web Server 文件以及相关技术简介

(1) 实验相关文件简介

本章我们在 ST 官方 Web Server 例程的基础上完成本章实验，该参考实验是在 STM32Cube_FW_F4_V1.27.0\Projects\STM324x9I_EVAL\Applications\LwIP\LwIP_HTTP_Server_Raw 路径下获取，这里我们打开《lwIP 例程 4 lwIP_RAW_Webserver 实验》实验，该实验用到 Middlewares\lwip\src\apps\http 文件夹下的文件，这些文件如下图所示。

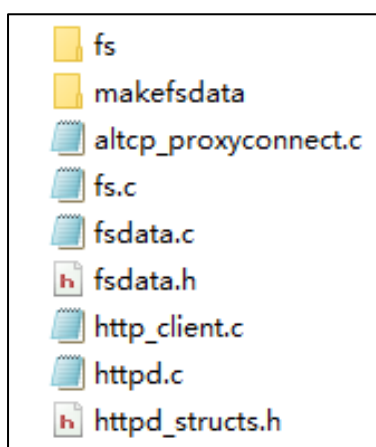


图 14.1.1 Web Server 实验所需文件

上图中 altcp_proxyconnect.c 和 http_client.c 文件并没有在本章实验中用到，下面我们讲解一下本章用到的文件，如下表所示：

文件	说明
Makefsdata 文件夹	这个文件中包含有原始网页文件和将原始的网页文件转换成网页数组的工具，这些文件没有明确的使用方式，本实验我们使用正点原子之前 lwIP 教程的方式生成 C 数组
fs.c	这两个文件用来管理生成的网页数组
fs.h	
fsdata.c	生成的网页数组
fsdata.h	
httpd.c	Http Server 的源代码，本实验的核心文件，这两个文件完成了将开发板配置成 Web Serve 的工作
httpd_structs.h	定义了 HTTPD 协议的结构体信息

表 14.1.1 Web Server 各文件说明

(2) 添加分组

在工程中添加 Middlewares/lwip/lwip/src/apps 分组，该分组添加上图 14.1.1 的 fs.c 以及 httpd.c 文件，如下图所示。

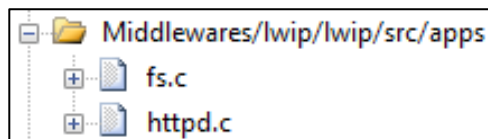


图 14.1.2 Middlewares/lwip/lwip/src/apps 分组添加的文件

(3) lwipopts.h 文件添加配置项

```
/** 支持 CGI */
#define LWIP_HTTPD_CGI 1
/** 支持 SSI */
#define LWIP_HTTPD_SSI 1
/* 设置为 1 将包含"fsdata_custom.c"而不是"fsdata.c" */
#define HTTPD_USE_CUSTOM_FSDATA 0
```

上述源码的第一和第二配置项表示支持 CGI 和 SSI 技术，这些技术我们稍后讲解，第三个配置项主要表示上图 14.1.2 中 fs.c 文件包含的文件路径，这里我们使用 fsdata.c 文件路径。注意：这个 fsdata.c 文件包含了网页数组，这些网页数组的生成方法我们稍后讲解，

(4) 添加工程路径

点击魔法棒并进入 C/C++配置项页面，我们在这个页面下添加 “..\..\Middlewares\lwip\src\include\lwip\apps” 头文件路径。

(5) 网页数组制作

在 lwip\src\apps\http\路径下包含了 makefsdata 文件夹，该文件夹中的文件可以把网页生成网页数组，这里我们不使用这个文件夹，我们使用正点原子之前的 makefsdata 文件夹，该文件夹包含了 makefsdata.exe，我们使用这个软件自动生成即可。

首先我们复制正点原子之前的 lwip 实验的下的 makefsdata 文件夹到桌面上，makefsdata 文件夹保存着网页数组生成器以及 fs 文件夹，网页数组生成如下图 14.1.3 所示，而 fs 文件夹保存着网页源文件如图 14.1.4 所示：

fs	文件夹	
cmd.reg	注册表项	1 KB
echotool.exe	应用程序	29 KB
makefsdata.exe	应用程序	197 KB
Tftpd32-3.51-setup.exe	应用程序	545 KB

图 14.1.3 makefsdata 文件夹的内容

image	文件夹	
STM32F407ADC_TEMPERATE.shtml	Microsoft Edge ...	3 KB
STM32F407LED_OFF_BEEP_OFF.shtml	Microsoft Edge ...	4 KB
STM32F407LED_OFF_BEEP_ON.shtml	Microsoft Edge ...	4 KB
STM32F407LED_ON_BEEP_OFF.shtml	Microsoft Edge ...	4 KB
STM32F407LED_ON_BEEP_ON.shtml	Microsoft Edge ...	4 KB
STM32F407RTC.shtml	Microsoft Edge ...	3 KB

图 14.1.4 本实验网页源文件

图 14.1.4 中的是网页源文件，显然这个文件不能直接放到 STM32 里面，我们要做一个转

换，使其可以放到 STM32 里面。这里我们通过 makefsdata.exe 这个工具将原始网页文件转换成.c 格式的网页数组，这样就可以添加到我们的工程中了，下面我们讲解 makefsdata 工具的使用。

(6) makefsdata 工具的使用

makefsdata 工具是用来将我们编辑网页文件等信息转换成二进制数的一个工具。接下来我们讲解一下这个工具的使用方法。

① 将 fs 文件夹和 makefsdada.exe 工具放到同一文件夹下,此处为 makefsdata 文件夹，打开 makefsdata 文件夹,如图 14.1.5 所示。其中图 14.1.3 中的 echotool.exe 和 Tftpd32-3.51-setup.exe 为其他工具，这里没有使用到，cmd.reg 稍后我们会讲到。

fs	文件夹	
cmd.reg	注册表项	1 KB
echotool.exe	应用程序	29 KB
makefsdata.exe	应用程序	197 KB
Tftpd32-3.51-setup.exe	应用程序	545 KB

图 14.1.5 makefsdata 文件内容

② 快捷键“win+r”并输入 cmd 进入命令行，我们复制上图的 makafsdata 文件夹路径并在命令行上进入该路径，如图 14.1.6 所示：



图 14.1.6 在命令行下进入 makafsdata 文件夹

如果我们不想使用命令行的方式生成网页数组，请使用我们提供的 cmd.reg 文件导入注册表注册,双击打开 cmd.reg，然后一路确定下去就可以了。

③ 在上图 14.1.6 命令行中输入“makefsdata -i”命令并按回车生成网页数组，如下图所示。

```
C:\Users\ATK\Desktop\makefsdata>makefsdata -i

makefsdata - HTML to C source converter
  by Jim Pettinato           - circa 2003
  extended by Simon Goldschmidt - 2009

HTTP 1.0 header will be statically included.
  Processing all files in directory fs and subdirectories...

processing subdirectory /image/...
processing /image/head.jpg...
processing /image/stm32.jpg...
processing /image/tab.jpg...
processing /STM32F407ADC_TEMPERATE.shtml...
processing /STM32F407LED_OFF_BEEP_OFF.shtml...
processing /STM32F407LED_OFF_BEEP_ON.shtml...
processing /STM32F407LED_ON_BEEP_OFF.shtml...
processing /STM32F407LED_ON_BEEP_ON.shtml...
processing /STM32F407RTC.shtml...

Creating target file...

Processed 9 files - done.
```

图 14.1.7 输入 makefsdata -i 命令

④ 打开 makefsdata 文件夹，打开后如图 14.1.8 所示，我们发现在 makefsdata 文件夹下多了一个 fsdata.c 的 C 文件，这个 fsdata.c 文件就是我们转换后的二进制网页文件，到此 makefsdata 工具的用法介绍完毕

名称	类型	大小
fs	文件夹	
cmd.reg	注册表项	1 KB
echotool.exe	应用程序	29 KB
fsdata.c	C 文件	482 KB
makefsdata.exe	应用程序	197 KB
Tftpd32-3.51-setup.exe	应用程序	545 KB

图 14.1.8 生成的 fsdata.c 文件

(7) 把上图的 fsdata.c 文件替换 Middlewares\lwip\src\apps\http 路径下的 fsdata.c 文件。注意：请把 file_404_html 网页数组和 file_index_html 网页数组添加到 fsdata.c 文件中，不然本实验无法运行。

(8) 添加 httpd_cgi_ssi.c 文件

该 httpd_cgi_ssi.c 文件是正点原子根据 ST 官方的一个 WebServer 的例程改编的，这里例程路径为：STM32Cube_FW_F4_V1.27.0\Projects\STM324xG_EVAL\Applications\LwIP\LwIP_HTTP_Server_Raw\Src 路径下的 httpd_cgi_ssi.c，注意：ST 官方的 WebServer 例程使用的是 fsdata_custom.c 文件，所以它把 HTTPD_USE_CUSTOM_FSDATA 配置项设置为 1。我们把正点原子改编的 httpd_cgi_ssi.c 文件添加到 Middlewares\lwip\lwip_app 分组当中，如图 14.1.10 所示：

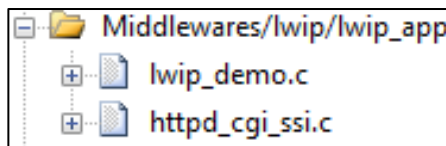


图 14.1.10 添加 httpd_cgi_ssi.c 文件

(9) 在 lwip_demo 函数添加 httpd 初始化源码

```
#include "httpd.h"
#include "httpd_cgi_ssi.h"

/**
 * @brief      lwip_demo 测试
 * @param      无
 * @retval     无
 */
void lwip_demo(void)
{
    /* Httpd Init */
    httpd_init();
    /* 配置 SSI 处理程序 */
    httpd_ssi_init();
    /* 配置 CGI 处理器 */
    httpd_cgi_init();
}
```

至此，我们已经完成了 WebServer 例程改编，下面我们来讲解一下 CGI 和 SSI 技术。

(10) CGI 技术简介

公共网关接口 CGI(Common Gateway Interface) 是 WWW 技术中最重要的技术之一，有着不可替代的重要地位。CGI 是外部应用程序与 Web 服务器之间的接口标准，是在 CGI 程序和 Web 服务器之间传递信息的规程。CGI 规范允许 Web 服务器执行外部程序，并将它们的输出发送给 Web 浏览器，CGI 在物理上是一段程序，运行在服务器上，提供同客户端 HTML 页面的接口。

绝大多数的 CGI 程序被用来解释处理来自表单的输入信息，并在服务器产生相应的处理，或将相应的信息反馈给浏览器，CGI 程序使网页具有交互功能。在我们本章实验中我们通过浏览器控制开发板上的 LED 和蜂鸣器就是使用的 CGI 技术。

(11) SSI 技术简介

服务器端嵌入：Server Side Include，是一种类似于 ASP 的基于服务器的网页制作技术。大多数的 WEB 服务器等均支持 SSI 命令。将内容发送到浏览器之前，可以使用“服务器端包含 (SSI)”指令将文本、图形或应用程序信息包含到网页中。例如，可以使用 SSI 包含时间/日期戳、版权声明或供客户填写并返回的表单。对于在多个文件中重复出现的文本或图形，使用包含文件是一种简便的方法。将内容存入一个包含文件中即可，而不必将内容输入所有文件。通过一个非常简单的语句即可调用包含文件，此语句指示 Web 服务器将内容插入适当网页。而且，使用包含文件时，对内容的所有更改只需在一个地方就能完成。因为包含 SSI 指令的文件要求特殊处理，所以必须为所有 SSI 文件赋予 SSI 文件扩展名。默认扩展名是 .stm、.shtm 和 .html。

SSI 是为 WEB 服务器提供的一套命令，这些命令只要直接嵌入到 HTML 文档的注释内容之中即可。如：<!--#include file="info.htm"-->就是一条 SSI 指令，其作用是将"info.htm"的内容拷贝到当前的页面中，当访问者来浏览时，会看到其它 HTML 文档一样显示 info.htm 其中的内容。其它的 SSI 指令使用形式基本同刚才的举例差不多，可见 SSI 使用只是插入一点代码而已，使用形式非常简单。<!-- -->是 HTML 语法中表示注释，当 WEB 服务器不支持 SSI 时，会忽略这些信息。

在本实验中我们可以通过网页查看开发板的 ADC，内部温度传感器和 RTC 的值就是通过 SSI 来实现的。

14.2 Web Server 实验

14.2.1 硬件设计

1. 例程功能

本实验通过网页控制开发板上的 LED 和 BEEP 外设，当然可以接收开发板的 RTC 以及 ADC 数据并在网页上显示出来。

该实验的实验工程，请参考《lwIP 例程 5 lwIP_RAW_WebServer 实验》。

14.2.2 软件设计

14.2.2.1 程序流程图

本实验的程序流程图，如下图所示。

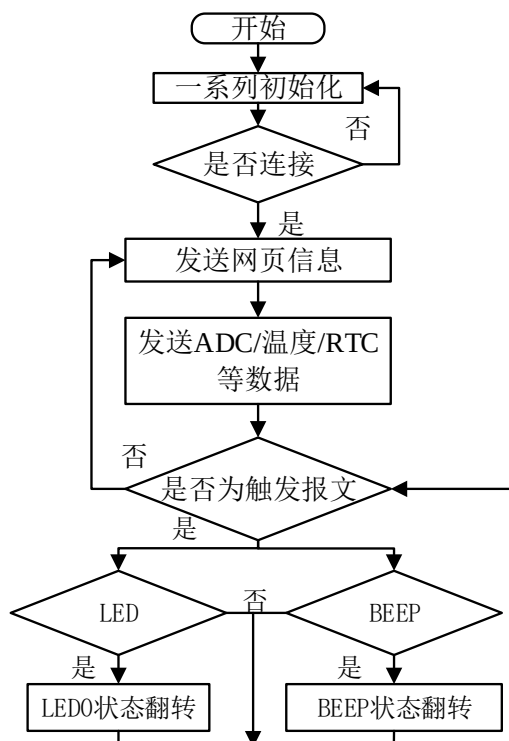


图 14.2.2.1.1 Webserver 实验流程图

14.2.2.2 程序解析

我们打开《lwIP 例程 4 lwIP_RAW_Webserver 实验》的工程，如图 14.2.2.2.1 所示，其中 fs.c 文件管理生成的网页数组文件这个文件由 lwIP 提供。httpd.c 文件是本章实验的重点，这个文件将开发板配置为 Web Server，这个文件也由 lwIP 官方提供的，阅读这个文件需要有网页相关的知识，这里对这个文件不做讲解。

我们在浏览器中输入网址，服务器就会返回给我们相应的网页，然后浏览器解析并呈现给我们。同样的，当我们通过浏览器访问开发板的时候，开发板这时是作为服务器的，服务器针对不同的 URL 在 fsdata.c 文件中找出相应的网页，并且返回给浏览器，在 fsdata.c 文件中查找网页的过程就需要 fs.c 里面的函数。接收浏览器发送的数据并且将网页返回给浏览器的过程都是由 httpd.c 文件里面的函数来完成的。

fs.c 和 httpd.h 文件本章不做讲解，感兴趣的朋友可以去看一下，本章中我们主要讲解的是 httpd_cgi_ssi.c 这个文件，这个文件中讲解了如何使用 CGI 和 SSI 技术完成浏览器与服务器的交互。

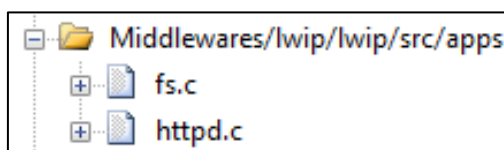


图 14.2.2.2.1 Web Server 工程文件

(1) CGI 实现

我们通过浏览器控制开发板上的 LED 灯和蜂鸣器就是使用的 CGI 技术，我们在开发板上对浏览器发送过来的 URL 进行分析，然后根据不同的 URL 调用不同的程序就可以了，下图 14.2.2.2.2 中我们控制 LED1 灯亮，注意图中的 URL。



图 14.2.2.2.2 打开 LED1

上图中的 URL 为：http://192.168.1.136/leds.cgi?LED1=LED1ON&button2=SEND。我们就是分析这一串字符串来做出相应的处理，其中 leds.cgi 表示为控制 LED 灯的 CGI，后面的“LED1”为变量，LEDION 为变量“LED1”的值。我们根据字符串“leds.cgi”调用处理 LED 灯的程序，然后根据后面的变量和变量值来决定是打开还是关闭 LED 灯。在 httpd_cgi_ssi.c 中我们定义了一个数组 ppcURLs，数组如下，在这个数组中 leds.cgi 对应的 LEDS_CGI_Handler 处理程序，beep.cgi 对应的是 BEEP_CGI_Handler 处理程序。

```
static const tCGI ppcURLs[] = /* cgi 程序 */
```



```
{
    {"/leds.cgi", LEDS_CGI_Handler},
    {"/beep.cgi", BEEP_CGI_Handler},
};
```

LEDS_CGI_Handler 是一个函数，代码如下，从下面代码中可以看出我们是根据变量“LED1”的值来做相应的处理，当为 LED1 ON 的时候就打开 LED1，当为 LED1 OFF 的时候就关闭 LED1。

那么蜂鸣器 BEEP 的处理过程也一样的，这里就不做讲解。最后我们还要初始化 CGI 句柄，初始化函数为 httpd_cgi_init，这个函数很简单。

```
/* CGI LED 控制句柄 */
const char *LEDS_CGI_Handler(int iIndex, int iNumParams, char *pcParam[],
                               char *pcValue[])
{
    uint8_t i = 0; /* 注意根据自己的 GET 的参数多少来选择 i 值范围 */

    iIndex = FindCGIParameter("LED1", pcParam, iNumParams);

    /* 只有一个 CGI 句柄 iIndex=0 */
    if (iIndex != -1)
    {
        LED1(1);
        /* 检查 CGI 参数: example GET /leds.cgi?led=2&led=4 */
        for (i = 0; i < iNumParams; i++)
        {
            if (strcmp(pcParam[i], "LED1") == 0) /* 检查参数"led" */
            {
                if (strcmp(pcValue[i], "LED1ON") == 0) /* 改变 LED1 状态 */
                {
                    /* 打开 LED1 */
                    HAL_GPIO_WritePin(LED1_GPIO_Port, LED1_Pin, GPIO_PIN_RESET);
                }
                else if (strcmp(pcValue[i], "LED1OFF") == 0)
                {
                    /* 关闭 LED1 */
                    HAL_GPIO_WritePin(LED1_GPIO_Port, LED1_Pin, GPIO_PIN_SET);
                }
            }
        }
    }

    if (READ_LED1 == 0 && READ_BEEP == 0)
    {
        return "/STM32F407LED_ON_BEEP_OFF.shtml"; /* LED1 开, BEEP 关 */
    }
}
```



```

else if(READ_LED1 == 0 && READ_BEEP == 1)
{
    return "/STM32F407LED_ON_BEEP_ON.shtml";        /* LED1 开,BEEP 开 */
}
else if(READ_LED1 == 1 && READ_BEEP == 1)
{
    return "/STM32F407LED_OFF_BEEP_ON.shtml";        /* LED1 关,BEEP 开 */
}
else
{
    return "/STM32F407LED_OFF_BEEP_OFF.shtml";        /* LED1 关,BEEP 关 */
}
}
    
```

(2) SSI 实现

我们通过网页查看开发板上的 ADC 值，内部温度传感器和 RTC 时间的时候就是用的 SSI，每隔 1s 刷新一次网页，然后通过 SSI 技术将这些值嵌入到网页中，这样我们就看到时钟在动态的跟新，如下图所示。



图 14.2.2.2.3 网页显示 RTC 时间

SSIHandler 函数为 SSI 的句柄函数，函数代码如下，在这个函数中我们根据参数 iIndex 调用不同的函数来完成向网页中添加数据，这几个函数比较简单，大家自行查阅一下。最后我们还要初始化 SSI 句柄，初始化函数为 httpd_ssi_init，这个函数很简单。

```

/* SSI 的 Handler 句柄 */
static u16_t SSIHandler(int iIndex, char *pcInsert, int iInsertLen)
{
    switch (iIndex)
    {
        case 0:
            ADC_Handler(pcInsert);
            break;
    }
}
    
```

```

    case 1:
        Temperate_Handler(pcInsert);
        break;

    case 2:
        RTCTime_Handler(pcInsert);
        break;

    case 3:
        RTCdate_Handler(pcInsert);
        break;
}

return strlen(pcInsert);
}

```

14.2.3 下载验证

在代码编译成功以后，我们下载代码到开发板中，通过网线连接开发板到路由器上，如果没有路由器的话就连接到电脑端的 RJ45 上，电脑端还要进行设置，设置过程和 UDP 实验一样。下载完成后等待开发板 LCD 出现如图 14.2.3.1 所示，然后我们在浏览器里面输入开发板的 IP 地址，我的开发板 IP 地址为：192.168.1.136，大家根据自己的实际情况输入就行了，按回车，服务器将网页返回给浏览器并显示出来，如图 14.2.3.2 所示。

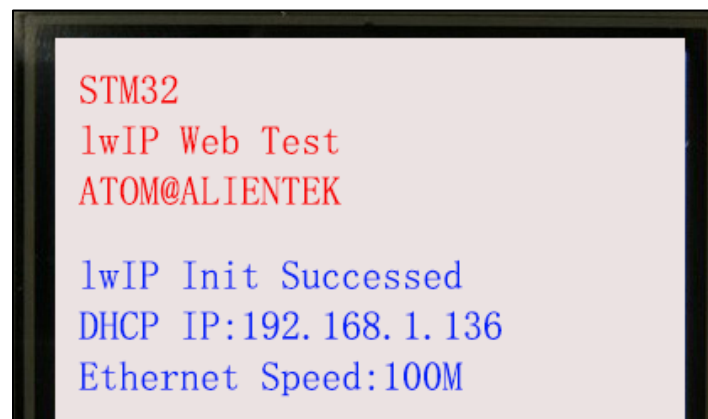


图 14.2.3.1 LCD 显示界面



ALIENTEK 探索者STM32F407开发板 LWIP实验

广州市星翼电子科技

主页	LED/BEEP控制	ADC/内部温度传感器	RTC实时时钟	联系我们
----	------------	-------------	---------	------

ALIENTEK 探索者STM32F407开发板是广州星翼电子科技于2014年11月份推出的新一代STM32F407旗舰开发板。

探索者STM32F407开发板载资源和特点包括：

- STM32F407ZGT6主控MCU, 1MB FLASH, 192K SRAM,168M主频
- 外扩1M字节SRAM, 16M字节SPI FLASH
- 支持音视频播放, 软解WAV/MP3/FLAC/APE/AVI等音视频格式
- 支持lwIP/ucOS/ucGUI/emWIN/FatFS
- 支持双RS232串口/RS485/CAN/USB Slave/USB Host
- 支持NES游戏 (全速,带音频), 性能超越infoNES
- 支持自动识别2.8/3.5/4.3寸电阻/电容式触摸屏
- 支持手写识别/拼音输入法/打电话/收发短信
- 支持六轴传感器(MPU6050), DMP姿态解算
- 支持百万高清摄像头, JPEG/BMP拍照

[关于探索者STM32F407开发板的详细介绍请移步](#)



图 14.2.3.2 WEB Server 主页

最后大家可以测试一下其他的功能，比如控制开发板的 LED1 灯，蜂鸣器等，可以查看 ADC 的值和内部温度传感器，查看 RTC 时间值。

第十五章 NETCONN 编程接口简介

前几章的实验我们都没有使用操作系统，因此它们采用的是 RAW 编程接口，RAW 编程接口使得程序效率高，但是需要对 lwIP 有深入的了解，而且不适合大数据量等场合。本章我们就讲解一下 NETCONN 编程接口，使用 NETCONN API 时需要有操作系统的支持，我们使用的是 FreeRTOS 操作系统。本章分为如下几个部分：

15.1 netbuf 数据缓冲区

15.2 netconn 连接结构

15.3 netconn 编程 API 函数

声明：本章内容参考自《嵌入式网络那些事 LWIP 协议深度剖析与实战演练》，作者朱升林！由于该书是使用 lwIP1.3.2 版本，所以它和本教程的 lwIP2.1.2 源码存在许多小差异。

15.1 netbuf 数据缓冲区

在前面笔者讲过了描述数据包的 pbuf 结构，这里我们就不做详细讲解了，本章我们来讲解一下另一个数据包的 netbuf 结构，netbuf 是 NETCONN 编程 API 接口使用的描述数据包的结构，我们可以使用 netbuf 来管理发送数据、接收数据的缓冲区。有关 netbuf 的详细描述在 netbuf.c 和 netbuf.h 这两个文件中，netbuf 是一个结构体，它在 netbuf.h 中定义了这个结构体，代码如下，

```
/* 网络缓冲区包含数据和寻址信息 */
struct netbuf {
    /* p 字段的指针指向 pbuf 链表 */
    struct pbuf *p, *ptr;

    ip_addr_t addr;          /* 记录了数据发送方的 IP 地址 */
    u16_t port;             /* 记录了数据发送方的端口号 */
};
```

从上述源码可知，其中 p 和 ptr 都指向 pbuf 链表，不同的是 p 一直指向 pbuf 链表的第一个 pbuf 结构，而 ptr 可能指向链表中其他的位置，lwIP 提供的 netbuf_next 和 netbuf_first 函数都是操作 ptr 字段的。addr 和 port 字段用来记录数据发送方的 IP 地址和端口号，lwIP 提供的 netbuf_fromaddr 和 netbuf_fromport 函数用于返回 addr 和 port 这两个字段。从上述的描述，我们可以知道 netbuf 是用来管理发送数据和接收数据的缓冲区，这些收发数据都存储在 pbuf 数据缓冲区当中，所以 netbuf 和 pbuf 肯定有着某种联系，下面我们根据上述的 netbuf 结构体来介绍一下 netbuf 和 pbuf 之间的关系如下图所示。

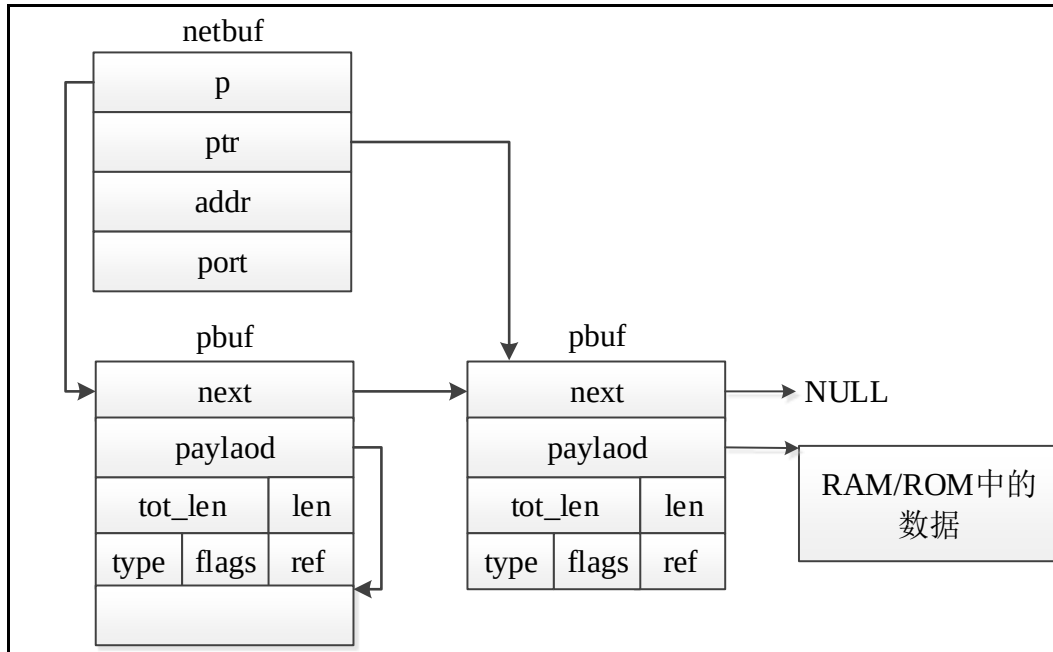


图 15.1.1 用户数据缓冲区

从上图可知：netbuf 结构体中的 p 指针永远指向 pbuf 链表的第一个，而 ptr 可能指向链表中其他的位置。

不管是 TCP 连接还是 UDP 连接，接收到数据包后会将数据封装在一个 netbuf 中，然后将这个 netbuf 交给应用程序去处理。在数据发送时，根据不同的连接有不同的处理：对于 TCP 连接，用户只需要提供待发送数据的起始地址和长度，内核会根据实际情况将数据封装在合适大小的数据包中，并放入发送队列中，对于 UDP 来说，用户需要自行将数据封装在 netbuf 结构中，当发送函数被调用的时候内核直接将数据包中的数据发送出去。

在 netbuf.c 中提供了几个操作 netbuf 的函数，如下表所示。

函数	描述
netbuf_new()	申请一个 netbuf 空间，但是不会分配任何数据空间(不指向任何的 pbuf)，真正的数据存储区域需要调用 netbuf_alloc()函数来分配
netbuf_delete()	释放一个 netbuf 的内存，如果 netbuf 中的 p 指针上还记录有数据，则相应的 pbuf 也会被释放掉
netbuf_alloc()	为 netbuf 结构分配指定大小的数据空间，数据空间是通过 pbuf 的形式表现的，函数返回成功分配的数据空间起始地址(pbuf 的 payload 指向的地址)
netbuf_free()	释放 netbuf 中的 p 指向的 pbuf 数据空间内存
netbuf_ref()	和 netbuf_alloc 类似，不过只是分配一个 pbuf 首部结构(首部结构中包含协议的首部空间)，pbuf 中的 payload 指向参数 dataptr，这种描述数据包的方式在静态数据包的发送时经常用到
netbuf_chain()	将参数 tail 的 pbuf 连接到参数 head 的 pbuf 的后面，调用此函数后参数 tail 结构会被删除掉
netbuf_data()	将 netbuf 结构中的 ptr 指针记录的 pbuf 数据起始地址填入参数 dataptr 中，同时将该 pbuf 中的数据长度填入到参数 len 中
netbuf_next()	将 netbuf 结构的 ptr 指针指向 pbuf 链表中的下一个 pbuf 结构
netbuf_first()	将 netbuf 结构的 ptr 指针指向 pbuf 链表中的第一个 pbuf 结构

表 15.1.1 netbuf 操作函数

注意：用户使用 NETCONN 编程 API 接口时必须在 lwipopts.h 把 LWIP_NETCONN 配置项设置为 1 启动 NETCONN 编程 API 接口。

15.2 netconn 连接结构

我们前面在使用 RAW 编程接口的时候，对于 UDP 和 TCP 连接使用的是两种不同的编程函数：udp_xxx 和 tcp_xxx。NETCONN 对于这两种连接提供了统一的编程接口，用于使用同一的连接结构和编程函数，在 api.h 中定了 netconn 结构体，代码如下。

```
/* netconn 描述符 */
struct netconn {
    /* 连接类型，TCP UDP 或者 RAW */
    enum netconn_type type;
    /* 当前连接状态 */
    enum netconn_state state;
    /* 内核中与连接相关的控制块指针 */
    union {
        struct ip_pcb *ip;      /* IP 控制块 */
        struct tcp_pcb *tcp;    /* TCP 控制块 */
        struct udp_pcb *udp;    /* UDP 控制块 */
        struct raw_pcb *raw;    /* RAW 控制块 */
    } pcb;
    /* 这个 netconn 最后一个异步未报告的错误 */
    err_t pending_err;
#ifdef LWIP_NETCONN_SEM_PER_THREAD
    /* 用于两部分 API 同步的信号量 */
    sys_sem_t op_completed;
#endif
    /* 接收数据的邮箱 */
    sys_mbox_t recvmbox;
#ifdef LWIP_TCP
    /* 用于 TCP 服务器端，连接请求的缓冲队列 */
    sys_mbox_t acceptmbox;
#endif /* LWIP_TCP */
    /* Socket 描述符，用于 Socket API */
#ifdef LWIP_SOCKET
    int Socket;
#endif /* LWIP_SOCKET */
#ifdef LWIP_SO_RCVTIMEO
    /* 接收数据时的超时时间 */
    u32_t recv_timeout;
#endif /* LWIP_SO_RCVTIMEO */
    /* 标识符 */
    u8_t flags;
#ifdef LWIP_TCP
    /* TCP:当传递到 netconn_write 的数据不适合发送缓冲区时，
     * 这将临时存储消息。
     * 也用于连接和关闭。 */

```

```
struct api_msg *current_msg;
#endif /* LWIP_TCP */

/* 连接相关回调函数，实现 Socket API 时使用 */
netconn_callback callback;
};
```

在 api.h 文件中还定义了连接状态和连接类型，这两个都是枚举类型。

```
/* 枚举类型，用于描述连接类型 */
enum netconn_type {
    NETCONN_INVALID        = 0,          /* 无效类型 */
    NETCONN_TCP            = 0x10,       /* TCP */
    NETCONN_UDP            = 0x20,       /* UDP */
    NETCONN_UDPLITE        = 0x21,       /* UDPLite */
    NETCONN_UDPNOCHKSUM    = 0x22,       /* 无校验 UDP */
    NETCONN_RAW            = 0x40        /* 原始链接 */
};

/* 枚举类型，用于描述连接状态，主要用于 TCP 连接中 */
enum netconn_state
{
    NETCONN_NONE,           /* 不处于任何状态 */
    NETCONN_WRITE,          /* 正在发送数据 */
    NETCONN_LISTEN,         /* 侦听状态 */
    NETCONN_CONNECT,        /* 连接状态 */
    NETCONN_CLOSE           /* 关闭状态 */
};
```

下面我们来结合一下 netconn 的 api 函数来讲解这个结构体的作用。

15.3 netconn 编程 API 函数

本节我们就讲解一下 NETCONN 编程的 API 函数，这些函数在 api_lib.c 文件中实现的,其中这个文件包含了很多 netconn 接口的函数，它们大部分是 lwIP 内部调用的，少部分是给用户使用的，用户能使用的函数如下表所示。

函数	描述
netconn_new()	这个函数其实就是一个宏定义，用来为新连接申请一个连接结构 netconn 空间
netconn_delete()	该函数的功能是删除一个 netconn 连接结构
netconn_getaddr()	该函数用来获取一个 netconn 连接结构的源 IP 地址和源端口号或者目的 IP 地址和目的端口号
netconn_bind()	将一个连接结构与本地 IP 地址和端口号进行绑定
netconn_connect()	将一个连接结构与目的 IP 地址和端口号进行绑定
netconn_disconnect()	只能用在 UDP 连接结构中，用来断开与服务器的连接
netconn_listen()	这个函数就是一个宏定义，只在 TCP 服务器程序中使用，用来将连接结构 netconn 置为侦听状态
netconn_accept()	这个函数也只用与 TCP 服务器程序中，服务器调用此函数可以获得一个新的连接
netconn_recv()	从连接的 recvmbox 邮箱中接收数据包，可用于 TCP 连接，也可用于 UDP 连接

netconn_send()	用于在已建立的 UDP 连接上发送数据
netconn_write()	用于在稳定的 TCP 连接上发送数据
netconn_close()	关闭一个 TCP 连接

图 15.3.1 NETCONN 编程的 API 函数描述

netconn_new 函数是函数 netconn_new_with_proto_and_callback 的宏定义，此函数用来为新连接申请一个 netconn 空间，参数为新连接的类型，连接类型在上一节已经讲过了，常用的值是 NETCONN_UDP 和 NETCONN_TCP，分别代表 UDP 连接和 TCP 连接。该函数如下所示：

```
#define netconn_new(t) netconn_new_with_proto_and_callback(t, 0, NULL)

struct netconn *
netconn_new_with_proto_and_callback(enum netconn_type t, u8_t proto,
                                    netconn_callback callback)
{
    struct netconn *conn;
    API_MSG_VAR_DECLARE(msg);
    /* 第一步：构建 api_msg 结构体 */
    API_MSG_VAR_ALLOC_RETURN_NULL(msg);
    /* 申请内存 */
    conn = netconn_alloc(t, callback);
    if (conn != NULL) {
        err_t err;
        /* 连接协议 */
        API_MSG_VAR_REF(msg).msg.n.proto = proto;
        /* 连接的信息 */
        API_MSG_VAR_REF(msg).conn = conn;
        /* 构建 API 消息 */
        err = netconn_apimsg(lwip_netconn_do_newconn, &API_MSG_VAR_REF(msg));
        if (err != ERR_OK) /* 构建失败 */
        {
            /* 释放信号量 */
            sys_sem_free(&conn->op_completed);
            /* 释放邮箱 */
            sys_mbox_free(&conn->recvmbox);
            /* 释放内存 */
            memp_free(MEMP_NETCONN, conn);
            API_MSG_VAR_FREE(msg);
            return NULL;
        }
    }
    API_MSG_VAR_FREE(msg);
    return conn;
}
```

上述源码可知，系统对 conn 申请内存，如果内存申请成功，则系统构建 API 消息。所谓 API 消息，其实就是两个 API 部分的交互的消息，它是由用户的调用 API 函数为起点，使用

IPC 通信机制告诉内核需要执行那个部分的 API 函数，API 消息的知识点笔者已经在第七章 7.4 小节讲解了，下面笔者使用一个示意图来描述 netconn_new 函数交互流程，如下图所示。

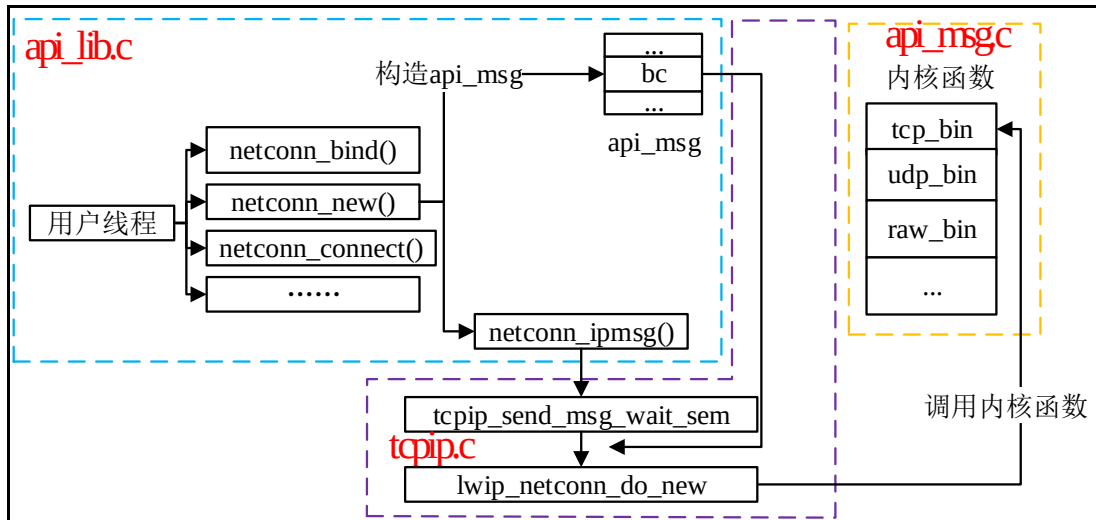


图 15.3.1 用户的应用线程与内核交互示意图

相信大家对上图很熟悉吧，没错，这个图我们在第七章 7.4.2 小节讲解过，这些用户函数就是以这个形式调用的。

netconn_delete 函数是用来删除一个 netconn 连接结构，如果函数调用时双方仍然处于连接状态，则相应连接将被关闭。其中对于 UDP 连接，它的连接会立即被关闭，UDP 控制块被删除；对于 TCP 连接，该函数执行主动关闭，内核完成剩余的断开握手过程，该函数如下所示：

```
err_t
netconn_delete(struct netconn *conn)
{
    err_t err;
    if (conn == NULL) {
        return ERR_OK;
    }
    /* 构建 API 消息 */
    err = netconn_prepare_delete(conn);
    if (err == ERR_OK) {
        netconn_free(conn);
    }
    return err;
}
```

此函数就是调用 netconn_prepare_delete 函数构建 API 消息，API 构建流程请参考图 15.3.1 的构建流程。

netconn_getaddr 函数是用来获取一个 netconn 连接结构的源 IP 地址和源端口号或者目的 IP 地址和目的端口号，IP 地址保存在 addr 当中，而端口信息保存在 port 当中，参数 local 表示是获取源地址还是目的地址，当 local 为 1 时表示本地地址，此函数原型如下。

```
err_t netconn_getaddr(struct netconn*conn,ip_addr_t*addr,u16_t*port,u8_t local);
```

netconn_bind 函数将一个连接结构与本地 IP 地址 addr 和端口号 port 进行绑定，服务器端程序必须执行这一步，服务器必须与指定的端口号绑定才能接受客户端的连接请求，该函数

原型如下。

```
err_t netconn_bind(struct netconn *conn, const ip_addr_t *addr, u16_t port);
```

`netconn_connect` 函数的功能是连接服务器，它将指定的连接结构与目的 IP 地址 `addr` 和目的端口号 `port` 进行绑定，当作为 TCP 客户端程序时，调用此函数会产生握手过程，该函数原型如下。

```
err_t netconn_connect(struct netconn *conn, const ip_addr_t *addr, u16_t port);
```

`netconn_disconnect` 函数只能使用在 UDP 连接中，功能是断开与服务器的连接。对于 UDP 连接来说就是将 UDP 控制块中的 `remote_ip` 和 `remote_port` 字段值清零，函数原型如下。

```
err_t netconn_disconnect (struct netconn *conn);
```

`netconn_listen` 函数只有在 TCP 服务器程序中使用，将一个连接结构 `netconn` 设置为侦听状态，既将 TCP 控制块的状态设置为 LISTEN 状态，该函数原型如下：

```
#define netconn_listen(conn) \
    netconn_listen_with_backlog(conn, TCP_DEFAULT_LISTEN_BACKLOG)
```

`netconn_accept` 函数也只用于 TCP 服务器程序，服务器调用此函数可以从 `acceptmbox` 邮箱中获取一个新建立的连接，若邮箱为空，则函数会一直阻塞，直到新连接的到来。服务器端调用此函数前必须先调用 `netconn_listen` 函数将连接设置为侦听状态，函数原型如下。

```
err_t netconn_accept(struct netconn *conn, struct netconn **new_conn);
```

`netconn_recv` 函数是从连接的 `recvmbox` 邮箱中接收数据包，可用于 TCP 连接，也可用于 UDP 连接，函数会一直阻塞，直到从邮箱中获得数据消息，数据被封装在 `netbuf` 中。如果从邮箱中接收到一条空消息，表示对方已经关闭当前的连接，应用程序也应该关闭这个无效的连接，函数原型如下。

```
err_t netconn_recv(struct netconn *conn, struct netbuf **new_buf);
```

`netconn_send` 函数用于在 UDP 连接上发送数据，参数 `conn` 指出了要操作的连接，参数 `buf` 为要发送的数据，数据被封装在 `netbuf` 中。如果 IP 层分片功能未使能，则 `netbuf` 中的数据不能太长，不能超过 MTU 的值，最好不要超过 1000 字节。如果 IP 层分片功能使能的情况下就可以忽略此细节，函数原型如下。

```
err_t netconn_send(struct netconn *conn, struct netbuf *buf);
```

`netconn_write` 函数用于在稳定的 TCP 连接上发送数据，参数 `dataptr` 和 `size` 分别指出了待发送数据的起始地址和长度，函数并不要求用户将数据封装在 `netbuf` 中，对于数据长度也没有限制，内核会直接处理这些数据，将他们封装在 `pbuf` 中，并挂接到 TCP 的发送队列中。

`netconn_close` 函数用来关闭一个 TCP 连接，该函数会产生一个 FIN 握手包的发送，成功后函数便返回，而后剩余的断开握手操作由内核自动完成，用户程序不用关心，该函数只是断开一个连接，但不会删除连接结构 `netconn`，用户需要调用 `netconn_delete` 函数来删除连接结构，否则会造成内存泄漏，函数原型如下。

```
err_t netconn_close(struct netconn *conn);
```

第十六章 NETCONN 编程接口 UDP 实验

本章，我们开始学习 NETCONN API 函数的使用，本章实验中我们通过电脑端的网络调试助手给开发板发送数据，开发板接收数据并通过串口将接收到的数据发送到串口调试助手上，也可以通过按键从开发板向网络调试助手发送数据。本章分为如下几个部分：

16.1 NETCONN 实现 UDP

16.2 NETCONN 接口的 UDP 实验

16.1 NETCONN 实现 UDP

上一章节，我们已经知道 lwIP 的 NETCONN 编程接口 API 的使用方法，本章实验就是调用这些 API 函数来实现 UDP 连接实验。用户使用 NETCONN 编程接口实现 UDP 连接分以下几个步骤：

- ① 调用函数 `netconn_new` 创建 UDP 控制块。
- ② 定义时间超时函数。
- ③ 调用函数 `netconn_bind` 绑定本地 IP 和端口。
- ④ 调用函数 `netconn_connect` 建立连接。

关于 UDP 的基础知识我们在第十一章时候已经讲过了，这里就不重复讲解。

16.2 NETCONN 接口的 UDP 实验

16.2.1 硬件设计

1. 例程功能

本实验使用 NETCONN 编程接口实现 UDP 连接，我们可通过按下 KEY0 按键发送数据至网络调试助手，还可以接收网络调试助手发送的数据，并在 LCD 显示屏上显示。

该实验的实验工程，请参考《lwIP 例程 7 lwIP_NETCONN_UDP 实验》。

16.2.2 软件设计

16.2.2.1 netconn 的 UDP 连接步骤

1) 创建 UDP 控制块

调用函数 `netconn_new` 创建 NETCONN 的 UDP 控制块。

2) 连接本地 IP 地址和端口号

调用函数 `netconn_bind` 绑定本地 IP 地址和本地端口号，注意：IP 地址必须是有效的。

3) 绑定远程 IP 地址与端口号

调用函数 `netconn_connect` 绑定远程 IP 地址和远程端口号。

4) 接收数据

`netconn_recv` 接收数据。

5) 发送数据

调用函数 `netconn_send` 发送数据。

16.2.2.2 程序流程图

本实验的程序流程图，如下图所示。

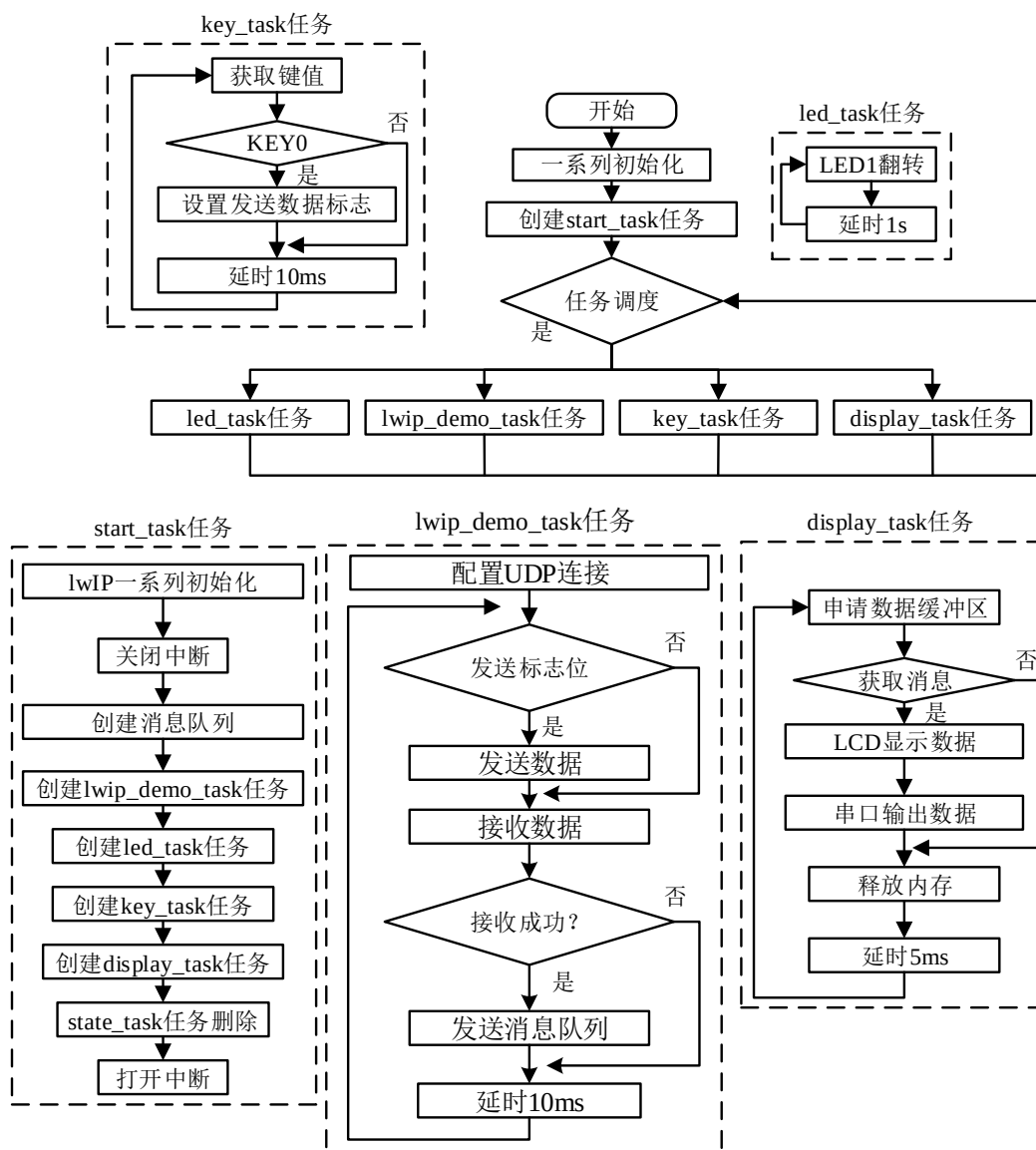


图 16.2.2.1 NETCONN 编程 UDP 实验流程图

16.2.2.3 程序解析

在本章实验中，我们最主要关注两个文件，它们分别为 lwip_demo.c 和 lwip_demo.h 文件，lwip_demo.h 文件主要定义了端口号、数据标识位以及声明 lwip_demo 函数，所以这些不需要我们去讲解，这里我们主要看 lwip_demo.c 文件的函数。在 16.1 小节中笔者已经列出实现 UDP 实验的步骤，在此基础上调用 NETCONN 接口配置 UDP 连接。

```

void lwip_demo(void)
{
    err_t err;
    static struct netconn *udpconn;
    static struct netbuf *recvbuf;

```

```
static struct netbuf *sentbuf;
ip_addr_t destipaddr;
uint32_t data_len = 0;
struct pbuf *q;
/* 第一步：创建 udp 控制块 */
udpconn = netconn_new(NETCONN_UDP);
/* 定义接收超时时间 */
udpconn->recv_timeout = 10;
if (udpconn != NULL) /* 判断创建控制块释放成功 */{
    /* 第二步：绑定控制块、本地 IP 和端口 */
    err = netconn_bind(udpconn, IP_ADDR_ANY, UDP_DEMO_PORT);
    IP4_ADDR(&destipaddr, DEST_IP_ADDR0,
            DEST_IP_ADDR1,
            DEST_IP_ADDR2,
            DEST_IP_ADDR3); /*构造目的 IP 地址 */
    /* 第三步：连接或者建立对话框 */
    netconn_connect(udpconn, &destipaddr, LWIP_DEMO_PORT);
    if (err == ERR_OK) /* 绑定完成 */
    {
        while (1)
        {
            /* 第四步：如果指定的按键按下时，会发送信息 */
            if ((lwip_send_flag & LWIP_SEND_DATA) == LWIP_SEND_DATA)
            {
                sentbuf = netbuf_new();
                netbuf_alloc(sentbuf, strlen((char *)udp_demo_sendbuf));
                memcpy(sentbuf->p->payload, (void *)udp_demo_sendbuf, |
                        strlen((char *)udp_demo_sendbuf));
                err = netconn_send(udpconn, sentbuf);
                if (err != ERR_OK)
                {
                    printf("发送失败\r\n");
                    netbuf_delete(sentbuf); /* 删除 buf */
                }
                lwip_send_flag &= ~LWIP_SEND_DATA; /* 清除数据发送标志 */
                netbuf_delete(sentbuf); /* 删除 buf */
            }
            /* 第五步：接收数据 */
            netconn_recv(udpconn, &recvbuf);
            if (recvbuf != NULL) /* 接收到数据 */
            {
                /*数据接收缓冲区清零 */
                memset(lwip_demo_recvbuf, 0, LWIP_DEMO_RX_BUFSIZE);
            }
        }
    }
}
```

```

        /*遍历完整个 pbuf 链表 */
        for (q = recvbuf->p; q != NULL; q = q->next)
        {
            if (q->len > (LWIP_DEMO_RX_BUFSIZE - data_len))
                memcpy(lwip_demo_recvbuf + data_len, q->payload,
                    (LWIP_DEMO_RX_BUFSIZE - data_len)); /* 拷贝数据 */
            else memcpy(lwip_demo_recvbuf + data_len,
                q->payload, q->len);
            data_len += q->len;
            if (data_len > LWIP_DEMO_RX_BUFSIZE) break;
        }
        data_len = 0; /* 复制完成后 data_len 要清零 */
        printf("%s\r\n", lwip_demo_recvbuf); /* 打印接收到的数据*/
        netbuf_delete(recvbuf); /* 删除 buf */
    }
    else vTaskDelay(5); /* 延时 5ms */
}

else printf("UDP 绑定失败\r\n");}
else printf("UDP 连接创建失败\r\n");}

```

可以看出，笔者调用 NETCONN 接口配置 UDP 连接，配置完成之后调用 netconn_send 发送数据到服务器当中，当然我们也可以调用 netconn_recv 函数接收服务器的数据。

16.2.3 下载验证

代码编译完成后下载到开发板中，初始化完成之后我们来看一下 LCD 显示的内容，如下图所示。

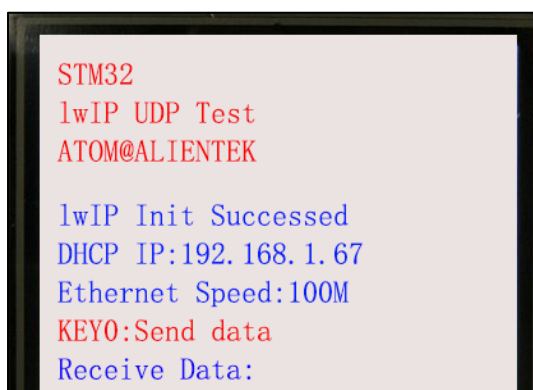


图 16.2.3.1 LCD 显示

我们在来看一下串口调试助手如图 16.2.3.2 所示，在串口调试助手上也输出了我们开发板的 IP 地址，子网掩码、默认网关等信息。

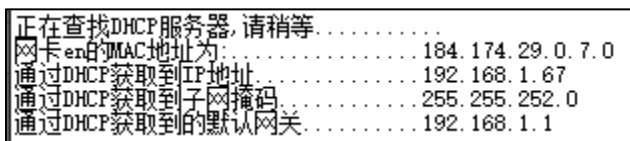


图 16.2.3.2 串口调试助手

我们通过网络调试助手发送数据到开发板当中，结果如图 16.2.3.3 所示，当然我们可以通过开发板上的 KEY0 发送数据到网络调试助手当中，如图 16.2.3.4 所示：

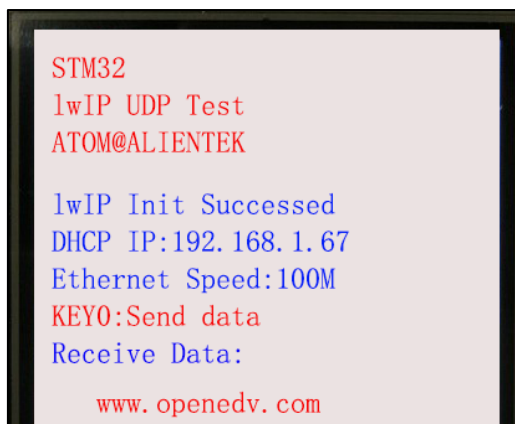


图 16.2.3.3 LCD 显示

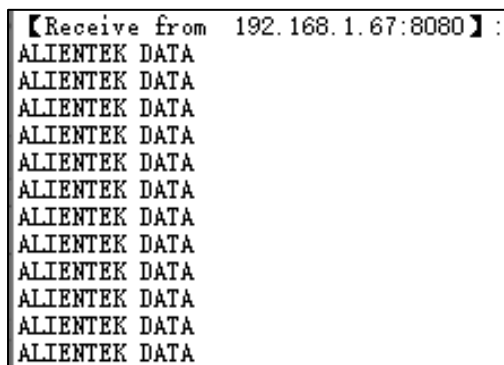


图 16.2.3.4 网络调试助手接收数据

第十七章 NETCONN 编程接口 TCP 客户端实验

本章实验中开发板做 TCP 客户端，网络调试助手为 TCP 服务器。开发板连接到 TCP 服务器(网络调试助手)，网络调试助手给开发板发送数据，开发板接收数据并通过串口将接收到的数据发送到串口调试助手上，也可以通过按键从开发板向网络调试助手发送数据。本章分为如下几个部分：

17.1 NETCONN 实现 TCP 客户端连接步骤

17.2 NETCONN 接口的 TCPClient 实验

17.1 NETCONN 实现 TCP 客户端连接步骤

NETCONN 实现 TCP 客户端连接有以下几步：

- ① 调用函数 `netconn_new` 创建 TCP 控制块。
- ② 调用函数 `netconn_connect` 连接服务器。
- ③ 设置接收超时时间 `tcp_clientconn->recv_timeout`。
- ④ 调用函数 `netconn_getaddr` 获取远端 IP 地址和端口号。
- ⑤ 调用函数 `netconn_write` 和 `netconn_recv` 收发数据。

至于 TCP 协议的知识，请读者擦看第十二章的内容。

17.2 NETCONN 接口的 TCPClient 实验

17.2.1 硬件设计

1. 例程功能

本实验使用 NETCONN 编程接口实现 TCPClient 连接，我们可通过按下 KEY0 按键发送数据至网络调试助手，还可以接收网络调试助手发送的数据，并在 LCD 显示屏上显示。

该实验的实验工程，请参考《lwIP 例程 8 lwIP_NETCONN_TCP 客户端实验》。

17.2.2 软件设计

17.2.2.1 netconn 的 TCPClient 连接步骤

1) 创建 TCP 控制块

调用函数 `netconn_new` 创建 TCP 控制块。

2) 绑定远程 IP 地址与端口号

调用函数 `netconn_connect` 绑定远程 IP 地址和远程端口号。

4) 接收数据

`netconn_recv` 接收数据。

5) 发送数据

调用函数 `netconn_write` 发送数据。

17.2.2.2 程序流程图

本实验的程序流程图，如下图所示。

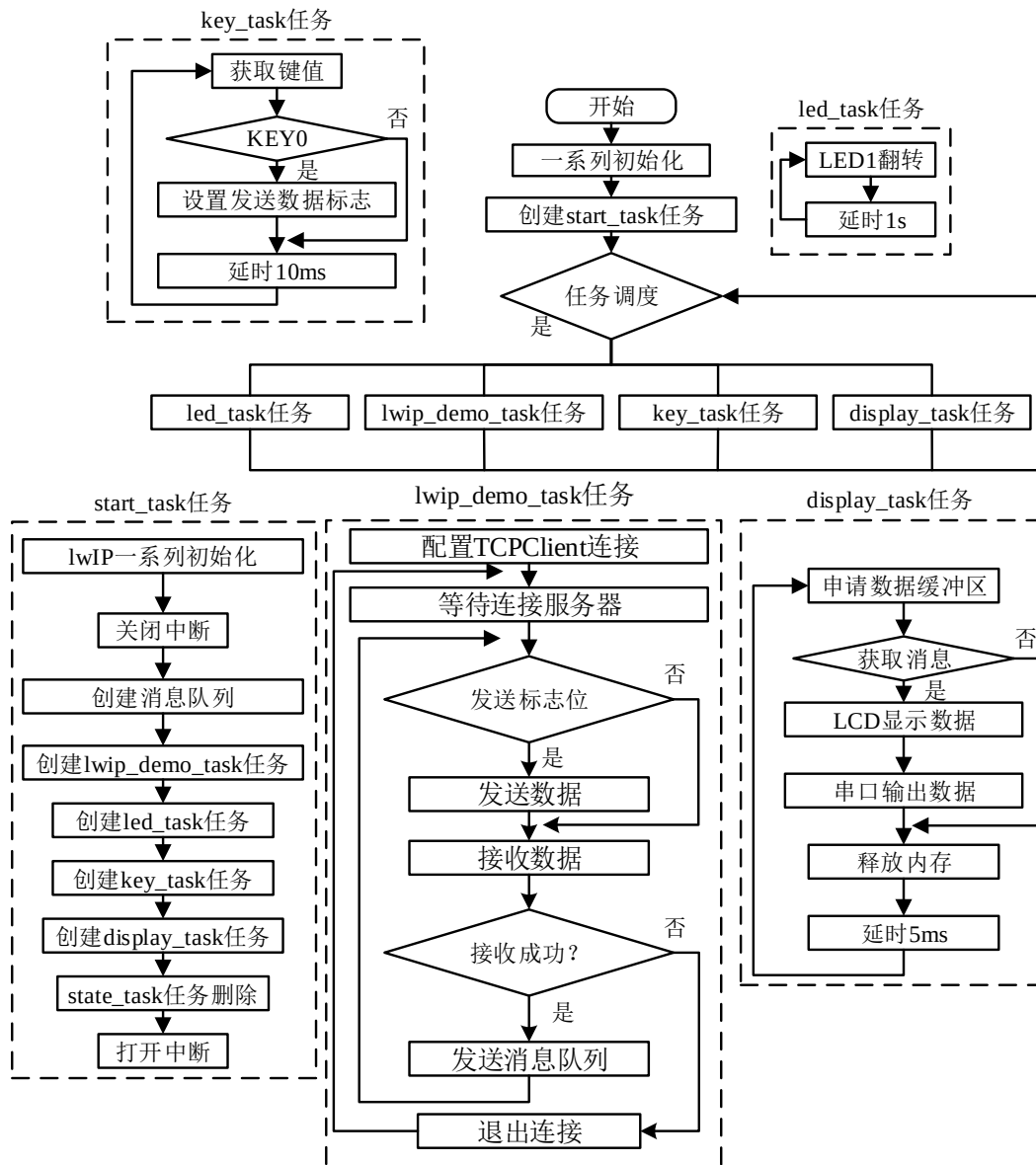


图 17.2.2.2.1 NETCONN 的 TCP 客户端实验流程图

17.2.2.3 程序解析

打开我们的例程，找到 lwip_demo.c 和 lwip_demo.h 两个文件，这两个文件就是我本章实验的源码，在 lwip_demo.c 中我们实现了一个函数 lwip_demo，同上一章一样，都有操作系统的支持下，如下源码所示：

```

void lwip_demo(void)
{
    uint32_t data_len = 0;
    struct pbuf *q;
    err_t err, recv_err;
    ip4_addr_t server_ipaddr, loca_ipaddr;
    static uint16_t server_port, loca_port;
    char *tbuf;

```

```
server_port = LWIP_DEMO_PORT;
IP4_ADDR(&server_ipaddr, DEST_IP_ADDR0, DEST_IP_ADDR1,
        DEST_IP_ADDR2, DEST_IP_ADDR3); /* 构造目的 IP 地址 */
tbuf = mymalloc(SRAMIN, 200); /* 申请内存 */
sprintf((char *)tbuf, "Port:%d", LWIP_DEMO_PORT); /* 客户端端口号 */
lcd_show_string(5, 150, 200, 16, 16, tbuf, BLUE);
myfree(SRAMIN, tbuf);
while (1)
{
    tcp_clientconn = netconn_new(NETCONN_TCP); /* 创建一个 TCP 链接 */
    /* 连接服务器 */
    err = netconn_connect(tcp_clientconn, &server_ipaddr, server_port);
    if (err != ERR_OK)
    {
        printf("接连失败\r\n");
        /* 返回值不等于 ERR_OK, 删除 tcp_clientconn 连接 */
        netconn_delete(tcp_clientconn);
    }
    else if (err == ERR_OK) /* 处理新连接的数据 */
    {
        struct netbuf *recvbuf;
        tcp_clientconn->recv_timeout = 10;
        /* 获取本地 IP 主机 IP 地址和端口号 */
        netconn_getaddr(tcp_clientconn, &loca_ipaddr, &loca_port, 1);
        printf("连接上服务器%d.%d.%d.%d, 本机端口号为:%d\r\n",
                DEST_IP_ADDR0,
                DEST_IP_ADDR1,
                DEST_IP_ADDR2,
                DEST_IP_ADDR3, loca_port);

        while(1)
        {
            /* 有数据要发送 */
            if((tcp_client_flag & LWIP_SEND_DATA) == LWIP_SEND_DATA)
            {
                /* 发送 tcp_server_sendbuf 中的数据 */
                err = netconn_write(tcp_clientconn, tcp_client_sendbuf,
                                    strlen((char *)tcp_client_sendbuf), NETCONN_COPY);
                if (err != ERR_OK)
                {
                    printf("发送失败\r\n");
                }
                tcp_client_flag &= ~LWIP_SEND_DATA;
            }
        }
    }
}
```

```

/*接收到数据*/
if((recv_err = netconn_recv(tcp_clientconn,&recvbuf)) == ERR_OK)
{
    taskENTER_CRITICAL();          /*进入临界区*/
    /*数据接收缓冲区清零*/
    memset(lwip_demo_recvbuf,0, LWIP_DEMO_RX_BUFSIZE);
    for(q=recvbuf->p;q!=NULL;q=q->next) /*遍历完整个 pbuf 链表*/
    {
        if(q->len > (LWIP_DEMO_RX_BUFSIZE-data_len))
        {
            memcpy(lwip_demo_recvbuf+data_len,q->payload,
                (LWIP_DEMO_RX_BUFSIZE-data_len));/* 拷贝数据 */
        }
        else
        {
            memcpy(lwip_demo_recvbuf+data_len,q->payload,
                q->len);
        }
        data_len += q->len;
        if(data_len > TCP_CLIENT_RX_BUFSIZE)
        {
            break; /*超出 TCP 客户端接收数组,跳出*/
        }
    }
    taskEXIT_CRITICAL();          /*退出临界区*/
    data_len=0; /*复制完成后 data_len 要清零*/
    printf("%s\r\n",lwip_demo_recvbuf);
    netbuf_delete(recvbuf);
}
else if(recv_err == ERR_CLSD) /*关闭连接 */
{
    netconn_close(tcp_clientconn);
    netconn_delete(tcp_clientconn);
    printf("服务器%d.%d.%d.%d 断开连接\r\n",DEST_IP_ADDR0,
        DEST_IP_ADDR1,DEST_IP_ADDR2,DEST_IP_ADDR3);
    lcd_fill(5, 89, lcddev.width,110, WHITE);
    break;
}
}
}
}
}

```

上述的源码结构和上一章节的 UDP 实验非常相似，它们唯一不同的是连接步骤以及发送

函数不同，注意：上述函数做了一个判断服务器与客户端的连接状态，如果这个连接状态是断开状态，则系统不断的调用函数 `netconn_connect` 连接服务器，直到连接成功才进入第二个 `while` 循环执行发送接收工作。

17.2.3 下载验证

代码编译完成后下载到开发板中，初始化完成之后我们来看一下 LCD 显示的内容，如下图所示。

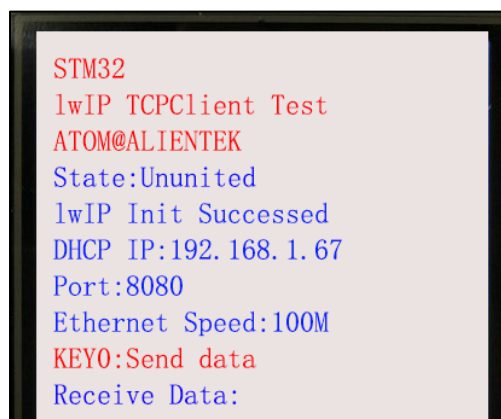


图 17.2.3.1 LCD 显示

我们在来看一下串口调试助手如图 17.2.3.2 所示，在串口调试助手上也输出了我们开发板的 IP 地址，子网掩码、默认网关等信息。

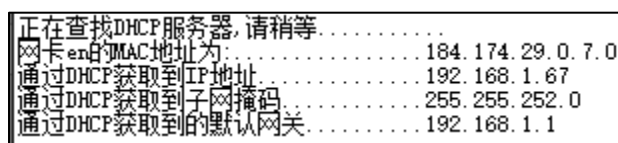


图 17.2.3.2 串口调试助手

我们通过网络调试助手发送数据到开发板当中，结果如图 17.2.3.3 所示，当然我们可以通过开发板上的 KEY0 发送数据到网络调试助手当中，如图 17.2.3.4 所示：



图 17.2.3.3 LCD 显示

```
【Receive from 192.168.1.67:57814】:  
ALIENTEK DATA  
ALIENTEK DATA  
ALIENTEK DATA  
ALIENTEK DATA
```

图 17.2.3.4 网络调试助手接收数据

第十八章 NETCONN 编程接口 TCP 服务器实验

本章实验中开发板做 TCP 服务器，网络调试助手做客户端，网络调试助手连接 TCP 服务器(开发板)。连接成功后网络调试助手可以给开发板发送数据，开发板接收数据并通过串口将接收到的数据发送到串口调试助手上，也可以通过按键从开发板向网络调试助手发送数据。本章分为如下几个部分：

19.1 NETCONN 实现 TCP 服务器

19.2 NETCONN 接口的 TCPServer 实验

18.1 NETCONN 实现 TCP 服务器步骤

NETCONN 实现 TCP 服务器有以下几步：

- ① 调用函数 `netconn_new` 创建 TCP 控制块。
- ② 调用函数 `netconn_bind` 绑定 TCP 控制块、本地 IP 地址和端口号。
- ③ 调用函数 `netconn_listen` 进入监听模式。
- ④ 设置接收超时时间 `conn->recv_timeout`。
- ⑤ 调用函数 `netconn_accept` 接收连接请求。
- ⑥ 调用函数 `netconn_getaddr` 获取远端 IP 地址和端口号。
- ⑦ 调用函数 `netconn_write` 和 `netconn_recv` 收发数据。

至于 TCP 协议的知识，请大家参看第十二章的内容。

18.2 NETCONN 接口的 TCPServer 实验

18.2.1 硬件设计

1. 例程功能

本实验使用 NETCONN 编程接口实现 TCPServer 连接，我们可通过按下 KEY0 按键发送数据至网络调试助手，还可以接收网络调试助手发送的数据，并在 LCD 显示屏上显示。

该实验的实验工程，请参考《lwIP 例程 9 lwIP_NETCONN_TCP 服务器实验》。

18.2.2 软件设计

18.2.2.1 netconn 的 TCPServer 连接步骤

1) 创建 TCP 控制块

调用函数 `netconn_new()` 创建 TCP 控制块。

2) 绑定 TCP 控制块、本地 IP 地址和端口号

调用函数 `netconn_bind()` 绑定本地 IP 地址和端口号。

3) 配置监听模式

调用函数 `netconn_listen()` 进入监听模式。

4) 接收连接请求

调用函数 `netconn_accept()` 接收连接请求

4) 接收数据

调用函数 `netconn_recv()` 接收数据。

5) 发送数据

调用函数 `netconn_write()` 发送数据。

18.2.2.2 程序流程图

本实验的程序流程图，如下图所示。

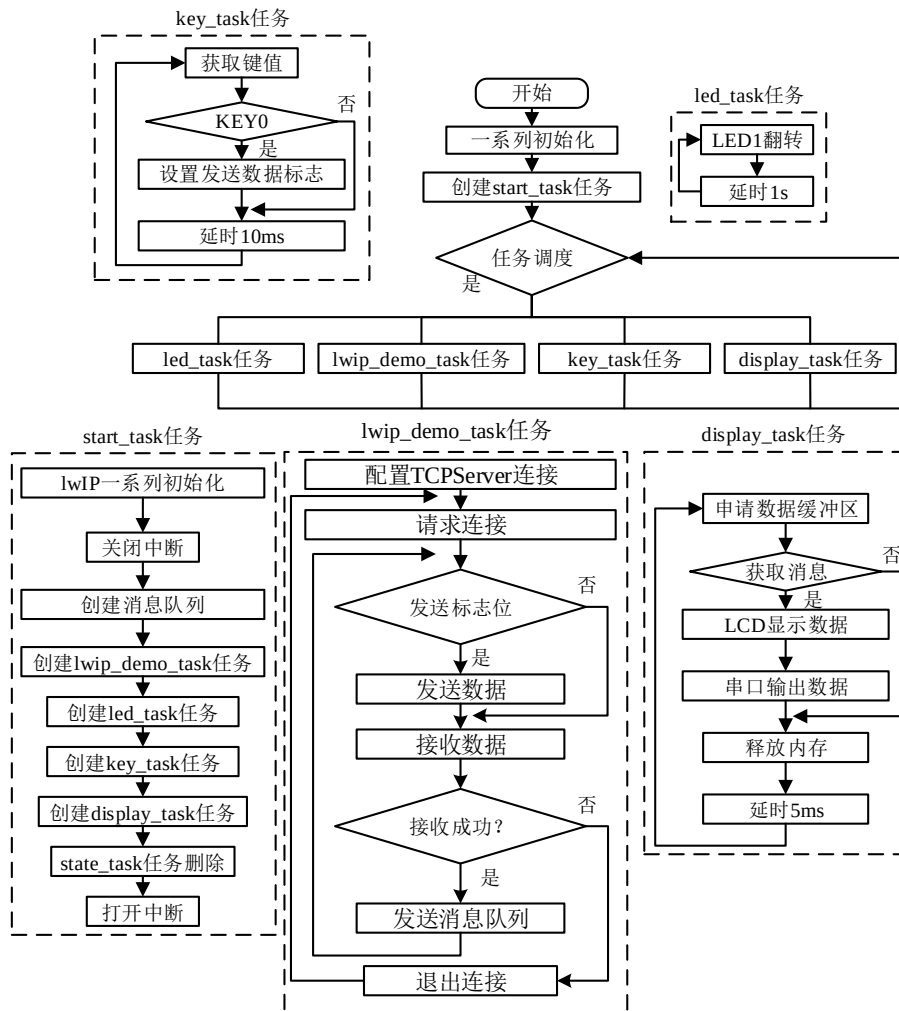


图 18.2.2.3.1 NETCONN 的 TCP 服务器实验

18.2.2.3 程序解析

打开我们的例程，找到 `lwip_demo.c` 和 `lwip_demo.h` 两个文件，这两个文件就是我本章实验的源码，在 `lwip_demo.c` 中我们实现了一个函数 `lwip_demo`，同上一章一样，都有操作系统的支持下，如下源码所示：

```

void lwip_demo(void)
{
    uint32_t    data_len = 0;
    struct      pbuf *q;
    err_t       err, recv_err;
    uint8_t     remot_addr[4];

```

```

struct      netconn *conn, *newconn;
static      ip_addr_t ipaddr;
static      u16_t  port;
/* 第一步：创建一个 TCP 控制块 */
conn=netconn_new(NETCONN_TCP);          /* 创建一个 TCP 链接 */
/* 第二步：绑定 TCP 控制块、本地 IP 地址和端口号 */
netconn_bind(conn, IP_ADDR_ANY, LWIP_DEMO_PORT); /* 绑定端口 8088 号端口 */
/* 第三步：监听 */
netconn_listen(conn);                  /* 进入监听模式 */
conn->recv_timeout = 10;                /* 禁止阻塞线程 等待 10ms */
while (1)
{
    /* 第四步：接收连接请求 */
    err = netconn_accept(conn, &newconn); /* 接收连接请求 */
    if(err==ERR_OK)newconn->recv_timeout = 10;
    if (err == ERR_OK)                  /* 处理新连接的数据 */
    {
        struct netbuf *recvbuf;
        netconn_getaddr(newconn, &ipaddr, &port, 0); /* 获取远端 IP 地址和端口号 */
        remot_addr[3] = (uint8_t) (ipaddr.addr >> 24);
        remot_addr[2] = (uint8_t) (ipaddr.addr >> 16);
        remot_addr[1] = (uint8_t) (ipaddr.addr >> 8);
        remot_addr[0] = (uint8_t) (ipaddr.addr);
        while(1)
        {
            /*有数据要发送*/
            if((lwip_send_flag & LWIP_SEND_DATA) == LWIP_SEND_DATA)
            {
                err = netconn_write(newconn ,tcp_server_sendbuf, \
                                    strlen((char*)tcp_server_sendbuf), \
                                    NETCONN_COPY); /*发送 tcp_server_sendbuf 中的数据*/
                if(err != ERR_OK)
                {
                }
                lwip_send_flag &= ~LWIP_SEND_DATA;
            }
            /*接收到数据*/
            if((recv_err = netconn_recv(newconn, &recvbuf)) == ERR_OK)
            {
                taskENTER_CRITICAL(); /*进入临界区*/
                /*数据接收缓冲区清零*/
                memset(lwip_demo_recvbuf, 0, LWIP_DEMO_RX_BUFSIZE);
                for(q=recvbuf->p; q!=NULL; q=q->next) /*遍历完整个 pbuf 链表*/
            }
        }
    }
}

```



```
正在查找DHCP服务器,请稍等.....
网卡en0的MAC地址为:.....184.174.29.0.7.0
通过DHCP获取到IP地址.....192.168.1.67
通过DHCP获取到子网掩码.....255.255.252.0
通过DHCP获取到的默认网关.....192.168.1.1
```

图 18.2.3.2 串口调试助手

我们通过网络调试助手发送数据到开发板当中，结果如图 18.2.3.3 所示，当然我们可以通过开发板上的 KEY0 发送数据到网络调式助手当中，如图 18.2.3.4 所示：

```
STM32
lwIP TCPServer Test
ATOM@ALIENTEK
State:Connection Successful
lwIP Init Successed
DHCP IP:192.168.1.67
Port:8080
Ethernet Speed:100M
KEY0:Send data
Receive Data:
www.openedv.com
```

图 18.2.3.3 LCD 显示

```
【Receive from 192.168.1.67:8080】:
ALIENTEK DATA
ALIENTEK DATA
ALIENTEK DATA
ALIENTEK DATA
ALIENTEK DATA
ALIENTEK DATA
ALIENTEK DATA
ALIENTEK DATA
ALIENTEK DATA
ALIENTEK DATA
ALIENTEK DATA
ALIENTEK DATA
ALIENTEK DATA
ALIENTEK DATA
ALIENTEK DATA
ALIENTEK DATA
ALIENTEK DATA
```

图 18.2.3.4 网络调试助手接收数据

第十九章 Socket 编程接口简介

lwIP 作者为了能更大程度上方便开发者将其他平台上的网络应用程序移植到 lwIP 上，也为了能让更多开发者快速上手 lwIP，他设计了第三种应用程序编程接口，即 Socket API，但是该接口受嵌入式处理器资源和性能的限制，部分 Socket 接口并未在 lwIP 中完全实现。本章分为如下几个部分：

19.1 Socket 编程接口简介

19.2 Socket API 函数

19.1 Socket 编程接口简介

说到 Socket，我们不得不提起 BSD Socket，BSD Socket 是由加州伯克利大学为 Unix 系统开发出来的，所以被称为伯克利套接字（Internet Berkeley Sockets），BSD Socket 是采用 C 语言进程间通信库的应用程序接口（API），允许不同主机或者同一个计算机上的不同进程之间的通信，支持多种 I/O 设备和驱动，具体的实现是依赖操作系统的。这种接口对于 TCP/IP 是必不可少的，所以是互联网的基础技术之一，所以 LWIP 也是引入该程序编程接口，虽然不能完全实现 BSD Socket，但是对于开发者来说，已经足够了。

在 lwIP 抽象出来的 Socket API 中，lwIP 内核为用户提供了最多 NUM_SOCKETS 个可使用的 Socket 描述符，并定义了结构体 lwip_socket（对 netconn 结构的封装和增强）来描述一个具体连接。内核定义了数组 Sockets，通过一个 Socket 描述符就可以索引得到相应的连接结构 lwip_socket，从而实现对连接的操作。连接结构 lwip_socket 的数据结构实现源码如下：

```
#define NUM_SOCKETS MEMP_NUM_NETCONN

#ifdef SELWAIT_T
#define SELWAIT_T u8_t
#endif

union lwip_sock_lastdata {
    struct netbuf *netbuf;
    struct pbuf *pbuf;
};

/* 包含套接字使用的所有内部指针和状态 */
struct lwip_sock {
    /* 套接字目前构建在网络上，每个套接字有一个 netconn */
    struct netconn *conn;
    /* 读上一次读取中留下的数据 */
    union lwip_sock_lastdata lastdata;
#ifdef LWIP_SOCKET_SELECT || LWIP_SOCKET_POLL
    /* 接收数据的次数，由 event_callback() 设置，
       通过接收和选择功能进行测试*/
    s16_t rcvevent;

```

```

/* 数据被隔离的次数(发送缓冲区)，由 event_callback() 设置，
   测试选择 */
u16_t sendevent;

/* 这个套接字发生错误，由 event_callback() 设置，由 select 测试 */
u16_t errevent;

/* 使用 select 计算有多少线程正在等待这个套接字 */
SELWAIT_T select_waiting;
#endif /* LWIP_SOCKET_SELECT || LWIP_SOCKET_POLL */
#if LWIP_NETCONN_FULLDUPLEX
/* 多少线程使用 struct lwip_sock(不是'int')的计数器 */
u8_t fd_used;

/* 挂起的关闭/删除操作的状态 */
u8_t fd_free_pending;
#define LWIP_SOCK_FD_FREE_TCP 1
#define LWIP_SOCK_FD_FREE_FREE 2
#endif
};

```

lwip_socket 结构是对连接结构 netconn 的再次封装，在内核中，它对 lwip_socket 的操作最终都会映射到对 netconn 结构的操作上。简单来讲，Socket API 完全依赖 netconn 接口实现的。

19.2 Socket API 函数

(1) socket 函数

该函数的原型，如下源码所示：

```
#define socket(domain,type,protocol) lwip_socket(domain,type,protocol)
```

向内核申请一个套接字，本质上该函数调用了函数 lwip_socket，该函数的参数如下表所示：

参数	描述
domain	创建的套接字指定使用的协议簇
	AF_INET 表示 IPv4 网络协议
	AF_INET6 表示 IPv6
	AF_UNIX 表示本地套接字（使用一个文件）
type	协议簇中的具体服务类型
	SOCK_STREAM（可靠数据流交付服务，比如 TCP）
	SOCK_DGRAM（无连接数据报交付服务，比如 UDP）
protocol	SOCK_RAW(原始套接字，比如 RAW)
	实际使用的具体协议(常见的有 IPPROTO_TCP、IPPROTO_UDP 等 若设置为"0"，表示根据前两个参数使用缺省协议)

表 19.2.1 函数 Socket()相关形成描述

(2) bind 函数

该函数的原型，如下源码所示：

```
#define bind(s,name,namelen) lwip_bind(s,name,namelen)
int bind(int s, const struct sockaddr *name, socklen_t namelen)
```

该函数与 netconn_bind 函数一样，用于服务器端绑定套接字与网卡信息，本质上就是对函

数 `netconn_bind` 再一次封装，从上述源码可以知道参数 `name` 指向一个 `sockaddr` 结构体，它包含了本地 IP 地址和端口号等信息；参数 `namelen` 指出结构体的长度。结构体 `sockaddr` 定义如下源码所示：

```
struct sockaddr {
    u8_t      sa_len;          /* 长度 */
    sa_family_t sa_family;     /* 协议簇 */
    char      sa_data[14];     /* 连续的 14 字节信息 */
};

struct sockaddr_in {
    u8_t sin_len;             /* 长度 */
    u8_t sin_family;          /* 协议簇 */
    u16_t sin_port;           /* 端口号 */
    struct in_addr sin_addr;   /* IP 地址 */
    char sin_zero[8];
};
```

可以看出，lwIP 作者定义了两个结构体，结构体 `sockaddr` 中的 `sa_family` 指向该套接字所使用的协议簇，本地 IP 地址和端口号等信息在 `sa_data` 数组里面定义，这里暂未用到。由于 `sa_data` 以连续空间的方式存在，所以用户要填写其中的 IP 字段和端口 `port` 字段，这样会比较麻烦，因此 lwIP 定义了另一个结构体 `sockaddr_in`，它与 `sockaddr` 结构对等，只是从中抽出 IP 地址和端口号 `port`，方便于用于的编程操作。

(3) connect 函数

该函数与 `netconn` 接口的 `netconn_connect` 函数作用是一样的，因此它是被 `netconn_connect` 函数封装了，该函数的作用是将 Socket 与远程 IP 地址和端口号绑定，如果开发板作为客户端，通常使用这个函数来绑定服务器的 IP 地址和端口号，对于 TCP 连接，调用这个函数会使客户端与服务器之间发生连接握手过程，并建立稳定的连接；如果是 UDP 连接，该函数调用不会有任何数据包被发送，只是在连接结构中记录下服务器的地址信息。当调用成功时，函数返回 0；否则返回-1。该函数的原型如下源码所示：

```
#define connect(s,name,namelen)    lwip_connect(s,name,namelen)
int lwip_connect(int s, const struct sockaddr *name, socklen_t namelen);
```

(4) listen 函数

该函数和 `netconn` 的函数 `netconn_listen` 作用一样，它是由函数 `netconn_listen` 封装得来的，内核同时接收到多个连接请求时，需要对这些请求进行排队处理，参数 `backlog` 指明了该套接字上连接请求队列的最大长度。当调用成功时，函数返回 0；否则返回-1。该函数的原型如下源码所示：

```
#define listen(s,backlog)    lwip_listen(s,backlog)
int lwip_listen(int s, int backlog);
```

注意：该函数作用于 TCP 服务器程序。

(5) accept 函数

该函数与 `netconn_accept` 作用是一样的，当接收到新连接后，连接另一端（客户端）的地址信息会被填入到地址结构 `addr` 中，而对应地址信息的长度被记录到 `addrlen` 中。函数返回新连接的套接字描述符，若调用失败，函数返回-1。该函数的原型如下源码所示：

```
#define accept(s,addr,addrlen)    lwip_accept(s,addr,addrlen)
int lwip_accept(int s, struct sockaddr *addr, socklen_t *addrlen);
```

注意：该函数作用于 TCP 服务器程序。

(6) send()/sendto()函数

该函数是被 netconn_send 封装的，其作用是向另一端发送 UDP 报文，这两个函数的原型如下源码所示：

```
#define send(s,dataptr,size,flags)    lwip_send(s,dataptr,size,flags)
#define sendto(s,dataptr,size,flags,to,tolen)
                                     lwip_sendto(s,dataptr,size,flags,to,tolen)
ssize_t lwip_send(int s, const void *dataptr, size_t size, int flags);
ssize_t lwip_sendto(int s, const void *dataptr, size_t size, int flags,
                    const struct sockaddr *to, socklen_t tolen);
```

可以看出，函数 sendto 比函数 send 多了两个参数，该函数如下表所示：

参数	描述
s	Socket 接口
dataptr	发送数据的起始地址
size	长度
flags	数据发送时的特殊处理，例如带外数据、紧急数据等，通常设置为 0
to(sendto())	目的地址信息
tolen(sendto())	信息的长度

表 20.2.2 函数 sendto()和 send()形参描述

(7) write 函数

该函数用于在一条已经建立的连接上发送数据，通常使用在 TCP 程序中，但在 UDP 程序中也能使用。该函数本质上是基于前面介绍的 send 函数来实现的，其参数的意义与 send 也相同。当函数调用成功时，返回成功发送的字节数；否则返回-1。

(8) read()/recv()/recvfrom()函数

函数 recvfrom 和 recv 用来从一个套接字中接收数据，该函数可以在 UDP 程序使用，也可在 TCP 程序中使用。该函数本质上是函数 netconn_recv 的封装，其参数与函数 sendto 的参数完全相似，如表 20.2.3 所示，数据发送方的地址信息会被填写到 from 中，fromlen 指明了缓存 from 的长度；mem 和 len 分别记录了接收数据的缓存起始地址和缓存长度，flags 指明用户控制接收的方式，通常设置为 0。两个函数的原型如下源码所示：

```
#define recv(s,mem,len,flags)        lwip_recv(s,mem,len,flags)
#define recvfrom(s,mem,len,flags,from,fromlen)
                                     lwip_recvfrom(s,mem,len,flags,from,fromlen)
ssize_t lwip_readv(int s, const struct iovec *iov, int iovcnt);
ssize_t lwip_recvfrom(int s, void *mem, size_t len, int flags,
                      struct sockaddr *from, socklen_t *fromlen);
#define read(s,mem,len) lwip_read(s,mem,len)
                           ssize_t lwip_read(
                               int s,
                               void *mem,
                               size_t len);
```

参数	描述
s	Socket 接口
mem	接收数据的缓存起始地址
len	缓存长度

flags	用户控制接收的方式，通常设置为 0
from(recvfrom())	发送方的地址信息
fromlen(recvfrom())	缓存 from 的长度

表 20.2.3 函数 recv()和 recvfrom()形参描述

(9) close 函数

函数 close 作用是关闭套接字，对应的套接字描述符不再有效，与描述符对应的内核结构 lwip_Socket 也将被全部复位。该函数本质上是被 netconn_delete 的封装，对于 TCP 连接来说，该函数将导致断开握手过程的发生。若调用成功，该函数返回 0；否则返回-1。该函数的原型如下源码所示：

```
#define close(s)      lwip_close(s)
int lwip_close(int s);
```

这些函数到底如何调用，请大家看第十五章节的内容，这里已经很详细讲解了 API 消息如何发送到内核并调用相关的函数。

第二十章 Socket 编程接口 UDP 实验

对于 lwIP 的 Socket 的使用方式，其实和文件操作相类似的。如果我们打开一个文件首先打开-读/写-关闭，在 TCP/IP 网络通信中同样存在这些操作，不过它使用的接口不是文件描述符或者 FILE*，而是一个称做 Socket 的描述符。对于 Socket，它也可以通过读、写、打开、关闭操作来进行网络数据传送。同时，还有一些辅助的函数，如域名/IP 地址查询、Socket 功能设置等。本章我们使用 Socket 编程接口实现 UDP 实验。本章分为如下几个部分：

20.1 Socket 编程 UDP 连接流程

20.2 Socket 接口的 UDP 实验

20.1 Socket 编程 UDP 连接流程

实现 UDP 协议之前，用户必须先配置结构体 `sockaddr_in` 的成员变量才能实现 UDP 连接，该配置步骤如下所示：

- ① `sin_family` 设置为 `AF_INET` 表示 IPv4 网络协议。
- ② `sin_port` 为设置端口号，笔者设置为 8080。
- ③ `sin_addr.s_addr` 设置本地 IP 地址。
- ④ 调用函数 `Socket` 创建 Socket 连接，注意：该函数的第二个参数 `SOCK_STREAM` 表示 TCP 连接，`SOCK_DGRAM` 表示 UDP 连接。
- ⑤ 调用函数 `bind` 将本地服务器地址与 Socket 进行绑定。
- ⑥ 调用收发函数接收或者发送。

20.2 Socket 接口的 UDP 实验

20.2.1 硬件设计

1. 例程功能

本实验使用 Socket 编程接口实现 UDP 服务器，并可通过按键发送 UDP 广播数据至其他的 UDP 客户端，也能够接收其他 UDP 客户端广播的数据，并实时显示至 LCD 屏幕上。

该实验的实验工程，请参考《lwIP 例程 10 lwIP_SOCKET_UDP 实验》。

20.2.2 软件设计

20.2.2.1 程序流程图

本实验的程序流程图，如下图所示。

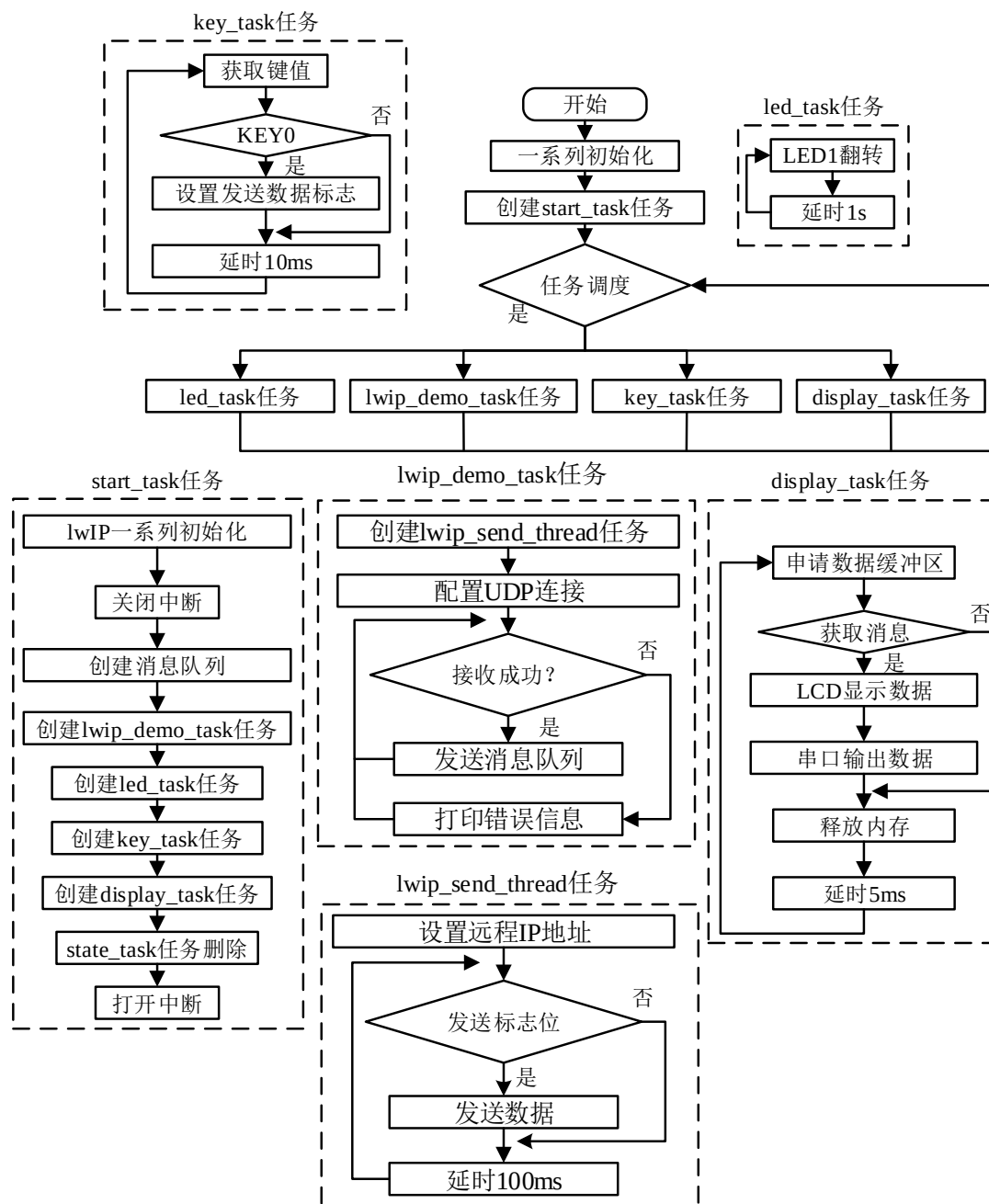


图 20.2.2.1.1 Socket 编程接口 UDP 实验

20.2.2.2 程序解析

在本章节中，我们最主要关注两个文件，分别为 lwip_demo.c 和 lwip_demo.h 文件，lwip_demo.h 文件主要定义了发送标志位以及声明 lwip_demo 函数，这里比较简单我们不需要去讲解，主要看 lwip_demo.c 文件的函数，我们在 lwip_demo 函数中配置相关 UDP 参数以及创建了一个发送数据线程 lwip_send_thread，这个发送线程就是调用 scokec 函数发送数据到服务器中，下面我们分别地讲解以下 lwip_demo 函数以及 lwip_send_thread 任务，如下源码所示：

```

/**
 * @brief      发送数据线程
 * @param      无
    
```

```

* @retval      无
*/
void lwip_data_send(void)
{
    sys_thread_new("lwip_send_thread", lwip_send_thread, NULL,
                    512, LWIP_SEND_THREAD_PRIO );
}

/**
* @brief      lwip_demo 实验入口
* @param      无
* @retval      无
*/
void lwip_demo(void)
{
    BaseType_t lwip_err;
    lwip_data_send(); /* 创建发送数据线程 */
    memset(&local_info, 0, sizeof(struct sockaddr_in)); /* 将服务器地址清空 */
    local_info.sin_len = sizeof(local_info);
    local_info.sin_family = AF_INET; /* IPv4 地址 */
    local_info.sin_port = htons(LWIP_DEMO_PORT); /* 设置端口号 */
    local_info.sin_addr.s_addr = htons(INADDR_ANY); /* 设置本地 IP 地址 */

    sock_fd = Socket(AF_INET, SOCK_DGRAM, 0); /* 建立一个新的 Socket 连接 */

    /* 建立绑定 */
    bind(sock_fd, (struct sockaddr *)&local_info, sizeof(struct sockaddr_in));

    while (1)
    {
        memset(lwip_demo_recvbuf, 0, sizeof(lwip_demo_recvbuf));
        recv(sock_fd, (void *)lwip_demo_recvbuf, sizeof(lwip_demo_recvbuf), 0);

        lwip_err = xQueueSend(Display_Queue, &lwip_demo_recvbuf, 0);

        if (lwip_err == errQUEUE_FULL)
        {
            printf("队列 Key_Queue 已满, 数据发送失败!\r\n");
        }
    }
}

/**

```

```

* @brief      发送数据线程函数
* @param      pvParameters : 传入参数 (未用到)
* @retval     无
*/
void lwip_send_thread(void *pvParameters)
{
    pvParameters = pvParameters;

    local_info.sin_addr.s_addr = inet_addr(IP_ADDR); /* 需要发送的远程 IP 地址 */

    while (1)
    {
        if((lwip_send_flag & LWIP_SEND_DATA) == LWIP_SEND_DATA)
        {
            sendto(sock_fd, /* Socket */
                   (char *)lwip_demo_sendbuf, /* 发送的数据 */
                   sizeof(lwip_demo_sendbuf), 0, /* 发送的数据大小 */
                   (struct sockaddr *)&local_info, /* 接收端地址信息 */
                   sizeof(local_info)); /* 接收端地址信息大小 */

            lwip_send_flag &= ~LWIP_SEND_DATA;
        }

        vTaskDelay(100);
    }
}

```

在源码中，lwip_demo 函数通过 lwip_data_send 创建了发送数据的线程 lwip_send_thread，并配置了 Socket 的 UDP 协议。该线程在发送前会检查标志位，有效时则通过 sendto 发送数据并重置标志位。同时，需设置目标 IP 地址以确保数据正确发送。此外，主函数的循环中不断通过 recv 接收数据并使用消息队列发送，其中 recv 可设为非阻塞模式。

20.2.3 下载验证

打开串口调试助手和网络调试助手，注意必须查看 PC 机上的 IP 地址为多少才能确定程序的宏定义 IP_addr 数值，如下源码所示：

```
#define IP_addr "192.168.1.37" /* 远程 IP */
```

代码编译完成后下载到开发板中，等待 lwIP 一系列初始化和等待 DHCP 分配 IP 地址，下面我们来看一下 LCD 显示的内容，如图 20.2.3.1 所示：

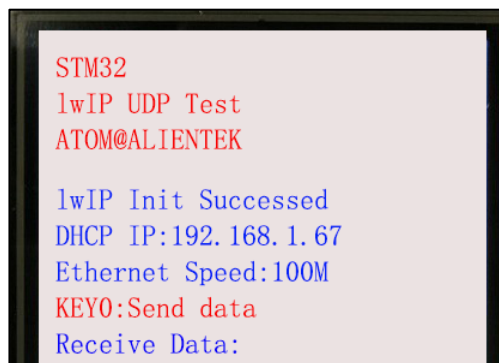


图 20.2.3.1 LCD 显示

我们在来看一下串口调试助手如图 20.2.3.2 所示，在串口调试助手上也输出了我们开发板的 IP 地址，子网掩码、默认网关等信息。

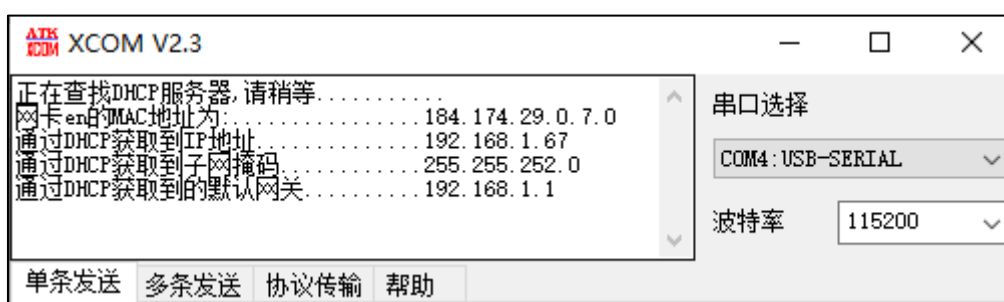


图 20.2.3.2 串口调试助手

我们通过网络调试助手发送数据到开发板当中，结果如图 20.2.3.3 所示，当然我们可以通过开发板上的 KEY0 发送数据到网络调试助手当中，如图 20.2.3.4 所示：

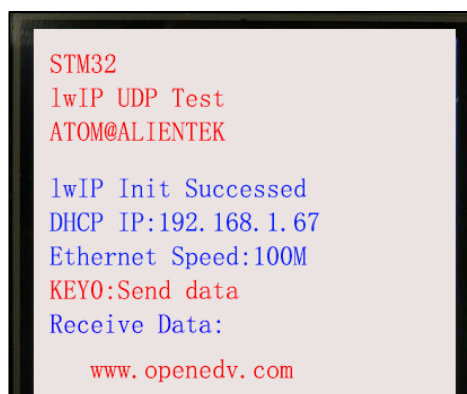


图 20.2.3.3 LCD 显示



图 20.2.3.4 网络调试助手接收数据

第二十一章 Socket 编程接口 TCP 客户端实验

关于 TCP 协议的相关知识，请参考第 12 章的内容。本章，笔者重点讲解 lwIP 的 Socket 接口如何配置 TCP 客户端，并在此基础上实现收发功能。本章分为如下几个部分：

21.1 Socket 编程 TCP 客户端流程

21.2 Socket 接口的 TCPClient 实验

21.1 Socket 编程 TCP 客户端流程

实现 TCP 客户端之前，用户必须先配置结构体 `sockaddr_in` 的成员变量才能实现 TCPClient 连接，该配置步骤如下所示：

- ① `sin_family` 设置为 `AF_INET` 表示 IPv4 网络协议。
- ② `sin_port` 为设置端口号。
- ③ `sin_addr.s_addr` 设置远程 IP 地址。
- ④ 调用函数 `Socket` 创建 Socket 连接，注意：该函数的第二个参数 `SOCK_STREAM` 表示 TCP 连接，`SOCK_DGRAM` 表示 UDP 连接。
- ⑤ 调用函数 `connect` 连接远程 IP 地址。
- ⑥ 调用收发函数实现远程通讯。

21.2 Socket 接口的 TCPClient 实验

21.2.1 硬件设计

1. 例程功能

本实验使用 Socket 编程接口实现 TCPClient 客户端，并可通过按键向所连接的 TCP 服务器发送数据，也能够接收来自 TCP 服务器的数据，并实时显示至 LCD 屏幕上。

该实验的实验工程，请参考《lwIP 例程 11 lwIP_SOCKET_TCP 客户端实验》。

21.2.2 软件设计

21.2.2.1 程序流程图

本实验的程序流程图，如下图所示。

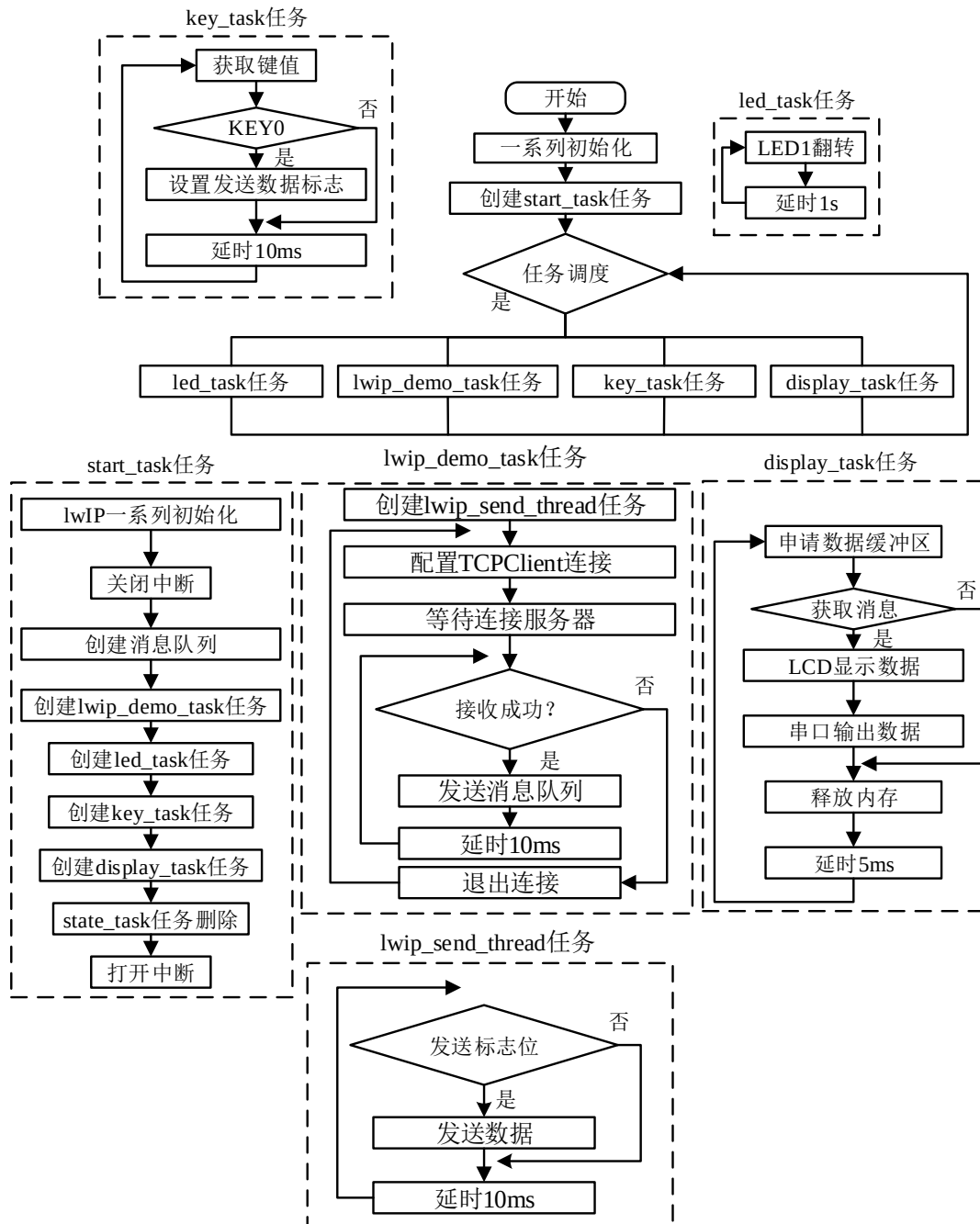


图 21.2.2.1.1 Socket 编程接口 TCP 客户端实验

21.2.2.2 程序解析

本实验，我们着重讲解 lwip_demo.c 文件，该文件实现了三个函数，它们分别为 lwip_data_send、lwip_demo 和 lwip_send_thread 函数，下面笔者分别地讲解它们的实现功能。

```

/**
 * @brief      发送数据线程
 * @param      无
 * @retval     无
 */

```

```
void lwip_data_send(void)
{
    sys_thread_new("lwip_send_thread", lwip_send_thread, NULL,
                   512, LWIP_SEND_THREAD_PRIO );
}
```

此函数调用 `sys_thread_new` 函数创建发送数据线程，它的线程函数为 `lwip_send_thread`，稍后我们重点会讲解。

```
/**
 * @brief      lwip_demo 实验入口
 * @param      无
 * @retval     无
 */
void lwip_demo(void)
{
    struct sockaddr_in atk_client_addr;
    err_t err;
    int recv_data_len;
    BaseType_t lwip_err;
    char *tbuf;

    lwip_data_send(); /* 创建发送数据线程 */

    while (1)
    {
sock_start:
        lwip_connect_state = 0;
        atk_client_addr.sin_family = AF_INET; /* 表示 IPv4 网络协议 */
        atk_client_addr.sin_port = htons(LWIP_DEMO_PORT); /* 端口号 */
        atk_client_addr.sin_addr.s_addr = inet_addr(IP_ADDR); /* 远程 IP 地址 */
        sock = Socket(AF_INET, SOCK_STREAM, 0); /* 可靠数据流交付服务既是 TCP 协议 */
        memset(&(atk_client_addr.sin_zero), 0,
               sizeof(atk_client_addr.sin_zero));

        tbuf = mymalloc(SRAMIN, 200); /* 申请内存 */
        sprintf((char *)tbuf, "Port:%d", LWIP_DEMO_PORT); /* 客户端端口号 */
        lcd_show_string(5, 150, 200, 16, 16, tbuf, BLUE);

        /* 连接远程 IP 地址 */
        err = connect(sock, (struct sockaddr *)&atk_client_addr,
                      sizeof(struct sockaddr));

        if (err == -1)
        {
```



```

        printf("连接失败\r\n");
        sock = -1;
        closeSocket(sock);
        myfree(SRAMIN, tbuf);
        vTaskDelay(10);
        goto sock_start;
    }

    printf("连接成功\r\n");

    lwip_connect_state = 1;

    while (1)
    {
        recv_data_len = recv(sock, lwip_demo_recvbuf,
                               LWIP_DEMO_RX_BUFSIZE, 0);
        if (recv_data_len <= 0 )
        {
            closeSocket(sock);
            sock = -1;
            lcd_fill(5, 89, lcddev.width, 110, WHITE);
            lcd_show_string(5, 90, 200, 16, 16, "State:Disconnect", BLUE);
            myfree(SRAMIN, tbuf);
            goto sock_start;
        }

        /* 接收的数据 */
        lwip_err = xQueueSend(Display_Queue, &lwip_demo_recvbuf, 0);

        if (lwip_err == errQUEUE_FULL)
        {
            printf("队列 Key_Queue 已满，数据发送失败!\r\n");
        }

        vTaskDelay(10);
    }
}

```

根据 21.1 小节的流程配置 `server_addr` 结构体的字段，配置完成之后调用 `connect` 连接远程服务器，接着调用 `recv` 函数接收客户端的数据，并且把数据以消息的方式发送至其他线程当中。

/**

* @brief 发送数据线程函数

```

* @param      pvParameters : 传入参数(未用到)
* @retval      无
*/
void lwip_send_thread(void *pvParameters)
{
    pvParameters = pvParameters;
    err_t err;
    while (1)
    {
        while (1)
        {
            if(((lwip_send_flag & LWIP_SEND_DATA) == LWIP_SEND_DATA)
                && (lwip_connect_state == 1)) /* 有数据要发送 */
            {
                err = write(sock, lwip_demo_sendbuf, sizeof(lwip_demo_sendbuf));

                if (err < 0)
                {
                    break;
                }
                lwip_send_flag &= ~LWIP_SEND_DATA;
            }
            vTaskDelay(10);
        }
        closeSocket(sock);
    }
}

```

此线程函数非常简单，它主要判断 `lwip_send_flag` 变量的状态，若该变量的状态为发送状态，则程序调用 `write` 函数发送数据，并且清除 `lwip_send_flag` 变量的状态。

21.2.3 下载验证

初始化完成之后 LCD 显示以下信息，如下图所示。

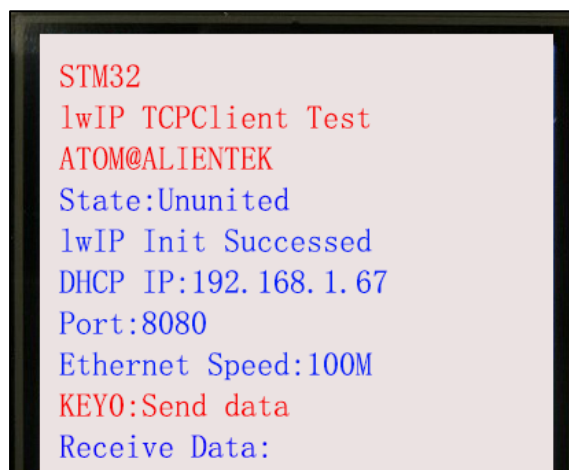


图 21.2.3.1 LCD 显示

我们通过网络调试助手发送数据至开发板，开发板接收完成之后 LCD 在指定位置显示接收的数据，如下图所示。



图 21.2.3.2 LCD 显示

当然，读者可通过 KEY0 按键发送数据至网络调试助手。

第二十二章 Socket 编程接口 TCP 服务器实验

关于 TCP 协议的相关知识，请参考第 12 章的内容。本章，笔者重点讲解 lwIP 的 Socket 接口如何配置 TCP 服务器，并在此基础上实现收发功能。本章分为如下几个部分：

22.1 Socket 编程 TCP 服务器流程

22.2 Socket 接口的 TCPServer 实验

22.1 Socket 编程 TCP 服务器流程

实现 TCP 服务器之前，用户必须先配置结构体 `sockaddr_in` 的成员变量才能实现 TCPServer 连接，该配置步骤如下所示：

- ① `sin_family` 设置为 `AF_INET` 表示 IPv4 网络协议。
- ② `sin_port` 为设置端口号。
- ③ `sin_addr.s_addr` 设置本地 IP 地址。
- ④ 调用函数 `Socket` 创建 Socket 连接，注意：该函数的第二个参数 `SOCK_STREAM` 表示 TCP 连接，`SOCK_DGRAM` 表示 UDP 连接。
- ⑤ 调用函数 `bind` 绑定本地 IP 地址和端口号。
- ⑥ 调用函数 `listen` 监听连接请求。
- ⑦ 调用函数 `accept` 监听连接。
- ⑧ 调用收发函数进行通讯。

上述的步骤就是 Socket 编程接口配置 TCPServer 的流程。

22.2 Socket 接口的 TCPServer 实验

22.2.1 硬件设计

1. 例程功能

本实验使用 Socket 编程接口实现 TCP 服务器，并可通过按键向连接的 TCP 客户端发送数据，也能够接收来自 TCP 客户端的数据，并实时显示至 LCD 屏幕上。

该实验的实验工程，请参考《lwIP 例程 12 lwIP_SOCKET_TCP 服务器实验》。

22.2.2 软件设计

22.2.2.1 程序流程图

本实验的程序流程图，如下图所示。

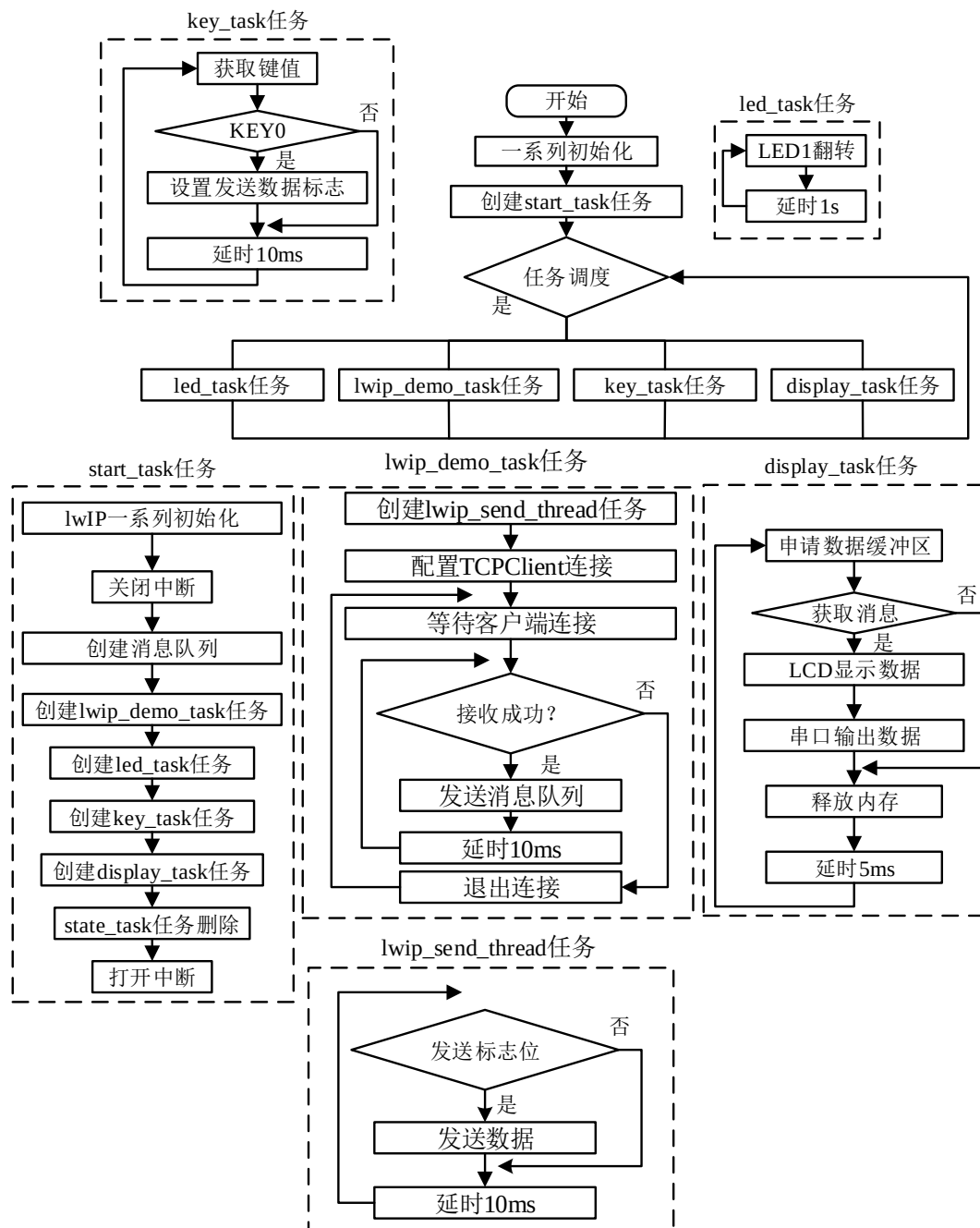


图 22.2.1.1 Socket 编程接口 TCP 客户端实验

22.2.2.2 程序解析

本实验，我们着重讲解 lwip_demo.c 文件，该文件实现了三个函数，它们分别为 lwip_data_send、lwip_demo 和 lwip_send_thread 函数，下面笔者分别地讲解它们的实现功能。

```

/**
 * @brief      发送数据线程
 * @param      无
 * @retval     无
 */

```

```
void lwip_data_send(void)
{
    sys_thread_new("lwip_send_thread", lwip_send_thread, NULL,
                   512, LWIP_SEND_THREAD_PRIO );
}
```

此函数调用 `sys_thread_new` 函数创建发送数据线程，它的线程函数为 `lwip_send_thread`，稍后我们重点会讲解。

```
/**
 * @brief      lwip_demo 实验入口
 * @param      无
 * @retval     无
 */
void lwip_demo()
{
    struct sockaddr_in server_addr;      /* 服务器地址 */
    struct sockaddr_in conn_addr;        /* 连接地址 */
    socklen_t addr_len;                 /* 地址长度 */
    int err;
    int length;
    int sock_fd;
    char *tbuf;
    BaseType_t lwip_err;
    lwip_data_send();                   /* 创建一个发送线程 */

    sock_fd = Socket(AF_INET, SOCK_STREAM, IPPROTO_TCP);
    memset(&server_addr, 0, sizeof(server_addr)); /* 将服务器地址清空 */
    server_addr.sin_family = AF_INET;          /* 地址家族 */
    server_addr.sin_addr.s_addr = htonl(INADDR_ANY); /* 注意转化为网络字节序 */
    /* 使用 SERVER_PORT 指定为程序头设定的端口号 */
    server_addr.sin_port = htons(LWIP_DEMO_PORT);

    tbuf = mymalloc(SRAMIN, 200); /* 申请内存 */
    sprintf((char *)tbuf, "Port:%d", LWIP_DEMO_PORT); /* 客户端端口号 */
    lcd_show_string(5, 150, 200, 16, 16, tbuf, BLUE);
    /* 建立绑定 */
    err = bind(sock_fd, (struct sockaddr *)&server_addr, sizeof(server_addr));

    if (err < 0) /* 如果绑定失败则关闭套接字 */
    {
        closeSocket(sock_fd); /* 关闭套接字 */
        myfree(SRAMIN, tbuf);
    }
}
```

```
err = listen(sock_fd, 4); /* 监听连接请求 */

if (err < 0) /* 如果监听失败则关闭套接字 */
{
    closeSocket(sock_fd); /* 关闭套接字 */
}

while(1)
{
    lwip_connect_state = 0;
    addr_len = sizeof(struct sockaddr_in); /* 将链接地址赋值给 addr_len */
    /* 对监听到的请求进行连接，状态赋值给 sock_conn */
    sock_conn = accept(sock_fd, (struct sockaddr *)&conn_addr, &addr_len);

    if (sock_conn < 0) /* 状态小于 0 代表连接故障，此时关闭套接字 */
    {
        closeSocket(sock_fd);
    }
    else
    {
        lwip_connect_state = 1;
    }

    while (1)
    {
        length = recv(sock_conn, (unsigned int *)lwip_demo_recvbuf,
                      sizeof(lwip_demo_recvbuf), 0);

        if (length <= 0)
        {
            goto atk_exit;
        }

        lwip_err = xQueueSend(Display_Queue, &lwip_demo_recvbuf, 0);

        if (lwip_err == errQUEUE_FULL)
        {
            printf("队列 Key_Queue 已满，数据发送失败!\r\n");
        }
    }
}

atk_exit:
```

```

        if (sock_conn >= 0)
        {
            closeSocket(sock_conn);
            sock_conn = -1;
            lcd_fill(5, 89, lcddev.width, 110, WHITE);
            lcd_show_string(5, 90, 200, 16, 16, "State:Disconnect", BLUE);
            myfree(SRAMIN, tbuf);
        }
    }
}

```

根据 22.1 小节的流程配置 `server_addr` 结构体的字段，配置完成之后调用 `listen` 和 `accept` 监听客户端连接请求，接着调用 `recv` 函数接收客户端的数据，并且把数据以消息的方式发送至其他线程当中。

```

/**
 * @brief      发送数据线程函数
 * @param      pvParameters : 传入参数(未用到)
 * @retval     无
 */
static void lwip_send_thread(void *pvParameters)
{
    pvParameters = pvParameters;

    while (1)
    {
        if((lwip_send_flag & LWIP_SEND_DATA) == LWIP_SEND_DATA
            && (lwip_connect_state == 1)) /* 有数据要发送 */
        {
            /* 发送数据 */
            send(sock_conn, lwip_demo_sendbuf, sizeof(lwip_demo_sendbuf), 0);
            lwip_send_flag &= ~LWIP_SEND_DATA;
        }

        vTaskDelay(10);
    }
}

```

此线程函数非常简单，它主要判断 `lwip_send_flag` 变量的状态，若该变量的状态为发送状态，则程序调用 `send` 函数发送数据，并且清除 `lwip_send_flag` 变量的状态。

22.2.3 下载验证

初始化完成之后 LCD 显示以下信息，如下图所示。

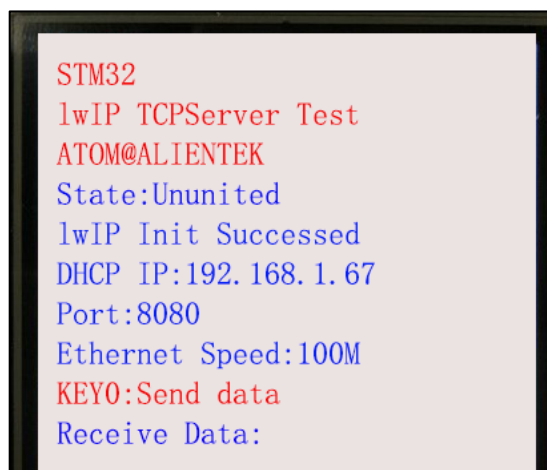


图 22.2.3.1 LCD 显示。

我们通过网络调试助手发送数据至开发板，开发板接收完成之后 LCD 在指定位置显示接收的数据，如下图所示。

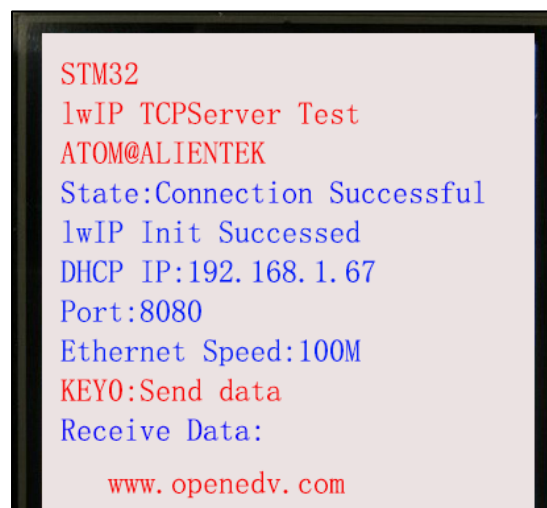


图 22.2.3.2 LCD 显示

当然，读者可通过 KEY0 按键发送数据至网络调试助手。

第二十三章 NTP 协议实验

NTP (Network Time Protocol) 网络时间协议基于 UDP，用于网络时间同步的协议，使网络中的计算机时钟同步到 UTC，再配合各个时区的偏移调整就能实现精准同步对时功能。本章在开发板上使用 UDP 协议连接阿里云的 NTP 服务器，向这个服务器发送 NTP 报文来获取实时时间。本章分为如下几个部分：

23.1 NTP 简介

23.2 NTP 实验

23.1 NTP 简介

NTP 服务器 (Network Time Protocol (NTP)) 是用来使计算机时间同步化的一种协议，它可以使计算机对其服务器或时钟源 (如石英钟，GPS 等等) 做同步化，它可以提供高精度的时间校正 (LAN 上与标准间差小于 1 毫秒，WAN 上几十毫秒)，且可介由加密确认的方式来防止恶毒的协议攻击。时间按 NTP 服务器的等级传播。按照离外部 UTC 源的远近把所有服务器归入不同的 Stratum (层) 中。

NTP 数据报文格式，如下图所示。

0	2	5	8	16	24	31
LI	VN	Mode	Strat	Poll	Prec	
Root Delay						
Root Dispersion						
Reference Identifier						
Reference Timestamp(64)						
Originate Timestamp(64)						
Recevice Timestamp(64)						
Transmit Timestamp(64)						
MAC(96or160)						

图 23.1.1 NTP 报文格式

NTP 数据报文格式的各个字段的作用，如下表所示：

字节段	描述
LI: 2 比特	11 为告警状态，表示时钟未被同步
VN: 3 比特	表示 NTP 的版本号
Mode: 3 比特	0 为未定义、1 为主动对等体模式、2 为被动对等体模式、3 为客户模式、4 为服务器模式、5 为广播模式或组播模式
Strat: 8 比特	表示系统时钟的层数，取值范围为 1~16
Poll: 8 比特	表示轮询时间，即两个连续 NTP 报文之间的时间间隔
Prec: 8 比特	表示系统时钟的精度
Root Delay: 32 比特	表示本地到主参考时钟源的往返时间

Root Dispersion: 32 比特	表示系统时钟相对于主参考时钟的最大误差
Reference Identifier: 32 比特	表示参考时钟源的标识
Reference Timestamp: 64 比特	表示系统时钟最后一次被设定或更新的时间
Originate Timestamp: 64 比特	表示 NTP 请求报文离开发送端时发送端的本地时间
Receive Timestamp: 64 比特	表示 NTP 请求报文到达接收端时接收端的本地时间
Transmit Timestamp: 64 比特	表示应答报文离开应答者时应答者的本地时间
Authenticator: 96 比特	表示验证信息我们怎么获取阿里云 NTP 实时时间数据呢

表 23.1.1 NTP 报文字段描述

从上表可知，NTP 报文的字段非常多，这些字段并不是每一个都必须设置的，请大家根据项目的需要来构建 NTP 请求报文。下面笔者使用网络调试助手制作一个简单的 NTP 实验，如下图所示。

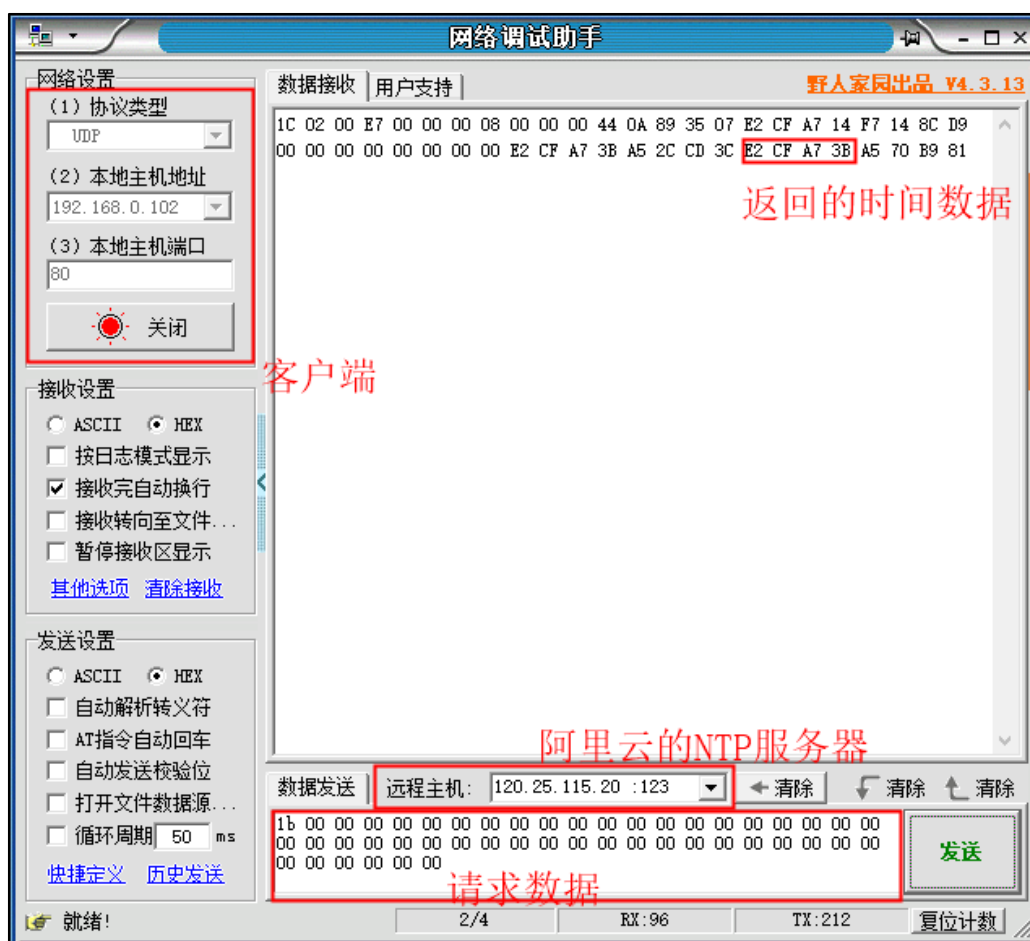


图 23.1.2 获取阿里云 NTP 数据

上图中，笔者使用网络调试助手以 UDP 协议连接阿里云 NTP 服务器，接着在发送框上填入 NTP 请求报文，发送完成之后网络调试助手接收到一段数据，这里我们只取第 40 位到 43 位的十六进制数值，该数值就是当前时间的总秒数。我们把总秒数转换成十进制，并且在在线转换器（<https://tool.lu/timestamp/>）上计算当前时间，如下图所示。

时间戳	1596270779	秒(s)	转换 >>	2020-08-01 16:32:59	北京时间
-----	------------	------	-------	---------------------	------

图 23.1.3 秒转成时间

从上面的内容可知，我们知道获取 NTP 实时时间需要哪些步骤了，这些步骤如下所示：

- ① 以 UDP 协议连接阿里云 NTP 服务器。
- ② 发送 NTP 报文到阿里云 NTP 服务器。
- ③ 获取阿里云 NTP 服务器返回的数据，取第 40 位到 43 位的十六进制数值。
- ④ 把 40 位到 43 位的十六进制数值转成十进制。
- ⑤ 把十进制数值减去 1900-1970 的时间差（2208988800 秒）。
- ⑥ 数值转成年月日时分秒。

23.2 NTP 实验

23.2.1 硬件设计

1. 例程功能

使用 UDP 协议连接阿里云的 NTP 服务器，并周期发送 NTP 请求报文，发送完成之后对阿里云 NTP 服务器返回的数据进行解析，把它转换成实时时间信息。

该实验的实验工程，请参考《lwIP 例程 13 lwip_ntp 实验》和《lwIP 例程 14 lwip_sntp 实验》。

23.2.2 软件设计

23.2.2.1 程序流程图

本实验的程序流程图，如下图所示。

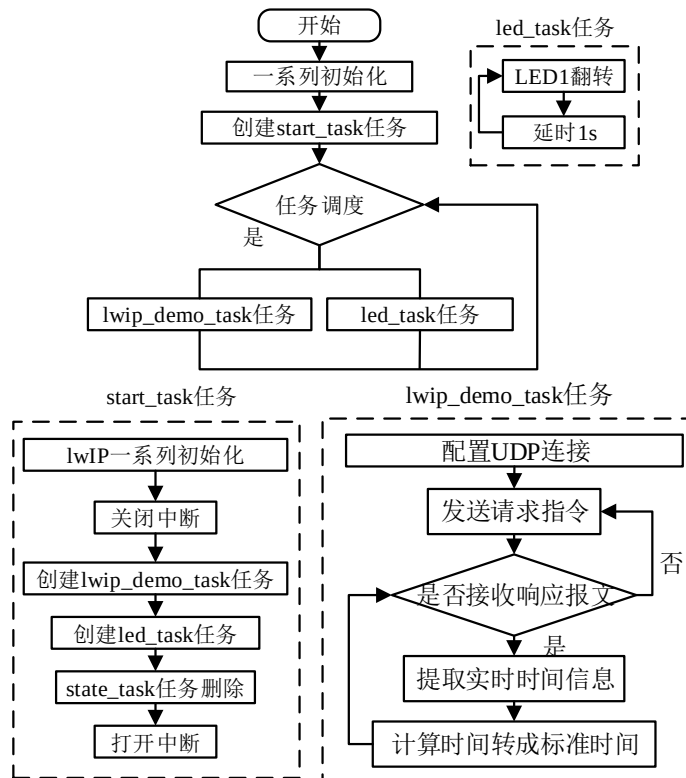


图 23.2.2.3.1 NTP 实验流程图

23.2.2.2 程序解析

为了描述 NTP 报文结构的字段，笔者在 lwip_demo.h 文件下定义了 NPTformat 结构体，它用来描述 NTP 报文结构体的各个字段，该结构体如下所示：

```
typedef struct _NPTformat
{
    char    version;           /* 版本号 */
    char    leap;              /* 时钟同步 */
    char    mode;              /* 模式 */
    char    stratum;           /* 系统时钟的层数 */
    char    poll;              /* 更新间隔 */
    signed char precision;     /* 精密度 */
    unsigned int    rootdelay; /* 本地到主参考时钟源的往返时间 */
    unsigned int    rootdisp;  /* 主时钟相对于主参考时钟的最大误差 */
    char    refid;             /* 参考识别码 */
    unsigned long long    reftime; /* 参考时间 */
    unsigned long long    org;     /* 开始的时间戳 */
    unsigned long long    rec;     /* 收到的时间戳 */
    unsigned long long    xmt;     /* 传输时间戳 */
} NPTformat;
```

该结构体的成员变量与表 23.1.1 的 NTP 报文结构体的字段是一一对应的。

打开 lwip_demo.c 文件，在此文件下定义了四个函数，这些函数的作用如下表所示：

函数	描述
lwip_demo()	实现 UDP 连接
lwip_ntp_client_init()	构建 NTP 请求报文
lwip_get_seconds_from_ntp_server()	获取 NTP 服务器的数据
lwip_calc_date_time()	计算日期时间

表 23.2.2.2.1 lwip_demo.c 相关函数描述

(1) lwip_ntp_client_init 函数

此函数用来构建 NTP 请求报文，通过设置 NPTformat 结构体的成员变量来描述 NTP 报文的字段信息，构建完成之后把该报文存储在缓冲区当中。构建 NTP 报文的源码如下所示：

```
/**
 * @brief    初始化 NTP Client 信息
 * @param    无
 * @retval    无
 */
void lwip_ntp_client_init(void)
{
    uint8_t flag;

    g_ntpformat.leap = 0;           /* 时钟同步 */
    g_ntpformat.version = 3;       /* 版本号 */
    g_ntpformat.mode = 3;          /* 模式 */
    g_ntpformat.stratum = 0;       /* 系统时钟的层数 */
    g_ntpformat.poll = 0;          /* 更新间隔 */
    g_ntpformat.precision = 0;     /* 精密度 */
```

```

g_ntpformat.rootdelay = 0;          /* 本地到主参考时钟源的往返时间 */
g_ntpformat.rootdisp = 0;          /* 统时钟相对于主参考时钟的最大误差 */
g_ntpformat.refid = 0;             /* 参考识别码 */
g_ntpformat.reftime = 0;           /* 参考时间 */
g_ntpformat.org = 0;               /* 开始的时间戳 */
g_ntpformat.rec = 0;               /* 收到的时间戳 */
g_ntpformat.xmt = 0;               /* 传输时间戳 */

flag = (g_ntpformat.version << 3) + g_ntpformat.mode;
memcpy(g_ntp_message, (void const *)&flag, 1);
}

```

可以看到，笔者只设置 NTP 报文的版本和模式字段，其他字段我们设置为 0。

(2) lwip_get_seconds_from_ntp_server 函数

此函数用来获取 NTP 服务器返回的数据，从这个数据截取 40~43 位的数值，并且强制转换成十进制数值，最后递交给其他函数处理。

```

/**
 * @brief      从 NTP 服务器获取时间
 * @param      buf: 存放缓存
 * @param      idx: 定义存放数据起始位置
 * @retval     无
 */
void lwip_get_seconds_from_ntp_server(uint8_t *buf, uint16_t idx)
{
    unsigned long long atk_seconds = 0;
    uint8_t i = 0;

    for (i = 0; i < 4; i++) /* 获取 40~43 位的数据 */
    {
        /* 把 40~43 位转成 16 进制再转成十进制 */
        atk_seconds = (atk_seconds << 8) | buf[idx + i];
    }

    /* 减去 1900-1970 的时间差 (2208988800 秒) */
    atk_seconds -= NTP_TIMESTAMP_DELTA;
    lwip_calc_date_time(atk_seconds); /* 由 UTC 时间计算日期 */
}

```

调用此函数时，该函数的 idx 形参为 40，经过 for 语句的作用，可在数据中截取 40~43 位的数值，截取完成之后强制转换成十进制并减去 1900-1970 的时间差，最后由 lwip_calc_date_time 函数计算时间。

(3) lwip_calc_date_time 函数

此函数是把总秒数转换成时间信息，该函数的源码如下所示：

```

/**
 * @brief      计算日期时间
 * @param      seconds UTC 世界标准时间

```

```
*@retval    无
*/
void lwip_calc_date_time(unsigned long long time)
{
    unsigned int Pass4year;
    int hours_per_year;
    if (time <= 0)
    {
        time = 0;
    }

    nowdate.second = (int)(time % 60);    /* 取秒时间 */
    time /= 60;

    nowdate.minute = (int)(time % 60);    /* 取分钟时间 */
    time /= 60;

    nowdate.hour = (int)(time % 24);      /* 小时数 */
    /* 取过去多少个四年，每四年有 1461*24 小时 */
    Pass4year = ((unsigned int)time / (1461L * 24L));

    nowdate.year = (Pass4year << 2) + 1970;    /* 计算年份 */

    time %= 1461 * 24;                      /* 四年中剩下的小时数 */

    for (;;)                                /* 校正闰年影响的年份，计算一年中剩下的小时数 */
    {
        hours_per_year = 365 * 24;          /* 一年的小时数 */

        if ((nowdate.year & 3) == 0)        /* 判断闰年 */
        {
            hours_per_year += 24;           /* 是闰年，一年则多 24 小时，即一天 */
        }

        if (time < hours_per_year)
        {
            break;
        }

        nowdate.year++;
        time -= hours_per_year;
    }
}
```

```

time /= 24;      /* 一年中剩下的天数 */

time++;         /* 假定为闰年 */

if ((nowdate.year & 3) == 0)      /* 校正闰年的误差，计算月份，日期 */
{
    if (time > 60)
    {
        time--;
    }
    else
    {
        if (time == 60)
        {
            nowdate.month = 1;
            nowdate.day = 29;
            return ;
        }
    }
}

/* 计算月日 */
for (nowdate.month = 0; Days[nowdate.month] < time; nowdate.month++)
{
    time -= Days[nowdate.month];
}

nowdate.day = (int)(time);

return;
}

```

总秒数经过算法的处理，计算得出的年、月、‘时、分和秒都保存在 DateTime 结构体当中。

(40) lwip_demo 函数

此函数调用 lwIP 相关的 API 接口，以 UDP 协议连接阿里云 NTP 服务器，连接完成之后开启定时器定时发送 NTP 请求报文，最后处理 NTP 服务器返回的数据。

```

/**
 * @brief      lwip_demo 程序入口
 * @param      无
 * @retval     无
 */
static void lwip_demo(void)
{

```



```
err_t err;

static struct netconn *udpconn;
static struct netbuf *recvbuf;
static struct netbuf *sentbuf;

ip_addr_t destipaddr;
uint32_t data_len = 0;
struct pbuf *q;

atk_ntp_client_init();

/* 第一步：创建 udp 控制块 */
udpconn = netconn_new(NETCONN_UDP);

/* 定义接收超时时间 */
udpconn->recv_timeout = 10;

if (udpconn != NULL) /* 判断创建控制块释放成功 */
{
    /* 第二步：绑定控制块、本地 IP 和端口 */
    err = netconn_bind(udpconn, IP_ADDR_ANY, NTP_DEMO_PORT);
    /* 域名解析 */
    netconn_gethostbyname((char *) (HOST_NAME), &(destipaddr));
    /* 第三步：连接或者建立对话框 */
    netconn_connect(udpconn, &destipaddr, NTP_DEMO_PORT); /* 连接到远端主机 */

    if (err == ERR_OK) /* 绑定完成 */
    {
        while (1)
        {
            sentbuf = netbuf_new();
            netbuf_alloc(sentbuf, 48);
            memcpy(sentbuf->p->payload, (void *) ntp_message,
                    sizeof(ntp_message));
            err = netconn_send(udpconn, sentbuf);
            if (err != ERR_OK)
            {
                printf("发送失败\r\n");
                netbuf_delete(sentbuf); /* 删除 buf */
            }
            netbuf_delete(sentbuf); /* 删除 buf */

            /* 第五步：接收数据 */
            netconn_recv(udpconn, &recvbuf);
            vTaskDelay(1000); /* 延时 1s */
            if (recvbuf != NULL) /* 接收到数据 */
            {
```

```

/* 数据接收缓冲区清零 */
memset(ntp_demo_recvbuf, 0, NTP_DEMO_RX_BUFSIZE);
/* 遍历完整个 pbuf 链表 */
for (q = recvbuf->p; q != NULL; q = q->next)
{
    /* 判断要拷贝到 UDP_DEMO_RX_BUFSIZE 中的数据是否大于
    UDP_DEMO_RX_BUFSIZE 的剩余空间，如果大于
    的话就只拷贝 UDP_DEMO_RX_BUFSIZE 中剩余长度的数据，
    否则的话就拷贝所有的数据 */
    if (q->len > (NTP_DEMO_RX_BUFSIZE - data_len))
    /* 拷贝数据 */
    memcpy(ntp_demo_recvbuf + data_len, q->payload,
    (NTP_DEMO_RX_BUFSIZE - data_len));
    else memcpy(ntp_demo_recvbuf + data_len,
    q->payload, q->len);

    data_len += q->len;
    /* 超出 TCP 客户端接收数组,跳出 */
    if (data_len > NTP_DEMO_RX_BUFSIZE) break;
}

data_len = 0; /* 复制完成后 data_len 要清零 */
/*从 NTP 服务器获取时间*/
atk_get_seconds_from_ntp_server(ntp_demo_recvbuf, 40);
printf("北京时间: %02d-%02d-%02d %02d:%02d:%02d\r\n",
    nowdate.year,
    nowdate.month + 1,
    nowdate.day,
    nowdate.hour + 8,
    nowdate.minute,
    nowdate.second);
sprintf((char*)lwip_time_buf,
    "Beijing time:%02d-%02d-%02d %02d:%02d:%02d",
    nowdate.year,
    nowdate.month + 1,
    nowdate.day,
    nowdate.hour + 8,
    nowdate.minute,
    nowdate.second);
lcd_show_string(5, 170, lcddev.width, 16, 16,
    (char*)lwip_time_buf, RED);
netbuf_delete(recvbuf); /* 删除 buf */
}

```

```

        else vTaskDelay(5); /* 延时 5ms */
    }
}

else printf("NTP 绑定失败\r\n");
}

else printf("NTP 连接创建失败\r\n");
}
}

```

23.2.3 下载验证

编译代码并下载到开发板中，打开串口调式助手查看当前时间如下图所示。

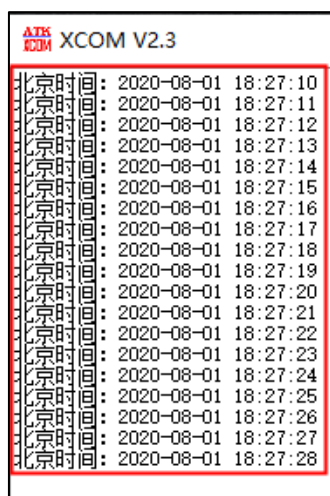


图 23.2.3.1 NTP 获取时间成功

第二十四章 lwIP 测试网速

我们为什么测试网速呢？原因很简单，在我们开发时候，有一些特殊的原因导致掉包、堵塞、延迟抖动等情况，一般都是发送和接收速率的问题，如果网速偏低或者达不到 PHY 芯片的最大网速，则开发过程中会遇到很多的问题。本章分为如下几个部分：

24.1 JPerf 网络测速工具

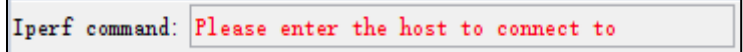
24.2 JPerf 网络实验

24.1 JPerf 网络测速工具

JPerf 网络测速工具是一个跨平台的网络性能测试工具，它支持 Win/Linux/Mac/Android/iOS 等多个平台，它也可以测试最大 TCP 和 UDP 带宽性能，具有多种参数和 UDP 特性，可以根据需要调整，可以报告带宽、延迟抖动和数据包丢失，该软件下载地址是：<https://iperf.fr/iperf-download.php>。

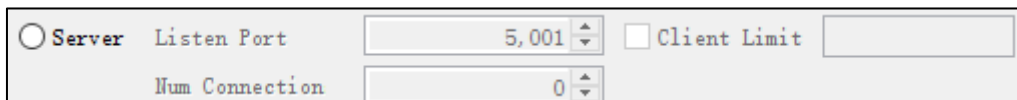
下载完成之后打开该软件，可以看到该软件划分为几个区域，这些区域的作用如下所示：

1. Iperf 命令行（无法直接输入）：



2. 服务端设置：

- (1) Listen Port 监听端口。
- (2) client limit:客户端限制，仅允许指定客户端连接。
- (3) Num Connections: 最大允许连接的数量，为 0 不限制。



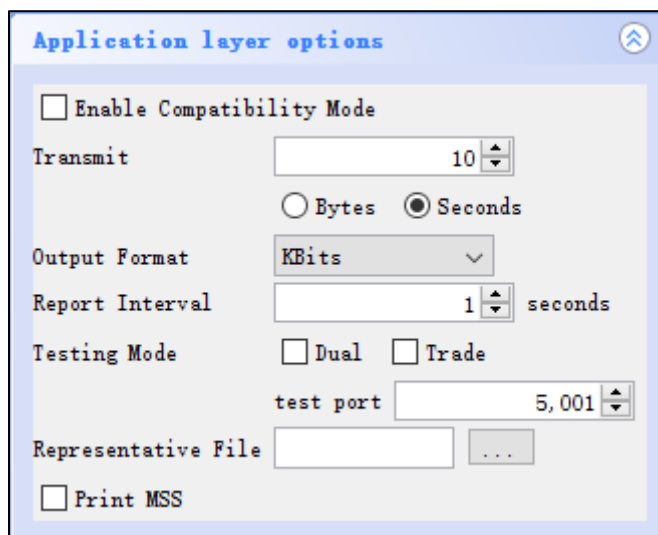
3. 客户端设置：

- (1) Server address 表示服务器地址。
- (2) Port 表示端口。
- (3) Parallel Streams 表示并发流。



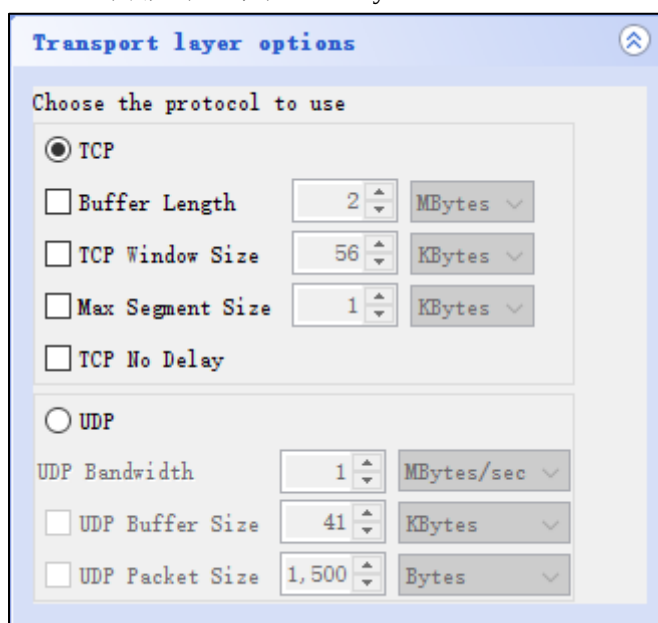
4. 应用层设置：

- (1) Enable Compatibility Mode 兼容旧版本（当 server 端和 client 端版本不一样时使用）。
- (2) Transmit 设置测试模式：传输字节总量大小 10Bytes 或者按传输时间总长度 10 秒。
- (3) Dual 复选框勾上表示同时进行双向传输测试。
- (4) Trade 复选框勾上表示单独进行双向传输测试，先测 c 到 s 的带宽。
- (5) Representative File 表示指定需要传输的文件。
- (6) Print MSS 表示显示 tcp 最大 mtu 值。



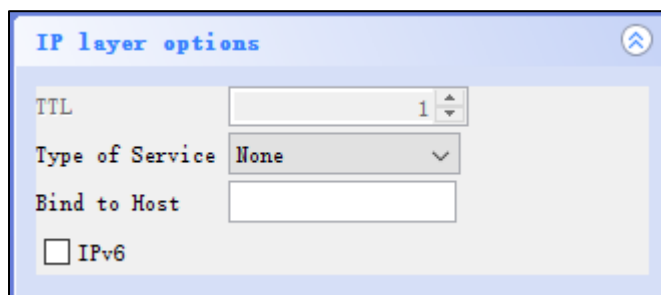
5. 传输层设置:

- (1) TCP 协议-设置缓冲区大小。
- (2) TCP 协议-指定 TCP 窗口大小。
- (3) TCP 协议-设定 TCP 数据包的最大 mtu 值。
- (4) TCP 协议-设定 TCP 不延时。
- (5) UDP 协议-设置 UDP 最大带宽。
- (6) UDP 协议-设置 UDP 缓冲区。
- (7) UDP 协议-UDP 包封装大小: 默认 1470 byte。

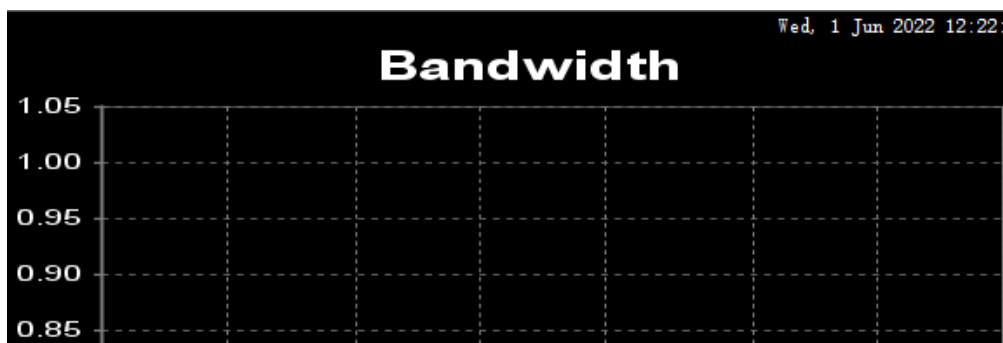


6. IP 层设置:

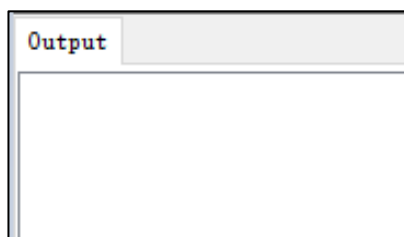
- (1) TTL 表示指定 ttl 值。
- (2) Type of Service 表示服务类型 (Type of Service, ToS), 大小范围从 0x10 (最小延迟) 到 0x2 (最少费用) 在使用 802.11e 来控制服务质量的 WLAN 中, ToS 是映射在 Wi-Fi 多媒体(WMM)存取范畴的。



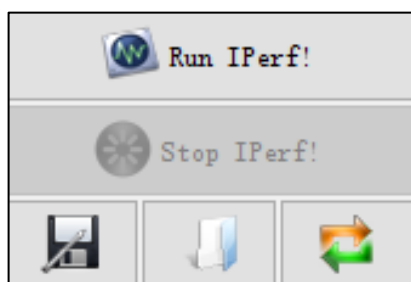
7. 网速显示窗口：折线图的形式显示



8. 网速相关数据输出窗口：以文本的形式输出



9. 开始和停止 JPerf



24.2 JPerf 网络实验

24.2.1 硬件设计

1. 例程功能

本实验的目标是：利用软件 JPerf 测试 PHY 的网速。

《lwIP 例程 15 lwIP 测试接收速度实验(裸机)》。

24.2.2 软件设计

24.2.2.1 程序解析

测试开发板收发速度的代码很简单，只需要移植 lwip-2.1.2\src\apps\lwiperf 的文件到工程中，接着在 lwip_demo.c 文件下添加以下源码，如下所示：

```
/* 报告状态 */
const char *report_type_str[] =
{
    "TCP_DONE_SERVER",          /* LWIPERF_TCP_DONE_SERVER*/
    "TCP_DONE_CLIENT",          /* LWIPERF_TCP_DONE_CLIENT*/
    "TCP_ABORTED_LOCAL",        /* LWIPERF_TCP_ABORTED_LOCAL */
    "TCP_ABORTED_LOCAL_DATAERROR", /*LWIPERF_TCP_ABORTED_LOCAL_DATAERROR*/
    "TCP_ABORTED_LOCAL_TXERROR", /* LWIPERF_TCP_ABORTED_LOCAL_TXERROR */
    "TCP_ABORTED_REMOTE",       /* LWIPERF_TCP_ABORTED_REMOTE */
    "UDP_STARTED",              /* LWIPERF_UDP_STARTED,*/
    "UDP_DONE",                 /* LWIPERF_UDP_DONE */
    "UDP_ABORTED_LOCAL",        /* LWIPERF_UDP_ABORTED_LOCAL*/
    "UDP_ABORTED_REMOTE"       /* LWIPERF_UDP_ABORTED_REMOTE */
};

/* 当测试结束以后会调用此函数，此函数用来报告测试结果 */
static void lwiperf_report(void *arg,
                           enum lwiperf_report_type report_type,
                           const ip_addr_t *local_addr,
                           u16_t local_port,
                           const ip_addr_t *remote_addr,
                           u16_t remote_port,
                           u32_t bytes_transferred,
                           u32_t ms_duration,
                           u32_t bandwidth_kbitpsec)
{
    printf("-----\r\n");
    if ((report_type < (sizeof(report_type_str)/sizeof(report_type_str[0])) \
        && local_addr && remote_addr)
    {
        printf(" %s \r\n", report_type_str[report_type]);
        printf(" Local address : %u.%u.%u.%u ", ((u8_t *)local_addr)[0], \
            ((u8_t *)local_addr)[1], \
            ((u8_t *)local_addr)[2], \
            ((u8_t *)local_addr)[3]);
        printf(" Port %d \r\n", local_port);
        printf(" Remote address : %u.%u.%u.%u ", ((u8_t *)remote_addr)[0], \
            ((u8_t *)remote_addr)[1], \
```

```

((u8_t *)remote_addr)[2],\
((u8_t *)remote_addr)[3]);

printf(" Port %d \r\n", remote_port);
printf(" Bytes Transferred %d \r\n", bytes_transferred);
printf(" Duration (ms) %d \r\n", ms_duration);
printf(" Bandwidth (kbitpsec) %d \r\n", bandwidth_kbitpsec);
}
else
{
printf(" IPERF Report error\r\n");
}
}

/**
 * @brief      lwip_demo 实验入口
 * @param      无
 * @retval     无
 */
void lwip_demo(void)
{
uint8_t t = 0;
if(lwiperf_start_tcp_server_default(lwiperf_report,NULL))
{
printf("\r\n*****\r\n");
printf(" IPERF Server example\r\n");
printf("*****\r\n");
printf(" IPv4 Address      : %u.%u.%u.%u\r\n", lwipdev.ip[0],
lwipdev.ip[1],
lwipdev.ip[2],
lwipdev.ip[3]);

printf(" IPv4 Subnet mask : %u.%u.%u.%u\r\n", lwipdev.netmask[0],
lwipdev.netmask[1],
lwipdev.netmask[2],
lwipdev.netmask[3]);

printf(" IPv4 Gateway      : %u.%u.%u.%u\r\n", lwipdev.gateway[0],
lwipdev.gateway[1],
lwipdev.gateway[2],
lwipdev.gateway[3]);

printf("*****\r\n");
}
else
{
printf("IPERF initialization failed!\r\n");
}
}

```



```

}

while(1)
{
    vTaskDelay(5);
}
}

```

测试网速的相关原理这里笔者不会讲解，有兴趣的小伙伴可以看一下 lwiperf.c/h 文件。

24.2.3 下载验证

编译程序并下载到开发板上，双击 jperf.bat，填写 IP 地址与端口号，如下所示：

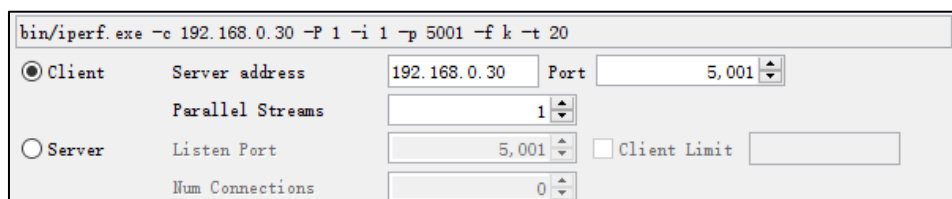


图 24.2.3.1 测试网速的 IP 地址和端口号

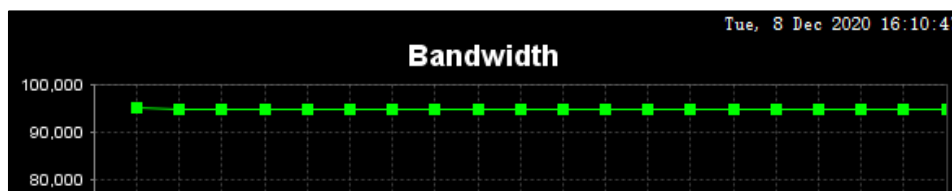


图 24.2.3.2 网速波形图

可以看到，我们的网速接近 95M，虽然离 100M 有一点点差距，但是速率受很多因素影响。提高网速的速率可设置以下几个配置项，如下所示：

```

/* 堆内存的大小，如果需要更大的堆内存，那么设置高一点 */
#define MEM_SIZE (30*1024)

/* MEMP_NUM_PBUF: 设置内存池的数量 */
#define MEMP_NUM_PBUF 25

/* MEMP_NUM_UDP_PCB: UDP 协议控制块的数量. */
#define MEMP_NUM_UDP_PCB 4

/* MEMP_NUM_TCP_PCB: TCP 的数量. */
#define MEMP_NUM_TCP_PCB 4

/* MEMP_NUM_TCP_PCB_LISTEN: 监听 TCP 的数量. */
#define MEMP_NUM_TCP_PCB_LISTEN 2

/* MEMP_NUM_TCP_SEG: 同时排队的 TCP 的数量段. */
#define MEMP_NUM_TCP_SEG 150

/* MEMP_NUM_SYS_TIMEOUT: 超时模拟活动的数量. */
#define MEMP_NUM_SYS_TIMEOUT 6

/* ----- Pbuf 选项 ----- */
/* PBUF_POOL 内存池中每个内存块大小 */

```

```
#define PBUF_POOL_SIZE                20

/* PBUF_POOL_BUFSIZE: pbuf 池中每个 pbuf 的大小. */
#define PBUF_POOL_BUFSIZE

LWIP_MEM_ALIGN_SIZE(TCP_MSS+40+PBUF_LINK_ENCAPSULATION_HLEN+PBUF_LINK_HLEN)

/* TCP 接收窗口 */
#define TCP_WND                        (20*TCP_MSS)
```

第二十五章 基于 MQTT 协议连接阿里云服务器

本章主要学习 lwIP 提供的 MQTT 协议文件使用，通过 MQTT 协议将设备连接到阿里云服务器，实现远程互通。由于 MQTT 协议是基于 TCP 的协议实现的，所以我们只需要在单片机端实现 TCP 客户端程序并使用 lwIP 提供的 MQTT 文件来连接阿里云服务器。本章分为如下几个部分：

25.1 MQTT 协议简介

25.1 MQTT 协议实验

25.1 MQTT 协议简介

(1) MQTT 是什么？

MQTT (Message Queuing Telemetry Transport, 消息队列遥测传输协议)，是一种基于发布/订阅 (Publish/Subscribe) 模式的轻量级通讯协议，该协议构建于 TCP/IP 协议上，由 IBM 在 1999 年发布，目前最新版本为 v3.1.1。MQTT 最大的优点在于可以以极少的代码和有限的带宽，为远程设备提供实时可靠的消息服务。做为一种低开销、低带宽占用的即时通讯协议，MQTT 在物联网、小型设备、移动应用等方面有广泛的应用，MQTT 协议属于应用层。

(2) MQTT 协议特点

MQTT 是一个基于客户端与服务器的消息发布/订阅传输协议。MQTT 协议是轻量、简单开放和易于实现的，这些特点使它适用范围非常广泛。在很多情况下，包括受限境中，如：机器与机器 (M2M) 通信和物联网 (IoT)。其在，通过卫星链路通信传感器、医疗设备、智能家居、及一些小型化设备中已广泛使用。

(3) MQTT 协议原理及实现方式

实现 MQTT 协议需要：客户端和服务端 MQTT 协议中有三种身份：发布者 (Publish)、代理 (Broker) (服务器)、订阅者 (Subscribe)。其中，消息的发布者和订阅者都是客户端，消息代理是服务器，消息发布者可以同时是订阅者，如下图所示。

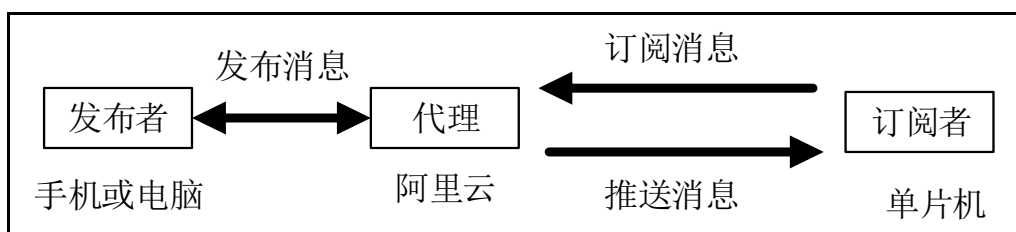


图 25.1.1 MQTT 订阅和发布过程

MQTT 传输的消息分为：主题 (Topic) 和消息的内容 (payload) 两部分。

Topic: 可以理解为消息的类型，订阅者订阅 (Subscribe) 后，就会收到该主题的消息内容 (payload)。

Payload: 可以理解为消息的内容，是指订阅者具体要使用的内容。

25.1.1 MQTT 协议实现原理

1. 要在客户端与代理服务端建立一个 TCP 连接，建立连接的过程是由客户端主动发起的，代理服务一直是处于指定端口的监听状态，当监听到有客户端要接入的时候，就会立刻去处理。客户端在发起连接请求时，携带客户端 ID、账号、密码（无账号密码使用除外，正式项目不

会允许这样)、心跳间隔时间等数据。代理服务收到后检查自己的连接权限配置中是否允许该账号密码连接,如果允许则建立会话标识并保存,绑定客户端 ID 与会话,并记录心跳间隔时间(判断是否掉线和启动遗嘱时用)和遗嘱消息等,然后回发连接成功确认消息给客户端,客户端收到连接成功的确认消息后,进入下一步(通常是开始订阅主题,如果不需要订阅则跳过)。如下图所示。

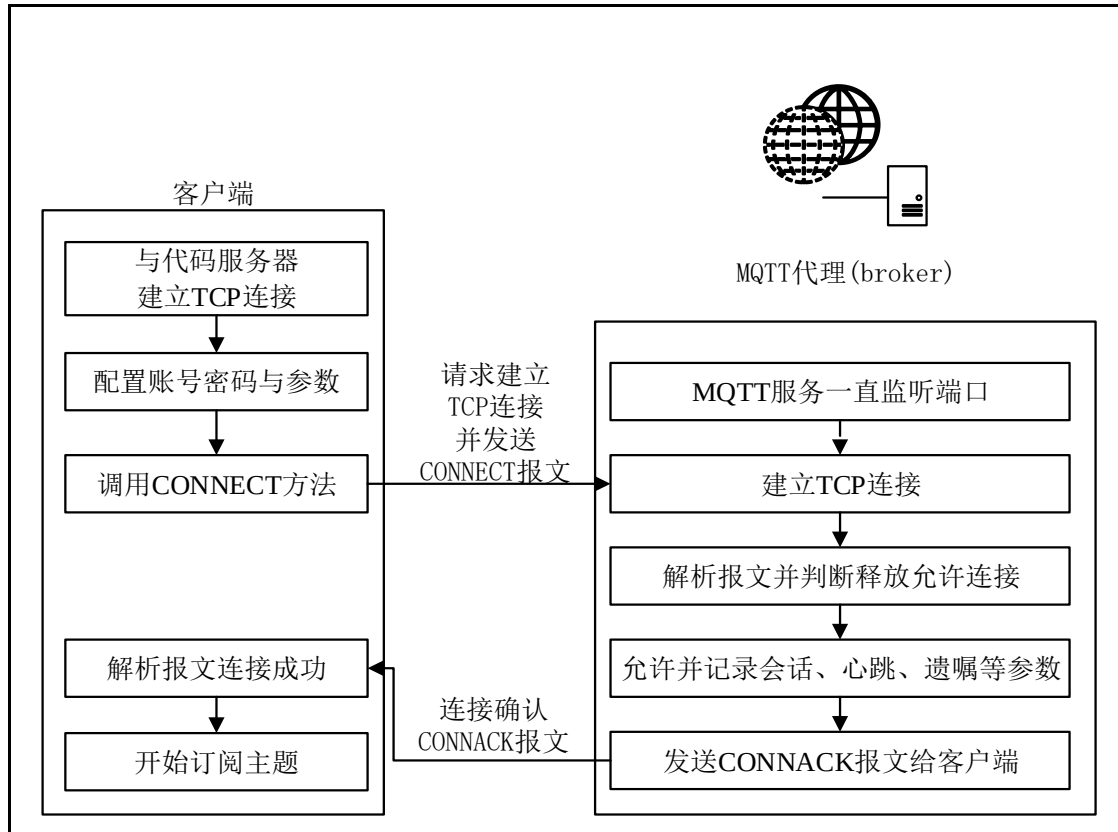


图 25.1.1.1 客户端与代理服务器建立连接示意图

2. 客户端将需要订阅的主题经过 SUBSCRIBE 报文发送给代理服务,代理服务则将这个主题记录到该客户端 ID 下(以后有这个主题发布就会发送给该客户端),然后回复确认消息 SUBACK 报文,客户端接到 SUBACK 报文后知道已经订阅成功,则处于等待监听代理服务推送的消息,也可以继续订阅其他主题或发布主题,如下图所示。

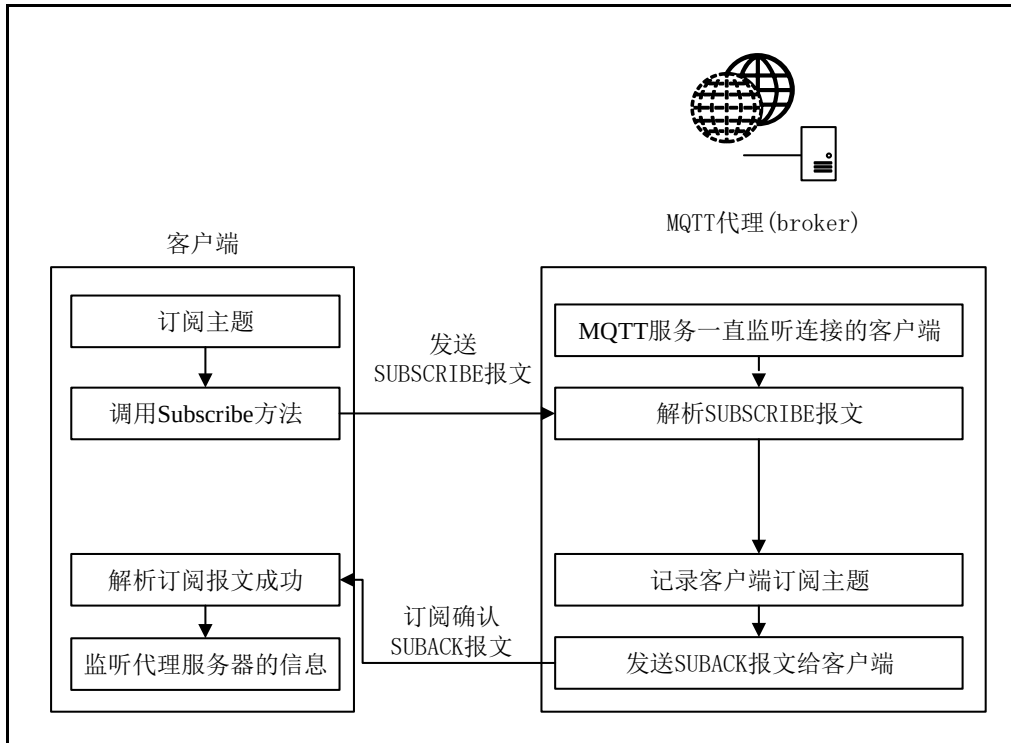


图 25.1.1.2 客户端向服务器订阅示意图

3. 当某一客户端发布一个主题到代理服务后，代理服务先回复该客户端收到主题的确认消息，该客户端收到确认后就可以继续自己的逻辑了。但这时主题消息还没有发给订阅了这个主题的客户端，代理要根据质量级别（QoS）来决定怎样处理这个主题。所以这里充分体现了是MQTT 协议是异步通信模式，不是立即端到端反应的，如下图所示。

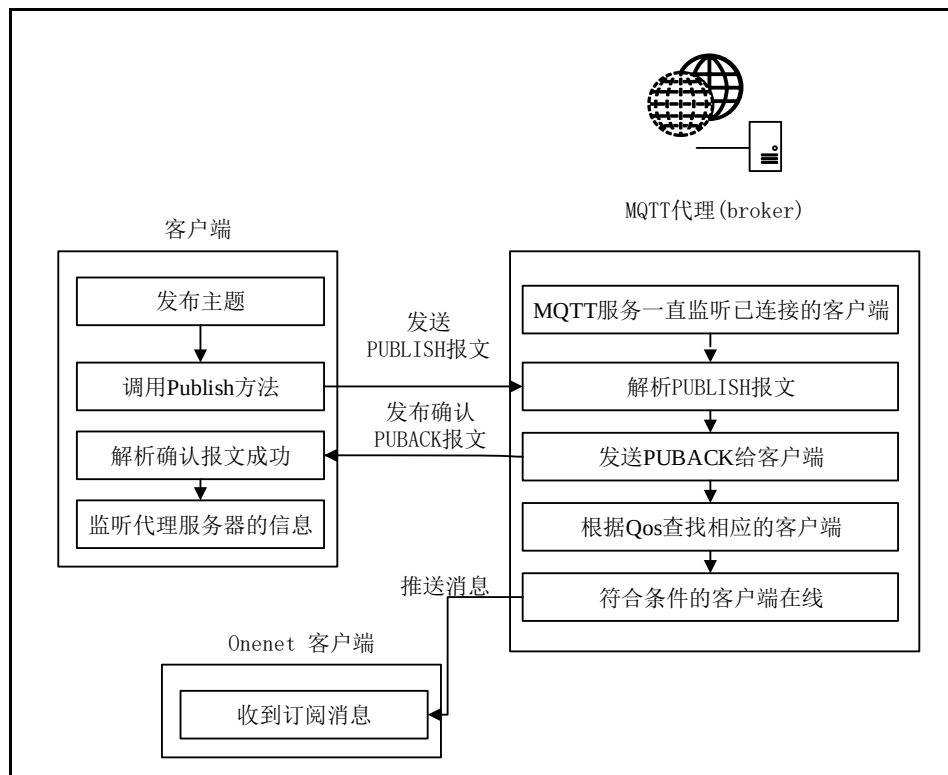


图 25.1.1.3 客户端向代理服务器发送主题

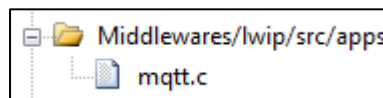
如果发布和订阅时的质量级别 QoS 都是至多一次，那代理服务则检查当前订阅这个主题的客户端是否在线，在线则转发一次，收到与否不再做任何处理。这种质量对系统压力最小。

如果发布和订阅时的质量级别 QoS 都是至少一次，那要保证代理服务和订阅的客户端都有成功收到才可以，否则会尝试补充发送（具体机制后面讨论）。这也可能会出现同一主题多次重复发送的情况。这种质量对系统压力较大。

如果发布和订阅时的质量级别 QoS 都是只有一次，那要保证代理服务和订阅的客户端都有成功收到，并只收到一次不会重复发送（具体机制后面讨论）。这种质量对系统压力最大。

25.1.2 移植 MQTT 协议

其实移植 lwIP 的 MQTT 文件是非常简单的，只将 lwip\src\apps\mqtt 路径下的 mqtt.c 文件添加到工程当中，这里我们在工程中添加一个名为 Middlewares/lwip/src/apps 分组，该分组用来添加 lwIP 应用层的文件，如下图所示所示：



25.1.2.1 添加 lwIP 的 mqtt.c 文件

mqtt.c 文件是 lwIP 根据 MQTT 协议规则编写而来的，如果用户不使用这个文件，请自行移植 MQTT 协议包。

在 Middlewares/lwip/lwip_app 分组添加 hmac_sha1 和 sha1 文件，这些文件用来计算核心密钥，这两个文件可在阿里云官方下载。

25.1.3 配置远程服务器

配置阿里云服务器步骤

第一步：注册阿里云平台，打开产品分类/物联网 Iot/物联网应用开发，如下图所示。

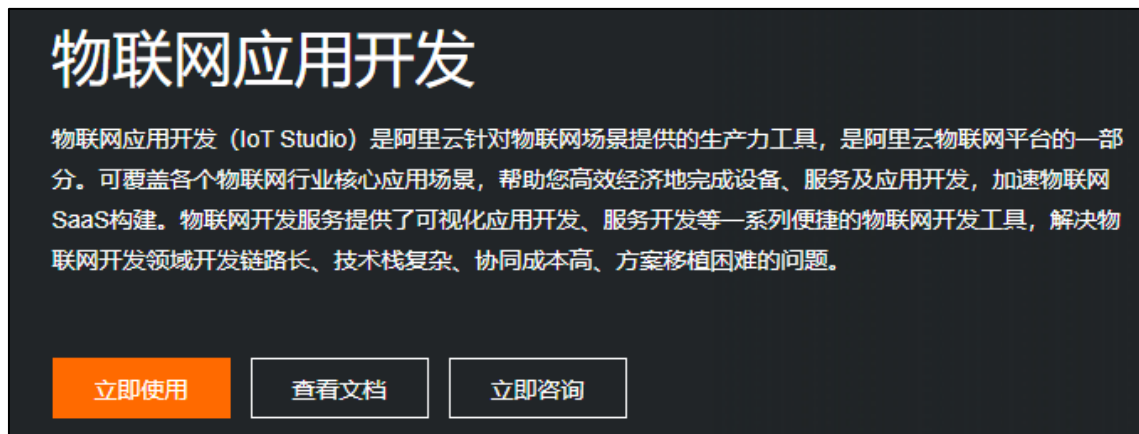


图 25.1.3.1 打开物联网应用开发

点击上图中的“立刻使用”按钮进去物联网应用开发页面。

第二步：在物联网应用开发页面下点击项目管理/新建项目/新建空白项目，在此界面下填写项目名称等相关信息，如下图所示。



新建空白项目

* 项目名称 ?

名称

描述

请输入内容

0/100

确认 取消

图 25.1.3.2 新建空白项目

创建项目完成之后在项目管理页面下点击项目进去子项目管理界面，如下图所示。



25.1.3.3 子项目管理页面

第三步：在上图中点击产品，如下图所示。



* 节点类型

直连设备 网关子设备 网关设备

连网与数据

* 连网方式

以太网

* 数据格式 ?

ICA 标准数据格式 (Alink JSON)

* 认证方式 ?

X.509 证书

* 使用私有 CA 证书

☐ 是 ☒ 否

图 25.1.3.4 产品参数填写

注：上图中的节点类型、连网方式、数据格式以及认证模式的选择，其他产品参数根据用户爱好设置。

第三步：创建产品之后点击图 26.1.3.3 中的设备选项添加设备，如下图所示。



添加设备 ?

i 特别说明：DeviceName 可以为空，当为空时，阿里云会颁发产品下的唯一标识符作为 DeviceName。

产品

请选择产品

ali_server_test

请输入 DeviceName

图 25.1.3.5 填写设备参数

第五步：在设备页面下找到我们刚刚创建的设备，如下图所示。

ProductKey	a1S0GvSCHqT 复制
DeviceName	ATK_MQTT_DEV 复制
DeviceSecret	39e260510264457f9a67c12b21eab692 复制

图 25.1.3.5 设备信息

这三个参数非常重要!!!!!!!!!!!!!!，在本章实验中会用到。

第六步：打开“产品/查看/功能定义”路径，在该路径下添加功能定义，如下图所示。

产品信息	Topic 类列表	功能定义	数据解析	服务端订阅	设备开发
当前展示的是已发布到线上的功能定义，如需修改，请点击 编辑草稿					

图 25.1.3.6 添加功能

第七步：打开自定义功能并发布上线，这里我们添加了两个 CurrentTemperature 和 RelativeHumidity 标签。

225.2 阿里云 MQTT 协议实验

25.2.1 硬件设计

1. 例程功能

本章的目标是，lwIP 连接阿里云实现数据上存。

该实验的实验工程，请参考《lwIP 例程 17 lwIP_Aliyun_MQTT 实验》。

25.2.2 软件设计

25.2.2.1 MQTT 配置步骤

1) 配置 MCU 为 TCP 客户端模式

配置为 TCP 客户端等步骤，请参考第 21 章。

2) DNS 解析阿里云网页转成 IP 地址

调用函数 gethostbyname 获取 DNS 解析的 IP 地址。

3) MQTT 连接

调用函数 mqtt_client_connect 连接服务器。

4) 连接状态

对服务器发布和订阅操作。

5) 循环发布数据到服务器当中

在 lwip_demo 函数的 while() 语句中定时 1s 调用函数 mqtt_publish 发布数据至服务器。

25.2.2.2 程序流程图

本实验的程序流程图，如下图所示。

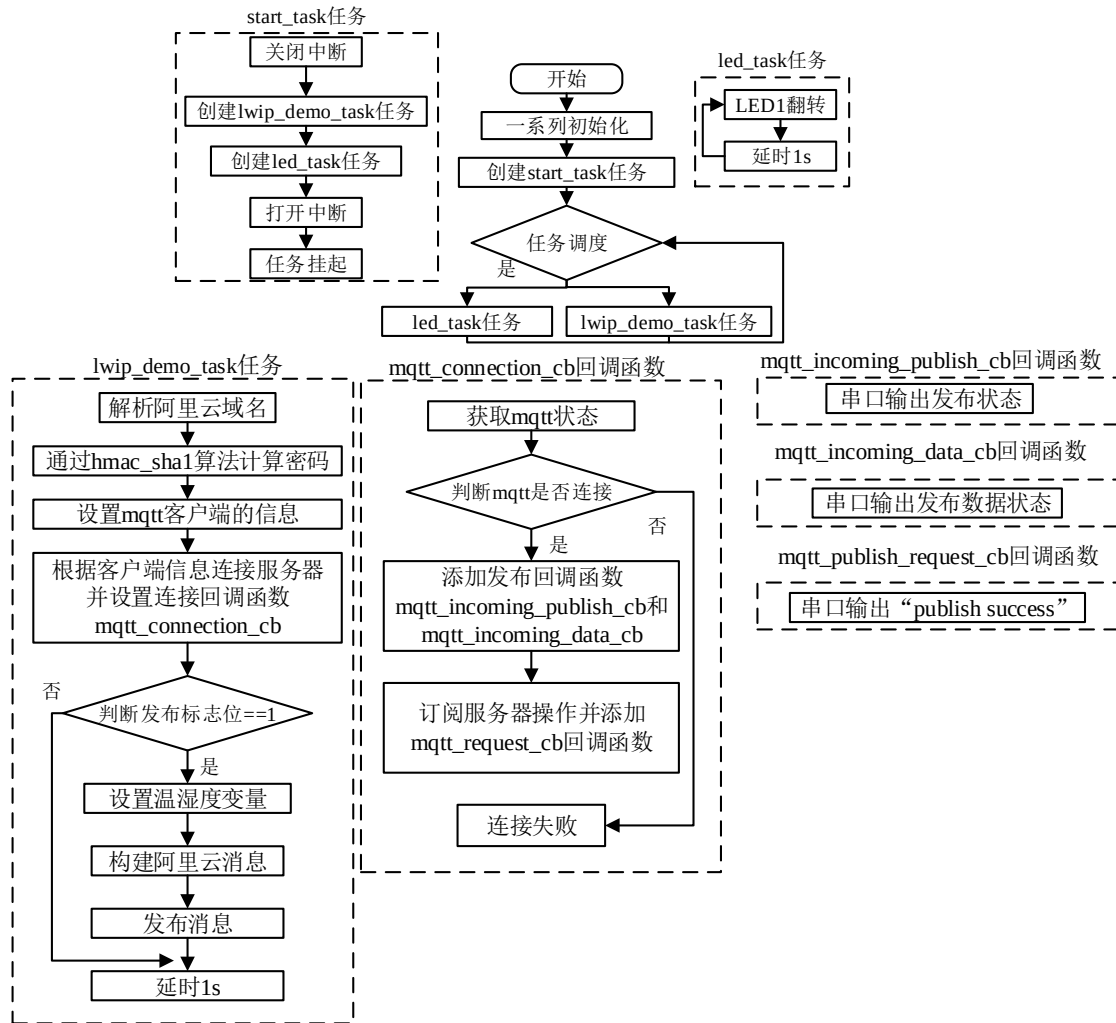


图 25.2.2.2.1 连接阿里云程序流程图

25.2.2.3 程序解析

我们打开 lwip_deom.h 文件，在这个文件中我们定义了阿里云服务器创建设备的配置项，另外还声明了 lwip_demo 函数，关于阿里云服务器的 MQTT 主题请大家查看阿里云相关手册。重点关注的是 lwip_deom.c 这个文件，在这个文件定义了 8 个函数，如下表所示。

函数	描述
lwip_demo()	解析域名、配置 mqtt 客户端、连接阿里云服务器的操作
lwip_ali_hextostr()	将 16 进制数转化为字符串
lwip_ali_get_password()	通过 hmac_sha1 算法获取 password
mqtt_connection_cb()	mqtt 连接回调函数
mqtt_request_cb()	mqtt 订阅响应回调函数
mqtt_publish_request_cb()	mqtt 发布响应回调函数
mqtt_incoming_publish_cb()	mqtt 进入发布回调函数
mqtt_incoming_data_cb()	mqtt 进入数据回调函数

表 25.2.2.3.1 阿里云 MQTT 实验函数

我们首先看一下 lwip_demo 函数，该函数的代码如下。

/**

```

* @brief      lwip_demo 进程
* @param      无
* @retval     无
*/
void lwip_demo(void)
{
    struct hostent *server;
    static struct mqtt_connect_client_info_t mqtt_client_info;

    server = gethostbyname((char *)HOST_NAME);      /* 对 oneNET 服务器地址解析 */
    /* 把解析好的地址存放在 mqtt_ip 变量当中 */
    memcpy(&mqtt_ip, server->h_addr, server->h_length);

    char *PASSWORD;
    PASSWORD = mymalloc(SRAMIN, 300); /* 为密码申请内存 */
    /* 通过 hmac_sha1 算法得到 password */
    lwip_alib_get_password(DEVICE_SECRET, CONTENT, PASSWORD);

    /* 设置一个空的客户端信息结构 */
    memset(&mqtt_client_info, 0, sizeof(mqtt_client_info));

    /* 设置客户端的信息量 */
    mqtt_client_info.client_id = (char *)CLIENT_ID;      /* 设备名称 */
    mqtt_client_info.client_user = (char *)USER_NAME;    /* 产品 ID */
    mqtt_client_info.client_pass = (char *)PASSWORD;     /* 计算出来的密码 */
    mqtt_client_info.keep_alive = 100;                   /* 保活时间 */
    mqtt_client_info.will_msg = NULL;
    mqtt_client_info.will_qos = NULL;
    mqtt_client_info.will_retain = 0;
    mqtt_client_info.will_topic = 0;

    myfree(SRAMIN, PASSWORD);                          /* 释放内存 */

    /* 创建 MQTT 客户端控制块 */
    mqtt_client = mqtt_client_new();

    /* 连接服务器 */
    mqtt_client_connect(mqtt_client,                  /* 服务器控制块 */
                        &mqtt_ip, MQTT_PORT,        /* 服务器 IP 与端口号 */
                        mqtt_connection_cb,
                        /* 设置服务器连接回调函数 */

```

```

        LWIP_CONST_CAST(void*, &mqtt_client_info),
        &mqtt_client_info); /* MQTT 连接信息 */

while(1)
{
    if (publish_flag == 1)
    {
        temp = 30 + rand() % 10 + 1; /* 温度的数据 */
        humid = 54.8 + rand() % 10 + 1; /* 湿度的数据 */
        sprintf((char *)payload_out,
            "{\"params\":{\"CurrentTemperature\":+
            %0.1f,\"RelativeHumidity\":%0.1f},\"method\":
            \"thing.event.property.post\"}", temp, humid);
        payload_out_len = strlen((char *)payload_out);
        mqtt_publish(mqtt_client, DEVICE_PUBLISH, payload_out,
            payload_out_len, 1, 0, mqtt_publish_request_cb, NULL);
    }

    vTaskDelay(1000);
}
}

```

此函数非常简单，首先我们调用 `gethostbyname` 函数解析阿里云的域名，根据这个域名来连接阿里云服务器，其次使用一个结构体配置 MQTT 客户端的信息并调用 `mqtt_client_new` 函数创建 MQTT 服务器控制块，接着我们调用 `mqtt_client_connect` 函数连接阿里云服务器并添加 `mqtt_connection_cb` 连接回调函数，最后在 `while()` 语句中判断是否订阅操作成功，如果系统订阅成功，则构建 MQTT 消息，并调用 `mqtt_publish` 函数发布。

接下来我们来讲解一下 `mqtt_client_connect` 函数的作用，如下源码所示：

```

/**
 * @brief      mqtt 连接回调函数
 * @param      client: 客户端控制块
 * @param      arg: 传入的参数
 * @param      status: 连接状态
 * @retval     无
 */
static void
mqtt_connection_cb( mqtt_client_t *client, void *arg,
                    mqtt_connection_status_t status)
{
    err_t err;

    const struct mqtt_connect_client_info_t* client_info =
        (const struct mqtt_connect_client_info_t*)arg;

    LWIP_UNUSED_ARG(client);
}

```

```
printf("\r\nMQTT client \"%s\" connection cb: status %d\r\n",
        client_info->client_id, (int)status);

/* 判断是否连接 */
if (status == MQTT_CONNECT_ACCEPTED)
{
    /* 判断是否连接 */
    if (mqtt_client_is_connected(client))
    {
        /* 设置传入发布请求的回调 */
        mqtt_set_inpub_callback(mqtt_client,
                                mqtt_incoming_publish_cb,
                                mqtt_incoming_data_cb,
                                NULL);

        /* 订阅操作，并设置订阅响应回调函数 mqtt_sub_request_cb */
        err = mqtt_subscribe(client, DEVICE_SUBSCRIBE, 1,
                              mqtt_request_cb, arg);

        if(err == ERR_OK)
        {
            printf("mqtt_subscribe return: %d\n", err);
            lcd_show_string(5, 170, 210, 16, 16,
                            "mqtt_subscribe succeed", BLUE);
        }
    }
}
else /* 连接失败 */
{
    printf("mqtt_connection_cb: Disconnected, reason: %d\n", status);
}
}
```

此函数也是非常简单，它主要调用函数 `mqtt_client_is_connected` 判断是否已经连接服务器，如果连接成功，则程序调用函数 `mqtt_set_inpub_callback` 添加 `mqtt_incoming_publish_cb` 和 `mqtt_incoming_data_cb` 回调函数，这些回调函数需要根据客户端以及服务器的发布操作才能进去该回调函数，最后我们调用函数 `mqtt_subscribe` 对服务器进行订阅操作并且添加 `mqtt_request_cb` 订阅响应回调函数。

25.2.3 下载验证

下载完代码后，在浏览器上打开阿里云平台，并在指定的网页查看上存数据，如下图所示。

温度	查看数据	湿度	查看数据
32.0 °C		62.8 %	
2020/12/09 13:01:06		2020/12/09 13:01:06	

图 25.2.3.1 温湿度显示

第二十六章 基于 MQTT 协议连接 OneNET 服务器

本章主要介绍 lwIP 如何通过 MQTT 协议将设备连接到 OneNET 平台，并通过 MQTT 协议远程互通。关于 MQTT 协议的知识，请参考第二十六章的内容。本章分为如下几个部分：

26.1 配置 OneNET 平台

26.2 工程配置

26.3 基于 OneNET 平台 MQTT 实验

26.1 配置 OneNET 平台

配置 OneNET 服务器步骤：

第一步：首先打开 OneNET 服务器并注册账号，注册之后在主界面下打开开发者中心页面，如下图所示。



图 26.1.1.1 MQTT 物联网套件（新版）

第二步：在上图中点击“产品开发”选项，页面跳转完成之后点击“创建产品”选项，此时该页面会弹出产品信息小界面，这里我们根据自己的项目填写相关的信息，如下图所示。

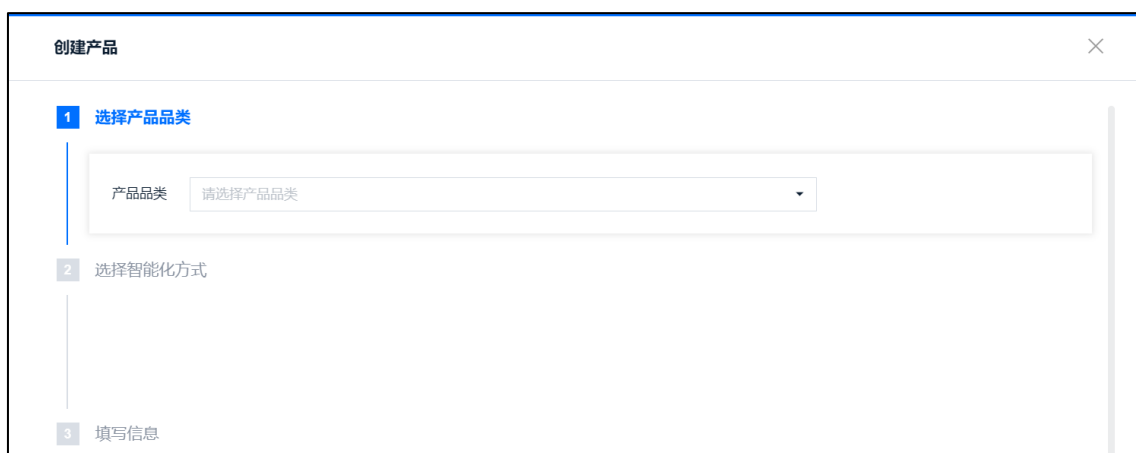


图 26.1.1.2 填写产品相关信息

到“选择智能化方式”时，我们选择设备接入，并编写产品信息，如下图所示。

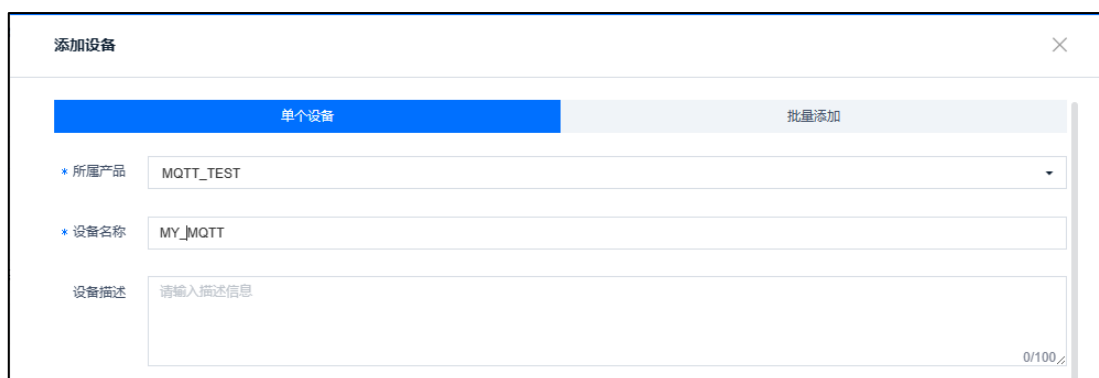


The image shows a configuration form for an MQTT product. It includes fields for '所属城市' (City), '节点类型' (Node Type) with radio buttons for '直连设备' (Selected), '网关设备' (Gateway), and '子设备' (Sub-device); '接入协议' (Access Protocol) set to 'MQTT'; '数据协议' (Data Protocol) set to '数据流' (Data Stream); '联网方式' (Networking Method) with radio buttons for '蜂窝' (Cellular), 'Wi-Fi', 'NB', '以太网' (Selected), and '其他' (Other); and '开发方案' (Development Solution) with radio buttons for '标准方案' (Selected) and '自定义方案' (Custom).

图 26.1.1.3 MQTT 产品信息

上图是作者选择的配置，值得注意的是，如果用户选择物模型数据交互，则设置数据协议为 OneJosn；如果用户选择数据流模型数据交互，则设置数据协议为数据流。

创建完成后，在产品开发下找到刚刚创建的产品，接着，我们点击该产品下的设备管理，进入添加设备界面，如下图所示。



The image shows the '添加设备' (Add Device) interface. It has two tabs: '单个设备' (Single Device) and '批量添加' (Batch Add). Under '单个设备', there are fields for '所属产品' (Product) set to 'MQTT_TEST', '设备名称' (Device Name) set to 'MY_MQTT', and a text area for '设备描述' (Device Description) with a character count of 0/100.

图 26.1.1.4 添加设备

第三步：创建设备成功后，点击设备右边的详情标签进入标签的链接页面，在这个页面下我们得到以下设备信息，如下图所示。



The image shows the 'MY_MQTT' device details page. It has tabs for '设备详情' (Device Details), '数据流' (Data Stream), '文件管理' (File Management), '命令下发' (Command Dispatch), and 'SIM卡查询' (SIM Card Query). The '设备信息' (Device Information) section shows: Device ID: 2132489619, Device Key: OVhEYzZBb0tpQUVoZWl0ZUYzTWVRKeTJ..., Creation Time: 2023-12-05 14:12:17, Activation Time: 2023-12-05 14:13:40, and OS Version: -. The '产品信息' (Product Information) section shows: Product ID: wKcHrEVL0n, Product Name: MQTT_TEST, Development Solution: 标准方案, and Node Type: 直连设备.

图 26.1.1.5 设备的 ID，设备名称和密钥 KEY

本实验会用到上图中的产品 ID (wKcHrEVL0n)、设备名称 MY_MQTT 以及“key”设备的密钥。

下面我们打开 OneNET 在线开发指南 (<https://open.iot.10086.cn/doc/v5/fuse/detail/919>), 在这个指南中找到服务器地址, 这些服务器地址就是 MQTT 服务器地址, 如下图所示。

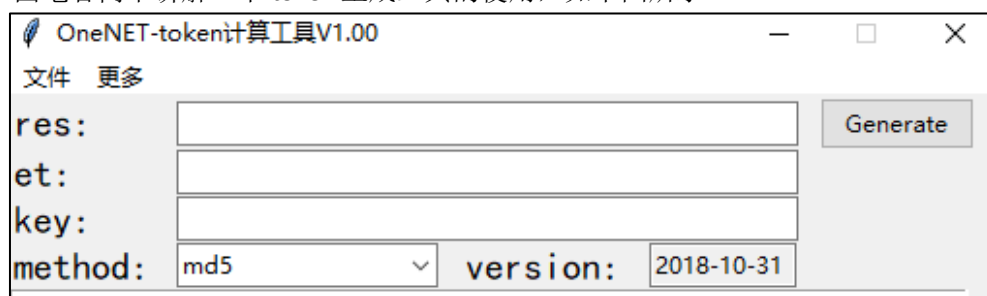
连接协议	证书	地址
MQTT	—	mqtt.heclouds.com:1883
MQTTS	证书下载	mqttstls.heclouds.com:8883

图 26.1.1.6 连接 OneNET 服务器的 IP 与端口

上图中, OneNET 的 MQTT 服务器具有两个连接方式, 一种是加密接口连接, 而另一种是非加密接口连接, 本章实验使用的是非加密接口连接 MQTT 服务器。

注: MQTT 物联网套件采用安全鉴权策略进行访问认证, 即通过核心密钥计算的 token 进行访问认证, 简单来讲, 用户想连接 OneNET 的 MQTT 服务器必须计算核心密钥, 这个密钥是根据我们前面创建的产品和设备相关的信息计算得来的, 密钥的计算方法可以使用 OneNET 提供的 token 生成工具计算, 该软件可在这个网址下载: <https://open.iot.10086.cn/doc/v5/development/detail/242>。

下面笔者简单讲解一下 token 生成工具的使用, 如下图所示。



OneNET-token计算工具V1.00

文件 更多

res:

et:

key:

method: version:

Generate

图 26.1.1.7 token 生成工具主界面

res: 输入格式为“products/{pid}/devices/{device_name}”, 这个输入格式中的“pid”就是我们 MQTT 产品 ID, 而“device_name”就是设备的名称。根据前面创建的产品和设备来填写 res 选项的参数, 如下图所示。



OneNET-token计算工具V1.00

文件 更多

res:

et:

key:

method: version:

Generate

图 26.1.1.8 填写 res 配置项

et: 访问过期时间 (expirationTime, unix) 时间, 这里笔者选择参考文档中的数值 (1672735919), 如下图所示。

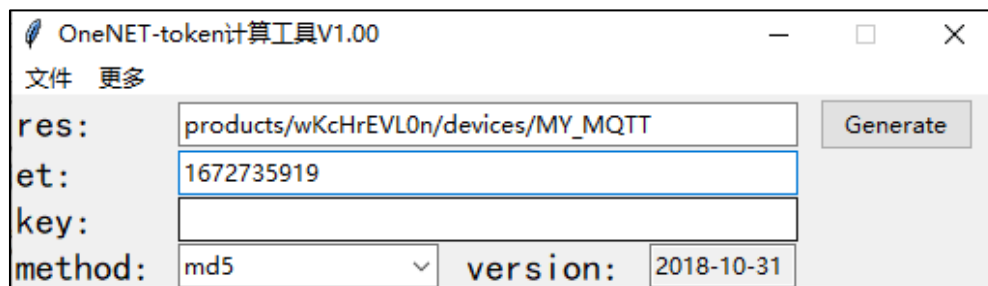


图 26.1.1.9 设置 et 配置项

key: 指选择设备的 key 密钥，如下图所示。



图 26.1.1.10 设置 key 配置项

最后按下上图中的“Generate”按钮生成核心密钥，如下图所示。

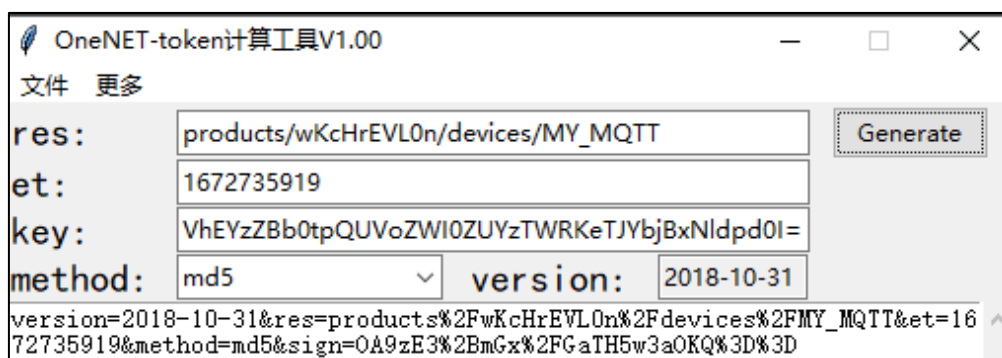


图 26.1.1.11 生成核心密钥

这个核心密钥会在 MQTT 客户端的结构体 client_pass 成员变量保存。

26.2 工程配置

小上节我们使用 token 生成工具根据产品信息以及设备信息来计算核心密钥，这样的方式导致每次创建一个设备都必须根据这个设备信息再一次计算核心密钥才能连接，这种方式会大大地降低我们的开发效率，为了解决这个问题，笔者使用另一个方法，那就是使用代码的方式计算核心密钥，它和上一章节中的方式不一样，因为阿里云和 OneNET 计算的方式不同，所以不能使用阿里云的那两个文件来计算 OneNET 的密钥。OneOS 源码中有几个文件是用来计算 MQTT 协议连接 OneNET 平台的核心密钥，这些文件在 oneos2.0\components\cloud\onenet\mqtt-kit\authorization 路径下，大家先下载 OneOS 源码并在该路径下复制这些文件到工程当中。

打开工程并在 Middlewares\lwip\lwip_app 分组下添加以下文件，如下图所示。

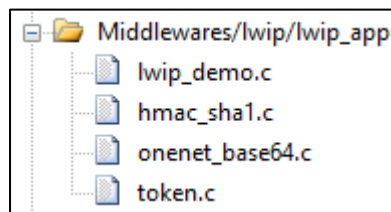


图 26.2.1 token 算法文件

这些文件都在 oneos2.0\components\cloud\onenet\mqtt-kit\authorization 路径下获取。

26.3 基于 OneNET 平台 MQTT 实验

26.3.1 硬件设计

1. 例程功能

本章目标是开发板使用 MQTT 协议连接 OneNET 服务器，并实现数据上存更新。

该实验的实验工程，请参考《lwIP 例程 18 lwIP_OneNET_MQTT 实验》。

26.3.2 软件设计

26.3.2.1 程序流程图

本实验的程序流程图，如下图所示。

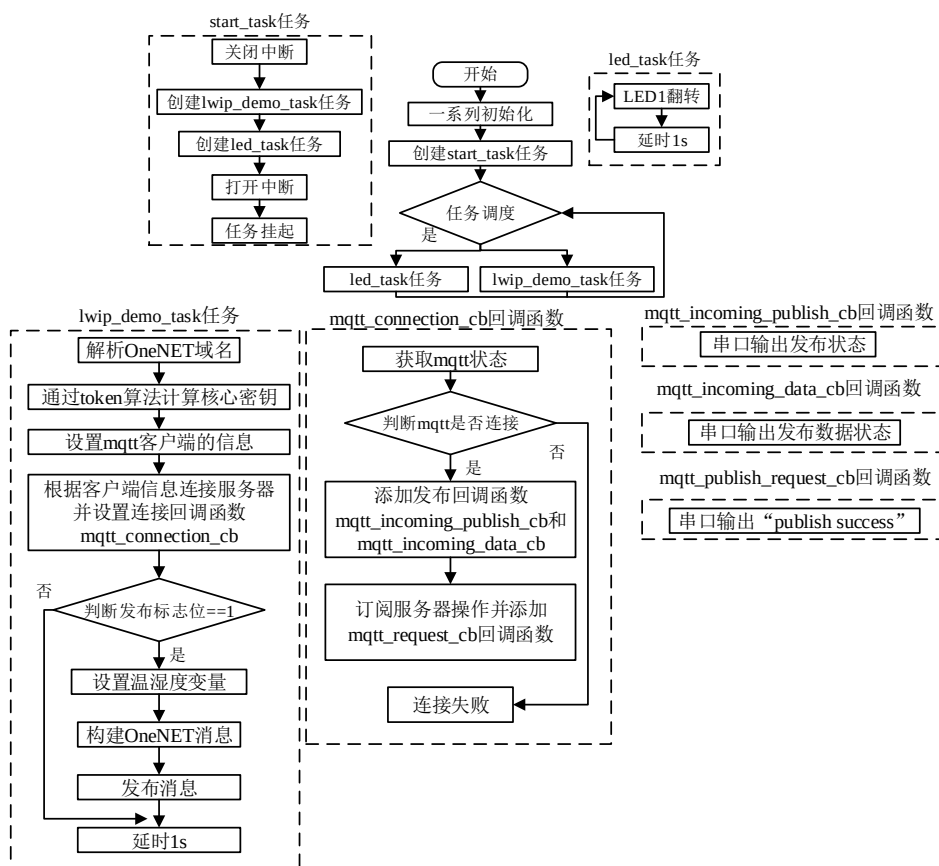


图 26.3.2.1.1 连接 OneNET 流程图

26.3.2.3 程序解析

我们打开 `lwip_deom.h` 文件，在这个文件中我们定义了 OneNET 服务器创建设备的配置项，另外还声明了 `lwip_demo` 函数，关于 OneNET 服务器的 MQTT 主题请大家查看 OneNET 相关手册，该手册地址为 <https://open.iot.10086.cn/doc/v5/develop/detail/251>，这个地址里面已经说明了 OneNET 的 MQTT 服务器相关主题信息。至于 `lwip_deom.c` 文件前面我们已经讲解过了，它们唯一不同的是计算核心密钥方式。

26.3.3 下载验证

我们编译代码，并把下载到开发板上运行，打开 OneNET 的 MQTT 服务器查看数据流展示，如下图所示。

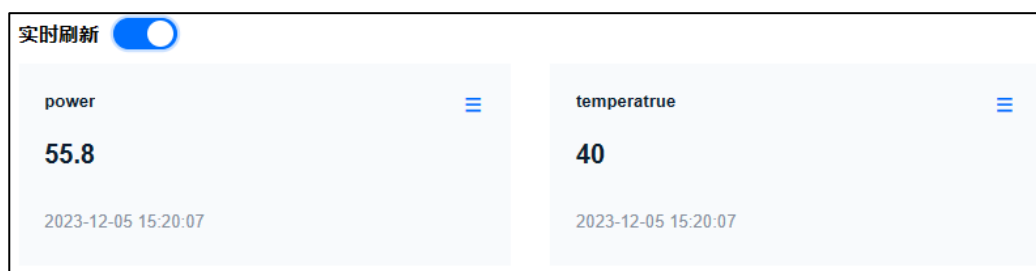


图 26.3.3.1 数据显示成功

第二十七章 HTTP 服务器实验

本章实验我们在开发板上搭建一个 HTTP 服务器，通过浏览器去访问我们的开发板，这个实验和第十四章的实验不同的是该实验使用字符串的形式描述网页数据，而第十四章的实验使用的是网页数组形式搭建 HTTP 服务器。本实验参考 contrib-2.1.0\apps\httpserver 路径下的 httpserver-netconn.c/h 下的例程。本章分为如下几个部分：

27.1 HTTP 协议简介

27.2 HTTP 服务器实验

27.1 HTTP 协议简介

HTTP 协议是 Hyper Text Transfer Protocol（超文本传输协议）的缩写,是用于从万维网（WWW:World Wide Web）服务器传输超文本到本地浏览器的传送协议。HTTP 是一种无状态协议，即服务器不保留与客户交易时的任何状态。这就大大减轻了服务器记忆负担，从而保持较快的响应速度。HTTP 是一种面向对象的协议。允许传送任意类型的数据对象。它通过数据类型和长度来标识所传送的数据内容和大小，并允许对数据进行压缩传送。当用户在一个 HTML 文档中定义了一个超文本链后，浏览器将通过 TCP/IP 协议与指定的服务器建立连接，如下所示：



图 27.1.1 HTTP 协议交互

HTTP：定义了与服务器交互的不同方法，其最基本的方法是 GET、PORT 和 HEAD。如下图所示。

- ① GET：从服务端获取数据。
- ② PORT：向服务器传送数据。
- ③ HEAD：检测一个对象是否存在。



图 27.1.2 HTTP 基本方法使用

可以知道，“GET”请求用来获取服务器的数据，而“PORT”请求是向服务器转送数据。

27.2 HTTP 服务器实验

27.2.1 硬件设计

1. 例程功能

在开发板上搭建一个 HTTP 服务器，并实时控制开发板上的 LED1 和蜂鸣器。

该实验的实验工程，请参考《lwIP 例程 16 lwIP_HTTPS 实验》。

27.2.2 软件设计

27.2.2.1 程序流程图

本实验流程图，如下图所示。

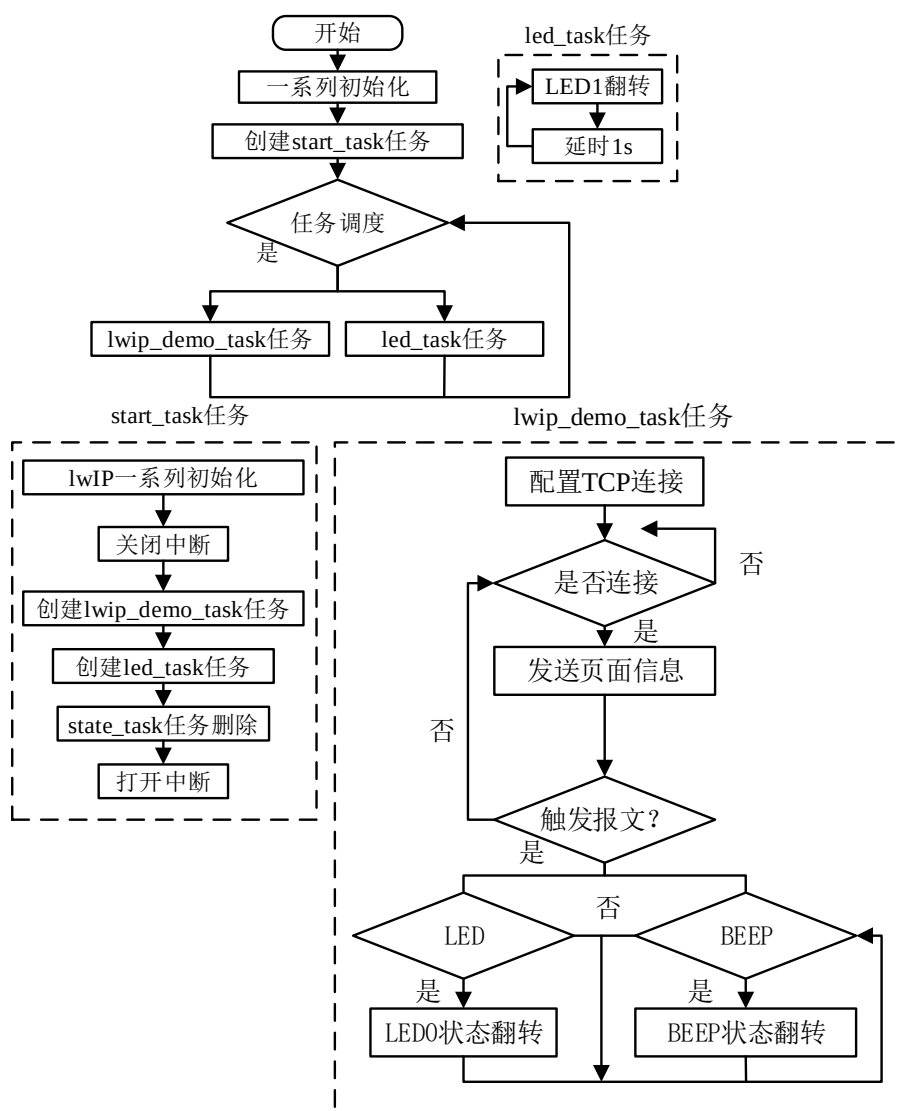


图 27.2.2.1.1 HTTP 服务器程序流程图

27.2.2.2 程序解析

本实验重点看 lwip_demo.c 文件，该文件定义了三个函数，如下表所示：

函数	描述
lwip_demo()	建立 TCP 连接
lwip_server_netconn_serve()	服务 HTTP 线程中接受的一个 HTTP 连接
lwip_data_locate()	寻找指定字符位置

表 27.2.2.2.1 lwip_demo.c 文件的相关函数描述

lwip_demo 函数用来配置网络环境，这里笔者把开发板设置为 TCP 服务器，其端口号为 80。

```
/**
 * @brief lwip_demo 程序入口
 * @param 无
 * @retval 无
 */
void lwip_demo(void)
{
    struct netconn *conn, *newconn;
    err_t err;

    /* 创建一个新的 TCP 连接句柄 */
    /* 使用默认 IP 地址绑定到端口 80 (HTTP) */

    conn = netconn_new(NETCONN_TCP);
    netconn_bind(conn, IP_ADDR_ANY, 80);

    /* 将连接置于侦听状态 */
    netconn_listen(conn);

    do
    {
        err = netconn_accept(conn, &newconn);

        if (err == ERR_OK)
        {
            http_server_netconn_serve(newconn);
            netconn_delete(newconn);
        }
    } while (err == ERR_OK);

    netconn_close(conn);
    netconn_delete(conn);
}
```

连接完成之后调用 http_server_netconn_serve 函数实现本章节的功能。

lwip_server_netconn_serve 函数源码如下所示：

```
static void
lwip_server_netconn_serve(struct netconn *conn)
```

```

{
    struct netbuf *inbuf;
    char *buf;
    u16_t buflen;
    err_t err;
    char *ptemp;

    /* 从端口读取数据，如果那里还没有数据，则阻塞。
       我们假设请求(我们关心的部分)在一个 netbuf 中 */
    err = netconn_recv(conn, &inbuf);

    if (err == ERR_OK)
    {
        netbuf_data(inbuf, (void **)&buf, &buflen);

        /* 这是一个 HTTP GET 命令吗?只检查前 5 个字符，因为
           GET 还有其他格式，我们保持简单) */
        if (buflen >= 5 &&
            buf[0] == 'G' &&
            buf[1] == 'E' &&
            buf[2] == 'T' &&
            buf[3] == ' ' &&
            buf[4] == '/')
        {
start_html:
            /* 发送 HTML 标题
               从大小中减去 1，因为我们没有在字符串中发送\0
               NETCONN_NOCOPY:我们的数据是常量静态的，所以不需要复制它 */
            netconn_write(conn, http_html_hdr, sizeof(http_html_hdr) - 1,
                          NETCONN_NOCOPY);

            /* 发送我们的 HTML 页面 */
            netconn_write(conn, http_index_html, sizeof(http_index_html) - 1,
                          NETCONN_NOCOPY);
        }
        else if (buflen >= 8 && buf[0] == 'P' && buf[1] == 'O' && buf[2] == 'S' && buf[3] == 'T')
        {
            ptemp = lwip_data_locate((char *)buf, "led1=");

            if (ptemp != NULL)
            {
                /* 查看 led1 的值。为 1 则灯亮，为 2 则灭，此值与 HTML 网页中设置有关 */
                if (*ptemp == '1')

```



```

        {
            /* 点亮 LED1 */
            LED0(0);
        }
        else
        {
            /* 熄灭 LED1 */
            LED1(1);
        }
    }

    /* 查看 beep 的值。为 3 则灯亮，为 4 则灭，此值与 HTML 网页中设置有关 */
    ptemp = atk_data_locate((char *)buf, "beep=");

    if (ptemp != NULL )
    {
        if (*ptemp == '3')
        {
            /* 打开蜂鸣器 */
            BEEP(1);
        }
        else
        {
            /* 关闭蜂鸣器 */
            BEEP(0);
        }
    }
    goto start_html;
}

/* 关闭连接(服务器在 HTTP 中关闭) */
netconn_close(conn);

/* 删除缓冲区(netconn_recv 给我们所有权，
   所以必须确保释放缓冲区) */
netbuf_delete(inbuf);
}

```

上述的源码很简单理解，主要分为三步：

- ① 当浏览器输入 IP 地址并且回车确认时，程序调用函数 `netconn_write` 把网页数据发送到浏览器当中。
- ② 当网页发送一个 PORT 命令时，程序调用函数 `lwip_data_locate` 判断触发源，判断完成之后根据触发源来执行相应的动作。

- ③ 程序执行 goto 语句重新发送网页字符串到网页当中，这个步骤相当于更新网页。

27.3.3 下载验证

编译程序并把程序下载到开发板中，打开网页同时，需要查看分配的 IP 地址为多少，接着在浏览器上输入 IP 地址，如下图所示。

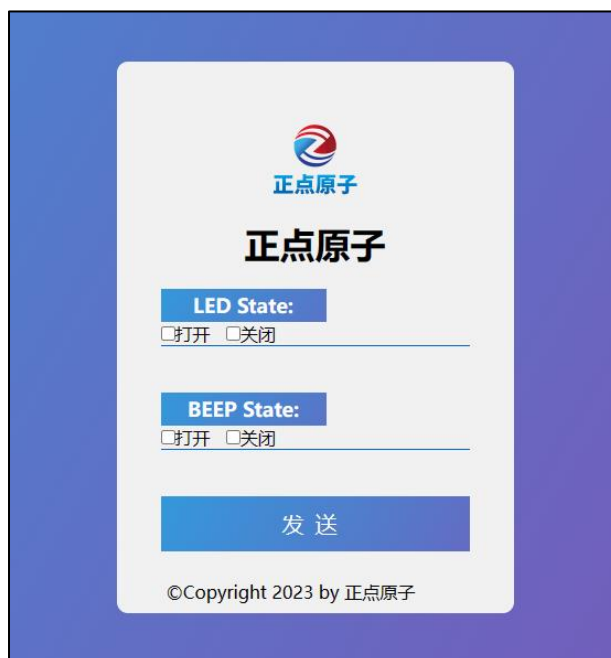


图 27.3.3.1 主界面

可以看出，我们通过选择复选框来开启 LED 和蜂鸣器器件。